ABACUS

原 | 子 | 算 | 筹

# 原子算筹（ABACUS）软件使用和开发手册（V0.1 版本）

作者：ABACUS 爱好者

组织：北京大学

时间：2025 年 8 月 6 日

# 目录


**第 1 章 基本介绍** **1**

1.1 前言 . . . 1
1.2 相关网址 . . . 1
1.3 交流平台 . . . 1
1.4 常见问题 . . . 1

**第 2 章 编译 ABACUS** **6**

2.1 GCC 编译 ABACUS 教程 . . . 6
2.1.1 介绍：关于 ABACUS . . . 6
2.1.2 检查当前服务器基础环境： . . . 6
2.1.3 安装仅支持 PW 基组的 ABACUS . . . 7
2.1.4 安装支持两种基组的 ABACUS . . . 8
2.2 Intel oneAPI 2024/2025 编译 ABACUS 教程 . . . 9
2.2.1 Intel OneAPI 工具链 . . . 9
2.2.2 ABACUS 安装 . . . 12
2.2.3 常见问题和解决方法 . . . 15
2.2.4 参考 . . . 17
2.3 Intel oneAPI 编译 ABACUS 教程 . . . 18
2.3.1 前言 . . . 18
2.3.2 关于 Intel® oneAPI Toolkits . . . 18
2.3.3 编译安装 . . . 18
2.4 Toolchain 编译 ABACUS 教程 (1) – GCC . . . 20
2.4.1 简介 . . . 20
2.4.2 什么是 ABACUS Toolchain . . . 20
2.4.3 基于 GCC Toolchain 安装 ABACUS . . . 21
2.5 Toolchain 编译 ABACUS 教程 (2) – Intel . . . 29
2.5.1 简介 . . . 29
2.5.2 基于 Intel Toolchain 安装 ABACUS . . . 29
2.5.3 基于 Intel Toolchain 引入 ABACUS 功能插件 . . . 42
2.6 Toolchain 编译 ABACUS 教程 (3) – AMD . . . 44
2.6.1 简介 . . . 44
2.6.2 已有方案回顾 . . . 45
2.6.3 AMD Toolchain . . . 45
2.7 编译 Nvidia GPU 版本的 ABACUS . . . 54
2.7.1 介绍 . . . 54
2.7.2 检查环境 . . . 55
2.7.3 编译 . . . 55
2.8 ABACUS LCAO 基组 GPU 版本使用说明 . . . 55
2.8.1 简介 . . . 55
2.8.2 安装 . . . 55
2.8.3 Toolchain 安装 . . . 56
2.8.4 使用 . . . 56

2.8.5 示例 . . . 57
2.9 在超算环境编译 ABACUS 的建议 . . . 65
2.9.1 关于 ABACUS . . . 65
2.9.2 超算平台的简单介绍 . . . 65
2.9.3 拓展教程 . . . 67
2.10 ABACUS 在曙光 DCU 集群与 AMD 显卡上的编译与使用 . . . 67
2.10.1 介绍 . . . 67
2.10.2 准备 . . . 67
2.10.3 流程 . . . 68
2.10.4 结语 . . . 71

**第 3 章 建模** **72**
3.1 ABACUS 结构文件 STRU 及其准备方式 . . . 72
3.1.1 背景 . . . 72
3.1.2 基于 ASE-ABACUS 快速准备 STRU 文件 . . . 72
3.1.3 STRU 文件的基本结构 . . . 73
3.1.4 基于 ASE-ABACUS 从其他结构文件准备 STRU . . . 74
3.1.5 基于 ATOMKIT 准备 STRU 文件 . . . 75
3.1.6 将 STRU 文件转化为其他格式 . . . 78
3.1.7 基于 abacustest 准备 ABACUS 输入文件 . . . 78
3.2 ABACUS 如何选择晶胞朝向获得最佳并行效率？以碳纳米管为例 . . . 79
3.2.1 背景 . . . 79
3.2.2 ABACUS 做不同真空方向计算的流程 . . . 79
3.2.3 测试结果 . . . 81
3.3 ABACUS 如何选择晶胞朝向获得最佳并行效率？以二维氮化硼为例 . . . 82
3.3.1 背景 . . . 82
3.3.2 ABACUS 计算的流程 . . . 82
3.3.3 测试结果 . . . 89
3.4 ABACUS 如何选择晶胞朝向获得最佳并行效率？以铜表面一氧化碳吸附为例 . . . 89
3.4.1 背景 . . . 89
3.4.2 ABACUS 做不同真空方向计算的流程 . . . 90
3.4.3 测试结果 . . . 98
3.5 模守恒赝势生成方法简介 . . . 99
3.5.1 介绍 . . . 99
3.5.2 从其他格式赝势转换 . . . 99
3.5.3 赝势的生成 . . . 99
3.5.4 结尾 . . . 110
3.6 数值原子轨道（一）：ABACUS 中的数值原子轨道命名和使用方法 . . . 110
3.6.1 数值原子轨道的背景知识 . . . 110
3.6.2 数值原子轨道的命名方法 . . . 111
3.6.3 ABACUS 中数值原子轨道的使用方法 . . . 112
3.6.4 参考文献 . . . 113
3.7 数值原子轨道（二）：生成给定模守恒赝势的数值原子轨道 . . . 113
3.7.1 ABACUS 中的数值原子轨道背景知识 . . . 113
3.7.2 ABACUS 中产生数值原子轨道的具体流程 . . . 116

3.7.3 ABACUS 中数值原子轨道的生成脚本 . . . 120
3.7.4 ABACUS 中数值原子轨道的测试流程 . . . 123
3.7.5 参考文献 . . . 124
3.8 数值原子轨道（三）：产生高精度数值原子轨道 . . . 124
3.8.1 PTG（PyTorch Gradient）方法 . . . 124
3.8.2 PTG_dpsi（PyTorch Gradient with dpsi）方法 . . . 124
3.8.3 安装 Pytorch . . . 125
3.8.4 产生高精度数值原子轨道流程 . . . 125
3.8.5 参考文献 . . . 127

**第 4 章 基础功能** **128**
4.1 Kohn-Sham 密度泛函理论计算 . . . 128
4.1.1 平面波计算与收敛性测试 . . . 128
4.1.2 ABACUS+LibRI 做杂化泛函计算教程 . . . 136
4.1.3 ABACUS 收敛性问题解决手册 . . . 140
4.2 分子动力学模拟 . . . 144
4.2.1 ABACUS 分子动力学使用教程 . . . 144
4.2.2 ABACUS+DeePMD-kit 做机器学习分子动力学 . . . 154
4.3 采用 ABACUS 进行表面计算（一）：静电势和功函数 . . . 158
4.3.1 背景 . . . 158
4.3.2 ABACUS 中计算静电势具体流程 . . . 158
4.4 采用 ABACUS 进行表面计算（二）：偶极修正 . . . 161
4.4.1 偶极修正介绍（Dipole Correction） . . . 161
4.4.2 文档与算例 . . . 162
4.4.3 偶极修正输入参数介绍 . . . 162
4.4.4 预期结果 . . . 164
4.4.5 参考文献 . . . 164
4.5 采用 ABACUS 进行表面计算（三）：表面能计算 . . . 165
4.5.1 背景 . . . 165
4.5.2 ABACUS 计算表面能具体流程 . . . 165
4.5.3 参考文献 . . . 167
4.6 采用 ABACUS 进行表面计算（四）：表面缺陷能和吸附能计算 . . . 167
4.6.1 背景 . . . 167
4.6.2 ABACUS 计算表面缺陷能具体流程 . . . 168
4.6.3 ABACUS 计算表面吸附能具体流程 . . . 169
4.6.4 参考文献 . . . 171
4.7 采用 ABACUS 进行表面计算（五）：外加电场 . . . 171
4.7.1 介绍 . . . 171
4.7.2 软件和算例准备 . . . 172
4.7.3 采用加电场进行二维材料的能带计算 . . . 172
4.7.4 预期计算结果 . . . 174
4.7.5 参考文献 . . . 176
4.8 采用 ABACUS 进行表面计算（六）：补偿电荷 . . . 176
4.8.1 介绍 . . . 176
4.8.2 文档与算例 . . . 177

4.8.3 偶极修正与补偿电荷结合的表面计算 . . . 177
4.8.4 预期结果 . . . 178
4.8.5 参考文献 . . . 179

**第 5 章 特色功能 180**
5.1 ABACUS 随机波函数 DFT 方法使用教程 . . . 180
5.1.1 介绍 . . . 180
5.1.2 软件和算例准备 . . . 180
5.1.3 采用 SDFT 进行电子自洽迭代计算 . . . 180
5.1.4 采用 SDFT 进行分子动力学模拟 . . . 182
5.1.5 采用 SDFT 计算态密度 . . . 182
5.1.6 结尾 . . . 183
5.2 ABACUS 随机波函数 DFT 计算电子电导热导教程 . . . 184
5.2.1 介绍 . . . 184
5.2.2 算例准备 . . . 184
5.2.3 采用 sDFT 进行电子的电导热导计算 . . . 185
5.3 ABACUS 实时演化含时密度泛函理论使用教程 . . . 187
5.3.1 背景 . . . 187
5.3.2 在 ABACUS 中使用 rt-TDDFT 时改变电子的占据数 . . . 187
5.3.3 ABACUS 中使用 rt-TDDFT 外加电场计算材料吸收谱 . . . 190
5.3.4 参考文献 . . . 193
5.4 ABACUS 无轨道密度泛函理论方法使用教程 . . . 193
5.4.1 无轨道密度泛函理论背景知识 . . . 193
5.4.2 ABACUS 中进行 OFDFT 计算的具体流程 . . . 195
5.4.3 参考文献 . . . 198
5.5 ABACUS 隐式溶剂模型使用教程 . . . 198
5.5.1 介绍 . . . 198
5.5.2 文档和算例 . . . 199
5.5.3 输入参数 . . . 199
5.5.4 预期结果 . . . 200
5.5.5 参考文献 . . . 201
5.6 ABACUS+DPGEN 使用教程 . . . 201
5.6.1 介绍 . . . 201
5.6.2 准备 . . . 202
5.6.3 结合 DPGEN 生成 DP 数据的流程 . . . 202
5.6.4 计算材料 EOS 和弹性常数的流程 . . . 212

**第 6 章 软件接口与后处理 215**
6.1 ABACUS+Atomkit 计算态密度和能带 . . . 215
6.1.1 介绍 . . . 215
6.1.2 准备 . . . 215
6.1.3 铝的态密度以及电子能带结构计算 . . . 216
6.1.4 铁的态密度以及电子能带结构计算 . . . 221
6.1.5 参考文献 . . . 222
6.2 ABACUS 计算 PDOS . . . 223

6.2.1 PDOS 文件分析 . . . 223
6.2.2 绘制 PDOS . . . 224
6.2.3 绘制 PDOS 的流程 . . . 227
6.3 ABACUS 输出部分的电荷密度和波函数及可视化教程 . . . 228
6.3.1 引言 . . . 228
6.3.2 基本概念与公式 . . . 228
6.3.3 输入输出概述 . . . 229
6.3.4 在平面波基组下进行计算 . . . 231
6.3.5 在数值原子轨道基组下进行计算 . . . 232
6.3.6 示例：对电荷密度进行可视化 . . . 234
6.3.7 结语 . . . 236
6.4 ABACUS 计算电子局域函数 ELF 使用教程 . . . 237
6.4.1 背景 . . . 237
6.4.2 定义 . . . 237
6.4.3 ABACUS 计算 ELF 具体流程 . . . 238
6.4.4 计算实例：自旋极化的体心立方铁 . . . 240
6.4.5 参考文献 . . . 241
6.5 ABACUS+Bader charge 分析教程 . . . 241
6.5.1 前言 . . . 241
6.5.2 Bader 分析软件部署 . . . 241
6.5.3 使用示例 . . . 242
6.5.4 附录：cube manipulator 更多的使用方法 . . . 245
6.5.5 Have a question? Submit issue! . . . 250
6.6 ABACUS+pymatgen 计算弹性常数 . . . 250
6.6.1 介绍 . . . 250
6.6.2 准备 . . . 251
6.6.3 金刚石的弹性常数计算 . . . 252
6.7 ABACUS+Phonopy 计算声子谱 . . . 253
6.7.1 介绍 . . . 253
6.7.2 准备 . . . 253
6.7.3 流程 . . . 254
6.8 ABACUS+ShengBTE 计算晶格热导率 . . . 258
6.8.1 介绍 . . . 258
6.8.2 准备 . . . 258
6.8.3 流程 . . . 258
6.8.4 结尾 . . . 262
6.9 ABACUS+Wannier90 使用教程 . . . 262
6.9.1 教程使用说明 . . . 262
6.9.2 Wannier90 软件介绍 . . . 262
6.9.3 使用 ABACUS 准备 Wannier90 需要的文件 . . . 262
6.9.4 参考文献 . . . 265
6.10 ABACUS+Candela 使用教程 . . . 265
6.10.1 介绍 . . . 265
6.10.2 准备 . . . 265
6.10.3 流程 . . . 266

6.11 ABACUS+USPEX 接口教程 . . . 270
6.11.1 背景介绍 . . . 270
6.11.2 接口介绍 . . . 270
6.11.3 流程（以 Si 为例） . . . 273
6.11.4 结语 . . . 276
6.12 ABACUS+Hefei NAMD 使用教程 . . . 277
6.12.1 介绍 . . . 277
6.12.2 计算流程 . . . 277
6.12.3 结语 . . . 280
6.12.4 参考文献 . . . 280
6.13 Pyabacus 文档一：用户手册 . . . 280
6.13.1 简介 . . . 280
6.13.2 模块 . . . 280
6.13.3 安装 . . . 281
6.13.4 调用 . . . 281
6.13.5 示例 . . . 281
6.13.6 测试 . . . 282
6.13.7 结语 . . . 282
6.14 Pyabacus 文档二：HSolver 模块 . . . 282
6.14.1 简介 . . . 282
6.14.2 安装 pyabacus . . . 284
6.14.3 调用对角化算法 . . . 284
6.14.4 结语 . . . 285
6.15 Pyabacus 文档三：开发者指南 . . . 285
6.15.1 简介 . . . 285
6.15.2 结构 . . . 285
6.15.3 资料汇总 . . . 290
6.15.4 结语 . . . 290

**第 7 章 开发者规范 291**
7.1 ABACUS 的 Github 仓库 Issues 处理流程 . . . 291
7.1.1 Github 的 Issues 机制 . . . 291
7.1.2 ABACUS 项目的 Issues 标签分类 . . . 292
7.1.3 Issues 处理流程 . . . 292
7.2 ABACUS 开源项目 C++ 代码规范 . . . 299
7.2.1 前言 . . . 299
7.2.2 命名约定 . . . 299
7.2.3 关于头文件 . . . 300
7.2.4 关于类（Class） . . . 301
7.2.5 关于函数（Function） . . . 302
7.2.6 关于作用域 . . . 303
7.2.7 其他 C++ 特性 . . . 305
7.2.8 关于 ABACUS 中常用的关键词缩写 . . . 308
7.2.9 Reference . . . 308
7.3 ABACUS 注释规范：Doxygen 入门 (c++) . . . 309

7.3.1 简介 . . . 309
7.3.2 如何使用 Doxygen . . . 312
7.3.3 附录：Doxygen 与 VS Code . . . 322
7.4 ABACUS 线上文档输入参数撰写规范 . . . 322
7.4.1 背景 . . . 322
7.4.2 规范格式 . . . 322
7.4.3 其他问题 . . . 324
7.5 ABACUS 代码存放规范 . . . 324
7.5.1 背景 . . . 324
7.5.2 代码目录 . . . 324
7.5.3 总结 . . . 327
7.6 如何在 ABACUS 中新增一个输入参数（截至 v3.5.3） . . . 327
7.6.1 在 input 类中申明和定义 . . . 327
7.6.2 将 input 成员转换为全局变量 . . . 329
7.6.3 添加文档 . . . 330
7.7 如何在 ABACUS 中新增一个输入参数（v3.7.0 后） . . . 330
7.7.1 开发者须知 . . . 330
7.7.2 设计文档参考 . . . 334
7.8 如何在 ABACUS 中进行异构计算 . . . 338
7.8.1 前言 . . . 338
7.8.2 管理设备类型 . . . 338
7.8.3 数据内存管理（旧） . . . 339
7.8.4 数据内存管理（新） . . . 345
7.8.5 执行数学计算 . . . 346
7.9 ABACUS formatter-2.0 版本使用说明书 . . . 347
7.9.1 前言 . . . 347
7.9.2 字符串格式化 . . . 347
7.9.3 制表功能 . . . 350
7.9.4 static Python-style 字符串函数 . . . 351
7.9.5 Find a bug? Submit issue! . . . 352
7.10 ABACUS 中使用格式化工具 clang-format . . . 352
7.10.1 引言 . . . 352
7.10.2 在 ABACUS 中使用 clang-format . . . 352
7.10.3 格式化时的注意事项： . . . 353
7.10.4 参考文献： . . . 354

# 第 1 章 基本介绍

## 1.1 前言

本文档素材主要收集自 ABACUS 线上中文文档：https://mcresearch.github.io/abacus-user-guide/，希望能够惠及 ABACUS 软件的开发者和使用者。若有任何疑问，请写邮件到 mohanchen@pku.edu.cn 进一步交流。

## 1.2 相关网址

ABACUS 的官网地址为：http://abacus.ustc.edu.cn/
ABACUS 在 DeepModeling 社区中的 GitHub 仓库地址为：https://github.com/deepmodeling/abacus-develop
ABACUS 的 Gitee 镜像仓库地址为：https://gitee.com/deepmodeling/abacus-develop
ABACUS 线上文档（包括输入输出参数介绍等）：https://abacus.deepmodeling.com/en/latest/
ABACUS 线上中文文档：https://mcresearch.github.io/abacus-user-guide/
ABACUS 开源社区提问与讨论：https://github.com/deepmodeling/abacus-develop/issues

## 1.3 交流平台

ABACUS 社区目前有微信群和 QQ 群。
ABACUS 用户 QQ 群：759914681。

## 1.4 常见问题

**常用缩写**

- **PW**: Plane Wave，指平面波
- **NAO**: Numerical Atomic Orbitals，指数值原子轨道
- **LCAO**: Linear Combination of Atomic Orbitals，指局域轨道，在 ABACUS 里面也指数值原子轨道

**常见问题** 基于过去几年来积累的用户问题，接下来本文档将尽量以问答的形式来替使用 ABACUS 的用户答疑解惑，例如：

1. **问：ABACUS 和 ABAQUS 是什么关系？**
   **答：**ABAQUS 是一款知名的多物理场有限元仿真分析软件，ABACUS 的英文单词翻译是算盘，ABACUS 作为一款国产密度泛函理论软件，中文名是原子算筹。
2. **问：使用 ABACUS 需要收费吗？**
   **答：**从软件的角度来讲不收费，因为是开源软件，但跑该软件的机器得自己准备，机器要交电费。
3. **问：ABACUS 的开发过程是怎样的，什么人可以成为开发者？**
   **答：**ABACUS 的代码开发记录完全公开，所有代码的修改都在 `https://github.com/deepmodeling/abacus-develop` 网页上可以看到，ABACUS 为所有感兴趣的参与者都保留了成为开发者的可能性，每个人都有可能成为开发者。

**ABACUS 使用建议**

1. **编译**

(a). **从哪里可以下载到 ABACUS 的代码？**

**答：**如果可以接 github，可以登录 https://github.com/deepmodeling/abacus-develop，建议下载最新版本使用，因为软件一直在更新，总体而言新版本的 bugs 会更少。

如果网络不方便登录 github，还可以登录国内网站 https://gitee.com/deepmodeling/abacus-develop.git 下载。

(b). **怎么安装 ABACUS？**

**答：**可以参考官方文档 https://abacus.deepmodeling.com/en/latest/quick_start/easy_install.html

也可以登录网址 https://mcresearch.gitee.io/abacus-user-guide/，上面有一些针对不同编译器的中文编译教程。

另外，注意“安装 ABACUS”这句话其实有歧义，ABACUS 有很多不同的安装选项，比如是否只安装支持平面波的功能，还是也要安装支持数值原子轨道的功能，是否要使用 LIBXC 等等不同选项，可根据自己的需要来安装具体的代码模块。

(c). **如何在曙光的 DCU 上编译 ABACUS？**

**答：**可参考文档 https://mcresearch.gitee.io/abacus-user-guide/abacus-dcu.html

2. **计算前准备**

(a). **PW 计算需要准备的文件？**

**答：**INPUT 文件控制输入参数，STRU 文件包含晶胞、原子信息和赝势；KPT 文件包含布里渊区 k 点信息。这三个文件默认是放在一个文件夹内的。但是 INPUT 文件里其实可以控制结构文件所在地址，以及 KPT 文件所在地址。

(b). **KPT 文件什么时候不适用？**

**答：**第一，在使用 DPGEN 的时候，强烈建议设置 kspacing 这个参数，而不是在 KPT 文件里设置 k 点的个数，这是为了做不同大小体系密度泛函计算的时候，使得计算的精度尽量持平，从而得到不同结构里精度相当的总能量和受力，这对于 DPMD 的训练很重要。第二，LCAO 可以在 INPUT 使用 gamma_only（设为 1），那么 KPT 文件不起作用，所有计算只用到 Gamma（0，0，0）这个 k 点，这种情况下体系有更快速的特殊算法，效率更高，计算内存的消耗也更小。

(c). **LCAO 基组相比于 PW 基组需要额外准备的文件？**

**答：**数值原子轨道文件，下载地址为 https://abacus.ustc.edu.cn/pseudo/list.htm。注意轨道文件要和对应的赝势相匹配。

关于数值原子轨道的背景知识，可以参考文献：

- https://mcresearch.gitee.io/abacus-user-guide/abacus-nac1.html
- https://mcresearch.gitee.io/abacus-user-guide/abacus-nac2.html
- https://mcresearch.gitee.io/abacus-user-guide/abacus-nac3.html

(d). **能量截断值收敛性测试是必须要做的吗？**

**答：**是的，除非你或者别人之前已经做过了，得到了推荐的能量截断值。否则，一般来讲一个给定的赝势（模守恒或者超软赝势），你需要从低到高逐步增加能量截断值来判断总能量已经收敛。

(e). **布里渊区 K 点收敛性和什么关系？**

**答：**布里渊区 k 点只和实空间的晶胞大小有关系，因此当换了晶胞之后需要测试 k 点的收敛性。然而，一般我们会有一些经验规律，比如实空间某个方向增大一倍，则倒空间相应的 k 点数量可以少取一半，一直小到只取 Gamma 点（0，0，0）为止。所以，改变赝势或者交换关联泛函的时候，并不需要重新测试 k 点的收敛情况。

(f). **关于长度单位**

**答：**没有注意到单位的转换是初学者容易犯的错之一。长度单位 Angstroms（埃）和 Bohr 的转换关系需要了解（1 埃 =1.8897259886 Bohr，反过来 1 Borh=0.529177249 Angstroms，转换时候尽可能多地取有效数字，会让原子坐标更准确），不同软件的长度单位可能有取得不同的情况，涉及到晶格矢量、原

子位置，以及坐标系转换（Direct 和 Cartesian 的区别，前者是晶格坐标系，后者单位才是长度单位），如果原子位置或者晶胞设错了，不同软件计算结果就会不同。一个小技巧判断不同软件的晶格结构和原子坐标是否一样：可以看不同软件算出来的 Ewald 能量（周期性边界条件下算离子-离子库伦排斥能的快速算法），如果 Ewald 能量能对上（精确到小数点后 10 位以上），那么说明两个软件的原子坐标和晶格矢量设置都是一样的。

(g). **关于能量单位**

**答：**涉及到能量单位的有多处。首先是平面波的能量截断值，VASP 经常用的是 eV，QE 和 ABACUS 用的是 Ry，这两者有转换关系（1 Ry=13.605703976 eV），所以用 VASP 习惯的用户需要把对应的 eV 转换成 Ry 再使用 ABACUS。其次是 smearing 的参数选择，例如 0.015，其单位是 Ry 或者 eV 也对结果有较大影响。

3. **基矢量**

(a). **ABACUS 支持的基矢量有几种？**

**答：**分成 PW（平面波）和 LCAO（具体来讲是 Numerical atomic orbitals，NAO，即数值原子轨道）两种，目前 ABACUS 都支持，像是 VASP 和 QE 只支持 PW，SIESTA 和 OpenMP 支持 LCAO。还有一种叫 lcao_in_pw，就是把 NAO 展开成平面波进行计算，但一般只用于测试目的。

(b). **怎么选择是采用 PW 还是用 LCAO 基矢量来做计算？**

**答：**如果是较小原子数的固体体系，建议可以采用 PW 效率较高。如果是体系原子数较大（例如大于 50 个原子），建议可以用 LCAO 试试，效率会更高。此外，当使用 LCAO 做计算时，可以先用 PW 这组相对完备的基组来测试赝势带来的能量收敛性。

(c). **如果感觉 LCAO 的效率慢，可能的原因是否和基矢量有关系？**

**答：**ABACUS 里采用的 LCAO 的局域轨道长度一般在 6-10 au 之间，轨道的截断越长，往往计算精度越高，但计算量和内存消耗也会显著的更大。如果要提高速度，可以在尽可能不影响精度（需要自己测试）的前提下，减小数值原子轨道的半径，这样可以大幅度提升计算效率以及降低内存占用量。

(d). **ABACUS 中 LCAO 的 Ecut 是做什么用的**

**答：**LCAO 和 PW 一样，都会有一个 Ecut，在使用 LCAO 的时候，增加 Ecut 只会增加局域势格点积分部分的精度（赝势的局域部分 + 交换关联势 +Hartree 势），并不增加数值原子轨道基矢量的完备性，所以原先用于 PW 的收敛性测试方法是不能这样直接用于 LCAO 的。如果要增加 LCAO 的基矢量使得基矢量更完备，需要从例如 DZP 增加到 TZDP，而不是增加 Ecut。

(e). **LCAO 算小分子和固体有什么要注意的？**

**答：**使用局域轨道的话，会有 BSSE（Basis set superposition error），对于小分子体系，BSSE 带来的误差会较为严重，使得对于一些分子体系，有必要采用更完备的基矢量（或者更长的局域轨道）或者 BSSE 修正。而对于固体系统，一般来讲每个原子会有若干近邻原子，这些近邻原子上的局域轨道会使得在描述每个原子的电子结构时基矢量总体更完备，所以可以相对选择更短的局域轨道。

(f). **如果算出来 SZ 结果比 DZP 好，是否可以用 SZ 结果？**

**答：**采用 LCAO 时，SZ（single zeta 轨道）的结果一般来讲都不收敛，一般不可以用来发表文章。一般至少是 DZP 及以上才认为基矢量较为准确，是学界可以认可的 NAO 精度。

4. **赝势**

(a). **PAW 是不是赝势？**

**答：**准确来说不是，是一种描述电子-离子相互作用的方法。

(b). **为什么不同软件算出来的同一体系的总能量差距可能会巨大？**

**答：**因为同样的元素，其不同的赝势（或 PAW 势）允许总能量有一个常数的偏移，因为都对库伦势进行了人为的修改。所以，我们一般在分析结果的时候也不用总能量这个数值，而是用总能量之间的差别，例如 bcc 相比于 fcc 更稳定这种结论，不同赝势哪怕总能量不对齐，其算出来的 bcc 和 fcc 之间的能量差也应该是对齐的。另外一件事情就是画态密度（Density of States，DOS）的时候，我们习惯把费米面取成 0 点，也是因为总能量是可以有一个常数的偏移的。

(c). **赝势为什么和交换关联泛函有关？**

**答：**因为赝势生成过程中解的是一维的 Kohn-Sham 方程，这个过程也用到了泛函，所以不同泛函生成的赝势就不同。

(d). **赝势改变了，什么参数需要重新测试？**

**答：**因为电子和离子的描述方式变了，所以需要重新测试能量截断值 Ecut 的收敛性，即增加 Ecut，看总能量变化，例如从 90 Ry 变到 100 Ry，如果总能量变化小于 1 meV/atom，则认为可以收敛。

(e). **从 PBE 泛函换到 Meta-GGA 泛函（如 SCAN）或杂化泛函，是否需要换赝势文件？**

**答：**一个约定俗成的赝势用法是：PBE 的赝势可以用于 Meta-GGA 泛函，例如 SCAN 泛函，甚至杂化泛函包括 PBE0 和 HSE06 泛函。但是，LDA 的赝势就不可以用于 SCAN 泛函或者 PBE0、HSE06 泛函。因为 SCAN 泛函和 HSE06 泛函也是基于 PBE 泛函发展起来的。

(f). **别人文献看到的某元素的能量截断值，是否可以直接使用？**

**答：**一般来说是不可以，例如用了 VASP 的 PAW 势，就难以直接拿到模守恒赝势里使用。

5. **电子自洽迭代过程**

(a). **为什么 ABACUS 的 SCF 计算比 VASP 慢很多？**

**答：**首先，先检查 Ecut 是否一样（注意 ABACS 的单位是 Ry，VASP 是 eV，要换算），一般来讲 VASP 用 PAW 方法所需的 Ecut 比 ABACUS 的模守恒赝势要低，目前来讲这部分参数 VASP 占优，你并不能降低 ABACUS 的 Ecut 来和 VASP 的 Ecut 持平。第二，检查你所计算的元素的电子数，往往电子数不同，计算效率会有很大差别。例如 Ga 的模守恒赝势可能有 13 个电子，但 VASP 的 Ga 的 PAW 势可能只有 3 个电子，那么 VASP 的计算效率会快很多也是可以预期的。

(b). **电子自洽迭代里的展宽技术是用来做什么的？**

**答：**电子自洽迭代过程中使用的展宽（一般称为 Smearing）是一种数值技术，它用于提高计算中的收敛稳定性和速度，尤其是在处理金属和其他导体体系时非常有用。展宽通常采用几种不同的办法来实施，例如费米-狄拉克展宽（Fermi-Dirac smearing）、高斯展宽（Gaussian smearing）、Methfessel-Paxton 展宽等，它们代表不同的电子占据数分布方法。

(c). **电子自洽迭代收敛结果和 smearing 相关吗？**

**答：**对于没有带隙的金属体系是相关的，采用 smearing 会在体系总能量里额外引入一个电子熵项。一般来讲 smearing_sigma 取得越大越容易收敛，但是要注意的是，smearing_sigma 取大了，总能量结果也愈发不准确。因此在迭代接近收敛时，通常需要逐步减少展宽值，以得到更加准确的接近零温的结果。对于半导体来讲，一般较小的 smearing 参数不影响总能量结果。

(d). **只要体系不收敛，就可以调整 smearing 吗？**

**答：**在项目里对一系列相关体系进行密度泛函理论计算，如果需要用到这些体系的总能量相对值，但发现有体系不收敛，我可以针对不同体系调整不同的 smearing 参数吗? 一般不可以，因为 smearing 可能会改变总能量的数值。

(e). **电荷密度混合是用来做什么的？**

**答：**在电子自洽迭代过程中，需要不断更新电荷密度，直到输入的电荷密度和由此计算得到的输出电荷密度之间的差异足够小，即达到自洽。混合的具体方法有很多种，比如简单的线性混合、Broyden 混合、Pulay 混合（也称为 DIIS，Direct Inversion in the Iterative Subspace）等。这些方法通过考虑前几次迭代的电荷密度和/或电荷密度差异，来改进当前迭代的电荷密度估计。

(f). **总能量收敛、受力和应力是否收敛了？**

**答：**一般来讲，总能量在 SCF 中收敛到几乎不变时，受力和应力不一定完全收敛。如果是对受力或者压力敏感的计算，建议也测试一下相应受力和应力是否收敛（即继续降低 scf_thr 数值，设置更严格收敛阈值）。

(g). **怎么得到准确的带隙？**

**答：**对称性、scf、nscf、电荷密度

6. **原子位置和晶胞结构优化 (弛豫)**

(a). **原子位置和晶胞结构优化有什么需要注意的？**

**答：**首先要注意的就是，原子位置和晶胞结构优化较慢，很耗机时，动辄就是几十上百个进程跑一两天。所以，如果不是很有把握了解你要算的体系，或者不是特别熟练密度泛函理论计算，建议动手算之前找导师聊一聊这样做的可行性，这样做一方面也是可以少走科研上的弯路，另外一方面也能有效的帮助导师节省机时费。

(b). **原子位置和晶胞结构优化有什么技巧？**

**答：**首先，就是要小心一些参数不要设多了，例如不要包含太多费米面上方非占据的轨道，如果体系没有磁矩，没有必要打开自旋极化计算（nspin=2）。其次，如果你能预估偏离最终的构型较大，那你可以先采用低精度的方法达到一个接近最终构型的一个临时构型。例如，没有必要用 18×18×18 的 k 点（虽然你认为这样才能完全收敛总能量）来做优化，你可以先选一个 9×9×9 的 k 点 mesh 来做。第三，如果你是做的一系列相关结构的原子位置和晶胞结构优化，可以想办法构造一些更好的初猜结构，除非考虑亚稳态的存在，否则一开始猜测的结构越接近，所需要的优化步数就越少。第四，BFGS 算法一般适用于偏离稳定结构较小的情况，而 cg 一般用于偏离稳定结构较大的情况。

(c). **做结构优化很多步都不收敛，怎么办？**

**答：**首先，检查每一步的 SCF 是否收敛，如果不收敛，先让 SCF 能收敛，再做结构优化。其次，如果能不用 cell-relax 而只用 relax，建议尽量先用 relax，把晶胞内的原子位置先固定住了，最后再做 cell-realx，这样效率更高。原因是如果把 cell 的自由度和原子的自由度一起考虑做 relax，往往效率较低。

(d). **采用 LCAO 和 PW，结构优化收敛有什么区别？**

**答：**PW 基组更完备，一般结构优化可以收敛到 0.001 eV/Angstroms。而 LCAO 由于基矢量不够完备，存在诸如 egg box effect 的误差，所以求力能收敛到的精度一般要差一些，通常 0.04 eV/Angstroms 以下是可以接受的，也有不少文章采用这个精度来发表结果，是被认可的。

7. **并行**

(a). **MPI 并行和 OpenMP 并行的区别是什么？什么叫进程，什么叫线程？**

**答：**MPI 进程和 OpenMP 线程的差别，前者内存是不共享的，后者内存是共享的。

(b). **原子位置和晶胞结构优化有什么技巧？**

**答：**首先，就是要小心一些参数不要设多了，例如不要包含太多费米面上方非占据的轨道，如果体系没有磁矩，没有必要打开自旋极化计算（nspin=2）。其次，如果你能预估偏离最终的构型较大，那你可以先采用低精度的方法达到一个接近最终构型的一个临时构型。例如，没有必要用 18×18×18 的 k 点（虽然你认为这样才能完全收敛总能量）来做优化，你可以先选一个 9×9×9 的 k 点 mesh 来做。第三，如果你是做的一系列相关结构的原子位置和晶胞结构优化，可以想办法构造一些更好的初猜结构，除非考虑亚稳态的存在，否则一开始猜测的结构越接近，所需要的优化步数就越少。第四，BFGS 算法一般适用于偏离稳定结构较小的情况，而 cg 一般用于偏离稳定结构较大的情况。

(c). **是不是进程或者线程越多，计算就越快？**

**答：**举个例子，用 7au 的 DZP 轨道算 512 个 Fe 原子的体系，$k$ 点 1×1×1，以下是运行时间（OMP 代表线程数，MPI 代表进程数）：1 OMP×32 MPI = 262s 每电子步；1 OMP×64 MPI = 581s 每电子步；1 OMP×128 MPI = 654s 每电子步；1 OMP×256 MPI = 765s 每电子步。可以看到进程越多，计算反而越慢，这是因为进程间的通讯量随着进程数增多变大了。所以，并不是用的计算资源越多，计算就会越快，建议做计算前先评估好并行效率。

(d). **$k$ 点并行（也就是 KPAR 这个参数），要怎么理解它的使用？**

**答：**对 PW 基组计算来说，不同 $k$ 点的 Kohn-Sham 方程求解（即寻找特征值和特征向量）是独立的，在给定总进程数的前提下，ABACUS 提供把不同 $k$ 点的方程求解并行的方案，通过参数 KPAR 实现，该参数代表要将 $k$ 点分成几组，原则上 KPAR 越大效率越高，但 KPAR 不应超过进程数或者 $k$ 点数。如果 KPAR 设成 1，则程序执行时只对 PW 进行并行。在有些情况下，仅用 PW 并行并行效率也挺高，因此看不出 $k$ 点并行的优势。

# 第 2 章 编译 ABACUS

## 2.1 GCC 编译 ABACUS 教程

### 2.1.1 介绍：关于 ABACUS

ABACUS（原子算筹）软件同时支持两种基矢量，一种是平面波（Plane wave，PW）基组，一种是 LCAO (Linear combination of atomic orbitals) 或者称 NAO(Numerical atomic orbitals) 基组。因为两种基组运行依赖的软件库不完全相同，所以我们分开介绍。

实际上，LCAO 基组依赖的软件库比 PW 基组更多，因此如果用户只使用 PW 基组，并不需要将所有依赖的数学库全都下载安装好。

下面介绍，两种基组分别都依赖那些数学库：

- **PW 基组依赖以下 3 个数学库：**

  **BLAS：**BLAS（Basic Linear Algebra Subprograms）是一组基本的线性代数库，用于高效执行常见的基础线性代数运算，如矩阵乘法和向量操作。

  **LAPACK：**LAPACK（Linear Algebra Package）是一个开源的数值线性代数库，用于解决线性代数问题，包括矩阵分解、方程组求解和特征值计算等。

  **FFTW3：**FFTW3（Fastest Fourier Transform in the West 3）是一个高性能、开源的快速傅里叶变换（FFT）库，用于高效计算各种傅里叶变换和逆变换。

- **LCAO 基组依赖以下 6 个软件库：**

  BLAS

  LAPACK

  FFTW3

  **ScaLAPACK：**ScaLAPACK（Scalable LAPACK）是一种并行计算库，构建在 LAPACK 之上，用于解决大规模线性代数问题，特别适用于分布式和并行计算环境。

  **CEREAL：**CEREAL 是一个 C++ 序列化库，用于将 C++ 对象转换为可存储或传输的数据格式，以及将其反序列化回对象。

  **ELPA：**ELPA（Eigenvalue SoLvers for Petaflop-Applications）是一个用于高性能计算的开源库，旨在解决大规模高性能计算中的特征值问题，特别是密集矩阵的特征值问题。（用户编译安装 abacus 的时候大多在这里出现问题）

ELPA 仅用于 LCAO 基组求解特征值与特征矩阵的过程，如果由于机器等原因导致 ELPA 难以安装，可以暂不安装 ELPA，在编译 abacus 的时候添加-DUSE_ELPA=0 即可绕过 ELPA 安装困难的问题。

在 ABACUS 中，LCAO 基组求解特征值与特征矩阵提供了两种方法，一种是调用 ELPA 进行求解（默认选项），一种是调用 ScaLAPACK 进行求解。

如果使用-DUSE_ELPA=0 编译选项，请参考使用文档配置 ks_solver 为 scalapack_gvx，调用 ScaLAPACK 进行求解。

### 2.1.2 检查当前服务器基础环境：

**1. 查看当前系统版本**

```
root@bohrium-11852-1041346:~# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.5 LTS
```

```
Release:        20.04
Codename:       focal
```

**2. 检查当前 GCC 编译器版本**

```
root@bohrium-11852-1041346:~# g++ --version
g++ (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0
Copyright (C) 2019 Free Software Foundation, Inc.
```

**3. 检查当前环境是否有 git**

```
root@bohrium-11852-1041346:~# git --version
git version 2.25.1
```

### 2.1.3 安装仅支持 PW 基组的 ABACUS

对于仅支持 PW 基组的 ABACUS，我们同时支持两种版本的编译：串行版本和并行版本。编译这两个版本的主要区别在于是否需要安装 MPI library。

**1. 编译串行版本**

**安装依赖库**

```
sudo apt update
sudo apt install -y libopenblas-openmp-dev
sudo apt install -y liblapack-dev
sudo apt install -y libfftw3-dev
```

**从仓库克隆代码到本地，并进入目录**

```
git clone https://github.com/deepmodeling/abacus-develop.git
cd abacus-develop/
```

**开始编译并安装 abacus**

```
cmake -B build -DENABLE_MPI=OFF
cd build && make -j`nproc`
```

可以看到在当前目录下，已经编译成功了 abacus_pw_serial 可执行文件,abacus_pw_serial 就是串行版本的支持平面波基组的 ABACUS。

**2. 编译并行版本**

**安装依赖库（同串行，如果已经安装，则不用重复操作）**

```
sudo apt update
sudo apt install -y libopenblas-openmp-dev
sudo apt install -y liblapack-dev
sudo apt install -y libfftw3-dev
```

**安装 MPI library, 这里选择更为常用的 open MPI**

```
sudo apt install -y libopenmpi-dev
```

**从仓库克隆代码到本地，并进入目录**

```
git clone https://github.com/deepmodeling/abacus-develop.git
cd abacus-develop/
```

**开始编译并安装 abacus**

```
cmake -B build -DENABLE_LCAO=OFF
cd build && make -j`nproc`
```

可以看到在当前目录下，已经编译成功了 abacus_pw 可执行文件,abacus_pw 就是并行版本的支持平面波基组的 ABACUS。

### 2.1.4 安装支持两种基组的 ABACUS

对于同时支持两种基组的 ABACUS，仅可以编译并行版本。并无串行版本。
在本文最初就提到支持 LCAO 基组的 ABACUS 所依赖软件库的更多。

**1. 安装 PW 基组依赖的软件库。（同上，如果已经安装，则不用重复操作）**

```
sudo apt update
sudo apt install -y libopenblas-openmp-dev
sudo apt install -y liblapack-dev
sudo apt install -y libfftw3-dev
```

**2. 安装 MPI library。这里选择更为常用的 open MPI。（同上，如果已经安装，则不用重复操作）**

```
sudo apt install -y libopenmpi-dev
```

**3. 安装 LCAO 基组依赖的 ScaLAPACK 和 CEREAL 软件库。**

```
sudo apt install -y libscalapack-mpi-dev
sudo apt install -y libcereal-dev
```

**4. 安装 ELPA 软件库（一般这里容易出问题）**

如果你的系统是 Ubuntu 22.04，那么你可以以很简单的方式成功安装 ELPA：

```
sudo apt install -y libelpa-dev
```

如果你的系统不是 Ubuntu 22.04，很可惜，你需要稍微辛苦一点来安装 ELPA。但别急，这也并不难！
手动编译安装 ELPA 可以分为以下几步：

- 下载并进入 ELPA 目录：

```
wget https://elpa.mpcdf.mpg.de/software/tarball-
archive/Releases/2021.05.002/elpa-2021.05.002.tar.gz
tar xzf elpa-2021.05.002.tar.gz
cd elpa-2021.05.002
mkdir build  && cd build
```

- configure

```
../configure --enable-openmp CFLAGS="-O3 -march=native -funsafe-
loop-optimizations -funsafe-math-optimizations -ftree-vect-loop-
version -ftree-vectorize" FCFLAGS="-O2 -mavx" --disable-avx512
```

- 编译安装

```
make -j`nproc`
make install
ln -s /usr/local/include/elpa_openmp-2021.05.002/elpa /usr/local/include/
```

注意：ln -s /usr/local/include/elpa_openmp-2021.05.002/elpa /usr/local/include/ 是非常重要的！（很多用户是这里出的问题！）

**5. 开始编译并安装 abacus**

```
cmake -B build
cd build && make -j`nproc`
```

可以看到在当前目录下，已经编译成功了 abacus 可执行文件。
abacus 就是完整版的 ABACUS，它同时支持两种基矢量，而且还是并行版本！

## 2.2 Intel oneAPI 2024/2025 编译 ABACUS 教程

### 2.2.1 Intel OneAPI 工具链

**1. 简介**

本教程介绍如何使用 Intel oneAPI 2024/2025 工具链编译 ABACUS。
和之前的版本相比，新的 oneAPI 工具链在编译器等方面有较大变化。
oneAPI 2024 中，Intel® C++ Compiler Classic (icc) 停止使用，并使用了新的目录布局。
oneAPI 2025 中，Intel® Fortran Compiler Classic (ifort) 停止使用。

#### OneAPI Base Toolkit

OneAPI Base Toolkit 提供了一系列基础工具和库，包括 BLAS, LAPACK, ScaLAPACK and FFTW3 等关键组件：

- Intel® oneAPI DPC++/C++ Compiler：面向 CPU、GPU、FPGA 的 C++ 编译器。
- Intel® oneAPI DPC++ Library：提供并行算法的库。
- Intel® oneAPI Math Kernel Library：即 MKL，提供高度优化的数学函数库，包括线性代数、FFT 等。
- Intel® VTune™ Profiler：性能分析优化工具。

#### OneAPI HPC Toolkit

OneAPI HPC Toolkit 是为高性能计算（HPC）特别优化的工具集，它在 OneAPI Base Toolkit 的基础上增加了更多针对 HPC 应用的特性和工具，包括：

- Intel® oneAPI DPC++/C++ Compiler：含有 C++ MPI 编译器。
- Intel® Fortran Compiler & Intel® Fortran Compiler Classic：Fortran 编译器。
- Intel® Intel® MPI Library：MPI 库。
- Intel® VTune™ Profiler：性能分析优化工具。

### 2. 安装

可以从官网上获得最新的安装包。安装需要管理员权限。

Download the Intel® oneAPI Base Toolkit

Download the Intel® HPC Toolkit

以 2024.2 版本为例，展示配置环境和安装 ABACUS 的流程。

#### Offline installer

#### Base Toolkit

- 下载

```
wget https://registrationcenter-download.intel.com/akdlm/IRC\_NAS/9a98af19-1c68-46ce-9fdd-e24
9240c7c42/l\_BaseKit\_p\_2024.2.0.634\_offline.sh
```

- 安装，可选图形化界面安装

```
sudo sh ./l\_BaseKit\_p\_2024.2.0.634\_offline.sh
```

- 或直接在 shell 中静默安装

```
sudo sh ./l\_BaseKit\_p\_2024.2.0.634\_offline.sh -a --silent --cli --eula accept
```

#### Base Toolkit

- 下载

```
wget https://registrationcenter-download.intel.com/akdlm/IRC_NAS/d4e49548-1492-45c9-b678-8268
cb0f1b05/l_HPCKit_p_2024.2.0.635_offline.sh
```

- 安装，类似

```
sudo sh ./l_HPCKit_p_2024.2.0.635_offline.sh -a --silent --cli --eula accept
```

- 按照默认配置，全局安装目录位于/opt/intel/oneapi。以 2024.2 版本为例，该目录如下：

```
  oneapi ls
2024.2   basekit  common  dal  dev-utilities  dnnl  dpl  installer  ippcp  logs
modulefiles-setup.sh  setvars.sh  tbb  vtune advisor  ccl  compiler  debugger  diagnostics
dpcpp-ct  hpckit  ipp  licensing  mkl  mpi  support.txt  tcm
```

- 其中有一些重要的目录和工具，如 2024.2/ 和 installer/

**3. 环境变量设置**

要使用 oneAPI 提供的编译器和库，需要正确设置环境。

2024.0 及之后的版本，使用了新的目录布局。这导致了和老版本不同的环境配置方式。参见 Use the setvars and oneapi-vars Scripts with Linux*

和之前的版本相比，由组件目录布局（Component Directory Layout）改为统一目录布局（Unified Directory Layout），新版本的所有组件（bin, lib, include, share）等统一放在以工具包版本号命名的顶级目录中。在原先的 Component Directory Layout 中，不同的组件有各自的环境变量设置脚本，由一个位于 oneAPI 安装目录的脚本 /opt/intel/oneapi/setvars.sh 统一管理。新版的 Unified Directory Layout 中，每个组件被集中到组件共用的共享文件夹中，即每个组件将其头文件提供给一个公共的 include 文件夹，将其库文件提供给一个公共的 lib 文件夹，以此类推。这样，不同版本工具包之间的切换更容易，无需维护通用的 setvars.sh，而是通过工具包版本号命名的目录提供的脚本 /opt/intel/oneapi/<toolkit-version>/oneapi-vars.sh 设置。

以 2024.2 为例，每次使用 icpx 等编译器之前，需要在 shell 环境中 source 一次脚本：

```
. /opt/intel/oneapi/2024.2/oneapi-vars.sh
# configure, build, ...
```

此时，编译器应该被添加到环境中，可以运行命令检查是否正确配置：

```
# source 前
-> oneapi mpiicpx -v
zsh: command not found: mpiicpx
# source 后
-> oneapi mpiicpx -v
mpiicpx for the Intel(R) MPI Library @IMPI_OFFICIALVERSION@ for Linux*
Copyright Intel Corporation.
Intel(R) oneAPI DPC++/C++ Compiler 2024.2.0 (2024.2.0.20240602)
Target: x86_64-unknown-linux-gnu
Thread model: posix
InstalledDir: /opt/intel/oneapi/compiler/2024.2/bin/compiler
Configuration file: /opt/intel/oneapi/compiler/2024.2/bin/compiler/../icpx.cfg
Found candidate GCC installation: /usr/lib/gcc/x86_64-linux-gnu/11
Selected GCC installation: /usr/lib/gcc/x86_64-linux-gnu/11
Candidate multilib: .;@m64
Selected multilib: .;@m64
```

```
icpx: warning: argument unused during compilation: '-I /opt/intel/oneapi/2024.2/include'
[-Wunused-command-line-argument]
```

- . 命令在 POSIX 标准中定义，因此它在所有 POSIX 兼容的 shell 中都应该可用。
- source 命令一般情况下是. 的别名，但在某些环境中不可用。如果脚本不能使用 source，请换成通用的..

这一步会配置环境变量，但是并不会设定默认的 C++ 编译器。因此，设置了 oneAPI 编译器的环境变量之后，还需要在配置（configure）时指定构建（build）采用的编译器，如 CXX=icpx。

- 2024.0 开始，原先的 classical 编译器停止使用。

请使用新的 icpx/mpiicpx 取代原来的 icpc/mpiicpc。

### 2.2.2 ABACUS 安装

**1. 依赖库**

**安装 cereal**

```
sudo apt install libcereal-dev
```

**安装 elpa**

在 Ubuntu22.04 等发行版中，可以通过 apt 获取预构建的 elpa 包（pre-build packages）。不幸的是，使用不同的 MPI 库构建 elpa 可能会导致冲突，apt 获取的 elpa 包和 oneAPI 2024.2 构建的 abacus 在运行时可能出现问题。

```
/usr/bin/ld: warning: libmpi.so.40, needed by /usr/lib/x86_64-linux-gnu/libelpa.so, may conflict
with libmpi.so.12
```

运行算例时报错：

```
* * * * * *
 << Start SCF iteration.
Abort(403251971) on node 0 (rank 0 in comm 0): Fatal error in internal_Bcast: Unknown error class,
error stack:
internal_Bcast(4152): MPI_Bcast(buffer=0x2c0e960, count=1, INVALID DATATYPE, 1, comm=0xc400001b)
failed
internal_Bcast(4112): Invalid datatype
Abort(67707651) on node 1 (rank 1 in comm 0): Fatal error in internal_Bcast: Unknown error class,
error stack:
internal_Bcast(4152): MPI_Bcast(buffer=0x2fcdac0, count=1, INVALID DATATYPE, 1, comm=0xc4000013)
failed
internal_Bcast(4112): Invalid datatype
```

因此，我们需要手动利用 oneAPI 工具链构建 elpa。

**使用 toolchain**

由于 elpa 的编译较复杂，我们可以通过 abacus toolchain 脚本自动构建和安装 elpa。该脚本用 icpx 和 mpiicpc 编译 elpa，同时会默认安装 cereal 和 libxc。

一键配置编译 ABACUS | toolchain 脚本的使用

```
. /opt/intel/oneapi/2024.2/oneapi-vars.sh
cd abacus-develop/toolchain
./toolchain_intel.sh # 脚本将会用 intel 工具链安装依赖。
```

安装完成的库目录为 abacus-develop/toolchain/install.

```
# 安装完成的库目录为 abacus-develop/toolchain/install
-> toolchain git:(develop)  ls install
cereal-1.3.2  cmake-3.28.1  elpa-2023.05.001  libxc-6.2.2  lsan.supp  setup  toolchain.conf
toolchain.env  tsan.supp  valgrind.supp
```

使用 toolchain 构建的 elpa，在构建 abacus 时有警告如下，暂未发现影响使用。该问题由 scalapack 库未指定 mkl 版本导致，“自行编译安装”方式（见下）无此问题。

参见2.4 Non standard paths or non standard libraries

```
/usr/bin/ld: warning: libmpi.so.40, needed by /lib/x86_64-linux-gnu/libscalapack-openmpi.so.2.1,
may conflict with libmpi.so.12
```

除了使用 toolchain，也可以根据 documentation/INSTALL.md · master · elpa / elpa · GitLab，自行编译安装 elpa。

#### 自行编译安装

自行下载和安装 elpa 到环境中。以 oneAPI 2024.2 和 elpa-2024.05.001 为例。

- 在官网下载 elpa 包。

```
wget https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2024.05.001/elpa-2024.05.001.tar.gz

tar xvf elpa-2024.05.001.tar.gz
```

- 以 oneAPI 2024.2 为例提供一个安装脚本，仅供参考。这一步可能需要几分钟。如果运行脚本时显示 oneAPI warnings 提示已经设置过环境变量并退出，请打开一个新的 shell 环境，执行安装脚本。最后的 make install 和 ln 需要 sudo 权限，如果无法提供，请手动完成这两步。
- oneAPI 2025.0 开始，Intel® Fortran Compiler Classic (ifort) 停止使用。

请使用新的 ifx/mpiifx 取代原来的 ifort/mpiifort。

请根据实际情况，指定安装脚本中的 oneAPI 工具链版本。如果使用 oneAPI 2025.0 之后的版本，请把 21 行 FC=mpiifort 替换为 FC=mpiifx。

```
cd elpa-2024.05.001
```

```
#!/bin/bash -e

# buildelpa.sh
# run in elpa main dir

# source oneAPI environments
echo "using oneAPI 2024.2"
```

```
. /opt/intel/oneapi/2024.2/oneapi-vars.sh \
|| { echo "Failed to load oneAPI environment. Please restart in a new shell without oneAPI vars
set."; false; }

# in elpa main dir
# check whether there is a 'build' directory
if [ -d "build" ]; then
    echo "rm -rf build"
    rm -rf build
fi
mkdir build \&\& cd build

MKL_HOME=/opt/intel/oneapi/2024.2

__CC=mpiicx CXX=mpiicpx FC=mpiifort ../configure \
--disable-avx --disable-avx2 --disable-avx512 --disable-sse --disable-sse-assembly \
SCALAPACK_LDFLAGS="-L\$MKL_HOME/lib/ -lmkl_scalapack_lp64 -lmkl_intel_lp64 -lmkl_sequential \
                           -lmkl_core -lmkl_blacs_intelmpi_lp64 -lpthread -lm -Wl,-rpath,
                           \$MKL_HOME/lib/" \
SCALAPACK_FCFLAGS="-L\$MKL_HOME/lib/ -lmkl_scalapack_lp64 -lmkl_intel_lp64 -lmkl_sequential \
                    -lmkl_core -lmkl_blacs_intelmpi_lp64 -lpthread -lm -I\$MKL_HOME/include/mkl/
                    intel64/lp64"

make -j\$(nproc) > make.log 2>\&1
echo "installation process may require administrative privileges."
read -p "Would you like to continue with 'sudo make install'? (y/n): " -n 1 -r
echo    # (Optional) Move to a new line

if [[ \$REPLY =~ ^[Yy]\$ ]]
then
    echo "Proceeding with installation using administrative privileges..."
    sudo make install > install.log 2>\&1
else
    echo "Installation has been canceled."
    echo "Please manually execute 'make install' and link elpa to default path."
fi

# link elpa to /usr/local/include

# The target path for the symbolic link
LINK_PATH="/usr/local/include/elpa"
# The source path for the original link (replace with the actual source path)
SOURCE_PATH="/usr/local/include/elpa-2024.05.001/elpa"

# Check if the link exists
if [ -L "\$LINK_PATH" ]; then
    # If the link exists, delete it
    echo "The link already exists, deleting the old link..."
```

```
    sudo rm "\$LINK_PATH"
else
    # If the link does not exist, check for the presence of a file or directory
    if [ -e "\$LINK_PATH" ]; then
        echo "A file or directory exists at the path, unable to create the link.
        Please delete or rename the file/directory first."
        exit 1
    fi
fi

# Create a new symbolic link
sudo ln -s "\$SOURCE_PATH" "\$LINK_PATH"

# Check if the link was created successfully
if [ -L "\$LINK_PATH" ]; then
    echo "The new symbolic link has been created successfully."
else
    echo "Failed to create the symbolic link."
    exit 1
fi

echo "elpa install over."
```

**2. 安装 ABACUS**

配置好依赖后，我们可以开始安装 ABACUS。

如果你使用 toolchain 配置环境，那么 toolchain 提供了一系列环境变量和构建工具。构建时，请利用 abacus-develop/toolchain/build_abacus_intel.sh 脚本直接构建 ABACUS（在其中修改配置选项），也可以手动构建。（不推荐）

如果需要手动安装 ABACUS 并使用 toolchain 的 ELPA：需要记住此前 toolchain 安装 elpa 的目录。

如果之前选择手动安装 ELPA，可以用默认方式自行用 cmake 构建。

```
# 设置环境
. /opt/intel/oneapi/2024.2/oneapi-vars.sh
# configure
# 在此选择 oneAPI 的编译器，添加编译选项，指定此前的安装路径，如
CXX=mpiicpx cmake -B build \
-DELPA_DIR=~/abacus-develop/toolchain/install/elpa-2023.05.001/cpu/
# 根据需要添加其他 cmake 选项

# build
cmake --build build -j`nproc`
# install
cmake --install build
```

### 2.2.3 常见问题和解决方法

在开始构建之前，请清除原有的 build 目录。

```
rm -rf build
```

找不到编译器，记得运行 vars 脚本设置 oneAPI 环境变量。

```
CMake Error at /usr/share/cmake-3.22/Modules/CMakeDetermineCXXCompiler.cmake:48 (message):
  Could not find compiler set in environment variable CXX:

  mpiicpx.

Call Stack (most recent call first):
  CMakeLists.txt:7 (project)


CMake Error: CMAKE_CXX_COMPILER not set, after EnableLanguage
-- Configuring incomplete, errors occurred!

# 请设置环境
. /opt/intel/oneapi/2024.2/oneapi-vars.sh
```

找不到 elpa，请在配置时指定安装路径。

```
CMake Error in CMakeLists.txt:
  Imported target "ELPA::ELPA" includes non-existent path

    "/usr/include/elpa"

  in its INTERFACE_INCLUDE_DIRECTORIES.  Possible reasons include:

  * The path was deleted, renamed, or moved to another location.

  * An install or uninstall procedure did not complete successfully.

  * The installation package was faulty and references files it does not
  provide.


# 请指定 ELPA_DIR（及其他自己手动构建的库）
CXX=mpiicpx cmake -B build \
-DELPA_DIR=~/abacus-develop/toolchain/install/elpa-2023.05.001/cpu/
```

期望使用 intel 工具链编译，但 cmake 显示使用 GNU 工具链。请使用 CXX=mpiicpx 指定编译器。

```
cmake -B build -DELPA_DIR=~/abacus-develop/toolchain/install/elpa-2023.05.001/cpu/
-- The CXX compiler identification is GNU 11.4.0

# CXX=mpiicpx cmake ...
# -- The CXX compiler identification is IntelLLVM 2024.2.0
```

```
1
```

cmake 提示编译器错误，请使用新版的 icpx/mpiicpx，而不是 icpc/mpiicpc

```
1 CMake Error at /usr/share/cmake-3.22/Modules/CMakeTestCXXCompiler.cmake:62 (message):
2   The C++ compiler
3
4     "/opt/intel/oneapi/2024.2/bin/mpiicpc"
5
6   is not able to compile a simple test program.
7
8 # use CXX=mpiicpx instead of CXX=mpiicpc
```

运行了 oneAPI 配置环境变量脚本，但是链接错误

请检查 oneAPI HPC kits 的安装；可进入 installer 查看当前安装的所有 Toolkits 和对应版本，见下条。

使用 /opt/intel/oneapi/2024.2/oneapi-vars.sh，而不是 /opt/intel/oneapi/setvars.sh

如果安装了多版本的 oneAPI 工具链，怀疑环境遭到破坏，可以使用 /opt/intel/oneapi/installer 中的 installer 工具修复和移除不需要版本以及更新。

```
1 cd /opt/intel/oneapi/installer
2 sudo ./installer
3
4 # 可以用 Repair 尝试修复环境
5 # 使用 Remove 移除不需要的组件
6 # 使用 Update 获得新版本
```

如果遇到 libmpi.so 相关报错，可以用 locate 查看所有相关库。

```
1 locate libmpi.so
```

运行算例或测试失败，请确保最新构建后运行了 install 命令，且没有因为权限不足安装失败。

### 2.2.4 参考

Abacus 文档

Easy Installation

一键配置编译 ABACUS | toolchain 脚本的使用

Intel oneAPI 编译 ABACUS 教程 · GitBook

elpa

documentation/INSTALL.md · master · elpa / elpa · GitLab

Intel 文档

Use the setvars and oneapi-vars Scripts with Linux*

Intel® C++ Compiler Classic Release Notes

Porting Guide for DPCPP or ICX

Porting Guide for ifort Users to ifx

Intel® oneAPI Base Toolkit Release Notes

## 2.3 Intel oneAPI 编译 ABACUS 教程

### 2.3.1 前言

非特殊情况，Linux 系统会默认安装 GCC（GNU Compiler Collection）编译器套件。因此如果你的机器默认没有 Intel® oneAPI Toolkits，那么建议，你可以直接看 2.1：GCC 编译 ABACUS 教程。

如果您的机器恰好有 Intel® oneAPI Toolkits，那么您也可以使用它来编译 ABACUS。

### 2.3.2 关于 Intel® oneAPI Toolkits

Intel® oneAPI Toolkits 分为很多个版本的 Toolkit，这里解释下 Intel® oneAPI Base Toolkit 和 Intel® oneAPI HPC Toolkit 的区别。

简单来说：Intel® oneAPI HPC Toolkit 包含 Intel® oneAPI Base Toolkit。

Intel® oneAPI Base Toolkit 中包含：Intel® oneAPI Math Kernel Library（简称为 MKL），这个软件库里面包含了大量常用的数学库，包括 ABACUS 依赖的 BLAS、LAPACK、FFTW3 和 ScaLAPACK 4 个软件库。

因此使用 Intel oneAPI 来编译 ABACUS，可以省去逐个去安装数学库的过程！

Intel® oneAPI HPC Toolkit 相比于 Intel® oneAPI Base Toolkit 多了一些必要的 Intel 的编译器以及 Intel® MPI Library（ABACUS 想要编译并行版本必须的关键库）。

总结一下，使用 Intel oneAPI 编译 ABACUS，必须确保你的机器中包含 Intel® oneAPI HPC Toolkit。

关于 Intel® oneAPI Toolkits 更详细的信息：
https://www.intel.com/content/www/us/en/developer/tools/oneapi/toolkits.html#hpc-kit

### 2.3.3 编译安装

开启 Intel® oneAPI Toolkits：

```
source /opt/intel/oneapi/setvars.sh
```

ELPA 仅用于 LCAO 基组求解特征值与特征矩阵的过程，如果由于机器等原因导致 ELPA 难以安装，可以暂不安装 ELPA，在编译 abacus 的时候添加-DUSE_ELPA=0 即可绕过 ELPA 安装困难的问题。在 ABACUS 中，LCAO 基组求解特征值与特征矩阵提供了两种方法，一种是调用 ELPA 进行求解（默认选项），一种是调用 ScaLAPACK 进行求解。如果使用-DUSE_ELPA=0 编译选项，请参考使用文档配置 ks_solver 为 scalapack_gvx，调用 ScaLAPACK 进行求解。（http://abacus.deepmodeling.com/en/stable/advanced/input_files/input-main.html#ks-solver）

#### 1. 安装仅支持 PW 基组的 ABACUS

对于仅支持 PW 基组的 ABACUS，我们同时支持两种版本的编译：串行版本和并行版本。

ABACUS 的 PW 基组只依赖：BLAS、LAPACK、FFTW3 三个数学库，而这三个数学库都已经被 Intel® oneAPI Base Toolkit 中包含：Intel® oneAPI Math Kernel Library（简称为 MKL）所包含，因此我们不需要再安装其他的软件库！

**编译串行版本**

```
CXX=icpx cmake -B build -DENABLE_MPI=OFF
cd build && make -j`nproc`
```

注意，这里必须指定 CXX=icpx，不然系统会使用默认的 CXX 编译器。同时这里的 CXX 也不要指定 icpc，icpc 是 Intel® C++ Compiler Classic，会带来一些报错。

可以看到在当前目录下，已经编译成功了 abacus_pw_serial 可执行文件。

abacus_pw_serial 就是串行版本的支持平面波基组的 ABACUS。

#### 编译并行版本

```
CXX=icpx cmake -B build -DENABLE_LCAO=OFF
cd build && make -j`nproc`
```

可以看到在当前目录下，已经编译成功了 abacus_pw 可执行文件。

abacus_pw 就是并行版本的支持平面波基组的 ABACUS。

### 2. 安装支持两种基组的 ABACUS

对于同时支持两种基组的 ABACUS，仅可以编译并行版本。并无串行版本。

完整版的 ABACUS 需要依赖更多的软件库：BLAS、LAPACK、FFTW3、ScaLAPACK、CEREAL、ELPA 共 6 个。

而前四个数学库（BLAS、LAPACK、FFTW3、ScaLAPACK）都已经被 Intel MKL 所包含，因此我们只需再额外安装 CEREAL、ELPA 即可。

#### 安装 CEREAL

```
sudo apt install -y libcereal-dev
```

#### 安装 ELPA

如果你的系统是 Ubuntu 22.04，那么你可以以很简单的方式成功安装 ELPA。

```
sudo apt install -y libelpa-dev
```

如果你的系统不是 Ubuntu 22.04，很可惜，你需要稍微辛苦一点来安装 ELPA。但别急，这也并不难！

注意；这里手动编译安装 ELPA 的方法与在 GCC 下编译不太一样，因为要保证 ELPA 和 ABACUS 都是基于 Intel oneAPI 来编译的！

手动编译安装 ELPA 可以分为以下几步：

下载并进入 ELPA 目录：

```
wget https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2021.05.002/elpa-2021.05.002.tar.gz
tar xzf elpa-2021.05.002.tar.gz
cd elpa-2021.05.002
mkdir build  && cd build
```

configure

```
CC=mpiicc CXX=mpiicpc FC=mpiifort ../configure --enable-openmp FCFLAGS="-qmkl=cluster"
```

编译安装

```
make -j`nproc`
make install
ln -s /usr/local/include/elpa_openmp-2021.05.002/elpa /usr/local/include/
```

注意：ln -s /usr/local/include/elpa_openmp-2021.05.002/elpa /usr/local/include/ 是非常重要的！（很多用户是这里出的问题！）

**编译安装 ABACUS**

```
CXX=icpx cmake -B build
cd build && make -j`nproc`
```

可以看到在当前目录下，已经编译成功了 abacus 可执行文件。

abacus 就是完整版的 ABACUS，它同时支持两种基矢量，而且还是并行版本！

## 2.4 Toolchain 编译 ABACUS 教程 (1) – GCC

### 2.4.1 简介

从前面的教程中我们可以发现，ABACUS 的源码编译涉及到众多依赖库，如 CMake 编译管理器，C/C++/Fortran 编译器，MPI 并行库、数学库、泛函库和其他功能依赖库（如 LCAO 基组，杂化泛函计算，abacus-json 支持）等，从源码编译需要正确编译这些依赖库，并在编译 ABACUS 时正确声明各个依赖库，对用户的软件编译能力提出了挑战。尤其对于广大对 Linux 并不十分熟悉的用户而言，编译软件会变得比使用软件更加困难。

那么，有相对简单高效的办法吗，答案是有的，这就是 ABACUS Toolchain。

### 2.4.2 什么是 ABACUS Toolchain

ABACUS Toolchain 是一套内置于 ABACUS 仓库中的 bash 脚本集，它能帮助用户在线或离线地编译安装 ABACUS 所需的软件依赖，自动处理各个依赖库的环境变量，并基于这些依赖库快速完成 ABACUS 源码编译过程。本教程基于 2025-02 版本的 ABACUS Toolchain 撰写，目前，ABACUS Toolchain 支持如下编译安装功能：

- GCC Toolchain，即从足够版本的 GCC（也称 GNU）编译套件（gcc, g++, gfortran）出发，从头编译安装 ABACUS 依赖库及 ABACUS 本体的 Toolchain 方法。
- Intel Toolchain，即基于 Intel 的编译器、数学库和并行库（通常打包在 Intel-OneAPI 或 Intel-parallel-xe-studio 内）编译安装 ABACUS 依赖库及 ABACUS 本体的 Toolchain 方法。
- AMD Toolchain，即基于 AMD 的编译器和数学库进行 ABACUS 编译安装的方法，细分为两种：
  - GCC-AOCL Toolchain，即基于系统 gcc 编译器和 AMD 的 AOCL 数学库编译安装 ABACUS 依赖及其本体。
  - AOCC-AOCL Toolchain，即基于 AMD 的 AOCC 编译器和 AOCL 数学库编译安装 ABACUS 依赖及其本体。

同时，ABACUS Toolchain 还支持一些高级功能，如：

- 支持几乎所有 ABACUS 的额外功能插件的编译（如 LibRI, DeePKS, DeePMD 等）。
- 离线安装 ABACUS 依赖库和 ABACUS 本体。
- 安装 GPU 支持的 ABACUS。
- 使用系统已有的依赖库配合 Toolchain 下载的依赖库安装 ABACUS（可通过环境变量或绝对路径指定两种方法）

- 将 ABACUS 的依赖库安装到指定位置。
- 切换不同的 ABACUS 依赖库版本.

总体来说，ABACUS Toolchain 希望达到的愿景是：

- 方便用户高效地从源码编译最适合当前服务器环境的 ABACUS，并能快速测试不同依赖库类型的 Toolchain 编译所得 ABACUS 的计算效率。
- 建立一套 ABACUS 源码编译的标准流程，ABACUS 开发者可以直接在 Toolchain 中控制 ABACUS 各个依赖库的版本和编译方式，而不需要自行编译并手动加入各种编译选项。与此同时，如果 ABACUS 的安装依赖有所更新，开发者也可以在 Toolchain 中直接更新相关信息，即可为用户使用。

### 2.4.3 基于 GCC Toolchain 安装 ABACUS

下面我们将以 GCC Toolchain 为例，展示安装 LTS-3.10.0 版本 ABACUS 的过程，并让各位用户快速上手 Toolchain 使用。GCC Toolchain 也是基于 Toolchain 的 ABACUS 编译安装方法中通用性最好的。

1. 下载 ABACUS 仓库: 可以通过 git clone 下载，这一做法允许用户通过 git 包管理器快速更新或切换版本。也可以通过 wget 拉取 ABACUS 仓库压缩包，这样得到的 ABACUS 仓库没有 git 相关的信息，文件体积更小。

```
# via github
git clone https://github.com/deepmodeling/abacus-develop.git -b LTS
# via wget from github by codeload: can be used under CN Internet
wget https://codeload.github.com/deepmodeling/abacus-develop/tar.gz/LTS -O abacus-LTS.tar.gz
tar -zxvf abacus-LTS.tar.gz
```

2. 进入 Toolchain 目录

```
cd abacus-develop/toolchain
# if you download abacus via wget from codeload
cd abacus-develop-LTS/toolchain
```

3. 运行 toolchain_gnu.sh 脚本

```
sh ./toolchain_gnu.sh
```

如此即可开始基于 Toolchain 的 ABACUS 依赖库编译安装。注：在使用 Toolchain 安装之前，务必确保你的服务器相关环境变量是基本干净的！如使用 GNU Toolchain 时，不能加载 Intel OneAPI 的环境变量；或是不能有奇特的 mpirun 存在于 conda 包内等。

以 GCC Toolchain 默认设置为例，如此运行后，会做如下操作：

1. 检查你的系统 GCC 编译器版本（2025-02 版本 Toolchain 新功能），并链接 GCC 编译器。输出类似于：

```
Checking system GCC version for gcc, intel and amd toolchain
Your System gcc/g++/gfortran version should be consistent
Minimum required version: 5
Your gcc version: 11.4.0
Your g++ version: 11.4.0
Your gfortran version: 11.4.0
Your GCC version seems to be enough for ABACUS installation.
Compiling with 128 processes for target native.
==================== Finding GCC from system paths ====================
path to gcc is /usr/bin/gcc
```

```
path to g++ is /usr/bin/g++
path to gfortran is /usr/bin/gfortran
Found include directory /usr/include
Step gcc took 0.00 seconds.
Step intel took 0.00 seconds.
Step amd took 0.00 seconds.
```

注：编译 ABACUS 及其相关依赖是有系统 C++ 编译器最低版本要求的，至少要求支持 C++11 特性。同时，有一些依赖库还要求一部分的 C++14 特性，这都要求 GCC 版本至少不能小于 5，即使是使用 Intel 或 AMD 相关依赖库和编译器进行编译安装时也是如此。然而，有一部分老旧超算依然维持着 CentOS 的内核和 4.8.5 版本的 GCC，这些超算上的 ABACUS 编译需要加载更高版本的 GCC，或基于 Toolchain 编译安装 GCC。用户可以通过 gcc –version 命令检查自己的 gcc 版本。

2. 下载 OpenBLAS，通过 OpenBLAS 内置的 genarch 程序识别系统指令集和架构，在不同服务器上的不同输出如下

```
# Run toolchain on Intel-CPU Machine
==================== Getting proc arch info using OpenBLAS tools ====================
wget  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29 -O OpenBLAS-0.3.29.tar.gz --no-check
--2025-05-06 16:58:40--  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'OpenBLAS-0.3.29.tar.gz'

OpenBLAS-0.3.29.tar.gz         [               <=>                  ]  23.53M  6.75MB/s    in 3.5s

2025-05-06 16:58:45 (6.75 MB/s) -  'OpenBLAS-0.3.29.tar.gz'  saved [24671913]

OpenBLAS-0.3.29.tar.gz: OK
Checksum of OpenBLAS-0.3.29.tar.gz Ok
OpenBLAS detected LIBCORE = skylakex
OpenBLAS detected ARCH    = x86_64

# Run toolchain on AMD-CPU Machine
==================== Getting proc arch info using OpenBLAS tools ====================
wget  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29 -O OpenBLAS-0.3.29.tar.gz --no-check
--2025-05-06 04:49:02--  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'OpenBLAS-0.3.29.tar.gz'

OpenBLAS-0.3.29.tar.gz         [                           <=>       ]  23.53M  2.77MB/s    in 10s

2025-05-06 04:49:13 (2.35 MB/s) -  'OpenBLAS-0.3.29.tar.gz'  saved [24671913]

```

```
OpenBLAS-0.3.29.tar.gz: OK
Checksum of OpenBLAS-0.3.29.tar.gz Ok
OpenBLAS detected LIBCORE = zen
OpenBLAS detected ARCH    = x86_64
```

3. 自动下载并编译安装：编译套件 CMake; 并行库 OpenMPI; 数学库 OpenBLAS, ScaLAPACK, FFTW, ELPA; 泛函库 LibXC; 以及其他功能需要的库 CEREAL, RapidJSON 和其他可选项 LibRI, LibComm, LibTorch, Libnpy 等。以 AMD-7b12 机器编译为例，完整输出如下：

```
MPI is detected and defaults to MPICH
Checking system GCC version for gcc, intel and amd toolchain
Your System gcc/g++/gfortran version should be consistent
Minimum required version: 5
Your gcc version: 11.4.0
Your g++ version: 11.4.0
Your gfortran version: 11.4.0
Your GCC version seems to be enough for ABACUS installation.
Compiling with 128 processes for target native.
==================== Finding GCC from system paths ====================
path to gcc is /usr/bin/gcc
path to g++ is /usr/bin/g++
path to gfortran is /usr/bin/gfortran
Found include directory /usr/include
Step gcc took 0.00 seconds.
Step intel took 0.00 seconds.
Step amd took 0.00 seconds.
==================== Getting proc arch info using OpenBLAS tools ====================
wget  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29 -O OpenBLAS-0.3.29.tar.gz --no-check
--2025-05-06 04:49:02--  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'OpenBLAS-0.3.29.tar.gz'

OpenBLAS-0.3.29.tar.gz         [                           <=>       ]  23.53M  2.77MB/s    in 10s

2025-05-06 04:49:13 (2.35 MB/s) -  'OpenBLAS-0.3.29.tar.gz'  saved [24671913]

OpenBLAS-0.3.29.tar.gz: OK
Checksum of OpenBLAS-0.3.29.tar.gz Ok
OpenBLAS detected LIBCORE = zen
OpenBLAS detected ARCH    = x86_64
==================== Installing CMake ====================
wget  https://cmake.org/files/v3.31/cmake-3.31.7-linux-x86_64.sh -O cmake-3.31.7-linux-x86_64.sh --no-chec
--2025-05-06 04:49:17--  https://cmake.org/files/v3.31/cmake-3.31.7-linux-x86_64.sh
Resolving cmake.org (cmake.org)... 66.194.253.25
Connecting to cmake.org (cmake.org)|66.194.253.25|:443... connected.
```

```
HTTP request sent, awaiting response... 200 OK
Length: 55005854 (52M) [text/x-sh]
Saving to:  'cmake-3.31.7-linux-x86_64.sh'

cmake-3.31.7-linux-x86_64. 100%[=====================================>]  52.46M  3.23MB/s    in 24s

2025-05-06 04:49:42 (2.23 MB/s) -  'cmake-3.31.7-linux-x86_64.sh'  saved [55005854/55005854]

cmake-3.31.7-linux-x86_64.sh: OK
Checksum of cmake-3.31.7-linux-x86_64.sh Ok
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/cm
Step cmake took 27.00 seconds.
==================== Installing OpenMPI ====================
wget  https://download.open-mpi.org/release/open-mpi/v5.0/openmpi-5.0.7.tar.bz2 -O openmpi-5.0.7.tar.bz2 -
--2025-05-06 04:49:44--  https://download.open-mpi.org/release/open-mpi/v5.0/openmpi-5.0.7.tar.bz2
Resolving download.open-mpi.org (download.open-mpi.org)... 2600:9000:2200:c000:16:d6ed:ec80:93a1, 2600:900
Connecting to download.open-mpi.org (download.open-mpi.org)|2600:9000:2200:c000:16:d6ed:ec80:93a1|:443...
HTTP request sent, awaiting response... 200 OK
Length: 30506871 (29M) [binary/octet-stream]
Saving to:  'openmpi-5.0.7.tar.bz2'

openmpi-5.0.7.tar.bz2      100%[=====================================>]  29.09M  33.0MB/s    in 0.9s

2025-05-06 04:49:46 (33.0 MB/s) -  'openmpi-5.0.7.tar.bz2'  saved [30506871/30506871]

openmpi-5.0.7.tar.bz2: OK
Checksum of openmpi-5.0.7.tar.bz2 Ok
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/op
Found directory /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi-5.0.7/b
Found directory /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi-5.0.7/l
Found directory /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi-5.0.7/i
mpiexec is installed as /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi
mpicc is installed as /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi-5
mpicxx is installed as /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi-
mpifort is installed as /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi
Step openmpi took 324.00 seconds.
==================== Installing OpenBLAS ====================
OpenBLAS-0.3.29.tar.gz is found
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/op
Installing OpenBLAS library for target ZEN
Step openblas took 84.00 seconds.
==================== Installing LIBXC ====================
wget  https://gitlab.com/libxc/libxc/-/archive/7.0.0/libxc-7.0.0.tar.bz2 -O libxc-7.0.0.tar.bz2 --no-check
--2025-05-06 04:56:32--  https://gitlab.com/libxc/libxc/-/archive/7.0.0/libxc-7.0.0.tar.bz2
Resolving gitlab.com (gitlab.com)... 2606:4700:90:0:f22e:fbec:5bed:a9b9, 172.65.251.78
Connecting to gitlab.com (gitlab.com)|2606:4700:90:0:f22e:fbec:5bed:a9b9|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/octet-stream]
```

```
Saving to: ‘libxc-7.0.0.tar.bz2’

libxc-7.0.0.tar.bz2           [       <=>                             ]  49.98M  38.5MB/s    in 1.3s

2025-05-06 04:56:34 (38.5 MB/s) - ‘libxc-7.0.0.tar.bz2’ saved [52408700]

libxc-7.0.0.tar.bz2: OK
Checksum of libxc-7.0.0.tar.bz2 Ok
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/li
Step libxc took 49.00 seconds.
==================== Installing FFTW ====================
wget  http://www.fftw.org/fftw-3.3.10.tar.gz -O fftw-3.3.10.tar.gz --no-check-certificate
--2025-05-06 04:57:21--  http://www.fftw.org/fftw-3.3.10.tar.gz
Resolving www.fftw.org (www.fftw.org)... 216.137.187.110
Connecting to www.fftw.org (www.fftw.org)|216.137.187.110|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4144100 (4.0M) [application/octet-stream]
Saving to: ‘fftw-3.3.10.tar.gz’

fftw-3.3.10.tar.gz         100%[=====================================>]   3.95M   859KB/s    in 4.7s

2025-05-06 04:57:27 (859 KB/s) - ‘fftw-3.3.10.tar.gz’ saved [4144100/4144100]

fftw-3.3.10.tar.gz: OK
Checksum of fftw-3.3.10.tar.gz Ok
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/ff
install float version fftw
clean
install double version fftw
Step fftw took 142.00 seconds.
==================== Installing ScaLAPACK ====================
wget  https://codeload.github.com/Reference-ScaLAPACK/scalapack/tar.gz/v2.2.2 -O scalapack-2.2.2.tar.gz --
--2025-05-06 04:59:43--  https://codeload.github.com/Reference-ScaLAPACK/scalapack/tar.gz/v2.2.2
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to: ‘scalapack-2.2.2.tar.gz’

scalapack-2.2.2.tar.gz         [           <=>                         ]   4.55M  1.64MB/s    in 2.8s

2025-05-06 04:59:47 (1.64 MB/s) - ‘scalapack-2.2.2.tar.gz’ saved [4769347]

scalapack-2.2.2.tar.gz: OK
Checksum of scalapack-2.2.2.tar.gz Ok
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/sc
Step scalapack took 12.00 seconds.
==================== Installing ELPA ====================
```

```
wget  https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2025.01.001/elpa-2025.01.001.tar.gz -O e
--2025-05-06 04:59:55--  https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2025.01.001/elpa-2025
Resolving elpa.mpcdf.mpg.de (elpa.mpcdf.mpg.de)... 130.183.17.94
Connecting to elpa.mpcdf.mpg.de (elpa.mpcdf.mpg.de)|130.183.17.94|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2169795 (2.1M) [application/gzip]
Saving to:  'elpa-2025.01.001.tar.gz'

elpa-2025.01.001.tar.gz    100%[=====================================>]   2.07M  1.68MB/s    in 1.2s

2025-05-06 04:59:58 (1.68 MB/s) -  'elpa-2025.01.001.tar.gz'  saved [2169795/2169795]

elpa-2025.01.001.tar.gz: OK
Checksum of elpa-2025.01.001.tar.gz Ok
Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/el
Step elpa took 121.00 seconds.
==================== Installing CEREAL ====================
===> Notice: This version of CEREAL is downloaded in GitHub master repository  <===
wget  https://codeload.github.com/USCiLab/cereal/tar.gz/master -O cereal-master.tar.gz --no-check-certific
--2025-05-06 05:01:56--  https://codeload.github.com/USCiLab/cereal/tar.gz/master
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'cereal-master.tar.gz'

cereal-master.tar.gz           [    <=>                                 ] 377.35K   381KB/s    in 1.0s

2025-05-06 05:01:59 (381 KB/s) -  'cereal-master.tar.gz'  saved [386409]

Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/ce
Step cereal took 3.00 seconds.
==================== Installing RAPIDJSON ====================
===> Notice: This version of rapidjson is downloaded in GitHub master repository <===
wget  https://codeload.github.com/Tencent/rapidjson/tar.gz/master -O rapidjson-master.tar.gz --no-check-ce
--2025-05-06 05:01:59--  https://codeload.github.com/Tencent/rapidjson/tar.gz/master
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'rapidjson-master.tar.gz'

rapidjson-master.tar.gz        [    <=>                                 ]   1.06M   919KB/s    in 1.2s

2025-05-06 05:02:01 (919 KB/s) -  'rapidjson-master.tar.gz'  saved [1116059]

Installing from scratch into /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/ra
Step rapidjson took 2.00 seconds.
```

```
Step libtorch took 0.00 seconds.
Step libnpy took 0.00 seconds.
Step libri took 0.00 seconds.
Step libcomm took 0.00 seconds.
========================= usage =========================
Done!
To use the installed tools and libraries and ABACUS version
compiled with it you will first need to execute at the prompt:
  source /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/setup
To build ABACUS by gnu-toolchain, just use:
    ./build_abacus_gnu.sh
To build ABACUS by intel-toolchain, just use:
    ./build_abacus_intel.sh
To build ABACUS by amd-toolchain in gcc-aocl, just use:
    ./build_abacus_gnu-aocl.sh
or you can modify the builder scripts to suit your needs.
```

编译成功后，根据输出提示，运行 build_abacus_gnu.sh

```
sh ./build_abacus_gnu.sh
```

即可完成 ABACUS 本体的编译安装，大致输出如下：

```
-- The CXX compiler identification is GNU 11.4.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/g++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- RapidJSON found. Headers:
-- Found Git: /usr/bin/git (found version "2.34.1")
-- Found git: attempting to get commit info...
fatal: not a git repository (or any of the parent directories): .git
fatal: not a git repository (or any of the parent directories): .git
CMake Warning at CMakeLists.txt:104 (message):
  Failed to get git commit info
-- Found Cereal: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/cereal-master/inclu
-- Found PkgConfig: /usr/bin/pkg-config (found version "0.29.2")
-- Found ELPA: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/elpa-2025.01.001/cpu/
-- Performing Test ELPA_VERSION_SATISFIES
-- Performing Test ELPA_VERSION_SATISFIES - Success
-- Found MPI_CXX: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openmpi-5.0.7/lib/
-- Found MPI: TRUE (found version "3.1")
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Found OpenMP_CXX: -fopenmp (found version "4.5")
-- Found OpenMP: TRUE (found version "4.5")
```

```
-- Looking for a CUDA compiler
-- Looking for a CUDA compiler - NOTFOUND
-- Found FFTW3: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/fftw-3.3.10/lib/libf
-- Looking for sgemm_
-- Looking for sgemm_ - not found
-- Looking for sgemm_
-- Looking for sgemm_ - found
-- Found BLAS: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openblas-0.3.29/lib/l
-- Looking for cheev_
-- Looking for cheev_ - found
-- Found LAPACK: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/openblas-0.3.29/lib
CMake Warning (dev) at CMakeLists.txt:439 (find_package):
  Policy CMP0144 is not set: find_package uses upper-case <PACKAGENAME>_ROOT
  variables.  Run "cmake --help-policy CMP0144" for policy details.  Use the
  cmake_policy command to set the policy and suppress this warning.

  Environment variable SCALAPACK_ROOT is set to:

    /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/scalapack-2.2.2

  For compatibility, find_package is ignoring the variable, but code in a
  .cmake module might still use it.
This warning is for project developers.  Use -Wno-dev to suppress it.

-- Found ScaLAPACK: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/scalapack-2.2.2/l
-- Checking for one of the modules 'libxc'
-- Found Libxc: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/install/libxc-7.0.0/lib/libx
-- Found Libxc: version 7.0.0
-- Configuring done (1.4s)
-- Generating done (0.2s)
-- Build files have been written to: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/build_abacus_gnu
[  0%] Building CXX object source/module_rdmft/CMakeFiles/rdmft.dir/rdmft.cpp.o
[  0%] Building CXX object source/CMakeFiles/device.dir/module_hamilt_pw/hamilt_pwdft/kernels/nonlocal_op.cpp.o
...
[100%] Building CXX object source/module_base/CMakeFiles/base.dir/module_mixing/pulay_mixing.cpp.o
[100%] Building CXX object source/module_base/CMakeFiles/base.dir/module_mixing/broyden_mixing.cpp.o
[100%] Built target io_input
[100%] Built target io_advanced
[100%] Built target cell
[100%] Built target hamilt_lcao
[100%] Built target operator_ks_lcao
[100%] Built target base
[100%] Built target esolver
[100%] Built target io_basic
[100%] Built target lr
[100%] Building CXX object CMakeFiles/abacus.dir/source/main.cpp.o
[100%] Linking CXX executable abacus
[100%] Built target abacus
```

```
-- Install configuration: ""
-- Installing: /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/bin/abacus
========================= usage ========================
Done!
To use the installed ABACUS version
You need to source /home/mikoto/tests/toolchain-dev-test/abacus-develop-LTS/toolchain/abacus_env.sh first !
```

如上输出即为完成了 ABACUS 的编译安装。若想使用如此编译安装的 ABACUS，只需要在 Toolchain 目录下加载自动生成的环境变量脚本：

```
source abacus_env.sh
```

即可加载 ABACUS 所有依赖库环境和 ABACUS 本体，此时可以通过 abacus –version 命令确认 ABACUS 已正确安装和加载：

```
(base) mikoto@amd-cpu:toolchain$ source abacus_env.sh
(base) mikoto@amd-cpu:toolchain$ abacus --version
ABACUS version v3.10.0
```

## 2.5 Toolchain 编译 ABACUS 教程 (2) – Intel

### 2.5.1 简介

此前的教程介绍如何使用 GCC Toolchain 简单直接地从头编译 ABACUS，这一方案兼容性最好，但编译所得的 ABACUS 不一定最高效，尤其对于很多配置了相应 Intel OneAPI 套件的 Intel-CPU 服务器。本教程则将侧重于如何利用 Intel Toolchain，让编译得到的 ABACUS 获得更高性能。与此同时，本教程还会介绍如何编辑 Toolchain 的关键脚本，达到切换不同编译依赖组件或是加入特定功能插件的效果。

### 2.5.2 基于 Intel Toolchain 安装 ABACUS

下面我们将以 Intel Toolchain 为例，展示安装 LTS-3.10.0 版本 ABACUS 的过程。其中，针对不同的服务器特点，我们需要针对性地调整 Intel Toolchain 的各个脚本，教程作者习惯使用 vim 进行调整，会基于 vim 教学，具体操作可以采用任何在 Linux 服务器上编辑文件的方法。

注意：使用 Toolchain 进行源码安装前，务必确保你正确加载了对应的 Intel OneAPI 环境或其他相关环境变量！

```
# load intel-oneapi environment via source
source /path/to/intel/oneapi/setvars.sh
# load intel-oneapi environment via module-env if exists
module load mkl mpi compiler
```

#### 在 Intel OneAPI 2024+ 依赖下直接安装 ABACUS

如果你的服务器 Intel OneAPI 版本在 2024.0 版本以上，一般来说，你可以直接通过下述步骤进行 ABACUS 的编译安装。（202502 版本 Toolchain 已通过拉取最新版本依赖库方式支持了 OneAPI 2025+ 版本）

1. 下载 ABACUS 仓库。可以通过 git clone 下载，这一做法允许用户通过 git 包管理器快速更新或切换版本。也可以通过 wget 拉取 ABACUS 仓库压缩包，这样得到的 ABACUS 仓库没有 git 相关的信息，文件体积更

小。

```
# via github
git clone https://github.com/deepmodeling/abacus-develop.git -b LTS
# via wget from github by codeload: can be used under CN Internet
wget https://codeload.github.com/deepmodeling/abacus-develop/tar.gz/LTS -O abacus-LTS.tar.gz
tar -zxvf abacus-LTS.tar.gz
```

2. 进入 toolchain 目录

```
cd abacus-develop/toolchain
# if you download abacus via wget from codeload
cd abacus-develop-LTS/toolchain
```

3. 运行 toolchain_intel.sh 脚本

```
sh ./toolchain_intel.sh
```

如此即可开始基于 Toolchain 的 ABACUS 依赖库编译安装。以 Intel Toolchain 默认设置为例，会做如下操作：

1. 检查你的系统 GNU 编译器版本（2025-02 版本 Toolchain 新功能），并链接 Intel 编译器，输出类似于：

```
MPI is detected and it appears to be Intel MPI
Checking system GCC version for gcc, intel and amd toolchain
Your System gcc/g++/gfortran version should be consistent
Minimum required version: 5
Your gcc version: 11.3.0
Your g++ version: 11.3.0
Your gfortran version: 11.3.0
Your GCC version seems to be enough for ABACUS installation.
Using MKL, so openblas is disabled.
Compiling with 16 processes for target native.
Step gcc took 0.00 seconds.
==================== Finding Intel compiler from system paths ====================
path to icx is /opt/intel/oneapi/compiler/2025.1/bin/icx
path to icpx is /opt/intel/oneapi/compiler/2025.1/bin/icpx
path to ifx is /opt/intel/oneapi/compiler/2025.1/bin/ifx
CC  is /opt/intel/oneapi/compiler/2025.1/bin/icx
CXX is /opt/intel/oneapi/compiler/2025.1/bin/icpx
FC  is /opt/intel/oneapi/compiler/2025.1/bin/ifx
Step intel took 0.00 seconds.
Step amd took 0.00 seconds.
```

注：编译 ABACUS 及其相关依赖是有系统 C++ 编译器最低版本要求的，GCC 版本至少不能小于 5，即使是使用 Intel 或 AMD 相关依赖库和编译器进行编译安装时也是如此。然而，有一部分老旧超算依然维持着 CentOS 的内核和 4.8.5 版本的 GCC，因而如果在 Intel Toolchain 编译过程中出现奇特报错，一大可能就是服务器 GCC 版本过低，使得 Intel 编译器不能正常编译 ABACUS 相关程序，此时请联系服务器管理员。

2. 下载 OpenBLAS（即使 Intel Toolchain 也会进行这一操作），通过 OpenBLAS 内置的 genarch 程序识别系统指令集和架构，在服务器上的输出如下：

```
# Run toolchain on Intel-CPU Machine
==================== Getting proc arch info using OpenBLAS tools ====================
wget  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29 -O OpenBLAS-0.3.29.tar.gz --no-check
--2025-05-06 16:58:40--  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'OpenBLAS-0.3.29.tar.gz'

OpenBLAS-0.3.29.tar.gz         [               <=>                  ]  23.53M  6.75MB/s    in 3.5s

2025-05-06 16:58:45 (6.75 MB/s) -  'OpenBLAS-0.3.29.tar.gz'  saved [24671913]

OpenBLAS-0.3.29.tar.gz: OK
Checksum of OpenBLAS-0.3.29.tar.gz Ok
OpenBLAS detected LIBCORE = skylakex
OpenBLAS detected ARCH    = x86_64
```

3. 自动下载 CMake 编译库.
4. 从环境变量中定位并链接 Intel MPI 并行库和 Intel MKL 数学库

```
==================== Finding Intel MPI from system paths ====================
path to mpiexec is /opt/intel/oneapi/mpi/2021.15/bin/mpiexec
path to mpiicx is /opt/intel/oneapi/mpi/2021.15/bin/mpiicx
path to mpiicpx is /opt/intel/oneapi/mpi/2021.15/bin/mpiicpx
path to mpiifx is /opt/intel/oneapi/mpi/2021.15/bin/mpiifx
Found lib directory /opt/intel/oneapi/mpi/2021.15/lib
libmpi is found in ld search path
libmpicxx is found in ld search path
I_MPI_CXX is icpx
I_MPI_CC  is icx
I_MPI_FC  is ifx
MPICXX    is /opt/intel/oneapi/mpi/2021.15/bin/mpiicpx
MPICC     is /opt/intel/oneapi/mpi/2021.15/bin/mpiicx
MPIFC     is /opt/intel/oneapi/mpi/2021.15/bin/mpiifx
Step intelmpi took 0.00 seconds.
==================== Finding MKL from system paths ====================
MKLROOT is found to be /opt/intel/oneapi/mkl/2025.1
libm is found in ld search path
libdl is found in ld search path
Step mkl took 0.00 seconds.
```

5. 自动下载并编译安装：数学库 ELPA，泛函库 LibXC，其他功能库 CEREAL, RapidJSON 以及可选功能库：LibRI, LibComm, LibTorch, Libnpy 等。完整输出如下：（本案例基于 Bohr 容器完成）

```
MPI is detected and it appears to be Intel MPI
Checking system GCC version for gcc, intel and amd toolchain
```

```
Your System gcc/g++/gfortran version should be consistent
Minimum required version: 5
Your gcc version: 11.3.0
Your g++ version: 11.3.0
Your gfortran version: 11.3.0
Your GCC version seems to be enough for ABACUS installation.
Using MKL, so openblas is disabled.
Compiling with 16 processes for target native.
Step gcc took 0.00 seconds.
==================== Finding Intel compiler from system paths ====================
path to icx is /opt/intel/oneapi/compiler/2025.1/bin/icx
path to icpx is /opt/intel/oneapi/compiler/2025.1/bin/icpx
path to ifx is /opt/intel/oneapi/compiler/2025.1/bin/ifx
CC  is /opt/intel/oneapi/compiler/2025.1/bin/icx
CXX is /opt/intel/oneapi/compiler/2025.1/bin/icpx
FC  is /opt/intel/oneapi/compiler/2025.1/bin/ifx
Step intel took 0.00 seconds.
Step amd took 0.00 seconds.
==================== Getting proc arch info using OpenBLAS tools ====================
wget  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29 -O OpenBLAS-0.3.29.tar.gz --no-check
--2025-05-09 17:15:04--  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29
Resolving ga.dp.tech (ga.dp.tech)... 10.255.254.18, 10.255.254.37, 10.255.254.7
Connecting to ga.dp.tech (ga.dp.tech)|10.255.254.18|:8118... connected.
Proxy request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to: 'OpenBLAS-0.3.29.tar.gz'

OpenBLAS-0.3.29.tar.gz         [                           <=>        ]  23.53M  4.64MB/s    in 5.5s

2025-05-09 17:15:11 (4.30 MB/s) - 'OpenBLAS-0.3.29.tar.gz' saved [24671913]

OpenBLAS-0.3.29.tar.gz: OK
Checksum of OpenBLAS-0.3.29.tar.gz Ok
./f_check: 100: [: Illegal number:
OpenBLAS detected LIBCORE = skylakex
OpenBLAS detected ARCH    = x86_64
==================== Installing CMake ====================
wget  https://cmake.org/files/v3.31/cmake-3.31.7-linux-x86_64.sh -O cmake-3.31.7-linux-x86_64.sh --no-chec
--2025-05-09 17:15:16--  https://cmake.org/files/v3.31/cmake-3.31.7-linux-x86_64.sh
Resolving ga.dp.tech (ga.dp.tech)... 10.255.254.37, 10.255.254.7, 10.255.254.18
Connecting to ga.dp.tech (ga.dp.tech)|10.255.254.37|:8118... connected.
Proxy request sent, awaiting response... 200 OK
Length: 55005854 (52M) [text/x-sh]
Saving to: 'cmake-3.31.7-linux-x86_64.sh'

cmake-3.31.7-linux-x86_64. 100%[=====================================>]  52.46M  7.10MB/s    in 8.7s

2025-05-09 17:15:25 (6.04 MB/s) - 'cmake-3.31.7-linux-x86_64.sh' saved [55005854/55005854]
```

```
51
52 cmake-3.31.7-linux-x86_64.sh: OK
53 Checksum of cmake-3.31.7-linux-x86_64.sh Ok
54 Installing from scratch into /opt/abacus-develop-LTS/toolchain/install/cmake-3.31.7
55 Step cmake took 11.00 seconds.
56 ==================== Finding Intel MPI from system paths ====================
57 path to mpiexec is /opt/intel/oneapi/mpi/2021.15/bin/mpiexec
58 path to mpiicx is /opt/intel/oneapi/mpi/2021.15/bin/mpiicx
59 path to mpiicpx is /opt/intel/oneapi/mpi/2021.15/bin/mpiicpx
60 path to mpiifx is /opt/intel/oneapi/mpi/2021.15/bin/mpiifx
61 Found lib directory /opt/intel/oneapi/mpi/2021.15/lib
62 libmpi is found in ld search path
63 libmpicxx is found in ld search path
64 I_MPI_CXX is icpx
65 I_MPI_CC  is icx
66 I_MPI_FC  is ifx
67 MPICXX    is /opt/intel/oneapi/mpi/2021.15/bin/mpiicpx
68 MPICC     is /opt/intel/oneapi/mpi/2021.15/bin/mpiicx
69 MPIFC     is /opt/intel/oneapi/mpi/2021.15/bin/mpiifx
70 Step intelmpi took 0.00 seconds.
71 ==================== Finding MKL from system paths ====================
72 MKLROOT is found to be /opt/intel/oneapi/mkl/2025.1
73 libm is found in ld search path
74 libdl is found in ld search path
75 Step mkl took 0.00 seconds.
76 ==================== Installing LIBXC ====================
77 wget  https://gitlab.com/libxc/libxc/-/archive/7.0.0/libxc-7.0.0.tar.bz2 -O libxc-7.0.0.tar.bz2 --no-check
78 --2025-05-09 17:15:27--  https://gitlab.com/libxc/libxc/-/archive/7.0.0/libxc-7.0.0.tar.bz2
79 Resolving ga.dp.tech (ga.dp.tech)... 10.255.254.18, 10.255.254.7, 10.255.254.37
80 Connecting to ga.dp.tech (ga.dp.tech)|10.255.254.18|:8118... connected.
81 Proxy request sent, awaiting response... 200 OK
82 Length: unspecified [application/octet-stream]
83 Saving to: 'libxc-7.0.0.tar.bz2'
84
85 libxc-7.0.0.tar.bz2           [                              <=>  ]  49.98M  7.63MB/s    in 7.5s
86
87 2025-05-09 17:15:36 (6.68 MB/s) - 'libxc-7.0.0.tar.bz2' saved [52408700]
88
89 libxc-7.0.0.tar.bz2: OK
90 Checksum of libxc-7.0.0.tar.bz2 Ok
91 Installing from scratch into /opt/abacus-develop-LTS/toolchain/install/libxc-7.0.0
92 Step libxc took 42.00 seconds.
93 Step fftw took 0.00 seconds.
94 Step scalapack took 0.00 seconds.
95 ==================== Installing ELPA ====================
96 wget  https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2025.01.001/elpa-2025.01.001.tar.gz -O e
97 --2025-05-09 17:16:09--  https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2025.01.001/elpa-2025
98 Resolving ga.dp.tech (ga.dp.tech)... 10.255.254.18, 10.255.254.7, 10.255.254.37
```

```
Connecting to ga.dp.tech (ga.dp.tech)|10.255.254.18|:8118... connected.
Proxy request sent, awaiting response... 200 OK
Length: 2169795 (2.1M) [application/gzip]
Saving to: 'elpa-2025.01.001.tar.gz'

elpa-2025.01.001.tar.gz    100%[=====================================>]   2.07M  7.64KB/s    in 4m 4s

2025-05-09 17:20:15 (8.69 KB/s) - 'elpa-2025.01.001.tar.gz' saved [2169795/2169795]

elpa-2025.01.001.tar.gz: OK
Checksum of elpa-2025.01.001.tar.gz Ok
Installing from scratch into /opt/abacus-develop-LTS/toolchain/install/elpa-2025.01.001/cpu
Step elpa took 535.00 seconds.
==================== Installing CEREAL ====================
===> Notice: This version of CEREAL is downloaded in GitHub master repository  <===
wget  https://codeload.github.com/USCiLab/cereal/tar.gz/master -O cereal-master.tar.gz --no-check-certific
--2025-05-09 17:25:04--  https://codeload.github.com/USCiLab/cereal/tar.gz/master
Resolving ga.dp.tech (ga.dp.tech)... 10.255.254.18, 10.255.254.7, 10.255.254.37
Connecting to ga.dp.tech (ga.dp.tech)|10.255.254.18|:8118... connected.
Proxy request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to: 'cereal-master.tar.gz'

cereal-master.tar.gz           [ <=>                                ] 377.35K  --.-KB/s    in 0.1s

2025-05-09 17:25:05 (2.73 MB/s) - 'cereal-master.tar.gz' saved [386409]

Installing from scratch into /opt/abacus-develop-LTS/toolchain/install/cereal-master
Step cereal took 1.00 seconds.
==================== Installing RAPIDJSON ====================
===> Notice: This version of rapidjson is downloaded in GitHub master repository <===
wget  https://codeload.github.com/Tencent/rapidjson/tar.gz/master -O rapidjson-master.tar.gz --no-check-ce
--2025-05-09 17:25:05--  https://codeload.github.com/Tencent/rapidjson/tar.gz/master
Resolving ga.dp.tech (ga.dp.tech)... 10.255.254.18, 10.255.254.7, 10.255.254.37
Connecting to ga.dp.tech (ga.dp.tech)|10.255.254.18|:8118... connected.
Proxy request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to: 'rapidjson-master.tar.gz'

rapidjson-master.tar.gz        [   <=>                              ]   1.06M  1.94MB/s    in 0.6s

2025-05-09 17:25:06 (1.94 MB/s) - 'rapidjson-master.tar.gz' saved [1116059]

Installing from scratch into /opt/abacus-develop-LTS/toolchain/install/rapidjson-master
Step rapidjson took 1.00 seconds.
Step libtorch took 0.00 seconds.
Step libnpy took 0.00 seconds.
Step libri took 0.00 seconds.
```

```
Step libcomm took 0.00 seconds.
========================= usage =========================
Done!
To use the installed tools and libraries and ABACUS version
compiled with it you will first need to execute at the prompt:
  source /opt/abacus-develop-LTS/toolchain/install/setup
To build ABACUS by gnu-toolchain, just use:
    ./build_abacus_gnu.sh
To build ABACUS by intel-toolchain, just use:
    ./build_abacus_intel.sh
To build ABACUS by amd-toolchain in gcc-aocl, just use:
    ./build_abacus_gcc-aocl.sh
To build ABACUS by amd-toolchain in aocc-aocl, just use:
    ./build_abacus_aocc-aocl.sh
or you can modify the builder scripts to suit your needs.
```

如此编译依赖库成功后，根据输出提示，运行 build_abacus_intel.sh

```
sh ./build_abacus_intel.sh
```

即可完成 ABACUS 本体的编译安装，大致输出如下：

```
-- The CXX compiler identification is IntelLLVM 2025.1.1
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /opt/intel/oneapi/compiler/2025.1/bin/icpx - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- RapidJSON found. Headers:
-- Found Git: /usr/bin/git (found version "2.34.1")
-- Found git: attempting to get commit info...
fatal: not a git repository (or any of the parent directories): .git
fatal: not a git repository (or any of the parent directories): .git
CMake Warning at CMakeLists.txt:104 (message):
  Failed to get git commit info


-- Found Cereal: /opt/abacus-develop-LTS/toolchain/install/cereal-master/include/cereal
-- Could NOT find PkgConfig (missing: PKG_CONFIG_EXECUTABLE)
-- ELPA : We need pkg-config to get all information about the elpa library
-- Found ELPA: /opt/abacus-develop-LTS/toolchain/install/elpa-2025.01.001/cpu/lib/libelpa_openmp.so
-- Performing Test ELPA_VERSION_SATISFIES
-- Performing Test ELPA_VERSION_SATISFIES - Success
-- Found MPI_CXX: /opt/intel/oneapi/mpi/2021.15/lib/libmpicxx.so (found version "3.1")
-- Found MPI: TRUE (found version "3.1")
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
```

```
-- Found OpenMP_CXX: -fiopenmp (found version "5.1")
-- Found OpenMP: TRUE (found version "5.1")
-- Looking for a CUDA compiler
-- Looking for a CUDA compiler - NOTFOUND
-- MKL_VERSION: 2025.1.0
-- MKL_ROOT: /opt/intel/oneapi/mkl/2025.1
-- MKL_ARCH: intel64
-- MKL_SYCL_LINK: None, set to ` dynamic` by default
-- MKL_LINK: None, set to ` dynamic` by default
-- MKL_SYCL_INTERFACE_FULL: intel_lp64
-- MKL_INTERFACE_FULL: intel_lp64
-- MKL_SYCL_THREADING: None, set to ` tbb_thread` by default
-- MKL_THREADING: None, set to ` intel_thread` by default
-- MKL_MPI: None, set to ` intelmpi` by default
-- Experimental oneMKL Data Fitting SYCL API does not support LP64 on CPU
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_scalapack_lp64.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_cdft_core.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_intel_lp64.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_intel_thread.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_core.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_blacs_intelmpi_lp64.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_blas.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_lapack.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_dft.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_sparse.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_data_fitting.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_rng.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_stats.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_sycl_vm.so
-- Found /opt/intel/oneapi/mkl/2025.1/lib/libmkl_tbb_thread.so
-- Found /opt/intel/oneapi/compiler/2025.1/lib/libiomp5.so
-- Could NOT find PkgConfig (missing: PKG_CONFIG_EXECUTABLE)
-- Found Libxc: version 7.0.0
-- Configuring done (1.3s)
-- Generating done (0.2s)
-- Build files have been written to: /opt/abacus-develop-LTS/build_abacus_intel
[  0%] Building CXX object source/CMakeFiles/driver.dir/driver.cpp.o
...
[100%] Built target io_basic
[100%] Building CXX object CMakeFiles/abacus.dir/source/main.cpp.o
[100%] Linking CXX executable abacus
[100%] Built target abacus
-- Install configuration: ""
-- Installing: /opt/abacus-develop-LTS/bin/abacus
========================= usage =========================
Done!
To use the installed ABACUS version
You need to source /opt/abacus-develop-LTS/toolchain/abacus_env.sh first !
```

如上输出即为完成了 ABACUS 的编译安装。注意，从系统内读入的环境变量并不会被写入到 abacus_env.sh 对应的 install/setup 文件中，因此在实际使用这一 abacus 时，除了 source abacus_env.sh，你还需要加载对应的 Intel-OneAPI 环境。

```
# after load intel-OneAPI env via source or module-load
source abacus_env.sh
```

如此即可加载 ABACUS 所有依赖库环境和 ABACUS 本体，此时可以通过 abacus –version 命令确认 ABACUS 已正确安装和加载：

```
root@bohrium:toolchain# source abacus_env.sh
root@bohrium:toolchain# abacus --version
ABACUS version v3.10.0
```

### 基于较旧的 Intel-OneAPI 安装 ABACUS

很多情况下，我们被迫使用旧版本的 OneAPI，如

- 使用 Intel 库编译 GPU 支持的 ABACUS（此时只能用 icpc 编译器编译 ABACUS）。
- 在 AMD-CPU 服务器上使用 Intel-OneAPI 编译 ABACUS（此时只能用 icpc 编译器编译 ELPA）。需要注意的是，在 AMD-CPU 服务器上使用 Intel-OneAPI 编译的 ABACUS，其计算性能将较其他方案偏低。
- 服务器上仅有旧版的 OneAPI（但最好在 2023 版及以上）或 parallel_xe_studio。

此时，我们可以通过编辑关键脚本来完成基于 Toolchain 的 ABACUS 编译，用户层面操作的关键脚本包括 toolchain_*.sh 和 build_abacus_*.sh。对于 Intel Toolchain 而言，相应脚本则是 toolchain_intel.sh 和 build_abacus_intel.sh。

通过 vim 打开 toolchain_intel.sh

```
#!/bin/bash
#SBATCH -J install
#SBATCH -N 1
#SBATCH -n 16
#SBATCH -o compile.log
#SBATCH -e compile.err

# JamesMisaka in 2025-05-05
# install abacus dependency by intel-toolchain
# use mkl and intelmpi
# but mpich and openmpi can also be tried
# libtorch and libnpy are for deepks support, which can be =no
# gpu-lcao supporting modify: CUDA_PATH and --enable-cuda
# export CUDA_PATH=/usr/local/cuda

# module load mkl mpi compiler

./install_abacus_toolchain.sh \
--with-intel=system \
--math-mode=mkl \
--with-gcc=no \
--with-intelmpi=system \
```

```
--with-cmake=install \
--with-scalapack=no \
--with-libxc=install \
--with-fftw=no \
--with-elpa=install \
--with-cereal=install \
--with-rapidjson=install \
--with-libtorch=no \
--with-libnpy=no \
--with-libri=no \
--with-libcomm=no \
--with-intel-classic=no \
| tee compile.log
# for using AMD-CPU or GPU-version: set --with-intel-classic=yes
# to enable gpu-lcao, add the following lines:
# --enable-cuda \
# --gpu-ver=75 \
# one should check your gpu compute capability number
```

输入:34 切到第 34 行，按 i 键或 a 键进入编辑模式（好奇的小伙伴可以观察二者的区别），将–with-intel-classic=no 修改为–with-intel-classic=yes, 呈现效果为：

```
./install_abacus_toolchain.sh \
--with-intel=system \
--math-mode=mkl \
--with-gcc=no \
--with-intelmpi=system \
--with-cmake=install \
--with-scalapack=no \
--with-libxc=install \
--with-fftw=no \
--with-elpa=install \
--with-cereal=install \
--with-rapidjson=install \
--with-libtorch=no \
--with-libnpy=no \
--with-libri=no \
--with-libcomm=no \
--with-intel-classic=yes \
| tee compile.log
```

可以发现 toolchain*.sh 脚本实际上是通过调用主脚本 install_abacus_toolchain.sh 来完成编译的，只是针对不同的 toolchain 进行了不同的编译选项设置，方便用户开箱即用和简单编辑。需要注意的，如此使用 bash 命令时，各个选项其实是连接在一起的，也即分行符号” 后˙ 面不能有任何空格或其他字符，并且各个分行之间不能有注释。

按一下退出键 ESC 退出编辑模式，输入:wq 保存并退出。如此这般，Toolchain 将在运行时识别传统 Intel 编译器（icc, icpc, ifort）及其 MPI 库进行依赖库编译，运行 toolchain_intel.sh 时，会有与此前不同的输出：

```
==================== Finding Intel compiler from system paths ====================
path to icc is /mnt/sg001/opt/intel/oneapi/compiler/2022.2.1/linux/bin/intel64/icc
path to icpc is /mnt/sg001/opt/intel/oneapi/compiler/2022.2.1/linux/bin/intel64/icpc
path to ifort is /mnt/sg001/opt/intel/oneapi/compiler/2022.2.1/linux/bin/intel64/ifort
CC  is /mnt/sg001/opt/intel/oneapi/compiler/2022.2.1/linux/bin/intel64/icc
CXX is /mnt/sg001/opt/intel/oneapi/compiler/2022.2.1/linux/bin/intel64/icpc
FC  is /mnt/sg001/opt/intel/oneapi/compiler/2022.2.1/linux/bin/intel64/ifort
Step intel took 0.00 seconds.
Step amd took 0.00 seconds.
==================== Getting proc arch info using OpenBLAS tools ====================
OpenBLAS detected LIBCORE = skylakex
OpenBLAS detected ARCH    = x86_64
==================== Finding CMake from system paths ====================
path to cmake is /mnt/sg001/home/fz_pku_jh/software/cmake/3.31.7/bin/cmake
Step cmake took 0.00 seconds.
==================== Finding Intel MPI from system paths ====================
path to mpiexec is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiexec
path to mpiicc is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiicc
path to mpiicpc is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiicpc
path to mpiifort is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiifort
Found lib directory /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/lib/release
libmpi is found in ld search path
libmpicxx is found in ld search path
I_MPI_CXX is icpc
I_MPI_CC  is icc
I_MPI_FC  is ifort
MPICXX    is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiicpc
MPICC     is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiicc
MPIFC     is /mnt/sg001/opt/intel/oneapi/mpi/2021.7.1/bin/mpiifort
Step intelmpi took 1.00 seconds.
==================== Finding MKL from system paths ====================
MKLROOT is found to be /mnt/sg001/opt/intel/oneapi/mkl/2022.2.1
libm is found in ld search path
libdl is found in ld search path
Step mkl took 0.00 seconds.
```

读者可思考：为什么上面的编译输出中，cmake 是来自于 system 的？我还修改了哪个编译选项？这个问题的答案将在下一部分揭露。

如此编辑并运行完 toolchain_intel.sh 后，进一步通过 vim 打开 build_abacus_intel.sh

```
#!/bin/bash
#SBATCH -J build
#SBATCH -N 1
#SBATCH -n 16
#SBATCH -o install.log
#SBATCH -e install.err
# JamesMisaka in 2025.03.09

```

```
# Build ABACUS by intel-toolchain

# module load mkl compiler mpi
# source path/to/setvars.sh

ABACUS_DIR=..
TOOL=$(pwd)
INSTALL_DIR=$TOOL/install
source $INSTALL_DIR/setup
cd $ABACUS_DIR
ABACUS_DIR=$(pwd)

BUILD_DIR=build_abacus_intel
rm -rf $BUILD_DIR

PREFIX=$ABACUS_DIR
ELPA=$INSTALL_DIR/elpa-2025.01.001/cpu
# ELPA=$INSTALL_DIR/elpa-2025.01.001/nvidia # for gpu-lcao
CEREAL=$INSTALL_DIR/cereal-master/include/cereal
LIBXC=$INSTALL_DIR/libxc-7.0.0
RAPIDJSON=$INSTALL_DIR/rapidjson-master/
# LIBTORCH=$INSTALL_DIR/libtorch-2.1.2/share/cmake/Torch
# LIBNPY=$INSTALL_DIR/libnpy-1.0.1/include
# LIBRI=$INSTALL_DIR/LibRI-0.2.1.0
# LIBCOMM=$INSTALL_DIR/LibComm-master
# DEEPMD=$HOME/apps/anaconda3/envs/deepmd # v3.0 might have problem

# Notice: if you are compiling with AMD-CPU or GPU-version ABACUS, then `icpc` and `mpiicpc` compilers are recon
cmake -B $BUILD_DIR -DCMAKE_INSTALL_PREFIX=$PREFIX \
        -DCMAKE_CXX_COMPILER=icpx \
        -DMPI_CXX_COMPILER=mpiicpx \
        -DMKLROOT=$MKLROOT \
        -DELPA_DIR=$ELPA \
        -DCEREAL_INCLUDE_DIR=$CEREAL \
        -DLibxc_DIR=$LIBXC \
        -DENABLE_LCAO=ON \
        -DENABLE_LIBXC=ON \
        -DUSE_OPENMP=ON \
        -DUSE_ELPA=ON \
        -DENABLE_RAPIDJSON=ON \
        -DRapidJSON_DIR=$RAPIDJSON \
#         -DENABLE_DEEPKS=1 \
#         -DTorch_DIR=$LIBTORCH \
#         -Dlibnpy_INCLUDE_DIR=$LIBNPY \
#         -DENABLE_LIBRI=ON \
#         -DLIBRI_DIR=$LIBRI \
#         -DLIBCOMM_DIR=$LIBCOMM \
#             -DDeePMD_DIR=$DEEPMD \
```

```
#         -DUSE_CUDA=ON \
#         -DENABLE_CUSOLVERMP=ON \
#         -D CAL_CUSOLVERMP_PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/2x.xx/math_libs/1x.x/targets/x86_64-linux/lib

cmake --build $BUILD_DIR -j `nproc`
cmake --install $BUILD_DIR 2>/dev/null

# if one want's to include deepmd, your system gcc version should be >= 11.3.0 for glibc requirements

# generate abacus_env.sh
cat << EOF > "${TOOL}/abacus_env.sh"
#!/bin/bash
source $INSTALL_DIR/setup
export PATH="${PREFIX}/bin":\${PATH}
EOF

# generate information
cat << EOF
========================== usage =========================
Done!
To use the installed ABACUS version
You need to source ${TOOL}/abacus_env.sh first !
"""
EOF
```

修改 ABACUS 编译所用的编译器，即按如下方式修改第 38 和 39 行:

```
cmake -B $BUILD_DIR -DCMAKE_INSTALL_PREFIX=$PREFIX \
        -DCMAKE_CXX_COMPILER=icpc \
        -DMPI_CXX_COMPILER=mpiicpc \
```

保存退出后，运行 build_abacus_intel.sh 即可。编译期间可能会有大量 ICC 相关的 Warning，这只是提醒你 icpc 等旧版 Intel 编译器不会在新版 Intel-OneAPI 中保持支持，不必理会。通过这一方法，即可基于旧版 Intel-OneAPI 编译 ABACUS。

有些 Intel-OneAPI 处于过渡期，如 Intel-OneAPI 2023.2 可能同时具备 icpx 和 mpiicpc，但不具备 mpiicpx。针对这一情况，可以使用另一个编译选项–with-intel-mpi-clas=yes。编辑 toolchain_intel.sh，使 install_abacus_toolchain.sh 部分呈现如下

```
./install_abacus_toolchain.sh \
--with-intel=system \
--math-mode=mkl \
--with-gcc=no \
--with-intelmpi=system \
--with-cmake=install \
--with-scalapack=no \
--with-libxc=install \
--with-fftw=no \
--with-elpa=install \
```

```
--with-cereal=install \
--with-rapidjson=install \
--with-libtorch=no \
--with-libnpy=no \
--with-libri=no \
--with-libcomm=no \
--with-intel-classic=no \
--with-intel-mpi-clas=yes \
| tee compile.log
```

即可使用 icpx 和 mpiicpc 编译 ABACUS 依赖软件。编译 ABACUS 本体时，按照与上述类似的方法在 build_abacus_intel.sh 中修改编译器即可。

```
cmake -B $BUILD_DIR -DCMAKE_INSTALL_PREFIX=$PREFIX \
        -DCMAKE_CXX_COMPILER=icpx \
        -DMPI_CXX_COMPILER=mpiicpc \
```

很好！你已经学会了通过编辑 toolchain_intel.sh 和 build_abacus_intel.sh 自定义 toolchain 以在不同机器条件下编译 Intel 依赖的 ABACUS 了，这种自定义同样能用于相关 ABACUS 功能插件的引入，如杂化泛函库 LibRI，或是 DeePKS 支持所需 Torch 依赖。

### 2.5.3 基于 Intel Toolchain 引入 ABACUS 功能插件

在之前的内容中，我们查看并编辑了 toolchain_*.sh 和 build_abacus_*.sh 这两个 toolchain 核心脚本。通过编辑这两个脚本，我们可以基于 Toolchain 方便快捷地实现不同需求的 ABACUS 编译。其中，toolchain_*.sh 主要通过调用 install_abacus_toolchain.sh 主脚本完成，涉及到几个编译选项，这些编译选项及其作用分为几类：

- –with-PKG=[install, system, no, [abspath]]，安装特定依赖库时下载并通过下载的软件包安装/从系统环境变量中读取对应依赖库/不使用对应依赖库/[高级] 从绝对路径中识别对应依赖库。
- –math-mode=[mkl, openblas, aocl]，指定 Toolchain 会用到的数学库类型，默认为 openblas，但如果识别到环境变量 $MKLROOT，则会切换到 MKL（所以使用 GCC Toolchain 时不能加载 OneAPI 环境）
- - –with-option=[yes,no] 等其他的编译选项，用于对特定依赖库指定不同版本，如 Intel Toolchain 的–with-intel-classic=[yes,no]，或 GNU Toolchain 的–with-openmpi4=[yes,no]

通过 README，或运行 install_abacus_toolchain.sh –help，能获取更多相关信息。上面内容中留了一个思考题，其实就只是我在具体编译时使用了–with-cmake=system 选项。了解了这些之后，我们就可以很方便地基于 Toolchain 编译一些 ABACUS 功能插件，下面以 Intel Toolchain 为例，简要介绍如何编译支持杂化泛函和 DeePKS 的 ABACUS。

#### 编译支持杂化泛函计算的 ABACUS

在 toolchain_intel.sh 中，修改如下两个选项为 install：

```
--with-libri=install \
--with-libcomm=install \
```

在编辑完其他选项后，运行 toolchain_intel.sh，此时 Toolchain 会自动下载安装 LibRI 库和 LibComm 库，并自动加入到环境变量管理文件 install/setup 中。运行完毕后，编辑 build_abacus_intel.sh，打开相关代码附近的注释（删除注释符号），并将相关代码连接起来：

```
PREFIX=$ABACUS_DIR
ELPA=$INSTALL_DIR/elpa-2025.01.001/cpu
# ELPA=$INSTALL_DIR/elpa-2025.01.001/nvidia # for gpu-lcao
CEREAL=$INSTALL_DIR/cereal-master/include/cereal
LIBXC=$INSTALL_DIR/libxc-7.0.0
RAPIDJSON=$INSTALL_DIR/rapidjson-master/
# LIBTORCH=$INSTALL_DIR/libtorch-2.1.2/share/cmake/Torch
# LIBNPY=$INSTALL_DIR/libnpy-1.0.1/include
LIBRI=$INSTALL_DIR/LibRI-0.2.1.0
LIBCOMM=$INSTALL_DIR/LibComm-master
# DEEPMD=$HOME/apps/anaconda3/envs/deepmd # v3.0 might have problem

# Notice: if you are compiling with AMD-CPU or GPU-version ABACUS, then `icpc` and `mpiicpc` compilers are recom
cmake -B $BUILD_DIR -DCMAKE_INSTALL_PREFIX=$PREFIX \
        -DCMAKE_CXX_COMPILER=icpx \
        -DMPI_CXX_COMPILER=mpiicpx \
        -DMKLROOT=$MKLROOT \
        -DELPA_DIR=$ELPA \
        -DCEREAL_INCLUDE_DIR=$CEREAL \
        -DLibxc_DIR=$LIBXC \
        -DENABLE_LCAO=ON \
        -DENABLE_LIBXC=ON \
        -DUSE_OPENMP=ON \
        -DUSE_ELPA=ON \
        -DENABLE_RAPIDJSON=ON \
        -DRapidJSON_DIR=$RAPIDJSON \
        -DENABLE_LIBRI=ON \
        -DLIBRI_DIR=$LIBRI \
        -DLIBCOMM_DIR=$LIBCOMM \
#         -DENABLE_DEEPKS=1 \
#         -DTorch_DIR=$LIBTORCH \
#         -Dlibnpy_INCLUDE_DIR=$LIBNPY \
```

再运行 build_abacus_intel.sh 脚本即可。再次强调：分行符号” 后 面不能有任何空格或其他字符，并且各个分行之间不能有注释。作者的经验是，Intel-OneAPI 编译的 ABACUS 往往具备更强的 OpenMP 并行性能，可以开很多 OpenMP 线程，通过线程并行加速计算，尤其是跑杂化泛函等需要运行 EXX 部分的计算时，效率通常来说会比较高。

**编译支持 DeePKS 的 ABACUS**

在 toolchain_intel.sh 中，修改如下两个选项为 install：

```
--with-libtorch=install \
--with-libnpy=install \
```

运行 toolchain_intel.sh 完毕后，在 build_abacus_intel.sh 中打开相关注释并修改好脚本，比如：

```
PREFIX=$ABACUS_DIR
ELPA=$INSTALL_DIR/elpa-2025.01.001/cpu
# ELPA=$INSTALL_DIR/elpa-2025.01.001/nvidia # for gpu-lcao
CEREAL=$INSTALL_DIR/cereal-master/include/cereal
LIBXC=$INSTALL_DIR/libxc-7.0.0
RAPIDJSON=$INSTALL_DIR/rapidjson-master/
LIBTORCH=$INSTALL_DIR/libtorch-2.1.2/share/cmake/Torch
LIBNPY=$INSTALL_DIR/libnpy-1.0.1/include
LIBRI=$INSTALL_DIR/LibRI-0.2.1.0
LIBCOMM=$INSTALL_DIR/LibComm-master
# DEEPMD=$HOME/apps/anaconda3/envs/deepmd # v3.0 might have problem

# Notice: if you are compiling with AMD-CPU or GPU-version ABACUS, then `icpc` and `mpiicpc` compilers are recom
cmake -B $BUILD_DIR -DCMAKE_INSTALL_PREFIX=$PREFIX \
        -DCMAKE_CXX_COMPILER=icpx \
        -DMPI_CXX_COMPILER=mpiicpx \
        -DMKLROOT=$MKLROOT \
        -DELPA_DIR=$ELPA \
        -DCEREAL_INCLUDE_DIR=$CEREAL \
        -DLibxc_DIR=$LIBXC \
        -DENABLE_LCAO=ON \
        -DENABLE_LIBXC=ON \
        -DUSE_OPENMP=ON \
        -DUSE_ELPA=ON \
        -DENABLE_RAPIDJSON=ON \
        -DRapidJSON_DIR=$RAPIDJSON \
        -DENABLE_LIBRI=ON \
        -DLIBRI_DIR=$LIBRI \
        -DLIBCOMM_DIR=$LIBCOMM \
        -DENABLE_DEEPKS=1 \
        -DTorch_DIR=$LIBTORCH \
        -Dlibnpy_INCLUDE_DIR=$LIBNPY \
```

按类似于如上修改的 build_abacus_intel.sh 运行，即可同时编译含 LibRI 和 DeePKS 支持的 ABACUS。

## 2.6 Toolchain 编译 ABACUS 教程 (3) – AMD

### 2.6.1 简介

此前两部 Toolchain 相关教程分别介绍如何了使用 GCC Toolchain 直接地从头编译 ABACUS，以及如何使用 Intel Toolchain 利用 Intel OneAPI 在 Intel-CPU 机器上编译具有高效率的 ABACUS，并同时利用 Toolchain 的可选项编辑方式引入 ABACUS 功能插件。这一教程我们将讲解如何在 AMD-CPU 机器上，通过 Toolchain 编译能高效运行的 ABACUS。

### 2.6.2 已有方案回顾

**GCC Toolchain**

这一方案最为通用，兼容性最好，自然也可以在 AMD-CPU 机器上完成 ABACUS 的编译，编译过程中并没有需要特别调整的选项，按照此前的教程操作即可。

**Intel Toolchain**

AMD-CPU 机器上其实也是可以使用此前介绍的 Intel Toolchain 基于 Intel-OneAPI 编译 ABACUS 的，不过这一做法存在以下问题：

1. 编译器只能选用 Intel classic compiler 系列，即 icc/icpc/ifort，不能采用 Intel-OneAPI 2024.0+ 之后更新的新版 Intel 编译器，即 icx/icpx/ifx，否则 ELPA 编译是过不去的。
2. 在 AMD-CPU 机器上，这一编译方案的计算效率一般来说较 GNU Toolchain 偏低（Intel 机器则相反），这是源于 Intel-OneAPI（主要是 Math Kernel Library）对 Intel-CPU 的特化处理。网络上有一些解除这种特化处理带来的限制，让 MKL 在 AMD-CPU 上发挥应有性能的办法，但这些方法往往都需要管理员权限和一些额外的技巧，也不推荐普通用户花过多精力去琢磨。

### 2.6.3 AMD Toolchain

**AMD AOCL 和 AOCC 简介**

AMD Optimizing CPU Libraries (AOCL) 和 AMD Optimizing C/C++ Compiler (AOCC) 是 AMD 开发的面向高性能计算 (HPC) 和科学应用的优化工具库，分别对标 Intel 的 Intel MKL 和 Intel Compiler

- AOCL: 是一组高度优化的数学库（如 BLAS、LAPACK、SCALAPACK, FFTW 等），专为 AMD EPYC 等处理器深度调优，显著提升科学计算、数据分析等任务的运行速度和效率。用户无需修改代码即可通过链接库获得性能提升。AOCL 的各个数学库与开源版本的数学库兼容性基本一致。点此进入 AOCL 官网。
- AOCC: 是基于 LLVM/Clang 的 C/C++ 编译器套件，针对 AMD Zen 架构处理器（如 EPYC）进行了深度优化。它利用高级优化技术（如多目标代码生成、循环优化），帮助开发者编译出在 AMD 平台上性能出色的应用。点此进入 AOCC 官网

AOCL 和 AOCC 的安装可以参见官网的下载部分，一般来说需要咨询服务器管理员获取其在服务器上的安装位置，并通过 module 等方式加载对应的环境变量。本教程中，AOCC 依赖的 AOCL 安装位置为/data/softwares/AMD/5.0.0-aocl-aocc/aocl，其中内容为：

```
(base) mikoto@amd-cpu:/data/softwares/AMD/5.0.0-aocl-aocc/aocl\$ ls
amd-blis         amd-libs.cfg                 include_ILP64
amd-compression  amd-rng                      include_LP64
amd-crypto       amd-scalapack                lib
amd-da           amd-securerng                lib_ILP64
amd-fftw         amd-sparse                   lib_LP64
amd-libflame     amd-utils                    README
amd-libm         aocl-linux-aocc-5.0.0_module  set_aocl_interface_symlink.sh
amd-libmem       include
```

AOCC 安装位置为/data/softwares/AMD/5.0.0-aocl-aocc/aocc，其中内容为：

```
(base) mikoto@amd-cpu:/data/softwares/AMD/5.0.0-aocl-aocc/aocc\$ ls
AMDALLOC-LICENSE.TXT         FLANG-LICENSE.TXT  LICENSERV.TXT
aocc-compiler-5.0.0_module   include            LICENSE.TXT
```

```
AOCC-LICENSE.TXT              install.sh          ompd
AOCC-prerequisites-check.sh   lib                 README
bin                           lib32               setenv_AOCC.sh
FLANG-CHANGELOG.TXT           libexec             share
```

通过 AOCL 和 AOCC 文件夹里面的 _module 后缀文件，服务器管理者可以快捷的配置二者的 Modulefile，并便利用户使用。其中 AOCL 的环境变量被命名为 AOCLhome，类似于 Intel MKL 的 MKLROOT。

**基于 AOCC-AOCL 的编译方案**

1. 正确加载 AOCC 和 AOCL 环境，同时确保系统内没有其他多余环境变量（尤其不能有 Intel OneAPI 的环境变量）。作者本人采用的方式是：

```
(base) mikoto@amd-cpu:~\$ module load aocl-5.0.0-aocc
Loading aocl-5.0.0-aocc
  Loading requirement: aocc-5.0.0
```

作者注：有很多人使用服务器喜欢把各种环境变量相关的 source 和 export 命令写在 $HOME/.bashrc 文件内，甚至有某些服务器运维喜欢把一些环境变量写在更公用的/etc/profile 文件内，这都是非常不好的习惯！久而久之，很容易忘记自己到底都加载了哪些环境变量。合理的环境变量管理方式应该是每个需要的环境变量单独做一个.sh 脚本，需要哪个 source 哪个。当然更好的做法是直接用 Modules-env 方式管理。后续“在超算环境编译 ABACUS 的建议”教程中有介绍何为 Modules。

2. 下载 ABACUS 仓库。
3. 进入 Toolchain 目录
4. 运行 toolchain_aocc-aocl.sh 脚本

```
sh ./toolchain_aocc-aocl.sh
```

如此即可开始基于 Toolchain 的 ABACUS 依赖库编译安装。以 Intel Toolchain 默认设置为例，会做如下操作：

1. 检查你的系统 GNU 编译器版本，其版本不能低于 5（2025-02 版本 Toolchain 新功能），并链接 AMD-AOCC 编译器，输出类似于：

```
WARNING: (./install_abacus_toolchain.sh, line 315) No MPI installation detected (ignore this message in Cr
Checking system GCC version for gcc, intel and amd toolchain
Your System gcc/g++/gfortran version should be consistent
Minimum required version: 5
Your gcc version: 11.4.0
Your g++ version: 11.4.0
Your gfortran version: 11.4.0
Your GCC version seems to be enough for ABACUS installation.
Compiling with 128 processes for target native.
Step gcc took 0.00 seconds.
Step intel took 0.00 seconds.
==================== Finding AMD compiler from system paths ====================
path to clang is /data/softwares/AMD/5.0.0-aocl-aocc/aocc/bin/clang
path to clang++ is /data/softwares/AMD/5.0.0-aocl-aocc/aocc/bin/clang++
path to gfortran is /usr/bin/gfortran
Found lib directory /usr/lib/x86_64-linux-gnu
```

```
CC  is /data/softwares/AMD/5.0.0-aocl-aocc/aocc/bin/clang
Check the AMD C compiler path
CXX is /data/softwares/AMD/5.0.0-aocl-aocc/aocc/bin/clang++
Check the AMD C++ compiler path
FC  is gfortran
Check the GNU Fortran compiler path
Step amd took 1.00 seconds.
```

细心的小伙伴可能发现这里链接的 Fortran 编译器是 gfortran 而不是 AMD-AOCC 的 flang，原因会在后续说明。

2. 下载 OpenBLAS，通过 OpenBLAS 内置的 genarch 程序识别系统指令集和架构：

```
==================== Getting proc arch info using OpenBLAS tools ====================
wget  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29 -O OpenBLAS-0.3.29.tar.gz --no-check
--2025-06-02 23:18:25--  https://codeload.github.com/OpenMathLib/OpenBLAS/tar.gz/v0.3.29
Resolving codeload.github.com (codeload.github.com)... 20.205.243.165
Connecting to codeload.github.com (codeload.github.com)|20.205.243.165|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [application/x-gzip]
Saving to:  'OpenBLAS-0.3.29.tar.gz'

OpenBLAS-0.3.29.tar.gz           [                                <=>        ]  23.53M  3.69MB/s    in 8.

2025-06-02 23:18:34 (2.92 MB/s) -  'OpenBLAS-0.3.29.tar.gz'  saved [24671913]

OpenBLAS-0.3.29.tar.gz: OK
Checksum of OpenBLAS-0.3.29.tar.gz Ok
OpenBLAS detected LIBCORE = zen
OpenBLAS detected ARCH    = x86_64
```

此处若出现提示：./f_check: 100: [: Illegal number: , 须知其只是运行 OpenBLAS genarch 程序中可能出现的问题，不影响 Toolchain 编译过程。

3. 自动下载和安装 CMake 编译库以及 OpenMPI 并行库

```
==================== Installing CMake ====================
wget  https://cmake.org/files/v3.31/cmake-3.31.7-linux-x86_64.sh -O cmake-3.31.7-linux-x86_64.sh --no-chec
cmake-3.31.7-linux-x86_64.sh: OK
Checksum of cmake-3.31.7-linux-x86_64.sh Ok
Installing from scratch into /home/mikoto/abacus-develop/toolchain/install/cmake-3.31.7
Step cmake took 26.00 seconds.
==================== Installing OpenMPI ====================
wget  https://download.open-mpi.org/release/open-mpi/v5.0/openmpi-5.0.7.tar.bz2 -O openmpi-5.0.7.tar.bz2 -
openmpi-5.0.7.tar.bz2: OK
Checksum of openmpi-5.0.7.tar.bz2 Ok
Installing from scratch into /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7
Found directory /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/bin
Found directory /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/lib
Found directory /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/include
```

```
mpiexec is installed as /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/bin/mpiexec
mpicc is installed as /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/bin/mpicc
mpicxx is installed as /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/bin/mpicxx
mpifort is installed as /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/bin/mpifort
Step openmpi took 541.00 seconds.
```

4. 从环境变量中定位并链接 AOCL 中的 BLAS，LAPACK，ScaLAPACK 和 FFTW
5. 自动下载并编译安装：数学库 ELPA，泛函库 LibXC，其他功能库 CEREAL, RapidJSON 以及可选功能库：LibRI, LibComm, LibTorch, Libnpy 等。后续输出如下：（本案例基于作者本人的 AMD 机器完成）

```
==================== Finding AOCL from system paths ====================
libblis is found in ld search path
libflame is found in ld search path
Found lib directory /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib_LP64
Found lib directory /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib_LP64
Step aocl took 0.00 seconds.
==================== Installing LIBXC ====================
wget  https://gitlab.com/libxc/libxc/-/archive/7.0.0/libxc-7.0.0.tar.bz2 -O libxc-7.0.0.tar.bz2 --no-check
libxc-7.0.0.tar.bz2: OK
Checksum of libxc-7.0.0.tar.bz2 Ok
Installing from scratch into /home/mikoto/abacus-develop/toolchain/install/libxc-7.0.0
Step libxc took 20.00 seconds.
==================== Finding FFTW from system paths ====================
libfftw3 is found in ld search path
libfftw3_omp is found in ld search path
libfftw3_mpi is found in ld search path
Found lib directory /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib_LP64
Step fftw took 0.00 seconds.
==================== Finding ScaLAPACK from system paths ====================
libscalapack is found in ld search path
Found lib directory /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib_LP64
Step scalapack took 1.00 seconds.
==================== Installing ELPA ====================
wget  https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2025.01.001/elpa-2025.01.001.tar.gz -O e
elpa-2025.01.001.tar.gz: OK
Checksum of elpa-2025.01.001.tar.gz Ok
Installing from scratch into /home/mikoto/abacus-develop/toolchain/install/elpa-2025.01.001/cpu
Step elpa took 158.00 seconds.
==================== Installing CEREAL ====================
===> Notice: This version of CEREAL is downloaded in GitHub master repository  <===
wget  https://codeload.github.com/USCiLab/cereal/tar.gz/master -O cereal-master.tar.gz --no-check-certific
Installing from scratch into /home/mikoto/abacus-develop/toolchain/install/cereal-master
Step cereal took 2.00 seconds.
==================== Installing RAPIDJSON ====================
===> Notice: This version of rapidjson is downloaded in GitHub master repository <===
wget  https://codeload.github.com/Tencent/rapidjson/tar.gz/master -O rapidjson-master.tar.gz --no-check-ce
Installing from scratch into /home/mikoto/abacus-develop/toolchain/install/rapidjson-master
Step rapidjson took 3.00 seconds.
```

```
Step libtorch took 0.00 seconds.
Step libnpy took 0.00 seconds.
Step libri took 0.00 seconds.
Step libcomm took 0.00 seconds.
========================= usage =========================
Done!
To use the installed tools and libraries and ABACUS version
compiled with it you will first need to execute at the prompt:
  source /home/mikoto/abacus-develop/toolchain/install/setup
To build ABACUS by gnu-toolchain, just use:
    ./build_abacus_gnu.sh
To build ABACUS by intel-toolchain, just use:
    ./build_abacus_intel.sh
To build ABACUS by amd-toolchain in gcc-aocl, just use:
    ./build_abacus_gcc-aocl.sh
To build ABACUS by amd-toolchain in aocc-aocl, just use:
    ./build_abacus_aocc-aocl.sh
or you can modify the builder scripts to suit your needs.
```

如此即为编译成功。此后，我们通过 vim 打开 build_abacus_aocc-aocl.sh

```
vim ./build_abacus_aocc-aocl.sh
```

查看该文件

```
#!/bin/bash
#SBATCH -J build
#SBATCH -N 1
#SBATCH -n 16
#SBATCH -o install.log
#SBATCH -e install.err
# JamesMisaka in 2025.03.09

# Build ABACUS by amd-openmpi toolchain

# module load openmpi aocc aocl

ABACUS_DIR=..
TOOL=$(pwd)
INSTALL_DIR=$TOOL/install
source $INSTALL_DIR/setup
cd $ABACUS_DIR
ABACUS_DIR=$(pwd)
#AOCLhome=/opt/aocl-linux-aocc-5.0.0/5.0.0/aocl/  # user should specify this parameter

BUILD_DIR=build_abacus_aocl
rm -rf $BUILD_DIR

```

```
PREFIX=$ABACUS_DIR
ELPA=$INSTALL_DIR/elpa-2025.01.001/cpu
# ELPA=$INSTALL_DIR/elpa-2025.01.001/nvidia # for gpu-lcao
CEREAL=$INSTALL_DIR/cereal-master/include/cereal
LIBXC=$INSTALL_DIR/libxc-7.0.0
RAPIDJSON=$INSTALL_DIR/rapidjson-master/
LAPACK=$AOCLhome/lib
SCALAPACK=$AOCLhome/lib
FFTW3=$AOCLhome
# LIBRI=$INSTALL_DIR/LibRI-0.2.1.0
# LIBCOMM=$INSTALL_DIR/LibComm-master
# LIBTORCH=$INSTALL_DIR/libtorch-2.1.2/share/cmake/Torch
# LIBNPY=$INSTALL_DIR/libnpy-1.0.1/include
# DEEPMD=$HOME/apps/anaconda3/envs/deepmd # v3.0 might have problem

# if clang++ have problem, switch back to g++

cmake -B $BUILD_DIR -DCMAKE_INSTALL_PREFIX=$PREFIX \
        -DCMAKE_CXX_COMPILER=clang++ \
        -DMPI_CXX_COMPILER=mpicxx \
        -DLAPACK_DIR=$LAPACK \
        -DSCALAPACK_DIR=$SCALAPACK \
        -DFFTW3_DIR=$FFTW3 \
        -DELPA_DIR=$ELPA \
        -DCEREAL_INCLUDE_DIR=$CEREAL \
        -DLibxc_DIR=$LIBXC \
        -DENABLE_LCAO=ON \
        -DENABLE_LIBXC=ON \
        -DUSE_OPENMP=ON \
        -DUSE_ELPA=ON \
        -DENABLE_RAPIDJSON=ON \
        -DRapidJSON_DIR=$RAPIDJSON \
#         -DENABLE_DEEPKS=1 \
#         -DTorch_DIR=$LIBTORCH \
#         -Dlibnpy_INCLUDE_DIR=$LIBNPY \
#         -DENABLE_LIBRI=ON \
#         -DLIBRI_DIR=$LIBRI \
#         -DLIBCOMM_DIR=$LIBCOMM \
#             -DDeePMD_DIR=$DEEPMD \
#         -DUSE_CUDA=ON \
#         -DENABLE_CUSOLVERMP=ON \
#         -D CAL_CUSOLVERMP_PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/2x.xx/math_libs/1x.x/targets/x86_64-linux/lib

# if one want's to include deepmd, your system gcc version should be >= 11.3.0 for glibc requirements

cmake --build $BUILD_DIR -j `nproc`
cmake --install $BUILD_DIR 2>/dev/null

```

```
# generate abacus_env.sh
cat << EOF > "${TOOL}/abacus_env.sh"
#!/bin/bash
source $INSTALL_DIR/setup
export PATH="${PREFIX}/bin":\${PATH}
EOF

# generate information
cat << EOF
========================== usage =========================
Done!
To use the installed ABACUS version
You need to source ${TOOL}/abacus_env.sh first !
"""
EOF
```

我们需要修改 AOCLhome 环境变量为自己的 AOCL 位置，这里需要手动修改的主要原因是 AOCLhome 这个环境变量在 AMD AOCL 的 modulefile 里面并没有写成全局的形式，说的更具体一点，其 modulefile 里面 AOCLhome 的定义用的是 set 指令而非 setenv：

```
AOCLhome=/data/softwares/AMD/5.0.0-aocl-aocc/aocl
```

完成这一修改之后，通过:wq 保存并退出，再运行这一脚本

```
sh ./build_abacus_aocc-aocl.sh
```

即可完成基于 AOCC-AOCL 的 ABACUS 编译，编译过程大致输出如下：

```
-- The CXX compiler identification is Clang 17.0.6
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /data/softwares/AMD/5.0.0-aocl-aocc/aocc/bin/clang++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- RapidJSON found. Headers:
-- Found Git: /usr/bin/git (found version "2.34.1")
-- Found git: attempting to get commit info...
-- Current commit hash: ede143784
-- Last commit date: Sat May 31 20:20:02 2025 +0800
-- Found Cereal: /home/mikoto/abacus-develop/toolchain/install/cereal-master/include/cereal
-- Found PkgConfig: /usr/bin/pkg-config (found version "0.29.2")
-- Found ELPA: /home/mikoto/abacus-develop/toolchain/install/elpa-2025.01.001/cpu/lib/libelpa_openmp.so
-- Performing Test ELPA_VERSION_SATISFIES
-- Performing Test ELPA_VERSION_SATISFIES - Success
-- Found MPI_CXX: /home/mikoto/abacus-develop/toolchain/install/openmpi-5.0.7/lib/libmpi.so (found version "3.1
-- Found MPI: TRUE (found version "3.1")
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
```

```
-- Found Threads: TRUE
-- Found OpenMP_CXX: -fopenmp=libomp (found version "5.1")
-- Found OpenMP: TRUE (found version "5.1")
-- Looking for a CUDA compiler
-- Looking for a CUDA compiler - NOTFOUND
-- Found FFTW3: /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib/libfftw3_omp.so
-- Looking for sgemm_
-- Looking for sgemm_ - not found
-- Looking for sgemm_
-- Looking for sgemm_ - found
-- Found BLAS: /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib/libblis.so
-- Looking for cheev_
-- Looking for cheev_ - not found
-- Looking for cheev_
-- Looking for cheev_ - found
-- Found LAPACK: /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib/libflame.so;/data/softwares/AMD/5.0.0-aocl-aocc/a
-- Found ScaLAPACK: /data/softwares/AMD/5.0.0-aocl-aocc/aocl/lib/libscalapack.so
-- Checking for one of the modules 'libxc'
-- Found Libxc: /home/mikoto/abacus-develop/toolchain/install/libxc-7.0.0/lib/libxc.so
-- Found Libxc: version 7.0.0
-- Configuring done (3.2s)
-- Generating done (0.2s)
-- Build files have been written to: /home/mikoto/abacus-develop/build_abacus_aocl
[  0%] Building CXX object source/CMakeFiles/driver.dir/driver_run.cpp.o
[  0%] Building CXX object source/module_base/module_container/CMakeFiles/container.dir/ATen/ops/einsum_op.cpp.
...
[100%] Built target lr
[100%] Built target io_basic
[100%] Building CXX object CMakeFiles/abacus.dir/source/main.cpp.o
[100%] Linking CXX executable abacus
[100%] Built target abacus
-- Install configuration: ""
-- Installing: /home/mikoto/abacus-develop/bin/abacus
========================== usage ==========================
Done!
To use the installed ABACUS version
You need to source /home/mikoto/abacus-develop/toolchain/abacus_env.sh first !
```

完成编译后，通过 toolchain 给出的 abacus_env.sh 环境变量文件，即可加载 ABACUS 相关环境。需注意，从系统内读入的环境变量并不会被写入到 abacus_env.sh 对应的 install/setup 文件中，因此在实际使用这一 abacus 时，除了 source abacus_env.sh，我们还需要加载对应的 AOCC-AOCL 环境。

```
# after load AOCC-AOCL env via module-load or source
source abacus_env.sh
```

如此即可加载 ABACUS 所有依赖库环境和 ABACUS 本体，此时可以通过 abacus –version 命令确认 ABACUS 已正确安装和加载：

```
(base) mikoto@amd-cpu:~/abacus-develop/toolchain\$ which abacus
/home/mikoto/abacus-develop/bin/abacus
(base) mikoto@amd-cpu:~/abacus-develop/toolchain\$ abacus --version
ABACUS version v3.10.0
```

这样编译就是完成了，此时我们回顾一下 toolchain_aocc-aocl.sh 脚本

```
(base) mikoto@amd-cpu:~/abacus-develop/toolchain\$ cat toolchain_aocc-aocl.sh
#!/bin/bash
#SBATCH -J install
#SBATCH -N 1
#SBATCH -n 16
#SBATCH -o compile.log
#SBATCH -e compile.err

# JamesMisaka in 2025-05-05
# install abacus dependency by aocc-aocl toolchain
# openmpi is recommended to use
# libtorch and libnpy are for deepks support, which can be =no
# if you want to run EXX calculation, you should set --with-libri=install
# gpu-lcao supporting modify: CUDA_PATH and --enable-cuda
# export CUDA_PATH=/usr/local/cuda

./install_abacus_toolchain.sh \
--with-amd=system \
--math-mode=aocl \
--with-intel=no \
--with-gcc=no \
--with-openmpi=install \
--with-cmake=install \
--with-scalapack=system \
--with-libxc=install \
--with-fftw=system \
--with-elpa=install \
--with-cereal=install \
--with-rapidjson=install \
--with-libtorch=no \
--with-libnpy=no \
--with-libri=no \
--with-libcomm=no \
--with-4th-openmpi=no \
--with-flang=no \
| tee compile.log
# to use openmpi-version4: set --with-4th-openmpi=yes
```

可以发现几个有趣的点，分别解释如下：

- –with-scalapack, –with-fftw 都是 system，并通过这一方案分别链接 AOCL 里面的 ScaLAPACK 和 FFTW。这点和 Intel Toolchain 的–with-scalapack, –with-fftw 都是 no 不同，因为在那时这些数学库都被打包在 MKL 里

面了。

- –with-openmpi=install : AOCL-AOCC 并不自带 MPI，并推荐采用 OpenMPI 作为 MPI 库。Toolchain 默认采用 OpenMPI v5，如果用户发现 OpenMPI 编译存在问题，可以指定–with-4th-openmpi=yes 来下载安装 v4 版本的 OpenMPI，这一版本通常兼容性更好，但计算效率会偏低。
- –with-flang=no: 若使用 flang 编译器，则 ELPA 的编译是会报错的，虽然可以通过修改 ELPA 源码里面的 libtool 文件绕过相关报错，但绕过这些报错之后所编译出来的 ABACUS 使用 LCAO-genelpa 计算时计算效率相对较差，故 Toolchain 默认方案是将 flang 编译器强行替换为 gfortran。在 Toolchain 中，用户可以修改–with-flang=yes 尝试使用 flang 编译器编译依赖库并验证这件事情，此时相关源码修改会自动进行。也正因这点，往往 AOCC-AOCL Toolchain 编译得到的 ABACUS 在运行 LCAO-genelpa SCF 计算时，效率不一定比 GCC Toolchain 高出很多，且可能存在潜在问题。

#### 基于 GCC-AOCL 的编译方案

如果我们仔细查看 AOCL 官网，会发现 AOCL 不仅有 AOCC 依赖的包，还有 GCC 依赖的包，我们也可以采用 GCC 依赖的 AOCL 进行编译，这些编译方案在 Toolchain 里面也提供了支持，具体可见 toolchain_gcc-aocl.sh 和 build_abacus_gcc-aocl.sh 脚本，操作流程与 AOCC-AOCL 方案基本一致。

1. 加载 GCC 依赖的 AOCL 环境
2. 运行 toolchain_gcc-aocl.sh 脚本
3. 安装完成后，编辑 build_abacus_gcc-aocl.sh 脚本中的 AOCLhome 环境变量
4. 编辑完成后保存退出，运行 build_abacus_gcc-aocl.sh 完成 ABACUS 本体编译

仅有的差别是编译器为 GCC 编译器，而 AOCL 库是 GCC 依赖的。与此同时，基于 Toolchain，如下方案也是可以尝试的：

- 可采用 AOCC 依赖的 AOCL 库，通过 GCC-AOCL Toolchain 编译 ABACUS 依赖和 ABACUS 本体，这点只需要在加载环境变量的时候加载 AOCC 和对应 AOCL 库，并在编译时采用 GCC-AOCL Toolchain 的编译脚本等。
- 可采用 AOCC-AOCL Toolchain 编译 ABACUS 依赖库，而用 GCC-AOCL 编译 ABACUS 本体，这只需要在加载好 AOCC-AOCL 库的环境变量后，先运行 toolchain_aocc-aocl.sh，再编辑并运行 build_abacus_gcc-aocl.sh 即可。

## 2.7 编译 Nvidia GPU 版本的 ABACUS

### 2.7.1 介绍

本教程旨在介绍 ABACUS 在支持 NVIDIA GPU 的服务器上的编译与使用。

#### 当前 ABACUS 支持的 GPU 并行计算

INPUT 文件中 device 参数需设置为 gpu。
目前 GPU 版本的 ABACUS 仅支持 PW 基组的计算，因此 INPUT 文件中 basis_type 参数仅能设置为 pw。
详细请看官方文档：https://abacus.deepmodeling.com/en/latest/advanced/acceleration/cuda.html
LCAO 基组的 GPU 计算版本近期会发布。

#### NVIDIA GPU & CUDA

NVIDIA GPU (Graphics Processing Unit) 是由一种专门设计来处理图形和并行计算任务的电子芯片。自从 NVIDIA 推出其首款 GPU 以来，它们已经从仅仅处理视频游戏图形的设备发展成为能处理各种高性能计算和深度学习任务的强大工具。

CUDA（Compute Unified Device Architecture），是显卡厂商 NVIDIA 推出的运算平台。CUDA 是一种由 NVIDIA 推出的通用并行计算架构，该架构使 NVIDAI GPU 能够解决复杂的计算问题。

为了使 ABACUS 的部分功能支持 NVIDIA GPU，ABACUS 开发团队针对 NVIDIA GPU 写了大量的 CUDA 代码，使其在 NVIDIA GPU 上有较高的运行效率。相比于 CPU 版本的计算任务，有极高的效率提升！

新闻稿：https://mp.weixin.qq.com/s/D8gcQb0bikMdgizLsbvCfQ

### 2.7.2 检查环境

检查当前环境是否有 NVIDIA GPU

nvidia-smi 命令帮助检测你的服务器是否有 NVIDIA GPU 设备。

nvcc –version 命令帮助你检测你的服务器软件环境中是否包含 cuda toolkit

更详细的内容查看：https://abacus.deepmodeling.com/en/latest/advanced/acceleration/cuda.html#required-hardware-software

### 2.7.3 编译

如果您已经尝试过使用 GCC 编译 ABACUS 教程或者 Intel oneAPI 编译 ABACUS 教程，那么编译支持在 NVIDIA GPU 上运行的 ABACUS 十分简单。

在保证可以正常编译 CPU 版本的 ABACUS 的环境下，使用：

```
cmake -B build -DUSE_CUDA=1 # for GCC
cd build && make -j`nproc`
# or
CXX=icpx cmake -B build -DUSE_CUDA=1 # for Intel oneAPI
cd build && make -j`nproc`h
```

这样编译出来的 abacus 可执行文件就是支持 NVIDIA GPU 运行的 ABACUS。

## 2.8 ABACUS LCAO 基组 GPU 版本使用说明

### 2.8.1 简介

为了提升 ABACUS 的运行效率，2024 年 10 月发布的 ABACUS 3.8 版本支持 GPU 环境下采用 LCAO（Linear Combination of Atomic Orbitals）基组（即数值原子轨道基组）对 Kohn-Sham 方程进行求解。目前该版本提供 GPU 支持的模块有广义本征值求解器以及实空间生成哈密顿量的格点积分，这两部分也是 LCAO 基组求解 Kohn-Sham 方程的热点。在广义本征值求解器方面，目前 3.8 版本集成了 cuSolver，cuSolverMP，ELPA 软件，从而可以支持单卡和多卡的 GPU 加速。在格点积分模块里我们自研了 GPU 积分加速算法。开发团队未来还会针对 LCAO 算的更多模块提供 GPU 硬件以及其它国产硬件的支持。

### 2.8.2 安装

要使用 GPU 版的 ABACUS，使用 CMAKE 编译 ABACUS 的时候（ABACUS 编译方法详见 https://abacus.deepmodeling.com/en/latest/quick_start/easy_install.html），需要进行如下设置。

- 格点积分 GPU 版支持，以及 cusolver 求解器支持：需要安装 cuda-toolkit，并在编译 ABACUS 时设置 -DUSE_CUDA=ON。

- (可选项)cusolvermp 求解器支持：编译 ABACUS 之前，需要确保系统上安装了 cusolvermp 相关库，具体安装方法见 https://docs.nvidia.com/cuda/cusolvermp/。除了设置 -DUSE_CUDA=ON 之外，还需要设置 -DENABLE_CUSOLVERMP=ON。
- (可选项)GPU 版 ELPA 求解器支持：编译 ABACUS 之前，需要确保系统上安装了支持 GPU 版本的 ELPA，安装方法详见 https://github.com/marekandreas/elpa/blob/master/documentation/INSTALL.md，安装过程可以参考 https://github.com/deepmodeling/abacus-develop/pull/4969。安装好 ELPA 之后，在编译 ABACUS 时，需要设置 -DUSE_CUDA=ON, -DUSE_ELPA=ON。

### 2.8.3 Toolchain 安装

Toolchain 也支持安装 GPU 版 ABACUS 并支持 GPU-LCAO 计算，只需要在 toolchain*.sh 里打开 GPU 相关选项即可自动安装 GPU 支持的 ELPA，以 toolchain_intel.sh 文件为例：

```
./install_abacus_toolchain.sh \
--with-intel=system \
--math-mode=mkl \
--with-gcc=no \
--with-intelmpi=system \
--with-cmake=install \
--with-scalapack=no \
--with-libxc=install \
--with-fftw=no \
--with-elpa=install \
--with-cereal=install \
--with-rapidjson=install \
--with-libtorch=no \
--with-libnpy=no \
--with-libri=no \
--with-libcomm=no \
--with-intel-classic=no \
--enable-cuda \
--gpu-ver=75 \
```

编译完依赖库之后，类似于上面的，在 build_abacus_*.sh 中加入-DUSE_CUDA=ON，同时切换 ELPA 的指向路径（子路径从 cpu 切换到 nvidia），即可使用 Toolchain 完成 GPU-LCAO 功能的 ABACUS 安装。

### 2.8.4 使用

**1. INPUT 参数设置**

目前 ABACUS 的 LCAO 基组已经 GPU 化的模块包括格点积分模块和广义特征值求解模块。这两个模块是否使用 GPU 加速可以分开设置。与格点积分和广义特征值求解相关的输入参数包括 device 以及 ks_solver。下面详细介绍一下这两个参数的设置，注意以下介绍基于 ABACUS 的 LCAO 基组（INPUT 文件中的 basis_type 需要设置为 lcao）。

- device:device 用于指定 ABACUS 是否使用 GPU 来加速运算，可以设置为 cpu 或者 gpu。如果编译的是 GPU 版的 ABACUS（编译时设置了 -DUSE_CUDA=ON），同时机器中至少有一张 GPU 卡，那么 device 的默认值为 gpu, 否则为 cpu。

  如果 device 设置为 gpu, 那么 ABACUS 将使用 GPU 来加速计算，对应的，格点积分将使用 GPU 加速计算，

广义特征值求解器 ks_solver 也将默认设置为 cusolver, 如果您的机器包含多张 GPU，也可以将 ks_solver 设置为 cusolvermp 或者 elpa 来调用多卡求解特征值；

如果 device 设置为 cpu，ABACUS 将使用 CPU 进行计算，格点积分部分也会使用 CPU 计算，广义特征值求解器 ks_solver 默认设置为 scalapack_gvx,genelpa, 或者 lapack(默认值详见 https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#ks-solver)

- ks_solver:ks_solver 用于设置广义特征值求解器，目前 LCAO 基组支持的广义特征值求解器有 lapack,scalapack_gvx,genelpa,cusolver,cusolvermp,elpa。其中 cusolver,cusolvermp 与 elpa 可以调用 GPU 进行广义特征值求解,cusolver 只支持单 GPU 卡求解，cusolvermp 与 elpa 支持多 GPU 卡求解。

在 INPUT 文件中设置好 device 和 ks_solver 参数后，运行 ABACUS，即可调用 GPU 进行运算。

**2. 多卡计算**

ABACUS 的格点积分模块以及广义特征值求解模块均支持多块 GPU 卡加速，ABACUS 调用的 GPU 数量通过 MPI 进程数来设置。如果设置 n 个进程，ABACUS 将会自动调用 n 张 GPU 卡来进行计算，若机器中的 GPU 卡数小于 n，ABACUS 则会调用机器中所有 GPU 卡来进行运算，不推荐进程数设置成大于 GPU 卡数。需要注意的是，ABACUS 的广义特征值求解部分只有 cusolvermp 和 elpa 支持多卡加速，如果 ks_solver 设置成 cusolver，无论设置多少个进程，ABACUS 都只会调用一张 GPU 卡来进行广义特征值求解。

### 2.8.5 示例

下面以 tests/performance/P102_si64_lcao 的 64 个金刚石结构的硅原子体系为例，展示如何使用 GPU 版 ABACUS 进行 LCAO 基组下的计算。

环境：GPU/双 3090, CPU / Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz

ABACUS 版本号：3.8.0, commit:72b1d7ce9

**1. GPU 算例和结果**

我们首先调用双 GPU 卡进行格点积分运算，同时使用 cusolver 进行广义特征值求解：修改 INPUT 文件，添加一行 “device gpu” , 同时将 ks_solver 设置为 cusolver，运行命令 “OMP_NUM_THREADS=12 mpirun -n 2 abacus”, 输出为：

```
Info: Local MPI proc number: 2,OpenMP thread number: 12,Total thread number: 24,Local thread limit: 56

                              ABACUS v3.8.0

               Atomic-orbital Based Ab-initio Computation at UStc

                    Website: http://abacus.ustc.edu.cn/
               Documentation: https://abacus.deepmodeling.com/
                  Repository: https://github.com/abacusmodeling/abacus-develop
                              https://github.com/deepmodeling/abacus-develop
                      Commit: 72b1d7ce9 (Sat Oct 5 16:46:53 2024 +0800)

 Sat Oct  5 19:04:21 2024
 MAKE THE DIR         : OUT.autotest/
 RUNNING WITH DEVICE  : GPU / NVIDIA GeForce RTX 3090
 UNIFORM GRID DIM        : 192 * 192 * 96
 UNIFORM GRID DIM(BIG)   : 48 * 48 * 24
```

```
 DONE(0.652318   SEC) : SETUP UNITCELL
 DONE(0.70875    SEC) : SYMMETRY
 DONE(0.789308   SEC) : INIT K-POINTS
 ----------------------------------------------------------
 Self-consistent calculations for electrons
 ----------------------------------------------------------
 SPIN    KPOINTS         PROCESSORS  NBASE
 1       1               2           832
 ----------------------------------------------------------
 Use Systematically Improvable Atomic bases
 ----------------------------------------------------------
 ELEMENT ORBITALS        NBASE       NATOM       XC
 Si      2s2p1d-8au      13          64
 ----------------------------------------------------------
 Initial plane wave basis and FFT box
 ----------------------------------------------------------
 DONE(0.818906   SEC) : INIT PLANEWAVE
 ------------------------------------------
 SELF-CONSISTENT :
 ------------------------------------------
 gemm_algo_selector::Fastest time: 0.03072 ms
 START CHARGE      : atomic
 DONE(8.17035    SEC) : INIT SCF
 * * * * * *
 << Start SCF iteration.
 ITER       ETOT/eV           EDIFF/eV         DRHO     TIME/s
 CU1     -6.83607351e+03   0.00000000e+00   1.7682e-01  13.42
 CU2     -6.83164916e+03   4.42435700e+00   1.2487e-01   7.31
 CU3     -6.83237991e+03  -7.30748856e-01   7.9826e-03   7.29
 CU4     -6.83238388e+03  -3.97353454e-03   1.2270e-03   7.22
 CU5     -6.83238393e+03  -5.42990673e-05   4.6013e-04   7.26
 CU6     -6.83238395e+03  -1.43977923e-05   3.8918e-05   7.30
 CU7     -6.83238395e+03  -3.55773794e-07   1.1467e-05   7.24
 CU8     -6.83238395e+03   3.54214631e-09   3.8113e-06   7.23
 CU9     -6.83238395e+03  -2.56457580e-09   6.6704e-08   6.62
 >> Leave SCF iteration.
 * * * * * *
---------------------------------------------------------------
 TOTAL-STRESS (KBAR)
---------------------------------------------------------------
        30.8367522660        -0.0000000000         0.0000000000
         0.0000000000        75.4069126649        36.2463263984
         0.0000000000        36.2463263984        75.4069126649
---------------------------------------------------------------
 TOTAL-PRESSURE: 60.550193 KBAR

TIME STATISTICS
-------------------------------------------------------------------------------
```

```
    CLASS_NAME               NAME            TIME/s  CALLS   AVG/s  PER/%
--------------------------------------------------------------------------------
                     total                   95.57   11      8.69   100.00
Driver               reading                 0.13    1       0.13   0.14
Input_Conv           Convert                 0.12    1       0.12   0.12
Driver               driver_line             95.43   1       95.43  99.86
UnitCell             check_tau               0.00    1       0.00   0.00
ESolver_KS_LCAO      before_all_runners      2.02    1       2.02   2.11
PW_Basis_Sup         setuptransform          0.29    1       0.29   0.30
PW_Basis_Sup         distributeg             0.27    1       0.27   0.28
mymath               heapsort                0.09    1097    0.00   0.09
Symmetry             analy_sys               0.06    1       0.06   0.06
PW_Basis_K           setuptransform          0.02    1       0.02   0.02
PW_Basis_K           distributeg             0.01    1       0.01   0.01
PW_Basis             setup_struc_factor      1.01    1       1.01   1.06
NOrbital_Lm          extra_uniform           0.04    5       0.01   0.04
Mathzone_Add1        SplineD2                0.00    5       0.00   0.00
Mathzone_Add1        Cubic_Spline_Interpolation 0.00 5       0.00   0.00
Mathzone_Add1        Uni_Deriv_Phi           0.04    5       0.01   0.04
ppcell_vl            init_vloc               0.04    1       0.04   0.04
Ions                 opt_ions                93.36   1       93.36  97.69
ESolver_KS_LCAO      runner                  77.64   1       77.64  81.24
ESolver_KS_LCAO      before_scf              6.01    1       6.01   6.29
ESolver_KS_LCAO      beforesolver            0.27    1       0.27   0.28
ESolver_KS_LCAO      set_matrix_grid         0.19    1       0.19   0.20
atom_arrange         search                  0.00    1       0.00   0.00
Grid_Technique       init                    0.13    1       0.13   0.14
Grid_BigCell         grid_expansion_index    0.01    2       0.01   0.01
Record_adj           for_2d                  0.05    1       0.05   0.05
Grid_Driver          Find_atom               0.01    1152    0.00   0.01
LCAO_domain          grid_prepare            0.00    1       0.00   0.00
Veff                 initialize_HR           0.01    1       0.01   0.01
OverlapNew           initialize_SR           0.01    1       0.01   0.01
EkineticNew          initialize_HR           0.00    1       0.00   0.00
NonlocalNew          initialize_HR           0.03    1       0.03   0.03
Charge               set_rho_core            0.00    1       0.00   0.00
Charge               atomic_rho              0.98    2       0.49   1.02
PW_Basis_Sup         recip2real              30.81   74      0.42   32.24
PW_Basis_Sup         gathers_scatterp        1.36    74      0.02   1.42
Potential            init_pot                3.73    1       3.73   3.90
Potential            update_from_charge      33.03   10      3.30   34.56
Potential            cal_fixed_v             0.42    1       0.42   0.44
PotLocal             cal_fixed_v             0.41    1       0.41   0.43
Potential            cal_v_eff               32.52   10      3.25   34.03
H_Hartree_pw         v_hartree               5.66    10      0.57   5.92
PW_Basis_Sup         real2recip              8.21    85      0.10   8.59
PW_Basis_Sup         gatherp_scatters        2.45    85      0.03   2.56
PotXC                cal_v_eff               26.76   10      2.68   28.00
```

| | | | | | | |
|---|---|---|---|---|---|---|
| XC_Functional | v_xc | 26.71 | 10 | 2.67 | 27.95 |
| Potential | interpolate_vrs | 0.05 | 10 | 0.00 | 0.05 |
| Symmetry | rhog_symmetry | 8.67 | 10 | 0.87 | 9.07 |
| Symmetry | group fft grids | 0.70 | 10 | 0.07 | 0.73 |
| H_Ewald_pw | compute_ewald | 0.02 | 1 | 0.02 | 0.03 |
| Charge_Mixing | init_mixing | 0.00 | 1 | 0.00 | 0.00 |
| HSolverLCAO | solve | 23.39 | 9 | 2.60 | 24.47 |
| HamiltLCAO | updateHk | 17.12 | 9 | 1.90 | 17.91 |
| OperatorLCAO | init | 15.90 | 27 | 0.59 | 16.63 |
| Veff | contributeHR | 11.04 | 9 | 1.23 | 11.55 |
| Gint_interface | cal_gint | 16.54 | 19 | 0.87 | 17.30 |
| Gint_interface | cal_gint_vlocal | 10.79 | 9 | 1.20 | 11.29 |
| Gint_k | transfer_pvpR | 0.25 | 9 | 0.03 | 0.26 |
| OverlapNew | calculate_SR | 1.18 | 1 | 1.18 | 1.23 |
| OverlapNew | contributeHk | 0.05 | 9 | 0.01 | 0.05 |
| EkineticNew | contributeHR | 1.18 | 9 | 0.13 | 1.23 |
| EkineticNew | calculate_HR | 1.17 | 1 | 1.17 | 1.23 |
| NonlocalNew | contributeHR | 3.59 | 9 | 0.40 | 3.76 |
| NonlocalNew | calculate_HR | 3.54 | 1 | 3.54 | 3.71 |
| OperatorLCAO | contributeHk | 0.05 | 9 | 0.01 | 0.05 |
| HSolverLCAO | hamiltSolvePsiK | 1.27 | 9 | 0.14 | 1.33 |
| DiagoCusolver | cusolver | 1.22 | 9 | 0.14 | 1.28 |
| ElecStateLCAO | psiToRho | 4.99 | 9 | 0.55 | 5.22 |
| elecstate | cal_dm | 0.26 | 10 | 0.03 | 0.27 |
| psiMulPsiMpi | pdgemm | 0.25 | 10 | 0.03 | 0.26 |
| DensityMatrix | cal_DMR | 0.10 | 10 | 0.01 | 0.10 |
| Gint | transfer_DMR | 0.18 | 9 | 0.02 | 0.19 |
| Gint_interface | cal_gint_rho | 4.39 | 9 | 0.49 | 4.60 |
| Charge_Mixing | get_drho | 0.04 | 9 | 0.00 | 0.04 |
| Charge | mix_rho | 4.85 | 8 | 0.61 | 5.07 |
| Charge | Broyden_mixing | 0.27 | 8 | 0.03 | 0.29 |
| ESolver_KS_LCAO | after_scf | 0.75 | 1 | 0.75 | 0.78 |
| ModuleIO | write_rhog | 0.16 | 1 | 0.16 | 0.17 |
| ESolver_KS_LCAO | cal_force | 15.72 | 1 | 15.72 | 16.45 |
| Force_Stress_LCAO | getForceStress | 15.72 | 1 | 15.72 | 16.45 |
| Forces | cal_force_loc | 0.94 | 1 | 0.94 | 0.99 |
| Forces | cal_force_ew | 0.77 | 1 | 0.77 | 0.81 |
| Forces | cal_force_cc | 0.00 | 1 | 0.00 | 0.00 |
| Forces | cal_force_scc | 1.02 | 1 | 1.02 | 1.06 |
| Stress_Func | stress_loc | 0.22 | 1 | 0.22 | 0.23 |
| Stress_Func | stress_har | 0.13 | 1 | 0.13 | 0.14 |
| Stress_Func | stress_ewa | 0.76 | 1 | 0.76 | 0.79 |
| Stress_Func | stress_cc | 0.00 | 1 | 0.00 | 0.00 |
| Stress_Func | stress_gga | 1.74 | 1 | 1.74 | 1.82 |
| Force_LCAO | ftable | 10.13 | 1 | 10.13 | 10.60 |
| Force_LCAO | allocate | 5.32 | 1 | 5.32 | 5.56 |
| LCAO_domain | build_ST_new | 2.55 | 2 | 1.28 | 2.67 |
| LCAO_domain | vnl_mu_new | 2.67 | 1 | 2.67 | 2.79 |

```
Force_LCAO        cal_fedm                  0.06   1        0.06   0.06
Force_LCAO        cal_ftvnl_dphi            0.02   1        0.02   0.03
Force_LCAO        cal_fvl_dphi              1.36   1        1.36   1.42
Gint_interface    cal_gint_force            1.36   1        1.36   1.42
Force_LCAO        cal_fvnl_dbeta            3.37   1        3.37   3.52
ESolver_KS_LCAO   cal_stress                0.00   1        0.00   0.00
ESolver_KS_LCAO   after_all_runners         0.00   1        0.00   0.00
ModuleIO          write_istate_info         0.00   1        0.00   0.00
----------------------------------------------------------------------------


START  Time  : Sat Oct  5 19:04:21 2024
FINISH Time  : Sat Oct  5 19:05:56 2024
TOTAL  Time  : 95
SEE INFORMATION IN : OUT.autotest/
```

**2. CPU 算例和结果**

可以看到 GPU 运行用了 95 秒，接下来我们调用 CPU 进行计算，在 INPUT 文件中设置 device 为 cpu，同时设置 ks_solver 为 scalapack_gvx，运行命令 “OMP_NUM_THREADS=12 mpirun -n 2 abacus”, 输出：

```
Info: Local MPI proc number: 2,OpenMP thread number: 12,Total thread number: 24,Local thread limit: 56

                             ABACUS v3.8.0

               Atomic-orbital Based Ab-initio Computation at UStc

                    Website: http://abacus.ustc.edu.cn/
               Documentation: https://abacus.deepmodeling.com/
                  Repository: https://github.com/abacusmodeling/abacus-develop
                              https://github.com/deepmodeling/abacus-develop
                      Commit: 72b1d7ce9 (Sat Oct 5 16:46:53 2024 +0800)

Sat Oct  5 19:06:27 2024
MAKE THE DIR         : OUT.autotest/
RUNNING WITH DEVICE  : CPU / Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz
UNIFORM GRID DIM        : 192 * 192 * 96
UNIFORM GRID DIM(BIG)   : 48 * 48 * 24
DONE(0.274561   SEC) : SETUP UNITCELL
DONE(0.330937   SEC) : SYMMETRY
DONE(0.411047   SEC) : INIT K-POINTS
---------------------------------------------------------
Self-consistent calculations for electrons
---------------------------------------------------------
SPIN    KPOINTS         PROCESSORS  NBASE
1       1               2           832
---------------------------------------------------------
Use Systematically Improvable Atomic bases
---------------------------------------------------------
```

```
 ELEMENT ORBITALS        NBASE       NATOM       XC
 Si      2s2p1d-8au      13          64
 ----------------------------------------------------------
 Initial plane wave basis and FFT box
 ----------------------------------------------------------
 DONE(0.438332   SEC) : INIT PLANEWAVE
 ------------------------------------------
 SELF-CONSISTENT :
 ------------------------------------------
 START CHARGE      : atomic
 DONE(7.72282    SEC) : INIT SCF
 * * * * * *
 << Start SCF iteration.
 ITER       ETOT/eV           EDIFF/eV         DRHO     TIME/s
 GE1     -6.83607351e+03   0.00000000e+00   1.7682e-01  25.64
 GE2     -6.83164916e+03   4.42435700e+00   1.2487e-01  19.58
 GE3     -6.83237991e+03  -7.30748856e-01   7.9826e-03  19.48
 GE4     -6.83238388e+03  -3.97353455e-03   1.2270e-03  19.56
 GE5     -6.83238393e+03  -5.42990735e-05   4.6013e-04  19.54
 GE6     -6.83238395e+03  -1.43977575e-05   3.8918e-05  19.67
 GE7     -6.83238395e+03  -3.55816331e-07   1.1467e-05  19.68
 GE8     -6.83238395e+03   3.53518576e-09   3.8113e-06  19.74
 GE9     -6.83238395e+03  -2.51817213e-09   6.6704e-08  19.04
 >> Leave SCF iteration.
 * * * * * *
---------------------------------------------------------------
 TOTAL-STRESS (KBAR)
---------------------------------------------------------------
        30.8367522660        -0.0000000000         0.0000000000
        -0.0000000000        75.4069126649        36.2463263984
         0.0000000000        36.2463263984        75.4069126649
---------------------------------------------------------------
 TOTAL-PRESSURE: 60.550193 KBAR

TIME STATISTICS
------------------------------------------------------------------------------
    CLASS_NAME                NAME            TIME/s  CALLS   AVG/s  PER/%
------------------------------------------------------------------------------
                   total                      224.00 11       20.36  100.00
 Driver            reading                    0.01   1        0.01   0.01
 Input_Conv        Convert                    0.00   1        0.00   0.00
 Driver            driver_line                223.99 1        223.99 99.99
 UnitCell          check_tau                  0.00   1        0.00   0.00
 ESolver_KS_LCAO   before_all_runners         1.75   1        1.75   0.78
 PW_Basis_Sup      setuptransform             0.03   1        0.03   0.01
 PW_Basis_Sup      distributeg                0.02   1        0.02   0.01
 mymath            heapsort                   0.09   1097     0.00   0.04
 Symmetry          analy_sys                  0.06   1        0.06   0.03
```

```
PW_Basis_K        setuptransform              0.02   1        0.02   0.01
PW_Basis_K        distributeg                 0.01   1        0.01   0.00
PW_Basis          setup_struc_factor          1.00   1        1.00   0.45
NOrbital_Lm       extra_uniform               0.04   5        0.01   0.02
Mathzone_Add1     SplineD2                    0.00   5        0.00   0.00
Mathzone_Add1     Cubic_Spline_Interpolation  0.00   5        0.00   0.00
Mathzone_Add1     Uni_Deriv_Phi               0.04   5        0.01   0.02
ppcell_vl         init_vloc                   0.04   1        0.04   0.02
Ions              opt_ions                    222.19 1        222.19 99.19
ESolver_KS_LCAO   runner                      188.63 1        188.63 84.21
ESolver_KS_LCAO   before_scf                  5.96   1        5.96   2.66
ESolver_KS_LCAO   beforesolver                0.20   1        0.20   0.09
ESolver_KS_LCAO   set_matrix_grid             0.13   1        0.13   0.06
atom_arrange      search                      0.00   1        0.00   0.00
Grid_Technique    init                        0.07   1        0.07   0.03
Grid_BigCell      grid_expansion_index        0.01   2        0.01   0.01
Record_adj        for_2d                      0.05   1        0.05   0.02
Grid_Driver       Find_atom                   0.01   1152     0.00   0.00
LCAO_domain       grid_prepare                0.00   1        0.00   0.00
Veff              initialize_HR               0.01   1        0.01   0.00
OverlapNew        initialize_SR               0.01   1        0.01   0.00
EkineticNew       initialize_HR               0.00   1        0.00   0.00
NonlocalNew       initialize_HR               0.02   1        0.02   0.01
Charge            set_rho_core                0.01   1        0.01   0.00
Charge            atomic_rho                  0.98   2        0.49   0.44
PW_Basis_Sup      recip2real                  30.88  74       0.42   13.78
PW_Basis_Sup      gathers_scatterp            1.37   74       0.02   0.61
Potential         init_pot                    3.74   1        3.74   1.67
Potential         update_from_charge          33.04  10       3.30   14.75
Potential         cal_fixed_v                 0.41   1        0.41   0.19
PotLocal          cal_fixed_v                 0.41   1        0.41   0.18
Potential         cal_v_eff                   32.59  10       3.26   14.55
H_Hartree_pw      v_hartree                   5.67   10       0.57   2.53
PW_Basis_Sup      real2recip                  8.18   85       0.10   3.65
PW_Basis_Sup      gatherp_scatters            2.45   85       0.03   1.09
PotXC             cal_v_eff                   26.82  10       2.68   11.97
XC_Functional     v_xc                        26.78  10       2.68   11.95
Potential         interpolate_vrs             0.04   10       0.00   0.02
Symmetry          rhog_symmetry               8.58   10       0.86   3.83
Symmetry          group fft grids             0.69   10       0.07   0.31
H_Ewald_pw        compute_ewald               0.03   1        0.03   0.01
Charge_Mixing     init_mixing                 0.00   1        0.00   0.00
HSolverLCAO       solve                       134.43 9        14.94  60.01
HamiltLCAO        updateHk                    71.31  9        7.92   31.83
OperatorLCAO      init                        70.08  27       2.60   31.29
Veff              contributeHR                65.22  9        7.25   29.12
Gint_interface    cal_gint                    135.97 19       7.16   60.70
Gint_interface    cal_gint_vlocal             64.86  9        7.21   28.95
```

```
Gint_Tools          cal_psir_ylm              22.23   41472     0.00    9.92
Gint_k              transfer_pvpR             0.25    9         0.03    0.11
OverlapNew          calculate_SR              1.19    1         1.19    0.53
OverlapNew          contributeHk              0.04    9         0.00    0.02
EkineticNew         contributeHR              1.19    9         0.13    0.53
EkineticNew         calculate_HR              1.18    1         1.18    0.53
NonlocalNew         contributeHR              3.59    9         0.40    1.60
NonlocalNew         calculate_HR              3.55    1         3.55    1.59
OperatorLCAO        contributeHk              0.05    9         0.01    0.02
HSolverLCAO         hamiltSolvePsiK           10.78   9         1.20    4.81
DiagoElpa           elpa_solve                10.73   9         1.19    4.79
ElecStateLCAO       psiToRho                  52.34   9         5.82    23.37
elecstate           cal_dm                    0.16    10        0.02    0.07
psiMulPsiMpi        pdgemm                    0.16    10        0.02    0.07
DensityMatrix       cal_DMR                   0.09    10        0.01    0.04
Gint                transfer_DMR              0.18    9         0.02    0.08
Gint_interface      cal_gint_rho              51.86   9         5.76    23.15
Charge_Mixing       get_drho                  0.04    9         0.00    0.02
Charge              mix_rho                   4.86    8         0.61    2.17
Charge              Broyden_mixing            0.28    8         0.03    0.12
ESolver_KS_LCAO     after_scf                 0.74    1         0.74    0.33
ModuleIO            write_rhog                0.16    1         0.16    0.07
ESolver_KS_LCAO     cal_force                 33.57   1         33.57   14.98
Force_Stress_LCAO   getForceStress            33.56   1         33.56   14.98
Forces              cal_force_loc             0.97    1         0.97    0.44
Forces              cal_force_ew              0.77    1         0.77    0.34
Forces              cal_force_cc              0.00    1         0.00    0.00
Forces              cal_force_scc             1.05    1         1.05    0.47
Stress_Func         stress_loc                0.23    1         0.23    0.10
Stress_Func         stress_har                0.13    1         0.13    0.06
Stress_Func         stress_ewa                0.75    1         0.75    0.34
Stress_Func         stress_cc                 0.00    1         0.00    0.00
Stress_Func         stress_gga                1.76    1         1.76    0.79
Force_LCAO          ftable                    27.88   1         27.88   12.45
Force_LCAO          allocate                  5.29    1         5.29    2.36
LCAO_domain         build_ST_new              2.55    2         1.28    1.14
LCAO_domain         vnl_mu_new                2.65    1         2.65    1.18
Force_LCAO          cal_fedm                  0.06    1         0.06    0.03
Force_LCAO          cal_ftvnl_dphi            0.02    1         0.02    0.01
Force_LCAO          cal_fvl_dphi              19.14   1         19.14   8.54
Gint_interface      cal_gint_force            19.14   1         19.14   8.54
Gint_Tools          cal_dpsir_ylm             6.23    2304      0.00    2.78
Gint_Tools          cal_dpsirr_ylm            1.84    2304      0.00    0.82
Force_LCAO          cal_fvnl_dbeta            3.36    1         3.36    1.50
ESolver_KS_LCAO     cal_stress                0.00    1         0.00    0.00
ESolver_KS_LCAO     after_all_runners         0.00    1         0.00    0.00
ModuleIO            write_istate_info         0.00    1         0.00    0.00
----------------------------------------------------------------------------
```

```
173
174
175 START  Time  : Sat Oct  5 19:06:27 2024
176 FINISH Time  : Sat Oct  5 19:10:11 2024
177 TOTAL  Time  : 224
178 SEE INFORMATION IN : OUT.autotest/
```

输出中的“RUNNING WITH DEVICE”一行标出了此次运算使用的设备，同时运行时间统计里可以看到格点积分以及广义特征值求解的具体耗时，通过这两项耗时可以对比 GPU 和 CPU 的计算效率。格点积分耗时对应 cal_gint, cal_gint_vlocal, cal_gint_rho,cal_gint_force 这几项，广义特征值求解对应 hamiltSolvePsiK 项（在上面输出中标黄项）。

可以看到 GPU 运行用了 95 秒，而同样体系用 CPU 运行用了 224 秒，GPU 对于该体系有一定的加速效果。

以上就是本教程的内容，希望对学习采用 ABACUS 结合 GPU 进行密度泛函理论计算的读者有所帮助，大家也可以在 bohrium 平台按照 ABACUS LCAO 基组 GPU 版本使用介绍 | Bohrium 跑一遍实际安装使用流程，有问题可以发邮箱给作者（见开头）。

## 2.9 在超算环境编译 ABACUS 的建议

由于不同超算环境提供的软件包版本各不相同，因此我们没办法提供给您确切的安装方法。一般来说，上述文档和 Toolchain 内已提供的安装模式能应对多数的 ABACUS 编译需求，但是某些时候超算环境的特点会使已有方法失效。我们将在本篇文档中介绍一些在超算环境中编译安装 ABACUS 的注意事项和相关知识。如果在具体编译操作过程中遇到了任何问题，请随时联系我们。

### 2.9.1 关于 ABACUS

首先，再次简单介绍一下 ABACUS。ABACUS（原子算筹）软件同时支持两种基矢量，一种是平面波（Plane wave，PW）基组，一种是 LCAO (Linear combination of atomic orbitals) 或者称 NAO (Numerical atomic orbitals) 基组。

用户可以编译仅支持 PW 基组的 ABACUS 或者同时支持两种基组的 ABACUS。同时，针对 PW 基组的 ABACUS，用户还可以选择编译串行版本或并行版本。但因为这个文档的主题是如何在超算环境中编译 ABACUS（应该没有人在超算上跑串行的 ABACUS 吧），我们接下来只会介绍并行版本的编译方法。如果想要在普通的服务器上安装串行版本的 PW 基组的 ABACUS，可以参考 GCC 编译 ABACUS 教程 · GitBook 和 Intel oneAPI 编译 ABACUS 教程 · GitBook。

### 2.9.2 超算平台的简单介绍

**1. 关于 Modules**

一般情况下，大多数超算平台是通过 Environment Modules 来管理基础软件库的。

如何使用 Modules？这里提供一些参考的教程：软件模块使用方法 - 上海交大超算平台用户手册 Documentation

**2. 超算上编译 ABACUS 的基本路线**

在超算平台上编译 ABACUS，同样也有两种选择：

一种是基于 GCC（the GNU Compiler Collection）编译一种是基于 Intel oneAPI 或者 Intel Parallel Studio 编译不得不说，使用 Intel oneAPI 将会简单很多！因此，如果你所使用的超算平台正好有 Intel oneAPI ，那么请毫不犹豫的使用它！

**ABACUS 依赖的软件库**

- C/C++ 编译器：由于 ABACUS 是由 C++ 语言编写的软件，所以一定需要 C++ 编译器。
- Fortran 编译器：Fortran 编译器并不直接用于编译 ABACUS 软件，而是需要用其编译 ABACUS 所依赖的数学库：BLAS、LAPACK、ScaLAPACK 和 ELPA。（这四个软件都是使用 Fortran 语言写的，因此需要依赖 Fortran 编译器）
- MPI 库：如果要编译并行版本的 ABACUS，那么 MPI Library 是必须的。同时也需要注意，如果你选择自己编译 ScaLAPACK 和 ELPA，那么 MPI Library 也是必须的。
- 数学库：
- BLAS：BLAS（Basic Linear Algebra Subprograms）是一组基本的线性代数库，用于高效执行常见的基础线性代数运算，如矩阵乘法和向量操作。
- LAPACK：LAPACK（Linear Algebra Package）是一个开源的数值线性代数库，用于解决线性代数问题，包括矩阵分解、方程组求解和特征值计算等。
- FFTW3：FFTW3（Fastest Fourier Transform in the West 3）是一个高性能、开源的快速傅里叶变换（FFT）库，用于高效计算各种傅里叶变换和逆变换。
- ScaLAPACK：ScaLAPACK（Scalable LAPACK）是一种并行计算库，构建在 LAPACK 之上，用于解决大规模线性代数问题，特别适用于分布式和并行计算环境。
- ELPA：ELPA（Eigenvalue SoLvers for Petaflop-Applications）是一个用于高性能计算的开源库，旨在解决大规模高性能计算中的特征值问题，特别是密集矩阵的特征值问题。（用户编译安装 abacus 的时候大多在这里出现问题）
- CEREAL：CEREAL 是一个 C++ 序列化库，用于将 C++ 对象转换为可存储或传输的数据格式，以及将其反序列化回对象。

如果你只编译仅支持 PW 基组的 ABACUS，那么你不需要安装 ScaLAPACK、ELPA 和 CEREAL。他们是 LCAO 基组才依赖的软件库。

**基于 Intel oneAPI 编译 ABACUS**

首先在此介绍一下 Intel oneAPI 和 Intel Parallel Studio 的区别。用户可能在超算平台上同时看到 Intel oneAPI 和 Intel Parallel Studio。这是因为 Intel Parallel Studio 是 oneAPI 的前身，或者说 Intel Parallel Studio 是 oneAPI 的子集。Intel Parallel Studio 在 2020 年已经停止维护，而 Intel oneAPI 却是在 2019 年 11 月正式提出。因此我们可以将 Intel oneAPI 看作是新版的 Parallel Studio 或加强版。

Intel oneAPI（Intel® oneAPI Toolkits）同时包含 C/C++ 编译器、Fortran 编译器和 Intel MPI Library。因此如果加载了 Intel oneAPI 环境，也就不用再次加载其他的 MPI Library。除此之外，Intel oneAPI 还包含了 Intel® oneAPI Math Kernel Library（简称为 MKL），这个软件库里面包含了大量常用的数学库，包括 ABACUS 依赖的 BLAS、LAPACK、FFTW3 和 ScaLAPACK 4 个软件库。

因此，如果你只编译支持 PW 基组的 ABACUS，那么 Intel oneAPI 就已经可以解决所有的依赖问题。

如果你要编译同时支持两种基组的 ABACUS，那么还需要手动编译安装 ELPA 和 CEREAL，安装完成后，可以参考官方文档中的 ABACUS-Installation-Options 编译 ABACUS。

注意：在使用 Intel oneAPI 编译 ABACUS 的时候，并不意味着软件环境中不需要 GCC。refer：How the Compiler Uses GCC Intel compilers 需要使用 GCC 中的头文件和库。因此同时也需要加载 GCC。

**基于 GCC 编译 ABACUS**

GCC（the GNU Compiler Collection）它同时包含 C/C++ 编译器和 Fortran 编译器，但其不包含 MPI Library。因此我们还需要额外的 MPI 库（Open MPI 或者 MPICH）。

由于这是基于 GCC 编译 ABACUS，如果用户所使用的超算环境中没有提供 BLAS、LAPACK、FFTW3、ScaLAPACK、ELPA 和 CEREAL 的 modules，那么这些数学库都需要用户一个个手动编译安装。

在安装完成后，可以参考官方文档中的 ABACUS-Installation-Options 编译 ABACUS。

### 2.9.3 拓展教程

**1. ABACUS 高级编译选择：**

Easy Installation ABACUS documentation
Advanced Installation Options ABACUS documentation

**2. 超算平台使用教程推荐：**

上海交大超算平台用户手册
中国科大超算中心用户使用手册
北大超算手册
超算小站

## 2.10 ABACUS 在曙光 DCU 集群与 AMD 显卡上的编译与使用

### 2.10.1 介绍

本教程旨在介绍 ABACUS 在曙光 DCU 计算平台上的编译与使用。在 AMD 显卡上的编译教程详见 ABACUS AMD 版本编译教程。

**1. DCU**

DCU (Deep Computing Unit) 是一款面向人工智能、科学计算的高性能全功能 GPGPU (General-Purpose computing on Graphics Processing Units) 加速卡。

中科海光基于 DCU 硬件提供完整的软件工具链，以 DTK(DCU toolkit) 为基础软件层为开发者提供运行、编译、调试和性能分析等功能，并提供多种深度优化的计算加速库。DCU 加速卡支持 ROCm/Hip 并行架构。

曙光计算集群采用 CPU 和 DCU 加速卡（Deep Computing Unit）相结合的异构融合计算体系结构。

**2. ABACUS 的异构并行计算**

INPUT 文件中 device 参数需设置为 gpu。

目前 GPU/DCU 版本的 ABACUS 仅支持 PW 基组的计算，因此 INPUT 文件中 basis_type 参数仅能设置为 pw。

### 2.10.2 准备

**1. 曙光计算平台**

用户需要在平台上申请异构计算资源：
https://ac.sugon.com/doc/1.0.6/30000/general-handbook/platform/source.html

**2. E-shell**

曙光计算平台采用 E-shell 来对管理节点进行操作。

- 可使用网页版或 E-Shell Client。
- 不可直接运行任务，而是使用 Slurm 调度系统。
- 采用 modules 工具来管理环境变量与系统依赖项。许多依赖如编译器版本等可以通过 modules 来处理。

**3. slurm**

Slurm 工作调度工具是面向 Linux 和 Unix 及类似内核的免费和开源工作调度程序，可以方便用户进行作业的提交、管理、监测。

- sinfo: 查看系统资源。
- squeue: 查看当前作业状态。
- salloc: 分配节点的作业提交，用 salloc 申请的节点会在当前终端退出时释放掉。用于程序测试以及中小型任务的提交。
- sbatch: 批处理模式的作业提交，需要编写 slurm 作业提交脚本。在 E-shell 的默认目录存有 slurm 脚本模板。在下面的流程中也会介绍提交 ABACUS 任务的脚本案例。
- srun: 交互式提交作业命令，有屏幕输出，但容易受网络波动影响，断网或关闭窗口会导致作业中断。

### 2.10.3 流程

**1. 配置超算环境**

```
module avail    # 列出已有环境
module list     # 查看当前已加载环境
module load     # 加载环境
module unload   # 卸载环境
```

昆山节点

```
module purge
1) compiler/devtoolset/7.3.1
2) compiler/rocm/dtk-22.10
3) compiler/cmake/3.17.2
4) mpi/hpcx/2.6.0/gcc-7.3.1
```

乌镇节点

```
1) compiler/devtoolset/7.3.1
2) compiler/dtk/23.04
3) compiler/cmake/3.23.1
4) mpi/hpcx/gcc-7.3.1
```

对于使用其他 DCU 节点（合肥、哈尔滨、西安）的用户，如果 module 中没有找到类似的环境，欢迎在 ABACUS 仓库提出 issue，我们将尽力协助解决。

**2. 编译 ABACUS 依赖软件包**

目前按照 DCU 版本已验证的编译方法，有三个数学库需要自行编译。

若曙光平台网络连接不畅，请在软件官网选择合适的软件包，再用曙光平台的 E-File 传送至节点。

FFTW: https://fftw.org/pub/fftw/fftw-3.3.10.tar.gz OpenBLAS: https://github.com/xianyi/OpenBLAS/releases/download/v0.3.21/O
0.3.21.tar.gz ScaLAPACK: https://github.com/Reference-ScaLAPACK/scalapack/archive/refs/tags/v2.2.0.tar.gz

**编译 FFTW**

```
tar -zxvf fftw-3.3.10.tar.gz

mkdir build
./configure --prefix=/work/home/your_username/fftw-3.3.10/build
cd ~/fftw-3.3.10/build
make
make install
```

FFTW 需要编译单精度版本和双精度版本：

```
cd ~/fftw-3.3.10/build
./configure --prefix=/work/home/your_username/fftw-3.3.10/build --enable-float
make
make install
```

**编译 OpenBLAS**

```
tar -zxvf OpenBLAS-0.3.23.tar.gz
cd OpenBLAS-0.3.23

make USE_OPENMP=1 NO_AVX512=1 FC="gfortran -fPIC" CC="gcc -fPIC" -j8
mkdir build
make PREFIX=/work/home/your_username/OpenBLAS-0.3.21/build install
```

**编译 ScaLAPACK**

```
cp SLmake.inc.example SLmake.inc

```

复制包中提供的 SLmake.inc.example 作为基准，并对 SLmake.inc 的内容作出一部分修改:

```
FC            = mpif90 -fPIC
CC            = mpicc -fPIC
BLASLIB       =
LAPACKLIB     = -L/work/home/your_username/OpenBLAS-0.3.21/build/lib -lopenblas
```

```
make
```

### 3. 编译 DCU 版本的 ABACUS

```
git clone https://gitee.com/deepmodeling/abacus-develop  # main 分支
cd abacus-develop
mkdir build
cd build
```

CMake 配置：
指定编译器为 clang，关闭 OpenMP、LCAO 计算模块，设定三个数学库的位置，设定 USE_ROCM=ON。

```
CC=clang CXX=clang++ cmake -B build -DUSE_OPENMP=OFF -DENABLE_LCAO=OFF \
-DFFTW3_DIR=/work/home/your_username/fftw-3.3.10/build/ \
-DLAPACK_DIR=/work/home/your_username/OpenBLAS-0.3.21/build/lib \
-DSCALAPACK_DIR=/work/home/your_username/scalapack-2.2.0/ \
-DUSE_ROCM=ON
```

Make 编译：make 部分建议不要并行编译。

```
make
```

### 4. 提交任务

salloc（中小型任务与交互性程序测试）

```
salloc -p [队列名] -N 1 -n 32 --gres=dcu:4

... load text ...

salloc: Waiting for resource configuration
salloc: Nodes node_num are ready for job  # 分配计算节点，可用 ssh 直接连接

ssh node_num

... 交互式进行计算任务
```

sbatch（大型任务与批量提交）
写一个作业提交脚本，可以参考以下结构：

```
#!/bin/bash
#SBATCH --job-name=ABACUS_GPU
#SBATCH --partition=kshdnormal
#SBATCH --nodes=1
#SBATCH --output=output.log
#SBATCH --ntasks-per-node=32
#SBATCH --mail-user=username@email
#SBATCH --gres=dcu:4  #dcu 个数
#SBATCH --time=01:00:00
#SBATCH --error=error.log

```

```
# 以上的 SBATCH 信息会由 slurm 识别
abacus=/work/home/your_username/abacus-develop/build/abacus_pw
# 设置环境
module purge
module load compiler/devtoolset/7.3.1
module load compiler/rocm/dtk-22.10
module load compiler/cmake/3.17.2
module load mpi/hpcx/2.11.0/gcc-7.3.1
# 运行脚本
cd your_task_path
mpirun -np 4 \$abacus
```

```
sbatch abacus_dcu.slurm
```

作业已提交，可在“作业管理”中查看。

### 2.10.4 结语

DCU 可以提高 ABACUS 计算性能，也充分利用了 ROCm 并行框架，使得 ABACUS 异构计算能应用在更多的平台上。

# 第 3 章 建模

## 3.1 ABACUS 结构文件 STRU 及其准备方式

### 3.1.1 背景

在使用 ABACUS 开展计算之前，我们需要准备好 ABACUS 计算用的输入文件，这些输入文件主要包括：

1. STRU 文件：ABACUS 的结构文件格式，可类比 VASP 的 POSCAR。
2. INPUT 文件：开展 ABACUS 计算的输入参数集合，可类比 VASP 的 INCAR。
3. KPT 文件：ABACUS 的 K 点文件格式，代表了开展第一性原理计算时的 K 空间采样情况，可类比 VASP 的 KPOINTS。

其中，结构文件 STRU 的准备是 ABACUS 输入文件准备的重要一环。一般来说，用户的结构文件来源主要包括：

1. 自行通过 Materials Studio Visualizer 等晶体结构建模可视化软件构建结构模型，并保存为这一建模软件所支持的格式
2. 从 Materials Project 等晶体结构数据库上下载的结构文件（多为 CIF 格式）
3. 其他来源，如求解晶体表征谱图得到的 CIF 文件，从文献中获取的结构文件等。

这些结构文件的格式大多为 CIF，POSCAR，XSD 等，不会直接是 ABACUS 的 STRU 文件。那么，我们该如何准备 ABACUS 的结构文件呢？与此同时，在 ABACUS 计算完成之后，我们有没有将计算结果转化为可被常见的晶体结构建模可视化软件导入并可视化的常见结构文件格式（如 CIF）的方法？

### 3.1.2 基于 ASE-ABACUS 快速准备 STRU 文件

ASE（Atomic Simulation Environment）是开源的原子尺度模拟 Python 工具库，由丹麦技术大学（DTU）等机构开发，广泛应用于计算材料科学和分子模拟领域。该工具库支持多种第一性原理软件，ABACUS 也有 ASE-ABACUS 接口，不过这一接口是独立于 ASE 主仓库以外的分支，需要单独下载安装。安装方式：

```
# install ase-abacus
git clone https://gitlab.com/1041176461/ase-abacus.git
cd ase-abacus
pip install .
# use tsinghua mirror in CN internet if needed
# pip install . -i https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

若是直接通过 pip install ase，可以下载到 PyPi 上官方的 ASE，但这个 ASE 里没有接入 abacus 的部分支持。ASE-ABACUS 是 ASE 的一个派生分支，专用于 ASE 的 ABACUS 支持，同时兼容 ASE 本体。安装好了 ASE-ABACUS 之后就可以使用这个 ASE 来准备 STRU 文件了。有两种可行方式，一种是直接基于 ASE 构建结构文件并导出为 STRU 格式。比如以下 Python 脚本（或 IPython 命令行）：

```
from ase.build import bulk
from ase.io import write

structure = bulk("Al", "fcc")
write("STRU", structure, format='abacus')
```

如此即可得到 FCC Al 的一个最简单的 STRU 文件，它是 FCC Al 的原胞。

### 3.1.3 STRU 文件的基本结构

通过上述最简单的方式准备的 STRU 文件内容如下：

```
ATOMIC_SPECIES
Al  26.9815385    Al_ONCV_PBE-1.0.upf

NUMERICAL_ORBITAL
Al_gga_7au_100Ry_4s4p1d.orb

LATTICE_CONSTANT
1.8897261258369282

LATTICE_VECTORS
0.0000000000      2.0250000000      2.0250000000
2.0250000000      0.0000000000      2.0250000000
2.0250000000      2.0250000000      0.0000000000

ATOMIC_POSITIONS
Direct

Al
0.0000000000
1
0.0000000000 0.0000000000 0.0000000000 1 1 1 mag 0.0
```

可以看到一个基础的 STRU 文件包括如下内容

- 元素类别信息。包括元素符号，摩尔质量，赝势文件名称。
- 原子轨道文件名称。在使用原子轨道基组计算时使用。
- 晶格常数单位。ABACUS 的默认长度单位为 Bohr，这一晶格常数将所有长度放大 1.889726 倍，将长度单位 Bohr 转化为更为人熟知的埃（Angstrom）
- 晶格矢量，为九分量形式。
- 原子坐标类别：有多种选项，可以在docs:atomic-positions处查看，常见选项包括分数坐标 Direct，笛卡尔坐标 Cartesian。其他选项比如 Cartesian_angstrom 选项可以避免对晶格坐标单位 LATTICE_CONSTANT 的显式指定，直接以埃为单位，不过这一选项在各个接口软件中支持的不多，也不广为人知。
- 每个元素的原子坐标。具体来说：

```
Al   # 元素类别
0.0000000000  # 该元素每个原子的磁矩（低优先级）
1               # 该元素包含的原子数目
0.0000000000 0.0000000000 0.0000000000 1 1 1 mag 0.0
# 坐标 x,y,z; 原子在 x,y,z 方向是否可动（1 代表可动），原子磁矩（高优先级）
```

对于不同计算任务，STRU 文件的原子坐标部分会略有不同，比如 nspin=4 时磁矩会有三个分量，跑 MD 时此 STRU 文件会有每个原子的速度信息。该 STRU 文件的赝势，轨道文件名称来自于ABACUS 官网上推荐的 SG15-v1.0 赝势和对应的第二代数值原子轨道，是 ASE-ABACUS 输出 STRU 时的默认值。

### 3.1.4 基于 ASE-ABACUS 从其他结构文件准备 STRU

除了基于 ASE 构建的结构准备 STRU 外，更常见的是，用户已经有自己想做计算的输入结构，只是要把它转化为 STRU 格式，这一点 ASE-ABACUS 当然也支持，以如下 FCC Al 的 CIF 文件为例：

```
data_image0
_chemical_formula_structural       Al
_chemical_formula_sum              "Al1"
_cell_length_a       2.8637824638055176
_cell_length_b       2.8637824638055176
_cell_length_c       2.8637824638055176
_cell_angle_alpha    60.00000000000001
_cell_angle_beta     60.00000000000001
_cell_angle_gamma    60.00000000000001

_space_group_name_H-M_alt    "P 1"
_space_group_IT_number       1

loop_
  _space_group_symop_operation_xyz
  'x, y, z'

loop_
  _atom_site_type_symbol
  _atom_site_label
  _atom_site_symmetry_multiplicity
  _atom_site_fract_x
  _atom_site_fract_y
  _atom_site_fract_z
  _atom_site_occupancy
  Al  Al1       1.0  0.0  0.0  0.0  1.0000
```

比较简单的 Python 操作方式如下：

```
from ase.io import read, write
# read existing cif file
Al_fcc = read("Al.cif", format="cif")
# write this structure to STRU
write("STRU", Al_fcc, format='abacus')
```

如果想自行指定输出的赝势文件和轨道文件，可以这么写你的 Python 脚本：

```
from ase.io import read, write
Al_fcc = read("Al.cif", format="cif")
# specify pseudopot from APNS-v1.0
pp = {"Al": "Al.upf"}
# specify orbital from APNS-v1.0
basis = {"Al": "Al_gga_8au_100Ry_2s2p1d.orb"}
write("STRU", Al_fcc, format='abacus', pp=pp, basis=basis)
```

还可以通过设置系统环境变量，让 ASE-ABACUS 自动识别你想指定的赝势文件和轨道文件所在路径。以下设置需要写在服务器账户的 /.bashrc 下

```
# Set up the environment for ABACUS
PP=${HOME}/pseudopotentials  # 赝势文件路径
ORB=${HOME}/orbitals  # 轨道文件路径
export ABACUS_PP_PATH=${PP}  # 设置环境变量
export ABACUS_ORBITAL_PATH=${ORB}  # 设置环境变量
```

### 3.1.5 基于 ATOMKIT 准备 STRU 文件

基于 ASE-ABACUS 准备 STRU 文件总还是有它不方便的地方，要么要自行写 Python 脚本并运行这一脚本，要么要安装 IPython 环境才能以命令行交互形式进行 STRU 文件准备。有没有更便捷的方式呢?

ATOMKIT 由 VASPKIT 开发团队开发，提供了另一种选项：通过命令行交互方式，集成晶体结构操作、建模、格式转化及可视化，并可输出多种常见材料模拟软件结构文件格式。同时，ATOMKIT 兼具一定的可视化功能。其最新版可在此处下载：ATOMKIT 0.9.0 新版发布: 新增可视化界面

下载好了之后，直接解压，然后安装即可，如:

```
# under target directory
tar -zxvf atomkit.0.9.0.linux.x64.tar.gz
cd atomkit.0.9.0.linux.x64
bash setup.sh
# reload your system environment
source ~/.bashrc
```

如此便可通过 atomkit 命令启动 ATOMKIT:

```
~> atomkit
           \\\///
          / _  _ \
        (| (o)(o) |)
o-----.OOOo--()--oOOO.----------------------------------------o
|         ATOMKIT Standard Edition 0.9.0 (26 Mar. 2024)        |
|         Lead Developer: Vei WANG (wangvei@icloud.com)        |
|      Contributors: Zhao-Ke WANG, Gang TANG, Ya-Chao LIU      |
o-----.oooO----------------------------------------------------o
       (   )   Oooo.                          ATOMKIT Made Simple
        \ (    (   )
         \_)    ) /
               (_/
===================== Structural Utilities =====================
1) Export Structures             2) Symmetry Analysis
3) Generate K-Mesh & K-Path      4) Edit Structure
5) Advanced Structure Models     6) 2D-Material Kit
8) Visualize Structure           0) Quit
------------>>
```

输入 1，即可基于已有结构输出各种格式的结构:

```
1
 +---------------------------- Tip -----------------------------+
 |   Supplementary parameters from GLOBAL_PARAMETERS.in will be  |
 |    automatically copied to the exported file, if available.   |
 +---------------------------------------------------------------+
 ==================== Export Format Options ====================
 101) ABACUS (*.stru)             102) ABINIT (*.in)
 105) ASE (*.py)                  107) ATAT (*.in)
 109) AtomEye (*.cfg)             112) CASTEP (*.cell)
 113) CIF Format (*.cif)          114) CP2K (*.inp)
 115) CPMD (*.inp)                123) Elk (*.in)
 124) exciting (*.xml)            126) DFTB+ (*.hsd)
 127) DMol3 (*.incoor)            129) FHI-aims (*.in)
 132) FLEUR (*.inp)               134) Gaussian (*.gjf)
 136) LAMMPS (*.lmp)              138) JDFTx (*.in)
 142) Octopus (*.in)              144) OpenMX (*.in)
 146) ORCA (*.in)                 149) PDB Format (*.pdb)
 156) Quantum-Espresso (*.in)     165) RSPt (*.inp)
 168) Siesta (*.fdf)              175) VASP (*.vasp)
 180) Wannier90 (*.win)           182) WIEN2K (*.struct)
 185) XCrySDen (*.xsf)            189) XYZ Format (*.xyz)

 0)   Quit
 9)   Back
 ------------>>
```

可以看到 ABACUS 的 STRU 格式对应命令 101，CIF 格式对应命令 113，VSAP 的 POSCAR 格式对应命令 175。输入 101 即为准备 ABACUS 的 STRU 文件：

```
101

 ==================== Import Format Options ====================
 101) ABACUS (STRU)               112) CASTEP (*.cell)
 113) CIF Format (*.cif)          123) Elk (*.in)
 134) Gaussian (*.gjf)            142) Materials Studio (*.xsd)
 144) MDL Molfile Format (*.mol)  149) PDB Format (*.pdb)
 156) Quantum-Espresso (*.in)     168) Siesta (*.fdf)
 175) VASP (POSCAR, *.vasp)       185) XCrySDen (*.xsf)
 189) XYZ Format (*.xyz)
 190) Build Crystal from Wyckoff Positions & Space Group Number

 +---------------------------- Tip -----------------------------+
 | Examples of Usage:                                            |
 | 113 Si.cif: Read structure from Si.cif file with cif format   |
 | 175 POSCAR: Read structure from POSCAR file with VASP format  |
 | 190 INPUT:  Build crystal From Wyckoff positions in INPUT file|
 +---------------------------------------------------------------+
 Input the code-ID listed above and the filename to be read:
```

然后，按照提示加载我们已有的结构文件。假设当前路径下已有结构文件是 Al.cif，如下输入即可：

```
113 Al.cif
 -->> (01) Reading Structure from Al.cif File...
 +------------------------------------------------------------------+
 | This utility is in experimental stage & needs to be improved. |
 +------------------------------------------------------------------+
 +------------------------------------------------------------------+
  Species and Wyckoff Sites
  [Al]  0.000  0.000  0.000
 +------------------------------------------------------------------+
 |      Cell Parameters After Applying Symmetry Operations       |
 +------------------------------------------------------------------+
     Import File Format:   CIF
      Lattice Constants:   2.864   2.864   2.864
         Lattice Angles:  60.000  60.000  60.000
            Total Atoms:   1
        Number of Atoms:  [Al]   1
 +------------------------------------------------------------------+
 -->> (02) Written Al.STRU File!
 o------------------------------------------------------------------o
 |      WARNING: Your are using a beta version of ATOMKIT        |
 |   Please Upgrade to the latest stable version if available    |
 |  Any Suggestions for Improvement are Welcome and Appreciated  |
 o------------------------------------------------------------------o
```

可以看到 ATOMKIT 自动读取了 Al.cif 文件，并将其转化为了 ABACUS 的 STRU 格式文件 Al.STRU，这一文件内容为：

```
ATOMIC_SPECIES
Al   26.982  Al_ONCV_PBE-1.0.upf

NUMERICAL_ORBITAL
Al_gga_7au_100Ry_4s4p1d.orb

LATTICE_CONSTANT
1.889726

LATTICE_VECTORS
2.863782463806  0.000000000000  0.000000000000
1.431891231903  2.480108364568  0.000000000000
1.431891231903  0.826702788189  2.338268590218

ATOMIC_POSITIONS
Direct

Al
0.000
1
```

```
0.000000000000  0.000000000000  0.000000000000  1  1  1  mag  0.0
```

读者可自行思考为什么 LATTICE_VECTORS 看起来和之前的 STRU 文件不同，这里能给出的结论是二者等价。

ATOMKIT 有非常丰富的结构操作功能，比如对原子按特定方向排序，按特定方向分层固定原子，交换晶体的晶格向量等，这里不再展开，读者可以自行探索。

### 3.1.6 将 STRU 文件转化为其他格式

很多时候我们也需要把 STRU 文件转化为其他结构格式（如 CIF），以便于在通用的结构可视化软件中导入。使用 ASE-ABACUS 进行转化的 Python 代码如下：

```
from ase.io import read, write
# read existing STRU file
structure = read("STRU", format="abacus")
# write this structure to CIF
write("my_stru.cif", stru, format='cif')
```

而使用 ATOMKIT 时，操作也和准备 STRU 文件类似，只是导出格式选项要输入 CIF 对应的 113，输入文件选项要输入对应的 101 STRU。同时，ATOMKIT 还支持另一种使用方式，用户可以把需要在 ATOMKIT 中输入的命令写在一个文本文件中，如在 example.txt 文件内输入如下内容：

```
1
113
101 STRU
```

然后通过如下方式

```
atomkit < example.txt
```

即可一次性完成所有 atomkit 操作。

### 3.1.7 基于 abacustest 准备 ABACUS 输入文件

除去上述两种方式以外，还有一个工具 abacustest 也能用来准备 ABACUS 的 STRU 等输入文件。abacustest用于对 ABACUS 进行前后处理，以及高通量任务的计算。目前 abacustest 有两种安装方式：通过 pip 从 PyPi 源安装，或者是 git 拉取仓库后源码安装：

```
# install via pip
pip install abacustest
# install via git+pip
git clone https://github.com/pxlxingliang/abacus-test.git
cd abacus-test
pip install .
```

abacustest 可以用来高通量地准备 ABACUS 的输入任务，高通量地基于 Bohrium 提交任务，或是高通量地收集整理 ABACUS 计算结果。通过 abacustest -h，用户可以了解到如何使用这一工具的各个功能。这里仅教学如何准备 STRU 等输入文件。用户需要准备类似于下面的 param.json 文件：

```
{
  "prepare":{
    "strus": ["1.vasp", "2.vasp"],
    "stru_format": "poscar",
    "input_template": "INPUT",
    "kpt_template": "KPT",
    "pp_dict": {"H": "H.upf", "O": "O.upf"},
    "orb_dict": {"H": "H.orb", "O": "O.orb"},
    "pp_path": "path/to/pp",
    "orb_path": "path/to/orb"
  }
}
```

其中，INPUT 模板，KPT 模板，赝势和轨道的路径等都不是必须指定的，但指定好有利于一次准备好 ABACUS 输入。然后，按照如下方式：

```
abacustest prepare -p param.json -s abacustest
```

即可在 abacustest 文件夹下准备好一系列 ABACUS 输入文件，包含转换好的结构文件 STRU。abacustest 支持对 POSCAR 和 CIF 等结构格式的转换。

以上内容讲解了 ABACUS 结构文件 STRU 的格式，以及通过 ASE-ABACUS，ATOMKIT 和 abacustest 准备 ABACUS 的 STRU 文件的方法，同时讲解了如何将 STRU 文件转化为其他结构文件。希望此教程能让用户更快了解 ABACUS，并上手 ABACUS 输入建模和计算！

## 3.2 ABACUS 如何选择晶胞朝向获得最佳并行效率？以碳纳米管为例

### 3.2.1 背景

不少用户会问当准备的 ABACUS 算例在 XYZ 三个方向的大小不一致时，摆放晶胞会不会对结果产生影响？如果会影响，应该怎么选取摆放的方向？相关的 Github Issue 有：https://github.com/deepmodeling/abacus-develop/issues/3015。为此，我们这里给了一个包含 1000 个碳原子的一维碳纳米管的测试。算例来自：https://github.com/MCresearch/TEAS/tree/main/2022-PRB-CNT/1_DHU/0.01。下面我们分别将真空方向设置为 X、Y、Z 方向，比较计算结果能量和时间来判断真空方向对计算的影响。

### 3.2.2 ABACUS 做不同真空方向计算的流程

本文算例的碳纳米管在一个方向的长度明显大于另外两个方向，我们会采用原子弛豫计算 (ABACUS 中设置 calculation 为 relax，只让原子移动而固定晶格常数)。改变真空层的方向通过对 STRU 和 KPT 文件的三列进行轮换实现。我们先来看看为 ABACUS 计算准备的 INPUT 和 KPT 输入文件。

**INPUT 文件**

需要注意的是，上文给出的算例链接中的 INPUT 文件参数属于 ABACUS 老版本，下面已更新为 v3.8.0 的参数名称，输入参数关键词可以参考：Full List of INPUT Keywords #

```
INPUT_PARAMETERS
#Parameters (1.General)
```

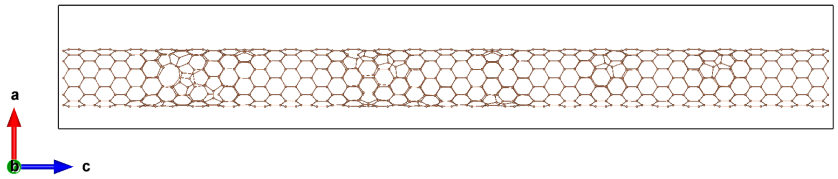
```
suffix              CNT_0.01
calculation         relax
nbands              2100
symmetry            0

#Parameters (2.Iteration)
ecutwfc             100
scf_thr             1e-6
scf_nmax            400

#Parameters (3.Basis)
basis_type          lcao
ks_solver           genelpa
gamma_only          1

#Parameters (4.Smearing)
smearing_method      gaussian
smearing_sigma       0.001

#Parameters (5.Mixing)
mixing_type          pulay
mixing_beta          0.1
mixing_ndim          20

cal_force            1
cal_stress           1
relax_nmax           1000
force_thr_ev         0.04
#move_method         cg
out_stru             1
```

**KPT 文件**

```
K_POINTS
0
Gamma
1 1 1 0 0 0
```

**STRU 文件**

一共有 1000 个 C 原子，STRU 文件较长所以不在此处展示，可以在 https://github.com/MCresearch/TEAS/blob/main/2022-PRB-CNT/1_DHU/0.01/STRU 中查看，结构如下图。




我们一起看看 STRU 文件中的 LATTICE_VECTORS，这是一个 3×3 矩阵，代表我们所计算体系在 X，Y，Z 三个方向上的晶格矢量大小，还要注意晶格矢量由 LATTICE_CONSTANT（晶格常数）进行缩放，此处晶格常数为 1 Bohr。可以参考：https://abacus.deepmodeling.com/en/latest/advanced/input_files/stru.html#lattice-vectors

```
LATTICE_CONSTANT
1
LATTICE_VECTORS
32.1253418058417  0.00000000000000  0.000000000000000
0.00000000000000  34.0150677944206  0.000000000000000
0.00000000000000  0.00000000000000  202.0428810989945
```

可以观察到此时位于矩阵 (3,3) 的元素值远大于 (1,1) 和 (2,2) 处的值，即此时长边在 Z 方向。我们要做的就是把长边分别放在不同的方向进行计算，比较效率。

放在 X 方向时：

```
LATTICE_CONSTANT
1
LATTICE_VECTORS
202.0428810989945  0.00000000000000  0.00000000000000
0.000000000000000  32.1253418058417  0.00000000000000
0.000000000000000  0.00000000000000  34.0150677944206
```

放在 Y 方向时：

```
LATTICE_CONSTANT
1
LATTICE_VECTORS
34.0150677944206  0.000000000000000  0.00000000000000
0.00000000000000  202.0428810989945  0.00000000000000
0.00000000000000  0.000000000000000  32.1253418058417
```

原子坐标的轮换也是相同的规则。

由于布里渊区的 K 点采样选择了 Gamma 点，故 KPT 文件轮换后结果不改变，测试中只轮换了 STRU 文件中的晶格矢量 (LATTICE_VECTORS) 和原子坐标 (ATOMIC_POSITIONS) 两部分来改变晶胞摆放。如果读者在做其他需要变换真空层方向的计算时，千万不要忘记当晶格矢量方向改变时，K 点也要改变！

### 3.2.3 测试结果

我们选择了 64 个型号为 8426Y+ 的 CPU 用 64 个进程对该一维碳纳米管结构进行结构弛豫。下表给出本次测试的总能量，将碳纳米管摆放在 XYZ 三个方向下做结构弛豫的步数都是 212 步。我们通过比较总能量和时间

差异来判断真空层方向对计算结果的影响，可以看到系统总能量并不会受到不同碳纳米管取向的影响，但计算时间会受到显著影响。虽然，但是真空层摆放在 X 方向时，计算时间明显慢于摆放在 Z 方向，每一电子步大概慢 8s 左右。

| 碳纳米管长边朝向 | CPU 型号 | CPU 个数进程个数 | 系统总能量 (eV) | 计算总时间 (s) |
|---|---|---|---|---|
| 计算总驰豫步数 | 运行一电子步的时间 (s) | FFT | UNIFORM GRID DIM(BIG) | |
| X | 8462Y+ | 64 核 64 进程 | -154919.0390601 | 64114 |
| 212 | 18 18.6 | 1296*216*225 | 324*54*45 | |
| Y | 8462Y+ | 64 核 64 进程 | -154919.0390599 | 56846 |
| 212 | 15.5 16 | 225*1296*216 | 45*324*54 | |
| Z | 8462Y+ | 64 核 64 进程 | -154919.0390613 | 40630 |
| 212 | 10 10.5 | 216*225*1296 | 54*45*432 | |

根据以上测试结果，建议读者们在进行真空层计算时，将碳纳米管的长边摆放在 Z 方向以获得 ABACUS 计算时的最大效率。

## 3.3 ABACUS 如何选择晶胞朝向获得最佳并行效率？以二维氮化硼为例

### 3.3.1 背景

我们已经测试过了一维材料的长边分别摆放在 XYZ 三个方向时，对计算效率的影响，并通过测试结果给出了建议，参考 ABACUS 如何选择晶胞朝向获得最佳并行效率? 以碳纳米管为例 · GitBook。现在我们考虑一个二维六方氮化硼体系，即材料在两个方向的长度明显大于第三个方向。

下面我们分别将短边摆放在 X、Y、Z 方向，比较计算结果能量和时间来判断短边摆放方式对计算的影响。

### 3.3.2 ABACUS 计算的流程

本文算例的氮化硼在一个方向的长度明显小于另外两个方向，我们采用原子弛豫计算 (ABACUS 中设置 calculation 为 relax，只让原子移动而固定晶格常数)。同样，通过对 STRU 和 KPT 文件的三列进行轮换改变摆放方式。我们先来看看为 ABACUS 计算准备的 INPUT 、STRU 和 KPT 三个输入文件。

**INPUT 文件**

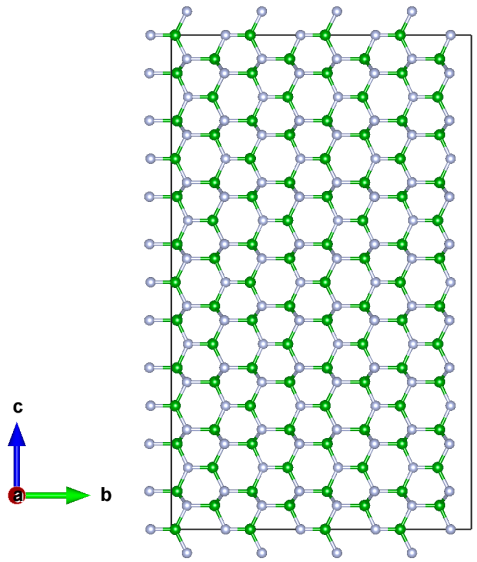
可以注意到，在本次计算中，我们使用了 kspacing 这个参数（Full List of INPUT Keywords —ABACUS documentation），可以根据晶胞大小自动生成 KPT 文件。因此不再需要我们手动调换 K 点的顺序。

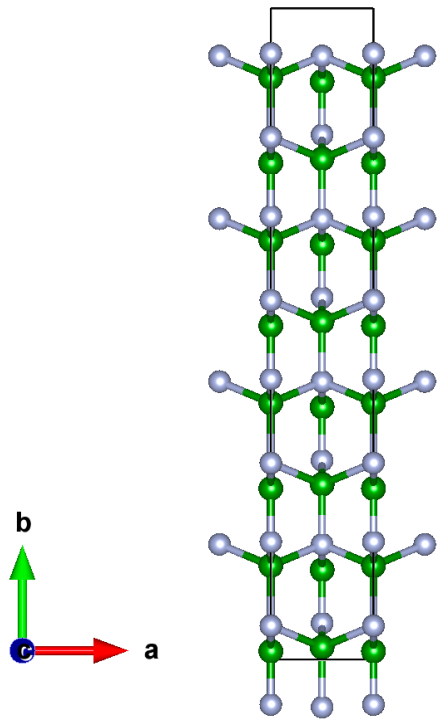
```
INPUT_PARAMETERS
suffix                  BN
pseudo_dir              ./
orbital_dir             ./
calculation             cell-relax
symmetry                0
basis_type              lcao
ecutwfc                 100
scf_thr                 1e-07
scf_nmax                100
smearing_method         gauss
smearing_sigma          0.002
```

```
mixing_type             pulay
mixing_beta             0.3
cal_force               1
cal_stress              1
force_thr_ev            0.01
stress_thr              0.5
relax_nmax              100
out_stru                1
kspacing                0.08
```

**STRU 文件**

在本体系中，B 和 N 原子各有 96 个，h-BN 结构图如下。






以发现，此时短边在晶胞的 X 方向上。

```
ATOMIC_SPECIES
B    10.81          B_ONCV_PBE-1.0.upf
N    14.007         N_ONCV_PBE-1.0.upf

NUMERICAL_ORBITAL
B_gga_8au_100Ry_2s2p1d.orb
N_gga_8au_100Ry_2s2p1d.orb

LATTICE_CONSTANT
1.8897261258369282

LATTICE_VECTORS
2.6665300000       0.0000000000       0.0000000000
0.0000000000       16.970664000       0.0000000000
0.0000000000       0.0000000000       28.020088000

ATOMIC_POSITIONS
Direct
B
0.0000000000
96
0.0000000000 0.0126740000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.0768190000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.0481810000 1 1 1 mag 0.0
```

```
0.5000000000 0.1376740000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.2018190000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.1731810000 1 1 1 mag 0.0
0.0000000000 0.2626740000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.0768190000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.0481810000 1 1 1 mag 0.0
0.5000000000 0.3876740000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.2018190000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.1731810000 1 1 1 mag 0.0
0.0000000000 0.5126740000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.0768190000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.0481810000 1 1 1 mag 0.0
0.5000000000 0.6376740000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.2018190000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.1731810000 1 1 1 mag 0.0
0.0000000000 0.7626740000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.0768190000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.0481810000 1 1 1 mag 0.0
0.5000000000 0.8876740000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.2018190000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.1731810000 1 1 1 mag 0.0
0.0000000000 0.0126740000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.3268190000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.2981810000 1 1 1 mag 0.0
0.5000000000 0.1376740000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.4518190000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.4231810000 1 1 1 mag 0.0
0.0000000000 0.2626740000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.3268190000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.2981810000 1 1 1 mag 0.0
0.5000000000 0.3876740000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.4518190000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.4231810000 1 1 1 mag 0.0
0.0000000000 0.5126740000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.3268190000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.2981810000 1 1 1 mag 0.0
0.5000000000 0.6376740000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.4518190000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.4231810000 1 1 1 mag 0.0
0.0000000000 0.7626740000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.3268190000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.2981810000 1 1 1 mag 0.0
0.5000000000 0.8876740000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.4518190000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.4231810000 1 1 1 mag 0.0
0.0000000000 0.0126740000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.5768190000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.5481810000 1 1 1 mag 0.0
```

```
0.5000000000 0.1376740000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.7018190000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.6731810000 1 1 1 mag 0.0
0.0000000000 0.2626740000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.5768190000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.5481810000 1 1 1 mag 0.0
0.5000000000 0.3876740000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.7018190000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.6731810000 1 1 1 mag 0.0
0.0000000000 0.5126740000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.5768190000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.5481810000 1 1 1 mag 0.0
0.5000000000 0.6376740000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.7018190000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.6731810000 1 1 1 mag 0.0
0.0000000000 0.7626740000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.5768190000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.5481810000 1 1 1 mag 0.0
0.5000000000 0.8876740000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.7018190000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.6731810000 1 1 1 mag 0.0
0.0000000000 0.0126740000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.8268190000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.7981810000 1 1 1 mag 0.0
0.5000000000 0.1376740000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.1439860000 0.9518190000 1 1 1 mag 0.0
0.5000000000 0.0189860000 0.9231810000 1 1 1 mag 0.0
0.0000000000 0.2626740000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.8268190000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.7981810000 1 1 1 mag 0.0
0.5000000000 0.3876740000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.3939860000 0.9518190000 1 1 1 mag 0.0
0.5000000000 0.2689860000 0.9231810000 1 1 1 mag 0.0
0.0000000000 0.5126740000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.8268190000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.7981810000 1 1 1 mag 0.0
0.5000000000 0.6376740000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.6439860000 0.9518190000 1 1 1 mag 0.0
0.5000000000 0.5189860000 0.9231810000 1 1 1 mag 0.0
0.0000000000 0.7626740000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.8268190000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.7981810000 1 1 1 mag 0.0
0.5000000000 0.8876740000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.8939860000 0.9518190000 1 1 1 mag 0.0
0.5000000000 0.7689860000 0.9231810000 1 1 1 mag 0.0

N
0.0000000000
```

```
96
0.0000000000 0.0518080000 0.0478800000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.0771200000 1 1 1 mag 0.0
0.0000000000 0.1807370000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.1728800000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.2021200000 1 1 1 mag 0.0
0.5000000000 0.0557370000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.0478800000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.0771200000 1 1 1 mag 0.0
0.0000000000 0.4307370000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.1728800000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.2021200000 1 1 1 mag 0.0
0.5000000000 0.3057370000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.0478800000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.0771200000 1 1 1 mag 0.0
0.0000000000 0.6807370000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.1728800000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.2021200000 1 1 1 mag 0.0
0.5000000000 0.5557370000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.0478800000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.0771200000 1 1 1 mag 0.0
0.0000000000 0.9307370000 0.0000000000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.1728800000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.2021200000 1 1 1 mag 0.0
0.5000000000 0.8057370000 0.1250000000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.2978800000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.3271200000 1 1 1 mag 0.0
0.0000000000 0.1807370000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.4228800000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.4521200000 1 1 1 mag 0.0
0.5000000000 0.0557370000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.2978800000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.3271200000 1 1 1 mag 0.0
0.0000000000 0.4307370000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.4228800000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.4521200000 1 1 1 mag 0.0
0.5000000000 0.3057370000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.2978800000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.3271200000 1 1 1 mag 0.0
0.0000000000 0.6807370000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.4228800000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.4521200000 1 1 1 mag 0.0
0.5000000000 0.5557370000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.2978800000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.3271200000 1 1 1 mag 0.0
0.0000000000 0.9307370000 0.2500000000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.4228800000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.4521200000 1 1 1 mag 0.0
```

```
0.5000000000 0.8057370000 0.3750000000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.5478800000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.5771200000 1 1 1 mag 0.0
0.0000000000 0.1807370000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.6728800000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.7021200000 1 1 1 mag 0.0
0.5000000000 0.0557370000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.5478800000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.5771200000 1 1 1 mag 0.0
0.0000000000 0.4307370000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.6728800000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.7021200000 1 1 1 mag 0.0
0.5000000000 0.3057370000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.5478800000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.5771200000 1 1 1 mag 0.0
0.0000000000 0.6807370000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.6728800000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.7021200000 1 1 1 mag 0.0
0.5000000000 0.5557370000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.5478800000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.5771200000 1 1 1 mag 0.0
0.0000000000 0.9307370000 0.5000000000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.6728800000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.7021200000 1 1 1 mag 0.0
0.5000000000 0.8057370000 0.6250000000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.7978800000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.8271200000 1 1 1 mag 0.0
0.0000000000 0.1807370000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.1768080000 0.9228800000 1 1 1 mag 0.0
0.0000000000 0.0518080000 0.9521200000 1 1 1 mag 0.0
0.5000000000 0.0557370000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.7978800000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.8271200000 1 1 1 mag 0.0
0.0000000000 0.4307370000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.4268080000 0.9228800000 1 1 1 mag 0.0
0.0000000000 0.3018080000 0.9521200000 1 1 1 mag 0.0
0.5000000000 0.3057370000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.7978800000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.8271200000 1 1 1 mag 0.0
0.0000000000 0.6807370000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.6768080000 0.9228800000 1 1 1 mag 0.0
0.0000000000 0.5518080000 0.9521200000 1 1 1 mag 0.0
0.5000000000 0.5557370000 0.8750000000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.7978800000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.8271200000 1 1 1 mag 0.0
0.0000000000 0.9307370000 0.7500000000 1 1 1 mag 0.0
0.5000000000 0.9268080000 0.9228800000 1 1 1 mag 0.0
0.0000000000 0.8018080000 0.9521200000 1 1 1 mag 0.0
```

```
217 0.5000000000 0.8057370000 0.8750000000 1 1 1 mag 0.0
```

改变摆放方式时，要同时轮换 LATTICE_VECTORS 和 ATOMIC_POSITIONS 的顺序。

**KPT 文件**

```
1 K_POINTS
2 0
3 Gamma
4 16 3 2 0 0 0
```

如前所述，该 KPT 文件是由 INPUT 文件里的 kspacing 参数所生成，因此不需要我们手动轮换，每次计算都会自动根据晶胞大小产生合适的 K 点。

### 3.3.3 测试结果

计算采用 ABAUCS v3.8.0 版本，选择 64 个型号为 8426Y+ 的 CPU 用 64 个进程对该二维氮化硼体系进行结构弛豫。下表给出本次测试的结果，将氮化硼的短边分别摆放在 XYZ 三个方向完成结构弛豫的步数都是 9 步。我们通过比较总能量和时间差异来判断摆放方式对计算结果的影响，可以看到系统总能量并不会受到短边取向的影响，但计算时间会受到显著影响。短边摆放在 Z 方向时，即两长边分别放在 X、Y 方向，计算时间明显慢于长边摆放在 Y、Z 和 Z、X 方向，每一电子步大概慢 6 s 左右。（注意：我们进行一维材料计算时，也建议将长边摆放在 Z 方向以提高计算效率）

| h-BN 的两个长边方向 | 系统总能量 (eV) | 计算总时间 (s) | 计算总驰豫步数 | 运行一电子步的时间 (s) |
|---|---|---|---|---|
| FFT | UNIFORMGRID DIM(BIG) | | | |
| XY | -33578.4871085887 | 2140 | 9 | 24.8 26.8 |
| 216*360*36 | 54*72*18 | | | |
| YZ | -33578.4871085889 | 1609 | 9 | 18.7 20.0 |
| 36*216*360 | 9*54*120 | | | |
| ZX | -33578.4871085888 | 1629 | 9 | 18.9 20.9 |
| 360*36*216 | 72*9*54 | | | |

根据以上测试结果，建议读者们在进行二维体系计算时，将短边摆放在 X 或 Y 方向以获得 ABACUS 计算时的最大效率。

## 3.4 ABACUS 如何选择晶胞朝向获得最佳并行效率？以铜表面一氧化碳吸附为例

### 3.4.1 背景

不少用户会问当准备的 ABACUS 算例在 XYZ 三个方向的大小不一致时，摆放晶胞会不会对结果产生影响？如果会影响，应该怎么选取摆放的方向？相关的 Github Issue 有：abacus/issues/3015。此前已有针对一维材料和（不含真空层的）二维材料其不同晶胞朝向的计算效率测试，并通过测试结果给出了最佳计算效率方案，但这两种情况并没有囊括所有的用户需求，其中一个非常重要的需求是表面体系和表面反应体系。这类体系虽然和二维材料体系一样是二维体系，但其在垂直于表面的方向的晶格长边（一般为 Z 轴方向）是具有不含有任何原子的真空层的，这一真空层厚度一般在 10-15 Angstrom，从而使得真空层方向显著长于另外两个表面延展方向。

下面我们分别将真空方向设置为 X、Y、Z 方向，比较计算结果能量和时间来判断真空方向对计算的影响。

值得强调的是，后续晶胞构建，ABACUS 计算和结果提取全部由 ABACUS 智能体完成，这一智能体以GaliLeo科学智能体平台为智能体平台，以ABACUS-agent-tools中的智能体工具实现为基础构建。GaliLeo 科学智能体平台聚焦“自然语言 + 科研计算”，用户可在平台网页通过自然语言向智能体 ADAM 提出需求，如“计算蛋白–小分子结合自由能”、“计算分子在表面的吸附能”等，ADAM 将自动解析 → 查询 ADAM-KB 知识库 → 调用 HPC/云原生科学计算引擎 → 返回科学解读与可视化报告，实现从需求到结果的全流程自动化。GaliLeo 已在理论计算化学、计算生物学、计算材料学、AI for Science 与实验-计算交互等领域成功落地，其社区已有 300+ 用户，涵盖 70 余家高校与科研机构的硕博科研人员，对平台交互体验与计算效果高度认可。欢迎访问sidereus-ai.com，关注微信公众号“星使智算”或发送邮件至 sidereus_ai@163.com，共创科研智能体未来！

### 3.4.2 ABACUS 做不同真空方向计算的流程

本文算例的 Cu(111) 表面吸附 CO 结构在真空层方向的长度明显大于另外两个方向，我们会采用默认的单单点电子结构计算（ABACUS 中设置 calculation 为 scf）。之前的前两个测试案例和我个人对表面体系的计算经验表明，不同晶胞朝向对原子驰豫步数（即离子步步数）和优化收敛结构并没有影响，因而选用 scf 而非结构优化（原子驰豫）计算 relax 即可体现计算效率差距。

在此前的教程中，改变真空层的方向通过对 STRU 和 KPT 文件的三列进行手动轮换实现，但这一做法其实只适合对称性较高的晶胞，并且需要对分数坐标操作。这种晶胞朝向改变本质上是晶胞自然坐标系 a, b, c 方向的轮换，可以通过手写旋转矩阵的方式来实现，也可以通过已有程序完成，如ATOMKIT中的 405 功能可以便捷、可靠地完成这件事情，基于ASE也可以快速构建相关的代码。

在本教程中，Cu(111) 表面吸附 CO 的结构由 ABACUS 智能体建模，ABACUS 计算所用的 INPUT 文件由 ABACUS 智能体按照用户需求生成，KPT 文件则通过 INPUT 设置的 kspacing 选项在 ABACUS 计算时自动生成，所用 ABACUS 版本为LTS-v3.10.0，并基于 202502 版本的gnu-toolchain安装。计算采用 ABACUS 赝势-轨道库 APNS-PPORB-v1的推荐赝势及其对应效率基组完成。

#### 结构建模和 STRU 文件

本次测试的 Cu(111) 表面吸附 CO 结构由 ABACUS 智能体建模，我们让智能体首先构建了一个 Cu 晶胞，然后针对其 (111) 面进行切面，构造层数为 4 且进行了 (2*2) 扩胞，一共有 64 个原子的 Cu(111) 表面，并添加 15 Angstrom 的真空层，最后在切面上放置了一个 CO 分子。结构构建的对话过程如下：

图 1. 构建 Cu(111)-CO 表面吸附体系

构建完成后，智能体将结构保存为 CIF 文件，其结构如下图所示：

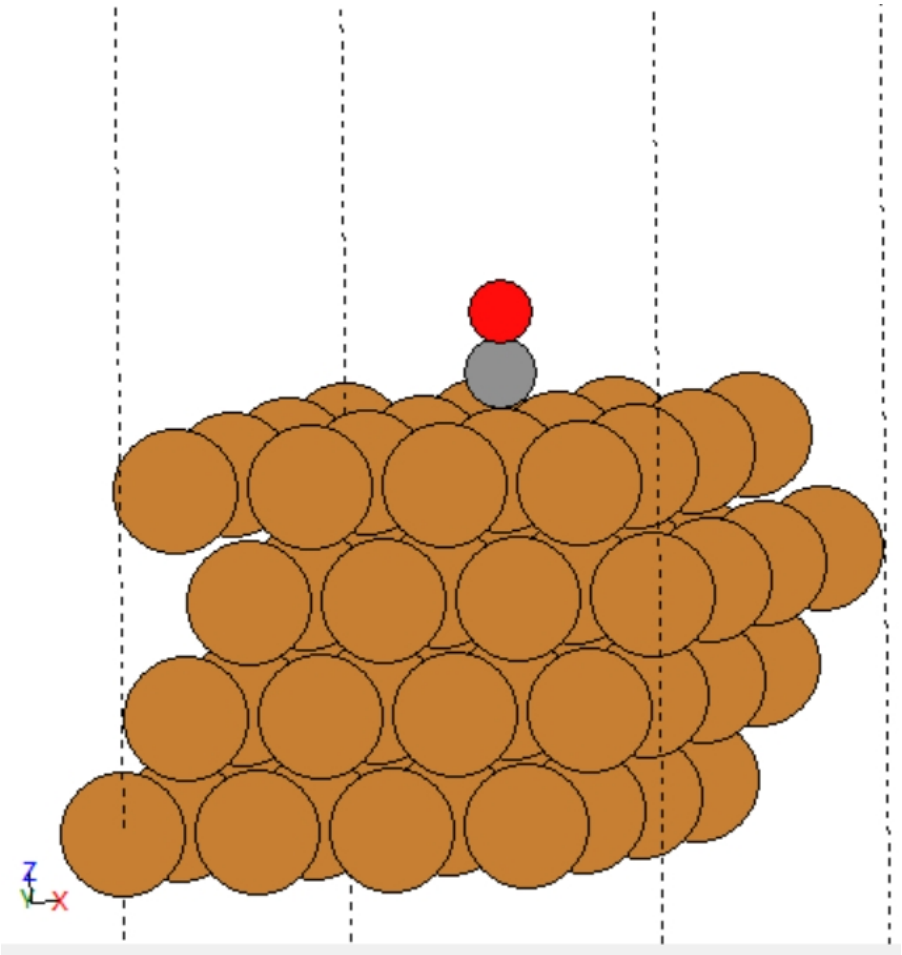



图 2. 目标表面吸附结构可视化

在后续的智能体操作中，这一结构将被转化为 ABACUS 的 STRU 结构文件。这一结构文件内容如下：

```
ATOMIC_SPECIES
Cu 63.546000 Cu_ONCV_PBE-1.0.upf
O 15.999400 O.upf
C 12.010700 C.upf

NUMERICAL_ORBITAL
Cu_gga_9au_150Ry_4s2p2d1f.orb
O_gga_6au_100Ry_2s2p1d.orb
C_gga_8au_100Ry_2s2p1d.orb

LATTICE_CONSTANT
1.889726

LATTICE_VECTORS
   10.21062192033      0.00000000000      0.00000000000
    5.10531096017      8.84265797145      0.00000000000
    0.00000000000      0.00000000000     39.25270341532

ATOMIC_POSITIONS
Cartesian
```

```
21
22 Cu
23 0.000000
24 64
25      0.00000000000      0.00000000000     15.00000000000 1 1 1
26      1.27632774004      2.21066449286     15.00000000000 1 1 1
27      5.10531096017      4.42132898572     15.00000000000 1 1 1
28      3.82898322013      2.21066449286     15.00000000000 1 1 1
29      5.10531096017      2.94755265715     17.08423447178 1 1 1
30      3.82898322013      0.73688816429     17.08423447178 1 1 1
31      2.55265548008      2.94755265715     17.08423447178 1 1 1
32      1.27632774004      0.73688816429     17.08423447178 1 1 1
33      2.55265548008      1.47377632858     19.16846894355 1 1 1
34      3.82898322013      3.68444082144     19.16846894355 1 1 1
35      5.10531096017      1.47377632858     19.16846894355 1 1 1
36      6.38163870021      3.68444082144     19.16846894355 1 1 1
37     12.76327740042      4.42132898572     21.25270341532 1 1 1
38     11.48694966038      2.21066449286     21.25270341532 1 1 1
39      5.10531096017      4.42132898572     21.25270341532 1 1 1
40      3.82898322013      2.21066449286     21.25270341532 1 1 1
41      2.55265548008      4.42132898572     15.00000000000 1 1 1
42      3.82898322013      6.63199347858     15.00000000000 1 1 1
43      2.55265548008      0.00000000000     15.00000000000 1 1 1
44      6.38163870021      6.63199347858     15.00000000000 1 1 1
45      7.65796644025      7.36888164287     17.08423447178 1 1 1
46      6.38163870021      5.15821715001     17.08423447178 1 1 1
47      5.10531096017      7.36888164287     17.08423447178 1 1 1
48      3.82898322013      5.15821715001     17.08423447178 1 1 1
49      5.10531096017      5.89510531430     19.16846894355 1 1 1
50      6.38163870021      8.10576980716     19.16846894355 1 1 1
51      7.65796644025      5.89510531430     19.16846894355 1 1 1
52      8.93429418029      8.10576980716     19.16846894355 1 1 1
53      5.10531096017      8.84265797145     21.25270341532 1 1 1
54      3.82898322013      6.63199347858     21.25270341532 1 1 1
55      7.65796644025      8.84265797145     21.25270341532 1 1 1
56      6.38163870021      6.63199347858     21.25270341532 1 1 1
57      5.10531096017      0.00000000000     15.00000000000 1 1 1
58      6.38163870021      2.21066449286     15.00000000000 1 1 1
59     10.21062192033      4.42132898572     15.00000000000 1 1 1
60      8.93429418029      2.21066449286     15.00000000000 1 1 1
61     10.21062192033      2.94755265715     17.08423447178 1 1 1
62      8.93429418029      0.73688816429     17.08423447178 1 1 1
63      7.65796644025      2.94755265715     17.08423447178 1 1 1
64      6.38163870021      0.73688816429     17.08423447178 1 1 1
65      7.65796644025      1.47377632858     19.16846894355 1 1 1
66      8.93429418029      3.68444082144     19.16846894355 1 1 1
67     10.21062192033      1.47377632858     19.16846894355 1 1 1
68     11.48694966038      3.68444082144     19.16846894355 1 1 1
```

```
    7.65796644025     4.42132898572    21.25270341532 1 1 1
    6.38163870021     2.21066449286    21.25270341532 1 1 1
   10.21062192033     4.42132898572    21.25270341532 1 1 1
    8.93429418029     2.21066449286    21.25270341532 1 1 1
    7.65796644025     4.42132898572    15.00000000000 1 1 1
    8.93429418029     6.63199347858    15.00000000000 1 1 1
    7.65796644025     0.00000000000    15.00000000000 1 1 1
   11.48694966038     6.63199347858    15.00000000000 1 1 1
   12.76327740042     7.36888164287    17.08423447178 1 1 1
   11.48694966038     5.15821715001    17.08423447178 1 1 1
   10.21062192033     7.36888164287    17.08423447178 1 1 1
    8.93429418029     5.15821715001    17.08423447178 1 1 1
   10.21062192033     5.89510531430    19.16846894355 1 1 1
   11.48694966038     8.10576980716    19.16846894355 1 1 1
   12.76327740042     5.89510531430    19.16846894355 1 1 1
   14.03960514046     8.10576980716    19.16846894355 1 1 1
   10.21062192033     8.84265797145    21.25270341532 1 1 1
    8.93429418029     6.63199347858    21.25270341532 1 1 1
   12.76327740042     8.84265797145    21.25270341532 1 1 1
   11.48694966038     6.63199347858    21.25270341532 1 1 1

O
0.000000
1
    7.65796644025     4.42132898572    24.25270341532 1 1 1

C
0.000000
1
    7.65796644025     4.42132898572    23.10236341532 1 1 1
```

可以看到：

智能体构建的表面吸附体系，其真空层（长边）方向在 Z 轴方向，这是符合表面吸附体系模拟的常规做法的。

这一表面结构不是四方晶系，而是六方晶系。在进行晶胞朝向调整时，不一定能用简单的 LATTICE_VECTORS 轮换来实现，而是需要对晶胞的自然坐标系进行旋转。同时，也不能对笛卡尔坐标进行轮换操作，而是要对分数坐标进行轮换操作。

通过与智能体对话，我们可以通过旋转晶格矢量，进一步构建不同真空层方向的这一表面体系。这一步骤也可以通过 ATOMKIT 等工具自行实现，相关对话如下：

图 4. 将真空层旋转到 X 轴

图 3. 将真空层旋转到 Y 轴

这些表面体系的 LATTICE_VECTORS 是一个 3×3 矩阵，代表我们所计算体系在 X, Y, Z 三个方向上的晶格矢量 a, b, c 大小，它们由 LATTICE_CONSTANT（晶格常数）进行缩放，此处晶格常数为 1.889726 Bohr, 即 1 Angstrom。可以参考 https://abacus.deepmodeling.com/en/latest/advanced/input_files/stru.html#lattice-vectors。它们的对比如下：

```
# 真空层在 Z 轴方向的 LATTICE_VECTORS
LATTICE_VECTORS
   10.21062192033      0.00000000000      0.00000000000
    5.10531096017      8.84265797145      0.00000000000
```

```
    0.00000000000     0.00000000000    39.25270341532
# 真空层在 Y 轴方向的 LATTICE_VECTORS
LATTICE_VECTORS
   10.21062192033     0.00000000000     0.00000000000
    0.00000000000    39.25270341532     0.00000000000
    5.10531096017     0.00000000000     8.84265797145
# 真空层在 X 轴方向的 LATTICE_VECTORS
LATTICE_VECTORS
   39.25270341532     0.00000000000     0.00000000000
    0.00000000000    10.21062192033     0.00000000000
    0.00000000000     5.10531096017     8.84265797145
```

不难看出，虽然各个 LATTICE_VECTORS 的分量数值没有变化（这得益于六方晶系的对称性依然不低），但其位置不再是简单轮换。因而对于非四方晶系的体系，相关的晶胞朝向调整需要采用专门代码或者专门工具进行。

**INPUT 文件**

本次计算的 INPUT 文件和 STRU 文件由 ABACUS-Agent 基于 CIF 文件准备，准备方式如下所示：

用户Jennifer 用

基于对Cu_111_CO_adsorbate_swapped.cif，构建ABACUS计算任务，并设置kspacing为0.14

A ADAM

准备根据用户需求生成ABACUS输入文件，未经指定的计算关键词将被赋予默认值，这些默认值未必能给出高效良好的计算结果，请用户仔细检查生成的输入文件。

Structure files: ['/share/data/adam-data-stage/workspace/2/1/475/Cu_111_CO_adsorbate_swapped.cif']
Structure type: cif

显示更多 >>

思考完毕
开始时间：2025-07-15 13:51:50 已耗时：20小时23分 Token: 17584

执行ABACUS输入文件生成器完毕
开始时间：2025-07-15 13:52:03 已耗时：6秒 CPU: 1c · 内存: 4000MB

思考完毕
开始时间：2025-07-15 13:52:15 已耗时：17秒 Token: 19164

A ADAM

## 任务列表

- [✅] 构建Cu晶体
  - 成功。构建好的晶体结构保存在： /share/data/adam-data-stage/workspace/2/1/475/Cu_bulk.cif。
- [✅] 基于构建好的Cu晶体，构建Cu(111)表面吸附CO分子的结构，表面层数为4，并做2x2扩胞，CO到表面距离为3埃，真空层厚度为15埃
  - 成功。构建好的表面吸附结构模型文件保存在： /share/data/adam-data-stage/workspace/2/1/475/Cu_111_CO_adsorbate.cif。
- [✅] 对Cu_111_CO_adsorbate.cif做晶格变换，使之真空层的Z轴方向朝向Y轴
  - 成功。晶格变换后的结构模型文件保存在： /share/data/adam-data-stage/workspace/2/1/475/Cu_111_CO_adsorbate_swapped.cif。
- [✅] 对Cu_111_CO_adsorbate.cif做晶格变换，使之真空层的Z轴方向朝向X轴
  - 成功。晶格变换后的结构模型文件保存在： /share/data/adam-data-stage/workspace/2/1/475/Cu_111_CO_adsorbate_swapped_to_X.cif。
- [✅] 基于对Cu_111_CO_adsorbate.cif，构建ABACUS计算任务，并设置kspacing为0.14
  - 成功。生成的ABACUS输入文件INPUT和结构文件STRU所在位置为： ./000000，INPUT文件内容为 {'ecutwfc': 100, 'calculation': 'scf', 'basis_type': 'lcao', 'symmetry': 1, 'scf_thr': 1e-07, 'scf_nmax': 100, 'smearing_method': 'gauss', 'smearing_sigma': 0.015, 'mixing_type': 'broyden', 'mixing_beta': 0.8, 'ks_solver': 'genelpa', 'kspacing': 0.14}，请确认ABACUS输入文件是否符合要求，若是可开展ABACUS计算。

图 5. 准备 ABACUS 输入文件

从智能体的输出中我们可以提取出所用的 INPUT。

```
INPUT_PARAMETERS
ecutwfc           100
calculation       scf
basis_type        lcao
kspacing          0.14
symmetry          1
scf_thr           1e-07
scf_nmax          100
smearing_method   gauss
smearing_sigma    0.015
mixing_type       broyden
mixing_beta       0.8
ks_solver         genelpa
```

可以看到，INPUT 文件中设置了 calculation 为 scf，即单点电子结构计算，kspacing 为 0.14，这一值是用户的要求设置。其他参数均为 ABACUS LTS 版本下开展 SCF 计算的合理默认值。

**KPT 文件**

在开展计算时，ABACUS 会根据 INPUT 文件中的 kspacing 参数自动生成 KPT 文件。对于本次测试的 Cu(111) 表面吸附 CO 结构，当真空层在 Z 轴方向时，生成的 KPT 文件内容如下：

```
K_POINTS
0
Gamma
3 3 1 0 0 0
```

需要注意的是，真空层方向的 K 点数目一般需要显式设置为 1，以防止计算中混入不应存在的层间周期性。此处由 kspacing 参数自动生成的 KPT 文件是满足要求的。

### 3.4.3 测试结果

测试计算由智能体调用计算资源完成，计算核数为 16 核，采用 MPI 方式并行，计算平台的 CPU 型号为 Intel(R) Xeon(R) Platinum 9242 CPU @ 2.30GHz。测试结果由智能体统计。下表给出本次测试的总能量，电子步数和计算时间消耗，我们通过比较总能量和时间差异来判断真空层方向对计算结果的影响，可以看到，系统总能量和电子步迭代步数并不会受到不同真空层取向的影响，但计算时间会受到显著影响。真空层摆放在 Z 方向时，单电子步用时和总计算时间显著大于其他两个方向，达到了其他两个方向的三倍以上。

| 真空层朝向 | 系统总能量（eV） | 计算总时间（s） | 总电子步数 | 单电子步用时（s） |
|---|---|---|---|---|
| X | -318603.9704461048 | 1432.47 | 39 | 35.77 47.09 |
| Y | -318603.9704469827 | 1421.25 | 39 | 35.37 46.94 |
| Z | -318603.9704470374 | 4725.36 | 39 | 120.24 131.01 |

根据以上测试结果，建议读者们在进行表面和表面反应等含真空层体系的计算时，让真空层规避 Z 方向，并将表面的长边摆放在 Z 方向，以获得 ABACUS 计算时的最大效率。同时，对于六方晶系或者类似于六方晶系的表面，则可将真空层摆放在 Y 方向，以获得最大的计算效率。

# 3.5 模守恒赝势生成方法简介

## 3.5.1 介绍

UPF（unified PP format）是一种类似 XML 格式的文件格式，用于存储赝势及其相关参数。该格式由 Quantum ESPRESSO（QE）团队开发，并已成为许多量子计算软件包的标准格式之一。

ABACUS（截至 v3.2.3）的 Kohn-Sham 密度泛函理论计算主要支持 UPF 格式的模守恒赝势，在使用 ABACUS 过程中，一般可以直接使用网上已经生成好的赝势，常见的下载位置：

ABACUS 官网

SG15-ONCV，参考文献：Optimization algorithm for the generation of ONCV pseudopotentials

Quantum ESPRESSO 官网中的模守恒赝势

pseudo-dojo 官网（psp8 格式，需要转换）

PWmat 官网

但是，在实际计算中，可能这些赝势并不符合当前的需求，这时候就需要自己生成模守恒赝势。可以通过其他格式赝势转换或者利用赝势软件生成两种方式来获得赝势。

注：ABACUS 的无轨道密度泛函理论计算（Orbital-Free Density Functional Theory，简称 OFDFT）需要用到一种特殊的局域势（不包含非局域势），一般采用 BLPS 局域赝势文件而非上文提到的 UPF 文件，如果做 OFDFT 计算的，可以阅读介绍 OFDFT 计算的文档，或者登录 Emily A. Carter 教授主页寻找相关的信息。

## 3.5.2 从其他格式赝势转换

### 使用 QE upflib 中的 upfconv 进行转换

其支持将 UPF v.1 格式、vdb/van 格式（Vanderbilt US pseudopotential）、cpi/fhi 格式（FHI/abinit）转化成 UPF v.2 格式

```
path_to_QE/upflib/upfconv.x -u  *.upf/UPF/vdb/van/cpi/fhi
```

注：这里的 path_to_QE 代表下载的 Quantum Espresso 的软件包地址。

注：文件夹 upflib 在 QE 5.x 和 6.x 版本名称为 upftools，在 QE 7.x 版本为 upflib。

### psp8 格式转换

psp8 格式是 ONCVPSP 软件生成的一种赝势格式，在 Abinit 官网（pseudo-dojo）中，其使用的就是 psp8 的格式，目前没有直接将 psp8 格式的赝势直接转换成 UPF 格式的脚本，但可以将 psp8 中输入文件部分摘抄下来，用 ONCVPSP 软件重新生成 UPF 格式的赝势（具体生成见下面 ONCVPSP 的讲解）。

可供参考的批量处理脚本：https://github.com/pipidog/ONCVPSP

## 3.5.3 赝势的生成

下面介绍三个可以生成 UPF 模守恒赝势的软件，分别是 ONCVPSP，Opium，和 ld1.x

### 1. ONCVPSP

#### 介绍

ONCVPSP（Optimized Norm-Conserving Vanderbilt PSeudoPotential）是由 D.R. Hamann 等人提出的优化版模守恒赝势，其有更高的精度与效率。ONCVPSP 依赖 Libxc，支持多种交换关联泛函。

参考文献：Optimized norm-conserving Vanderbilt pseudopotentials。

**安装**

安装 Libxc

Libxc 网址：Libxc - a library of exchange-correlation functionals for density-functional theory

推荐下载 libxc-4.3.4

然后执行如下命令：

```
cd libxc-4.3.4
autoreconf -i
./configure --prefix='PATH/TO/LIBXC' CC=icc FC=ifort F77=ifort
make
make install
```

命令执行完毕后即可在 PATH/TO/LIBXC 目录下看到 bin,include 和 lib 三个目录，代表安装成功

安装 oncvpsp

推荐下载 oncvpsp-4.0.1

然后执行 cd oncvpsp-4.0.1，进入文件夹

这时需要修改 make.inc 文件

修改 F77 = ifort, F90 = ifort, CC = icc

FFLAGS 删去 -ffast-math

LIBS 改为 -qmkl=”sequential”

LIBS+ 和 FFLAGS 中的地址改为自己的 PATH/TO/LIBXC

具体如下：

```
# System-dependent makefile options for ONCVPSP
# This must be carefully edited before executing "make" in src
#
# Copyright (c) 1989-2019 by D. R. Hamann, Mat-Sim Research LLC and Rutgers
# University

##### Edit the following lines to correspond to your compilers ####

F77        = ifort
F90        = ifort
CC         = icc
FCCPP      = cpp

FLINKER     = \$ (F90)

#FCCPPFLAGS = -ansi -DLIBXC_VERSION=301  #Use this for versions <301
FCCPPFLAGS = -ansi -DLIBXC_VERSION=400  #Use this for versions >400

##### Edit the following optimization flags for your system ####

FFLAGS     = -O3 -funroll-loops
CFLAGS     = -O3

##### Edit the following LAPACK and BLAS library paths for your system ####
```

```

LIBS = -qmkl="sequential"

##### Edit the following for to use libxc if available #####

OBJS_LIBXC =        exc_libxc_stub.o

# oncvpsp is compatible with libxc
# To build oncvpsp with libxc, uncomment the e following lines and edit
# the paths to point to your libxc library and include directories
# make clean in src before rebuilding after changing this

LIBS += -L/PATH/TO/LIBXC/lib -lxcf90 -lxc
FFLAGS += -I/PATH/TO/LIBXC/include

#LIBS += -L/home/drh/abinit/fallbacks/exports/lib -lxcf90 -lxc
#FFLAGS += -I/home/drh/abinit/fallbacks/exports/include

OBJS_LIBXC =        functionals.o exc_libxc.o
```

然后执行如下命令：

```
make
# 如果 make -j 报错，可以忽略再 make，即可编译成功
```

安装测试完成后，即可在 src 目录下看到 oncvpsp.x
可以在 /.bashrc 中增加如下命令将 oncvpsp.x 加入环境变量，方便直接调用：

```
export PATH=\$PATH:/PATH/TO/ONCVPSP/src
```

**输入文件**

输入文件的准备可以参考 PATH_TO_ONCVPSP/doc/32_Ge_annotated.dat，或者参考已知 ONCV 的赝势里面的 <PP_INPUTFILE> 部分来写。

这里以铝（Al）为例，输入文件 Al.dat:

```
# ATOM AND REFERENCE CONFIGURATION
# atsym, z, nc, nv, iexc   psfile
Al   13   1   4   4   upf
#
# n, l, f   (nc+nv lines)
1 0 2
2 0 2
2 1 6
3 0 2
3 1 1
#
# PSEUDOPOTENTIAL AND OPTIMIZATION
```

```
# lmax
1
#
# l, rc, ep, ncon, nbas, qcut   (lmax+1 lines, l's must be in order)
0 1.29163 0 5 8 10.3003
1 2.40653 0 5 8 7.02214
#
# LOCAL POTENTIAL
# lloc, lpopt, rc (5) , dvloc0
4 5 0.932267 0
#
# VANDERBILT-KLEINMAN-BYLANDER PROJECTORs
# l, nproj, debl   (lmax+1 lines, l's in order)
0 2 0
1 2 0
#
# MODEL CORE CHARGE
# icmod, fcfact
0 0
#
# LOG DERIVATIVE ANALYSIS
# epsh1, epsh2, depsh
-5.0  3.0  0.02
#
# OUTPUT GRID
# rlmax, drl
6.0  0.01
#
# TEST CONFIGURATIONS
# ncnf
0
# nvcnf
#   n    l    f
```

- nc : c = core，芯电子层数
- nv: v=valence，价电子层数，调整 nc 和 nv 可以改变碟势的价电子数
- iexc : 生成厦势所用的交换关联泛男，详情可参考 ONCVPSP-4.0.1 目录下 doc 内的 pwscf_exc.txt 文件，常用的如下

```
iexc==3 .or. iexc==-001009, 'functional="PZ"'
iexc==4 .or. iexc==-101130, 'functional="PBE"'
```

- psfile：赝势格式，有 upf 和 psp8 两种。ABACUS 支持 upf 的格式
- rc：rc 中较小的为赝势的截断半径。在温度较高或密度较大的温稠密体系，通常需要调整赝势的截断半径，推荐小于 0.7 倍维格纳半径，单位为 Bohr；每个 1 对应一个 rc ，其中最小的为赝势的截断半径
- rc（5）：rc 都要大于等于 rc（5）。例如 Al 的 1=0 的 rc 为 1.29163 Bohr，1=1 的 rc 为 2.40653 Bohr，rc（5）为 0.932267 Bohr，因此该赝势的截断半径为 1.29163 Bohr。
- qcut：通过调整 (+0.1)，直到推荐 ECUT 达到最小

- dvloc0：通过调整 (+0.5)，直到消除赝势的 GHOST 态（可参考文献 Phys. Rev. B 41, 12264 (1990)）

**生成赝势**

执行命令：

```
oncvpsp.x < Al.dat > Al.out
sed -n '/<UPF version="2.0.1">/,/<\/UPF>/p' Al.out > Al.UPF
```

生成完赝势需要查看 Al.out 文件，确定没有 GHOST 态（可参考文献 Phys. Rev. B 41, 12264 (1990)）：
Testing for highly-localized positive-energy ghosts l rc E Basis Diag. E Cutoff
1 0.974041 120.256772 338.98 WARNING - GHOST（+）
如上，出现 WARNING - GHOST 说明有 GHOST，需要调整 dvloc0 （+-0.5）直到消除 GHOST 态
sed 命令是为了从 Al.out 文件中截取赝势文件到 Al.UPF 中
如果同时要画出类似 Phys. Rev. B 88, 085117 (2013) 的图，需要准备输入文件 Al.dat 并执行命令：

```
sh PATH_TO_ONCVPSP/scripts/run.sh Al
```

可以画出如下的图：

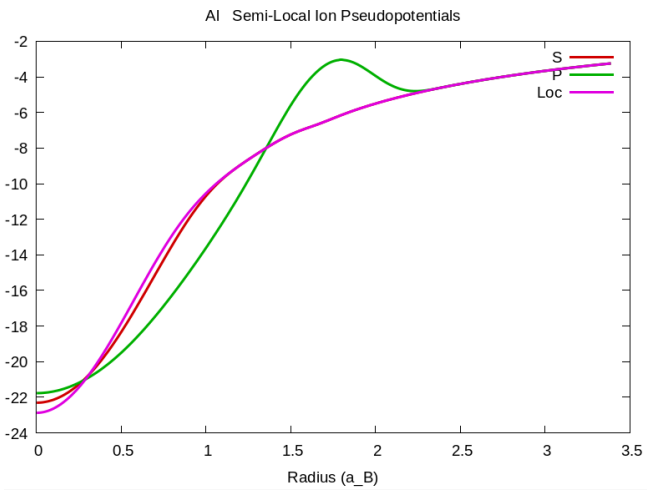



图 1. 局域势函数与非局域势不同轨道角动量对应的半局域径向势函数

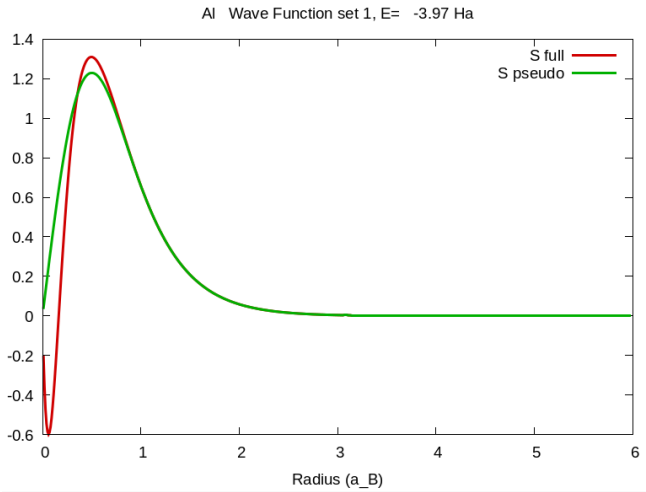



图 2. S 赝波函数与全电子波函数对比

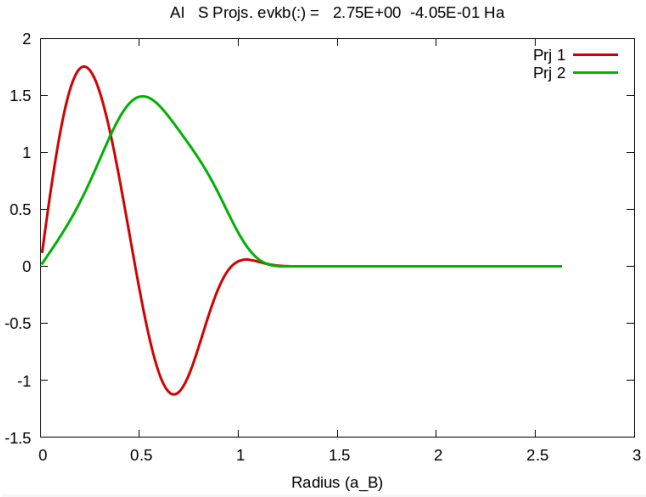



图 3. S 的双投影波函数

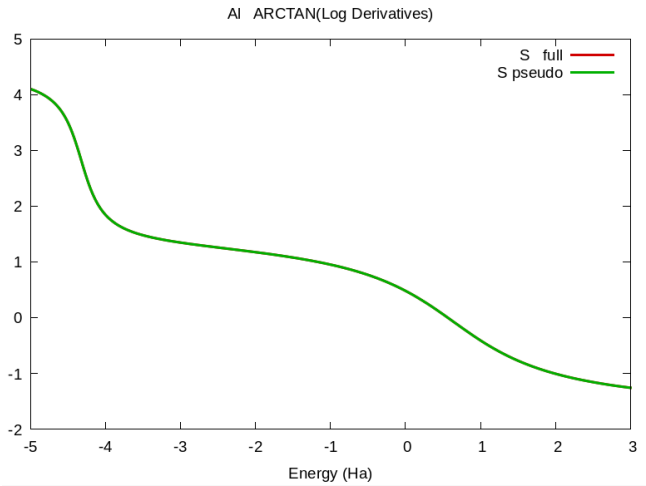



图 4. 不同能级波函数在截断半径处 log 导数对比，其影响散射性质的计算

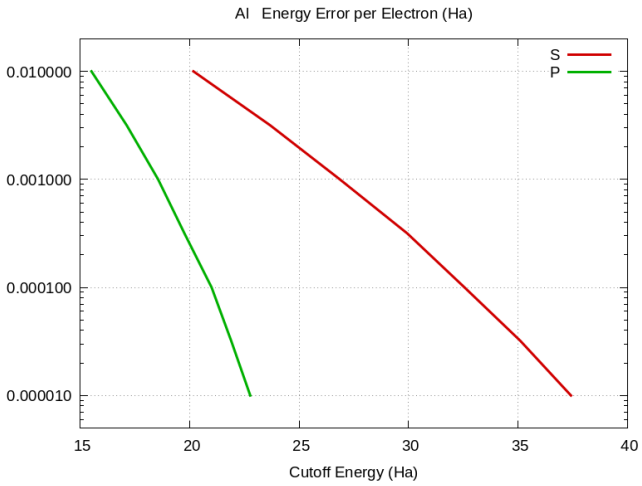



图 5. 不同轨道角动量对应的截断能

**优化赝势**

依次调节不同 l 对应的 qcut（+-0.1），检查对应 ECUT（1）变化情况，直到 ECUT（1）达到最小
参考的脚本：

```
!/bin/bash

qcut=17
dq=0.1
# 遍历 l=0 的 qcut 从 17 到 25，间隔为 0.1，输出其对应的 ECUT
while [ `echo "$qcut <= 25" | bc` == "1" ]
do
cat > test.dat <<EOF
# ATOM AND REFERENCE CONFIGURATION
# atsym  z    nc    nv    iexc   psfile
  Al 13.00     1     4     4      upf
#
#   n    l    f        energy  (Ha)
    1    0    2.00
    2    0    2.00
    2    1    6.00
    3    0    2.00
    3    1    1.00
#
# PSEUDOPOTENTIAL AND OPTIMIZATION
# lmax
    1
#
#   l,   rc,     ep,   ncon, nbas, qcut
    0   0.68  -3.97500    5    8   $qcut
    1   0.75  -2.55934    5    8   19.4
#
# LOCAL POTENTIAL
# lloc, lpopt,  rc (5) ,   dvloc0
```

```
    4    5       0.60     0.00000
#
# VANDERBILT-KLEINMAN-BYLANDER PROJECTORs
# l, nproj, debl
    0    2   3.69008
    1    2   2.45967
#
# MODEL CORE CHARGE
# icmod, fcfact
    0   0.00000
#
# LOG DERIVATIVE ANALYSIS
# epsh1, epsh2, depsh
   -5.00    3.00    0.02
#
# OUTPUT GRID
# rlmax, drl
    6.00    0.01
#
# TEST CONFIGURATIONS
# ncnf
    0
# nvcnf
#   n    l    f
EOF

../../src/oncvpsp.x < test.dat > qcut$qcut.out
grep -A 5 "Energy error" qcut$qcut.out > _tmp.txt
E1=`sed -n "6p" _tmp.txt | awk '{print \$3}'`
E2=`sed -n "13p" _tmp.txt | awk '{print \$3}'`
E3=`sed -n "20p" _tmp.txt | awk '{print \$3}'`
E4=`sed -n "27p" _tmp.txt | awk '{print \$3}'`

# echo $qcut $E1 $E2 $E3 $E4
if [ `echo "$E1>$E2" | bc` == '1' ] ;then
Emax1=$E1
else
Emax1=$E2
fi

if [ `echo "$E3>$E4" | bc` == '1' ];then
Emax2=$E3
else
Emax2=$E4
fi

echo $qcut $Emax1 $Emax2

```

```
qcut=`echo "$qcut+$dq" | bc`
done
```

不同的 l 都会有对应的能量截断值 ECUT(l)。一般 rc(l) 越小，对应的 ECUT(l) 越大，赝势也就越精确。由于 DFT 计算中的截断能 ECUT 是由赝势中较大的那个 ECUT(l) 决定，如果不同 l 的 ECUT(l) 相差很大，可以适当减小较小的 ECUT（l）对应 l 的截断半径 rc，使得不同 l 对应的 ECUT(l) 更接近，这样并不会增加赝势计算的 ECUT，却可以提升精度。

这样调完之后如果有 GHOST 态，需要调整 dvloc0 （+-0.5）直到消除 GHOST 态，这个赝势才可使用

### 2. Opium

Opium 软件包可以生成 RRKJ 、TM 或 Kerker 径向波函数的赝势，官网有 LDA、GGA 赝势库（但是某些输入文件已经不匹配最新的 4.1 版本，需要稍作修改）

下载 Opium 之后利用 configure 安装：

```
./configure
make
```

输入文件可以参考官网 LDA、GGA 赝势库，也可以参考 PATH_TO_OPIUM/tests，这里以铝（Al）为例，输入文件 al.param:

```
[Atom]
Al
6
100 2.00  -13.0   # reference configuration
200 2.00   -1.4   # reference configuration
210 6.00   -0.4   # reference configuration
300 2.00   -0.1   # reference configuration
310 0.95  -13.0   # reference configuration
320 -1.0   -0.0   # 这条能带是非束缚态，需要将占据数调成负数，官网给错了
[Pseudo]
3 1.85 1.85 1.97
opt

[Optinfo]
7.07 10    # rc[a.u.] qc[sqrt (Ry) ] Nb
7.07 10
7.07 10

[Configs]
3                 # number of valence configurations
#
300  2.00  -13.0   # nlm occ eigen (- means auto-generate)
310  1.00   -6.0
320  0.00   -6.0
#
300  1.00  -13.0
310  2.00   -6.0
```

```
320  0.00   -6.0
#
300  2.00  -13.0
310  0.00   -6.0
320  0.00   -6.0

[XC]
gga

[Relativity]
srl
```

这里的参数可以参考官网手册
之后执行

```
opium al log all  # 输出在 log 文件里，依次执行全电子计算、计算赝势、计算非局域势、可移植性测试
opium al log upf  # 生成 al.upf 的输出文件
upfconv.x -u  al.upf # 利用 QE 的 upflib 将 UPFv.1 转化成 UPFv.2 格式
```

**3. ld1.x**

QE 的 atomic 模块中的 ld1.x 支持生成赝势。其不仅可以生成模守恒赝势、还支持超软赝势、PAW 方法，支持全相对论、标量（非）相对论赝势（rel, non-rel/sca-rel），其径向波函数支持 TM（更稳定）与 RRKJ 两种方法，支持交换关联近似 7 类 LDA（pz）,GGA（pbe, pbesol, revpbe, bp, wc, pw91）。

下载好 QE 软件后，可以直接安装：

```
./configure
make ld1
```

**pslibrary 赝势库**

推荐 ld1.x 生成其自带的 pslibrary 赝势库，下载好 pslibrary.1.0.0.tar.gz，解压，进入文件夹，修改 QE_path 文件，指定 QE 的路径。然后打开 make_ps 文件，解锁相应赝势：

```
#   These two files generate PAW and US PPs for all elements. These are
#   high accuracy - high kinetic energy cut-off PPs.
#
. ../paw_ps_high.job # 默认打开的
. ../us_ps_high.job
#
#   These two files generate additional PAW and US PPs for some elements.
#   These are less accurate PP than the previous one but require
#   lower kinetic energy cut-off or have less projectors or less semicore
#   states.
#
. ../paw_ps_low.job
. ../us_ps_low.job

```

```
#  Uncomment the following line to generate the old pslibrary 0.3.1 PPs.
#  打开下面的注释，可以解锁其余赝势
#
#. ../paw_ps_collection.job    # 可以打开注释
#. ../us_ps_collection.job     # 可以打开注释

#  Uncomment the following line to generate the NC-PPs. Be very careful
#  these PPs are completely untested.
#  虽然可以生成，但其也提示该赝势的准确性并不能保证
#
#. ../nc_ps_collection.job     # 可以打开注释
```

下面运行

```
./make_all_ps
```

就能看到在一个个生成赝势库里的赝势：

```
Making   Ac.pz-spfn-kjpaw_psl.1.0.0.in  ...  Done
Making   Ac.pz-spfn-rrkjus_psl.1.0.0.in  ...  Done
Making   Ag.pz-n-kjpaw_psl.1.0.0.in  ...  Done
Making   Ag.pz-n-rrkjus_psl.1.0.0.in  ...  Done
Making   Ag.pz-spn-kjpaw_psl.1.0.0.in  ...  Done
Making   Ag.pz-spn-rrkjus_psl.1.0.0.in  ...  Done
Making   Al.pz-n-kjpaw_psl.1.0.0.in  ...  Done
Making   Al.pz-n-rrkjus_psl.1.0.0.in  ...  Done
Making   Al.pz-nl-kjpaw_psl.1.0.0.in  ...  Done
Making   Al.pz-nl-rrkjus_psl.1.0.0.in  ...  Done
```

如果不想生成所有的元素的赝势，则可以修改 make_ps 的 element，例如：

```
element='C Si Ge'
```

**生成自己的赝势**

参数的详细解释见 PATH_TO_QE/atomic/Doc/INPUT_LD1.html 或线上的文档（不是最新的，建议前者），输入文件可以参考 pslibrary 赝势库的例子。

这里就简单给个 Al（铝）的例子，准备输入文件 al.in：

```
&input
   title='Al',
   zed=13.0,
   rel=1,
   config='[Ne] 3s2 3p1 3d-2.0 4f-2.0',
   iswitch=3,
   dft='PBE'
/
&inputp
```

```
  pseudotype=2,
  file_pseudopw='Al.pbe-n-nc.UPF', # 输出赝势
  lloc=2,
  nlcc=.true.,
  tm=.true.
/
3
3S  1  0  2.00  0.00  2.60  2.60  0.0
3P  2  1  1.00  0.00  2.60  2.60  0.0
3D  3  2  0.00  0.10  2.60  2.60  0.0
```

运行

```
ld1.x < al.in
```

即可生成 Al.pbe-n-nc.UPF 模守恒赝势。

### 3.5.4 结尾

ABACUS 使用的是模守恒赝势，基于模守恒赝势还可以产生数值原子轨道，进行基于数值原子轨道的高效率密度泛函理论计算。有些情况下，网上提供的赝势不能满足需求，这个时候赝势的生成需要对赝势理论有比较深入的了解之后，才能调整好相关参数，生成质量较好的赝势。此外，生成之后，应该经过较为充分的测试，才能保证赝势的可移植性和正确性。如果大家有任何问题，欢迎发 email 至文档开头的邮箱。

## 3.6 数值原子轨道（一）：ABACUS 中的数值原子轨道命名和使用方法

### 3.6.1 数值原子轨道的背景知识

**数值原子轨道**

对电子结构的求解可以在不同的表象下进行，电子的波函数以及势函数也可以在不同基矢量下表示，常用的有平面波和局域轨道等。LCAO（Linear Combination of Atomic Orbitals）方法就是通过局域的原子轨道来求解量子力学问题。其中局域轨道的选取方式有多种，例如高斯轨道（Gaussian type Orbitals，GTOs 或 Gaussians）、数值原子轨道（Numerical Atomic Orbitals）、万尼尔函数（Wannier Functions）等。

**数值原子轨道作为基矢量的优点**

数值原子轨道作为基矢量有几个优点：第一，基矢量个数相比于一些常用的基矢量（例如平面波和实空间网格）大幅度降低；第二，数值原子轨道是局域的，空间上可以严格截断，采用数值原子轨道来构建体系的哈密顿量的效率可以达到线性标度的时间复杂度。

**构造数值原子轨道基组的几种方案**

构造精度高、可系统提升数量、可移植性好的原子轨道基组颇有挑战，因此也有多种方案被提出。例如，Junquera 等人提出在一维薛定谔方程中加入不同形式的约束势场，从而求解出具有严格截断的数值原子轨道 [1]。Ozaki 在 OpenMX 软件中采用变分的方法来优化局域轨道的形状，从而得出一组最优的数值原子轨道 [2]。Volker 等人提出在一个大的局域轨道基组中挑选最合适的局域轨道组成不同等级的基组轨道，该方案用于全电子密度泛函理论软件 FHI-aims 中 [3]。Chen 等人提出利用前人提出的溢出函数（Spillage function）[4-5] 来构造可系统

提高数量的数值原子轨道，其中每个轨道都由一组球贝塞尔函数作为基矢量展开，该轨道被用在 ABACUS（原子算筹）软件中 [6]。

### 3.6.2 数值原子轨道的命名方法

**数值原子轨道的组成**

数值原子轨道（Numerical Atomic Orbitals，简称 NAO）是目前在 ABACUS 程序中支持的一种基矢量。从数学形式上来看，数值原子轨道可以分解为径向函数 $f_l$、和球谐函数 $Y_lm$ 的乘积。

$$\phi_{lm\zeta}(\mathbf{r}) = f_l\zeta(r)Y_{lm}(\hat{r}),$$

其中 1 是角量子数，$m$ 是磁量子数，$\zeta$ 代表了每个角量子数上对应的多个径向轨道，实际计算中通常采用多于 1 个轨道来增加基矢量的完备性。

**数值原子轨道的命名方法**

数值原子轨道有一套常用的命名方案用来表示选取的基组大小，早期该命名方案在 SIESTA 软件中被采用 [7][8][9][10]，后来在 OpenMX、FHI-aims 和 ABACUS 中也采取这套命名方案。具体来说是，对于每个被电子占据的角量子数 l，若采用 1 条径向轨道，则称该基组为 Single-ζ 轨道，简称 SZ 轨道基组。若采用 2 条径向轨道，则称该基组为 Double-ζ 轨道，简称 DZ 轨道基组。

目前，在许多赝势结合数值原子轨道的程序里，通常会在 DZ 轨道的基础上引入 1 条极化（polar）的径向轨道，即角量子数更高的轨道，来组成 DZP（Double-ζ valence orbitals plus SZ polarization）轨道基组。此外，还有基组数量更大的 TZDP（Triple-ζ valence orbitals plus DZ polarization）轨道等。

**数值原子轨道基组的个数**

数值原子轨道基组的具体个数除了取决于轨道类别（比如 SZ、DZP、TZDP）外，也取决于元素种类、所选取的赝势。

**轨道类别：**比如对于 O 元素的赝势而言，一般将两个 1s 电子作为核内电子，在构造赝势的时候只考虑它的外层 6 个价电子部分 2s、2p 轨道，则该 O 赝势下的电子的极化轨道为 d 轨道。故它的 SZ（Single-ζ）轨道包含 1 组 s 轨道、1 组 p 轨道，共 1*1 + 1*3 = 4 个轨道；DZP 轨道包含 2 组 s 轨道、2 组 p 轨道、1 组 d 轨道，共包含 2*1 + 2*3 + 1*5 = 13 个轨道；TZDP 轨道包含 3 组 s 轨道、3 组 p 轨道、2 组 d 轨道，共包含 3*1 + 3*3 + 2*5 = 22 个轨道。

**元素种类：**比如同为 DZP 轨道，对于 H 元素（1 个价电子）即为 2 组 s 轨道、1 组 p 轨道（p 轨道也是 H 的极化轨道），共包含 2*1 + 1*3 = 5 个轨道。对于 O 元素即为 2 组 s 轨道、2 组 p 轨道、1 组 d 轨道，共包含 2*1 + 2*3 + 1*5 = 13 个轨道。

**赝势：**比如对于 Fe 元素而言，同为 DZP 轨道。若选取的赝势为，

$$[\mathrm{Ne}]3s^23p^64s^23d^6$$

即

$$[\mathrm{Ne}] = 1s^22s^22p^6$$

部分为芯电子，$3s^23p^64s^23d^6$

部分为 16 个价电子，则 DZP 轨道包含 2*2 套 s 轨道（3s 和 4s）、2 套 p 轨道（3p）、2 套 d 轨道（3d）、1 套 f 轨道（极化轨道），即每个原子的 DZP 数值原子轨道个数为

$$2^*2^*1 + 2^*3 + 2^*5 + 1^*7 = 27$$

个轨道。若选取的赝势为

$$[\mathrm{Ar}]4s^2 3d^6$$

则 DZP 轨道包含 2 套 s 轨道、2 套 d 轨道、1 套 p 轨道，共 $2^*1 + 2^*5 + 1^*3 = 15$ 个轨道。

### 3.6.3 ABACUS 中数值原子轨道的使用方法

**ABACUS 中的数值原子轨道文件**

ABACUS 提供了已经生成好的数值原子轨道库打包文件供下载（官网下载链接），对于绝大多数计算任务，这些数值原子轨道是经过精度和可靠性验证，可以直接使用的。这些轨道文件的开头提供了关于该轨道的重要信息。以 O_gga_7au_100Ry_2s2p1d.orb 轨道文件为例，首先文件名包含的信息有：氧元素（o），GGA 泛函（gga），数值原子轨道截断半径（7au，即 7 Bohr），推荐能量截断值（100 Ry），数值原子轨道个数（2s2p1d，2 个 s 的径向轨道，2 个 p 的径向轨道，1 个 d 的径向轨道）。值得注意的三个点是：

ABACUS 里用到的模守恒赝势和轨道是需要匹配的。因为不同的赝势可能有不同的价电子，而数值原子轨道是用来描述这些价电子的，如果赝势的价电子多，则相应的默认数值原子轨道基组数量也会增多。

推荐用户直接使用轨道文件包含的能量截断值做计算。ABACUS 里的数值原子轨道在生成时，是尽量的去匹配平面波输出的波函数，从而优化得到的。而平面波计算是有一个能量截断值的，因此为了尽可能的保证 LCAO 的精度，我们建议直接使用推荐的能量截断值，而不需要真正做计算的时候做体系能量随着能量截断值变化的收敛性测试。当你在 LCAO 计算时改变能量截断值时候，其实只是改变了平面波的个数，这些平面波是用来做一些数值计算的，本质上并没有改变基矢量的个数，而基矢量的个数增加是靠改变基组大小，例如 DZP 到 TZDP 来实现的。

特殊情况可以自己生成数值原子轨道。如果用户有自己特殊的赝势，或者对目前提供的原子轨道的参数或者个数感觉不满意，例如对于导带的描述能力较差，也可以自己生成数值原子轨道，具体生成的方法可以参考这个系列文档的第二和第三篇。打开以上提到的氧的数值原子轨道文件，文件的开头如下：

```
-------------------------------------------------------------------------
Element                     O
Energy Cutoff(Ry)           100
Radius Cutoff(a.u.)         7
Lmax                        2
Number of Sorbital-->       2
Number of Porbital-->       2
Number of Dorbital-->       1
-------------------------------------------------------------------------
SUMMARY  END
```

这里包括元素种类（Element），生成轨道时指定的截断能量值（Energy Cutoff）、截断半径（Radius Cutoff），最大角量子数（Lmax）及各角量子数轨道的个数（Number of * orbital，* 为 S、P、D 等轨道角动量）。根据这些信息可以知道该轨道的类别，比如该举例文件即包含 O 的 DZP 轨道。

上述 ABACUS 提供的数值原子轨道文件中，在取名中即包含文件开头的重要信息，包括各角量子数的轨道个数。由这些轨道的文件名即可判断轨道类型。

轨道文件中包含的后续内容为不同类型、不同角量子数（L）的多个径向数值原子轨道（即不同 N）的具体数据，目前轨道是存在均匀格点上的，之后有可能也会支持非均匀格点。

**如何选择数值原子轨道文件**

用户需要根据精度要求，选择数值原子轨道合适的截断能量值、截断半径及轨道类型。截断能量值、截断半径越大，轨道类型提升（SZ、DZP、TZDP），精度越高，结果更接近平面波。注意对于不同元素，要达到同样的精度，以上数值设置并不一定相同。

**如何修改数值原子轨道文件的设置，获得低精度轨道类型文件**

在 ABACUS 中生成数值原子轨道文件时，可以设置同时生成多种精度的轨道文件，即设置 SIAB_INPUT 文件中的参数 Save Orbitals，详见篇目（三）。

如果已有一份数值原子轨道文件，但希望直接用此文件进行低精度计算，比如希望使用 DZP 轨道文件进行 SZ 计算。可以修改文件开头的信息实现，具体是 Lmax 及各角量子数轨道个数，将这些参数调整到低精度计算对应的数值。文件的后续内容不需要做更改。

比如对于 O 元素而言，DZP 轨道文件参数如 3.1 所示，若要进行 SZ 基矢量的计算，则将 Lmax 设置为 1，Number of Sorbital/Porbital/Dorbital=1/1/0 即可。

### 3.6.4 参考文献

[1] J. Junquera, Ó. Paz, D. Sánchez-Portal, and E. Artacho, Numerical Atomic Orbitals for Linear-Scaling Calculations, Phys. Rev. B, 64, 235111 (2001).

[2] T. Ozaki, Variationally Optimized Atomic Orbitals for Large-Scale Electronic Structures, Phys. Rev. B, 67, 155108 (2003).

[3] V. Blum, R. Gehrke, F. Hanke, P. Havu, V. Havu, X. Ren, K. Reuter, and M. Scheffler, Ab Initio Molecular Simulations with Numeric Atom-Centered Orbitals, Comput. Phys. Commun., 180, 2175 (2009).

[4] M. Chen, G.-C. Guo, and L. He, Systematically Improvable Optimized Atomic Basis Sets for Ab Initio Calculations, J. Phys.: Condens. Matter, 22, 445501 (2010).

[5] M. Chen, G.-C. Guo, and L. He, Electronic Structure Interpolation via Atomic Orbitals, J. Phys.: Condens. Matter, 23, 325501 (2011).

[6] Liu Xiao-Hui et al., Introduction to first-principles simulation package ABACUS based on systematically improvable atomic orbitals, Acta Phys. Sin., 64, 187104 (2015).

[7] Sánchez-Portal D, Ordejon P, Artacho E, et al. Density-functional method for very large systems with LCAO basis sets, J. International Journal of Quantum Chemistry, 65, 453-461 (1997).

[8] Artacho E, Sánchez-Portal D, Ordejón P, et al. Linear-scaling ab-initio calculations for large and complex systems, J. Physica Status Solidi (b), 215, 809-817 (1999).

[9] Junquera J, Paz Ó, Sánchez-Portal D, et al. Numerical atomic orbitals for linear-scaling calculations, J. Physical Review B, 64, 235111 (2001).

[10] Soler J M, Artacho E, Gale J D, et al. The SIESTA method for ab initio order-N materials simulation, J. Phys.: Condens. Matter, 14, 2475 (2002).

## 3.7 数值原子轨道（二）：生成给定模守恒赝势的数值原子轨道

### 3.7.1 ABACUS 中的数值原子轨道背景知识

**数值原子轨道**

ABACUS 中的三维数值原子轨道 [1]：可以分解为径向部分与角向球谐函数的乘积，即

$$\phi_\mu(r) = f_{\mu,l}(r)Y_{lm}(\hat{r}),$$

其中 $\mu = \{\alpha, i, \xi, l, m\}$，$\alpha$ 是原子种类的指标，$i$ 是每个原子种类对应的原子个数的指标，$\xi$ 是角动量 l 对应的径向函数的个数，$m$ 是磁量子数。

径向部分函数：第 $\mu$ 个轨道的径向部分函数可以写为球贝塞尔函数的线性组合（可参考文献 [1]）

$$f_{\mu l}(r) = \begin{cases} \sum_q c_{\mu q} j_l(qr), & r < r_c \\ 0, & r \geq r_c \end{cases},$$

其中 $j_l(qr)$ 是波矢为 q 的球贝塞尔函数，$r_c$ 是径向部分的截断半径，满足关系 $j_l(qr_c) = 0$。q 的个数由能量截断值决定（以下会介绍）。在实际计算里，通过用户指定每种元素的数值原子轨道截断半径和能量截断值来确定需要用到多少个球贝塞尔函数。组合系数 $c_{\mu q}$ 是可优化的量，决定了数值原子轨道的形状。

**参考体系**

参考系统：在 ABACUS 中构造某个元素对应的数值原子轨道的最普遍方式是为每个元素选取一组参考体系做平面波计算（可以是单 k 点也可以是多 k 点），得到这些参考体系的电子波函数，然后通过最小化自定义的一个溢出函数（下面会介绍）来优化 $c_{\mu q}$ 系数。换句话说，我们需要选定一组参考系统，生成数值原子轨道基组，并期望它们有良好的迁移性，以便可以在更一般的情况下使用。

Dimer（二聚体）：我们发现二聚体是较为合适的参考系统。具体来说，推荐选择的是若干个不同键长的二聚体（dimer），有时也可选择三聚体（trimer）或四聚体。对同一个元素，我们通常选择几个键长不同的二聚体（例如 5-6 个），这些键长覆盖二聚体稳定键长附近能量区间，通常可以取与最稳定能量差别为 1-2 个 eV 的能量曲线上的点。

周期性体系：数值原子轨道还有一些特殊用途，例如能带插值。面对这种场景，可以选择一系列带多个 k 点的周期性体系，此时参考态中的每个原子构型就对应了不同 k 点指标和不同能带指标的波函数，这些波函数会被用于最小化溢出函数来生成数值原子轨道，可以参考文献 [2]。

**溢出函数**

溢出函数（英文为 spillage）：由给定参考体系的多个电子波函数与生成的数值原子轨道之间的差别来定义。换个角度理解，溢出函数的大小反映的是由一组" 精确" 波函数所张成的希尔伯特空间与局域轨道所张成的空间的差别。溢出函数定义如下：

$$S = \frac{1}{N_n} \sum_{n=1}^{N_n} \left\langle \Psi_n | 1 - \hat{P} | \Psi_n \right\rangle,$$

其中 $\Psi_n$ 表示平面波基组下的本征态，$N_n$ 是选定的参考电子态的个数，$\hat{P}$ 是由所有原子轨道张成的投影子

$$\hat{P} = \sum_{\mu v} |\phi_\mu\rangle S^{-1}_{\mu v} \langle\phi_v|,$$

其中 $S_{\mu v} = \langle\phi_\mu|\phi_v\rangle$ 是局域轨道 $\phi_\mu$ 与 $\phi_v$ 之间的重叠矩阵，投影子用到了重叠矩阵的逆。

**轨道正交：**事实上，如果我们有了给定的数值原子轨道，我们可以在此基础上再次定义新的溢出函数来得到一组新的数值原子轨道，这也是为什么该算法可以系统性的增加数值原子轨道的原因。定义的方式也比较直接，可以将 $|\Psi_n\rangle$ 和已有原子轨道形成的投影子 $\hat{P}$ 正交，公式为

$$|\Psi_n^{new}\rangle = (1 - \hat{P}) |\Psi_n\rangle,$$

这个公式里，左边的波函数经过正交已经包含原子轨道的成分，这样可以再次作为新的参考体系波函数生成新的数值原子轨道，以上算法已经被实现在 SIAB 这个程序里。

**最小化溢出函数：**优化原子轨道时，最终溢出函数值定义为选定的几个二聚体的平均溢出函数值。溢出函数可以用来评估获得的原子轨道基组的质量，溢出函数越小，则原子轨道所张成的空间越接近于参考态波函数。

在 ABACUS 中，通过最小化溢出函数的方式来优化 $c_{\mu q}$，该系数决定了原子轨道的径向函数的形状。这种方法的好处在于

- **轨道形状可调：**可以由球贝塞尔函数基组的线性组合自动生成，而不需要任何先决条件，因此原则上可获得完全优化的原子轨道。
- **基组数量可调：**可以用统一的方式系统的增加轨道数量，包括增加径向轨道的数量（multi-ζ）和增加更高角动量的轨道（极化轨道）两种方式。为此，我们可以首先生成具有给定角动量的轨道，然后更高级的轨道可以使用相同的程序生成，但是此时只需最小化剩余的希尔伯特空间中的溢出函数，该空间与所有先前生成的原子轨道所张成的空间正交。
- **参考系统可调：**通过选择参考系统及对应能级，可以提高原子轨道基组在不同环境的迁移性。例如，对于涉及到激发态能级的计算，往往需要在生成数值原子轨道的时候，考虑这组轨道对于非占据态能级的拟合能力，这个时候可以在生成参考体系的时候加入一定数量的非占据能级。

**模拟退火算法 SA（Simulated Annealing）**

我们首先介绍模拟退火法 Simulated Annealing（SA）来最小化溢出函数 [1]，从而优化球贝塞尔函数的系数 $c_{\mu q}$。在模拟退火算法中我们用的是 Metropolis 的算法，需要设置一个初始温度，并且指定温度下降的方式。代码部分我们之后会介绍，这里先简要介绍一下算法。

**初始系数：**如果没有特别指定，一开始的系数 $c_{\mu q}$ 是随机产生的，每一步 SA 会更新其中一个系数，当溢出函数被改变的数值在允许接收的范围内时，参数 $c_{\mu q}$ 会被更新。

**优化过程：**当要优化的所有轨道的所有参数 $c_{\mu q}$ 都被更新过一次之后，我们称为一步优化。我们发现大约在每个温度行走 500 ~ 1000 步就已经足够产生好的原子轨道。最开始的温度我们一般取为 1.0e-5，事实上我们发现即便初始温度取得更小，最后的结果也是不错的。在每一步优化之后，我们将温度乘上一个比率，来达到降温的目的从而得到新的温度，这个参数我们一般取为 0.5 0.8，最后的温度一般降到 1.0e-9 1.0e-12。接收的概率可以控制在 0.2 0.4 范围内。

**能量截断值：**这里的能量截断值实际上是能量截断值，我们知道每个平面波 $e^{iq}r$ 都对应一个二阶求导后的动能 $q^2/2$。同样的，每个球贝塞尔函数 $j_i(qr)$ 也有一个对应的动能是 $q^2/2$。在平面波计算里我们通过能量截断值来控制平面波的数量，从而控制计算精度。在生成数值原子轨道的时候，我们也通过能量截断值来控制球贝塞尔函数的个数，从而控制生成的数值原子轨道的精度。

对于 ABACUS 的平面波计算和产生球贝塞尔函数 $j_i(qr)$ 个数的计算，我们建议采用同样的能量截断值，该截断值取得越高，基矢量个数越多同时越组也越完备。对于采用腰势的平面波计算，15 30 个球贝塞尔函数已经足够得到质量比较高的数值原子轨道。

**数值原子轨道的二阶可导：**除了以上操作之外，还有一个需要注意的地方。为了使数值原子轨道的动能积分是定义的，需要使原子轨道的二阶导数连续。这可以通过将原子轨道的径向部分乘以一个平滑函数 $g(r)$ 来实现

$$g(r) = 1 - \exp\left[-\frac{(r - r_c)^2}{2\sigma^2}\right],$$

该径向轨道会在数值原子轨道的截断值以内起作用，使其在截断值处 2 阶连续，超过阶段半径之后就严格截断。在我们的测试中，我们发现参数 σ 对轨道的质量影响不大，所以我们一般固定 σ 为常数 0.1。

**数值原子轨道动能和再次使用模拟退火算法：**如果仅仅最小化溢出函数值，原子轨道的径向部分经常会出现震荡，原因是震荡有时有利于溢出函数的值尽可能的低，但震荡厉害的函数往往能高，可移植性较差。因此，为了去除这些实际非物理的震荡，在算法中我们还定义了原子轨道的动能：

$$T_\mu = \sum_q c_{\mu q}^2 q^2/2 + k,$$

并再次通过模拟退火算法最小化原子轨道的动能，其中 $k$ 则是模拟退火的一个惩罚函数：

$$k = \begin{cases} 0, & S/S_0 - 1 < \Delta \\ \infty, & S/S_0 - 1 > \Delta \end{cases},$$

这里 $S_0$ 是在优化动能之前所得到的数值原子轨道算出来的溢出函数的值，而 $S$ 则是当前在参数 $c_{\mu q}$ 下面的溢出函数的值。我们发现参数 $\Delta = 0.002 \sim 0.005$ 足够使得原子轨道平滑了。轨道动能也是通过同样的模拟退火算法得到，经过动能优化之后，轨道的形状会变平滑，同时溢出函数的值也会相比较有可能优化的轨道会有一点增加，但是对精度的影响是很小的。

优化动能用的模拟退火算法几乎和优化溢出函数一样，除了一点点参数的区别。目标函数选择的是所有轨道中最大的动能，而不是像溢出函数一样取平均值。对于每个轨道来说，温度是不一样的，因为每个轨道会有一个不同的动能，在这步动能优化中，我们发现结果对初始的温度不敏感，但最终达到的温度必须取得足够小。实际中我们发现只需要 100 200 步，动能就可以下降得很快。所以一般来说优化动能的步数就取得比优化溢出函数要少。

### 3.7.2 ABACUS 中产生数值原子轨道的具体流程

注：本文档介绍的是模拟退火算法 SA，不适用于基于 Pytorch 的 PyTorch-Gradient 方法，关于这两个算法，可以参考本数值原子轨道中文文档系列的第三篇文档。

#### 平面波计算

简化计算量：前文我们提到的溢出函数公式，如果把投影子 $\hat{P}$ 带进公式，则可以发现我们不需要在计算溢出函数的时候显示的包含任何基矢量（例如平面波）的操作。实际上，我们只需要先算好数值原子轨道 $|\phi_\mu\rangle$ 和电子波函数 $|u_i\rangle$ 的重叠矩阵（Overlap Matrix）$\langle u_i|\phi_\mu\rangle$ 并存下来，以及数值原子轨道之间的重叠矩阵 $\langle\phi_\mu|\phi_\nu\rangle$ 存下来就可以算溢出函数了。更进一步，我们意识到数值原子轨道 $|\phi_\mu\rangle$ 的径向部分在程序里是写成了球贝塞尔函数的线性组合，根据线性叠加的原理，我们只需要计算球贝塞尔函数和电子波函数的内积，以及球贝塞尔函数之间的内积，并把这些结果存下来，就可以优化系数 $c_{\mu q}$ 了！注意这里指标准遍历体系的所有电子态，包括所有的 $k$ 点以及能带对应的电子波函数。

换句话说，只需要存储线性代数里的内积结果而不是每个波函数在基矢量下的展开系数就可以优化我们想要的系数，因此数值原子轨道的优化过程可以极大提速！

采用 ABACUS 做平面波计算：

因此，第一步我们先调用 ABACUS 主程序，计算出以上 overlap 矩阵，首先需要准备 INPUT 文件如下：

```
INPUT_PARAMETERS
suffix              H
pseudo_dir          /home/liuyu/github/abacus-develop/tests/PP_ORB
wannier_card        INPUTw
calculation         scf
ks_solver           dav
nspin               1
lmaxmax             1

symmetry            0
nbands              10

ecutwfc             100
scf_thr             1.0e-7
scf_nmax            1000
```

```
16
17 smearing_method     gauss
18 smearing_sigma      0.01
19
20 mixing_type         pulay
21 mixing_beta         0.4
22 mixing_ndim         8
23 printe              1
24
25 bessel_nao_ecut     100    // same as ecutwfc
26 bessel_nao_rcut     6      // rcut
```

如上所示，该 INPUT 与平常的平面波计算相比，只有 3 个参数（已标红）需要注意：

- wannier_card：指定 wannier 相关参数的输入文件名 INPUTw
- bessel_nao_ecut：球贝塞尔函数的截断能，根据公式 $\sqrt{E_{cut}} \times r_{cut}/\pi$ 来决定求贝塞尔函数的数量
- bessel_nao_rcut：数值原子轨道的截断半径 $r_{cut}$

此外，我们还需要准备输入文件 INPUTw 如下（将来计划会把 INPUTw 文件去掉，把参数加入 INPUT 里）：

```
1 WANNIER_PARAMETERS
2 out_spillage 2           // output overlap
3 spillage_outdir OUT.H    // could be the same as OUT.\${suffix}
```

通过 ABACUS 做平面波基矢量计算可以计算出：

1. 电子波函数 $\{\Psi_i\}$ 和球贝塞尔函数 $j_l(qr)$ 之间的 overlap $\langle\Psi_i|j_l(qr)\rangle$；
2. 球贝塞尔函数之间的 overlap $\langle j_{l_2}(q_2r)|j_{l_1}(q_1r)\rangle$；
3. 电子波函数 $\{\Psi_i\}$ 之间的 overlap $\langle\Psi_i|\Psi_i\rangle$。

这些输出将存在文件 our.f(suffix)/orb_matrix.o.dat 中，而这个文件将被读入进行数值原子轨道的生成。

### 编译 SIAB

SIAB 的全称是 Systematically Improvable Atomic orbital Basis generator based on spillage formula，目前是包含在 ABACUS 软件里的一个独立程序，可用来读入以上 ABACUS 产生的数据文件并且生成数值原子轨道，具体编译方法如下：

拉取 deepmodeling/abacus-develop 仓库（github 和 gitee 二选一即可，国内建议用 gitee）

```
1 # github 仓库
2 \$ git clone -b develop https://github.com/deepmodeling/abacus-develop.git
3
4 # gitee 仓库
5 \$ git clone -b develop https://gitee.com/deepmodeling/abacus-develop.git
```

编译 SIAB 模拟退火（SA）方法生成数值原子轨道的代码在 SIAB 文件夹里，采用 C++ 编写，目前 Makefile 仅支持 intel 编译器。

SIAB 程序有串行（make s）和并行（make p）两种版本，一般情况下串行版本即可满足使用需求。

```
1 \$ cd abacus-develop/tools/SIAB/SimulatedAnnealing/source
2 \$ make clean
```

```
\$ make s
```

编译成功后，在目录 abacus-develop/tools/SIAB/SimulatedAnnealing/source 下可以看到生成数值原子轨道的可执行程序 SIA_s.exe，这里 s 代表编译串行版 (serial)。此外，对于并行版本，SIAB 还可以结合 MPI 编译并行版，通过命令 make p 指令得到 SIA_p.exe 可执行程序，这里 p 代表 parallel，但注意并行版只对某些特定算法进行并行，之后对代码进行一个更好整理之后我们会给出并行版的教程。

### 运行例子

进入 tests_s 目录，可以看到 H-6-0.6.15.dat 文件，这是用来生成数值原子轨道所需要准备的输入文件，从 ABACUS 的平面波自洽迭代计算后获得，现在该文件已经改名为 OUT.$suffix/orb_matrix.0.dat。

此外，还有一个输入文件 INPUT 如下：

```
<PW_QS>
1               // bool: calculate the spillage. 1/0
0               // bool: restart or not. 1/0
1               // bool: output the file. 1/0
1               // integer: number of structures.
H-6-0.6.15.dat  // Each structure's file name occupies one line
</PW_QS>

<PARALLEL>
1               // integer: number of k points (for parallel version)
1               // integer: number of pools (for parallel version)
</PARALLEL>

<BLOCK_NE>
15              // can be ignored now
</BLOCK_NE>

<SCHEME_VALUE>
2               // The way to calculate spillage, 2 means averaged spillage
<SCHEME_VALUE>

<METROPOLIS>
1.0e-8          // real: start temperature for minimizing spillage
0.5             // real: cooling rate for the temperature in minimizing spillage
2               // integer: number of temperatures (spillage)
500             // integer: number of steps per temperature (spillage)

1.0e-2          // real: starting temperature for the kinetic energy
0.8             // real: cooling rate for temperature in minimizing kinetic energy
1               // integer: number of temperatures (kinetic)
100             // integer: number of steps per temperature (kinetic)

0.05            // real: delta kappa for kinetic energy
49              // integer: selectly output information every n steps

```

```
100             // integer: change accept rate every n steps, can be ignored now
0.4             // real: acceptance rate (high), can be ignored now
0.2             // real: acceptance rate (low), can be ignored now

50              // real: max allowed kinetic energy (Rydberg)
0.01            // real: 'dr' for the 1d r grid to compute kinetic energy
1               // integer: 1: Kin 2: Ecut, can be ignored now
</METROPOLIS>

<BANDS>
1               // bool: to control the number of bands
1               // int, the start band index(>0).
1               // int, the end band index(<provied bands).
<BANDS>

<OPTIMIZE>
2               // Number of levels.
# label / na / skip / lmax / each L /
01 2 new 1 1 1
01 2 new 1 1 1
01 2 new 1 1 1
</OPTIMIZE>

<PLOT>
0.01    //dr(a.u.) of uniform mesh. Attention!!dr will affect kinetic energy minmized largely.
-6      //xmin
1       //zed, chosen as valence charge.
0.01    //dx
6.0     //xmax
</PLOT>

<CAL_C4>
0       // print out the C4, this section can be ignored now
2
./FILE/Si-S.ORBITAL
0
./FILE/Si-P.ORBITAL
1
</CAL_C4>

<TEST>
0                       // for tests, can be ignored now
14.0                    // rcut, only useful for test program
0.01                    // dr, for simpson integral
2                       // test eigenvalue index
2                       // lmax
</TEST>
```

C4：代表 4 个重要参数：α,ξ,l,nq ，α 是原子种类的指标，ξ 是角动量 l 对应的径向函数的个数，nq 是对应的球贝塞尔函数的个数。

该输入文件包含 SIAB 程序需要读取的一系列参数，基本上重要参数的说明都有列上去，一些可忽略的参数可以先不用设置，之后我们会推出更详细的关于此输入文件的说明文档。

准备好该输入文件后，可以通过如下命令执行 SIAB 程序：

```
\$ ~/abacus-develop/tools/SIAB/SimulatedAnnealing/source/SIA_s.exe
```

运行完毕后，可以得到一系列输出文件，其中 ORBITAL_1U.dat 则是我们需要的数值原子轨道文件。我们可以通过文件的前几行获取关键信息：这是 H 原子的轨道文件，平面波截断能为 100 Ry，截断半径为 6 Bohr，最大角动量为 1，角动量 s 和 p 的径向函数的个数都为 2，即 2 条 s 轨道和 6 条 p 轨道，共 8 条轨道。为了体现这些关键信息，在使用轨道的时候我们建议重命名该文件为 H_gga_100Ry_6au_2s2p.orb。

```
-------------------------------------------------------------------------
Element                      H
Energy Cutoff(Ry)          100
Radius Cutoff(a.u.)          6
Lmax                         1
Number of Sorbital-->        2
Number of Porbital-->        2
-------------------------------------------------------------------------
SUMMARY  END
```

### 3.7.3 ABACUS 中数值原子轨道的生成脚本

我们需要采用平面波计算不同键长的二聚体，并且存在不同层级的轨道，因此也可以通过一个脚本来实现自动化整个流程：abacus-develop/tools/SIAB/Generate_Orbital_AllInOne.sh。该脚本需要读入一个输入文件 ORBITAL_INPUT 如下：

```
#1.exe_dir
#-------------------------------------------------------------------------------
EXE_mpi       mpirun -np 4

EXE_pw        /home/liuyu/github/abacus-develop/build/abacus

EXE_orbital   /home/liuyu/github/abacus-develop/tools/SIAB/SimulatedAnnealing/source/SIA_s.exe

#-------------------------------------------------------------------------------
#( In this part, the direction of the two used exe is provided )

#2.electronic calculatation
#-------------------------------------------------------------------------------

targets     14_Si_60   # element
ref_bands   4   6      # reference bands
nbands      8   12     # num of bands for calculate
Ecut        60         # cutoff energy (in Ry)
Rcut        6   7      # cutoff radius (in a.u.)
```

```
Pseudo_dir  /home/liuyu/github/abacus-develop/tests/PP_ORB
Pseudo      Si_ONCV_PBE-1.0.upf
smearing_sigma       0.01          # energy range for gauss smearing (in Ry)
#-----------------------------------------------------------------------------------
#( In this part , some parameters of calculating are given )

#3.structure information
#-----------------------------------------------------------------------------------
#the Bond Length list for each STRU
BLSTRU1    1.8 2.0
BLSTRU2    2.4
# if skip the orbital optimization for each STRU (1:skip, 0:not skip).
SkipSTRU   0   0
# set the objective structure list to fit for orbital optimization
ListSTRU   dimer    trimer
#-----------------------------------------------------------------------------------
#(In this part, it gives us the bond length of the reference system(in angstrom) )

#4.orbital calculatation
#-----------------------------------------------------------------------------------

maxL       2     # the max angular momentum
Level      2  3  # num of levels to generate orbitals(<=5)
#(num)  (the max ang) (num of S) (num of P) (num of D) (num of F) (num of G)
level1     1    1    1
level2     2    1    1    1
level3     2    1    1    1
#-----------------------------------------------------------------------------------
#( In this part, some information of orbital is given )

#5.Metropolis parameters (in most cases do not need to change)
#-----------------------------------------------------------------------------------
Start_tem_S  1.0e-3  # start temperature for optimize Spillage (default 1.0e-4)
Start_tem_K  1.0e-1  # start temperature for optimize Kinetic  (default 1.0e-2)
Step_S       20      # num of steps for optimize Spillage (default 20)
Step_K       10     # num of steps for optimize Kinetic  (default 15)
Delta_kappa  0.01    # delta kappa (default 0.01)
#-----------------------------------------------------------------------------------
#( In this part , some parameters of Metropolis is given. In most cases , they
#do not need to be changed , only when you run into a situation , that the
#Kinnetic energy is larger than the  maximum value allowed , you can enlarge
#the start temperature appropritely , or you can enlarge the delta_kappa, e.g.
#start_tem_k 1.0e-2 to 5.0e-1, delta_kappa 0.01 to 0.02. more steps can make
#the orbitals better , too)
SUMMARY  END
```

该输入文件主要分为 5 个部分：

**exe_dir**

EXE_mpi：MPI 并行计算命令，这里我们采用的是 4 核 MPI 并行进行 dimer（或者 trimer）的平面波计算。

EXE_pw：ABACUS 可执行程序的绝对路径。

EXE_orbital：数值原子轨道生成程序 SIA_s.exe 的绝对路径。

**electronic calculatation**

targets：可以任取，一般格式是 $ 原子序数 _$ 元素名 _$ 截断能。

ref_bands：参考体系的能带数，取值为电子总数的一半。注意这里有多个取值，原因是我们选择了多组参考构型，既有 dimer，也有 trimer。

nbands：平面波计算中的实际能带数，必须大于 ref_bands，以保证包含非占据态。同理，由于多组参考构型，nbands 也有多个取值。

Ecut：平面波截断能，单位为 Ry。

Rcut：数值原子轨道的截断半径，单位为 Bohr。这里的多个取值表明，采用相同的一套参数，生成多个不同截断半径的数值原子轨道文件，根据第一部分的基础知识，截断半径会极大地影响数值原子轨道的质量，因此该参数可以帮助我们一次性生成多个不同截断半径的数值原子轨道文件。

Pseudo_dir：平面波计算采用的赝势文件所在文件夹的绝对路径。

Pseudo：平面波计算采用的赝势文件名。

smearing_sigma：平面波计算采用 Gaussian smearing 的展宽，单位为 Ry，一般取 0.01 即可。

**structure information**

BLSTRU1：第一组参考构型的键长，单位为埃。如上所示，第一组参考构型包含两个构型，键长分别为 1.8 和 2.0 埃。

BLSTRU2：第二组参考构型的键长，单位为埃。如上所示，第二组参考构型包含一个构型，键长为 2.4 埃。以此类推，可以存在 BLSTRU3、BLSTRU4 等等。

SkipSTRU：是否跳过该组参考构型的优化，一般而言，都取为 0 即可。

ListSTRU：指定 BLSTRU1、BLSTRU2 等等参考体系的具体构型，这里说明 BLSTRU1 是 dimer，BLSTRU2 是 trimer。如果取值缺省（第一个不可缺省），则默认为 dimer。

**orbital calculatation**

maxL：最大角动量。

Level1：根据上文可知，轨道可以分为多个层级，这里 level1 是 SZ 层级，最大角动量为 1，角动量 s 和 p 的径向函数的个数为 1 和 1，即 1 条 s 轨道和 3 条 p 轨道。

Level2：同上，这里 level2 的最大角动量为 2，角动量 s，p 和 d 的径向函数的个数为 1，1 和 1，即 1 条 s 轨道，3 条 p 轨道和 5 条 d 轨道。level1 和 level2 合在一起，是 DZP 层级。

Level3：同 Level2。

Level：BLSTRU1 和 BLSTRU2 分别负责一部分轨道的生成，Level 的 2 个取值对应的是 BLSTRU1 和 BLSTRU2 的 endlevel，即 BLSTRU1 用于 level1 和 level2 轨道的生成，BLSTRU2 用于 level3 轨道的生成。

**Metropolis parameters**

如注释所说，这部分参数实际上大多数情况下不需要改变。

Start_tem_S：优化 Spillage 的初始温度，更高的初始温度可以搜索更大的温度范围。

Start_tem_K：优化动能的初始温度，更高的初始温度可以搜索更大的温度范围。

Step_S：优化 Spillage 的步数。

Step_K：优化动能的步数。

Delta_kappa：模拟退火的惩罚函数 κ 所用的参数 Δ 。

准备好以上输入文件后，通过以下命令即可开始生成轨道：

```
\$ ~/abacus-develop/tools/SIAB/Generate_Orbital_AllInOne.sh ORBITAL_INPUT
```

### 3.7.4 ABACUS 中数值原子轨道的测试流程

在生成轨道之后，为了保证基于数值原子轨道基组的第一性原理计算拥有与平面波基组相当的精度，我们需要针对该元素的一些常见的晶体结构做一些验证性计算。

以 Al 元素为例，我们可以选择 simple cubic（sc），body center cubic（bcc），face center cubic（fcc）和 hexagonal close-packed（hcp）这 4 种常见的晶体结构，采用平面波（Plane wave）基组和数值原子轨道基组计算这些晶体结构的平衡晶格常数、体弹性模量等基态性质，评估两种基组之间的误差随着截断半径、最大角动量以及轨道数量的收敛趋势，选择合适的数值原子轨道文件。例如以下就是用 ABACUS 的平面波和数值原子轨道 DZP 计算的铝的四种体相材料的体积-能量曲线，来自文献 [1]。从图中可以看到，DZP 轨道描述不同体相的铝的相对能量还是精度较高的。细心的读者会发现纵坐标给出的是绝对能量，因为两种基矢量采用了相同的赝势，但左图给出的能量更低，这是因为平面波更加完备，所以根据 Honhenberg-Kohn 定理，体系总能量的密度泛函可以在更完备的基矢量下得到更好的收敛，即总能量更低。一般来说，平面波得到的总能量比 DZP 轨道小一些，大概在 0.1-0.2 eV/atom，是比较合理的。如果采用了同样的赝势，但两种基矢量给出的能量差别远远大于这个数值，则要检查一下是不是数值原子轨道产生出了问题。最后，大家有任何问题，欢迎写邮件到文档开头邮箱。

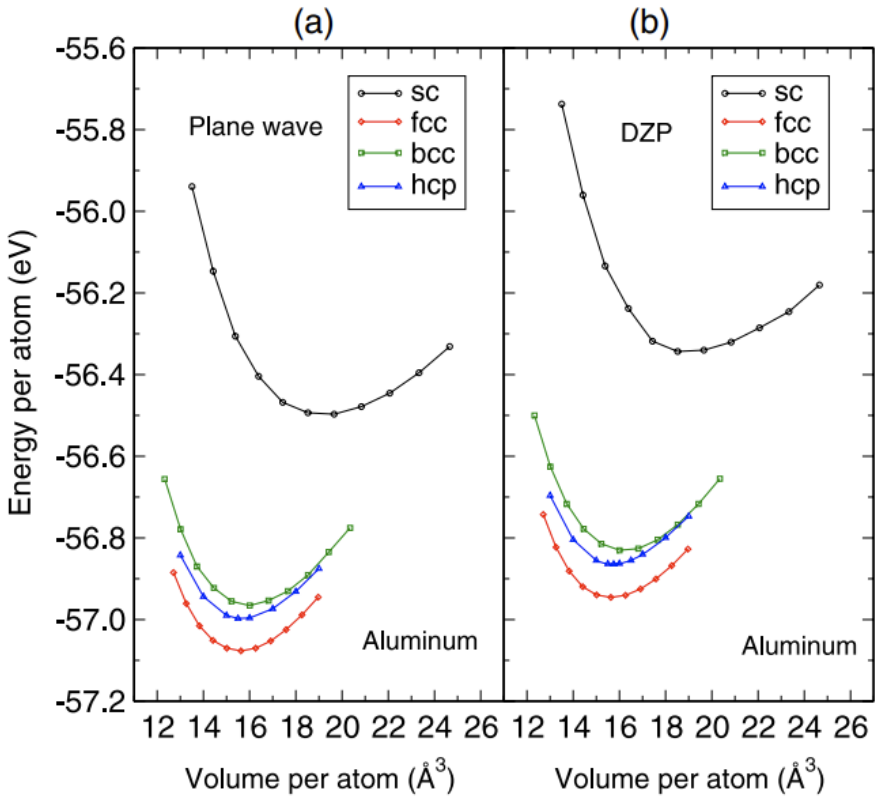



**Figure 8.** Comparison of the total energies of Al sc, fcc, bcc, hcp structures as functions of volume per atom (a) using the plane wave basis (b) using the atomic DZP basis.

### 3.7.5 参考文献

[1] M. Chen, G.-C. Guo, and L. He, Systematically Improvable Optimized Atomic Basis Sets for Ab Initio Calculations, J. Phys.: Condens. Matter 22, 445501 (2010).

[2] M. Chen, G.-C. Guo, and L. He, Electronic Structure Interpolation via Atomic Orbitals, J. Phys.: Condens. Matter 23, 325501 (2011).

## 3.8 数值原子轨道（三）：产生高精度数值原子轨道

### 3.8.1 PTG（PyTorch Gradient）方法

这篇文章是数值原子轨道系列的第三篇文档，除了第二篇文档提到的模拟退火算法之外，还可以使用 Pytorch 中的自动微分算法来最小化溢出函数。首先定义损失函数如下，并可以证明它与溢出函数是等价的：

$$\Delta PSI \overset{\text{def}}{=} \frac{1}{N_n}\sum_{n=1}^{N_m} \| |\Psi_n\rangle - |\tilde{\Psi}_n\rangle \|^2,$$

其中

$$|\tilde{\Psi}_n\rangle \overset{\text{def}}{=} \hat{P}|\Psi_n\rangle,$$

且 $\hat{P}$ 是由所有原子轨道张成的投影子，即 $\hat{P} = \sum_{\mu v} |\phi_\mu\rangle S^{-1}_{\mu v} \langle\phi_v|$。由 $\hat{P}^2 = \hat{P}$ 及 $\langle\Psi_n|\Psi_n\rangle = \delta_{mn}$ 可知，此处的损失函数 $\Delta PSI$ 与溢出函数是等价的。

### 3.8.2 PTG_dpsi（PyTorch Gradient with dpsi）方法

为了增加局域轨道做电子结构计算时的精度，损失函数的定义还可以被拓展，即在其中加入波函数的梯度，将总的损失函数定义为：

$$\Delta PSI \overset{\text{def}}{=} \frac{1}{N_n}\sum_{n=1}^{N_m} [\| |\Psi_n\rangle - |\tilde{\Psi}_n\rangle \|^2 + \| |\nabla\Psi_n\rangle - |\nabla\tilde{\Psi}_n\rangle \|^2].$$

由于投影波函数 $\tilde{\Psi}_n$ 是数值原子轨道的线性组合：

$$|\tilde{\Psi}_n\rangle = \sum_\mu a_{\mu n} |\phi_\mu\rangle,$$

其中系数的表达式为

$$a_{\mu n} = \sum_\nu S^{-1}_{\mu\nu} \langle\phi_\nu|\Psi_n\rangle.$$

那么投影波函数的梯度可以表示为数值原子轨道梯度的线性组合：

$$|\nabla\tilde{\Psi}_n\rangle = \sum_\mu a_{\mu n} |\nabla\phi_\mu\rangle.$$

因此，损失函数的梯度项可以转化为：

$$\| |\nabla\Psi_n\rangle - |\nabla\tilde{\Psi}_n\rangle \|^2 = \langle\nabla\Psi_n|\nabla\Psi_n\rangle - \sum_\mu a_{\mu n}\langle\nabla\Psi_n|\nabla\phi_\mu\rangle - \sum_\nu a^*_{\nu n}\langle\nabla\phi_\nu|\nabla\Psi_n\rangle + \sum_{\mu\nu} a^*_{\mu n} a^*_{\nu n}\langle\nabla\phi_\nu|\nabla\phi_\mu\rangle.$$

根据经验，使用这个算法生成的双重-ζ+ 极化 (DZP) 基组，精度与模拟退火法或 PTG 法生成的三重-ζ+ 双极化 (TZDP) 甚至四倍-ζ+ 三重极化 (QZTP) 基组相当。

在平面波计算中，除了数值原子轨道（二）：生成给定模守恒腰势的数值原子轨道提到的 overlap 输出文件 our.$suffix$/orb_matrix.o.dat 之外，overlap 输出文件 our.$suffix$/orb_matrix.1.dat 同样会作为生成轨道的输入文件之一，该文件保存与波函数梯度相关的如下 overlap 项：

1. 电子波函数梯度 $\nabla\Psi_i$ 和局域轨道梯度 $\nabla\phi_\mu$ 的 overlap，也就是电子波函数梯度 $\nabla\Psi_i$ 以及球贝塞尔函数梯度 $\nabla j_l(qr)$ 之间的 overlap$\langle\nabla\Psi_i|\nabla j_l(qr)\rangle$；
2. 球贝塞尔函数梯度 $\nabla j_l(qr)$ 之间的 overlap$\langle\nabla j_{l_2}(q_2r)|\nabla j_{l_1}(q_1r)\rangle$；
3. 电子波函数梯度 $\nabla\Psi_i$ 之间的 overlap$\langle\nabla\Psi_i|\nabla\Psi_i\rangle$。

### 3.8.3 安装 Pytorch

PTG 和 PTG_dpsi 方法采用 python 语言，无需编译，但是依赖 pytorch 包，下面介绍利用 conda 安装 pytorch 的方法：

```
# 确定 conda 版本
\$ conda -V
conda 4.8.3

# 确定 python3 版本
\$ python3 -V
Python 3.5.2

# 创建 python 环境
\$ conda create -n pytorch python=3.5

# 激活环境（每次使用 pytorch 需要激活该环境）
\$ source activate pytorch

# 安装 pytorch
\$ conda install pytorch torchvision torchaudio cpuonly -c pytorch

# 安装依赖库
\$ pip3 install --user scipy numpy
\$ pip3 install --user torch_optimizer

# 退出 python 环境（使用完毕后）
\$ source deactivate
```

### 3.8.4 产生高精度数值原子轨道流程

首先下载 PTG_dpsi 仓库

```
# github 仓库
git clone -b main https://github.com/abacusmodeling/ABACUS-orbitals
```

接着准备输入文件 SIAB_INPUT

```
#---------------------------------------------------------------------------
#1. CMD & ENV
EXE_mpi    mpirun -np 4
EXE_pw     /home/liuyu/github/abacus-develop/build/abacus
EXE_opt    /home/liuyu/github/ABACUS-orbitals/SIAB/opt_orb_pytorch_dpsi/main.py
#---------------------------------------------------------------------------
#2. Electronic calculatation
 element     Si          # Element Name
 Ecut        100         # in Ry
 Rcut        6 7         # in Bohr
 Pseudo_dir  /home/liuyu/github/abacus-develop/tests/PP_ORB
 Pseudo_name Si_ONCV_PBE-1.0.upf
 sigma       0.01        # energy range for gauss smearing (in Ry)

#---------------------------------------------------------------------------
#3. Reference structure related parameters for PW calculation
#For the built-in structure types (including 'dimer', 'trimer' and 'tetramer'):
#STRU Name   #STRU Type  #nbands #MaxL   #nspin  #Bond Length list
 STRU1       dimer       8       2       1       1.8 2.0 2.3 2.8 3.8
 STRU2       trimer      10      2       1       1.9 2.1 2.6

#---------------------------------------------------------------------------
#4. SIAB calculatation
 max_steps    200
# Orbital configure and reference target for each level
#LevelIndex   #Ref STRU Name    #Ref Bands   #InputOrb    #OrbitalConf
 Level1       STRU1             4            none         1s1p
 Level2       STRU1             4            fix          2s2p1d
 Level3       STRU2             6            fix          3s3p2d

#---------------------------------------------------------------------------
#5. Save Orbitals
#Index       #LevelNum    #OrbitalType
 Save1       Level1       SZ
 Save2       Level2       DZP
 Save3       Level3       TZDP
```

该输入文件同样包含 5 个部分：

**CMD & ENV**

EXE_mpi：MPI 并行计算命令，这里我们采用的是 4 核 MPI 并行进行 dimer（或者 trimer）的平面波计算。
EXE_pw：ABACUS 可执行程序的绝对路径。
EXE_opt：数值原子轨道生成程序 PTG_dpsi 的绝对路径。

**Electronic calculatation**

element：元素名。
Ecut：平面波截断能，单位为 Ry。

Rcut：数值原子轨道的截断半径，单位为 Bohr。这里的多个取值表明，采用相同的一套参数，生成多个不同截断半径的数值原子轨道文件，根据第一部分的基础知识，截断半径会极大地影响数值原子轨道的质量，因此该参数可以帮助我们一次性生成多个不同截断半径的数值原子轨道文件。

Pseudo_dir：平面波计算采用的赝势文件所在文件夹的绝对路径。

Pseudo：平面波计算采用的赝势文件名。

sigma：平面波计算采用 Gaussian smearing 的展宽，单位为 Ry，一般取 0.01 即可。

**Reference structure related parameters for PW calculation**

STRU Name：参考构型组别 STRU1、STRU2、STRU3.....

STRU Type：参考体系构型，一般可取 dimer，trimer，tetramer。

nbands：平面波计算中的实际能带数，必须保证包含非占据态。

MaxL：最大角动量。

nspin：自旋量子数。

Bond Length list：键长取值列表，单位为埃。以 STRU1 为例，这一组共有 5 个不同键长的 dimer 作为生成轨道的参考构型。

**SIAB calculatation**

max_steps：优化步数

LevelIndex：轨道分层的 index，取 Level1、Level2、Level3.....

Ref STRU Name：该层级的轨道采用哪一组参考构型生成。以上面的输入参数为例，Level1 和 Level2 的轨道采用 STRU1 组别的 dimer 参考构型生成，Level3 的轨道采用 STRU2 组别的 trimerr 参考构型生成。

Ref Bands：拟合参考体系的能带数，若输入“auto”则自动拟合所有基态。

InputOrb：是否在已有的轨道基础上生成新轨道，一般 Level1 设为 none，即没有旧轨道；Level2 一般以 Level1 生成的轨道为基础继续生成轨道，Level3 往后同理。

OrbitalConf：轨道的具体配置，这里 Level1 是 SZ 层级，Level2 是 DZP 层级，Level3 是 TZDP 层级，实际上还可以继续设置更高层级的轨道。

**Save Orbitals**

Index：取值 Save1、Save2、Save3.....

LevelNum：将特定 Level 的轨道保存到一个轨道文件中。以上面的输入参数为例，分别将 Level1、Level2、Level3 对应的轨道保存到单独的轨道文件中。

OrbitalType：即保存的轨道文件中，该轨道基组所属的层级。以上面的输入参数为例，分别为 SZ、DZP、TDZP。

准备好输入参数文件 SIAB_INPUT 后，运行如下命令即可开始生成轨道：

```
\$ python3 ~/github/ABACUS-orbitals/SIAB/SIAB.py SIAB_INPUT
```

程序正常结束后，轨道会分别保存在 Orbital_Si_SZ、Orbital_Si_DZP、Orbital_Si_TZDP 中，并自动命名成标准格式（如 Si_gga_6au_100Ry_2s2p1d.orb）。

### 3.8.5 参考文献

[1] Peize Lin, Xinguo Ren and Lixin He, Strategy for constructing compact numerical atomic orbital basis sets by incorporating the gradients of reference wavefunctions, Phys. Rev. B 103, 235131 (2021).

# 第 4 章 基础功能

## 4.1 Kohn-Sham 密度泛函理论计算

### 4.1.1 平面波计算与收敛性测试

**背景介绍**

ABACUS(原子算筹) 作为一款国产的电子结构软件，可用于对材料进行密度泛函理论（Density Functional Theory，简称 DFT）的第一性原理（first-principles）计算，其主要核心功能之一是电子自洽迭代计算 (Self Consistent Field，简称 SCF)，在给定材料微观层面的晶胞和原子位置的条件下，我们可以通过 SCF 流程的计算获得电子结构的总能量、能级、电子波函数、电子密度等关键信息，这些信息可用于进一步计算得出材料体系的其它性质。然而，对于初学者来说，初次运行 ABACUS 难免不熟悉输入参数，容易出错，要熟练掌握好 SCF 计算要领也颇具难度。因此，本文将主要针对初学者，详细介绍（1）如何采用 ABACUS 设置合理的输入参数完成一个 SCF 计算，以及（2）如何对体系做收敛性测试，以便确定所有计算参数。这里收敛性测试包括在平面波（Plane Wave，简称 pw）基矢量下面对 ecut（电子动能截断值）进行测试，以及对布里渊区的 k 点个数进行测试来确定这些参数。

对密度泛函理论计算不太了解的读者，我们想先介绍密度泛函理论的三个特点。第一，只要给定原子坐标和种类，密度泛函理论就可以用于预测该体系的许多物理和力学等性质。第二，对于不同的原子坐标排列方式以及不同元素种类，密度泛函理论软件提供了一系列参数用于调节计算的精度和效率，只有将参数设定在某一些正确的范围之内，得到的结果才有意义。反之，如果参数过于粗糙，则得到的结果可能也不收敛，因此不具有价值。第三，密度泛函理论计算是一种虽然较为流行的但也相对昂贵的科学计算方法。笔记本电脑只能跑得动几个原子的小体系计算，真正数十个原子乃至更大的体系需要借助超算进行长时间的计算（几个小时甚至几天），而选择不同的参数可能会对最终运行时间有较大的影响。因此，基于以上三点认识，有必要更加细致地掌握密度泛函理论计算的具体参数，下面我们予以介绍。

**输入文件简介**

使用 ABACUS 中进行密度泛函理论或者第一性原理分子动力学计算时，一般需要 STRU、KPT、INPUT 三个最基本的文件作为 ABACUS 的基本输入文件。STRU 文件提供元素、晶格、原子的基本结构信息；KPT 文件提供周期性边界条件下布里渊区采样的网格设置。INPUT 文件主要提供计算所需的各种设置参数，在图一里我们分成了五部分且将在下面详细介绍。

**STRU 和 KPT 文件简单介绍**

**STRU 文件**

STRU 文件提供元素种类、原子质量（SCF 中不会用到）、赝势文件（如果做密度泛函理论计算）、数值原子轨道（如果采用该基组做计算）、晶格矢量、原子位置、原子位置是否可移动等基本结构相关的信息。其中，晶格是晶体中原子有序排列的具体形式，我们可以选取一组晶格矢量来描述晶格。晶格确定后，原子的相对位置也可以通过选取不同坐标系（例如 Cartesian 直角坐标系或者 Direct 晶格坐标系）确定下来。

例如，以下是一个金刚石硅的结构文件（8 个硅原子，周期性边界条件）

```
ATOMIC_SPECIES
Si  28.085  Si_ONCV_PBE-1.0.upf

```

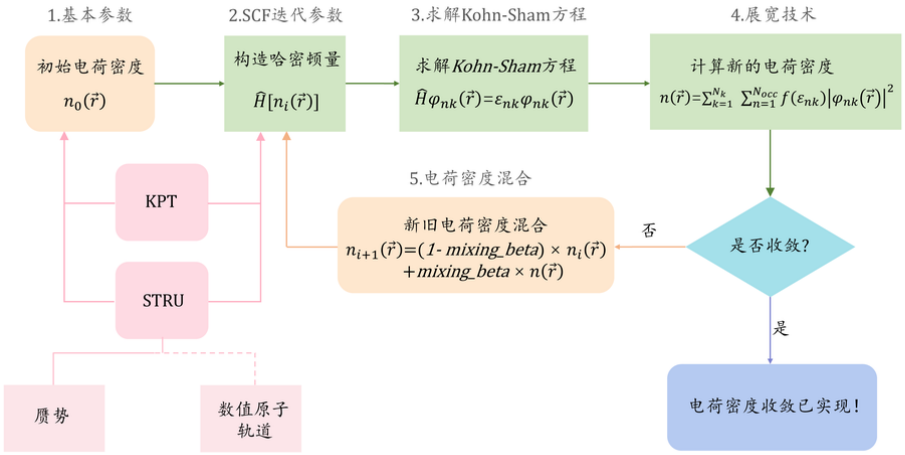



**图 4.1:** 电子自洽迭代计算流程。

```
4  NUMERICAL_ORBITAL
5  Si_gga_8au_60Ry_2s2pid.orb
6
7  LATTICE_CONSTANT
8  1.8897261258369282
9
10 LATTICE_VECTORS
11 5.4307000000    0.0000000000    0.0000000000
12 0.0000000000    5.4307000000    0.0000000000
13 0.0000000000    0.0000000000    5.4307000000
14
15 ATOMIC_POSITIONS
16 Direct
17 Si
18 0.0000000000
19 8
20 0.0000000000    0.0000000000    0.0000000000    1 1 1 mag 0.0
21 0.0000000000    0.5000000000    0.5000000000    1 1 1 mag 0.0
22 0.5000000000    0.0000000000    0.5000000000    1 1 1 mag 0.0
23 0.5000000000    0.5000000000    0.0000000000    1 1 1 mag 0.0
24 0.7500000000    0.7500000000    0.2500000000    1 1 1 mag 0.0
25 0.7500000000    0.2500000000    0.7500000000    1 1 1 mag 0.0
26 0.2500000000    0.7500000000    0.7500000000    1 1 1 mag 0.0
27 0.2500000000    0.2500000000    0.2500000000    1 1 1 mag 0.0
```

- 第 2 行“**元素原子质量赝势**”中的“**原子质量**”在进行 DFT 计算时不可以省略不写，哪怕在 SCF 中不会用到。那么什么时候会用到呢？比如在进行分子动力学（*Molecular Dynamics*，简称 *MD*，在 `INPUT` 的 `calculation` 参数里设置）计算时，涉及运动的积分方程时用的是牛顿的第二定律 $F = ma$，此时会使用原子的质量。另外注意，此行需要提供所使用的“**赝势**”文件名称。
- 第 5 行提供 LCAO 基组所需的“某种元素的数值原子轨道”文件名称，而如果采用 pw 基组计算，不需要写第 4 5 行内容。

- 第 8 行代表晶格整体缩放的一个长度，注意

$$1\ \text{Angstrom} = 1.8897261258369282\ \text{bohr}$$

  这里写 1.8 开头的小数意味着接下来的 `LATTICE_VECTORS` 部分可以写以 Angstrom 为单位的晶格。
- 第 11 13 行表示 $x, y, z$ 方向分别对应的三条晶格矢量。
- 第 16 行“`Direct`”表示给出原子位置的方法是分数坐标（或者称为晶格坐标），常用的还有 `Cartesian`(笛卡尔坐标方法)，就是直角坐标系下的坐标。
- 第 17 行是元素种类的名称。
- 第 18 行设置原子初始磁矩，如果 `INPUT` 里的 `nspin` 参数设为 1，则不考虑磁性，这个参数不起作用。
- 第 19 行表示体系的硅原子个数。
- 第 20 27 行给出所有 8 个硅原子的坐标；前三个数是坐标，之后的 (1 1 1) 三个数代表允许该原子在对应的 $x, y, z$ 对应方向上移动，相反，(0 0 0) 表示不允许在对应的方向上移动；同样，当考虑磁性计算的时候，“`mag 0.0`”指定每个原子的初始磁矩，若设置此参数，则第 18 行的值将被覆盖。

关于赝势，还值得多提一些。平面波基组计算涉及采用描述电子和离子吸引作用的赝势，例如模守恒赝势或者超软赝势。需要注意的是，如果赝势换了，则相应的 ecut 能量截断值也需要重新测试收敛性。而基于局域原子轨道 (Linear Combination of Atomic Orbitals，简称 LCAO) 基组的计算则涉及赝势和数值原子轨道（Numerical Atomic Orbitals，简称 NAO），需要使用者准备好相应文件。这里提到的赝势是一种描述核外电子和离子之间相互作用的近似方法，数值原子轨道是通过数值方法构建的描述电子波函数和电子密度等性质的基函数。需要注意的是，LCAO 计算所采用的数值原子轨道要和对应的赝势匹配，即数值原子轨道是从给定赝势生成出来的，赝势如果改变了，相应的数值原子轨道也要重新生成，才能保证较好的精度。一般来说数值原子轨道的截断半径越长或轨道越多，则基矢量越完备，计算精度越高，但同时计算量也越大。

### KPT 文件

KPT 文件提供周期性边界条件下布里渊区 k 点采样的网格设置

```
1 K_POINTS
2 0
3 Gamma
4 k k k 0 0 0
```

- 第 2 行表示 k 点的总数，如果设置成“0”代表 k 点是自动生成的
- 如果第二行是 0，则第 3 行“Gamma” (Γ-centered Monkhorst-Pack method) 是选择以 Gamma 点为中心的 Monkhorst-Pack 方法划分布里渊区网格，此外还可以使用“mp”方法，即最常用的 Monkhorst-Pack 方法。
- 第 4 行的前三个整数代表网格沿着每个方向划分成几份，后三个数代表网格的平移量，000 即代表不平移。
- 计算立方形晶格时，k 点各方向应取相同个数。在本次计算中，我们会将 k 的各方向从 2 取到 8，测试不同 k 点下体系能量的收敛情况。
- 在后文我们将介绍如何进行 k 点的收敛性测试，当选定一个 k 点后，若体系在某方向扩胞 n 次，该方向 k 点个数大致可以缩小到原来的 $\frac{1}{n}$。对于 N 原子以上的立方大体系，可通过在 `INPUT` 文件中设置参数 `gamma_only` 取 1，即只使用 Gamma 点计算，这个时候计算所需内存会显著下降，计算效率也会有提升。
- 此外，k 点还有所谓的 line mode，或者离散的 k 点模式，若有兴趣的读者可以查看 KPT 的介绍文档 https://abacus.deepmodeling.com/en/latest/advanced/input_files/kpt.html

### INPUT 文件关键输入参数

我们以金刚石结构硅原子体系（8 个原子）作为示例，进行电子自洽迭代计算 (Self Consistent Field，SCF)。接下来让我们需要准备一个 INPUT 文件，我们把一些计算的关键参数分成如下 5 个部分。

### 基本参数

- **suffix**: suffix 是用户可以自定义的后缀，运行 ABACUS 可执行程序之后，输入文件所在的文件夹里会生成一个包含大部分运行信息的 our.suffix 文件夹。例如，在这个例子里我们可以设为 “Si”，运行后就会产生一个 our.si 文件夹。

- **calculation**: 设置本次计算类型，例如本次文档主要展示 “scf”（自洽电子结构计算）。scf(Self Consistent Field)、relax、cell-relax、md(Molecular Dynamics) 是较常用的四类计算。

  1. **scf(自洽电子结构计算)** 是一种用于求解电子基态电子密度的自洽迭代计算方法，通过迭代来持续更新体系的电子密度，以达到收敛条件。通过电子自洽迭代，计算是否达到收敛由参数 scf_thr 决定。
  2. **relax(结构驰豫计算)** 是通过调整体系中原子的位置来达到系统最稳定的状态的计算方法，每一次 relax 中都包含若干步的 scf 计算。在 relax 中，需要设置参数 force_thr_ev (eV/Angstrom)，代表所有原子中最大受力的收敛阈值。而 relax 计算的收敛需要每一步离子迭代时 scf 中的 scf_thr 都满足，以及最后多步离子迭代后 force_thr_ev 也满足。
  3. **cell-relax(结构驰豫计算)** 与 relax 不同的是，其晶胞参数，比如晶格常数、晶格形状也会发生变化。除 force_thr_ev 外，还需要设置参数 stress_thr (kbar)，是晶胞感受到的应力的阈值。
  4. **md(分子动力学)** 基于牛顿力学原理，通过数值积分模拟粒子的运动轨迹，根据原子的势能计算相互作用力，因此势函数的选择比较关键，在 ABACUS 里可以选择 DFT 来计算相互作用力，也可以选择 DP 势函数。

- **symmetry**: 是否考虑对称性，有如下三个选项

  -1: 不进行对称性分析

  0: 仅考虑时间反演对称性

  1: 进行对称性分析

  如果打开对称性（设置为 1），布里渊区 k 点可以根据对称性进行简化处理，若体系有对称性，则可以减少所需计算的 k 点。因为每个 k 点都会进行一次 Kohn-Sham 方程的求解，对称性分析后若 k 点减少则将提升计算效率。本次计算中开启对称性分析，设置 “1”，关于对称性分析的测试还在进一步完善，如果计算结果奇怪，建议设置成 “0” 之后再进行计算，比较结果是否一致。

- **pseudo_dir**: 计算中需要使用赝势来近似离子和电子相互作用势能，为计算提供赝势文件。pseudo_dir 指定 STRU 文件中赝势文件所在的目录。本次计算将赝势和轨道文件都与 INPUT、STRU、KPT 文件放在一起，因此填入 “.”，表示处在当前文件夹中。

  ABACUS 支持的赝势文件——Si_ONCV_PBE-1.0.upf

  “ONCV” 代表模守恒赝势的种类，“PBE” 是采用的交换关联泛函。

- **orbital_dir**: 本次计算将采取 pw（平面波轨道）基组（后文 basis_type 中设置）。若采用 LCAO 基组进行计算，则需要提供作为基组的轨道文件，在 STRU 文件中指定轨道文件所在的目录。若选用 pw 基组，则不需要轨道文件，不需要填写此参数。常用的轨道有：DZP（Double-ζ valence orbitals plus SZ polarization，两条径向轨道和一条杂化轨道）、TZDP（Triple-ζ valence orbitals plus DZ polarization，三条径向轨道和两条杂化轨道）。

  ABACUS 支持的轨道文件——Si_gga_7au_100Ry_2s2p1d.orb

  “gga” 代表 GGA 泛函；“7au” 是轨道半径，半径越大，计算结果会更准确，但花费时间也更久；“100Ry” 是推荐的 ecutwfc 值；“2s2p1d” 表明采用了 DZP 轨道，轨道数目越多，即基矢量更完备，计算也会更准确。

- **basis_type**: 计算的基组（指描述电子波函数的基函数），在 ABACUS 中常用的有两种：

pw: 平面波基组（由于基矢量更完备，pw 计算将更准确，同时计算时间也更长）
LCAO: 局域原子轨道基组（没有 pw 准确，但效率高）

- **ecutwfc**: 平面波函数的能量截止（单位：Ry），在平面波基组里是很重要的一个参数，其大小决定着作为基矢的平面波函数的个数多少，而基矢的多少，也决定了计算精度的高低。
  在本次计算选用的 LCAO 基组中，一般选取轨道文件上推荐的值，设置“100”Ry 即可。

本次计算中选择 pw 基组。如前所述，pw 中不会用到轨道文件。
我们目前填写好的基本参数如下：

```
#Parameters (1.General)
suffix                  Si
calculation             scf
symmetry                1
pseudo_dir              .
orbital_dir             .
basis_type              pw
ecutwfc                 100
```

### SCF 迭代参数

- **scf_nmax**: 针对每个离子构型，scf 电子迭代的最大次数为 scf_nmax，可设为“50”或“100”次。半导体和绝缘体是较容易收敛的体系，一般 20 步 scf 以内即可达到收敛，对于金属体系或者费米面较为复杂的体系，有可能 50 步仍未收敛。注意，如果做 relax, cell-relax, 或者 md 的时候发现有某些 scf 迭代次数达到最大时仍未达到收敛条件，建议先把计算停下来弄清楚原因后再继续算，有可能是因为离子构型或者 k 点等参数的设置不合理引起的。
- **scf_thr**: 代表两个相邻电子迭代步之间的电荷密度误差，也是 SCF 计算中判断是否收敛、完成计算的标准，对于 LCAO，一般设置为“1e-7”，可认为精度足够。对于 pw，建议设置 1e-8 或 1e-9 的精度。

```
#Parameters (2. SCF iterations)
scf_nmax                100
scf_thr                 1e-8
```

### 求解 Kohn-Sham 方程

- **nbands**: 计算的 Kohn-Sham 轨道数目，在本次计算中无磁性，参数 $nspin$ 取 1（默认值），程序目前采取 $0.5 \times \max(1.2 \times \text{occupied_bands}, \text{occupied_bands} + 10)$ 计算 $nbands$。对于 Si，价电子数为 4，每个能级填充可以填充自旋向上和向下 2 个电子，金刚石结构中共有 8 个原子，则

$$nbands = \max(1.2 \times 8 \times 2, 8 \times 2 + 10) = \max(19.2, 26) = 26$$

- **ks_solver**: 在不同基组中展开哈密顿矩阵的对角化方法，对于 pw，可以选择 *cg* (Conjugate Gradient，默认方法)，*bpcg* (还不是太稳定、测试中)，*dav* (Davidson 算法)；对于 LCAO，可以选择 *genelpa* (默认值)，*scalapack_gvx* (Scalable Linear Algebra PACKage)。如果选用 LCAO 基组，$ks_solver$ 可设置为“genelpa”。

```
#Parameters (3. Solve KS equation)
nbands                  26
ks_solver               cg
```

### 展宽技术

- **smearing_method**: 对于金属体系或者费米面附近较复杂的体系，电子自洽迭代方法往往不容易收敛，这个时候可以选择给电子提供一个光滑的占据函数，用来调节程序计算电荷密度和总电子数时候的电子占据函数，这对于费米面附近的电子态尤为重要。我们这里称为“smearing method”，或者称为展宽方法。具体来说，对于金属体系可以选择 `mp` (methfessel-paxton)，`mp2` (2-nd methfessel-paxton)，也可以使用 `gauss` (也可以写作 `gaussian`)。因为我们计算金刚石 Si 具有半导体性质，所以 smearing 方法基本不起作用，默认可以选择 `gauss`。此外，非导体计算可以选择 `fixed`，半导体和非导体也可以选择 `fd` (Fermi-Dirac) 方法。
- **smearing_sigma**: 给定展宽方法的能量范围（单位：Ry），默认是 0.015 Ry，我们按照通常情况给定“0.01”。

```
#Parameters (4.Smearing)
smearing_method         gauss
smearing_sigma          0.01
```

### 电荷密度混合

- **mixing_type**: 进行新旧电荷密度混合时选用的方法，可选 “plain”(简单电荷密度混合方法，如图 1 所示)、“pulay”、“broyden”，默认选择 “broyden” 算法。
- **mixing_beta**: 新旧电荷密度混合时，新电荷的比例。这个参数越大，收敛越快，但不收敛的风险也会变大，不过取值大小并不会改变基态能量的结果。本计算中设置此参数为 “0.7”。
- **mixing_ggo**: 电子迭代过程中，可能出现混合电荷密度不收敛的情况，此时可以通过一种叫 Kerker 预处理的方法来加快收敛。本计算关闭 Kerker 预处理，设置此参数为 “0”。

```
#Parameters (5.Mixing)
mixing_type             broyden
mixing_beta             0.7
mixing_gg0              0
```

到此，我们完成了对主要计算参数的设置，此时集成了以上主要参数的 INPUT 文件如下：

```
#Parameters (1.General)
suffix                  Si
calculation             scf
symmetry                1
pseudo_dir              .
orbital_dir             .
basis_type              pw
ecutwfc                 100

#Parameters (2. SCF iterations)
scf_nmax                100
scf_thr                 1e-8

#Parameters (3. Solve KS equation)
nbands                  26
ks_solver               cg

#Parameters (4.Smearing)
```

```
smearing_method         gauss
smearing_sigma          0.01

#Parameters (5.Mixing)
mixing_type             broyden
mixing_beta             0.7
mixing_gg0              0
```

**收敛性测试**

**Ecut 收敛（只对平面波有效）**

平面波（pw）作为一种可以用于描述周期性边界条件下的电子波函数和电荷密度基矢量，它们都是正交的，且可以通过一个 `ecutwfc` 参数来控制基矢量的个数，ecut 其代表了每一个平面波所对应的动能，如果 ecut 取得越大，则基矢量可以描述震荡得越剧烈的物理量（例如氧原子的 2p 轨道），那么计算结果就会越精确，但同时所带来的机时成本消耗也越大。因此我们采用 pw 基组做真正计算前，需要对 ecut 进行测试来获得一个足够准确且效率也高的取值。

保持前文用于计算的基组为 “pw”，进行 ecut 的收敛性测试，ecut 取值范围为：20~100 Ry。这个例子在 4 进程下启动 MPI 进行并行计算，一般推荐总核数 (= 线程数 × 进程数) 取 2 的 n 次方或者 n 倍。一个较为粗糙、但基本不会造成计算资源浪费的选取 n 值原则是：体系有多少个原子，不要用超过这个原子个数太多的总核数进行并行计算。例如，体系如果有 16 个原子，不要用远大于 16 的总核数进行计算，一般取 16 的总核数或者更少就够了，具体需要测试。

```
mpirun -n 4 abacus
```

计算完成后，建议用脚本提取 ecut 数值下 `OUT.Si` 文件夹里 `running_scf.log` 文件中收敛的系统总能量，计算单原子能量并绘制其随着 ecut 的变化曲线。如图 2，可以看到随 ecut 增大，系统总能量趋于收敛。在 ecut=60 Ry 时，认为总能量收敛 (与 ecut=50 Ry 的能量差小于 1 meV/atom)。

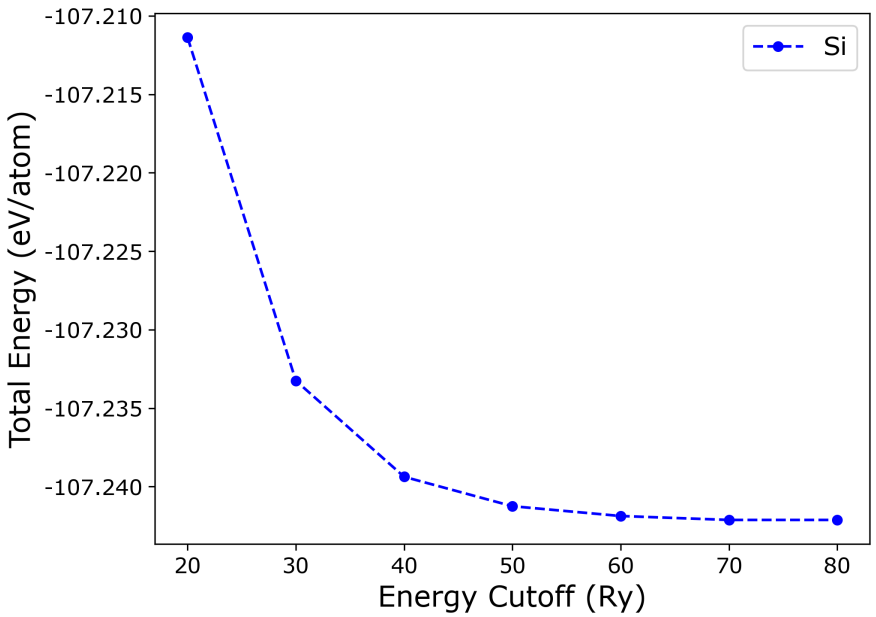



**图 4.2:** 体系里平均单个 Si 原子能量 (in eV/atom) 随 ecut (in Ry) 变化。

**k 点收敛**

对具有周期性的体系进行计算时，DFT 计算实际会对第一布里渊区中不同离散化 k 点的单独进行计算（例如每个 k 点都会单独求解一次依赖于该 k 点的 Kohn-Sham 方程），并将计算结果进行积分。k 点越多，则需要求的离散积分就越多，也将越近似连续积分的结果，但计算资源也会增加。因此，pw 和 LCAO 都有必要对“需要多少 k 点”进行测试并得出合适的 k 点选取方案。

金刚石 Si 的 k 点测试结果。同样在 4 进程下进行并行计算。

```
mpirun -n 4 abacus
```

计算完成后，提取各 K 点下 `OUT.Si` 文件夹里 `running_scf.log` 文件中收敛的系统总能量和计算时间，绘制这两个参数随 K 点的变化曲线。此处，$N_k$ 表示各方向上的 k 点取值个数，“Energy - $N_k$ ”图中采用单原子的能量。

分析图 3.3 和图 3.4 可知，在 $N_k = 6$ 时，结果已收敛，能量不会再发生较大的变化 (与 $N_k = 5$ 的能量差小于 1 meV/atom)，而此时花费时间 132 秒，相比于更大的 $N_k$ 点较低。因此选择 6×6×6 的 k 点可以同时获得准确的结果和高的计算效率。

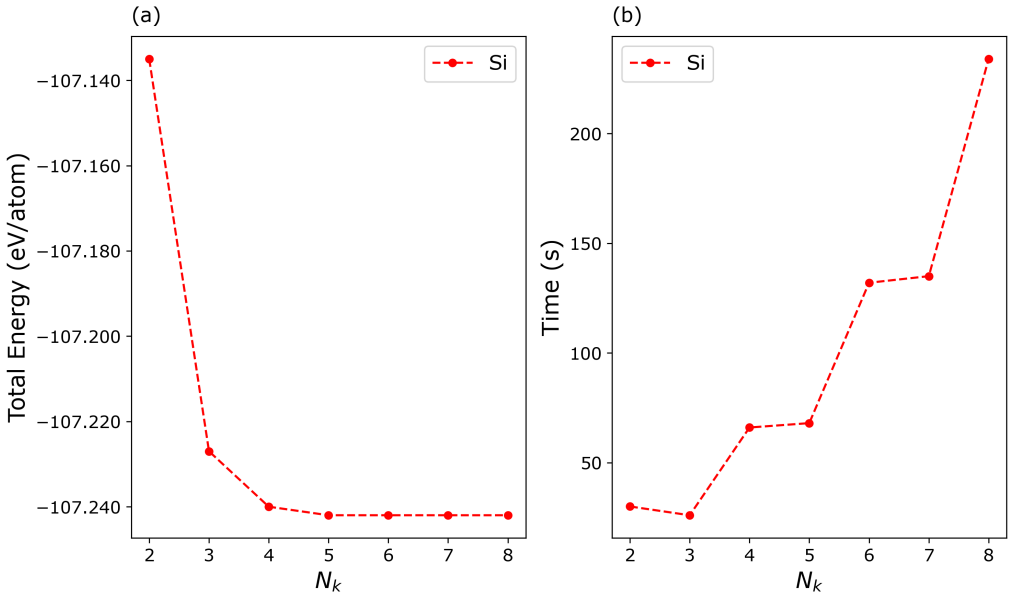



**图 4.3:** (a) 体系里平均单个 Si 原子能量 (in eV/atom) 随 K 点变化。(b) 计算时间随 K 点变化。

提取出每个计算中实际使用到 k 点个数，如表一所示，实际使用 k 点数相比总 k 点数有较明显的简化。如前所述，INPUT 文件中我们设置了参数 `symmetry` 取“1”，因此在实际计算中利用了晶胞结构中的对称性，即倒易空间的积分没有使用所有的 k 点计算，而是巧妙的利用了对称性得到其它 k 点的信息，简化了布里渊区中的计算工作量。这也解释了图 4 中明明总 k 点数不同，计算时间却十分接近的原因。

**表 4.1:** $N_k$ 点数，总 k 点数，实际使用 k 点数

| $N_k$ | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|
| 总 k 点个数 | 8 | 27 | 64 | 125 | 216 | 343 | 512 |
| 实际计算 k 点数（开了对称性） | 4 | 4 | 10 | 10 | 20 | 20 | 35 |

### 4.1.2 ABACUS+LibRI 做杂化泛函计算教程

#### 一、介绍

杂化泛函（Hybrid Functional）是指在密度泛函理论框架中的交换关联项里面加入一部分的 Hartree Fock (简称 HF) 的精确交换能。开源密度泛函理论软件 ABACUS 可以结合另一款国产开源软件 LibRI 软件进行杂化密度泛函计算，目前仅支持在数值原子轨道基组下使用该功能。可以通过 dft_functional 参数指定所使用的杂化泛函类型，如可以选择 hf (Hartree-Fock), pbe0(PBE0), hse(HSE06) 以及 scan0 杂化泛函。本教程以 HSE 杂化泛函为例，介绍如何在 ABACUS 里调用 LibRI 做杂化泛函自洽迭代、求力和应力以及结构优化。

注 1：使用 ABACUS+LibRI 做杂化泛函计算时，最大并行核数是 $N_a^4 N_k^3$，其中 $N_a$ 是原子个数，$N_k$ 是 $k$ 点个数。计算资源超出时可以运行，但会造成浪费。

注 2：使用 ABACUS+LibRI 做杂化泛函计算时，因为内存消耗比较大，推荐给定计算资源的前提下，先尽量使用 OpenMP 多线程并行，再考虑使用 MPI 多进程并行。

#### 二、杂化泛函的使用

**1. ABACUS 编译准备**

如果要在 ABACUS 中使用杂化泛函进行计算，需要在编译 ABACUS 的时候也编译 Libxc、LibRI 和 LibComm 三个软件包，具体请见线上文档 Advanced Installation Options ABACUS documentation。

注意在链接 LibRI、LibComm 时如果报错未定义的引用等，可以先注意检查 ABACUS 源代码下 deps 文件夹下是否包含 LibRI、LibComm 两个文件夹。如果未包含这两个文件夹，或文件夹中无内容，在本地 Github 仓库中可以尝试如下两条语句，以获取这两个子仓库内容：

```
git submodule init
git submodule update --remote --recursive
```

**2. 采用杂化泛函进行电子自洽迭代计算**

本教程在 Gitee 上准备了一个硅晶体使用杂化泛函做自洽计算（SCF）的例子（Gitee 的下载链接），以下是 INPUT 文件及相关的参数。由 dft_functional 设置为 hse 可知，该例子使用的杂化泛函为 HSE 泛函。KPT 文件取的是 444 的布里渊区 k 点。

```
INPUT_PARAMETERS
calculation scf
basis_type lcao
ntype 1
nbands 8
ecutwfc 60.0
scf_nmax 100

dft_functional hse
scf_thr 1e-7
```

杂化泛函相关的完整参数列表及解释见 Full List of INPUT Keywords / exact-exchange ABACUS documentation。这里再进行简单概述：

- **泛函相关参数**：
  - `exx_hybrid_alpha`: 杂化泛函中加入的 HF 精确交换能（Fock 交换能）的比例 $\alpha$，即有 $E_x = \alpha E_x^{\mathrm{HF}} + (1-\alpha)E_x^{\mathrm{LDA/GGA}}$。如果 `df_1 functional` 设置为 `hf`，则默认值为 1。目前其他杂化泛函的默认值是 0.25。但

是，如果是 SCANO 泛函，有的文献取的是 0.1，所以需要根据你想取的值进行设定 [1]。

- `exx_hse_omega`: 为 HSE 泛函中的区间分割参数（range-separation parameter）$\omega$，即有 $\frac{1}{r}=\frac{\mathrm{erfc}(\omega r)}{r}+\frac{\mathrm{erf}(\omega r)}{r}$。默认值为 0.11（$\mathrm{bohr}^{-1}$），此时为 HSE06 泛函 [2]。
- `exx_lambda`: 在 `basis_type` 设置为 `lcao_in_pw` 的情况下，用于补偿使用 `lcao_in_pw` 方法评估精确交换能时 $G=0$ 处的发散点。默认值为 0.3。

- `exx_real_number`: 该参数设定为 `True` 时，强制 LibRI 使用 double 数据类型，当设定为 `False` 时，强制 LibRI 使用 complex 数据类型。当 `gamma_only=1` 时，默认为 `True`，`gamma_only=0` 时默认为 `False`。
- **循环相关参数**：
  - `exx_separate_loop`: ABACUS 提供了两种迭代方法来评估精确交换能 $E_{\mathrm{exx}}$。当 `exx_separate_loop` 设置为 `False` 时：采用单层循环，即先进行 GGA 循环，然后进行 Hybrid 循环，在该过程中，使用电子迭代来更新 $E_{\mathrm{exx}}$ 对应的哈密顿量 $H_{\mathrm{exx}}$。当 `exx_separate_loop` 设置为 `True` 时：采用双层循环，在内层循环中，进行自洽迭代并更新密度矩阵，在外层循环中，根据在内层循环中收敛的密度矩阵来计算 $H_{\mathrm{exx}}$。默认值为 `True`，即采用双层循环计算。单层循环有利于难以自洽收敛的体系达到收敛，但会显著增加内存消耗。
  - `exx_hybrid_step`: 在 `exx_separate_loop` 设置为 `True` 的情况下，外层循环的最大迭代步数。默认值为 100。
  - `exx_mixing_beta`: 在 `exx_separate_loop` 设置为 `True` 的情况下，内层循环每次迭代时，密度矩阵混合的 `mixing_beta` 取值，默认为 1.0。
- `exx_pca_threshold`: 为了加速四中心积分 $\langle ik|jl\rangle$ 的计算，ABACUS 采用 LRI 方法，将原子轨道的乘积在辅助基函数 (ABF) 的基础上展开，即

$$\Phi_i\Phi_j\approx\sum_a C_{ij}^a P_a$$

并利用 PCA 减小辅助基函数 (ABF) 的大小 (即 $P_a$ 个数)。阈值越大，ABF 的数目越少，计算速度越快，计算精度越低。一个相对安全的值是 1e-4，也是默认值。

- `exx_ccp_rmesh_times`: 此参数决定计算 Column 势所需的截断半径比原子轨道的截断半径大多少倍。对于 HSE 泛函，设置为 1 就足够了。但是对于 PBEO，必须使用一个大得多的数字。当使用 HSE 泛函时，默认值为 1.5，其他情况下默认值为 5。
- **张量筛选相关参数**：针对杂化泛函计算过程中的物理量进行筛选可以加速计算。具体来说，`exx_c_threshold`、`exx_v_threshold`、`exx_dm_threshold`、`exx_c_grad_threshold`、`exx_v_grad_threshold` 分别是针对 $C_{ij}^a$、

$$V_{ab}=\langle P_a|P_b\rangle$$

、密度矩阵、$VC_{ij}^a$、$VV_{ab}$。阈值越大，筛掉的张量越多，计算速度越快，计算精度越低。具体请查看完整 INPUT 参数文档。

- **Cauchy-Schwartz 不等式相关参数**：
  - `exx_cauchy_threshold`: 在实际中，Fock 交换矩阵是稀疏的，利用 Cauchy-Schwartz 不等式，我们可以在进行显式求值之前找到每个矩阵元素的上界。小于 `exx_cauchy_threshold` 的值将被截断。阈值越大，筛掉的张量越多，计算速度越快，精度越低。一个相对安全的值是 1e-7，也是默认值。不等式算法参见参考文献 [3]。
  - `exx_cauchy_force_threshold`、`exx_cauchy_stress_threshold` 与 `exx_cauchy_threshold` 类似，区别在于它们分别针对的是求力、应力计算中的 Fock 交换矩阵元。
- **opt_orb 相关参数**：当 **dft_functional** 设置为 `opt_orb` 时使用，opt_orb 参考文献 [4]。本功能仅用于生成 opt 辅助基组，不进行杂化泛函计算。
  - **exx_opt_orb_lmax**：球贝塞尔函数的最大角动量 L 值，opt-ABF 的径向部分用球贝塞尔函数的线性组合生成。
  - **exx_opt_orb_ecut**：球贝塞尔函数展开的截断，在优化 opt-ABF 的时候采用的是球贝塞尔函数基组。
  - **exx_opt_orb_tolerance**：解球贝塞尔函数零点时的阈值。

### 3. 杂化泛函计算代价

杂化泛函的计算精度高，与此同时它的计算代价也比较高。在 ABACUS 的输入参数文件 INPUT 中，若 exx_separate_loop 参数设为 True（默认），仅在 SCF 步骤中就涉及两层循环。每次内层循环完成，外层循环往前推进一步时，屏幕输出 Updating EXX and rerun SCF。

一次 SCF 需要的时间至少是以上两个循环涉及的单次电子迭代时间之和。对于单次电子迭代所需时间，在此提出一些已有的经验。以一步电子迭代的时间为衡量尺标，使用 DZP 基组，CPU 型号为 Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz，使用 4 核计算一个水分子为 0.6s 左右，使用 14 个核计算 32 个水分子为 0.8s 左右，使用 14 个核计算 64 个水需要 1.9s 左右。

若将 exx_separate_loop 参数设为 False，即使用单层循环时，首先会进行 GGA 迭代直到自洽收敛，然后屏幕输出 Entering 2nd SCF, where EXX is updated，进行 Hybrid 迭代，此时每进行一次电子步得到新的密度后，都会更新一次精确交换能。以一步电子迭代 + 更新精确交换能的时间为衡量尺标，使用 DZP 基组，CPU 型号为 Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz，使用 4 核计算一个水分子为 0.7s 左右，使用 14 个核计算 32 个水分子为 115s 左右，使用 14 个核计算 64 个水需要 330s 左右。对于更大的体系，如 2048 个 Si 原子的晶体，使用 DZP 基组，CPU 型号为 Intel(R) Xeon(R) Silver 4310 CPU @ 2.10GHz，用一个节点（56 核）算时，PBE 下一步电子迭代大概需要 380s，而 HSE 一步电子迭代 + 更新精确交换能大概需要 1680s。

在进行杂化泛函计算时推荐尽量使用多线程计算（OpenMP），此时内存开销相对较小，计算速度相对较快。

## 三、使用杂化泛函做结构优化

### 1. 数据准备

在 Gitee 上我们准备了一个简单的使用杂化泛函做结构优化的例子。该例子是在 LCAO 基组下，使用 HSE 泛函，优化单个水分子的结构。文件夹中 log.ref 是使用 3.2.1 版本的 ABACUS 软件包，v0.1.0 版本的 LibRI 和 LibComm 计算所得的屏幕输出。

### 2. 输入文件

准备计算所需的 INPUT 文件、STRU 文件、KPT 文件，以及 H、O 原子对应的数值原子轨道文件。其中 INPUT 文件如下。注意该文件中指明了计算类型为 relax，即不对晶胞做优化（cell relax），只对原子位置做优化（relax）。更多结构优化类型请看文档 Geometry Optimization  ABACUS documentation。

```
INPUT_PARAMETERS
calculation relax
basis_type lcao
ntype 2
ecutwfc 60.000000
scf_nmax 100
gamma_only 1

dft_functional hse
relax_nmax 100
scf_thr 1e-6
force_thr_ev 1e-2
```

在该例子中，结构优化包括多个离子步，每个离子步中都要做一次 SCF。由 INPUT 文件可知，SCF 收敛的标准由 scf_thr=1e-6 指定，或达到 SCF 的最大步数 scf_nmax=100，并计算受力。根据上一个离子步计算得到的受力，计算下一个离子步的原子位置，计算收敛的标准此时为 force_thr_ev=1e-2，或达到离子步的最大步数 relax_nmax=100。STRU 文件如下，可见在结构弛豫步骤中，三个原子都可以移动。

```
ATOMIC_SPECIES
O 16.00 O_ONCV_PBE-1.0.upf
H 1.00 H_ONCV_PBE-1.0.upf

LATTICE_CONSTANT
1

LATTICE_VECTORS
28 0 0
0 28 0
0 0 28

ATOMIC_POSITIONS
Direct

O #label
0 #magnetism
1 #number of atoms
0.677639488918  0.5227809096584  0.232500040128  m  1  1  1

H #label
0 #magnetism
2 #number of atoms
0.641808457616  0.5785821615863  0.228644198512  m  1  1  1
0.708889637644  0.5204300746076  0.175087721492  m  1  1  1

NUMERICAL_ORBITAL
O_gga_6au_60Ry_2s2p1d.orb
H_gga_6au_60Ry_2s1p.orb
```

**3. 结果**

结构弛豫（relax）后的原子结构可见 OUT.ABACUS/STRU_ION_D。由输出文件可知，即使该例子中采用了相对稳定的构型，且 scf_thr 仅设为 1e-6，使用 HSE 做结构弛豫的计算代价仍然很高，使用 6 个核（Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz）计算需要 5 分钟左右。

## 四、参考文献

[1] Staroverov V N, Scuseria G E, Tao J, et al. Comparative assessment of a new nonempirical density functional: Molecules and hydrogen-bonded complexes[J]. The Journal of chemical physics, 2003, 119(23): 12129-12137.

[2] Aliaksandr V. Krukau, Oleg A. Vydrov, Artur F. Izmaylov, Gustavo E. Scuseria; Influence of the exchange screening parameter on the performance of screened hybrid functionals. J. Chem. Phys. 14 December 2006; 125 (22): 224106.

[3] Lin P, Ren X, He L. Efficient hybrid density functional calculations for large periodic systems using numerical atomic orbitals. Journal of Chemical Theory and Computation, 2021, 17(1): 222–239.

[4] Lin P, Ren X, He L. Accuracy of localized resolution of the identity in periodic hybrid functional calculations with numerical atomic orbitals[J]. The Journal of Physical Chemistry Letters, 2020, 11(8): 3082-3088.

### 4.1.3 ABACUS 收敛性问题解决手册

**一、可选参数**

mixing_type
描述：混合电荷密度所使用的方法
可选：broyden，pulay，plain
默认值：broyden
说明：没有特殊理由，不建议更改这个参数

mixing_ndim
描述：DIIS 混合计算混合系数时会保存的历史电荷密度数目
可选：int
默认值：8
说明：一般而言，这个值越大，收敛性越好，但是代价是内存消耗线性增加，8 一般是够用的。

mixing_beta
描述：电荷密度的混合系数，1.0 时表示全部用输出电荷，0.0 时表示全部用输入电荷
可选：double
默认值：0.8（nspin=1）；0.4（nspin=2|4）
说明：一般而言，这个值越大，收敛越快，但是 SCF 跑飞不收敛的风险也越大；这个值越小，收敛越慢，但是收敛会更稳定，不收敛的情况会减少。

mixing_gg0
描述：电荷密度混合时所做的 kerker 预处理的强度。Kerker 预处理会显著降低一些低频长程的电荷波动，对于金属性体系的收敛帮助极大。
可选：double
默认值：1.0
说明：默认 1.0 代表 ABACUS 默认打开 kerker 预处理，你可以手动设置成 0.0 将其关闭。一般而言，绝缘体/分子/原子体系中 kerker 预处理的效果不明显，甚至有负优化，可以关闭。

mixing_gg0_min
描述：电荷密度的混合系数，所做的 kerker 预处理中，小于这一 $|q|$ 的电荷分量不再进一步衰减
可选：double
默认值：0.1
说明：没有特殊理由，不建议更改这个参数

mixing_beta_mag
描述：磁密度的混合系数，只在 nspin=2|4 时才会启用
可选：double
默认值：4*mixing_beta，但是最大不超过 1.6
说明：与电荷密度的混合类似，这个值越大，收敛越快，但是 SCF 跑飞不收敛的风险也越大；这个值越小，收敛越慢，但是收敛会更稳定，不收敛的情况会减少。

mixing_gg0_mag
描述：磁密度的混合时所做的 kerker 预处理的强度

可选：double
默认值：0.0
说明：默认 0.0 代表 ABACUS 默认关闭对磁密度的 kerker 预处理，你可以手动设置成非零将其打开。
一般而言，对于磁密度的 Kerker 预处理物理意义并不明确，不建议打开。

mixing_angle
描述：使用 J. Phys. Soc. Jpn. 82 (2013) 114706 中所描述的角度混合来更新电荷密度，
只在非共线计算，即 nspin=4 时才可以启用
可选：double
默认值：0.0
说明：默认 0.0 代表 ABACUS 默认不使用这个方法，还是默认优先使用传统的 DIIS 混合。
你可以手动设置成非零将其打开。这一方法可以解决非共线计算中磁矩收敛困难、收敛到错误磁态的问题。

mixing_tau
描述：是否混合动能密度，只在 metaGGA 计算中才能使用
可选：bool
默认值：false
说明：ABACUS 默认不混合动能密度。但是打开一般会提升收敛性。

mixing_dmr
描述：使用混合电荷密度的系数，同样混合密度矩阵。打开这一参数的前提必须同时设置 mixing_restart
可选：bool
默认值：false
说明：ABACUS 默认不混合密度矩阵。如果你在进行需要密度矩阵构建算符的计算，
例如 DFT+U/EXX/DeePKS，你可以试图打开 mixing_dmr。经过我们大量的 DFT+U 算例测试，
mixing_dmr 配合一个合适的 mixing_restart 会显著地提升 ABACUS 计算的收敛性。

mixing_restart
描述：在第 mixing_restart 步 SCF 清空 mixing 的历史，直接使用上一步的输出作为这一步的输入，
一般与 mixing_dmr 配合使用
可选：int
默认值：0
说明：ABACUS 默认不进行 restart，一个不适合的 mixing_restart 设置可能导致计算收敛不了。

**二、如果收敛困难怎么办**

**非磁计算（nspin=1）**

非磁实际上可调的自由度不多，所有体系都可以遵循的原则如下：

1. 尝试调小 mixing_beta
2. 尝试调大 mixing_ndim

分情况，你可以进一步有如下特殊的操作：

原子分子体系

原子分子的电荷高度局域，Kerker 预处理一般都难以实际取得正优化，你可以选择关闭 Kerkerk 预处理，mixing_gg0=0.0。

半导体/绝缘体

虽然绝缘体部分电子态也很局域，但是 Kerker 预处理一般是安全的，即使负优化也不会很严重，但是你仍可以选择关闭 Kerkerk 预处理，mixing_gg0=0.0。

金属

调节不同的 mixing_gg0、mixing_gg0_min。

metaGGA 计算

打开 mixing_tau=true。一般会解决 metaGGA 收敛困难，以及 EDIFF 慢于 DRHO 的问题。

检查初始构型

一般而言，你不会在非磁计算遇到收敛性问题。如果真的遇到了，有很大概率初始构型太差，你可以调整构型，或者用先用宽松的收敛判据做几步弛豫计算，让体系构型更合理之后再调小 scf_thr。

**共线磁性计算（nspin=2）**

磁性计算中所有体系都可以遵循的原则如下：

1. 尝试调小 mixing_beta，并等比例调小 mixing_beta_mag
2. 尝试调大 mixing_ndim

分情况，你可以进一步有如下特殊的操作：

原子分子体系

孤立体系的磁性计算如果出现无法收敛，那大概率是出现一种极端的情况，即某一自旋全部来自某一轨道，不会叠加态。这种情况一旦出现，默认参数无法收敛。这时，请保证电荷密度和磁密度的混合系数完全一致：

```
mixing_beta      0.4
mixing_beta_mag  0.4
mixing_gg0       0.0
mixing_gg0_mag   0.0
```

相信这个设置之后，问题就会解决。

半导体/绝缘体

与非磁建议一致。

金属

与非磁建议一致。

metaGGA 计算

打开 mixing_tau=true。一般会解决 metaGGA 收敛困难，以及 EDIFF 慢于 DRHO 的问题。

DFT+U 计算

如果你的 DFT+U 计算无法收敛，我们推荐你做如下设置：

```
mixing_restart   10
mixing_dmr       1
```

mixing_restart=10 代表 SCF 的计算会在第 10 步直接用第 9 步的输出电荷开启一个新的 SCF，在那之后 DFT+U 的计算同时也会混合密度矩阵。在大多数情况下，10 是一个安全有效的设置，但请务必记住 mixing_restart 的设置是危险的，设置之后，你可能会遇到：

本来不设置这 2 个参数还可以收敛，只是收敛的不快，设置之后完全不能收敛了。

答：那是因为 mixing_restart 的位置选取不合适，在 mixing_restart 前一步的输出密度很差，用其做初始密度使得计算直接跑飞。这个时候，我们建议你观察不开 mixing_restart 时候，drho 的收敛曲线，找一个距离基态近一些的位置做 mixing_restart（例如 drho<10^-3）。

本来不能收敛，设置这 2 个参数之后，仍然不能收敛

答：大概率是 mixing_restart 的晚了，此时 SCF 已经跑飞，结合 drho 的收敛曲线，适当提前 mixing_restart 的位置。

弛豫计算

在弛豫计算中可以分多步做 relax，会使用上一步的 SCF 的磁矩做下一步优化的初始磁矩。这样对于弛豫计算的收敛性帮助巨大。

**非共线磁性计算（nspin=4）**

在使用传统的 DIIS 计算时，即 mixing_angle=0.0，参数建议与共线磁性一致。但是如果尝试之后，仍难以收敛，或收敛到错误的磁态，那么我们建议你进行如下的设置：

```
mixing_angle    1.0
```

我们实现了 J. Phys. Soc. Jpn. 82 (2013) 114706 中的新方法，这个方法寻找非共线基态的能力大大强于传统的 DIIS 混合。相信会对你有帮助。

### 三、其他 FAQ

我可以在 STRU 中不设置初始磁矩，就设置成 0 吗?

答：可以。当 STRU 中所有元素的初始磁矩都是 0 的时候，我们会自动为体系中的所有原子都赋予大小为 1 a.u. 的原子磁矩。但是我们仍建议你设置它，因为一个好的初始磁矩对于计算的收敛速度和正确性都有巨大的帮助。

我可以在计算中设置 NUPDOWN 来帮助收敛吗?

答：可以。一个合理的 NUPDOWN 设置一般会使得计算收敛地比较稳定。但是这么做的前提是，一定要对所做的体系很可靠的先验知识。

## 4.2 分子动力学模拟

### 4.2.1 ABACUS 分子动力学使用教程

**一、分子动力学方法简介**

**分子动力学（Molecular Dynamics，简称 MD）**：一种模拟原子体系随时间运动的方法，被广泛的用来研究与原子运动路径相关的一些基本过程，如相变、扩散、化学反应等等。本教程旨在介绍 ABACUS 中的分子动力学功能，分子动力学模拟重要的一件事情是如何选取精度和效率的平衡，其中精度高低主要取决于势函数的准确程度，而效率主要受限于求出每个原子的能量和受力的计算量。

**经典分子动力学（Classical MD，简称 CMD）**：通过构建描述原子间相互作用的势函数，获得每个原子的受力（受力等于能量对原子位置的导数），再通过积分运动方程来获得每个原子的下一时刻位置，从而获得随时间演化的粒子位置和速度。当系统处在一定的密度、温度和压强等物理条件限制下，可以结合统计物理的方法计算物质的性质。具体来说，采取某个系综后，对粒子位置和速度采样，之后统计出体系的热力学宏观性质。

**从头算分子动力学（ab initio molecular dynamics，简称 AIMD）**：也称为第一性原理的分子动力学方法（first-principles molecular dynamics，简称 FPMD），该方法采用的是第一性原理方法（例如密度泛函理论）来计算体系的势能面，因此计算量相比于经典分子动力学方法要昂贵很多。

**ABACUS 的分子动力学功能**：支持第一性原理分子动力学 FPMD 方法，也支持经典的 Lennard-Jones（LJ 对势）的分子动力学模拟。此外，ABACUS 还支持深度势能分子动力学（Deep Potential Molecular Dynamics，简称 DPMD）方法，此时需要编译 DeePMD-kit 软件包并在编译原子算筹软件时进行动态库的链接。

**二、文档和算例下载地址**

ABACUS 里关于分子动力学方法的线上文档地址为：
http://abacus.deepmodeling.com/en/latest/advanced/md.html
ABACUS 里关于分子动力学参数的线上文档地址为：
http://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#molecular-dynamics
ABACUS 的 MD 算例下载地址为（国内 gitee）：
https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/md
可以采用的下载命令是：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

之后进入 /abacus-user-guide/examples/md 目录或者采用 Github 仓库地址：
https://github.com/MCresearch/abacus-user-guide/tree/master/examples/md
注：算例仓库里面包含 1_AIMD，2_LJMD（采用 Lennard-Jones 经典势做 MD），3_DPMD（采用深度势能方法做 MD）和 4_FIRE（采用 FIRE 算法做结构优化）四个目录。
注：以上算例要在 ABACUS v3.2.1 版本及以上可以运行成功，并且我们强烈建议下载使用 ABACUS 最新版本！

**1. 第一性原理分子动力学（AIMD）**

进入 1_AIMD 目录，该目录提供了 8 个输入文件，使用的时候将 INPUT_* 复制为 INPUT 即可（INPUT_* 代表从 INPUT_0 到 INPUT_7 共 8 个不同的 INPUT 文件，对应 8 种不同的 MD 算法）。该算例采用 8 原子的金刚石结构 Si，晶格结构放在 STRU 文件里，布里渊区的 k 点采样放在 KPT 文件里。
8 个 INPUT 文件对应的 MD 方法如下：
INPUT_0：采用 NVE 微正则系综

INPUT_1：采用 NVT 正则系综
INPUT_2：采用 Langevin 热库
INPUT_3：采用 Anderson 热库，NVT 系综
INPUT_4：采用 MSST 方法
INPUT_5：采用 Berendsen 热库，NVT 系综
INPUT_6：采用 rescaling 方法，NVT 系综
INPUT_7：采用 rescale_v 方法，NVT 系综
以 INPUT_0 为例：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix                  Si_nve
calculation             md
nbands                  20
symmetry                0
pseudo_dir              ../../../tests/PP_ORB
orbital_dir             ../../../tests/PP_ORB

#Parameters (2.Iteration)
ecutwfc                 30
scf_thr                 1e-5
scf_nmax                100

#Parameters (3.Basis)
basis_type              lcao
ks_solver               genelpa
gamma_only              1

#Parameters (4.Smearing)
smearing_method         gaussian
smearing_sigma          0.001

#Parameters (5.Mixing)
mixing_type             pulay
mixing_beta             0.3
chg_extrap              second-order

#Parameters (6.MD)
md_type                 nve
md_nstep                10
md_dt                   1
md_tfirst               300
```

这些参数在 ABACUS 线上文档中均有说明，在这里再进行简单概述：

- calculation：设置 ABACUS 计算类型，做分子动力学模拟请设置为 md。
- esolver_type：给定原子位置进行系统能量的计算，默认 Kohn-Sham 密度泛函理论（ksdft），还可以设置 LJ 势（lj）或者深度势能（dp）。理论上来说，所有的 esolver_type 都可以用于 MD 计算。
- chg_extrap：电荷外插法，在 relax 和 md 计算中可用于优化电荷密度初始化，加速电子迭代收敛，MD 中可以

设置为 second-order。

- md_type：MD 算法种类，默认是正则系统 nvt，这里的例子选择的是 nve，即 NVE 微正则系综。
- md_nstep：MD 模拟的总步数。
- md_dt：MD 计算每一步的时间步长（单位是 fs），与 md_nstep 共同决定 MD 总时长。
- md_tfirst：MD 系统的初始温度（单位是 K）。

**2. Lennard-Jones 势函数**

进入 2_LJMD 目录，ABACUS 软件包中提供了一个采用 Lennard-Jones（LJ）经典势函数进行 MD 模拟的算例，该算例采用 LJ 对势作为能量求解器来做经典分子动力学，初始结构是边长 20 埃的正方体盒子中的 256 个无序 He 原子。

**LJ 对势**

LJ 对势的解析形式如下：$u^{ij}$ 表示原子 i 与原子 j 之间的相互作用能，$r^{ij}$ 表示这两个原子之间的距离，$\epsilon$，$\sigma$ 和 $r_{\text{cut}}$ 则是 LJ 势需要的 3 个参数。LJ 势只考虑截断半径 $r_{\text{cut}}$ 内的近邻原子 j 对中心原子 i 的能量贡献。

$$u^{ij} = 4\epsilon \left[ \left( \frac{\sigma}{r_{ij}} \right)^{12} - \left( \frac{\sigma}{r_{ij}} \right)^{6} \right]$$

$$F_i = -\nabla_i U = \sum_j 4\epsilon \left[ 12 \left( \frac{\sigma}{r_{ij}} \right)^{12} - 6 \left( \frac{\sigma}{r_{ij}} \right)^{6} \right] \frac{\mathbf{r}_i - \mathbf{r}_j}{r_{ij}^2} H\left(r_{\text{cut}} - r_{ij}\right)$$

$$H(x) = \begin{cases} 0 & (x < 0) \\ 1 & (x > 0) \end{cases}$$

**STRU**

LJMD 是经典分子动力学，不需要提供 KPT，赝势和轨道文件，STRU 的前面几部分如下：

```
ATOMIC_SPECIES
He 4.0026

LATTICE_CONSTANT
1.889727

LATTICE_VECTORS
20 0 0
0 20 0
0 0 20

ATOMIC_POSITIONS
Cartesian

He #label
0 #magnetism
256 #number of atoms
```

ATOMIC_SPECIES：不需要提供赝势信息

NUMERICAL_ORBITAL：不需要添加轨道信息

**INPUT**

INPUT 文件中的参数也需要做少量修改：

```
INPUT_PARAMETERS
#Parameters (General)
suffix              autotest

calculation         md
pseudo_dir          ../../PP_ORB

#Parameters (esolver)
esolver_type        lj
lj_rcut             8.5
lj_epsilon          0.01032
lj_sigma            3.405

cal_force           1
cal_stress          1

#Parameters (MD)
md_nstep            5
md_type             nve
md_dt               1
md_tfirst           300

init_vel            1
```

esolver_type：指定为 lj 就可以做 Lennard-Jones 经典势函数。

lj_rcut：LJ 对势的参数，即截断半径 rcut 。lj_epsilon：LJ 对势的参数，即 $\epsilon$ 。lj_sigma：LJ 对势的参数，即 $\sigma$ 。init_vel：读取 STRU 中的原子速度信息，原子单位制。

### 3. 深度势能（Deep Potential）

ABACUS 软件包中提供了一个 DPMD 的算例，进入 3_DPMD 目录。该算例采用 DP 模型（DeePMD-kit 产生的深度势能模型）作为能量求解器来做基于机器学习的分子动力学，初始结构是 864 个无序铝原子。

**DP 模型**

DeePMD-kit 是一种基于机器学习的分子动力学模拟方法，该方法是通过使用第一性原理计算数据对深度神经网络模型进行训练，从而得到通用的多体势能模型（DP 模型）。

**编译方式**

ABACUS 采用 DP 模型做 MD 计算需要编译与 DeePMD-kit 的接口，cmake 以及 makefile 编译方式可以参考 ABACUS 线上文档。

**INPUT 文件设置**

DPMD 是经典分子动力学，不需要提供 KPT，赝势和轨道文件

INPUT 文件中的参数需要做少量修改：

```
INPUT_PARAMETERS
#Parameters        (General)
suffix              autotest
calculation         md
```

```
esolver_type        dp
pot_file            ../../PP_ORB/Al-SCAN.pb

cal_force           1
cal_stress          1

md_nstep            3
md_type             msst
md_dt               1
md_tfirst           200
md_dumpfreq         1
md_restartfreq      1

msst_qmass          200
msst_vel            0.028
msst_vis            0.3

init_vel            1
```

esolver_type：设置为 dp

pot_file：DP 模型文件路径

md_dumpfreq：MD 输出文件 MD_dump 中原子以及晶胞信息的输出频率

md_restartfreq：结构文件 STRU_MD_$istep 的输出频率，MD 续算文件 Restart_md.dat 的更新频率

## 三、MD 输出和续算功能

### 1. MD 输出文件

ABACUS 的 MD 模拟会产生 3 类输出文件：结构文件 STRU_MD_$istep，续算文件 Restart_md.dat，信息文件 MD_dump。

**结构文件 STRU_MD_$istep**

参数 md_restartfreq 控制 OUT.$suffix/STRU/文件夹中结构文件 STRU_MD_$istep 的输出频率，该文件格式与输入结构文件 STRU 相同，主要用于 MD 续算。这里 istep 代表离子步，从 0 开始计数。

**续算文件 Restart_md.dat**

参数 md_restartfreq 控制 OUT.$suffix/Restart_md.dat 的更新频率，用于 MD 续算。

**信息文件 MD_dump**

参数 md_dumpfreq 控制 OUT.$suffix/MD_dump 文件的追加输出频率，该文件输出每 md_dumpfreq 步的 MD 信息，包括 MD 步数，晶格常数，晶格矢量，晶格维里，原子编号，原子位置，原子受力，原子速度。其中，晶格维里，原子受力，原子速度可以通过输入参数 dump_virial，dump_force，dump_vel 控制是否输出。

```
MDSTEP:  0
LATTICE_CONSTANT: 5.397605400000 Angstrom
LATTICE_VECTORS
  0.500000000000  0.500000000000  0.000000000000
  0.500000000000  0.000000000000  0.500000000000
  0.000000000000  0.500000000000  0.500000000000
VIRIAL (kbar)
```

```
  123.045275146001  -3.807535058112  -13.541439282961
  -3.807535058112  124.421300186299  24.047959743672
  -13.541439282961  24.047959743672  125.016024430342
INDEX    LABEL    POSITION (Angstrom)    FORCE (eV/Angstrom)    VELOCITY (Angstrom/fs)
  0  Si  0.000000000000  0.000000000000  0.000000000000  -0.885362725233  0.500467424348
  0.150239620206  -0.003506076235  0.000412570794  0.000094403053
  1  Si  1.300822817264  1.376389287977  1.354798867769  0.885362725233  -0.500467424348
  -0.150239620206  0.003506076235  -0.000412570794  -0.000094403053

MDSTEP:  1
LATTICE_CONSTANT: 5.397605400000 Angstrom
LATTICE_VECTORS
  0.500000000000  0.500000000000  0.000000000000
  0.500000000000  0.000000000000  0.500000000000
  0.000000000000  0.500000000000  0.500000000000
VIRIAL (kbar)
  123.051574098220  -2.465332696362  -10.608474638302
  -2.465332696362  123.587154919736  16.307187552709
  -10.608474638302  16.307187552709  123.960669810770
INDEX    LABEL    POSITION (Angstrom)    FORCE (eV/Angstrom)    VELOCITY (Angstrom/fs)
  0  Si  5.389827754452  2.701629481339  2.699621723525  -0.602062173578  0.391990779819
  0.096323485153  -0.010548689866  0.004659227844  0.001267888485
  1  Si  1.308600113700  1.373562332082  1.353979669688  0.602062173578  -0.391990779819
  -0.096323485153  0.010548689866  -0.004659227844  -0.001267888485

MDSTEP:  2
LATTICE_CONSTANT: 5.397605400000 Angstrom
LATTICE_VECTORS
  0.500000000000  0.500000000000  0.000000000000
  0.500000000000  0.000000000000  0.500000000000
  0.000000000000  0.500000000000  0.500000000000
VIRIAL (kbar)
  122.953007762375  -0.240072876170  -4.211946086929
  -0.240072876170  122.930693852071  3.378536464243
  -4.211946086929  3.378536464243  122.992607262385
INDEX    LABEL    POSITION (Angstrom)    FORCE (eV/Angstrom)    VELOCITY (Angstrom/fs)
  0  Si  5.376757913710  2.708010452044  2.701308224779  -0.124946902346  0.155367961328
  0.009378252255  -0.012739769308  0.006644009927  0.001613591879
  1  Si  1.321669954442  1.367181361377  1.352293168434  0.124946902346  -0.155367961328
  -0.009378252255  0.012739769308  -0.006644009927  -0.001613591879

MDSTEP:  3
LATTICE_CONSTANT: 5.397605400000 Angstrom
LATTICE_VECTORS
  0.500000000000  0.500000000000  0.000000000000
  0.500000000000  0.000000000000  0.500000000000
  0.000000000000  0.500000000000  0.500000000000
VIRIAL (kbar)
```

```
  122.881221914392  1.206247195689  2.345023988024
  1.206247195689  123.110079832453  -8.436042066928
  2.345023988024  -8.436042066928  123.123976300784
INDEX    LABEL    POSITION (Angstrom)    FORCE (eV/Angstrom)    VELOCITY (Angstrom/fs)
  0  Si  5.364727401370  2.714719749381  2.702800880457  0.310711926370  -0.086233730382
  -0.043760493686  -0.009222308089  0.005510922784  0.001122226311
  1  Si  1.333700466782  1.360472064040  1.350800512757  -0.310711926370  0.086233730382
  0.043760493686  0.009222308089  -0.005510922784  -0.001122226311
```

**2 MD 续算功能**

如果 ABACUS 运行达到指定的 MD 步数，或者计算中断，可以通过 MD 续算功能继续之前的 MD 计算。以上一个 INPUT 文件为例：

```
INPUT_PARAMETERS
#Parameters        (General)
suffix              autotest
calculation         md
pseudo_dir          ../../PP_ORB

esolver_type        dp
pot_file            ../../PP_ORB/Al-SCAN.pb

cal_force           1
cal_stress          1

md_nstep            3
md_type             msst
md_dt               1
md_tfirst           200
md_dumpfreq         1
md_restartfreq      1
md_restart          1

msst_qmass          200
msst_vel            0.028
msst_vis            0.3

init_vel            1
```

md_restart：控制续算的开关，在 MD 续算时将这个参数设为 1，其他参数不变。当 md_restart 设为 1，ABACUS 会读取 $read_file_dir/Restart_md.dat 文件，从中获取当前 MD 步数 istep 以及续算所需的其他参数如恒温器以及恒压器的信息，根据 istep 从 OUT.$suffix/STRU/ 文件夹中读取相应的结构文件 STRU_MD_$istep，之后就可以进行 MD 续算了。

**四、MD 后处理**

目前主流的可视化软件如 VESTA、VMD、OVITO 并不支持 ABACUS 的文件格式，因此我们需要通过一些后处理软件实现 ABACUS 的 STRU、MD_dump 文件与常用文件格式的转化。我们比较推荐的后处理软件有：

### 1 ASE

官网：https://wiki.fysik.dtu.dk/ase/
用法参考 ABACUS 线上文档：http://abacus.deepmodeling.com/en/latest/advanced/interface/ase.html

### 2 dpdata

github 网址：https://github.com/deepmodeling/dpdata
gitee 网址：https://gitee.com/deepmodeling/dpdata

### 3 案例

现在我们用 LJMD 算例来演示如何采用 dpdata+OVITO 的方式在 MD 计算完成后制作分子动力学轨迹动画。

### MD 计算

下载案例文件

```
$ git clone https://gitee.com/mcresearch/abacus-user-guide
```

进入 LJMD 文件夹

```
$ cd abacus-user-guide/examples/md/2_LJMD
```

修改 INPUT 参数，把 md_nstep 改为 100

```
INPUT_PARAMETERS
#Parameters (General)
suffix              autotest

calculation         md
pseudo_dir          ../../PP_ORB

#Parameters (esolver)
esolver_type        lj
lj_rcut             8.5
lj_epsilon          0.01032
lj_sigma            3.405

cal_force           1
cal_stress          1

#Parameters (MD)
md_nstep            100
md_type             nve
md_dt               1
md_tfirst           300

init_vel            1
```

运行 abacus

```
$ abacus
```

### dpdata 转化格式

安装 dpdata

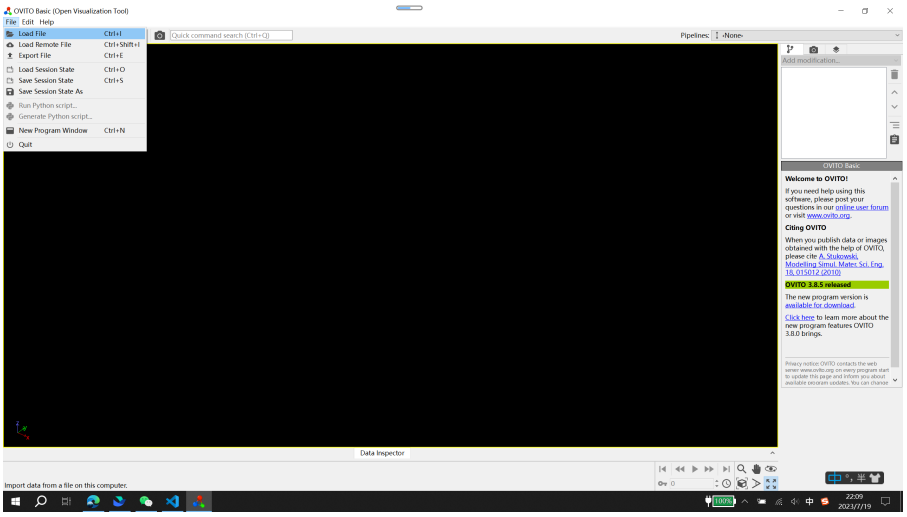
```
$ git clone https://github.com/deepmodeling/dpdata.git dpdata
$ cd dpdata
$ pip install .
```

运行 dpdata，将 MD_dump 文件转化为 GROMACS 的 gro 格式的文件 data.gro

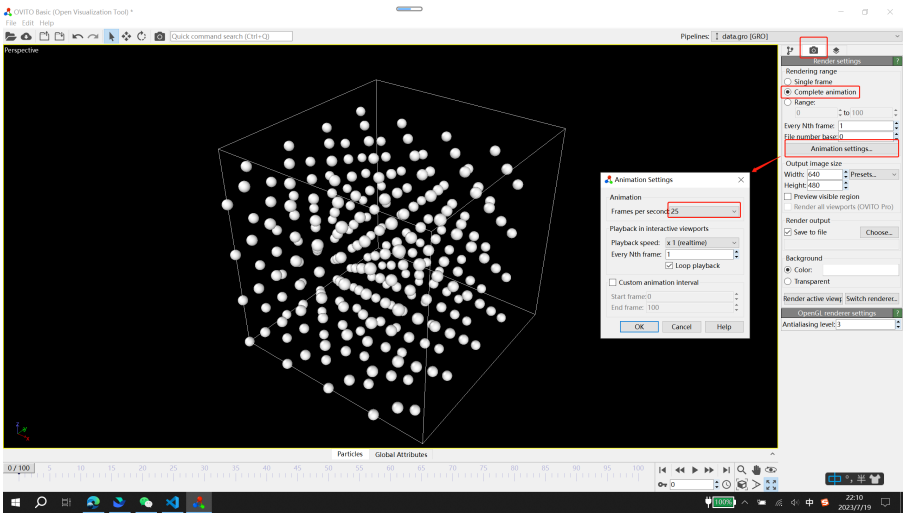
```
\$ python3
>>> import dpdata
>>> data = dpdata.LabeledSystem("./", fmt="abacus/md")
>>> data.to_gro("data.gro")
```

### OVITO 制作动画

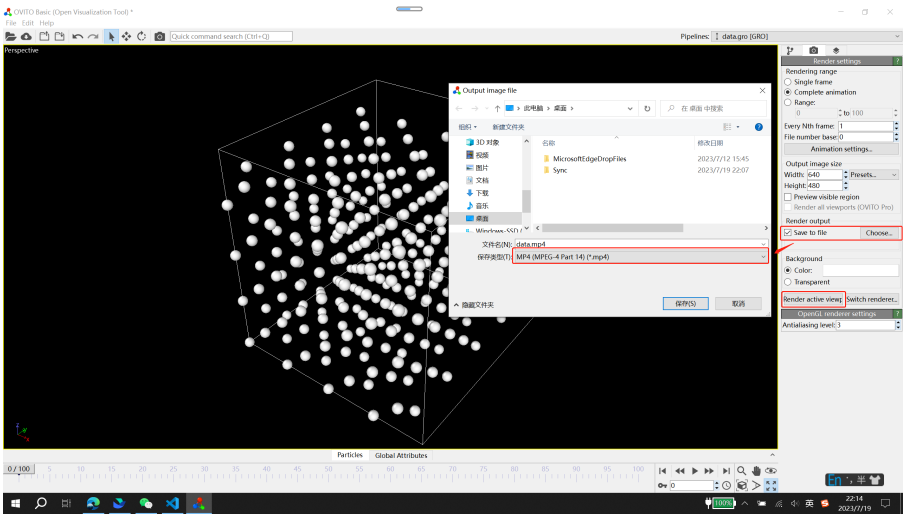
打开 OVITO，点击左上角 load file，选择上一步的 data.gro 文件




右上角 rendering settings



点击 Render active viewport 即可保存动画文件



## 五、特殊的 MD 方法

### 1. Fast Inertial Relaxation Engine

ABACUS 中实现了一种特殊的基于 MD 的 Fast Inertial Relaxation Engine (FIRE) 结构优化方法，该算法比共轭梯度法的标准实现快得多，具有额外的速度修改和自适应时间步长，具有优异的优化效率。

以上面提供的 4_FIRE 算例为例，我们可以介绍一下所需的参数如下：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix                  Si_fire
calculation             md
nbands                  6
symmetry                0
pseudo_dir              ../../PP_ORB
orbital_dir             ../../PP_ORB

```

```
#Parameters (2.Iteration)
ecutwfc                 30
scf_thr                 1e-8
scf_nmax                100

#Parameters (3.Basis)
basis_type              pw

#Parameters (4.Smearing)
smearing_method         gaussian
smearing_sigma          0.001

#Parameters (5.Mixing)
mixing_type             pulay
mixing_beta             0.3
chg_extrap              second-order

#Parameters (6.MD)
md_type                 fire
md_nstep                100
md_dt                   1
md_tfirst               300

force_thr_ev            0.001
```

calculation：FIRE 算法是基于 MD 方法的，因此需要将 calculation 设为 md

md_type：采用 FIRE 算法，需要设为 fire

md_tfirst：MD 需要离子速度，因此需要通过设置初始温度来随机生成离子速度，或者在 STRU 文件中手动设置初始速度

force_thr_ev：离子受力的收敛阈值，当离子受力的每个分量均小于该阈值或者 MD 运行至最大步数时，MD 终止

### 4.2.2 ABACUS+DeePMD-kit 做机器学习分子动力学

**一、介绍**

本教程旨在介绍结合 ABACUS（中文名原子算筹，这里基于 ABACUS 3.8.0 版本）和 DeePMD-kit 软件进行基于机器学习原子间势函数的分子动力学模拟。

DeePMD-kit 是一款基于神经网络拟合第一性原理数据得到势能模型，用于分子动力学模拟的软件。无需人工干预，其可以端对端地将用户提供的数据在数个小时内转化为深度势能模型，可以保持量子力学精度准确性的基础上，将分子动力学的计算速度提升数个量级。

如果不熟悉 DeePMD-kit 的读者，这里可以推荐大家先阅读 DeePMD-kit 的相关原理文档和说明，熟悉这部分的读者可以跳过：

https://docs.deepmodeling.com/projects/deepmd/en/r2/

https://bohrium-doc.dp.tech/docs/software/DeePMD-kit

https://mp.weixin.qq.com/s/hLULtXgwKbhuT2Zvy6XZLA

https://bohrium.dp.tech/notebooks/2649266844

## 二、准备

### 1. 案例下载地址

ABACUS 的 MD 算例下载地址为（国内 gitee）：
https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/md
可以采用的下载命令是：
git clone https://gitee.com/mcresearch/abacus-user-guide.git
之后进入 /abacus-user-guide/examples/md/3_DPMD 目录
或者采用 Github 仓库地址：
https://github.com/MCresearch/abacus-user-guide/tree/master/examples/md

### 2. 安装相关软件

#### ABACUS

这里默认读者已经准备好 ABACUS 的安装环境和依赖软件，如还没有可以参考相关教程。

#### DeePMD-kit

DeePMD-kit 安装教程请参考 DeePMD-kit 的线上文档：
https://docs.deepmodeling.com/projects/deepmd/en/r2/
新手用户建议采用 offline package 方式安装 DeePMD-kit：
https://docs.deepmodeling.com/projects/deepmd/en/r2/getting-started/install.html#install-off-line-packages
如果用户希望直接使用 DPA2 做基于 ABACUS 软件的分子动力学模拟，需要满足以下条件：

安装 DeePMD-kit v3 版本
DeePMD-kit v3 版本应该与采用的 DPA2 训练使用的 DeePMD-kit 版本一致
DeePMD-kit v3 支持 PyTorch backend

#### ABACUS + DeePMD-kit

cmake 安装方式：
在普通 abacus 构建项目命令的基础上，指定 DeePMD-kit 的地址即可编译支持 DP 势场的 ABACUS：

```
$ cmake -B build -DDeePMD_DIR=/dir_to_deepmd-kit ...
$ cmake --build build -j4
$ cmake --install build
```

如果采取的不是 offline packages 安装方式，则 TensorFlow 和 PyTorch 的地址可能与 DeePMD-kit 不同，则需要额外指定他们的地址：

```
$ cmake -B build -DDeePMD_DIR=/dir_to_deepmd-kit -DTensorFlow_DIR=/dir_to_tensorflow -DTorch_DIR=
/dir_to_pytorch ...
$ cmake --build build -j4
$ cmake --install build
```

TensorFlow 和 PyTorch 后端可能只有一个，也可能都有，用户应该根据自己的需求安装支持对应后端的 DeePMD-kit。如果只有 TensorFlow 后端，则只需要提供 TensorFlow_DIR 即可，PyTorch 后端同理
make 安装方式:
修改 Makefile.vars 文件，指定 DeePMD_DIR，LIBTORCH _DIR（可选）和 TensorFlow_DIR（可选）：

```
DeePMD_DIR      = /dir_to_deepmd-kit
LIBTORCH_DIR    = /dir_to_pytorch
TensorFlow_DIR  = /dir_to_tensorflow
```

或者执行 make 命令时，指定 DeePMD_DIR，LIBTORCH_DIR（可选）和 TensorFlow_DIR（可选）：

```
$ make DeePMD_DIR=/dir_to_deepmd-kit  LIBTORCH_DIR=/dir_to_pytorch  TensorFlow_DIR=
/dir_to_tensorflow
```

编译可能出现的问题:
采用 offilne 方式安装 DeePMD-kit，并使用 intel 编译器编译 ABACUS 时，可能会出现 mkl 库冲突，如

```
lib/libmkl_sequential.so.2: 对'mkl_lapack_sgesvda_batch_kernel' 未定义的引用
```

为避免影响原有的 DeePMD-kit，可以新建目录/another_dir_to_deepmd，将原地址下的 lib 和 include 文件夹复制至该目录下，然后删除 lib 中所有带 mkl 字符的库文件

```
$ cd /another_dir_to_deepmd/lib
$ rm *mkl*
```

### 三、机器学习分子动力学模拟

#### INPUT 文件设置

```
INPUT_PARAMETERS
#Parameters        (General)
suffix              autotest
calculation         md

esolver_type        dp
pot_file            ../../PP_ORB/Al-SCAN.pb

cal_force           1
cal_stress          1

md_nstep            3
md_type             msst
md_dt               1
md_tfirst           200
md_dumpfreq         1
md_restartfreq      1

msst_qmass          200
msst_vel            0.028
msst_vis            0.3

init_vel            1
```

这些参数在 ABACUS 线上文档中均有说明，在这里再进行简单概述：

calculation：设置 ABACUS 计算类型，做分子动力学模拟请设置为 md

esolver_type：给定原子位置后的能量求解方式，默认 Kohn-Sham 密度泛函理论（ksdft），还可以设置 LJ 势（lj）或者深度势能（dp），这里设置为 dp

md_type：MD 算法种类，例如选择不同的系综 nve、nvt、npt 等

md_nstep：MD 模拟的总步数

md_dt：MD 计算每一步的时间步长（单位是 fs），与 md_nstep 共同决定 MD 总时长

md_tfirst：MD 系统的初始温度（单位是 K）

pot_file：DP 模型文件路径

md_dumpfreq：MD 输出文件 MD_dump 中原子以及晶胞信息的输出频率

md_restartfreq：结构文件 STRUMD$step 的输出频率，MD 续算文件 Restart_md.dat 的更新频率

init_vel：从 STRU 文件中读取原子速度

**STRU 文件**

```
ATOMIC_SPECIES
Al 26.9815

LATTICE_CONSTANT
36.9005650344

LATTICE_VECTORS
1 0 0 #latvec1
0 1 0 #latvec2
0 0 1 #latvec3

ATOMIC_POSITIONS
Cartesian

Al #label
0 #magnetism
864 #number of atoms
0  0  0  m  1  1  1  v  0.000116448153697  -3.54062829069e-05  6.37221822545e-05
0.0833333333321  0.0833333333321  0  m  1  1  1  v  6.31461715716e-05
-7.68881397243e-05  0.000126053061667
0.0833333333321  0  0.0833333333321  m  1  1  1  v  -0.000203773691434
5.08681990052e-05  -5.02990480666e-05
0  0.0833333333321  0.0833333333321  m  1  1  1  v  0.00024862569861
-0.000253289379344  2.04134042994e-05
0  0  0.166666666668  m  1  1  1  v  8.86312919936e-05  0.000160542135344  -0.000121839904824
0.0833333333321  0.0833333333321  0.166666666668  m  1  1  1  v  2.55030351768e-05
-3.68630188311e-05  2.89192521276e-05
0.0833333333321  0  0.25  m  1  1  1  v  -0.000135330676077  -3.26957881398e-05  3.15103467938e-07
0  0.0833333333321  0.25  m  1  1  1  v  0.000131365976691  9.92067819339e-05  -0.00017808361164
...
...
...
```

DPMD 除了不需要提供 KPT，赝势和轨道文件之外，其他设置与正常计算一致。

## 4.3 采用 ABACUS 进行表面计算（一）：静电势和功函数

### 4.3.1 背景

静电势（electrostatic potential），也叫电势，是指将单位电荷从参考点移动到场内某一点所做的功。

在密度泛函理论的框架下，静电势 $V_{static}$ 定义为

$$V_{static} = V_H + V_{ext} + V_{efield} + V_{dipole} + \cdots$$

其中 $V_H = \int \frac{\rho(r')}{|r-r'|} dr'$ 为电子产生的静电势，$V_{ext}$ 为离子势（赝势的局域部分），$V_{field}$ 为外加电场项，$V_{dipole}$ 为偶极修正项，输出静电势和输出局域势的差别在于有没有输出交换关联势。

在处理表面等体系时，静电势可用于计算功函数等物理量，并进一步用于催化活性、化学反应机理以及电子输运性质等性质的计算与分析。

ABACUS 提供了直接输出静电势的接口，并提供了对其进行后处理的 python 脚本。

### 4.3.2 ABACUS 中计算静电势具体流程

此功能在平面波基组、原子轨道基组下的 Kohn-Sham DFT（KSDFT），以及随机波函数密度泛函理论和无轨道密度泛函理论中均适用。接下来，我们以平面波基组下的 KSDFT 为例介绍其使用方法，如果需要使用其它的能量求解器，只需要设置 INPUT 文件里的 esolver_type、basis_type 更换能量求解器并设置相应求解器的参数即可。

#### 计算实例

#### 输出静电势

下面是输入文件的示例：

INPUT 文件记录计算所需主要参数，为了输出静电势，只需要设置 out_pot 2 即可（设置 out_pot 1 时，会输出总的局域势，2 相比于 1 少输出了交换关联势，具体可以参考线上文档 https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#out-pot）。

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix                  example
calculation             scf
ntype                   1
nbands                  40
symmetry                1
pseudo_dir              .

#Parameters (2.Iteration)
ecutwfc                 60
scf_nmax                100

#Parameters (3.Basis)
basis_type              pw

#Parameters (4.Smearing)
```

```
smearing_method         gauss
smearing_sigma          0.0074

#Parameters (5.Mixing)
mixing_type             pulay
mixing_beta             0.7

out_pot                 2
```

STRU 文件记录元素种类、质量、赝势，晶格矢量，原子坐标等信息

```
ATOMIC_SPECIES
Al 13 Al_ONCV_PBE-1.0.upf

LATTICE_CONSTANT
7.6513590200098225  // add lattice constant

LATTICE_VECTORS
0.707106781187    0.000000000000    0.000000000000
0.000000000000    0.707106781187    0.000000000000
0.000000000000    0.000000000000    5.000000000000

ATOMIC_POSITIONS
Direct

Al
0.0
5
    0.000000000000    0.000000000000    0.000000000000 1 1 1
    0.500000000000    0.500000000000    0.100000000000 1 1 1
    0.000000000000    0.000000000000    0.200000000000 1 1 1
    0.500000000000    0.500000000000    0.300000000000 1 1 1
    0.000000000000    0.000000000000    0.400000000000 1 1 1
```

KPT 文件

```
K_POINTS
0
Gamma
20 20 1 0 0 0
```

如上所示，为了输出静电势，只需要在 INPUT 中设置 out_pot“ “2，计算完成后即可在 OUTOUT.$suffix 文件夹下找到下面的 ElecStaticPot.cube 文件。

ElecStaticPot.cube 文件（记录了实空间格点上的静电势，单位是 A.U.，下面的文档中，我们在括号中加入了注释。此文件可以直接在 VESTA 中可视化）

```
Cubefile created from ABACUS SCF calculation. The inner loop is z index, followed by y index, x index in turn.
1 (nspin)
```

```
5 (总原子数) 0.0 0.0 0.0 (原点坐标)
27 (沿 x 轴格点数 nx) 0.200383 0.000000 0.000000 (晶格矢量 a1 的 1/nx, in Bohr。即晶格常数 a*a1/nx)
27 (沿 y 轴格点数 ny) 0.000000 0.200383 0.000000 (晶格矢量 a2 的 1/ny, in Bohr)
192 (沿 z 轴格点数 nz) 0.000000 0.000000 0.199254 (晶格矢量 a3 的 1/nz, in Bohr)
 13 (第一个原子的原子序数) 11 (赝势中的价电子数) 0.000000 0.000000 0.000000 (该原子位置的笛卡尔坐标, in Bohr)
 13 11 2.705164 2.705164 3.825680
 13 11 0.000000 0.000000 7.651359
 13 11 2.705164 2.705164 11.477039
 13 11 0.000000 0.000000 15.302718
 (空间格点上的数据, z 坐标变化最快, 接下来依次是 y, x, 一般为 6 个数据一行, 每走完 z 的一次循环换一行)
 -1.808618025e+01 -1.683249579e+01 -1.326226499e+01 -8.726520963e+00 -4.859516723e+00 -2.648153424e+00
 -1.754646057e+00 -1.082969997e+00 -7.712730497e-01 -5.301070017e-01 -1.865686936e-01 1.178808012e-03
 1.531960521e-01 2.434659807e-01 3.236379649e-01 3.580121711e-01 3.871087692e-01 4.067752520e-01
 4.191236489e-01 4.303464392e-01 4.269139054e-01 4.072584618e-01 4.012863417e-01 3.694437712e-01
 3.419696568e-01 2.751686292e-01 1.986707660e-01 7.265002162e-02 -9.800789596e-02 -3.699329212e-01
 ...
 5.605648983e-01 5.596641131e-01 5.601221783e-01 5.558024312e-01 5.582286739e-01 5.509517334e-01
 5.484144631e-01 5.474458329e-01 5.413456797e-01 5.310960172e-01 5.220139261e-01 5.057932112e-01
 4.893954664e-01 4.661977108e-01 4.379332539e-01 3.906110446e-01 3.390039472e-01 2.575599458e-01
 1.632711220e-01 1.092880266e-02 -1.657977733e-01 -5.027618535e-01 -7.758727829e-01 -1.023213291e+00
 -1.643162715e+00 -2.406825325e+00 -4.139043775e+00 -7.343485448e+00 -1.119104878e+01 -1.435478802e+01
```

**后处理**

为了方便计算沿着 z 方向的平均静电势，我们在 tools/average_pot 下提供了 python 后处理脚本 aveElecStatPot.py，在存放 ElecStaticPot.cube 的文件夹下运行

python path_to_abacus/tools/average_pot

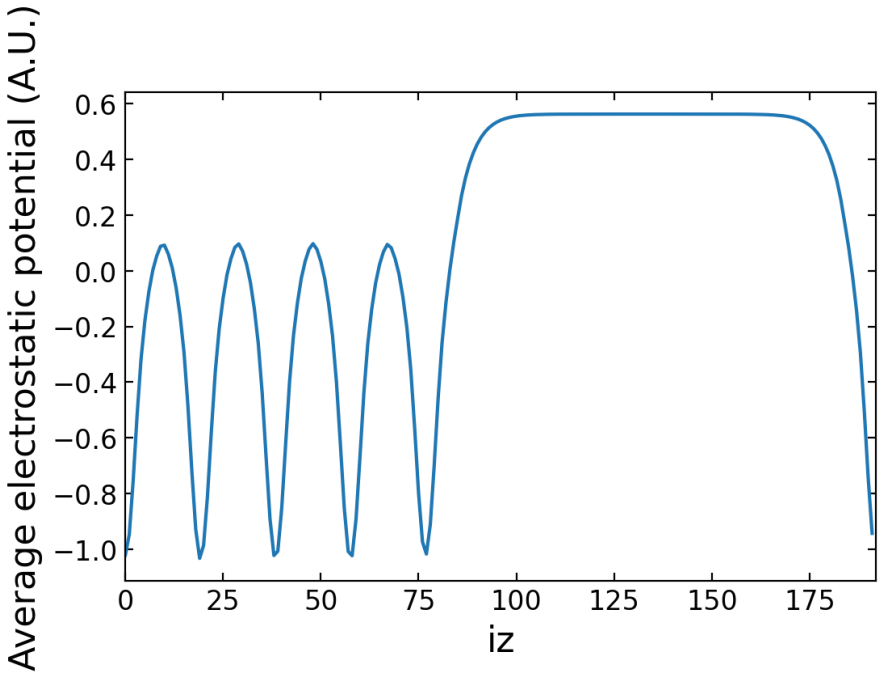
即可在此文件夹中得到存放了平均静电势的 ElecStaticPot_AVE 文件，如下所示。

ElecStaticPot_AVE 文件（存放了沿 z 方向的平均静电势，单位是 A.U.。此文件的前 11 行与 ElecStaticPot.cube 相同，之后的第一列数为沿着 z 轴的格点指标，第二列数为平均静电势）

```
Average electrostatic potential along z axis
1 (nspin)
5 0.0 0.0 0.0
27 0.200383 0.000000 0.000000
27 0.000000 0.200383 0.000000
192 0.000000 0.000000 0.199254
 13 11 0.000000 0.000000 0.000000
 13 11 2.705164 2.705164 3.825680
 13 11 0.000000 0.000000 7.651359
 13 11 2.705164 2.705164 11.477039
 13 11 0.000000 0.000000 15.302718
iz                 average
0                 -1.021155030e+00
1                 -9.476023401e-01
2                 -7.542571394e-01
3                 -5.200099451e-01
...
```

```
189                -5.039597208e-01
190                -7.447469356e-01
191                -9.431350596e-01
```

可视化后如图所示：




**功函数**

功函数（work function），又称逸出功，描述把电子从固体内部移到真空中所需的最小能量。

在 DFT 计算中，它可以定义为 $V_{vaccum} - E_F$，其中 $V_{vaccum}$ 为真空静电势，可以在 `ElectricPot_AVE` 中读到，$E_F$ 为体系费米能，可以用 `grep EFERMI OUT.$suffix/running_scf.log` 命令从 `OUT.$suffix/running_scf.log` 文件中提取出来。

对于上面的例子，$V_{vaccum} = 0.563\,\text{Ry}$，$E_F = 3.3293551883\,\text{eV} \approx 0.245\,\text{Ry}$，因此功函数为 $V_{vaccum} - E_F = 0.318\,\text{Ry}$。

如果大家使用有问题，欢迎写信联系（见上）。

## 4.4 采用 ABACUS 进行表面计算（二）：偶极修正

### 4.4.1 偶极修正介绍（Dipole Correction）

偶极矩（Dipole Moment）是一个矢量，由正、负电荷中心距离和其所带电量乘积得到，方向规定为正电中心指向负电中心，采用原子建模表面时，往往表面会产生偶极矩。

ABACUS 在 xyz 三个方向都采用周期性边界条件（Periodic Boundary Conditions），当使用密度泛函理论（Density Functional Theory，简称 DFT）进行表面体系的电子结构计算时常采用超胞（Supercell）近似，这时所研究的体系在真空方向会出现两个表面。当两个表面不对称时（例如在某表面进行吸附原子计算），这两个表面的电荷密度分布就会不同，这个时候两个表面的偶极密度不同且表面附近的静电势（Electrostatic Potential）会呈现

不同的值。然而，周期性边界条件的存在会强制使得静电势也要满足周期性边界条件，这个时候周期性边界条件的存在相当于制造了一个非物理的人造电场（Artificial Electric Field）。

通常情况下，降低这个人造电场的方法是加大真空层的厚度，使得相邻的超胞之间没有相互作用，但这往往要大幅增加密度泛函理论的计算量。因此，为了消除周期性边界条件对表面偶极矩带来的影响，我们可以采用偶极修正（Dipole Correction）的方法，通过引入另外一个外电场来抵消这个人造电场，但注意引入的外电场需要存在于真空中（其位置是程序输入参数的一部分）才不至于影响所要研究的表面体系。经过修正后，人造电场将被抵消，真空电势则维持不变。

ABACUS 中实现了偶极修正功能 [1]，可以有效的加速静电势收敛，避免超胞的真空取得太长，从而可以降低计算量。该修正功能主要体现在 Hartree 项的处理上，并且存在相应的原子受力的修正，可以用于平面波（Plane-Wave）基组和数值原子轨道（Numerical Atomic Orbitals）基组计算，但注意该功能只能用于带真空层的表面体系。

### 4.4.2 文档与算例

ABACUS 里关于偶极修正的线上文档地址为：

http://abacus.deepmodeling.com/en/latest/advanced/scf/advanced.html#dipole-correction

ABACUS 里关于偶极修正参数的线上文档地址为：https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#electric-field-and-dipole-correction

ABACUS 的偶极修正算例下载地址为（gitee）：

https://gitee.com/MCresearch/abacus-user-guide/tree/master/examples/dipole_correction/H2O

可以采用的下载命令是：

```
git clone https://gitee.com/MCresearch/abacus-user-guide
```

之后进入 /abacus-user-guide/examples/“dipole_correction/H2O/ 目录

或者采用 Github 仓库地址：

https://github.com/MCresearch/abacus-user-guide/tree/master/examples/dipole_correction/H2O

### 4.4.3 偶极修正输入参数介绍

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix              H2O
calculation         scf
nbands              6
symmetry            0
pseudo_dir          ../../PP_ORB

#Parameters (2.Iteration)
ecutwfc             100
scf_thr             1e-6
scf_nmax            100

#Parameters (3.Basis)
basis_type          pw

#Parameters (4.Smearing)
```

```
smearing_method     fixed

#Parameters (5.Mixing)
mixing_type         pulay
mixing_beta         0.7

#Parameters (6.Dipole correction)
efield_flag         1
dip_cor_flag        1
efield_pos_max      0.0
efield_pos_dec      0.1
efield_amp          0.00

#Parameters (7.Output)
out_pot             2
```

以算例提供的 INPUT 文件为例，使用偶极修正方法只需添加如下参数即可：

efield_flag：类型是 Bool，指定外加电场的开 (1) 或者关 (0)，默认值为 0，注意应用偶极修正功能时，efield_flag 必须设为 true

dip_cor_flag：类型是 Bool，指定偶极修正的开 (1) 或者关 (0)，默认值为 0

efield_dir：类型是 Int，外加电场方向，可取 0，1，2 分别表示 x，y，z 方向，默认值为 2

efield_pos_max：类型是 Real，锯齿状电势最大值所在分数坐标，范围 [0,1)，默认值为 0.5

efield_pos_dec：类型是 Real，锯齿状电势从最大值到最小值的长度（同样以分数坐标为度量），默认值为 0.1，范围 [0,1)。以图 1 为例，0.5 是最大值，0.6 是最小值，那么 0.5  0.6 的区域必须放置在没有原子的真空层中

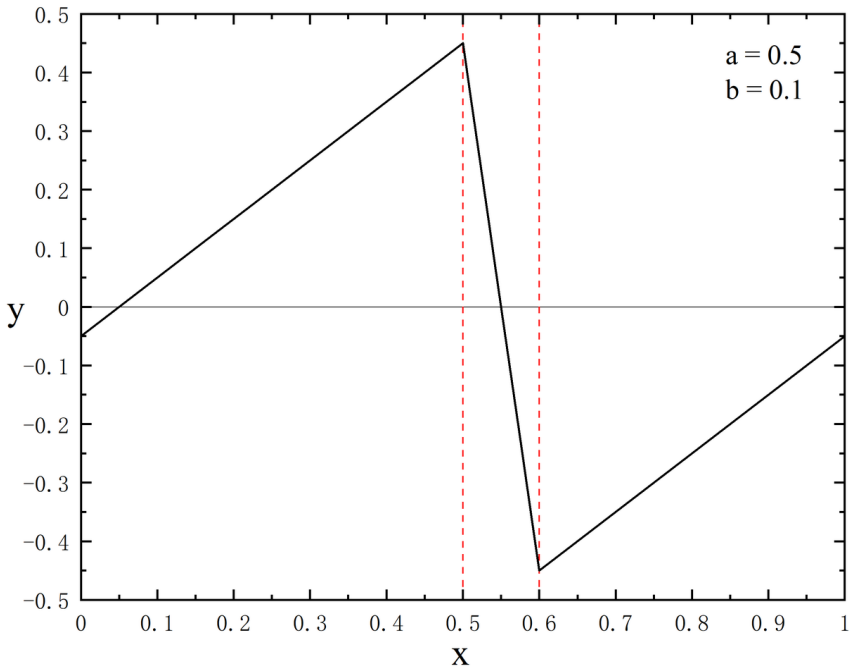



图 1. 锯齿状势场分布图

efield_amp：类型是 Real，电场强度，默认值为 0，单位为 a.u.，其中 1 a.u. = 51.4220632*1010 V/m。在偶极修正时需要设为 0，不为 0 则说明同时添加了外加电场和偶极修正

### 4.4.4 预期结果

如图 2 所示，一个水分子位于超胞（尺寸为 $3 \times 3 \times 18$ A$^3$）的中部（即位于 9 Å），那么按照上一部分的说明，锯齿应该放在没有原子的真空处，因此我们将电势的最大值所在的位置设为 efield_pos_max 设为 0.0，之后设置电势从最大值到最小值的长度，即将 efield_pos_dec 设为 0.1 （在这个例子是 1.8 Å），如此锯齿即位于 0.0 ∼ 0.1 的区域。

为了验证结果的正确性，我们可以画出静电势沿 $z$ 轴（$c$ 方向）的变化如图 3 所示。在水分子上方（9-18 Å）和下方（0-9 Å）的真空层中，静电势在很长一段区间保持常数不变，出现了两个平台，这说明我们的偶极修正功能使用无误。静电势计算和作图方法请参考静电势和功函数计算文档。

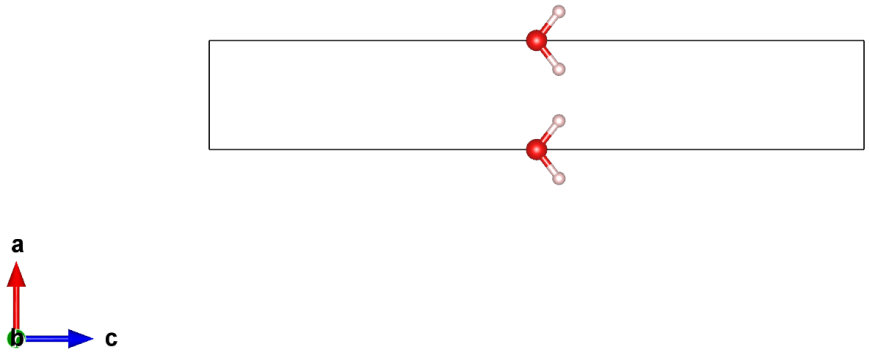



图 2. 一个水分子位于超胞中，红色代表氧原子，白色代表氢原子

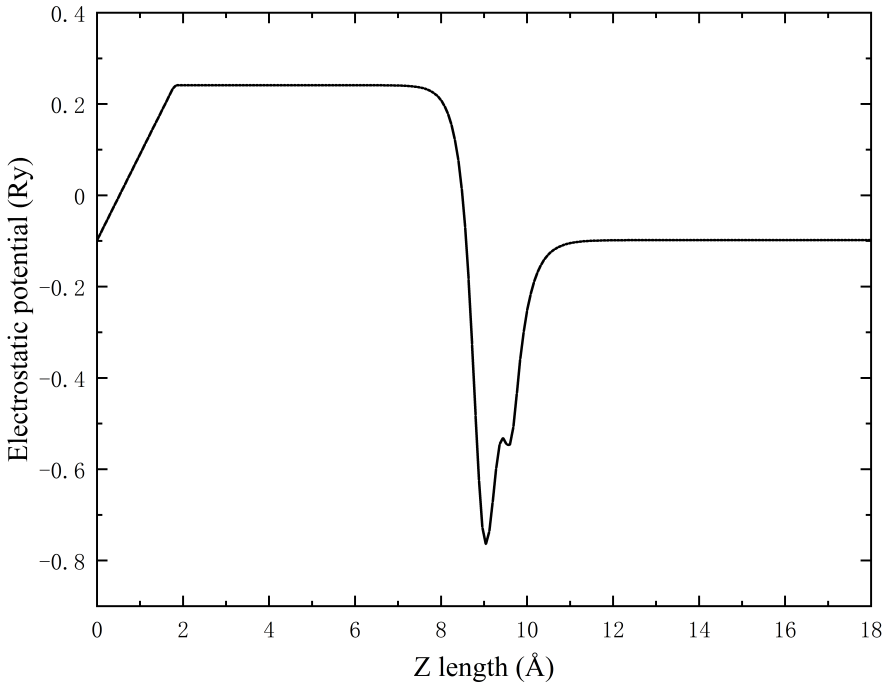



图 3. 静电势沿超胞 Z 轴变化图

### 4.4.5 参考文献

[1] Phys. Rev. B 59, 12301 (1999), https://doi.org/10.1103/PhysRevB.59.12301

# 4.5 采用 ABACUS 进行表面计算（三）：表面能计算

## 4.5.1 背景

将某一固体沿着某一晶面方向切开会形成相应的表面，许多化学反应都是依托在一些特定的表面系统上展开的。因此，计算表面能也是第一性原理计算经常会遇见的。

表面能（surface energy）定义为单位面积内表面体系与相应的体材料（bulk systems）能量之差，或者建立某特定表面需要做的功。不同表面的原子排布会对应不同大小的表面能，从能量的观点来看，表面能越低的表面体系越稳定，在现实世界里就越有可能稳定存在。

另外，需要注意的是，当沿着固体的某一晶面切开形成表面时，这个表面不一定是稳定结构。由于表面原子所经历的周围环境相比于体材料有所变化，所以往往表面原子会重新排布，只是排布的幅度有大有小。所以，一般来说，需要对表面进行结构优化计算，达到能量最低的稳定态。有一些常见的表面重构，已经被实验证实，例如硅的表面重构。

密度泛函理论（Density Functional Theory，简称 DFT）也常用来计算材料体系的表面能，但如果使用的是周期性边界条件，往往构造的原子体系会有两个表面，以及一段真空。注意真空的长度一般不能太小（一般大于 10 埃），要长到能量达到收敛。另外，如果表面是有极化的，可以采用 dipole correction 功能进行修正，这样真空往往可以取短一些。另外一方面，构成表面体系的原子层数一般来讲不少于 3-5 层，有时候需要检验更多层以达到收敛，达到收敛时体系最靠近中间的层一般和体材料的原子性质较为接近，这个可以作为表面厚度是否足够的判据之一，另外一个判据就是随着层数的增加，所得的表面能的值是否已经收敛。

用 DFT 计算材料体系的表面能时，需要进行两次自洽迭代（self-consistent field，简称 scf）计算，一次用于计算体材料体系中每原子的能量 $E_0$，一次用于计算表面体系的总能量 $E_{slab}$，最终表面能 σ 定义为

$$\sigma = \frac{E_{slab} - NE_0}{2A}$$

其中 $N$ 为表面体系包含的原子数，$A$ 为表面面积，分母上的 2 来自于如果计算中采用了周期性边界条件，即表面体系实际有上下两个表面。计算表面能时，需要计算 $E_0$ 和 $E_{slab}$ 两个能量。ABACUS 软件中，我们可以采用平面波基组、原子轨道基组下的 Kohn-Sham DFT（KSDFT），以及随机波函数密度泛函理论和无轨道密度泛函理论来计算它们。

## 4.5.2 ABACUS 计算表面能具体流程

接下来，我们以平面波基组下的 KSDFT 为例介绍其计算方法，如果需要使用其它的能量求解器，只需要设置 INPUT 文件里的 esolver_type、basis_type 更换能量求解器并设置相应求解器的参数即可。

以下是几个构造表面结构的注意点：

由于 ABACUS 在实空间的并行策略是将格点沿 z 轴分层，关于并行我们有两个建议：第一，为了使得 MPI 并行时负载平衡，建议沿 x 方向设置真空，即让表面的法线沿着 x 轴，避免有些核分不到原子，影响计算效率；第二，如果并行的核数较多，建议沿 z 方向设置真空，即让表面的法线沿着 z 轴，因为每个核至少要分到一些 z 方向的格点，选取长边会使得可以并行的核数更多。

真空层厚度一般设置在 10 埃以上，具体需要调整真空层厚度做收敛性测试；

表面结构（slab）需要用内部的原子模拟体材料，因此其厚度也要做收敛性测试。

**铝面心立方晶体 (100) 表面能计算**

算例链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Al_fcc100

**计算体材料能量**

这一步用晶格驰豫方法找到面心立方晶体 fcc Al 的平衡构型以及平衡能量。

链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Al_fcc100/0_bulk

运行完算例后，用 `grep FINAL_ETOT_IS OUT.example/running_cell-relax.log` 得到总能量为 `-1883.225 eV`，由于原胞内只有一个原子，因此 $E_0 = -1883.225\,\text{eV}$。

从 `OUT.example/STRU_ION_D` 文件中可以读到平衡晶格常数为 $a_0 = 7.586\,\text{Bohr}$，它将用于表面构型的构造。

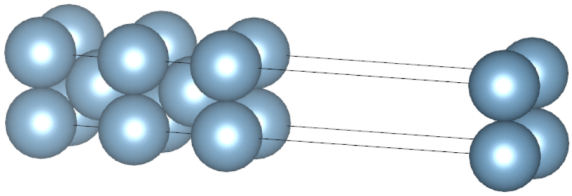
注意，如果是带有磁性的原子，计算体材料的时候也要加上磁性的计算，例如在 INPUT 文件里设置 `nspin=2`。

**计算表面体系能量**

这一步我们搭建表面体系并计算其总能量。

链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Al_fcc100/1_surface_5layers
https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Al_fcc100/2_surface_7layers

实际计算时，需要对表面构型的原子层数做收敛性测试，作为例子，我们提供了 5 层和 7 层的算例，其构型如下所示：



计算总能量时，可以根据实际需要选择是否做结构优化，这里为了结果更准确，同时兼顾效率，我们固定了内部原子，对表面上的原子结构进行了优化。

对于 5 层构型，用 `grep FINAL_ETOT_IS OUT.example/running_relax.log` 得到总能量为 `-9415.005 eV`，因此 $E_{slab} = -9415.005\,\text{eV}$。

对于 7 层构型，总能量 $E_{slab} = -13181.372\,\text{eV}$。

**计算表面能**

对于上述构型，表面面积 $A = (a_0/\sqrt{2})^2 = 8.057\,\text{Å}^2$。

对于 5 层构型，表面构型原子数 $N = 5$，因此表面能 $\sigma = \frac{E_{\text{slab}} - NE_0}{2A} = 0.070\,\text{eV/Å}^2 = 1113\,\text{mJ/m}^2$。

对于 7 层构型，表面构型原子数 $N = 7$，因此表面能 $\sigma = \frac{E_{\text{slab}} - NE_0}{2A} = 0.075\,\text{eV/Å}^2 = 1196\,\text{mJ/m}^2$。

可以看到 7 层的构型相比于 5 层算出的表面能还差别较大，所以需要增加层数一直到表面能收敛。

**2. 硅金刚石结构 (100) 表面能计算**

这里金刚石结构我们又名为 CD（Cubid Diamond）结构

算例链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Si_CD100

**计算体材料能量**

这一步我们用晶格驰豫方法找到 CD Si 的平衡构型以及平衡能量。

链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Si_CD100/0_bulk

运行完算例后，用命令 `grep FINAL_ETOT_IS OUT.example/running_cell-relax.log`

得到体系总能量为 `-214.516 eV`，由于原胞内有两个原子，因此 $E_0 = -107.258\,\text{eV}$。

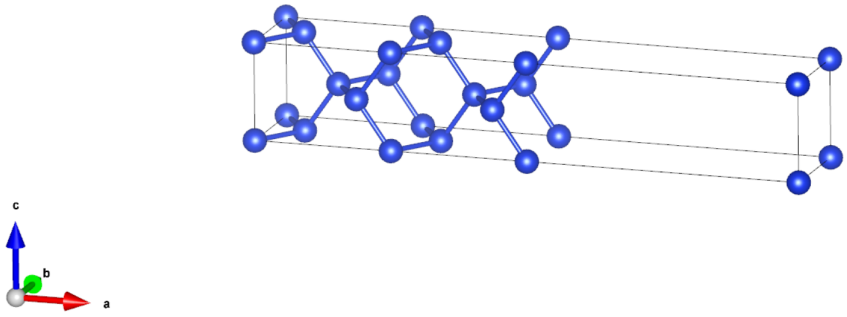
从 `OUT.example/STRU_ION_D` 文件中可以读到平衡晶格常数为 $a_0 = 10.351\,\text{Bohr}$，它将用于表面构型的构造。

**计算表面体系能量**

这一步我们搭建表面体系并计算其总能量。

链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_energy/Si_CD100/1_surface

我们搭建了一个 9 层的表面构型，如下图所示：




计算总能量时，可以根据实际需要选择是否做结构优化，这个例子里，为了计算省时，我们没有做结构优化。

用 `grep FINAL_ETOT_IS OUT.example/running_scf.log` 得到总能量为 `-961.337 eV`，因此

$$E_{\text{slab}} = -961.337\,\text{eV}.$$

**计算表面能**

对于上述构型，表面面积 $A = (a_0/\sqrt{2})^2 = 15.015\,\text{Å}^2$，表面构型原子数 $N = 9$，因此表面能

$$\sigma = \frac{E_{\text{slab}} - NE_0}{2A} = 0.133\,\text{eV/Å}^2 = 2126\,\text{mJ/m}^2.$$

与实验值 2131 $\text{mJ/m}^2$ [1] 非常接近，但略大于另一个实验值 1361 $\text{mJ/m}^2$ [2]。

注：以上为表面能计算示例，若有问题欢迎邮件联系（见上）。

### 4.5.3 参考文献

[1] Eaglesham D J, White A E, Feldman L C, et al. Equilibrium shape of Si[J]. Physical Review Letters, 1993, 70(11): 1643.

[2] Messmer C, Bilello J C. The surface energy of Si, GaAs, and GaP[J]. Journal of Applied Physics, 1981, 52(7): 4623-4629.

## 4.6 采用 ABACUS 进行表面计算（四）：表面缺陷能和吸附能计算

### 4.6.1 背景

在研究表面的稳定性时，我们经常关注表面上形成空穴、或者表面上吸附其它分子的难易程度，这两种现象可以分别用表面缺陷能和表面吸附能描述。

表面缺陷能（surface vacancy energy）定义为在表面上形成原子的空穴所需能量，缺陷能越低，则该表面越容易形成缺陷。

表面吸附能（surface adsorption energy）定义为原子、离子或分子（吸附物，adsorbate）附着在固体表面（吸附剂，adsorbent）的吸附过程中，二者结合时的能量下降，它描述了吸附剂对吸附物的吸附能力，吸附能越低，则吸附能力越强。

由于缺陷间或吸附物间的相互作用，上述两种物理量都与缺陷或吸附物在表面的覆盖率（coverage）θ 有关，其定义为 $\theta = N_{v/a}/N_{\text{surface}}$，即缺陷数或吸附物数与表面原子数之比。覆盖率越低，则缺陷间或吸附物间相互作用越弱，计算时所需的超跑也越大，实际计算中，可根据需求选择合适的覆盖率。本文档中，为了节省计算量，采取较大的覆盖率 $\theta = 0.25$。

下面我们分别介绍这两个物理量的计算方法，并展示如何使用 ABACUS 计算。

ABACUS 中，我们可以采用平面波基组、原子轨道基组下的 Kohn-Sham DFT（KSDFT）和无轨道密度泛函理论等电子结构计算方法来计算所需的能量，也可以调用 DeepMD 势函数。接下来，我们以平面波基组下的 KSDFT 为例介绍其计算方法，如果需要使用其它的能量求解器，只需要设置 INPUT 文件里的 esolver_type 、basis_type 更换能量求解器并设置相应求解器的参数即可。

### 4.6.2 ABACUS 计算表面缺陷能具体流程

表面缺陷能 $E_{vac}$ 可定义为

$$E_{vac} = E_{slab/vac} + E_{bulk} - E_{slab}$$

其中 $E_{slab/vac}$ 为带缺陷表面构型的总能量，$E_{bulk}$ 为体材料中每原子的能量，$E_{slab}$ 为无缺陷表面构型的总能量，注意计算 $E_{slab/vac}$ 所需构型比 $E_{slab}$ 少一个原子，即缺陷。

因此为了计算某种表面的缺陷能，我们需要进行三次自洽迭代（scf）计算，分别得到上述三个能量。

注意下面的例子中，为了提升效率，我们采用了较低的能量截断值（20 Ry）以及较少的 k 点（gamma 点），实际计算中，需要对二者做收敛性测试。

如果计算体系有磁性原子，要注意体材料的时候是否要加上磁性的计算。

#### 计算实例：钼体心立方 110 表面缺陷能

链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_vacancy_adsorption/0_vacancy/Mo

#### 计算体材料能量

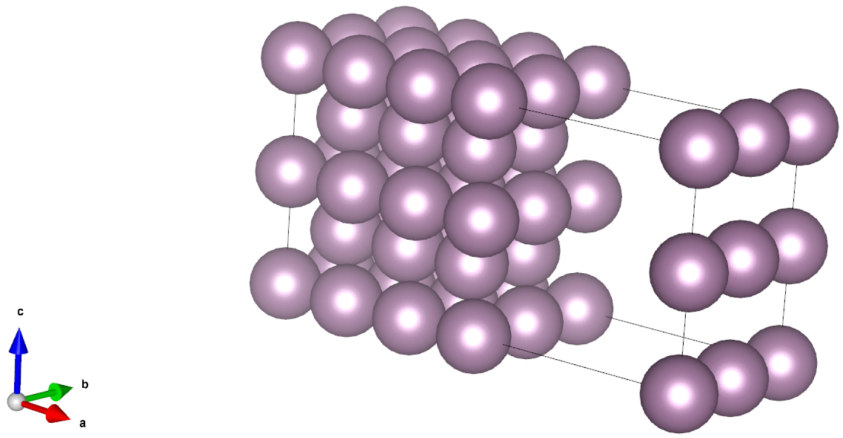
这一步我们用晶格驰豫方法找到体心立方（bcc）Mo 的晶格常数以及平衡能量。

运行算例后，用 `grep FINAL_ETOT_IS OUT.example/running_cell-relax.log` 得到总能量为 -1861.827 eV，由于原胞内只有一个原子，因此 $E_{bulk} = -1861.827\,\text{eV}$。

从 `OUT.example/STRU_ION_D` 文件中可以读到平衡晶格常数为 $a_0 = 5.924\,\text{Bohr}$，它将用于表面构型的构造。

#### 计算表面体系能量

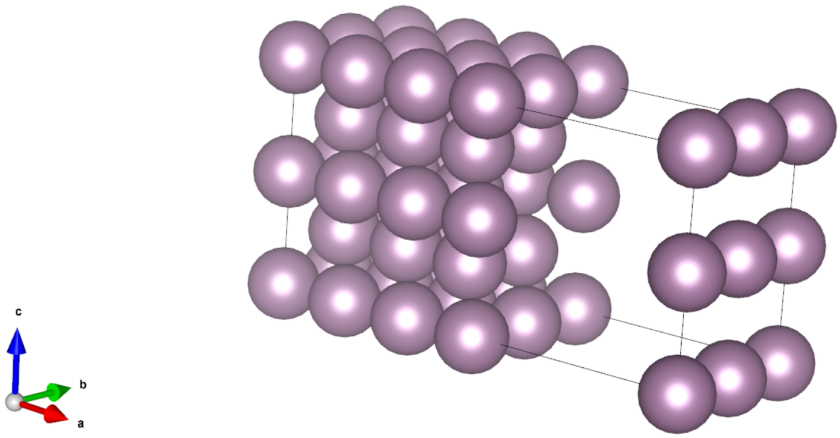
这一步我们参考文献 [1]，搭建了一个 7 层的 Mo bcc(110) 表面体系，每层包含 4 个原子，如下图所示：




计算其总能量时，可以根据实际需要选择是否做结构优化，这号
为了效率更高，我们没有做结构优化。
grep FINAL_ETOT_IS OUT.example/running_scf.log 得到总能量为”“-52129.48@ ev ，因此
Ejay = —52129.480 eV.

**计算带缺陷的表面体系能量**

第三步我们搭建带缺陷的表面体系，方法是从表面上移除一个原子，如下图所示，移除了表面中心的原子：




此算例同样没有做结构优化。

用 `grep FINAL_ETOT_IS OUT.example/running_scf.log` 得到总能量为 `-50265.933 eV`，因此 $E_{slab/vac} = -50265.933\,\text{eV}$.

**计算缺陷能**

对于上述构型，最终的缺陷能 $E_{vac} = E_{slab/vac} + E_{bulk} - E_{slab} = 1.720\,\text{eV}$.

### 4.6.3 ABACUS 计算表面吸附能具体流程

表面吸附能 $E_{ads}$ 可定义为

$$E_{ads} = E_{adsorbate/adsorbent} - E_{adsorbent} - E_{adsorbate} + \Delta ZPE$$

其中 $E_{adsorbate/adsorbent}$ 为吸附物与吸附剂结合后的总能量，$E_{adsorbent}$ 为吸附剂体系的总能量，$E_{adsorbate}$ 为吸附剂体系的总能量，$\Delta ZPE$ 为零点能，注意计算 $E_{adsorbate/adsorbent}$ 所需构型的原子数为 $E_{adsorbent}$ 与 $E_{adsorbate}$ 之和。

因此为了计算吸附能，我们需要进行三次自治迭代（scf）计算，分别得到上述三个能量。

**计算实例：Li 原子在 Mo bcc(110) 表面的吸附能**

链接：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/surface_vacancy_adsorption/1_adsorption/Li-Mobcc110

**计算 Li 原子能量**

这一步我们计算孤立 Li 原子的能量，方法是将一个 Li 原子放在一个足够大的原胞中，以排除 Li 原子间的相互作用。比如本例子中 Li 原子间距为 20 Å。

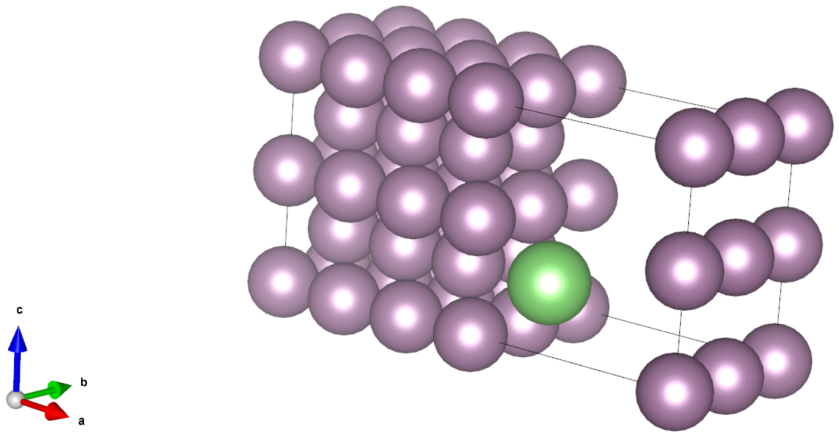
用 grep FINAL_ETOT_IS OUT.example/running_cell-relax.log 得到总能量为 -183.156 eV，由于原胞内有两个原子，因此 $E_{adsorbent} = -183.156\,\text{eV}$.

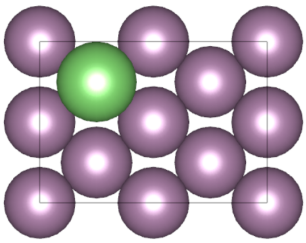
**计算 Mo bcc(110) 体系总能量**

上文已经计算过 Eslab=−52129.480 eV ，因此这里 Eadsorbate=−52129.480 eV 。

**计算吸附体系总能量**

这一步我们参考文献 [1] 搭建了以 long bridge 形式吸附了 Li 原子的 Mo bcc(110) 表面体系，其中 Li 到 Mo 表面的距离为 2.229 Å ，如下图所示，绿色的原子为 Li 原子。





这里我们没有做结构优化，但在实际计算中，Li 原子到 Mo 表面的距离应该通过结构优化得到。

用 grep FINAL_ETOT_IS OUT.example/running_scf.log 得到总能量为 -52315.069 eV，因此 $E_{adsorbate/adsorbent} = -52315.069\,\text{eV}$.

**计算吸附能**

对于上述构型，吸附能 $E_{ads} = E_{adsorbate/adsorbent} - E_{adsorbent} - E_{adsorbate} = -2.433\,\text{eV}$.（此处忽略了零点能）

### 4.6.4 参考文献

[1] Chen M, Roszell J, Scoullos E V, et al. Effect of temperature on the desorption of lithium from molybdenum (110) surfaces: Implications for fusion reactor first wall materials[J]. The Journal of Physical Chemistry B, 2016, 120(26): 6110-6119.

## 4.7 采用 ABACUS 进行表面计算（五）：外加电场

### 4.7.1 介绍

电场是电荷及变化磁场周围空间里存在的一种特殊物质。这种物质与通常的实物不同，它虽然不是由分子原子所构成的，但它却是客观存在的特殊物质，具有通常物质所具有的力和能量等客观属性。许多材料在外加电场作用下会表现出不同寻常的性质，例如：铁电体在外加电场的作用下会产生电极化，也可以通过施加的电场来控制磁性和自发极化，因此在信息储存和传输中有广泛应用；二维材料（如石墨烯）在外加电场下表现出不同的电子性质和可控制的物理性质，这种特性使其具有在电子学领域的广泛应用前景。在电子结构计算中，周期性边界条件下对体系施加电场较为容易，而施加磁场则困难很多。

因此，外加电场是密度泛函理论（Density Functional Theory，简称 DFT）软件中可以被实现且重要的功能。ABACUS 中实现了在表面体系中外加恒定电场的功能 [1]。注意，在 ABACUS 中，外加电场功能存在相应的原子受力的修正，可以用于平面波（Plane-Wave）基组和数值原子轨道（Numerical Atomic Orbitals）基组计算，但注意该功能只能用于带真空层的表面体系。

本教程中将会展示如何在 ABACUS 计算中使用加电场功能，为使得这个教程较为有趣，我们举一个半金属二维材料 nanoribbon（见图 1）加电场产生半金属的例子来讲解如何给体系添加电场，该 DFT 的计算工作曾发在 2006 年的 Nature 杂志上，具体可参考文献 [2]。在这个教程里，我们使用的是数值原子轨道基矢量（Numerical Atomic Orbitals），碳和氢的轨道半径都是 8 a.u.，但实际上也可以使用平面波基矢量，只是前者效率更高所以在这里采用。我们还会使用到电子的自洽场（self-consistent field，简称 SCF）计算，非自洽计算（non-self-consistent field，简称 NSCF），自旋非极化和极化计算等概念。在不考虑自旋时（即设置 nspin=1 时），计算出的能带图中没有带隙，为金属；考虑自旋之后，带隙打开，成为半导体；在自旋极化基础上，外加恒定电场之后，其中一个自旋表现为半导体，另一个自旋则为金属。

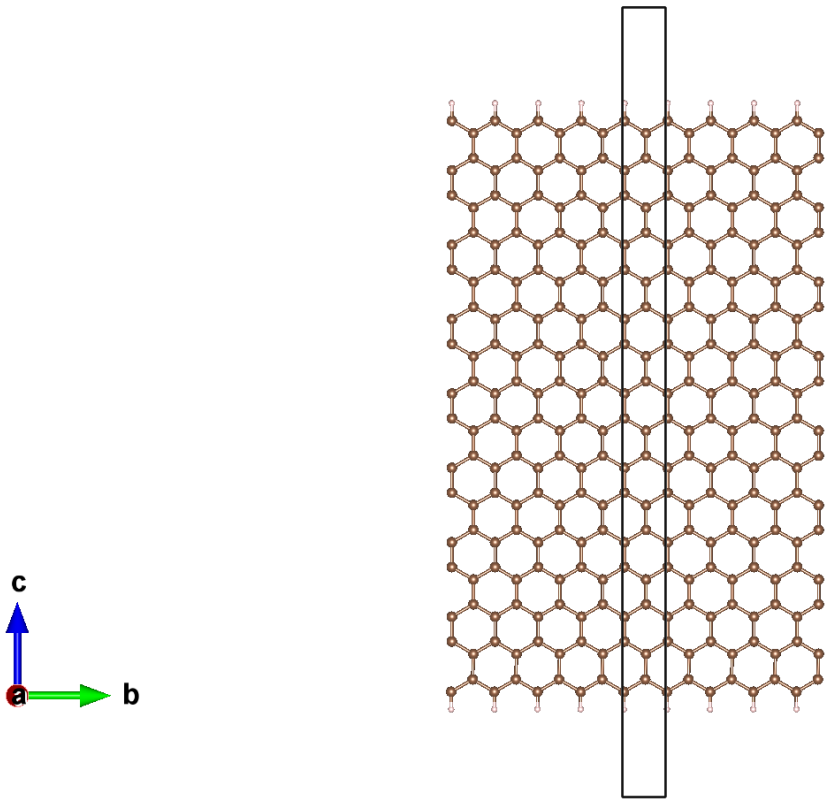



图 1. Nanoribbon 结构图，黑框代表超胞大小，有真空。超胞里包含 32 个碳原子（棕色），超胞里接触真空的 2 个碳原子（每个表面一个碳原子）被 2 个氢原子（白色）饱和。

### 4.7.2 软件和算例准备

ABACUS 里关于外加电场的线上文档地址为：

http://abacus.deepmodeling.com/en/latest/advanced/scf/advanced.html#external-electric-field

ABACUS 里关于外加电场参数的线上文档地址为：https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#electric-field-and-dipole-correction

ABACUS 的外加电场算例下载地址为（gitee）：

https://gitee.com/MCresearch/abacus-user-guide/tree/master/examples/electric_field/

可以采用的下载命令是：

```
git clone https://gitee.com/MCresearch/abacus-user-guide
```

下载后解压，之后进入 /abacus-user-guide/examples/“electric_field 目录

或者采用 Github 仓库地址：

https://github.com/MCresearch/abacus-user-guide

### 4.7.3 采用加电场进行二维材料的能带计算

该算例中含有六个文件夹，解释如下：

1_unspin_scf：做非自旋极化（INPUT 文件中的 nspin 设为 1）的自洽场迭代 SCF 计算（INPUT 文件中的 calculation 设为 scf），输出自洽迭代之后的电子密度（将 INPUT 中的 out_chg 设为 1），注意二维材料的 KPT 文件，因为超胞大小是 $102.4595121466453$，所以对应的布里渊区 k 点取的是 1101，因为实空间周期长度小的方向对应的布里渊区大，所以需要取更多 k 点得到收敛结果。

2_unspin_band：采用非自洽计算 NSCF（INPUT 文件中的 calculation 设为 nscf），读入上一个步骤输出的电荷密度（INPUT 文件中的 init_chg 设为 file），从而可以读入体系的基态电荷密度进行能带计算。

3_spin_scf：同 1_unspin_scf 的计算，只是把 nspin 设为 2。

4_spin_band：同 2_unspin_band 的计算，只是把 nspin 设为 2。

5_spin_elec_scf：同 3_spin_scf 的计算，只是又外加了电场的参数。

6_spin_elec_band：同 4_spin_band 的计算，只是又外加了电场的参数。

以下是算例 5 的输入文件（包含电场参数）：

```
INPUT_PARAMETERS
#Parameters (General)
suffix                  nanoribbon
calculation             scf
nbands                  70
nspin                   2
pseudo_dir              ../../PP_ORB
orbital_dir             ../../PP_ORB

#Parameters (Basis)
basis_type              lcao

#Parameters (Accuracy)
ecutwfc                 100
scf_thr                 1e-6
scf_nmax                200

#Parameters (Smearing)
smearing_method         gaussian
smearing_sigma          0.001

#Parameters (Charge mixing)
mixing_type             pulay
mixing_beta             0.1
mixing_ndim             20
mixing_gg0              1.5

#Parameters (Efield)
efield_flag             1
efield_dir              2
efield_amp              0.0019440124
efield_pos_max          0.95
efield_pos_dec          0.10
```

以算例提供的 INPUT 文件为例，使用外加电场功能只需添加如下参数即可：

efield_flag：类型是 Bool，指定外加电场的开 (1) 或者关 (0)，默认值为 0

efield_dir：类型是 Int，外加电场方向，可取 0，1，2 分别表示 x，y，z 方向，默认值为 2

efield_pos_max：类型是 Real，锯齿状电势最大值所在分数坐标，范围 [0,1)，默认值为 0.5

efield_pos_dec：类型是 Real，锯齿状电势从最大值到最小值的长度（同样以分数坐标为度量），默认值为 0.1，范围 [0,1)。以图 2 为例，0.5 是最大值，0.6 是最小值，那么 0.5  0.6 的区域必须放置在没有原子的真空层中，注

意这个例子设置的是 0.95 到 0.10 为电场区间。

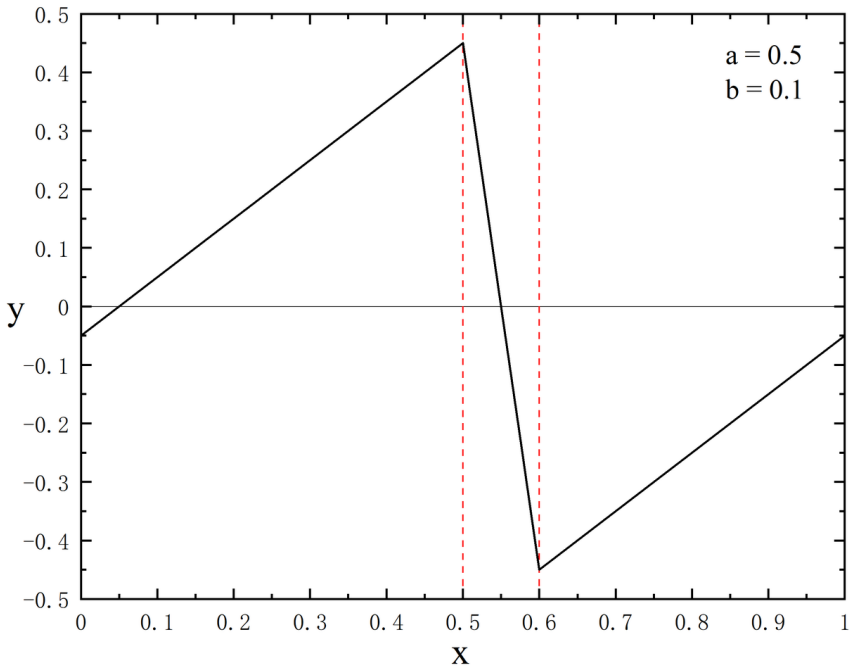



图 2. 锯齿状势场分布图

efield_amp：类型是 Real，电场强度，默认值为 0，单位为 a.u.，其中 1 a.u. = $51.4220632 * 10^10 V/m$

以上参数的使用方法在采用 ABACUS 进行表面计算（二）：偶极修正中有更详细的说明。

### 4.7.4 预期计算结果

运行完算例 2_unspin_band 后会通过非自洽计算得到体系非自旋极化能带图，横坐标是对二维材料布里渊区取 k 点，从 Gamma 点（坐标为 0 0 0）取到 π 点（坐标为 0 0.5 0），取了 100 个 k 点算出来的能带图；纵坐标是能级，将费米面设置为 0，取了费米面附近 2 eV 的能量区间画出能带。以下两张图是同样取法。

运行完算例 4_spin_band 后会得到非自洽计算得到的自旋极化能带图（图 3）。

运行完算例 6_spin_elec_band 后会得到自旋极化加电场的能带图（图 4），注意这里所加的电场为 0.1 V/Å，由输入文件 INPUT 中的 efield_amp 设置，但 efield_amp 的单位是 a.u.，需要换算。

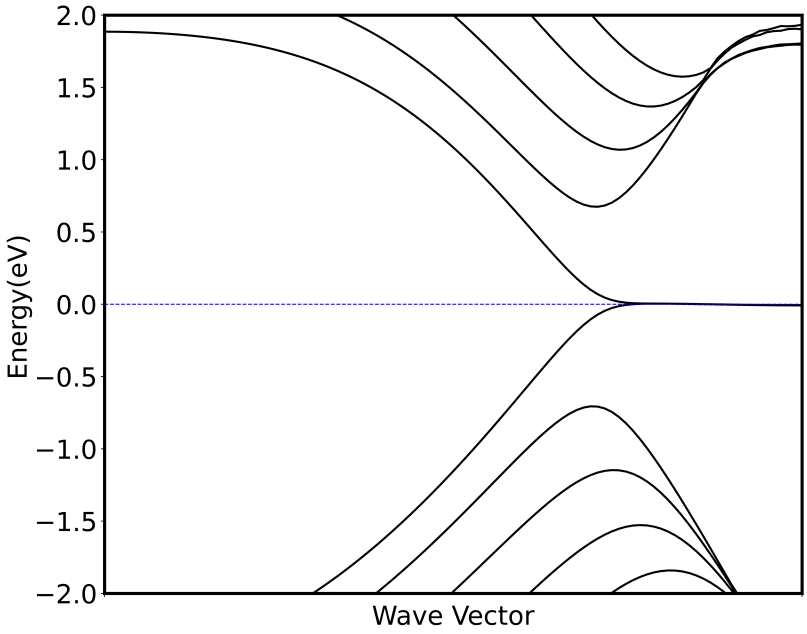



图 3. 采用 PBE 交换关联泛函和非自旋极化得到的二维 nanoribbon 的能带图，可以看出费米面附近 CBM（Conduction Band Minimum）和 VBM（Valence Band Maximum）重合，无带隙。

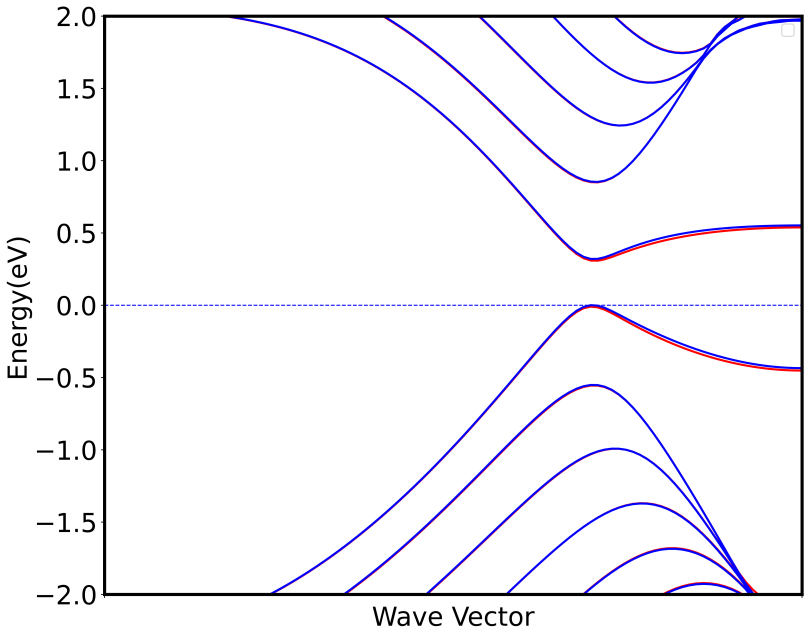



图 4. 采用 PBE 交换关联泛函和自旋极化得到的二维 nanoribbon 的能带图。蓝色和红色代表自旋极化方向不同时对应的两副能带图像，可以看出不加电场时，两个自旋方向的能带图几乎一样，都有带隙。

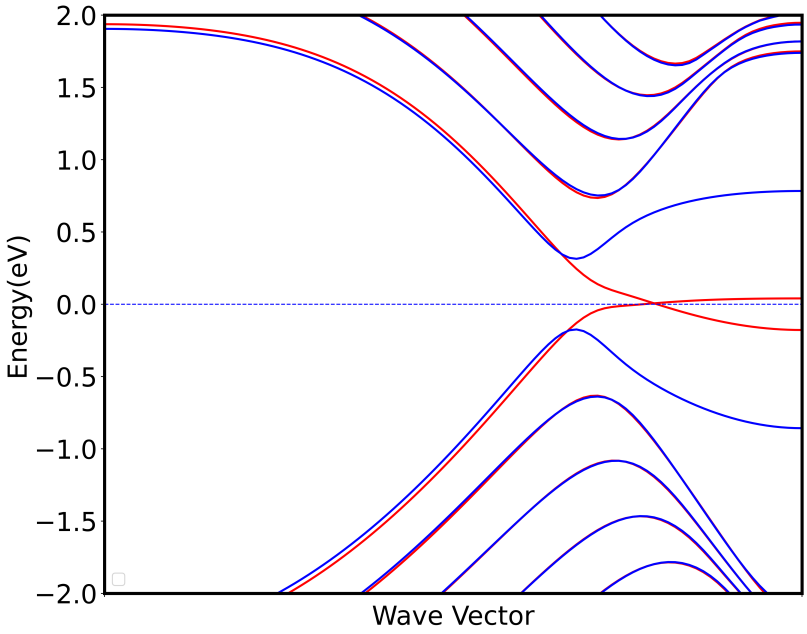



图 5. 采用 PBE 交换关联泛函和自旋极化，再给体系加上 0.1 V/Å 的电场得到的二维 nanoribbon 的能带图。蓝色和红色代表自旋极化方向不同时对应的两副能带图像，可以看出加了能带之后，其中一个自旋方向的能带图出现费米面附近的交叠，呈现金属性质，另外一个自旋方向的能带图依旧保持在费米面处的能隙。

### 4.7.5 参考文献

[1] Phys. Rev. B 59, 12301 (1999), https://doi.org/10.1103/PhysRevB.59.12301

[2] Nature 444, 347–349 (2006), https://doi.org/10.1038/nature05180

## 4.8 采用 ABACUS 进行表面计算（六）：补偿电荷

### 4.8.1 介绍

现实中的化学反应多数时候发生在恒定电势条件下，而传统的 DFT 计算软件则是基于恒定电荷的条件。若要在计算模拟中保持恒定电势，在电化学表面反应建模时就需要调整晶胞中的电子数。在开源密度泛函理论软件 ABACUS 里，模拟体系时使用的是周期性边界条件，因此需要添加额外的算法保持模拟过程中同时保证超胞的电中性。其中的一种方案就是当从系统中添加/提取额外的电子时，需要在表面模型的真空区域中实现补偿电荷的分布。

目前 ABACUS 软件中实现了补偿电荷功能 [1]，与增减电子数相对应的补偿电荷会以带电平板的形式（不考虑平板厚度）予以补偿进而保持电中性，并且存在相应的原子受力的修正，可以用于平面波（Plane-Wave）基组和数值原子轨道（Numerical Atomic Orbitals）基组计算，但注意目前该功能只能用于带真空层的表面体系。我们以下的例子以一个水分子为例，如图 1 所示。

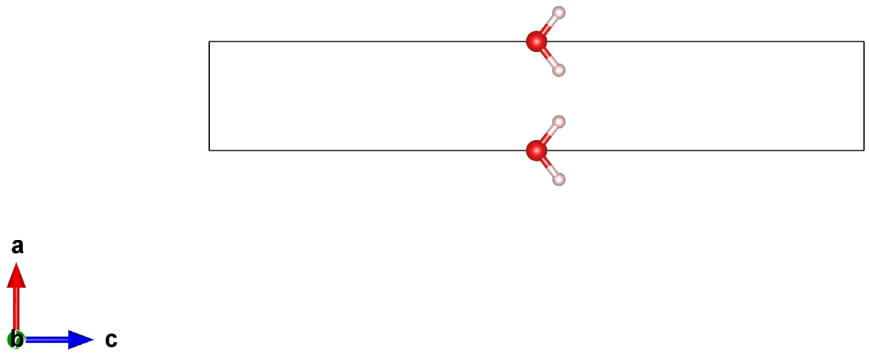



图 1. 一个水分子位于超胞中，红色代表氧原子，白色代表氢原子

### 4.8.2 文档与算例

ABACUS 里关于补偿电荷的线上文档地址为：

http://abacus.deepmodeling.com/en/latest/advanced/scf/advanced.html#compensating-charge

ABACUS 里关于补偿电荷参数的线上文档地址为：https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#gate-field-compensating-charge

ABACUS 的补偿电荷算例下载地址为（gitee）：

https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/compensating_charge/H2O

可以采用的下载命令是：

```
git clone https://gitee.com/mcresearch/abacus-user-guide
```

之后进入 /abacus-develop/“examples/compensating_charge/H2O 目录

或者采用 Github 仓库地址：

https://github.com/MCresearch/abacus-user-guide/tree/master/examples/compensating_charge/H2O

### 4.8.3 偶极修正与补偿电荷结合的表面计算

在采用 ABACUS 进行表面计算（二）：偶极修正文档中，我们给出了水分子层计算中应用偶极修正之后的结果。在此基础上，我们可以进一步应用补偿电荷功能。如图 1 所示，我们采用了完全相同的一个水分子在周期性晶胞的结构，只有 INPUT 文件中添加了补偿电荷相关参数。

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix              H2O
calculation         scf
nbands              8
symmetry            0
pseudo_dir          ../../PP_ORB

#Parameters (2.Iteration)
ecutwfc             100
```

```
scf_thr             1e-6
scf_nmax            100

#Parameters (3.Basis)
basis_type          pw

#Parameters (4.Smearing)
smearing_method     gauss
smearing_sigma      0.001

#Parameters (5.Mixing)
mixing_type         broyden
mixing_beta         0.1

#Parameters (6.Dipole correction)
efield_flag         1
dip_cor_flag        1
efield_pos_max      0.0
efield_pos_dec      0.1
efield_amp          0.0

#Parameters (7.Compensating charge)
gate_flag           1
zgate               0.7
nelec               9

#Parameters (8.Output)
out_pot             2
```

以算例提供的 INPUT 文件为例，使用补偿电荷功能只需添加如下参数即可：

- `gate_flag`：类型是 Bool，指定补偿电荷功能的开 (1) 或者关 (0)，默认值为 0
- `efield_dir`：类型是 Int，指定与带电平板垂直的方向，可取 0，1，2 分别表示 x，y，z 方向，默认值为 2
- `zgate`：类型是 Real，指定带电平板（位于真空中）在晶胞中的位置（分数坐标），范围 [0,1]，默认值为 0.5
- `nelec`：类型是 Real，设定的体系总电子数，这里是 9 个电子（氧原子因为质势有 6 个电子，每个氢原子有 1 个电子，总共 8 个电子），即原体系多加了 1 个电子，那么为了保持电中性，带电平板会带 1 个正电荷
- `block`：类型是 Bool，添加势垒以防止电子溢出的功能（可选），1 代表开，0 代表关，默认值为 0
- `block_down`：类型是 Real，势垒的起点（分数坐标），默认是 0.45
- `block_up`：类型是 Real，势垒的终点（分数坐标），默认是 0.55
- `block_height`：类型是 Real，势垒高度，单位是里德堡（Rydberg），默认是 0.1

### 4.8.4 预期结果

如左下图所示，一个水分子位于超胞（尺寸为 $3318^3$ ）的中部（即位于 9Å ）。按照采用 ABACUS 进行表面计算（二）：偶极修正文档中偶极修正参数设置的说明，锯齿即位于 0.0 0.1 的区域。注意在采用偶极修正/外加电场功能的情况下，带电平板的位置应该在真空区域，并且不同于偶极修正锯齿所在的区域。

为了验证结果的正确性，我们可以画出静电势沿 Z 轴（c 方向）的变化如图 2 所示。与只应用偶极修正功能的静电势相比（可参考表面计算二里面的图 3），在分数坐标 0.7（Z 轴 12.6 埃）处，静电势出现了一个向下的凹

陷，这来自于此处的补偿电荷带电平板的影响。

静电势计算和作图方法请参考静电势和功函数计算文档。

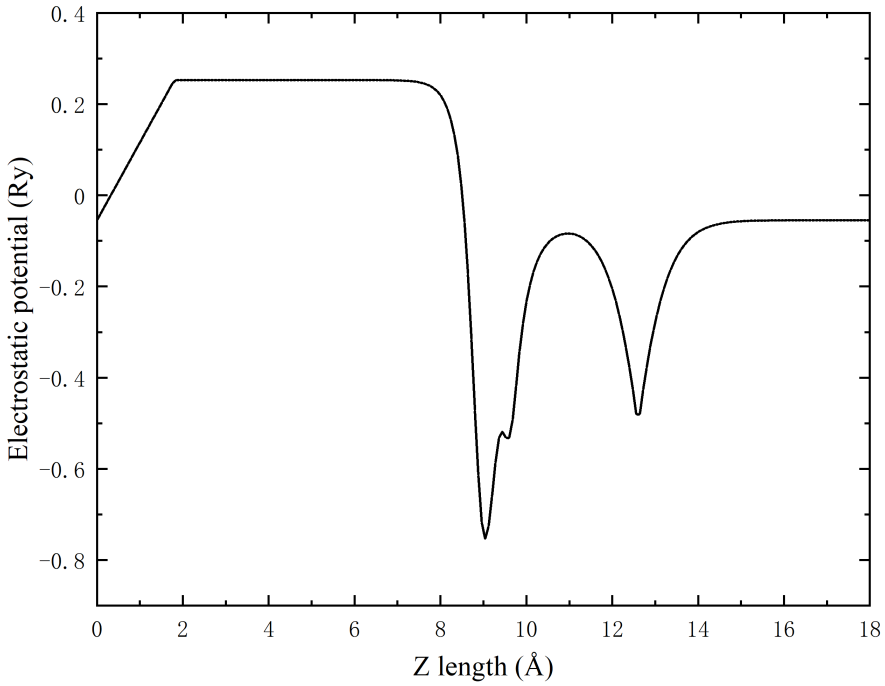



图 2. 静电势（Electrostatic Potential）沿超胞 Z 轴变化图

### 4.8.5 参考文献

[1] Phys. Rev. B 89, 245406 (2014), https://doi.org/10.1103/PhysRevB.89.245406

# 第 5 章 特色功能

## 5.1 ABACUS 随机波函数 DFT 方法使用教程

### 5.1.1 介绍

本教程旨在介绍 ABACUS 中随机波函数密度泛函理论（Stochastic Density Functional Theory，以下简称 SDFT）计算功能。目前 ABACUS 使用 SDFT 主要聚焦在高温高压物质的模拟，特别是温稠密物质（Warm Dense Matter，简称 WDM）。在进行温稠密物质计算时（温度高达数十到上千 eV，1 eV=11604.5 K），传统的 Kohn-Sham 密度泛函理论（KSDFT）需要用到极大数量的占据态电子波函数导致计算困难，而 SDFT 使用随机波函数轨道，可以有效地避开对角化哈密顿矩阵这个问题，应用于高温计算。关于 ABACUS 中实现 SDFT 算法的细节可以参考 Qianrui Liu and Mohan Chen*, "Plane-wave-based stochastic-deterministic density functional theory for extended systems," Phys. Rev. B, 106, 125132 (2022)。本教程中将会展示如何在 ABACUS 计算中使用 SDFT 功能，此外还会介绍混合随机波函数密度泛函理论方法使用（mixed stochastic-deterministic DFT，简称 MDFT），即在 SDFT 计算中，混入一部分的低能 Kohn-Sham 轨道，从而加速结果收敛。

### 5.1.2 软件和算例准备

ABACUS 的软件包（3.2.0 版本）中提供了一个 SDFT 的算例，可以从 Gitee 上下载。可以在网页右侧点击克隆/下载-> 下载 ZIP 得到算例，或者在 linux 终端执行如下命令得到算例：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

下载后解压，之后进入/abacus-user-guide/examples/stochastic 文件夹。算例中有三个文件夹，pw_Si2、pw_md_Al 和 186_PW_SDOS_10D10S。

### 5.1.3 采用 SDFT 进行电子自洽迭代计算

pw_Si2 文件夹：这是一个电子温度为 0.6 Ry（约 8.16 eV）的 2 个原子的金刚石结构硅（Si）的电子自洽迭代（Self Consistent Field，简称 SCF）算例，包含布里渊区 K 点的 KPT 文件和包含原子位置的 STRU 文件与传统的 KSDFT 计算并无区别，主要的不同在于输入文件 INPUT，注意目前 SDFT 仅支持 smearing_method 为 fd。INPUT 文件如下：

```
INPUT_PARAMETERS
#Parameters     (General)
calculation     scf
esolver_type    sdft
pseudo_dir      ../../PP_ORB
nbands          4
nbands_sto      64
nche_sto        100
method_sto      1
#Parameters (Accuracy)
ecutwfc         50
scf_nmax        20
symmetry        1
```

```
#Parameters (Smearing)
smearing_method fd
smearing_sigma  0.6
```

以上参数在 ABACUS 的线上文档中均有详细说明，这里再进行简单概述：

- **calculation** 设置为 `scf`，代表进行电子自治迭代计算（self-consistent field）。
- **esolver_type** 是选择系统总能量求解方法的，默认为 `ksdft`（Kohn-Sham density functional theory），这里需要设置为 `sdft` 才会使用 **SDFT** 或者 **MDFT** 进行计算。
- **nbands** 是使用的 **Kohn-Sham** 轨道的数目（也叫 **deterministic orbitals**，是通过严格对角化矩阵计算出来的）。如果 **nbands** 设置为 `0`，且 **nbands_sto**（随机轨道数目，这里设成 `64`）不为 `0`，则会进行 **SDFT** 计算；如果 **nbands>0**，且 **nbands_sto>0**，则会进行混合 **KS** 电子轨道和随机轨道（stochastic orbitals）的 **MDFT** 计算。注意：一般 **nbands** 设置为能量低于费米能级对应的能带数，计算效率会比较高。
- **nbands_sto** 是使用随机波函数（stochastic orbitals）轨道数目，原则上取得越大则随机误差越小，但计算效率也会相应降低。
  - 判断随机波函数个数是否足够的一个经验法则是：测试能量误差。实际计算中，一般可以采用 `10` 个左右不同的随机数种子（可以参考 **seed_sto** 参数设置随机数种子，下面有介绍）生成的相同数目的随机波函数轨道进行 **SDFT** 计算，得到系统能量并求得相应误差，增加随机波函数轨道数目直到控制能量误差小于万分之一即可。
  - 当 **nbands_sto** 设为 `0` 时，程序会自动转成 **KSDFT** 进行计算（ABACUS 3.2.2 版本以后）。
- **nche_sto** 是将电子体系的哈密顿量进行切比雪夫展开的阶数，这个数取得越大则用到的切比雪夫展开阶数越多，相应的计算精度也会越高但效率会降低。大致关系为与温度成反比，温度越高，阶数可以取得越小；**ecut**（正比关系）越大，阶数越大；推荐使用的 **nche_sto** 的大小是使得输出文件 `running_scf.log` 中的 **Chebyshev Precision** 小于 `1e-8`。
- **method_sto** 是进行 **SDFT** 计算使用的方法：`1` 代表消耗内存较少但稍慢的方法，`2` 代表更快但需要更大内存的方法，默认是 `2`。

此外还有如下参数可能会用到：

- **seed_sto**: 生成随机轨道的随机种子。默认是 `0`，代表随时间随机生成；若要控制使用相同的随机种子，可以设置一个大于 `1` 的整数。
- **bndpar**: 将所有并行的进程分成 `bndpar` 个组，计算所用随机轨道将平均分布在每个组中，可以提高并行效率，默认为 `1`。值得注意的是，这个参数并不是越大越好，并且不如 K 点并行（`kpar` 参数）有效，实际计算中应该优先使用 K 点并行，然后测试不同大小的 `bndpar`，确定最佳的 `bndpar`。

平面波能量截断值 ecut 的测试：由于随机误差的出现，SDFT 的 ecut 的测试与传统的 KSDFT 稍有区别，但是原理是类似的。在确定好 nbands_sto 后，就可以测试 ecut 了。与测试 nbands_sto 类似，在不同的 ecut，需要采用 10 个左右不同的随机种子生成的相同数目的随机波函数轨道进行 SDFT 计算，然后取平均能量。由于温稠密物质能量一般都比较高，因此只要控制相邻二个 ecut（相差为 10 Ry）对应的平均能量差小于一定标准即可（例如万分之一）。

注 1：在这个例子里我们提供的赝势是 Si.pz-vbc.UPF 文件，这个文件包含 4 个硅的价电子。事实上，当温度特别高的时候，一般的赝势可能会面临可移植性差的问题，例如高温会使得内壳层电离。这个时候，要选择合理的赝势进行计算，甚至可能需要自己造一个新的赝势，目前 ABACUS 3.2.0 支持的是模守恒的赝势。

注 2：ABACUS 的 SDFT 和 MDFT 支持多个 k 点采样，因此可以在 KPT 文件里设置不同的 k 点个数，在某些性质的计算里，要注意计算性质随着 k 点的收敛。

### 5.1.4 采用 SDFT 进行分子动力学模拟

pw_md_Al 文件夹：这是一个电子温度为 7.35 Ry（约 100 eV）、包含 16 个铝（Al）原子的结构，我们对其进行分子动力学（Molecular Dynamics，简称 MD）的模拟。INPUT 文件如下：

```
INPUT_PARAMETERS
#Parameters     (General)
calculation     md
esolver_type    sdft
pseudo_dir      ../../PP_ORB
nbands          0
nbands_sto      64
nche_sto        20
method_sto      2
#Parameters (Accuracy)
ecutwfc         50
scf_nmax        20
scf_thr         1e-6
symmetry        1
#Parameters (Smearing)
smearing_method fd
smearing_sigma  7.34986072
#Parameters (MD)
md_tfirst       1160400
md_dt           0.2
md_nstep        10
```

注意要进行分子动力学模拟，calculation 参数需设置为 md。esolver_type 需设置为 sdft，才能进行 SDFT 的计算。这里 nbands 设置为 0，nbands_sto 设置为 64，代表仅仅使用随机轨道而没有 KS 轨道的 SDFT 计算。

### 5.1.5 采用 SDFT 计算态密度

186_PW_SDOS_10D10S 文件夹：采用 SDFT 还可以计算给定体系的态密度（Density of States，简称 DOS）。例如，186_PW_SDOS_10D10S 是一个 1 个 Si 原子的算例，电子温度约为 0.6 Ry（约 8.16 eV）。如下所示：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix          autotest
calculation     scf
esolver_type    sdft
method_sto      2

nbands          10
nbands_sto      10
nche_sto        120
emax_sto        0
emin_sto        0
seed_sto        20000
pseudo_dir      ../../PP_ORB
```

```
symmetry        1
kpar            1
bndpar          2

#Parameters (2.Iteration)
ecutwfc         20
scf_thr         1e-6
scf_nmax        20

#Parameters (3.Basis)
basis_type      pw

#Parameters (4.Smearing)
smearing_method fd
smearing_sigma  0.6

#Parameters (5.Mixing)
mixing_type     broyden
mixing_beta     0.4

out_dos          1
dos_emin_ev      -20
dos_emax_ev      100
dos_edelta_ev    0.1
dos_sigma        4
dos_nche         240
npart_sto        2
```

以上参数在 ABACUS 的线上文档中均有说明，这里再进行简单概述：

- **out_dos**: 需要设置为 1，才能输出能态密度。
- **dos_emin_ev**: 能态密度的能量最小范围，单位 eV。
- **dos_emax_ev**: 能态密度的能量最大范围，单位 eV。
- **dos_edelta_ev**: 控制输出 DOS 时的能量间隔，单位 eV。
- **dos_sigma**: 能态密度的高斯展宽的因子，单位 eV。
- **dos_nche**: 计算能态密度时切比雪夫展开阶数，默认为 100。
- **npart_sto**: 当使用 method_sto=2 运行例如 DOS 的 SDFT 后处理时，将控制使用内存大小为正常的 1/npart_sto，防止内存不够导致无法计算，默认为 1。

注：态密度的输出文件是 OUT 文件夹下的 DOS1_smearing.dat。

### 5.1.6 结尾

总体来讲，随机波函数密度泛函理论方法（SDFT 或者 MDFT）的使用与 KSDFT 并无太大的区别，直接运行 ABACUS 程序即可，但是对一些关键参数的选取会影响精度和效率（例如 nbands, nbands_sto, nche_sto, method_sto, kpar, bnd_par）。对于极端高温计算（>10 eV），使用 SDFT 可以大大提高计算速度，是比普通的 KSDFT 更好的选择。如果大家使用有问题，欢迎写信联系（见上）。

# 5.2 ABACUS 随机波函数 DFT 计算电子电导热导教程

## 5.2.1 介绍

对于物质的电导热导，一般可以分成电子和离子贡献，而对于电子贡献的部分可以使用 Kubo-Greenwood 的公式进行计算 [Phys. Rev. B 83, 235120 (2011)]，该方法已被广泛应用于液态金属、温稠密 (Warm Dense matter, WDM) 等物质的计算。Kubo-Greenwood 通过直接计算含频的昂萨格系数 $L_{mn}(\omega)$，来计算电子的电导与热导：

$$L_{mn}(\omega) = (-1)^{m+n}\frac{2\pi e^2}{3\omega\Omega}$$

$$\times\sum_{ijak} W(k)\left(\frac{\epsilon_{ik}+\epsilon_{jk}}{2}-\mu\right)^{m+n-2}|\langle\Psi_{ik}|\hat{v}_\alpha|\Psi_{jk}\rangle|^2[f(\epsilon_{ik})-f(\epsilon_{jk})]\delta(\epsilon_{jk}-\epsilon_{ik}-\hbar\omega),$$

这里的 n,m 指标取值范围为 1, 2，$\Omega$ 为体积，$W(k)$ 为 k 点的权重，$\epsilon_{ik}$ 为第 i 个波函数 $\Psi_{ik}$ 对应的本征能量，$f(\epsilon)=\frac{1}{1+\exp\left(\frac{\epsilon-\mu}{kT}\right)}$ 为费米狄拉克分布，$\mu$ 为化学势，$\delta(x)$ 为 delta 函数，在实际计算中 delta 函数可以用高斯函数

$$G(x)=\frac{1}{\sqrt{2\pi}s}\exp\left(-\frac{x^2}{2s^2}\right)$$

或洛伦兹函数 $L(x)=\frac{1}{\pi}\frac{\gamma}{x^2+\gamma^2}$ 代替，$\hat{v}_\alpha$ 为速度算符的第一个分量 ($\alpha=x,y,z$) 对于速度算符，速度算符的定义为 $\hat{v}=\frac{i}{\hbar}[\hat{H},\hat{r}]$。

含频电导 $\sigma(\omega)=L_{11}(\omega)$，直流电导 $\sigma_0=\lim_{\omega\to0}\sigma(\omega)$。

含频热导 $\kappa(\omega)=\frac{1}{e^2T}\left(L_{22}-\frac{L_{12}^2}{L_{11}}\right)$，热导 $\kappa_0=\lim_{\omega\to0}(\omega)$。

以上便是传统 KSDFT 计算的方法，通过 KSDFT 求解的波函数、本征能量，可以轻易的带入公式进行计算。

对于随机波函数密度泛函理论 (sDFT)，其没有波函数、本征能量不能通过该方法进行计算。而应从原始的 Kubo 公式中出发：

电流、热流响应函数：

$$C_{mn}(t)=-2\theta(t)J\left\{Tr\left[\sqrt{\hat{f}j_m(1-\hat{f})}e^{i\frac{\hat{H}}{\hbar}t}j_n e^{-i\frac{\hat{H}}{\hbar}t}\sqrt{\hat{f}}\right]\right\},$$

这里 $\theta(x)$ 为阶跃函数，$J$ 为取虚部，$\hat{f}=\frac{1}{1+\exp\left(\frac{\hat{H}-\mu}{kT}\right)}$ 为费米狄拉克算符，$j_1=e\hat{v}$ 为电流算符，$j_2=(\hat{v}\hat{H}+\hat{H}\hat{v})/2-\mu\hat{v}$ 为热流算符，$e^{-i\hat{H}t/\hbar}$ 为演化算符。这里求迹便可通过随机波函数进行计算。

而昂萨格系数就可通过响应函数的傅里叶变换进行计算：

$L_{mn}=\frac{2e^{m+n-2}}{3\Omega\hbar\omega}J[\tilde{C}_{mn}(\omega)]$, 这里计算$\tilde{C}_{mn}(\omega)$可以使用不同的窗函数：

高斯型：

$$\tilde{C}_{mn}(\omega)=\int_0^\infty C_{mn}(t)e^{-i\omega t}e^{-\frac{1}{2}s^2t^2}dt$$

洛伦兹型：

$$\tilde{C}_{mn}=\int_0^\infty C_{mn}(t)e^{-i\omega t}e^{-rt}dt$$

其分别与前面提到的高斯函数 $G(x)$ 和洛伦兹函数 $L(x)$ 对应。

## 5.2.2 算例准备

下载链接：

https://github.com/MCresearch/abacus-user-guide/tree/master/examples/stochastic/cond_Si

### 5.2.3 采用 sDFT 进行电子的电导热导计算

**输入文件：**

cond_Si 文件夹：这是一个电子温度为 0.6 Ry（约 8.16 eV）的单原子硅（Si）的电导热导计算算例，包含布里渊区 K 点的 KPT 文件和包含原子位置的 STRU 文件与传统的 KSDFT 计算并无区别，主要的不同在于输入文件 INPUT。为了计算电导热导，我们只需要把 cal_cond 参数打开即可，INPUT 文件如下：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix          Si
calculation     scf
esolver_type    sdft
nbands          10
nbands_sto      40
nche_sto        120
seed_sto        20000
symmetry        1
kpar            1
bndpar          2

#Parameters (2.Iteration)
ecutwfc         40
scf_thr         1e-6
scf_nmax        100

#Parameters (3.Basis)
basis_type      pw

#Parameters (4.Smearing)
smearing_method     fd
smearing_sigma      0.6

#Parameters (5.Mixing)
mixing_type     broyden
mixing_beta     0.4

#Parameters (6.Conductivity)
cal_cond         1
cond_smear       1
cond_fwhm        0.4
cond_wcut        20
cond_dw          0.02
cond_dt          0.1
```

以上参数在 ABACUS 的线上文档中均有详细说明，这里再进行简单概述：

- **cal_cond**: 控制是否计算电子贡献的电导热导
- **cond_smear**: 设置展宽类型，1：高斯展宽（默认），2：洛伦兹展宽
- **cond_fwhm**: 设置半高宽 FWHM，对于高斯展宽，FWHM $= 2\sqrt{2\ln 2}s$；对于洛伦兹展宽 FWHM $= 2\gamma$

- **cond_wcut**: 计算频率的截断值，决定输出含频数据的频率范围，单位：eV
- **cond_dw**: 频率的间隔，单位：eV
- **cond_dt**: 对响应函数积分时的积分间隔，原子单位，响应函数振荡过快需要较小积分间隔，一般取值在 0.01-0.2 a.u.，在屏幕输出中有推荐的取值：

  **Recommended dt**: 1.553e-01 a.u.
- **cond_dtbatch**: 其与 cond_dt 共同决定演化算符

$$\exp(iH * \text{cond_dtbatch} * dt)$$

的切比雪夫展开阶数，一般来说该值设的越大，内存消耗越多，但速度越快，当其达到一定值，速度就不会增加。当其设为 0 时，程序会自动决定 cond_dtbatch 以使切比雪夫展开阶数在 100 附近。

计算时间的说明：

- 使用 sDFT 计算电导热导还是比较慢的，计算时间正比于切比雪夫展开阶数和

$$nt = \frac{t_{\text{total}}}{(\text{cond_dt_batch} * dt)}$$

- 切比雪夫阶数由平面波截断能决定，阶数约正比于截断能
- $nt$ 由 $t_{\text{total}}$ 决定，而 $t_{\text{total}}$ 由 cond_fwhm 决定，其长度反比于 cond_fwhm，因此该方法不能使用任意小的展宽。
- 增加 cond_dt_batch $*$ $dt$ 会使切比雪夫展开阶数增加，会使 $nt$ 减小，总体效应是其值越大，计算效率越高，但达到一定程度，效率就不再增高了。

**输出文件：**

注：结果仅供参考，以最新版本算出的结果为准

屏幕输出：

```
set cond_dtbatch to 69                    自动设置 cond_dtbatch 参数
set N order of Chebyshev for KG as 113    演化算符的切比雪夫展开阶数
Calculating conductivity....
nw: 1000 ; dw: 2.000e-02 eV               总频率数，频率间隔
nt: 4862 ; dt: 1.000e-01 a.u.(ry^-1)      总时间数，时间间隔
Emin_KS(2): 1.373e+01 eV; Emax: 5.442e+02 eV; Recommended max dt: 1.553e-01 a.u. 推荐 cond_dt
ik=0: (Time left 1.673e+02 s) 估计剩余计算时间
nt: 计算进度
69      138     207     276     345     414     483     552     621     690
759     828     897     966     1035    1104    1173    1242    1311    1380
1449    1518    1587    1656    1725    1794    1863    1932    2001    2070
2139    2208    2277    2346    2415    2484    2553    2622    2691    2760
2829    2898    2967    3036    3105    3174    3243    3312    3381    3450
3519    3588    3657    3726    3795    3864    3933    4002    4071    4140
4209    4278    4347    4416    4485    4554    4623    4692    4761    4830

DC electrical conductivity: 2.454778e+05 Sm^-1    直流电导值
 (直接线性外推的简单估计值，实际需要根据含频电导拟合外推)
Thermal conductivity: 5.968738e+02 W(mK)^-1       热导值
Lorenz number: 3.456403e+00 k_B^2/e^2             洛伦兹常数
```

je-je.txt:

```
#t(a.u.)        c11(t)        c12(t)        c22(t)        decay
       0            -0            -0            -0            1
     0.1      -1.39981       6.61846      -38.6529     0.999999
     0.2      -1.91869       6.94748      -25.4566     0.999997
     0.3      -1.87453       5.46871      -15.4962     0.999993
     0.4        -1.613       3.75367      -6.29163     0.999988
     0.5      -1.36998        3.0228      -6.02995     0.999981
     0.6       -1.1229       2.41076       -5.2121     0.999972
     0.7      -0.84644       1.75119      -5.82033     0.999962
     0.8     -0.529567      0.779473      -2.13103      0.99995
```

储存响应函数的文件，各列分别为均为原子单位，时间 t, 响应函数 $C_{11}(t), C_{12}(t), C_{22}(t)$ , 窗函数 Onsager.txt:

```
## w(eV)        sigma(Sm^-1)     kappa(W(mK)^-1)       L12/e(Am^-1)      L22/e^2(Wm^-1)
    0.01              250820             604.318      -4.66249e+06        1.4392e+08
    0.03              261503             619.205      -4.83345e+06       1.47997e+08
    0.05              283291             649.006      -5.18082e+06       1.56228e+08
    0.07              317019              693.79      -5.71545e+06       1.68767e+08
    0.09              363915             753.684      -6.45332e+06       1.85835e+08
    0.11              425574              828.87      -7.41519e+06       2.07723e+08
    0.13              503909             919.555      -8.62601e+06       2.34774e+08
    0.15              601083             1025.94      -1.01141e+07       2.67375e+08
    0.17              719414             1148.17      -1.19101e+07       3.05944e+08
```

储存昂萨格系数的文件，各列分别为频率，电导，热导，$L_{12}(), L_{22}()$

## 5.3 ABACUS 实时演化含时密度泛函理论使用教程

### 5.3.1 背景

实时演化含时密度泛函理论（Real-time Time-Dependent Density Functional Theory，简称 rt-TDDFT）是一种基于第一性原理的模拟激发态电子体系动态演化的方法。与传统的密度泛函理论（DFT）不同，rt-TDDFT 是建立在 Runge-Gross 定理之上，主要研究时间依赖的 Kohn-Sham 方程（TDKS 方程）。该理论通过构建时间传播子迭代求解 TDKS 方程，直接演化电子的波函数。此外，它还采用半经典的 Ehrenfest 动力学来模拟离子实的运动，可以实时地得到激发电子态以及离子位置的信息，因此具有较为广泛的应用前景。

周期性体系具有能带结构，而非周期性的原子或分子体系则具有离散能级。体系的基态是指价电子从低到高依次填充至能带或能级中。当体系吸收特定能量后，低能态的电子跃迁至高能态，此时体系便处于激发态。研究激发态性质是 TDDFT 的一大优势，因此掌握如何模拟体系的激发态变得尤为关键。ABACUS 为此提供了两种方式：一种是直接改变占据数，使得叠加态的波函数拥有更多高能态分量；另一种是向一个处于基态的体系施加一个外加电场，为其提供激发所需的能量。下面将分别介绍这两种方式的具体使用。文档中提到的算例可以在这个仓库中找到。有关 TDDFT 输入参数的细节可参考输入参数线上文档

### 5.3.2 在 ABACUS 中使用 rt-TDDFT 时改变电子的占据数

算例：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/tddft/occupation

算例输入文件 INPUT 设置如下：

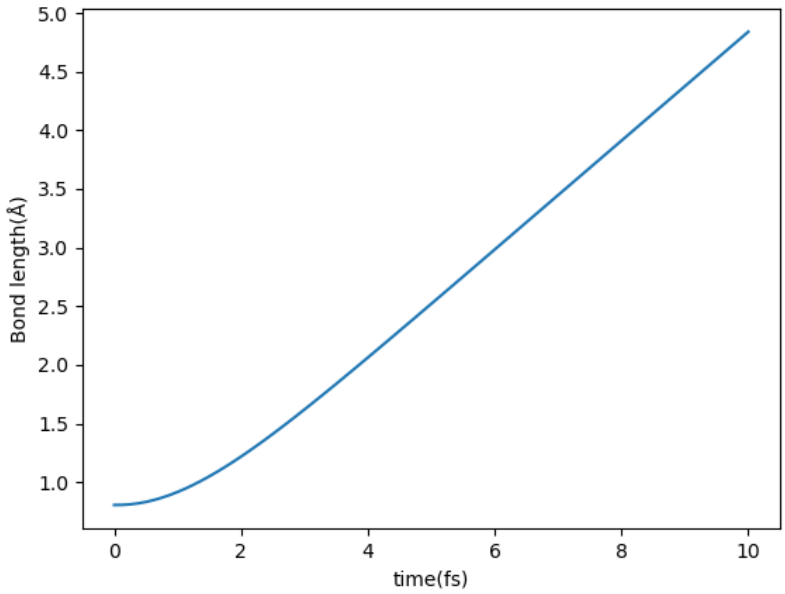
```
INPUT_PARAMETERS
#Parameters (1.General)
suffix              H2_ocp
calculation         md
esolver_type        tddft
nbands              5
nspin               1
pseudo_dir          ../../../tests/PP_ORB
orbital_dir         ../../../tests/PP_ORB

#Parameters (2.Iteration)
ecutwfc             60
scf_thr             1e-6
scf_nmax            100

#Parameters (3.Basis)
basis_type          lcao
gamma_only          0

#Parameters (4.Smearing)
smearing_method     gauss

#Parameters (5.MD Parameters)
md_type             nve
md_nstep            1000
md_dt               0.05
md_tfirst           0

#Parameters (6.TDDFT Occupation Parameters)
ocp                 1
ocp_set             1 1 0 0 0
```

具体参数设置请参考线上文档，这里仅对部分重要参数进行说明：

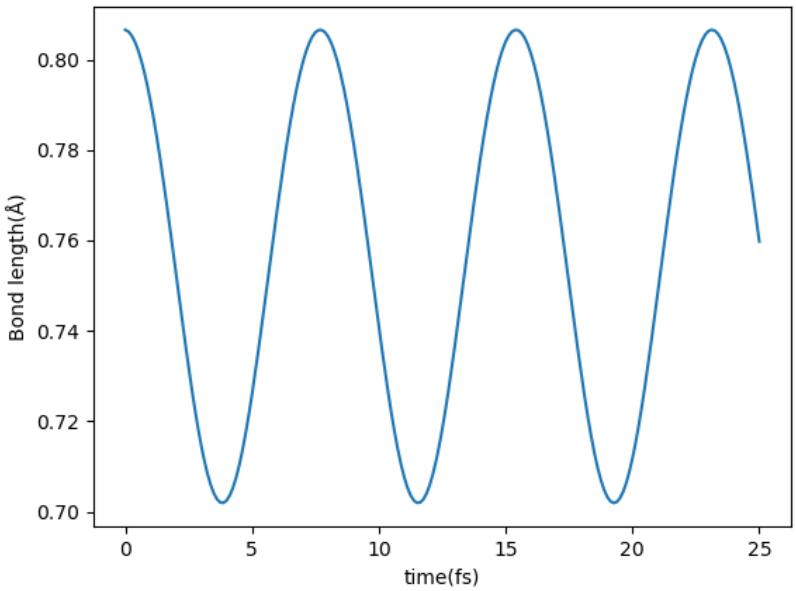
- **gamma_only**：必须设置为 0，gamma_only 为 1 会采用 double 类型计算，rt-TDDFT 使用传播子迭代计算波函数，需要 complex 类型存储数据，不兼容 gamma_only 为 1 的情况。
- **basis_type**：必须设置为 lcao，目前 ABACUS 仅支持基于数值原子轨道基组的 rt-TDDFT 功能。
- **nbands**：能带数，由于 ocp_set 需要手动设置每一条能带的占据数到最高带，建议手动设置 nbands，防止二者不对应。
- **calculation**：必须为 md，同时一般 MD (Molecular Dynamics) 计算所需的参数都需要设置，这部分参数会对应 rt-TDDFT 的离子实运动的计算。
- **esolver_type**：tddft，表示采用 rt-TDDFT 求解器。
- **ocp**：改变占据数功能的开关，为 1 的时候开启。
- **ocp_set**：占据数的设置，$1\cdot 1\cdot 0\cdot 0$ 表示这个算例所计算的五条能带，各自占据的电子数，其总和为 2，代表 Hz 的两个价电子，可以简化为 $2*1\cdot 3*\cdot 0$。另外，对于有多个 k 点的体系，ABACUS 支持对各个 k 点单独设置占据数。假设体系拥有 5 条能带，2 个 k 点，则 ocp_set 需要指定 10 个数值，前五个代表第一个 k 点五条能带的占据，后五个代表第二个 k 点五条能带的占据。

按照分子轨道理论，两个 H 原子的原子轨道发生相互作用形成一条成键轨道、一条反键轨道，H 分子的基

态占据可以表示为 2 0 0 0“ 0，最低的能态代表成键轨道，则 1 1 0 0“ 0 占据数表示将一个电子从成键轨道上激发到反键轨道，会导致 H 分子发生解离。通过输出目录 OUT.H2_ocp/STRU 下的结构文件，可以看到随着时间的推移，两个 H 原子逐渐解离。运行 occupation 文件夹下的算例，通过 ASE-ABACUS 读取各个时刻 STRU 文件，并计算了两个 H 原子实时的间距 d ，结果如下图所示：




为了说明这个解离是由于改变占据数引起的，可以将 INPUT 替换为 occupation 文件夹中的 INPUT-1，其中 ocp_set 被改为：2 0 0 0“ 0，同样输出实时的原子间距，可以看到，两个原子间距稳定的振荡，并未发生解离：



### 5.3.3 ABACUS 中使用 rt-TDDFT 外加电场计算材料吸收谱

在任意外加电场的影响下，rt-TDDFT 能够实时模拟并获得材料的电子态和离子位置信息。按照 Runge-Gross 定理（相当于 KSDFT 中的 Hohenberg-Kohn 定理），这等价于材料的全部物理性质。在众多的物理性质中，吸收谱作为材料基本的光电响应性质，处理简单，结果直观，可参考对比的数据丰富，很适合用于介绍 TDDFT 外加电场功能的基本细节。

#### 规范选择

如何加入电场存在规范问题，根据电动力学的知识：

$$E = -\nabla V - \frac{\partial A}{\partial t}$$

第一种是长度规范，把电场的影响归为纯标势的影响，向势场中加入：

$$V \propto E \cdot r$$

因为晶胞很小，按照通常所用的光的波长，体系内的电场在空间上是近似匀强的，故这里没有写成积分的形式。这种形式相对简单，需要处理的项较少，程序处理较快，但是由于 $n$ 的存在，它在周期性体系的边界是不连续的，只能采用锯齿状的方式在边界下降来衔接这种不连续性。对于非周期性体系，边界存在真空层，这种锯齿状衔接不会影响到材料的响应，但是对周期性体系，这种做法就行不通了。

对于周期性体系必须采用速度规范，它用矢势而非标势来引入电场：

$$A(t) \propto \int_0^t E(t')\,dt'$$

在这种规范下，需要对正常的项进行一系列变换，这里不详细论述，想进一步了解的可以参考文献 2 的内容。因此，我们实际上在程序里用的并不是直接的电场，而是标势或者矢势，但是它们都对应一个电场，程序指定电场的参数而不是势场的参数，这样可以统一处理长度和速度规范的场。

简单来说：长度规范只能处理非周期性体系，但是速度较快；速度规范可以处理周期性体系和非周期性体系，但是速度较慢，故一般建议对非周期性体系采用长度规范，周期性体系采用速度规范。

#### 长度规范示例

算例：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/tddft/absorption_H2_length

输入文件 INPUT：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix              H2_absoprion
calculation         md
esolver_type        tddft
pseudo_dir          ../../../tests/PP_ORB
orbital_dir         ../../../tests/PP_ORB

#Parameters (2.Iteration)
ecutwfc             60
scf_thr             1e-6
scf_nmax            100

```

```
#Parameters (3.Basis)
basis_type          lcao
gamma_only          0

#Parameters (4.Smearing)
smearing_method     gauss

#Parameters (5.MD Parameters)
md_type             nve
md_nstep            1000
md_dt               0.005
md_tfirst           0

#Parameters (6.Efield Parameters)
td_vext             1
td_stype            0

td_tstart           1
td_tend             1000

td_vext_dire        3 3
td_ttype            0 0
td_gauss_freq       3.66 1.22
td_gauss_phase      0.0 0.0
td_gauss_sigma      0.2 0.2
td_gauss_t0         300 300
td_gauss_amp        0.01 0.01

#Parameters (7.Output)
out_chg             1
out_efield          1
out_dipole          1
```

其他的注意事项和占据数一样，这里注意补充一些电场参数的细节：

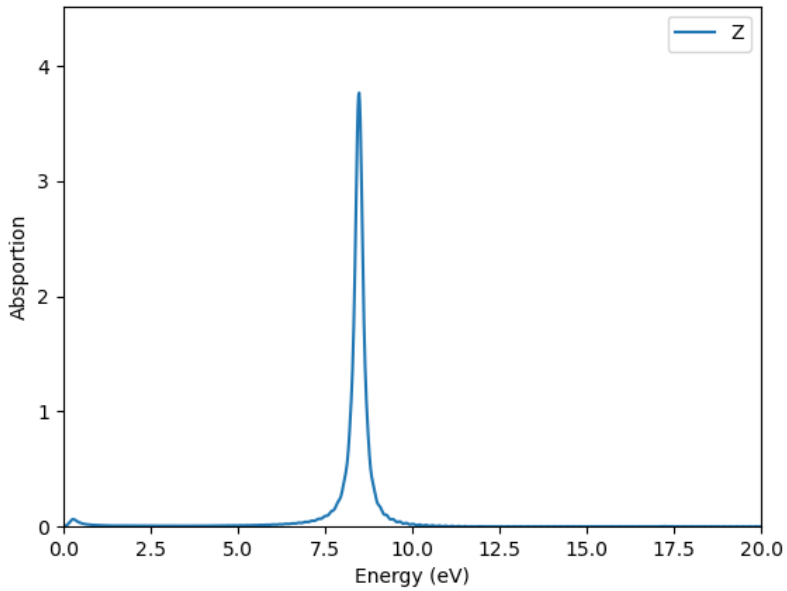
- **md_nstep**：MD 步，这里为了做演示只用了 1000 步，演化总时长 5 fs，为了精度考虑，实际计算时应当增加步数，具体视所需体系有所差异，一般保证 20 fs 以上总时长较为妥当。
- **td_vext**：外加电场的开关，1 为打开外加电场。
- **td_stype**：规范的选择，0 为长度规范，1 为速度规范。
- **td_tstart**：开始加入电场的步数，在 td_tstart 步之前，为无电场状态，一般为 1 即可。
- **td_tend**：电场结束的步数，td_tend 后不再计算电场，用于无电场时节约资源，最大可以等同 md_nstep 的值。td_tend 后，标势为 0，而矢势固定为 td_tend 处的值不再变化。
- **td_type**：电场的波包形状，0 - 3 分别对应高斯、梯形、三角、阶跃等不同类型的电场。
- **td_gauss_xxx**：由于这里指定的是高斯型，对应高斯型的参数，具体可以参考输入参数的说明文档：https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#tddft-time-dependent-density-functional-theory
- **out_dipole**：输出每个时刻的电偶极矩，我们通过对电偶极矩的分析得到吸收谱。
- **out_efield**：输出每个时刻的电场。

注意，这里为了覆盖更大的频率范围，通过对电场形式的参数多加一列的方式，在一次计算中加入了两个不同频率的电场。理论上可以用这种方式得到各种形状的场，当加的较多时，支持采用正则表达式 $m^*x$ 的形式进行简化。

运行算例，会输出全时刻的电偶极矩文件 `SPIN1_DIPOLE`，如果 `nspin` 为 2 则会有 `SPIN2_DIPOLE` 文件分别对应两种自旋的电子各自的电偶极矩；以及电场文件 `efield_x.dat`，$x$ 的数量视所加的电场数量而定，这个算例就是两个：`efield_0.dat` 和 `efield_1.dat`。不考虑单位，通过下列公式得到分子的吸收谱信息：

$$\alpha_i(\omega) = \frac{\int D_i(t) e^{i\omega t}\, dt}{\int E_i(t) e^{i\omega t}\, dt}$$

$D_i$ 为 $i$ 方向的电偶极矩，$E_i$ 为 $i$ 方向的电场。程序自带了相应的后处理功能，具体请参考 `tools`/`plot-tools` 下的说明进行使用，其结果如下：




这表明在 z 方向加的电场激发了一个频率为 8.5 eV 左右的吸收峰。

**速度规范示例**

算例：https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/tddft/absorption_H2_velocity

速度规范的流程与长度规范基本相同，主要不同的参数有两个：

- **td_stype**：变为了 1，代表进行速度规范计算。
- **out_current**：输出电流的开关。对无限大的周期性体系无法定义电偶极，采用电流来代替，公式可以参考本文的参考文献 2。简单来说：

$$I(t) \propto \frac{dD(t)}{dt}$$

因此在傅里叶变换时：

$$D(\omega) \propto \frac{I(\omega)}{i\omega}$$

因此二者可以得到类似的结果，不过 current 的结果由于分母的原因，在极低频时会无法避免的发散。相应的问题在文献 [3] 中有讨论。

### 5.3.4 参考文献

[1] Meng S, Kaxiras E. Real-Time, Local Basis-Set Implementation of Time-Dependent Density Functional Theory for Excited State Dynamics Simulations[J]. The Journal of Chemical Physics, 2008, 129(5), https://doi.org/10.1063/1.2960628.

[2] Pemmaraju C D, Vila F D, Kas J J, et al. Velocity-Gauge Real-Time TDDFT Within a Numerical Atomic Orbital Basis Set[J]. Computer Physics Communications, 2018, 226: 30-38, https://doi.org/10.1016/j.cpc.2018.01.013.

[3] Yabana K, Sugiyama T, Shinohara Y, et al. Time-Dependent Density Functional Theory for Strong Electromagnetic Fields in Crystalline Solids[J]. Physical Review B, 2012, 85(4): 045134, https://link.aps.org/doi/10.1103/PhysRevB.85.045134.

## 5.4 ABACUS 无轨道密度泛函理论方法使用教程

### 5.4.1 无轨道密度泛函理论背景知识

无轨道密度泛函理论 (Orbital free density functional theory, OFDFT) 是一种第一性原理模拟方法，相比于 Kohn Sham DFT (KSDFT)，它的优势之一在于 O(NlnN) 的算法复杂度，这使得 OFDFT 可以用于上万原子甚至更大体系的电子基态计算，或者大体系、长时间的第一性原理分子动力学等。

目前，OFDFT 已被应用于简单金属、合金、半导体、小分子、温稠密物质等体系。

**无轨道密度泛函理论**

在 OFDFT 的框架下，体系的总能量泛函可以写为

$$E_{OF}[\rho] = T_s[\rho] + E_{ext}[\rho] + E_{\Pi}[\rho] + E_{xc}[\rho] + E_{\Pi}.$$

依次为无相互作用动能 $T_s[\rho]$，电子-离子相互作用能 $E_{ext}[\rho]$，电子-电子相互作用能 $E_{\Pi}[\rho]$，交换关联能 $E_{xc}[\rho]$，离子-离子相互作用能 $E_{\Pi}$，其中 $\rho$ 为电荷密度。

为了在粒子数守恒的条件下求解其极小值，利用拉格朗日乘子法，定义

$$L = E_{OF}[\rho] - \mu\left(\int \rho(r)dr - N\right),$$

可以证明，这里的乘子 $\mu$ 就是无相互作用体系中最富占据态的能量，即化学势。为了保证密度处处为正，一般对 $\sqrt{\rho}$ 进行优化，因此求 $L$ 对 $\sqrt{\rho}$ 的导数，为了方便，定义

$$\phi(r) = \sqrt{\rho(r)},$$

则有

$$\frac{\delta L}{\delta \phi} = \frac{\delta E_{OF}[\rho]}{\delta \phi} - 2\mu\phi = 0 \rightarrow V(r)\phi(r) = \mu\phi(r). \tag{*}$$

其中 $V(r) = \frac{\delta E_{OF}[\rho]}{\delta \rho(r)}$ 为势能。这就是 OFDFT 求解的方程，一般用共轭梯度（CG）法、截断牛顿（TN）法或 L-BFGS 等优化算法求解。目前 ABACUS 中实现了 TN 法和两种 CG 法（Polak-Ribire 形式和 Hager-Zhang 形式），默认采用 TN 法。

ABACUS 基于平面波基矢量，实现了上述流程，可以进行基于 OFDFT 的自洽计算，分子动力学计算，以及结构弛豫。

**动能泛函**

OFDFT 的精度高度依赖于动能泛函（kinetic energy density functional, 简称 KEDF）的精度，目前 ABACUS 中实现了 Thomas-Fermi (TF) [1], von Weizsäcker (vW) [2], TFAvW [3], Wang-Teter (WT) [4], Luo-Karasiev-Trickey (LKT) [5] 共五种动能泛函。

下面我们对这些泛函做简单介绍，并且介绍在 ABACUS 的 INPUT 文件中如何设置相关的参数。

Thomas-Fermi KEDF:

设置 INPUT 文件中的 `of_kinetic tf` 参数

$$T_{TF} = \frac{3}{10}(3\pi^2)^{2/3}\int \rho^{5/3}(r)dr,$$

对均匀电子气精确成立，可用于极高温体系，比如处于温稠密状态的金属。可通过 `of_tf_weight` 调整其权重，默认为 `1`。

von Weizsäcker KEDF:

设置 `of_kinetic vw`

$$T_{vw} = -\frac{1}{2}\int \sqrt{\rho(r)\nabla^2}\sqrt{\rho(r)}dr,$$

对单电子、双电子体系（只有一条轨道）严格成立，一般不单独使用。可通过 `of_vw_weight` 调整其权重，默认为 `1`。

TFλvW KEDF:

设置 `of_kinetic tf+`

$$T = T_{TF} + \lambda T_{vw},$$

当 $\lambda = \frac{1}{9}$ 时就是 TF KEDF 的二阶梯度展开，一般 $\lambda = \frac{1}{5}$ 时表现最好。

参数 $\lambda$ 可通过 `of_vw_weight` 设置，默认为 `1`。

Wang-Teter KEDF:

设置 `of_kinetic wt`

$$T_{WT} = \frac{3}{10}(3\pi^2)^{2/3}\iint \rho^{\alpha}(r)W(r-r')\rho^{\beta}(r')drdr' + T_{vW} + T_{TF},$$

基于 Lindhard 响应函数推导，在简单金属 Li、Mg、Al 中有着不错的表现。是 ABACUS 默认采用的 KEDF。

参数 $\alpha, \beta$ 可通过 `of_wt_alpha` 和 `of_wt_beta` 设置，默认值均为 $\frac{5}{6}$。

Luo-Karasiev-Trickey KEDF:

设置 `of_kinetic lkt`

$$T_{LKT} = \int T_{TF}\frac{1}{\cosh \alpha s}dr + T_{vW}, \quad s = \frac{1}{2(3\pi^2)^{1/3}}\left|\nabla\rho\right|\rho^{4/3}.$$

可用于简单金属和半导体，计算效率较高，但在简单金属中精度低于 WT KEDF。

参数 $\alpha$ 可通过 `of_lkt_a` 设置，默认值为 `1.3`。

**局域赝势**

由于 OFDFT 中舍弃了单电子轨道，无法采用常用的非局域赝势，如模守恒赝势，而必须采用局域赝势。

目前 ABACUS 支持 BLPS (bulk-derived local pseudopotential)。

下载地址：https://github.com/EACcodes/local-pseudopotentials 实空间赝势：ABINIT, ABACUS；倒空间赝势：PROFESS。赝势生成 (需要和 ABINIT 7.0.5 结合)：https://github.com/EACcodes/BLPSGenerator 覆盖 Li, Mg, Al, Si, P, Ga, As, In, Sb 九种元素

使用 BLPS 时，需要在 ABACUS 里调整的参数有：

INPUT 中：pseudo_rcut 16

STRU 中：赝势种类设置为 blps，比如 Al 26.98 al.lda.lps blps

### 5.4.2 ABACUS 中进行 OFDFT 计算的具体流程

**自洽计算**

示例

下面是输入文件的示例：

INPUT 文件记录 OFDFT 计算所需主要参数

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix      example
calculation scf
esolver_type    ofdft

symmetry    1
pseudo_dir  ../../PP_ORB/
pseudo_rcut 16
nspin        1

#Parameters (2.Iteration)
ecutwfc     60
scf_nmax    50

#Parameters (3.Basis)
basis_type  pw

#OFDFT
of_kinetic  wt
of_method   tn
```

STRU 文件记录元素种类、质量、赝势，晶格矢量，原子坐标等信息

```
ATOMIC_SPECIES
Al 26.98 al.lda.lps blps

LATTICE_CONSTANT
7.50241114482312  // add lattice constant

LATTICE_VECTORS
0.000000000000    0.500000000000    0.500000000000
0.500000000000    0.000000000000    0.500000000000
0.500000000000    0.500000000000    0.000000000000

ATOMIC_POSITIONS
Direct

Al
0
1
```

```
18      0.000000000000      0.000000000000      0.000000000000 1 1 1
```

KPT 文件（因为 OFDFT 没有电子波函数，所以不需要布里渊区的多个 k 点，Gamma 点就可以）

```
1 K_POINTS
2 0
3 Gamma
4 1 1 1 0 0 0
```

如上所示，与 KSDFT 的自洽计算相比，OFDFT 自洽计算的输入文件有以下几个区别：

- **INPUT**
  - 不需要设置 smearing 和 charge mixing 相关参数，如果设置了也没有关系，这些参数不会影响 OFDFT 计算；
  - 将 `esolver_type` 设置为 `ofdft`；
  - 将 `pseudo_rcut` 设置为 16，以适配 BLPS 优势。
- **STRU**
  - 将优势种类设置为 `blps`。

做完以上调整后，即可使用默认参数进行 OFDFT 的自洽计算。

下面列举一些其它的重要参数：

- `of_kinetic`：用于选择功能泛函，可选项有 `tf`, `vw`, `tf+`, `wt`, `lkt`，默认值为 `wt`，具体介绍见 1.2 节；
- `of_method`：用于选择优化方法，可选项有 `tn`, `cg1`, `cg2`，分别对应截断牛顿法和两种 CG 法（Polak-Ribire 形式和 Hager-Zhang 形式），默认为 `tn`。一般而言，效率上 `tn > cg2 > cg1`；
- `of_full_pw`：做快速傅里叶变换（FFT）时，是否使用全部的平面波，默认为 `True`。建议打开，可以保证计算的稳定性和精度；
- `of_full_pw_dim`：控制 FFT 维数的奇偶性，可选项有 `0`, `1`, `2`，分别表示可奇偶，保证为奇数，保证为偶数，默认为 `0`。FFT 维数为偶数时，可能导致微小的误差，但一般来说可以忽略。需要注意的是，如果打开了 `nbspline`，则需要设置 `of_full_pw_dim 1`，否则会导致计算不稳定。

注意事项

目前 ABACUS 的 OFDFT 模块并不是十分完善，使用时请注意以下几个注意事项：

- 目前 OFDFT 不支持 gamma only，因此使用 OFDFT 功能时请关闭 `gamma_only`；
- 目前 OFDFT 只支持自旋简并，即 `nspin 1` 的计算；
- 如果使用 PBE 泛函，建议用 `dft_functional XC_GGA_X_PBE+XC_GGA_C_PBE` 调用 Libxc 中的 PBE，否则可能导致计算不稳定。

### 分子动力学与结构弛豫

ABACUS 中支持使用 OFDFT 作为能量、力和应力的求解器，进行分子动力学模拟与结构弛豫。

与使用 KSDFT 进行分子动力学或结构弛豫相比，使用 OFDFT 时，不需要对 MD，relax，或 cell-relax 相关参数进行修改，只需要按照 2.1 中的方式，将能量、力和应力的求解器替换为 OFDFT。

下面是几个实际的 INPUT 例子：

分子动力学（MD）

```
1 INPUT_PARAMETERS
2 #Parameters (1.General)
3 suffix          test
4 calculation     md
5 esolver_type    ofdft
```

```
pseudo_dir      ../../PP_ORB
pseudo_rcut     16

#Parameters (2.Iteration)
ecutwfc     60
scf_nmax    100

#OFDFT
of_kinetic  wt
of_method   tn
of_full_pw_dim   1

#Parameters (3.Basis)
basis_type  pw

md_restart  0
md_type     nvt
md_nstep    2
md_dt       0.25
md_tfirst   58022.52706
md_dumpfreq 10
md_tfreq    1.08
md_tchain   1
nbspline    10
```

原子结构弛豫（relax）

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix          test
calculation     relax
esolver_type    ofdft
pseudo_dir      ../../PP_ORB
pseudo_rcut     16

#Parameters (2.Iteration)
ecutwfc     60
scf_nmax    100

#OFDFT
of_kinetic  wt
of_method   tn

#Parameters (3.Basis)
basis_type  pw

relax_nmax  50
```

晶格弛豫（cell-relax）

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix          test
calculation     cell-relax
esolver_type    ofdft
pseudo_dir      ../../PP_ORB
pseudo_rcut     16

#Parameters (2.Iteration)
ecutwfc     60
scf_nmax    100

#OFDFT
of_kinetic  wt
of_method   tn

#Parameters (3.Basis)
basis_type  pw

relax_nmax  50
```

### 5.4.3 参考文献

[1] Fermi E. Statistical method to determine some properties of atoms[J]. Rend. Accad. Naz. Lincei, 1927, 6(602-607): 5.

[2] Weizsäcker C F. Zur theorie der kernmassen[J]. Zeitschrift für Physik, 1935, 96(7-8): 431-458.

[3] Berk A. Lower-bound energy functionals and their application to diatomic systems[J]. Physical Review A, 1983, 28(4): 1908.

[4] Wang L W, Teter M P. Kinetic-energy functional of the electron density[J]. Physical Review B, 1992, 45(23): 13196.

[5] Luo K, Karasiev V V, Trickey S B. A simple generalized gradient approximation for the noninteracting kinetic energy density functional[J]. Physical Review B, 2018, 98(4): 041111.

## 5.5 ABACUS 隐式溶剂模型使用教程

### 5.5.1 介绍

近年来，电化学表面反应，例如一些电催化系统，无论是在工业界还是科学研究领域，都受到了广泛的关注。而对于一个真实电极表面反应体系，往往需要考虑许多复杂的情况，例如电极与电解液之间的固液界面，在材料模拟中经常遇到和使用。在这种系统的第一性原理计算中应该考虑溶剂化效应，以便获得准确的性质预测。

溶剂化是指溶质和溶剂之间的相互作用，使溶质在溶液中稳定。这种相互作用的强度和性质影响体系的许多特性。在计算化学中，溶剂模型 (Solvation model) 是描述溶剂化行为的一种计算方法。

隐式溶剂化模型 (Implicit solvation model) 是一种处理溶剂化效应的成熟方法，在有限系统和周期系统中得到了广泛的应用。这种方法将溶剂视为连续介质，而不是单独的“显式”溶剂分子，这意味着嵌入隐式溶剂中的

溶质以及溶剂自由度的平均值将隐含在溶剂浴的性质中。与“显式”方法相比，这种隐式溶剂化模型可以以低得多的计算成本提供定性正确的结果，特别适用于大型复杂系统。

ABACUS 中实现的隐式溶剂化模型采用了 Mathew、Sundararaman、Letchworth-Waver、Arias 和 Hennig 在 2014 年开发的方法 [1]。

### 5.5.2 文档和算例

ABACUS 里关于隐式溶剂模型的线上文档地址为：http://abacus.deepmodeling.com/en/latest/advanced/scf/advanced.html#implicit-solvation-model

ABACUS 里关于隐式溶剂模型参数的线上文档地址为：

https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#implicit-solvation-model ABACUS 的隐式溶剂模型算例下载地址为（gitee）：

https://gitee.com/deepmodeling/abacus-develop/tree/develop/examples/32_imp_sol_model//Pt-slab

可以采用的下载命令是：

```
git clone https://gitee.com/deepmodeling/abacus-develop.git
```

之后进入 /abacus-develop/examples/32_imp_sol/Pt-slab 目录

或者采用 Github 仓库地址：

https://github.com/deepmodeling/abacus-develop/tree/develop/examples/32_imp_sol_model/Pt-slab

### 5.5.3 输入参数

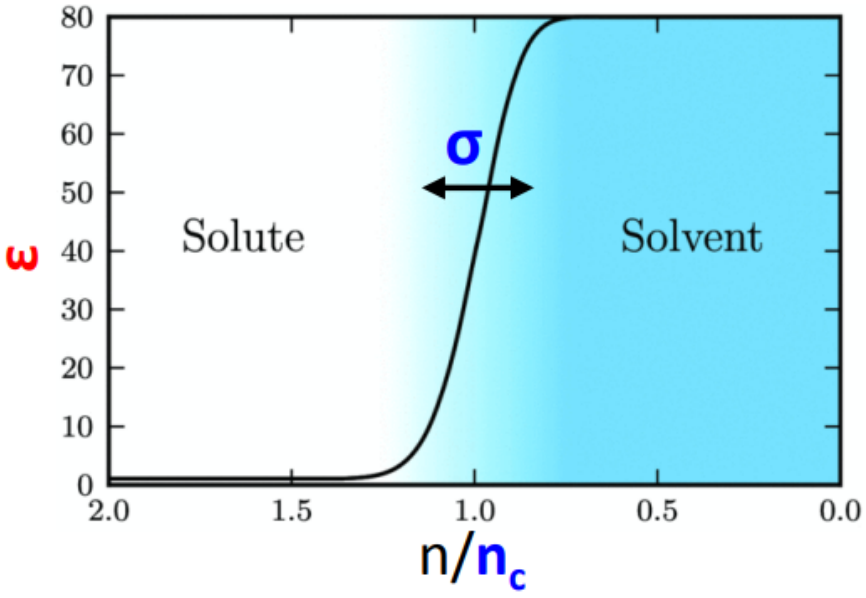
```
INPUT_PARAMETERS
#Parameters (1.General)
suffix              H2
calculation         scf
ntype               1
nbands              2
symmetry            0
pseudo_dir          ./

#Parameters (2.Iteration)
ecutwfc             60
scf_thr             1e-6
scf_nmax            100

#Parameters (3.Basis)
basis_type          pw

#Parameters (Solvation Model)
imp_sol             1
eb_k                80
tau                 0.000010798
sigma_k             0.6
```

```
nc_k                    0.00037
```

以算例提供的 INPUT 文件为例，使用隐式溶剂模型方法只需添加如下参数即可：

- **imp_sol**：类型是 Bool，指定隐式溶剂模型的开 (1) 或者关 (0)，默认值为 0
- **eb_k**：类型是 Real，溶剂的相对介电常数，水为 80
- **tau**：类型是 Real，有效表面张力参数，用于描述未被静电项捕获的溶质和溶剂之间的空化、分散和排斥相互作用，单位是 $Ry/Bohr^2$，默认值为 1.0798e-05
- **sigma_k**：类型是 Real，由溶质的电子结构隐含地确定的扩散腔的宽度（溶质电子密度与 nc_k 的比值，无量纲），默认值为 0.6
- **nc_k**：类型是 Real，介电腔形成时的电子密度值，单位是 $Bohr^3$，默认值为 0.00037

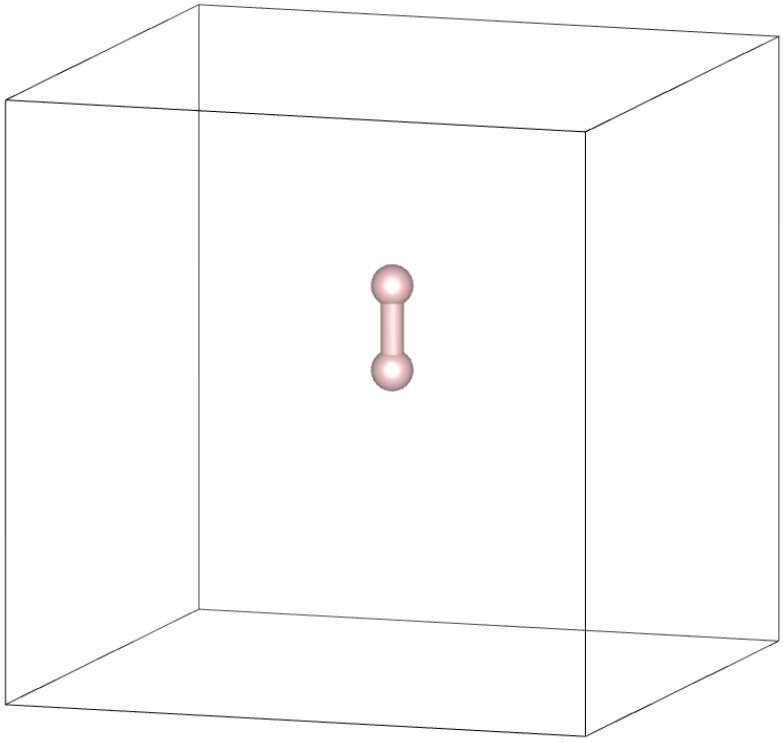
注意，eb_k 可根据需要设置为模拟溶剂分子的相对介电常数参数，默认值 80 为水分子情况。tau，sigma_k，nc_k 的默认值均由在水溶液情况下与实验结果拟合得到，如需考虑其他溶剂，可自行测试拟合。




### 5.5.4 预期结果

如图所示，一个氢分子位于超胞中，模拟其处于溶液环境时的状态。将 imp_sol 设为 1，其他参数使用默认值，表示溶剂为水。在输出文件 OUT.ABACUS/running_scf.log 中，E_sol_el 和 E_sol_cav 分别表示静电作用和空腔作用引起的能量修正。如案例所示，E_sol_el 代表溶质和溶剂分子之间的静电感应作用，贡献的能量修正一般为负值。E_sol_cav 代表溶质分子在溶剂中产生空腔所引起的能量修正，一般为正值。

体系溶剂化能的计算：打开隐式溶剂模型（imp_sol = 1）时体系总能量 (结构优化之后) 与真空情况（imp_sol = 0）体系总能量（结构优化之后）之差。



| Energy | Rydberg | eV |
|---|---|---|
| E_KohnSham | -2.33128015246 | -31.7186937078 |
| E_Harris | -2.3318039001 | -31.72581966 |
| E_band | -1.48509692705 | -20.2057802902 |
| E_one_elec | -2.78774594423 | -37.9292294179 |
| E_Hartree | +1.52957079998 | +20.8108783741 |
| E_xc | -1.37573099267 | -18.7177804156 |
| E_Ewald | +0.302102236829 | +4.11031179942 |
| E_demet | +0 | +0 |
| E_descf | +0 | +0 |
| E_exx | +0 | +0 |
| E_sol_el | -0.00119898588563 | -0.0163130398661 |
| E_sol_cav | +0.00172273352515 | +0.0234389920776 |
| E_Fermi | -0.742548463525 | -10.1028901451 |

### 5.5.5 参考文献

[1] J. Chem. Phys. 140, 084106 (2014), https://doi.org/10.1063/1.4865107

## 5.6 ABACUS+DPGEN 使用教程

### 5.6.1 介绍

本教程旨在介绍结合 ABACUS（中文名原子算筹，这里基于 ABACUS 3.2.0 版本）和 DPGEN 软件结合生成机器学习 DP 势，这里 ABACUS 扮演了提供第一性原理训练数据的作用。本教程以碳化硅（SiC）材料为案例，

并且教大家使用 DPGEN autotest 功能，计算 SiC 的 Equation of State (EOS) 和弹性常数等性质。先介绍主要软件和计算平台：

DPGEN 软件：是一个结合第一性原理软件来自动产生训练数据标签的软件，如果不熟悉 DPGEN 的读者，这里可以推荐大家先阅读 DPGEN 的相关原理文档和说明，熟悉这部分的读者可以跳过：

https://docs.deepmodeling.com/projects/dpgen/en/latest/

https://bohrium-doc.dp.tech/docs/software/DP-GEN

https://bohrium-doc.dp.tech/docs/software/DP-GEN_init

https://bohrium-doc.dp.tech/docs/software/DP-GEN_autotest

https://www.sciencedirect.com/science/article/pii/S001046552030045X?via%3Dihub

计算平台：由于本案例通过 Bohrium 平台教学，所以所用到的 machine.json 文件仅限于 Bohrium 平台使用，如需在其他平台上使用，可参考下面网址，修改 machine.json 文件：

https://docs.deepmodeling.com/projects/dpgen/en/latest/run/example-of-machine.html

### 5.6.2 准备

本教程用到的所有输入文件，均可点击这里下载。
(https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/abacus-dpgen)
此外，在 Linux 下可以采用的下载命令是：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

**案例下载地址**

之后进入 /abacus-user-guide/examples/md 目录
或者采用 Github 仓库地址：
https://github.com/MCresearch/abacus-user-guide/tree/master/examples/md
下载完之后，应该可以看到三个文件夹：
init，run，以及 auto-test
init 文件夹：包含用于 dpgen init_bulk 步骤所需的输入文件，用户可参考其设置自己的体系参数。
run 文件夹：包含用于 dpgen run 步骤所需的输入文件，用户可参考其设置自己的体系参数。
auto-test 文件夹：包含用于计算弹性常数等性质的输入文件，用户可参考其设置自己的体系参数。

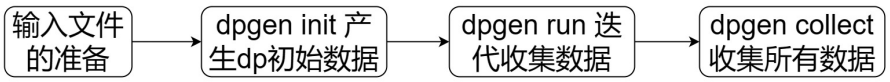
**安装相关软件**

这里默认读者已经安装好了 ABACUS 软件，本算例中 ABACUS 计算采用的是数值原子轨道的 LCAO（Linear Combination of Atomic Orbitals）方法，因为一般来说 LCAO 会比平面波效率更高，此外读者也可以把密度泛函理论计算部分改成平面波基矢量（后续会讲到）。

开始之前，请务必安装/更新 dpgen 和 dpdata（用来转换数据）到最新版本：
1. 安装/更新 dpdata pip install –user git+“https://gitee.com/deepmodeling/dpdata.git@devel
2. 安装/更新 dpgen pip install –user git+“https://gitee.com/deepmodeling/dpgen.git@devel

### 5.6.3 结合 DPGEN 生成 DP 数据的流程

DPGEN 使用流程如下，接下来我们将一一展开介绍：




**准备赝势和数值原子轨道文件**

本例子所需文件有：

模守恒赝势文件，格式为 *.upf，也可以使用其他 ABACUS 支持的赝势格式，注意 ABACUS 目前不支持 PAW 方法或者超软赝势。

数值原子轨道文件，格式为 *.orb，注意这里的数值原子轨道文件要和赝势文件匹配，也就是数值原子轨道是由指定赝势产生的，关于如何生成数值原子轨道，我们会提供中文教程。

注：在 ABACUS 官网上可以下载到赝势和数值原子轨道文件：

http://abacus.ustc.edu.cn/pseudo/list.htm

本例采用的赝势和轨道是：

C 的 PBE 模守恒 ONCV 模守恒赝势：C_ONCV_PBE-1.0.upf

Si 的 PBE 模守恒 ONCV 模守恒赝势：Si_ONCV_PBE-1.0.upf

C 的 PBE 模守恒 ONCV 赝势对应的数值原子轨道：C_gga_9au_100Ry_3s3p2d.orb

Si 的 PBE 模守恒 ONCV 赝势对应的数值原子轨道：Si_gga_9au_100Ry_3s3p2d.orb

注：如果是使用平面波基矢量来做密度泛函理论计算，则不需要提供数值原子轨道文件，结合 DPGEN 使用的流程是一致的（仅需要删除调用轨道文件的输入参数即可）。如果采用其他 ABACUS 支持的赝势文件，可自行生成轨道文件（请参考本教程系列的其它相关中文教程）.

注：这里 C_gga_9au_100Ry_3s3p2d.orb 的意思是 C 元素、采用 GGA 泛函、轨道截断半径是 9 a.u.（轨道截断半径越大，则 ABACUS 所需求解 Kohn-Sham 方程的时间会越长，而且这个时间对于轨道半径极其敏感，推荐可以测试不同轨道半径，选取合适的、截断半径小的轨道可以加速计算），100 Ry 是推荐的平面波截断值（哪怕用了数值原子轨道，程序里还是会用平面波来做某些物理量的计算，比如模守恒赝势的局域赝势部分），3s3p2d 表示 3 个 s 径向轨道、3 个 p 轨道、2 个 d 轨道。

**准备原子体系的结构文件**

本步骤所需的描述原子结构的文件有：

*.cif

可以从多个渠道下载所需体系的结构文件，本案例主要从 materials project 网站下载 cif 文件，使用的 SiC 对应多个原子构型，主要有 3C、2H、4H、6H 四个构型，采用哪些构型取决于用户所研究的体系和性质，一般可选取所关心的物质温度和压强区间内可能出现的结构。本案例主要采用 3C 和 2H 构型来演示。

3C-SiC：https://materialsproject.org/materials/mp-8062?formula=SiC

2H-SiC: https://materialsproject.org/materials/mp-7140?_skip=15&formula=SiC

4H-SiC: https://materialsproject.org/materials/mp-11714?formula=SiC

6H-SiC: https://materialsproject.org/materials/mp-7631?formula=SiC

得到了 cif 之后，可以使用下列方法把 cif 转换成 STRU 文件：

1. 安装 ASE-ABACUS 接口：

```
git clone https://gitlab.com/1041176461/ase-abacus.git
cd ase-abacus
python3 setup.py install
```

2. 设置环境变量（可选）

ABACUS 支持两种基组：平面波（PW）和数值原子轨道（LCAO）。赝势和轨道文件的存放路径可以通过环境变量设置，分别为：ABACUS_PP_PATH 和 ABACUS_ORBITAL_PATH, 设置方法如下：

```
PP=\${HOME}/pseudopotentials
ORB=\${HOME}/orbitals
export ABACUS_PP_PATH=\${PP}
export ABACUS_ORBITAL_PATH=\${ORB}
```

PW 计算只需要设置 ABACUS_PP_PATH 。LCAO 需要两个都设置：ABACUS_PP_PATH and ABACUS_ORBITAL_PATH 。

3. CIF 转 STRU

```
from ase.io import read, write
from pathlib import Path

cs_dir = './'
cs_vasp = Path(cs_dir, 'SiC.cif') \# 修改为自己的 cif 文件名
cs_atoms = read(cs_vasp, format='cif')
cs_stru = Path(cs_dir, 'STRU')
pp = {'Si':'Si_ONCV_PBE-1.0.upf','C':'C_ONCV_PBE-1.0.upf'} \# 修改为所对应的赝势文件名
basis = {'Si':'Si_gga_9au_100Ry_3s3p2d.orb','C':'C_gga_9au_100Ry_3s3p2d.orb'} \#  修改为所对应的轨道文件名
write(cs_stru, cs_atoms, format='abacus', pp=pp, basis=basis)
```

运行上述 python 脚本，把 cif 转成 ABACUS 所需的 STRU 文件。

**产生初始训练数据**

本步骤所需文件有：

原子结构文件：STRU

数值原子轨道文件：*.orb

赝势文件：*.upf

布里渊区 k 点采样：KPT

提交任务的文件：machine.json

DPGEN 的输入参数文件：param.json

为每一个构型新建一个名为 init 的文件夹。进入该文件夹后，为每个构型都新建一个单独的文件夹，之后单独运行 dpgen init_bulk 命令，用于产生接下来的 dpgen run 步骤的初始数据集。本案例所提供的初始原子构型为 3C 和 2H 结构，并把赝势文件、轨道文件、STRU 文件放入对应的文件夹里。按照下面链接的案例准备适合自己机器的 machine.json 文件，并分别放入 3C 和 2H 文件夹。

https://docs.deepmodeling.com/projects/dpgen/en/latest/run/example-of-machine.html 本教程所用到的文件可以从 Gitee 下载，修改为符合自己的配置即可。

接下来准备 param.jsion 文件，本步骤最关键的设置也是 param.json 文件设置，案例如下：

```
{
    "init_fp_style":     "ABACUS",
    "stages":            [1,2,3,4],
    "cell_type":        "diamond",
    "super_cell":        [1, 1, 1],
    "elements":          ["Si","C"],
```

```
    "from_poscar":       true,
    "from_poscar_path":  "./STRU",
    "potcars":           ["Si_ONCV_PBE-1.0.upf","C_ONCV_PBE-1.0.upf"],
    "orb_files":         ["Si_gga_9au_100Ry_3s3p2d.orb","C_gga_9au_100Ry_3s3p2d.orb"],
    "relax_incar":       "./INPUT_rlx",
    "md_incar" :         "./INPUT_md",
    "relax_kpt":         "./KPT",
    "md_kpt":            "./KPT",
    "md_nstep":          10,
    "scale":             [0.99,1.00,1.01],
    "skip_relax":        false,
    "pert_numb":         10,
    "pert_box":          0.03,
    "pert_atom":         0.01,
    "deepgen_templ":     "../generator/template/",
    "coll_ndata":        5000,
    "_comment":          "that's all"
}
```

init_fp_style 关键字需设置为 ABACUS，如果 from_poscar 为 true，则会读取 from_poscar_path 指定的 STRU 文件，且 cell_type 关键字失效。super_cell 用于括胞，推荐直接使用括胞之后的 STRU 文件，则 super_cell 设置为 [1, 1, 1]。括胞一般根据需求而定，使得三个方向的 cell 长度约为 6 埃以上（经验性的）。其中还用到了 *.“upf 和 *.“orb 文件，修改为自己所计算的体系即可。

init_bulk 的基本操作流程分为以下 4 个顺序步骤：

生成初始 (平衡) 结构：输出文件存储在 00.place_ele。

产生 MD 起始构象：放缩扰动 00.place_ele 中的结构，输出文件存储在 01.scale_pert。

执行 AIMD：取 01.scale_pert 起始构象调用 vasp 执行少步数 AIMD，输出文件存储在 02.md。

整理初始数据：调用 dpdata 提取 02.md 中 AIMD 各 snapshots 的数据作为初始数据，输出文件存储在 02.md。

param.json 中将使用 stages 关键词的取值 1 4 控制以上 4 步的执行，操作中，各步骤 (2, 3, 4 在所需前一步输出文件存在的前提下) 可独立执行。

其他具体参数含义可参考 dpgen 官网：

https://docs.deepmodeling.com/projects/dpgen/en/latest/init/init-bulk-jdata.html

另外，还需准备两个 ABACUS 输入文件 INPUT_rlx 和 INPUT_md。

INPUT_rlx 主要用于优化所下载的结构，如下：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix                  ABACUS
calculation             cell-relax
#Parameters (2.Iteration)
ecutwfc                 100
scf_thr                 1e-6
scf_nmax                100
#Parameters (3.Basis)
basis_type        lcao
#Parameters (4.Smearing)
smearing_method         gauss
```

```
smearing_sigma        0.002
#Parameters (5.Mixing)
mixing_type           pulay
mixing_beta           0.3
cal_stress            1
```

INPUT_md 主要用于产生一定数量的 AIMD 轨迹，用于 DPGEN 迭代的输出数据，如下：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix            ABACUS
calculation       md
#Parameters (2.Iteration)
ecutwfc           100
scf_thr           1e-6
scf_nmax          100
#Parameters (3.Basis)
basis_type        lcao
#Parameters (4.Smearing)
smearing_method   gauss
smearing_sigma    0.002
#Parameters (5.Mixing)
mixing_type       pulay
mixing_beta       0.3
cal_stress        1
#MD
chg_extrap second-order
md_type           2
md_tfirst         300
md_tlast          300
md_nstep          10
md_tfirst         10
md_tfreq          0.5
```

注：ABACUS 的 MD 关键字在此 3.2.0 新版本有所更改，如果是使用的最新版本，请严格参考手册设置关键字，如 md_type 在新版本里改为了 nvt、npt 等形式，而不再用数字（如：2）等：

此外，还需准备一个存放布里渊区 k 点的文件，名为 KPT：

```
K_POINTS
0
Gamma
4 4 4 0 0 0
```

也可在 INPUT 文件里直接指定 kspacing 参数（注意该参数可选 1 维或者 3 维，如果是表面系统可选 3 维因为真空方向不需要太多 k 点）。

用户可自行根据所研究体系，设置、更改上述参数。

所有以上输入文件准备完毕后，采用如下命令分别对 3C 和 2H 结构运行 dpgen：

nohup dpgen init_bulk param.json machine.json 1>log 2>err&

上述命令采用后台提交 dpgen，等所有计算完成之后，dpgen 会自动收集训练 DP 势所需的数据，且转换为 DeePMD-kit 的数据格式，保存在./STRU.01x01x01/02.md/sys-0036-0036/deepmd 目录里。

### 进行数据采样的迭代

run 主流程包含一系列迭代，各迭代会根据所选系统在给定的温度、压力或体积等条件下进行 DPMD 采样与筛选，实现同步学习，所有迭代希望能够有效覆盖训练可靠模型所需的势能面上样本空间。每个迭代依次：基于 init 准备的初始训练数据与之前迭代累积的训练数据训练多个新的模型-> 使用当前的模型进行 DPMD 采样，根据模型间预测偏差 (model-deviation) 挑选候选 snapshot 构象-> 将本轮候选 snapshot 构象进行第一性原理标定并加入数据集扩展数据集。

以上每个迭代中包含的 3 个步骤分别被命名为 `00.train`，`01.model_devi`，`02.fp`，细节如下：

- **00.train**: 使用 init 准备的初始训练数据与之前迭代累积的训练数据，调用 DeepMD-kit 训练多个（默认 4 个）模型。模型间的唯一区别来自于初始化神经网络时使用不同的随机数种子。
- **01.model_devi**: 指代 model-deviation。调用 LAMMPS 使用 `00.train` 的 1 个模型进行 MD 模拟。对于任一 MD 中 snapshot，模型间预测偏差越大意味着当前模型系统对该 snapshot 构象的精度越低，通过引入模型偏差作为误差判断并设定上下限，挑选出有希望有效改进模型对 PES 整体预测精度的 snapshot 构象，作为准备加入训练数据的候选构象。
- **02.fp**: 调用 ABACUS 对 `01.model_devi` 选取的候选构象进行第一性原理定标 (单点计算)，并调用 dpdata 收集整理所得数据加入到训练数据集中。

run 执行后，相关文件即存储在 `....../iter.*(迭代序号)/步骤文件夹`内。以上每个迭代中的每个步骤在实现中被分解为 3 个阶段。以 `00.train` 为例，3 阶段分别对应: 0: make_train，为训练任务准备脚本；1: run_train，依据机器配置上传文件并执行训练任务；2: post_train，收集整理分析训练任务的结果。`01.model_devi` 与 `02.fp` 的实现步骤与之类似，分别使用 3，4，5 及 6，7，8 来标记其阶段。

用输出文件 `record.dpgen` 来记录进程已经执行过当前正在执行的阶段。如进程中断后需要续算，DP-GEN 将根据此文件的记录来自前恢复 run 主进程。用户也可根据需求手动修改 `record.dpgen` 中的记录，来控制接下来将执行哪一阶段的任务：例如，删除最后一阶段 (可能未执行完成即中断) 的记录，并从前一个已完成阶段后恢复任务。可以通过查看 `record.dpgen` 文件监控当前任务的执行阶段。

**本步骤所需文件有：**

- `STRU`
- `*.orb`
- `*.upf`
- `machine.json` （可以直接用 `dpgen init bulk` 里的）
- `param.json`
- `dpgen init bulk` 产生的初始训练数据

**主要步骤如下：**

1. 新建一个文件夹，命名为：`run`，并进入此目录里
2. 在 `run` 文件夹里面新建 `init.conf` 和 `init_data` 文件夹，用于存放 `dpgen init bulk` 产生的构型和初始训练数据
3. 为了区分不同构型和初始训练数据，在 `init.conf` 和 `init_data` 文件夹里新建名为 `3C` 和 `2H` 的两个文件夹
4. 拷贝 `dpgen init bulk` 产生的构型和初始训练数据至 `init.conf` 和 `init_data` 文件夹里，如：

```
cp -rf yourPath/init/3C/STRU.01x01x01/02.md/sys-0032-0032 yourPath/run/init_conf/3C
cp -rf yourPath/init/2H/STRU.01x01x01/02.md/sys-0036-0036 yourPath/run/init_conf/2H

cp -rf yourPath/init/3C/STRU.01x01x01/02.md/sys-0032-0032/deepmd/* yourPath/run/init_data/3C
```

```
cp -rf yourPath/init/2H/STRU.01x01x01/02.md/sys-0036-0036/deepmd/* yourPath/run/init_data/2H
```

拷贝 machine.json、*.orb 和 *.upf 文件到 run 文件夹

新建一个 param.json 文件，如下案例：

```
{
    "type_map": ["Si","C"],
    "mass_map": [28.085,12.011],
    "init_data_prefix": "./init_data",
    "init_data_sys": [
        "3C",
        "2H"
    ],
    "sys_format":"abacus/stru",
    "sys_configs_prefix": "./init_conf",
    "sys_configs": [
        ["3C/sys-0032-0032/scale-1.000/00****/STRU"],
        ["2H/sys-0036-0036/scale-1.000/00****/STRU"]
    ],
    "_comment": " that's all ",
    "numb_models": 4,
    "default_training_param": {
        "model": {
            "type_map": ["Si","C"],
            "descriptor": {
                "type": "se_e2_a",
                "sel": "auto",
                "rcut_smth": 0.5,
                "rcut": 6.0,
                "neuron": [25, 50, 100],
                "resnet_dt": true,
                "axis_neuron": 12,
                "seed": 1
            },
            "fitting_net": {
                "neuron": [240, 240, 240],
                "resnet_dt": false,
                "seed": 1
            }
        },
        "learning_rate": {
            "type": "exp",
            "start_lr": 0.001,
            "stop_lr":  3.5e-8,
            "decay_steps": 2000
        },
        "loss": {
            "start_pref_e": 0.02,
```

```
            "limit_pref_e": 2,
            "start_pref_f": 1000,
            "limit_pref_f": 1,
            "start_pref_v": 0.0,
            "limit_pref_v": 0.0
        },
        "training": {
            "stop_batch": 400000,
            "disp_file": "lcurve.out",
            "disp_freq": 100,
            "numb_test": 10,
            "save_freq": 1000,
            "save_ckpt": "model.ckpt",
            "disp_training": true,
            "time_training": true,
            "profiling": false,
            "profiling_file": "timeline.json",
            "_comment": "that's all"
        }
    },
    "model_devi_dt": 0.002,
    "model_devi_skip": 1000,
    "model_devi_f_trust_lo": 0.05,
    "model_devi_f_trust_hi": 0.10,
    "model_devi_clean_traj": true,
    "model_devi_merge_traj": true,
    "model_devi_jobs": [
      {"sys_idx": [0,1], "temps":[50, 250, 450, 650], "press": [0, 100, 1000, 5000, 10000, 20000,
      50000], "trj_freq": 10, "nsteps": 3000, "ensemble": "npt-tri", "_idx": "01"},
      {"sys_idx": [0,1], "temps":[50, 250, 450, 650], "press": [0, 100, 1000, 5000, 10000, 20000,
      50000], "trj_freq": 10, "nsteps": 3000, "ensemble": "npt-tri", "_idx": "02"}
     ],
    "fp_style": "abacus",
    "shuffle_poscar": false,
    "fp_task_max": 100,
    "fp_task_min": 5,
    "fp_accurate_threshold": 0.999,
    "fp_accurate_soft_threshold": 0.998,
    "fp_pp_path": "./",
    "fp_pp_files": ["Si_ONCV_PBE-1.0.upf","C_ONCV_PBE-1.0.upf"],
    "fp_orb_files": ["Si_gga_9au_100Ry_3s3p2d.orb","C_gga_9au_100Ry_3s3p2d.orb"],
    "user_fp_params":{
       "ntype": 2,
       "symmetry": 0,
       "basis_type": "lcao",
       "ecutwfc": 100,
       "scf_thr": 1e-6,
       "scf_nmax": 100,
```

```
        "smearing_method": "gauss",
        "smearing_sigma": 0.002,
        "mixing_type": "pulay",
        "mixing_beta": 0.3,
        "cal_force": 1,
        "cal_stress": 1,
        "kspacing": 0.08
    }
}
```

- type_map：用列表方式设置了体系元素，注意顺序要和 STRU 结构文件里一致
- mass_map：记录了对应的原子质量，和 type_map 对应
- init_data_prefix：指定初始训练集的位置所在目录，如：./init_data
- init_data_sys：定位到初始训练集的具体目录，用列表调用多个，即：./init_data/2H
- sys_format：指定为 ABACUS 计算的结构，即：abacus/stru
- sys_configs_prefix：指定探索步所需调用的结构目录，如：./init_config
- sys_configs:指定具体的某个构型的目录,用列表调用多个,即:./init_config/3c/sys-0032-0032/scale-1.000/00***
  其中 00 为通配符，dpgen 会遍历 00 所有文件夹的 STRU 文件
- model_devi_f_trust_lo 和 model_devi_f_trust_hi：可由”init 数据” 中力绝对值的平均值，然后取该值的 20%-50%，设置成 trust level 的上下限。可通过以下 python 脚本计算：（注意指定初始数据位置）

```
import dpdata
import numpy as np
do06=dpdata.Labeledsystem("./", "deepmd/raw")
forces = np.absolute(d006["forces"])
af = np.average(forces)
f_trust_lo = af*0.2
f_trust_hi = af*0.5
print(f_trust_lo,f_trust_hi)
```

如果体系温度升高后力的幅度变大，trust level 也要增大设置

- model_devi_jobs 里面每一行代表一轮，每轮设置参数为 LAMMPS 进行 MD 模拟的参数
- fp_style 指定，使用 ABACUS 进行 SCF 计算
- user_fp_params 里面记录 ABACUS 做 SCF 所需的参数
- 其他参数含义见：https://docs.deepmodeling.com/projects/dpgen/en/latest/run/index.html

1. 准备好所有输入文件之后，运行命令：
   nohup dpgen run param.json machine.json lblog 2xerr&
   为后台提交，等待迭代计算完成即可。
2. 实时监测每轮的 accurate 数据，在 dpgen.log 文件里有记录
   - 1）模型精度是判断每个迭代中的训练是否收敛的重要指标。
   - 2）掌握训练策略有利于快速迭代和收敛。
   - 3）经验性：一般体系在 5-8 轮迭代内会上升到 80% 以上，若没有提升，如下方法排查：
     - a. 检查是否是探索步已经崩溃了，收集了许多不合理的结构
     - b. 检查 train 的 loss 是否是收敛
     - c. dp test 查看 fp 数据是否偏差很大，或者有不合理的情况
     - d. 检查是否是温度设置过高而导致的结构变形严重

- e. 及时调整 trust level，体系温度升高后力的幅度变大，trust level 也要增大设置

3. 一些经验

- **探索步已经崩溃的解决方案**

  原因：DPMD 跑崩的情况一般发生在训练温度高于熔点，而这个温区的结构是从”有序”到”无序态”的，其构型易变，采样空间增大，导致训练难度增加。DPMD 跑崩表明当前势函数对这类构型描述较差，大概率是因为前期数据集中这类构型采集的较少；特别是无序态结构的原子距离变小。

  策略：在熔点附近的参数设置需要平滑缓慢对待，增加”MD 轨迹保存频率”；缩短”MD 模拟步数”或降低”MD 步长”，增加迭代次数，其 DP 训练势函数稳定后再延长 MD 模拟步数。

- **ABACUS 收敛性测试**

  对每种体系一定要先做各个参数的收敛性测试，特别是：K 点、能量截断，也可测试 `scf_thr` 降低计算量。另外，对于原势文件也一定要做性质测试，看看能否能够正确描述目标体系的性质

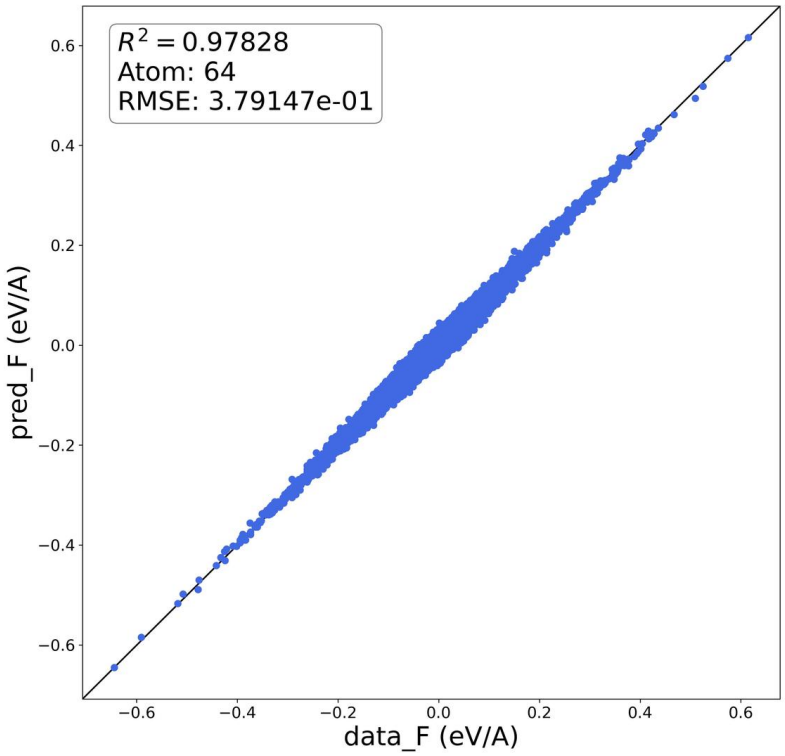
- **每隔几轮做一次 dp test**

  初始数据得到后，第一轮生成了 dp 势之后，一定要检查 train 步骤收敛情况和做 dp test。dp test 用如下命令：

  ```
  dp test -m frozen_model.pb -s ./collect_data -n 2000 -d out.out
  ```

frozen_model.pb 为生成的 4 个模型中的某一个，2000 为 collect_data 里面单个结构的总数。out.out 会生成 out.e.out out.f.out out.f.out 三个偏差数据文件，用于作图判断模型优劣屏幕会输出每个构型力、能量等误差平均值，也可用 *.out 文件做出 DP 和 DFT 偏差图，如：

```
OMP: Info #254: KMP_AFFINITY: pid 22901 tid 24474 thread 80 bound to
DEEPMD INFO    # number of test data : 62
DEEPMD INFO    Energy RMSE        : 6.660438e-01 eV
DEEPMD INFO    Energy RMSE/Natoms : 3.487140e-03 eV
DEEPMD INFO    Force  RMSE        : 1.471114e-01 eV/A
DEEPMD INFO    Virial RMSE        : 1.731723e+01 eV
DEEPMD INFO    Virial RMSE/Natoms : 9.066612e-02 eV
DEEPMD INFO    # ------------------------------------------------
DEEPMD INFO    # ---------------output of dp test---------------
DEEPMD INFO    # testing system : collect_data/init.000
DEEPMD INFO    Adjust batch size from 8192 to 16384
DEEPMD INFO    Adjust batch size from 16384 to 32768
DEEPMD INFO    Adjust batch size from 32768 to 65536
DEEPMD INFO    # number of test data : 302
DEEPMD INFO    Energy RMSE        : 1.120866e-01 eV
DEEPMD INFO    Energy RMSE/Natoms : 5.868411e-04 eV
DEEPMD INFO    Force  RMSE        : 1.311817e-01 eV/A
DEEPMD INFO    Virial RMSE        : 1.724487e+01 eV
DEEPMD INFO    Virial RMSE/Natoms : 9.028727e-02 eV
DEEPMD INFO    # ------------------------------------------------
DEEPMD INFO    # ----------weighted average of errors-----------
DEEPMD INFO    # number of systems : 2
DEEPMD INFO    Energy RMSE/Natoms : 1.535239e-03 eV
DEEPMD INFO    Force  RMSE        : 1.340288e-01 eV/A
DEEPMD INFO    Virial RMSE/Natoms : 9.035191e-02 eV
DEEPMD INFO    # ------------------------------------------------
```




经验性的，力的精度在 1e-2 量级，能量精度在 5e-3 量级，说明模型比较好。

### 5.6.4 计算材料 EOS 和弹性常数的流程

本例子里提供了一个 auto-test 文件夹，进入之后可以结合 DPGEN 的 auto-test 命令计算材料 EOS、弹性常数的流程

auto-test 的详细流程，可以从如下网页教程查阅：

https://bohrium-doc.dp.tech/docs/software/DP-GEN_autotest

这里贴上如何使用 ABACUS 来计算材料 EOS、弹性常数的相关输入文件

relaxation.json：

```
{
    "structures":    ["conf"],
    "interaction": {
        "type":         "abacus",
        "incar":        "./INPUT",
        "potcar_prefix":"./",
        "potcars": {"Si": "Si_ONCV_PBE-1.0.upf","C": "C_ONCV_PBE-1.0.upf"},
                "orb_files": {"Si":"Si_gga_9au_100Ry_3s3p2d.orb","C":"C_gga_9au_100Ry_3s3p2d.orb"}
    },
    "relaxation": {
            "cal_type": "relaxation",
            "cal_setting":{
```

```
            "input_prop": "./INPUT"
            }
     }
}
```

property.json：

```
{
    "structures":    ["conf"],
    "interaction": {
        "type":         "abacus",
        "incar":        "./INPUT",
        "potcar_prefix":"./",
        "potcars": {"Si": "Si_ONCV_PBE-1.0.upf","C": "C_ONCV_PBE-1.0.upf"},
        "orb_files": {"Si":"Si_gga_9au_100Ry_3s3p2d.orb","C":"C_gga_9au_100Ry_3s3p2d.orb"}
    },
    "properties": [
        {
         "type":         "eos",
         "vol_start":    0.9,
         "vol_end":      1.1,
         "vol_step":     0.02
        },
        {
         "type":         "elastic",
         "norm_deform":  1e-2,
         "shear_deform": 1e-2
        }
        ]
}
```

INPUT：

```
INPUT_PARAMETERS
calculation       cell-relax
symmetry          0
basis_type        lcao
ecutwfc           100.0
scf_thr           1.0e-06
scf_nmax          100
smearing_method   gauss
smearing_sigma    0.002
mixing_type       pulay
mixing_beta       0.3
cal_force         1
cal_stress        1
kspacing          0.10
```

在当前目录建立一个 conf 文件夹，把 STRU 文件放进去，依次运行下面命令：

```
dpgen autotest make relaxation.json
nohup dpgen autotest run relaxation.json machine.json \&
dpgen autotest post relaxation.json
dpgen autotest make property.json
nohup dpgen autotest run property.json machine.json \&
dpgen autotest post property.json
```

计算完成之后，会在 conf/elastic_00/result.out 文件里记录弹性常数、模量等信息，conf/eos_00/result.out 里记录 EOS 等数据。

以上即为采用 ABACUS 3.2.0 和 DPGEN 软件结合的介绍，若有任何问题，欢迎给我们反馈。

# 第 6 章 软件接口与后处理

## 6.1 ABACUS+Atomkit 计算态密度和能带

### 6.1.1 介绍

电子的态密度（Density of States，简称 DOS）和电子能带结构（Electronic band structure）在凝聚态物理和材料科学中分析物质的光学、磁学等诸多性质都有重要用途，因此学会计算和分析材料的 DOS 和能带结构也是一项重要技能。

电子能带结构描述了材料中电子能量与动量（或波矢 K）之间的关系。通过电子能带可以揭示材料中电子能级的分布情况，包括能级的高低、能带的宽度、能带之间的间隙（带隙，band gap）等信息。能带结构中的价带和导带之间的能隙大小决定了材料是导体、半导体还是绝缘体。金属没有能隙，半导体有能隙但带隙不大，绝缘体拥有较大的能隙。

DOS 是指在能量为 E 的能量附近可供电子占据的电子状态数目，DOS 是关于能量的函数，通常以每单位能量范围的态数目来表示。DOS 和能带结构紧密相关，通过对能带结构在能量上的积分得到。DOS 的峰值通常对应于能带结构中的范霍夫奇点，这些点是电子浓度变化剧烈的地方，且在费米能级（通常把这个能量设为 0 点，负数代表费米面以下，正数代表费米面以上）附近的行为对材料的电学性质有显著影响。例如，金属的高 DOS 导致高电导率，而半导体和绝缘体在费米能级附近的低 DOS 则导致低电导率。

使用密度泛函理论软件来计算电子态密度和能带一般包括以下两个步骤：第一步，对结构弛豫后的稳定晶体结构（一般来讲是这样操作）做 self-consistent field（简称 scf）自洽计算输出收敛的体系基态电子密度文件；第二步，读入上一步的电子密度，选取想要计算的能级和 K 点，固定电子密度，做一次非自洽（non-self-consistent field，简称 nscf）计算得到态密度以及能带结构。

### 6.1.2 准备

**1. 下载并安装 Atomkit**

在本教程里会用到 atomkit（并非一定要使用，只是会方便计算流程）来处理 ABACUS 的输入文件。首先推荐大家阅读以下文档并安装好 atomkit：

ATOMKIT 0.9.0 新版发布: 新增可视化界面

通过 sourceforge 可以下载到 Atomkit v0.8.0。地址：Download atomkit.0.8.0.linux.x64.tar.gz (vaspkit)

```
wget https://jaist.dl.sourceforge.net/project/vaspkit/Binaries/atomkit.0.8.0.linux.x64.tar.gz
tar -zxvf atomkit.0.8.0.linux.x64.tar.gz
cd atomkit/
bash setup.sh
# modify ~/.atomkit file based on your machine environment
source ~/.bashrc
```

**2. 下载例子**

可以从 Gitee 上下载。可以在网页右侧点击克隆/下载-> 下载 ZIP 得到算例，或者在 linux 终端执行如下命令得到算例：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

下载后解压，之后进入 abacus-user-guide/examples/dos_band 文件夹

里面有 Al 和 Fe 两个文件夹，分别演示铝（nspin=1，适用无磁性体系）和铁（nspin=2，适用有磁性体系）的计算，均使用平面波基组

### 6.1.3 铝的态密度以及电子能带结构计算

**1. 自洽计算**

首先进入 Al 文件夹，铝在常温常压下是面心立方结构（fcc），以下是它经过结构优化（弛豫）后的构型文件：

```
ATOMIC_SPECIES
Al 26.982 Al_ONCV_PBE-1.0.upf upf201

LATTICE_CONSTANT
1.88972612546

LATTICE_VECTORS
    4.0450551637     0.0000000000     0.0000000000 #latvec1
    0.0000000000     4.0450551637     0.0000000000 #latvec2
    0.0000000000     0.0000000000     4.0450551637 #latvec3

ATOMIC_POSITIONS
Direct

Al #label
0 #magnetism
4 #number of atoms
    0.0000000000     0.0000000000     0.0000000000 m  0  0  0
    0.5000000000     0.5000000000     0.0000000000 m  0  0  0
    0.5000000000     0.0000000000     0.5000000000 m  0  0  0
    0.0000000000     0.5000000000     0.5000000000 m  0  0  0
```

这是一个标准的 fcc 单胞结构，但是我们做 DOS 和电子能带结构计算时，常用原胞，因此需要做一步结构转换，转换公式如下图所示：

*A.2. Face-centered cubic (FCC, cF)*

(See Fig. 2 and Table 3.)

Conventional lattice
$\mathbf{a}_1 = (a, 0, 0)$
$\mathbf{a}_2 = (0, a, 0)$
$\mathbf{a}_3 = (0, 0, a)$

Primitive lattice
$\mathbf{a}_1 = (0, a/2, a/2)$
$\mathbf{a}_2 = (a/2, 0, a/2)$
$\mathbf{a}_3 = (a/2, a/2, 0)$

这时也可以使用 Atomkit，执行如下命令，即可得到原胞 PRIMCELL.STRU

```
echo -e "2\n 202\n 101 STRU\n 101" | atomkit
# 这里为了方便演示，使用了命令行，表示将'2'、'202'、'101 STRU' 和'101' 依次传入 atomkit，
实际可以按照 atomkit 的提示依次输入
# 2 代表进行 Symmetry Analysis
# 202 代表进行 Find Primitive Cell
# 101 STRU 代表读取 ABACUS 的 STRU 文件
# 101 代表输出格式为 ABACUS
```

接着执行如下命令进行自洽计算，要注意这里 calculation 为 scf，且需要设置 out_chg 为 1 以输出电荷密度文件（SPIN1_CHG.cube）。此外，ABACUS 默认读取 STRU 文件，这里我们设置 stru_file 为 PRIMCELL.STRU，读取得到的原胞

```
cp INPUT-scf INPUT
cp KPT-scf KPT
mpirun -n 8 abacus
```

**2. 非自洽计算得到态密度**

做态密度计算中的非自洽计算通常情况下需设置更加密的 K 点网格。执行如下命令进行非自洽计算，要注意这里 calculation 为 nscf，且需要设置 init_chg 为 file 以读取电荷密度文件（SPIN1_CHG.cube），out_dos 也需要设置为 1 以输出态密度

电荷密度文件（SPIN1_CHG.cube）默认是放在 OUT.suffix 目录下面，非自洽计算会自动去 OUT.suffix 目录下面找这个文件，如果找不到就会报错。熟悉 VASP 的用户可能习惯将这个文件移到另一个目录，这样也可以，但是需要在 INPUT 里设置 read_file_dir 为 SPIN1_CHG.cube 所在目录，read_file_dir 默认值是 OUT.suffix

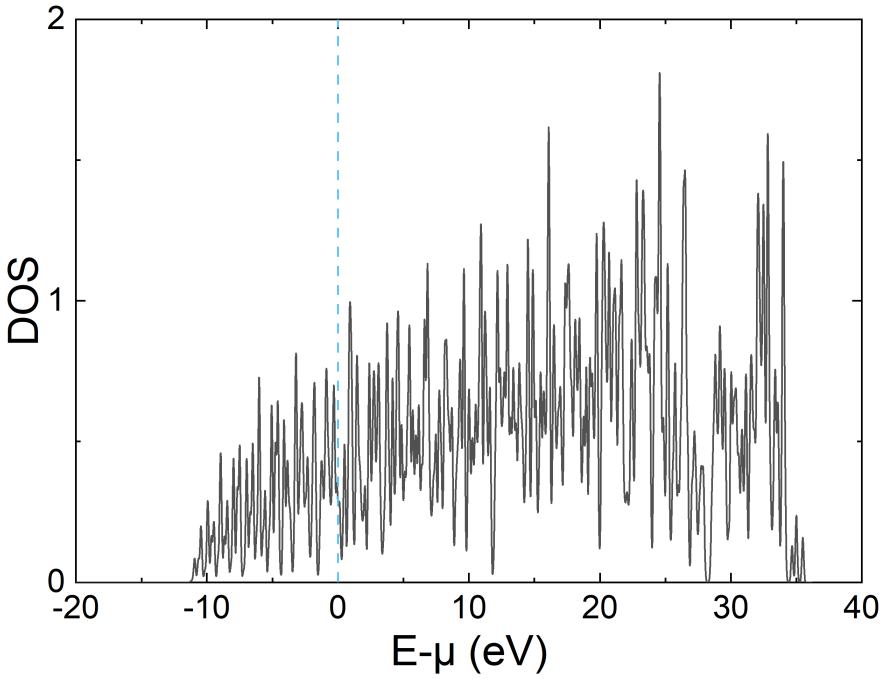
```
cp INPUT-nscf INPUT
cp KPT-nscf KPT
mpirun -n 8 abacus
```

计算得到的态密度存在 DOS1 和 DOS1_smearing.dat 内，我们常用 DOS1_smearing.dat 绘图，第一列是能量（单位为 eV），第二列是态密度，第三列是对态密度的积分

若需要减去费米能，可使用如下命令抓取费米能，然后在绘图时将第一列的数据减去费米能即可

```
grep EFERMI OUT.*/running_scf.log
#  EFERMI = 10.963171515 eV
```

下面是使用 Origin 绘制的态密度图，出现很多毛刺的原因是为了减少计算量 K 点取得较少，可以通过展宽方法做得更平滑（dos_sigma，默认值为 0.07），实际计算也可根据需要增大非自洽计算的 K 点网格




**3. 非自洽计算得到电子能带**

上一步的自洽计算目的是为了得到收敛的电子密度，有了收敛的电子密度，我们就能够快速的得到任意 K 点以及任意多条能带所对应的波函数。根据能带的习惯画法，这一步的 KPT 文件需要选取晶胞的布里渊区高对称点和路径，不同晶胞的高对称点和路径可以参考这个文献 [1]。

这些高对称点能够反映出材料的电子性质，如能隙大小、有效质量、载流子迁移率等。例如，能隙大小直接影响材料是金属、半导体还是绝缘体；有效质量影响载流子的响应速度；而能带的形态和分布则与材料的光学吸收和发射特性密切相关。因此，通过研究布里渊区内的高对称点的能带结构，我们可以预测和解释材料的多种物理性质，这对于材料的设计和应用具有重要意义

对于 fcc 的原胞，它的高对称点和可选的路径如下图 [1] 所示：

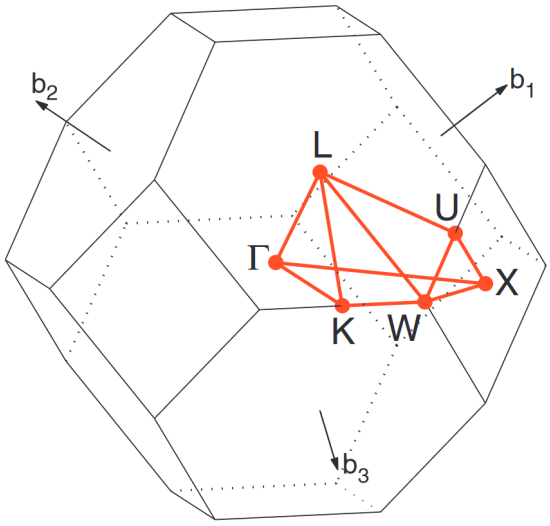



**Fig. 2.** Brillouin zone of FCC lattice. Path: $\Gamma$–$X$–$W$–$K$–$\Gamma$–$L$–$U$–$W$–$L$–$K$|$U$–$X$. An example of band structure using this path is given in Fig. 27.

**Table 3**
Symmetry **k**-points of FCC lattice.

| $\times\mathbf{b}_1$ | $\times\mathbf{b}_2$ | $\times\mathbf{b}_3$ | | $\times\mathbf{b}_1$ | $\times\mathbf{b}_2$ | $\times\mathbf{b}_3$ | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | $\Gamma$ | 5/8 | 1/4 | 5/8 | $U$ |
| 3/8 | 3/8 | 3/4 | $K$ | 1/2 | 1/4 | 3/4 | $W$ |
| 1/2 | 1/2 | 1/2 | $L$ | 1/2 | 0 | 1/2 | $X$ |

KPT 需要选择 Line 模式（Line 模式的介绍可以参考这个链接），并且需要设置 symmetry 为 0（INPUT 文件里若没有设置 symmetry，则会默认设置为 0）

这里可以使用 Atomkit 软件自动生成 KPT，执行如下命令

```
echo -e "3\n 301\n 3\n 101 PRIMCELL.STRU\n 0.06" | atomkit
# 这里为了方便演示，使用了命令行，表示将'3'、'301'、'3'、'101 PRIMCELL.STRU' 和'0.06' 依次传入
atomkit，实际可以按照 atomkit 的提示依次输入
# 3 代表进行 Generate K-Mesh & K-Path
# 301 代表输出 K 点的格式为 ABACUS
# 3 代表 Kpath for Bulk Structure
# 101 PRIMCELL.STRU 代表读取 ABACUS 的 PRIMCELL.STRU 文件
# 0.06 为 kspacing 的取值
```

生成的 KLINES 即是计算所需的 KPT 文件，前三个数是高对称点的分数坐标，第四个数是两个高对称 K 点间采点数，# 号后面是该高对称点的名称

```
K_POINTS
  8
Line
   0.00000000   0.00000000   0.00000000   25     # GAMMA
   0.50000000   0.00000000   0.50000000    9     # X
```

```
  0.62500000   0.25000000   0.62500000    1     # U
  0.37500000   0.37500000   0.75000000   27     # K
  0.00000000   0.00000000   0.00000000   22     # GAMMA
  0.50000000   0.50000000   0.50000000   18     # L
  0.50000000   0.25000000   0.75000000   12     # W
  0.50000000   0.00000000   0.50000000    1     # X
```

接着执行如下命令进行非自洽计算，要注意这里 calculation 为 nscf，同样需要设置 init_chg 为 file 以读取电荷密度文件（SPIN1_CHG.cube），out_band 也需要设置为 1 以输出电子能带

```
cp INPUT-band INPUT
cp KLINES KPT
mpirun -n 8 abacus
```

计算结束之后在 OUT.* 目录下面会有 BANDS_1.dat 文件，其中第一列为 K 点序号，第二列是 K 点在布里渊区里的间隔（以笛卡尔坐标计算），从第三列往后是每条能带的电子能量，单位为 eV

```
1   0.00000000 -92.49968796 -54.00929724 ...
2   0.00988862 -92.49960078 -54.00938081 ...
3   0.01977723 -92.50345008 -54.00963444 ...
......
```

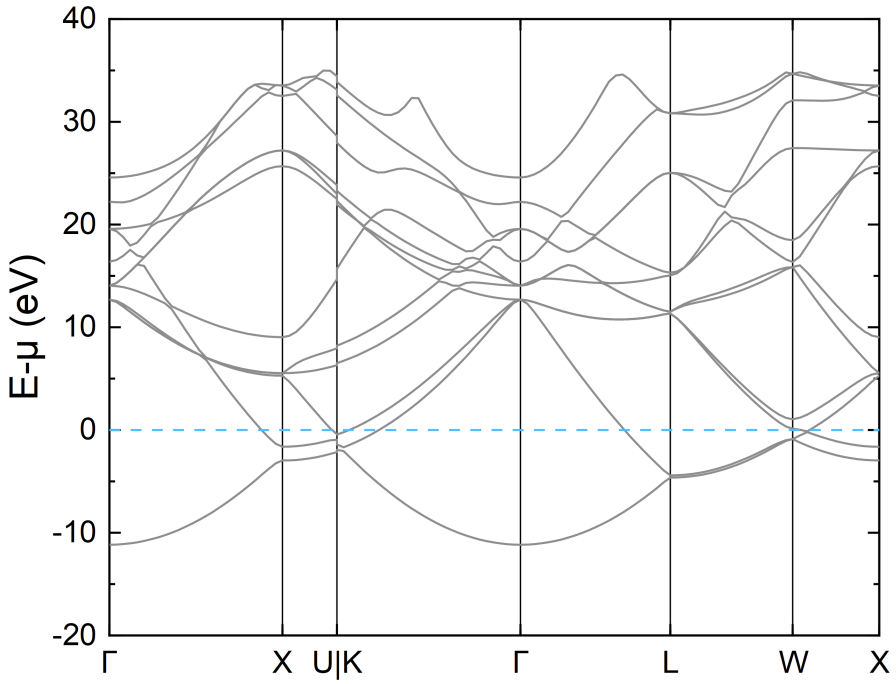
这里，可以使用目录下的 gene_band_dat.py 生成绘图所需的能带文件

```
# 假设当前目录存在 KPT 文件存储能带信息，OUT.* 目录下有 BANDS_*.dat 文件
python gene_band_dat.py 10.963171515
# 10.963171515 即为费米能数值，也可空缺，则不将能量减去费米能
```

运行完之后会在当前目录生成 plot_BANDS_1.dat 文件，即为绘图所需的能带文件，第一行为高对称 K 点的坐标，从第二行开始，第一列为 K 点在布里渊区里的间隔，第二列为每条能带的电子能量，单位为 eV。

```
# 0.0 0.24721543 0.32490774 0.47629565 0.73850721 0.95260205 1.12740976 1.25101747
0.000000 -103.462859
0.009889 -103.462772
......
```

下面是绘制的能带图，图中有不够光滑连线的原因是为了减少计算量，K 点取得较少，实际计算可根据需要增多 K 点间采点数目




### 6.1.4 铁的态密度以及电子能带结构计算

进入 Fe 文件夹，由于铁具有磁性，因此需要在 INPUT 中打开自旋极化选项，即增加参数 nspin 并设置为 2（默认值为 1），并且将铁的初始磁矩设置为非 0（直接设成 0 得不到铁磁基态）。注意此时再对铁做自洽迭代（相比于 nspin 设置为 1 的计算），程序对自旋向上和向下的电子在每一个布里渊区 K 点上都分别进行了 Kohn-Sham 方程的求解，因此计算时间将会几乎翻倍。

计算铁的态密度和电子能带结构的过程与铝几乎相同，但是要注意计算得到的态密度和电子能带结构文件均有两个（分别对应自旋向上和向下），分别为 DOS1 和 DOS2、BANDS_1.dat 和 BANDS_2.dat

绘图仍推荐使用 DOS1_smearing.dat 和 DOS2_smearing.dat，以及 gene_band_dat.py 生成的 plot_BANDS_1.dat 和 plot_BANDS_2.dat，使用绘图软件可得如下 DOS 以及能带图

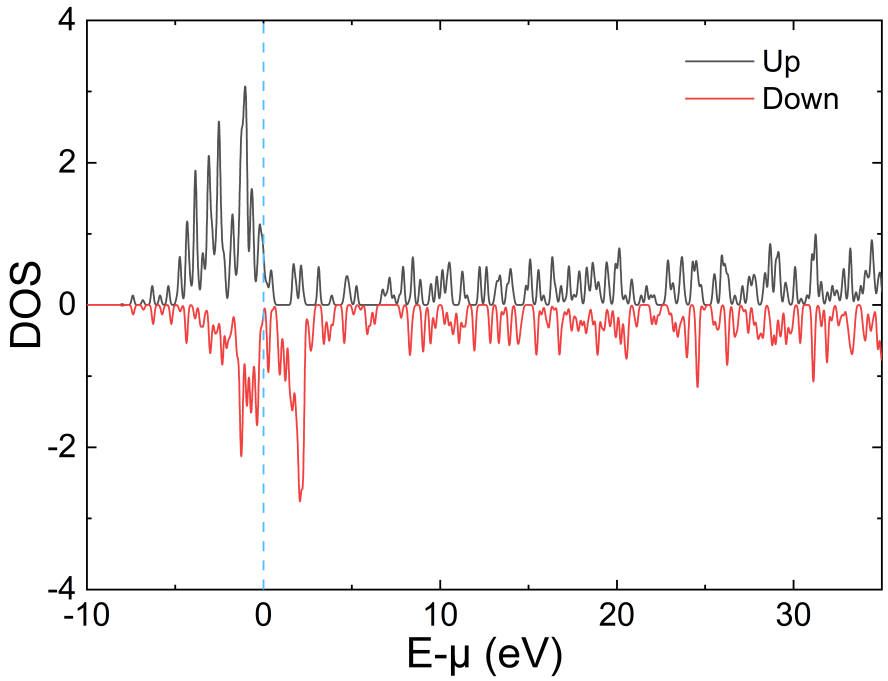



图 1. 铁的态密度（DOS），红线表示自旋向下电子的能带图，黑线表示自旋向上电子的能带图。费米面设为 0 点，即蓝色虚线所在能量位置。可以看出铁的两个自旋方向态密度并不相等，因此铁具有磁性。

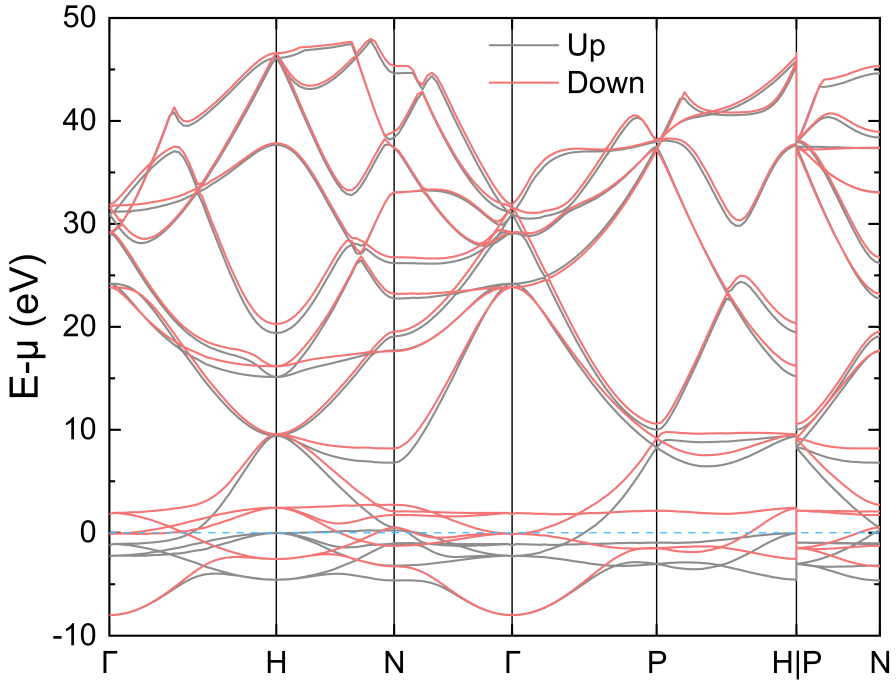



图 2. 铁的能带图，红线表示自旋向下电子的能带图，黑线表示自旋向上电子的能带图。费米面设为 0 点。

### 6.1.5 参考文献

[1] Wahyu Setyawan and Stefano Curtarolo, High-throughput electronic band structure calculations: Challenges and tools, Comp. Mater. Sci., 49, 299-312 (2010).

# 6.2 ABACUS 计算 PDOS

## 6.2.1 PDOS 文件分析

ABACUS 使用 LCAO 基组进行自洽迭代场计算（Self-Consistent Field，SCF）或 NSCF（Non-Self-Consistent Field，NSCF）计算时，可以在 INPUT 文件里设置 out_dos 为 2 在输出 DOS（Density Of States）的同时输出 PDOS（Partial DOS），PDOS 的文件格式是 xml，其文件内容在英文文档里有介绍：Calculating DOS and PDOS —ABACUS documentation

下面简单介绍一下 PDOS 文件：

**能量范围（横坐标）**

第一行 <pdos> 代表输出为 pdos，PDOS 文件最后一行会有 </pdos> 代表 pdos 内容结束

第二行的 2 表示自旋极化计算（npsin=2）

第三行表示该 PDOS 文件一共有 720 个不同的轨道，不同轨道的内容会在后面介绍

第四行 <energy_values units="eV"> 到 </energy_values> 中的内容代表 pdos 的能量范围，即横坐标值

```
<pdos>
<nspin>2</nspin>
<norbitals>720</norbitals>
<energy_values units="eV">
0.00000
0.01000
0.02000
0.03000
0.04000
0.05000
......
24.96000
24.97000
24.98000
24.99000
25.00000
</energy_values>
......
</pdos>
```

**不同轨道的 DOS**

<orbital> 中 index="1" 表示第"1" 条轨道，atom_index="1" 表示第"1" 个原子，species="Li" 表示元素为"Li" l="0"，m="0"，z="1" 分别代表角量子数为"0"，磁量子数为"0"，每个角量子数上对应的径向轨道为"1" 例如 Li 在标准原子轨道库里是 DZP，具体轨道为 4s1p，如下表所示共 7 个：4 个 s 轨道（分别对应 z 取 1 到 4；l 取 0；m 取 0 到 2l，即 0），3 个 p 轨道（对应 z 取 1；l 取 1；m 取 0 到 2l，即 0，1，2），更具体的介绍可以参考数值原子轨道基组的个数

<data> 到 </data> 中的内容代表该轨道的 dos，两列分别对应 spin 为上和下的 dos

| Li | s | | | | p | | |
|---|---|---|---|---|---|---|---|
| l | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| m | 0 | 0 | 0 | 0 | 0 | 1 | 2 |
| z | 1 | 2 | 3 | 4 | 1 | 1 | 1 |

```
<orbital
index="1"
atom_index="1"
species="Li"
l="0"
m="0"
z="1"
>
<data>
0.00000000    0.00000000
0.00000000    0.00000000
0.00000000    0.00000000
0.00000000    0.00000000
0.00000000    0.00000000
......
0.02085550    0.02085546
0.01939430    0.01939436
0.01791958    0.01791972
0.01638296    0.01638315
0.01475548    0.01475571
</data>
</orbital>
```

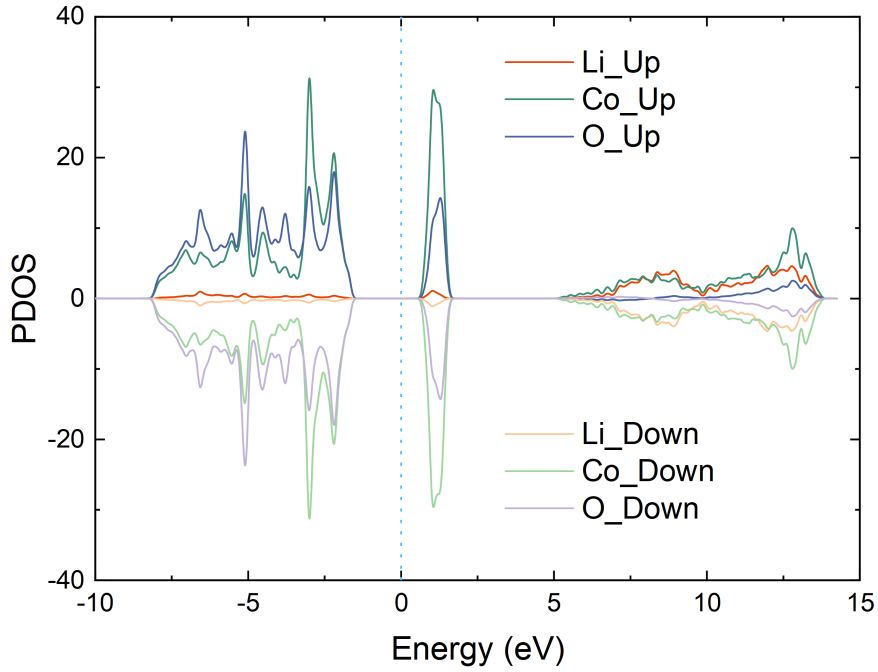
### 6.2.2 绘制 PDOS

有三种方式，分别对应 species, atom_index, index

**species**

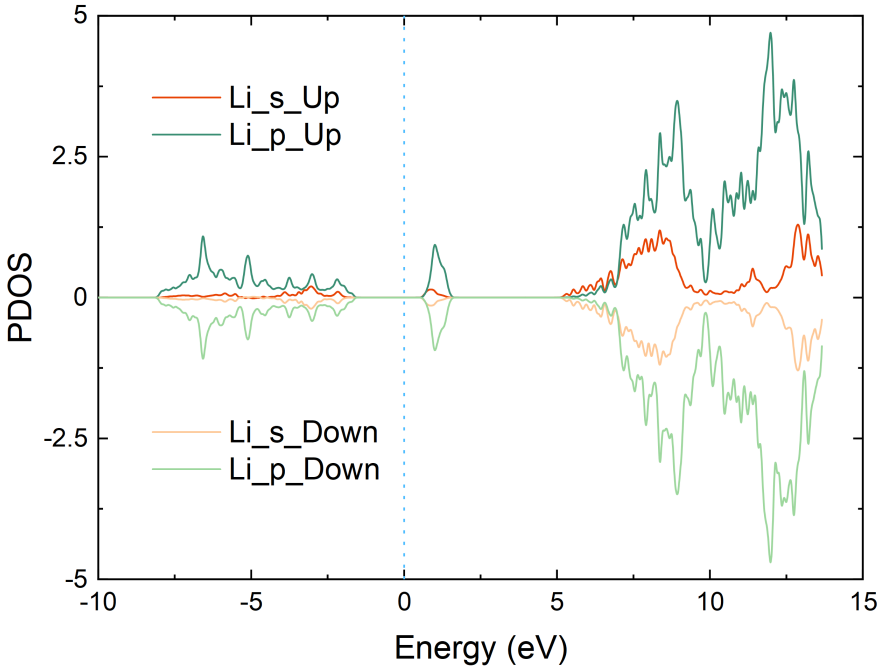
提取指定元素的 PDOS，会在 PDOS_FILE 下生成 species-*.dat 文件，其中 * 代表指定的元素，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
species: [species_1, species_2, species_3, ...]
# eg:
# species : ["Li", "Co", "O"]
```



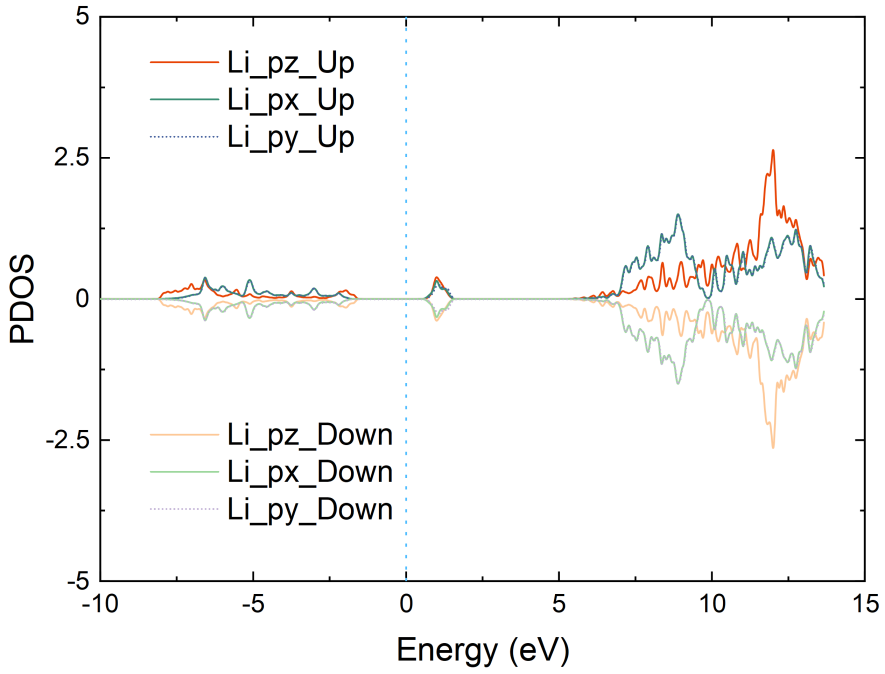

提取指定元素的不同 l（s:0, p:1, d:2, f:3, .....）的 PDOS，会在 PDOS_FILE 下生成 species-* 文件夹，每个文件夹下生成 species-*_l.dat 文件，其中 * 代表指定的元素，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
species : { species_1: [s, p, d], species_2: [s, p], ... }
# eg:
# species : {"Li": [0, 1], "Co": [0, 1, 2, 3], "O": [0, 1, 2]}
```



提取指定元素的不同 l 和 m（m:0, 1, ..., 2l）的 PDOS，会在 PDOS_FILE 下生成 species-* 文件夹，每个文件夹下生成 species-*_l_m.dat 文件，其中 * 代表指定的元素，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
species : { species_1: { s: [m_0], p: [m_0, m_1] }, species_2: { s: [m_0], p: [m_0, m_1], ... }}
# eg:
# species : {"Li": { 0: [0],  1: [0, 1, 2] }, "Co": { 0: [0],  1: [0, 1, 2], 2: [0, 1, 2, 3, 4],
3: [0, 1, 2, 3, 4, 5, 6] }, "O": { 0: [0],  1: [0, 1, 2], 2: [0, 1, 2, 3, 4] }}
```




**atom_index**

提取指定原子的 PDOS，会在 PDOS_FILE 下生成 atom_index-*.dat 文件，其中 * 代表指定的原子，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
atom_index : {atom_index_1, atom_index_2, atom_index_3, ...}
# eg:
# atom_index : {1, 2, 3, 16, 32}
```

提取指定原子的不同 l（s:0, p:1, d:2, f:3, .....）的 PDOS，会在 PDOS_FILE 下生成 atom_index-* 文件夹，每个文件夹下生成 atom_index-*_l.dat 文件，其中 * 代表指定的元素，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
atom_index : { atom_index_1: [s, p, d], atom_index_2: [s, p], ... }
# eg:
# atom_index : {1: [0, 1], 17: [0, 1, 2, 3], 34: [0, 1, 2]}
```

提取指定原子的不同 l 和 m（m:0, 1, ..., 2l）的 PDOS，会在 PDOS_FILE 下生成 atom_index-* 文件夹，每个

文件夹下生成 atom_index-*_l_m.dat 文件，其中 * 代表指定的元素，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
atom_index : { atom_index_1: { s: [m_0], p: [m_0, m_1] },
atom_index_2: { s: [m_0], p: [m_0, m_1], ... }}
# eg:
# atom_index : {1: { 0: [0],  1: [0, 1, 2] }, 17: { 0: [0],  1: [0, 1, 2], 2: [0, 1, 2, 3, 4],
3: [0, 1, 2, 3, 4, 5, 6] }, 34: { 0: [0],  1: [0, 1, 2], 2: [0, 1, 2, 3, 4] }}
```

**index**

提取指定原子轨道对应的 PDOS，会在 PDOS_FILE 下生成 index-*.dat 文件，其中 * 代表指定的原子轨道，第一列为横坐标能量范围，第二列到最后一列分别为不同 spin 的 pdos

```
index : {index_1, index_2, index_3, ...}
# eg:
# index : {1, 2, 3, 4}
```

### 6.2.3 绘制 PDOS 的流程

https://github.com/deepmodeling/abacus-develop/tree/develop/tools/plot-tools
使用 abacus-plot 程序可以绘制 PDOS，具体步骤如下：

**安装 abacus-plot**

```
cd /path_to_abacus/tools/plot-tools
python setup.py install
```

**准备输入文件 config.json**

```
{
    "pdosfile": "PDOS",
    "efermi": 6.585653952007503,
    "species" : ["Li", "Co", "O"]
}
```

pdosfile：实际 PDOS 文件的路径（相对路径和绝对路径均可）
efermi：费米能，单位是 eV，可以在输出文件 running_scf.log 里得到（关键字：EFERMI）
species：对应第二节介绍的三种输出模式参数

**运行 abacus-plot 输出 PDOS**

```
abacus-plot -d -o
```

# 6.3 ABACUS 输出部分的电荷密度和波函数及可视化教程

## 6.3.1 引言

在第一性原理计算中，电荷密度和波函数的计算及其可视化是理解材料性质的关键步骤之一。电荷密度描述了电子在实空间中的分布情况，直接反映了材料中的电子结构和化学键信息。通过电荷密度或者波函数的可视化，我们可以直观地观察到材料内部的电荷转移、价键的形成及其强弱，以及可能存在的空穴或电荷积累区域，这对于分析材料的导电性、磁性、化学反应活性等各种性质至关重要。

ABACUS 作为第一性原理计算软件，支持两种不同的基组：平面波基组（Plane Wave，PW）和数值原子轨道基组（Linear Combination of Atomic Orbitals，LCAO），两种基组下都支持输出实空间下电荷密度和波函数的信息，并且可以方便地使用其他软件/工具包进行可视化。

电荷密度既可以输出总的，也可以按照不同能带、不同 k 点、不同自旋分开输出（能带分解电荷密度）。但是它们的操作逻辑可能会略有不同，例如在 LCAO 下，能带分解电荷密度的输出需要在 SCF 结束后单独再开一次计算，读入上一步 SCF 的结果才可以进行后处理；但是输出总的电荷密度则可以直接在 SCF 时附带目标参数 out_chg=1 即可。

波函数作为复数，既可以输出其模（envelope function），也可以选择实部和虚部分开输出。在 ABACUS 里需要特定的参数组合进行控制，包括 if_separate_k、bands_to_print、out_wfc_norm 等参数。

这篇文档归类汇总了这类相关信息。

## 6.3.2 基本概念与公式

下面以较为复杂的 LCAO 为例，说明一下基本概念与公式。我们知道，LCAO 中的波函数由如下公式描述：

其中 $n$ 是能带指标，$k$ 是 Brillouin 区点的指标。$\phi_\mu(\mathbf{r}-\tau_{ai}-\mathbf{R})$ 是位于布拉维格矢 $\mathbf{R}$ 处的原胞中的种类为 $\alpha$ 的第 $i$ 个原子的原子轨道，复合指标 $\mu=\{\alpha,i,l,m,\zeta\}$ 用于标记原子轨道，$N$ 是 Born-von Karman 周期性边界条件下超胞中所含原胞的个数，$c_{n\mu,k}$ 是原子轨道基组的叠加系数。

那么，电荷（在实空间的）密度就可以计算如下：

其中，$N_k$ 是 Brillouin 区中采样的 $k$ 点个数，$f_{nk}$ 是 Fermi 占据因子。可以看到，在多 $k$ 点的情况下，存在因子 $e^{-ik\cdot R}$，因此多 $k$ 点情况下主要还要对各个 $k$ 点的因子 $e^{-ik\cdot R}$ 进行求和。这样实空间电荷密度 $\rho(\mathbf{r})$ 就和密度矩阵 $\rho_{\mu\nu}(\mathbf{R})$（DMR）联系起来：

$$\rho_{\mu\nu}(\mathbf{R})=\frac{1}{N_k}\sum_{nk}f_{nk}c^*_{n\mu,k}c_{n\nu,k}e^{-ik\cdot R}$$

然而，能带分解电荷密度（band-decomposed charge density，有时又称为 partial charge density）则是不论某能带是否占据，都把该能带单独拿出来，计算相应的电荷密度，并且不同自旋、不同 $k$ 点都要求可以单独计算和绘制，也就是这样一个

$$\rho^\sigma_{nk}(\mathbf{r}):$$

$$\rho^\sigma_{nk}(\mathbf{r})=\psi^{\sigma*}_{nk}(\mathbf{r})\psi^\sigma_{nk}(\mathbf{r})$$

$$=\sum_{\mathbf{R}}\sum_{\mu\nu}\underbrace{c^{\sigma*}_{n\mu,k}c^\sigma_{n\nu,k}}_{\rho^\sigma_{\mu\nu,n}(\mathbf{k})}e^{-ik\cdot R}\phi^*_\mu(\mathbf{r}-\tau_{ai}-\mathbf{R})\phi_\nu(\mathbf{r}-\tau_{\beta j})$$

$$= \sum_{\mathbf{R}} \sum_{\mu\nu} \underbrace{\rho^{\sigma}_{\mu\nu,n}(\mathbf{k}) e^{-ik\cdot R} \phi_{\mu}(\mathbf{r} - \tau_{ai} - \mathbf{R}) \phi_{\nu}(\mathbf{r} - \tau_{\beta j})}_{\rho^{\sigma}_{\mu\nu,n}(\mathbf{R})}$$

$$= \sum_{\mathbf{R}} \sum_{\mu\nu} \rho^{\sigma}_{\mu\nu,nk}(\mathbf{R}) \phi_{\mu}(\mathbf{r} - \tau_{ai} - \mathbf{R}) \phi_{\nu}(\mathbf{r} - \tau_{\beta j})$$

能带分解电荷密度可以用来分析不同轨道（或能量范围）对实空间中特定区域的贡献，有助于深入了解和可视化电子、磁性或输运性质，特别是在模拟扫描隧道显微镜（STM）图像时尤为重要。

### 6.3.3 输入输出概述

ABACUS 软件在平面波基组（Plane Wave，PW）和数值原子轨道基组（Linear Combination of Atomic Orbitals，LCAO）下均支持计算基于能带分解或 k 点分解的电荷密度和波函数。由于这两种基组在程序内部实现逻辑上有所不同，因此输入文件的编写和读取上也略有不同，在下面的部分中将会分开详细介绍。这一部分中，我们以电荷密度的计算为例，对一些共性的内容进行说明。

**输出文件类型：cube 文件**

最终的输出文件类型为.cube，这是一种用于存储三维网格数据的文件格式，主要用于描述电荷密度、波函数等在实空间中的信息。由于该文件格式最初是为 Gaussian 软件包设计的，因此通常也被称为 Gaussian CUBE file，但被其他第一性原理计算软件所广泛使用。cube 文件可以在免费软件 VESTA（Visualization for Electronic and STructural Analysis）中方便地进行可视化，因此后面的可视化步骤也均由 VESTA 进行。

一个典型的 cube 文件通常形如：

```
INPUT_PARAMETERS
nbands                          20
stru_file                       STRU
kpoint_file                     KPT
pseudo_dir                      path/to/your/pseudo_potential/folder

ecutwfc                         100.0   # Rydberg

basis_type                      pw

calculation                     scf
gamma_only                      0
symmetry                        -1      # Turn off symmetry

bands_to_print                  2*1 2*0 1
if_separate_k                   false

nspin                           1
```

cube 文件的前两行不会被 VESTA 读取，因此可以填写任意信息，ABACUS 在第二行输出了自旋数 nspin 和 Fermi 能级 fermi energy，以支持 NSCF（non self-consistent field，非自洽）计算。其他信息的含义如下，供参考：

```
Cubefile created from ABACUS. Inner loop is z, followed by y and x
1 (nspin) -0.094637 (fermi energy, in Ry)
2 (总原子数)  0.0 0.0 0.0 (格点数据原点的 x,y,z 坐标)
45  (FFT nx)   0.103305 0.000000 0.000000 (LATTICE_CONSTANT*a1/nx, 单位 Bohr)
45  (FFT ny)  -0.051653 0.089465 0.000000 (LATTICE_CONSTANT*a2/nx, 单位 Bohr)
180 (FFT nz)   0.000000 0.000000 0.104985 (LATTICE_CONSTANT*a3/nx, 单位 Bohr)
 6 (第 1 个原子的原子序数)  4 (价电子数)  0.000000 0.000000 9.448630
 (该原子位置的笛卡尔坐标, 单位 Bohr)
 6 (第 2 个原子的原子序数)  4 (价电子数) -0.000000 2.683943 9.448630
 (该原子位置的笛卡尔坐标, 单位 Bohr)
 (下面开始是 FFT 格点上的数据, z 坐标变化最快, 接下来依次是 y 和 x, 一般为 6 个数据一行,
 每走完 z 的一次循环换一行)
 7.033e-08 7.920e-08 1.024e-07 1.236e-07 1.439e-07 1.765e-07
 2.130e-07 2.281e-07 2.298e-07 2.592e-07 2.988e-07 2.991e-07
 (下面还有很多其他数据……)
```

因此，对于电荷密度输出的 cube 格式文件，其 FFT 格点上数值的单位为 $1/\text{Bohr}^3$。值得注意的是，电荷密度积分后的电子数目应该是守恒的，即在晶胞内对电子密度积分应等于该能带上占据的电子数目：

$$\int_{cell} \rho^{\sigma}_{nk}(r)\, d^3r = \sum_{N_x,N_y,N_z} \text{data}(N_x,N_y,N_z) \times \Delta V_{\text{FFT grid}} = \text{number of electrons occupying band } n$$

这可以作为检查输出正确性的依据（sanity check）。例如，对于 $nspin = 1$ 的情况，电子成对占据能带 $n$，那么积分后的结果应该是 2；对于 $nspin = 2$ 的情况，自旋向上和向下是分开考虑的，因此积分结果就应该是 1。在 ABACUS 的能带分解电荷密度计算中，对于空带采取的约定是占据数与 HOMO 的占据数一致。

### 输出文件命名规则

对于多能带、多 k 点、多自旋的计算，电荷密度输出文件名形如

`BAND*_K*_SPIN*_CHG.cube`，`*` 是相应的编号。这些编号都是从 1 开始的（如果发现有从 0 或者非 1 开始命名的编号，请在 ABACUS GitHub issue 中向我们反馈）。波函数的命名规则是类似的。

- 能带（BAND）编号与 SCF 计算中生成的 `istate.info` 文件中的约定一致。
- 自旋（SPIN）编号在 `nspin=1` 时只有为 1 的编号；`nspin=2` 时编号为 1 表示自旋向上的分量，编号为 2 表示自旋向下的分量；目前 ABACUS 还不支持对 `nspin=4`（非共线磁矩）的能带分解电荷密度的计算。
- k 点（K）编号顺序与输出的 `kpoints` 文件中的约定一致。

### 其他参数与注意事项

关于多 k 点的计算，可以选择是否启用输入参数 `if_separate_k`。默认情况下，该参数设置为 `false`，此时所有 k 点的贡献都会被计入，从而算得的电荷密度实际上是

$$\rho^{\sigma}_{n}(r) = \sum_{k} \rho^{\sigma}_{nk}(r)$$

输出文件名形如 `BAND*_SPIN*_CHG.cube`。当 `if_separate_k` 设置为 `true` 时，程序会分别计算不同 k 点的贡献并输出，输出文件名形如

`BAND*_K*_SPIN*_CHG.cube`。

有时候对称性分析会约化一些 k 点，并且相应地，k 点权重会发生改变。例如当 `symmetry=1`（开启晶格对称性分析）时，对于一个 $2 \times 2 \times 2$ 的 k 点采样，下面是一个可能的 `kpoints` 文件：

```
nkstot now = 4
K-POINTS DIRECT COORDINATES
 KPOINTS    DIRECT_X    DIRECT_Y    DIRECT_Z  WEIGHT
       1  0.00000000  0.00000000  0.00000000  0.1250
       2  0.50000000  0.50000000  0.00000000  0.3750
       3  0.00000000  0.00000000  0.50000000  0.1250
       4  0.50000000  0.50000000  0.50000000  0.3750

nkstot = 8                                      ibzkpt
K-POINTS REDUCTION ACCORDING TO SYMMETRY
     KPT    DIRECT_X    DIRECT_Y    DIRECT_Z     IBZ    DIRECT_X    DIRECT_Y    DIRECT_Z
       1  0.00000000  0.00000000  0.00000000       1  0.00000000  0.00000000  0.00000000
       2  0.50000000  0.00000000  0.00000000       2  0.50000000  0.50000000  0.00000000
       3  0.00000000  0.50000000  0.00000000       2  0.50000000  0.50000000  0.00000000
       4  0.50000000  0.50000000  0.00000000       2  0.50000000  0.50000000  0.00000000
       5  0.00000000  0.00000000  0.50000000       3  0.00000000  0.00000000  0.50000000
       6  0.50000000  0.00000000  0.50000000       4  0.50000000  0.50000000  0.50000000
       7  0.00000000  0.50000000  0.50000000       4  0.50000000  0.50000000  0.50000000
       8  0.50000000  0.50000000  0.50000000       4  0.50000000  0.50000000  0.50000000
```

可以看到一些 k 点在计算中被约化，从而拥有了更高的 WEIGHT。

虽然我们已经做了很多代码分析、测试与 benchmark，以保证结果的正确性，但是在打开对称性时仍不排除可能会因为 k 点约化与权重改变而产生不可预料的后果（例如电荷密度要根据对称性进行对称化；未来可能还要加入参数来控制纳入哪些用户选择的 k 点的贡献，此时打开对称性后用户操作得到的结果的正确性是不可控的：例如上述情况中纳入编号为 1 和 2 的 k 点，实际上是有 4 个 k 点的贡献，并且电荷密度没有经过对称性操作在实空间还原，因此会得到错误的结果）。因此，如果可能的话，我们强烈建议在完全关闭对称性（symmetry=-1）或者只启用时间反演对称性（symmetry=0）的情况下进行多 k 点 SCF 和能带分解电荷密度的计算。

### 6.3.4 在平面波基组下进行计算

**电荷密度**

平面波基组下，能带分解电荷密度的计算是较为简单的，因为实现逻辑中该功能被归类为 SCF 的附带后处理，因此只需要在 SCF 时附带输入参数 bands_to_print 即可。一个可能的 INPUT 文件如下：

```
INPUT_PARAMETERS
nbands                          20
stru_file                       STRU
kpoint_file                     KPT
pseudo_dir                      path/to/your/pseudo_potential/folder

ecutwfc                         100.0   # Rydberg

basis_type                      pw

calculation                     scf
gamma_only                      0
symmetry                        -1      # Turn off symmetry
```

```
14
15 bands_to_print              2*1 2*0 1
16 if_separate_k               false
17
18 nspin                       1
```

参数 bands_to_print 的具体含义可以在输入参数文档 https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html#bands-to-print 中进行查看。简单来说，上面 bands_to_print 被指定为 2*1 2*0 1，2*1 意味着前 2 个（第 1 个、第 2 个）能带会被计算，2*0 意味着紧接着的 2 个（第 3 个、第 4 个）能带会被跳过，最后一个 1 意味着紧接着的 1 个（第 5 个）能带会被计算；因此，最终会在 OUT.ABACUS 文件夹下得到第 1、2、5 个能带的分解电荷密度。

参数 if_separate_k 则控制是否将每个 k 点对电荷密度的贡献分开输出，false 表示不分开，因此在多 k 点的情形下，输出文件的电荷密度是各个 k 点贡献之和。

如果想要输出包含了所有 k 点、所有占据能带的贡献的电荷密度，可以加上

```
1 out_chg                      1
```

**波函数**

只需在 INPUT 文件中再添加

```
1 out_wfc_norm                   1
```

便可以在 OUT.ABACUS 文件夹下得到文件夹 wfc_realspace，里面存放了不同能带、不同 k 点、不同自旋的波函数的模。

当前版本中，平面波基组下的波函数不支持实部、虚部分开的输出方式。

### 6.3.5 在数值原子轨道基组下进行计算

**电荷密度**

在数值原子轨道基组下，能带分解电荷密度的计算比平面波稍微复杂一些，我们需要两次计算：一次计算先得到 LCAO 的轨道叠加系数文件，一次计算再根据这些系数文件计算得到分解电荷密度。

在 SCF 时计算并输出波函数系数文件

一个可能的 INPUT 文件如下：

```
1  INPUT_PARAMETERS
2  nbands                      20
3  stru_file                   STRU
4  kpoint_file                 KPT
5  pseudo_dir                  path/to/your/pseudo_potential/folder
6  orbital_dir                 path/to/your/orbital/folder
7
8  ecutwfc                     100.0   # Rydberg
9
10 basis_type                  lcao
11
12 calculation                 scf
```

```
gamma_only                  0
symmetry                    -1      # Turn off symmetry

nspin                       1

out_wfc_lcao                1
out_chg                     1
```

这里，重要的是需要设置参数 out_wfc_lcao，此时你将在 OUT.ABACUS 文件夹下得到各个 k 点的波函数系数文件，名称形如 WFC_NAO_K*.txt，这些文件将在下一步计算中被读取，并计算能带分解电荷密度。

这里的 out_chg 控制的是是否输出体系真实的总电荷密度，包含了所有 k 点、所有占据能带的贡献。因此如果你不需要总电荷密度，大可将 out_chg 设为 0。

计算并输出能带分解电荷密度

此时，我们最好再单独开一个文件夹，用来计算能带分解电荷密度。KPT 和 STRU 和第 1 步保持一致，但是 INPUT 文件需要将计算类型 calculation 从 scf 更改为 get_pchg。一个可能的 INPUT 文件如下：

```
INPUT_PARAMETERS
nbands                      20
stru_file                   STRU
kpoint_file                 KPT
pseudo_dir                  path/to/your/pseudo_potential/folder
orbital_dir                 path/to/your/orbital/folder

ecutwfc                     100.0   # Rydberg

basis_type                  lcao

calculation                 get_pchg
gamma_only                  0
symmetry                    -1      # Turn off symmetry
read_file_dir               ./

# nbands_istate               2       # pchg above/below Fermi band
bands_to_print              2*1 2*0 1
if_separate_k               true

nspin                       1
```

我们还要将上一步得到的波函数系数文件复制到该文件夹中，推荐使用 ln -s 命令进行软链接。

其他参数和平面波时基本一致，仍然可以使用 bands_to_print 指定需要计算的能带（见平面波计算部分）。不过在 LCAO 情况下，你也可以选择使用参数 nbands_istate，它会选择计算 Fermi 面上下 nbands_istate 条能带的分解电荷密度。当已经设置了 bands_to_print 的情况下，nbands_istate 的效果将会被覆盖。

这里选择将 if_separate_k 设置为 true，表明将不同 k 点的贡献分开输出。

**波函数**

波函数的计算流程和电荷密度并无二异，同理第一步仍然是 SCF 计算，从而得到各个 k 点的波函数系数文件。

第二步中，我们仅需将之前的计算类型 calculation 参数从 get_pchg 相应地改为 get_wf。这样，你就可以在 OUT.ABACUS 下得到波函数的模文件。如果此时你还打开了 out_wfc_norm=1，还可以在 OUT.ABACUS 下看到文件夹 wfc_realspace，里面存放了（不同能带、不同 k 点、不同自旋的）波函数的实部文件和虚部文件。

### 6.3.6 示例：对电荷密度进行可视化

以单层石墨烯为例，其原胞的 STRU 文件如下：

```
ATOMIC_SPECIES
C   12.011        C_ONCV_PBE-1.0.upf

NUMERICAL_ORBITAL
6_C_TZDP/C_gga_10au_100Ry_3s3p2d.orb

LATTICE_CONSTANT
1.889726

LATTICE_VECTORS
2.4600000000      0.0000000000      0.0000000000
-1.230000000      2.1304220000      0.0000000000
0.0000000000      0.0000000000      10.0000000000

ATOMIC_POSITIONS
Direct

C
0.0000000000
2
0.0000000000 0.0000000000 0.5000000000 0 0 0 mag 0.0
0.3333333333 0.6666666666 0.5000000000 0 0 0 mag 0.0
```

我们选择围绕 Γ 点 2×2×1 的 k 点采样（仅作演示用，并不是对该体系收敛的最优采法），KPT 文件如下：

```
K_POINTS
0
Gamma
2 2 1 0 0 0
```

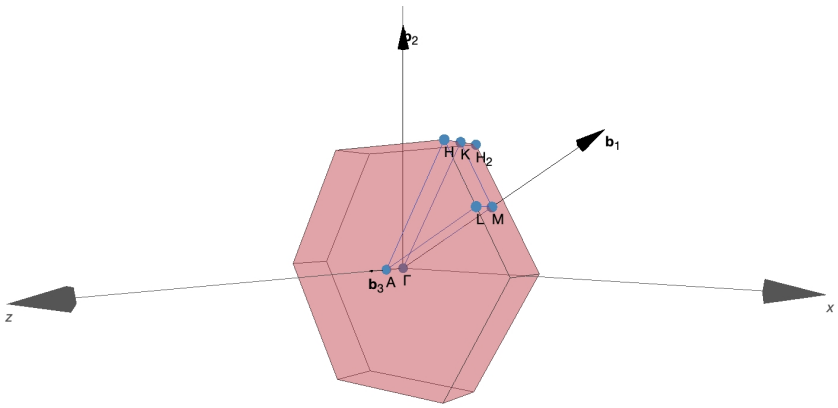
SCF 计算中可以得到输出的 kpoints 文件，指示了我们输出文件中 k 点编号和倒空间（分数）坐标的对应关系：

```
nkstot now = 4
K-POINTS DIRECT COORDINATES
 KPOINTS     DIRECT_X     DIRECT_Y     DIRECT_Z  WEIGHT
       1  0.00000000  0.00000000  0.00000000  0.2500
       2  0.50000000  0.00000000  0.00000000  0.2500
       3  0.00000000  0.50000000  0.00000000  0.2500
       4  0.50000000  0.50000000  0.00000000  0.2500
```

倒空间基矢的选择可以在输出日志 running_scf.log 中看到，形如：

```
Reciprocal vectors: (Cartesian coordinate: in unit of 2 pi/a_0)
            +0.406504           +0.234695                  -0
                   -0           +0.469391                  -0
                   +0                  -0                +0.1
```

采用 SeeK-path 工具中可以看到 Brillouin 区的样子：




这里 SeeK-path 是一个能够自动生成标准化的 k 点路径的工具，方便用于各种电子结构计算，可通过网址 https://www.materialscloud.org/work/tools/seekpath 访问。

不难发现，实际上 2×2×1 的 Brillouin 区采样只能采到 Γ 点和 M 点（编号 2、3、4 的 k 点都等价为 M 点，在 symmetry=1 时会显式约化）。可以预想到，K2、K3、K4 在实空间中的分布形状应该是一致的，它们只相差相应的晶格对称性操作。

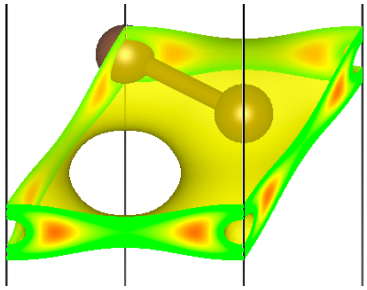
以 LCAO 为例，开启共线磁矩 nspin=2（仅作为演示，没有磁性的石墨烯自旋上下结果应该完全一致），在第一步 SCF 后可以在 OUT.ABACUS 文件夹下得到如下波函数系数文件：WFC_NAO_K1.txt、……、WFC_NAO_K8.txt。这里 K1 到 K8 代表 8 个 k 点，k 点编号比实际设置时多了一倍是因为其中编号 1 ~ 4 是代表自旋向上的波函数文件，而编号 5 ~ 8 是代表自旋向下的波函数文件。

SCF 计算结束后，将得到的这些系数文件复制/链接到另一个我们自己创建的新文件夹下，根据上面的教程设置控制文件，开启计算类型 calculation=get_pchg 的新计算，就可以在 OUT.ABACUS 文件夹下得到如下的电荷密度文件。若选择 if_separate_k=true，输出文件将形如：

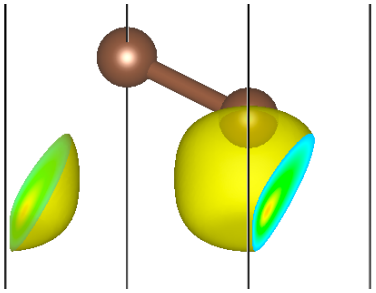
```
BAND1_K1_SPIN1_CHG.cube
BAND1_K1_SPIN2_CHG.cube
BAND1_K2_SPIN1_CHG.cube
BAND1_K2_SPIN2_CHG.cube
......
```

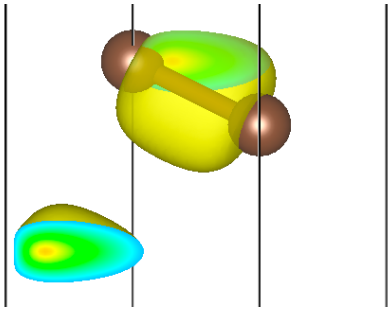
若选择 if_separate_k=false，则输出文件名中不会带有 k 点信息。这些 cube 格式文件可以直接在 VESTA 软件中打开并可视化。

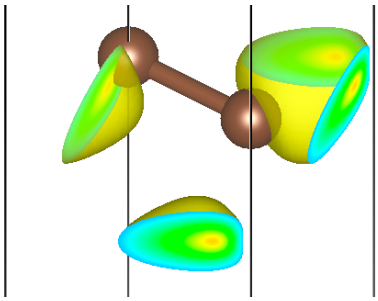
例如，第一条能带 Γ 点自旋向上的贡献存储在 BAND1_K1_SPIN1_CHG.cube 文件里，可视化效果为：



第二条能带三个不同 M 点的贡献为 BAND2_K2_SPIN*_CHG.cube、BAND2_K3_SPIN*_CHG.cube、BAND2_K4_SPIN*_CHG.cube：







可以发现，它们确实形状一致，但是在实空间被旋转了不同的角度，符合预期。

### 6.3.7 结语

以上就是本教程的内容，希望对读者有所帮助。若有问题，请在 ABACUS GitHub issue 中向我们反馈，或者发送邮件联系本教程作者。感谢大家对 ABACUS 软件的支持。

# 6.4 ABACUS 计算电子局域函数 ELF 使用教程

## 6.4.1 背景

电子局域函数（Electron localization function，ELF）最早在 1990 年由 Becke 和 Edgecombe 提出 [1]，它描述电子的局域化程度。根据定义，ELF 是一个 0 到 1 之间的数，越接近 1，表示电子越局域，ELF=1/2 时，对应均匀电子气，越接近 0，表示电子越离域。

目前，ABACUS 支持使用平面波基组、原子轨道基组下的 Kohn-Sham DFT（KSDFT）和无轨道密度泛函理论计算并输出 ELF。

## 6.4.2 定义

**自旋非极化**

自旋非极化时，ELF 定义为

$$ELF = \frac{1}{1+\chi^2}$$

其中 $\chi = \frac{\tau_{KS}-\tau_{vW}}{\tau_{TF}}$，$\tau_{KS} = \frac{1}{2}\sum_i f_i|\nabla\psi_i|^2$ 为 KSDFT 的正定动能密度，

$$\tau_{vW} = \frac{|\nabla\rho|^2}{8\rho}$$

为 von Weizsäcker 泛函的正定形式，$\tau_{TF} = \frac{3}{10}(3\pi^2)^2/3\rho^{5/3}$ 为 Thomas-Fermi 泛函。

**自旋极化**

自旋极化状态下，我们可以针对一种自旋的电子定义 ELF，即

$$\mathrm{ELF}_\sigma = \frac{1}{1+\chi_\sigma^2}$$

其中 $\chi_\sigma = \frac{\tau_\sigma^{KS}-\tau_\sigma^{v}W}{\tau_\sigma^{TF}}$，$\tau_\sigma^{KS} = \frac{1}{2}\sum_i f_i|\nabla\psi_{i,\sigma}|^2$ 为自旋为 $\sigma$ 的电子对应的正定动能密度。对于动能泛函，根据自旋标度率，有 $T_\sigma = \frac{1}{2}T[2\rho_\sigma]$，因此，$\tau_\sigma^{v}W = \frac{|\nabla\rho_\sigma|^2}{8\rho_\sigma}$，$\tau_\sigma^{TF} = \frac{1}{2}\frac{3}{10}(3\pi^2)^{2/3}(2\rho_\sigma)^{5/3} = \frac{3}{10}(6\pi^2)^{2/3}\rho_\sigma^{5/3}$。

注：Becke 和 Edgecombe 对 ELF 的原始定义为

$$\mathrm{ELF} = \left[1+\left(\frac{\tau_\sigma - \frac{(\nabla\rho_\sigma)^2}{4\rho_\sigma}}{\frac{3}{5}(6\pi^2)^{2/3}(\rho_\sigma^{5/3})}\right)^2\right]^{-1}$$

由于这里 $\tau_\sigma = \sum_i f_i|\nabla\psi_{i,\sigma}|^2 = 2\tau_\sigma^{KS}$，因此分子分母同除以 2 后与上述定义相同。

同时，我们也可以定义总的 ELF，即

$$\mathrm{ELF}_{tot} = \frac{1}{1+\chi_{tot}^2}$$

其中 $\chi_{tot} = \frac{\tau_\alpha^{KS}+\tau_\beta^{KS}-\tau_\alpha^{v}W-\tau_\beta^{v}W}{\tau_\alpha^{TF}+\tau_\beta^{TF}}$。

**稳定性修正**

在远离原子核的区域，$\chi$ 的分子和分母均趋于零，但分母更快地趋于零，因此 $\chi$ 趋于无穷，最终 ELF 趋于零。

但在 LCAO 基组下，由于基组的不完备性，在远离原子核的某些区域，$\chi$ 的分子可能更快地趋于零，导致 $\chi$ 趋于零或得到一个有限的值，最终得到非零的 ELF，即电子出现不物理的" 局域化"。在 PW 基组下，则没有上述问题。

为了修正基组带来的不稳定性，我们采用了文献 [2] 中的建议，在 $\chi$ 的分子上增加一个很小的修正值，以保证 $\chi$ 在远离原子核的区域正确地趋于零。经测试后，ABACUS 取此修正值为 $10^{-5}$，即可以提升稳定性，又不会影响近核区域的结果。

### 6.4.3 ABACUS 计算 ELF 具体流程

相关参数：`out_elf [int] [int]`：`out_elf` 的第一个值控制是否输出 ELF，设置为 `1` 则输出，默认值为 `0`；第二个值控制输出的有效数字位数，默认为 `3`。

在 INPUT 中设置 `out_elf 1` 并运行 ABACUS 后，ELF 会被存储到 OUT 文件夹下。

自旋非极化情况下（`nspin 1`）ELF 会被存储到 `ELF.cube` 文件中；自旋极化的情况下（`nspin 2`），除了 `ELF.cube` 中存储的总 ELF，自旋为上和自旋为下的 ELF 会分别存储在 `ELF_SPIN1.cube` 和 `ELF_SPIN2.cube` 中。这些 cube 文件都可以直接在 VESTA 中可视化。

下面举两个具体的例子，一个自旋非极化的水分子，另一个是自旋极化的体心立方铁。

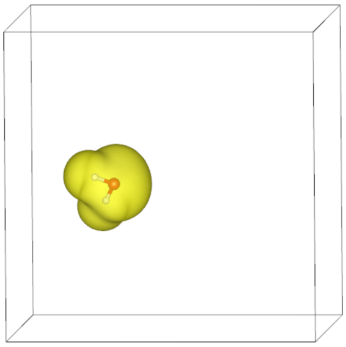
**计算实例：自旋非极化的水分子**

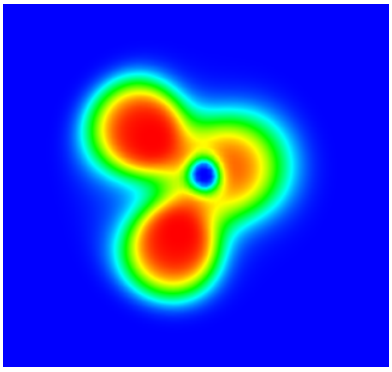
算例地址：https://github.com/MCresearch/abacus-user-guide/tree/master/examples/elf/H20-pw

运行此算例得到的 ELF.cube 文件如下，由于此算例中的水分子放置在真空中，远离水分子的区域内 ELF 都为零。

```
STEP: 0  Cubefile created from ABACUS. Inner loop is z, followed by y and x
1 (nspin)
3 0.0 0.0 0.0
180 0.155556 0.000000 0.000000
180 0.000000 0.155556 0.000000
180 0.000000 0.000000 0.155556
 1 1.000000 11.967714 18.726812 8.322864
 1 1.000000 14.244302 20.657387 7.503683
 8 6.000000 13.550429 19.681072 9.139011
 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
 ......
  0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
```

将此文件在 VEASTA 中可视化，得到下图，其中左图为 ELF 的等高面，右图为在 H-O-H 截面上的切面图。

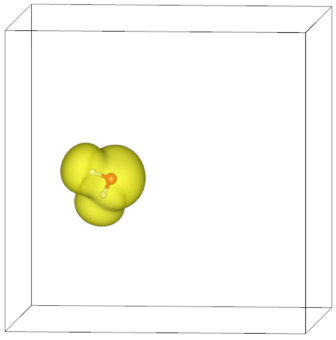


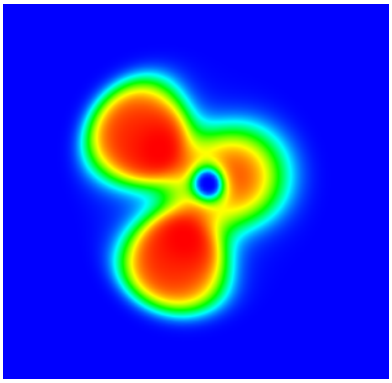


**原子轨道基组**

算例地址：https://github.com/MCresearch/abacus-user-guide/tree/master/examples/elf/H20-lcao

运行此算例后，可得到与上例格式相同的 ELF.cube 文件，用 VESTA 可视化的结果如下图，其中左图为 ELF 的等高面，右图为在 H-O-H 截面上的切面图。



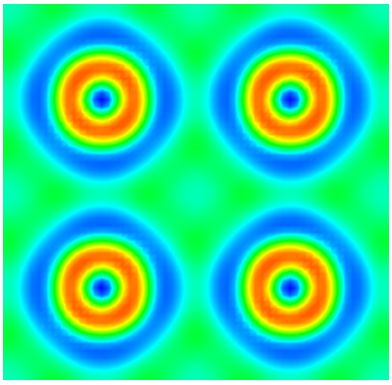


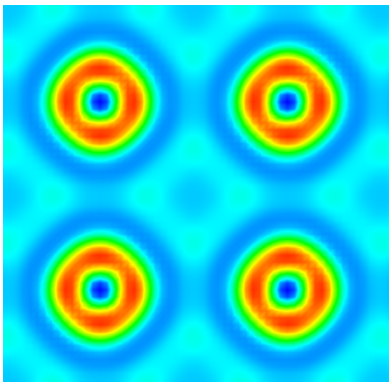
由于使用的基组不同，得到的 ELF 也略有不同。

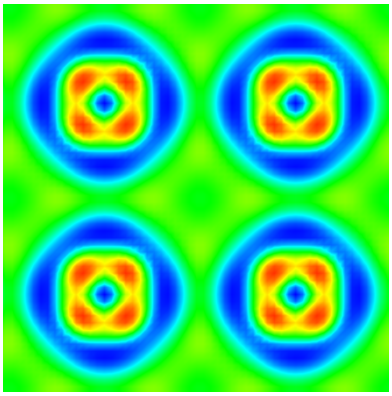
### 6.4.4 计算实例：自旋极化的体心立方铁

算例地址：https://github.com/MCresearch/abacus-user-guide/tree/master/examples/elf/bcc-Fe-pw

此算例使用 PW 基组下的 KSDFT。运行此算例后，可以得到 ELF.cube，ELF_SPIN1.cube 与 ELF_SPIN2.cube 三个文件，分别存储总 ELF，自旋为上电子的 ELF 以及自旋为下电子的 ELF，下面为三种不同的 ELF 在 (100) 晶面上的分布图。







### 6.4.5 参考文献

[1] Becke A D, Edgecombe K E. A simple measure of electron localization in atomic and molecular systems[J]. The Journal of chemical physics, 1990, 92(9): 5397-5403.

[2] 卢天, 陈飞武. 电子定域化函数的含义与函数形式 [J]. 物理化学学报, 2011, 27(12): 2786-2792.

## 6.5 ABACUS+Bader charge 分析教程

### 6.5.1 前言

我们在软件开发时不可避免遇到化学相关应用场景的需求，其中一个可能重点关注的信息便是“价态”。尽管从原理上考虑，“价态”这个量的定义并不明确，但化学场景的使用者总希望找到各式各样的方法来契合他们的“化学直觉”，便诞生了各种布居分析（Population Analysis）方法：如 Mulliken、Hirshfeld、Hirshfeld-Iterative (Hirshfeld-I)、ADCH、DDEC6、CM5、Mayer、Bader 等，以及各种电子的定域化分析方法，如 Wannier、ELF 等。尽管他们各自存在或致命或物理上定义不明的问题，如 Mulliken 电荷具有非常显著的基组相关效应，Hirshfeld 电荷经常对过渡元素计算出携带负电，Bader 对于电荷分布极其不均匀的体系会得出异常结果等，但仍然有足够普遍的受众群体。

Bader charge 是将电子密度进行类似于 Voronoi tesellation 的剖分，在原子间以电荷密度的梯度为 0 处创建截面，然后将原子周围所有截面包络出的 cell 进行实空间积分，即为 Bader charge。考虑实际计算，我们总能对价电子的密度进行切分，但对近核电子的处理则有所差别：CP2K 支持构造出核电荷分布，VASP 支持从 PAW 势中重建核电荷分布，而 QE 和 ABACUS 则不支持以上处理方式。

### 6.5.2 Bader 分析软件部署

网站：https://theory.cm.utexas.edu/henkelman/code/bader/

我们可以直接运行 linux 命令以下载 Bader 电荷分析软件的可执行文件版本，也可以选择下载源码然后手动编译：

```
wget https://theory.cm.utexas.edu/henkelman/code/bader/download/bader_lnx_64.tar.gz
tar -zxvf bader_lnx_64.tar.gz
chmod +x bader
```

下载源码：

```
wget https://theory.cm.utexas.edu/henkelman/code/bader/download/bader.tar.gz
tar -zxvf bader.tar.gz
cd bader
make
chmod +x bader
```

### 6.5.3 使用示例

在了解 bader charge 的原理后，我们首先从 nspin 1（only one spin channel）开始讲解具体使用。

**nspin 1 case**

生成所需 cube 文件

bader 程序需要读取以 cube 方式，或者 CHGCAR 格式存储的实空间电荷密度，因此运行 ABACUS 时，只需要添加

```
out_chg 1
```

即可输出电荷密度。

运行 bader 电荷分析

只需要以

```
/path/to/bader Si2-ELECTRON_DENSITY.cube
```

即可，屏幕会有如下输出：

```
GRID BASED BADER ANALYSIS  (Version 1.05 08/19/23)

 OPEN ... Si2-ELECTRO
 GAUSSIAN-STYLE INPUT FILE
 DENSITY-GRID:   60 x  60 x  60
 CLOSE ... Si2-Si2.cube-ELECTRO
 RUN TIME:    0.02 SECONDS

 CALCULATING BADER CHARGE DISTRIBUTION
               0  10  25  50  75  100
 PERCENT DONE:  **********************

 REFINING AUTOMATICALLY
 ITERATION: 1
 EDGE POINTS:         97653
 REASSIGNED POINTS:    7908

 RUN TIME:       0.23 SECONDS

 CALCULATING MINIMUM DISTANCES TO ATOMS
               0  10  25  50  75  100
 PERCENT DONE:  **********************
```

```
 RUN TIME:    0.02 SECONDS

 WRITING BADER ATOMIC CHARGES TO ACF.dat
 WRITING BADER VOLUME CHARGES TO BCF.dat

 NUMBER OF BADER MAXIMA FOUND:              8
     SIGNIFICANT MAXIMA FOUND:              8
                 VACUUM CHARGE:         0.0000
           NUMBER OF ELECTRONS:        8.00001
```

同时生成了 ACF.dat，AVF.dat 和 BCF.dat 文件，ACF.dat 中即是 bader 切分该电荷密度后在“Voronoi cell”切分的结果：

```
#         X           Y           Z       CHARGE      MIN DIST   ATOMIC VOL
--------------------------------------------------------------------------------
   1    3.653762    2.109500    1.491642    3.999891     1.952476   137.965434
   2    0.000000    0.000000    0.000000    4.000124     1.952474   137.962879
--------------------------------------------------------------------------------
   VACUUM CHARGE:               0.0000
   VACUUM VOLUME:               0.0000
   NUMBER OF ELECTRONS:         8.0000
```

我们将“CHARGE”列数据和赝势中 PP_HEADER 部分的 z_valence 值相减，即可得到该原子所携带电荷量。

**nspin 2 case**

ABACUS 目前分 spin channel 进行电荷密度的输出，若 nspin 2，则会在目录下输出 SPIN1_CHG.cube 和 SPIN2_CHG.cube。然而，直接对每一个 spin channel 进行体积剖分，然后按照原子序号相加的结果可能是错误的，因为两个 spin channel 中不一定每个原子均具有位置相同的“0 梯度面”。因此对同一原子，可能在两个 spin channel 中具有不同大小的 cell。为了避免这种情况，可以首先将两 cube 文件进行加和，之后再调用 bader.x。

Cube manipulator

Cube manipulator 是处理 Gaussian cube 格式 3D 实空间格点数据的小工具，可以在 ABACUS 代码包 tools 文件夹下找到（https://github.com/deepmodeling/abacus-develop/blob/develop/tools/plot-tools/cube_manipulator.py）。

通过 –help 可以查看可用选项：

```
myaccount@mycomputer:~/abacus-develop/examples/spin_polarized/AFM/OUT.ABACUS# python3
cube_manipulator.py --help
usage: cube_manipulator.py [-h] [-i INP] [-o OUT] [-s SCALE] [--p1d P1D] [--s2d S2D] [-p PLUS]
[-m MINUS]

manipulate the Gaussian cube format volumetric data.

options:
  -h, --help            show this help message and exit
  -i INP, --inp INP     the input Gaussian cube file.
  -o OUT, --out OUT     the output file.
  -s SCALE, --scale SCALE
                        scale the Gaussian cube file by a factor.
```

```
  --p1d P1D             integrate the Gaussian cube file in 2D, followed by the axis: 'x', ...
  --s2d S2D             slice the Gaussian cube file along one axis, followed by string like
  'x=0.0', 'y=0.0', 'z=0.0'. Note: should be fractional coodinate.
  -p PLUS, --plus PLUS  plus the two Gaussian cube files.
  -m MINUS, --minus MINUS
                        minus the two Gaussian cube files.

Once meet any problem, please submit an issue at: https://github.com/deepmodeling/
abacus-develop/issues
```

生成所需 cube 文件

我们选择 abacus-develop/examples/spin_polarized/AFM 算例，将 INPUT 文件中 out_chg flag 修改为 1，得到两个 cube 文件分别对应于两个 spin channel。此时我们只需要将两个 SPIN*_CHG.cube 相加即可：

```
python3 cube_manipulator.py -i SPIN1_CHG.cube -p SPIN2_CHG.cube -o ELECTRONIC_DENSITY.cube
```

运行 bader 电荷分析

对文件 ELECTRONIC_DENSITY.cube 使用 bader.x，得到：

```
GRID BASED BADER ANALYSIS  (Version 1.05 08/19/23)

 OPEN ... ELECTRONIC_DENSITY.c
 GAUSSIAN-STYLE INPUT FILE
 DENSITY-GRID:   36 x  36 x  36
 CLOSE ... ELECTRONIC_DENSITY.c
 RUN TIME:    0.00 SECONDS

 CALCULATING BADER CHARGE DISTRIBUTION
               0  10  25  50  75  100
 PERCENT DONE:  **********************

 REFINING AUTOMATICALLY
 ITERATION: 1
 EDGE POINTS:         39985
 REASSIGNED POINTS:    6753

 RUN TIME:       0.07 SECONDS

 CALCULATING MINIMUM DISTANCES TO ATOMS
               0  10  25  50  75  100
 PERCENT DONE:  **********************
 RUN TIME:    0.00 SECONDS

 WRITING BADER ATOMIC CHARGES TO ACF.dat
 WRITING BADER VOLUME CHARGES TO BCF.dat

 NUMBER OF BADER MAXIMA FOUND:            108
     SIGNIFICANT MAXIMA FOUND:            108
```

```
                  VACUUM CHARGE:          0.0000
               NUMBER OF ELECTRONS:       31.99909
```

该例为，呈反铁磁态，每个 Fe 原子有 16 个价电子，最终积分得到电子数量为 31.99909，因此符合预期。同样查看 ACF.dat 文件：

```
#         X           Y           Z       CHARGE      MIN DIST   ATOMIC VOL
--------------------------------------------------------------------------------
   1    0.000000    0.000000    0.000000   15.999544     2.060461    76.704205
   2    2.676622    2.676622    2.676622   15.999544     2.060459    76.704205
--------------------------------------------------------------------------------
   VACUUM CHARGE:               0.0000
   VACUUM VOLUME:               0.0000
   NUMBER OF ELECTRONS:        31.9991
```

发现两 Fe 原子均为电中性，同样符合预期。

### 6.5.4 附录：cube manipulator 更多的使用方法

Cube manipulator 还有除了加法之外的更多功能，例如两 cube 文件进行减法、一个 cube 文件进行乘法等，以及还有面向更实际的应用场景，2D 平面积分与 cube 3D 内容切片。

#### AXPY 类操作

AXPY 是 BIAS 中函数，代表了诸如：

$$y = \alpha x + \beta y$$

的操作。因此 `-p` flag 即为 $\alpha = 1, \theta = 1$, `-m` flag 即为 $\alpha = 1, \theta = -1$, `-s` 为 $\alpha = ?, \theta = 0$。

例如将 `SPIN1_CHG.cube` 和 `SPIN2_CHG.cube` 相减可以获得自旋密度：

```
python3 cube_manipulator.py -i SPIN1_CHG.cube -m SPIN2_CHG.cube -o SPIN_DENSITY.cube
```

#### Profile1d 操作

在进行表面模型的功函数计算时，需要对 Hartree 势进行面积分，得到沿某个轴的 1 维曲线。我们分别使用 ELECTRON_DENSITY.cube 和 SPIN_DENSITY.cube 对该功能进行演示：

```
python3 cube_manipulator.py -i ELECTRON_DENSITY.cube --p1d x -o charge1d.dat
python3 cube_manipulator.py -i SPIN_DENSITY.cube --p1d x -o spin1d.dat
```

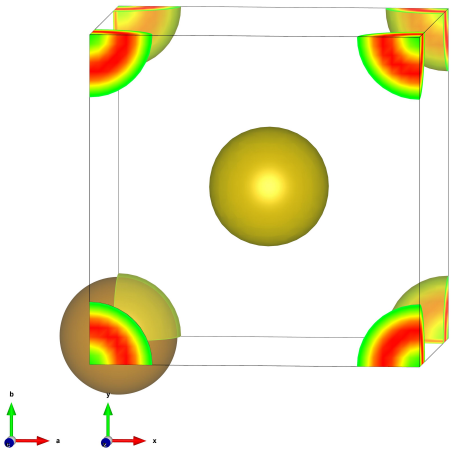


图 1. SPIN1_CHG.cube

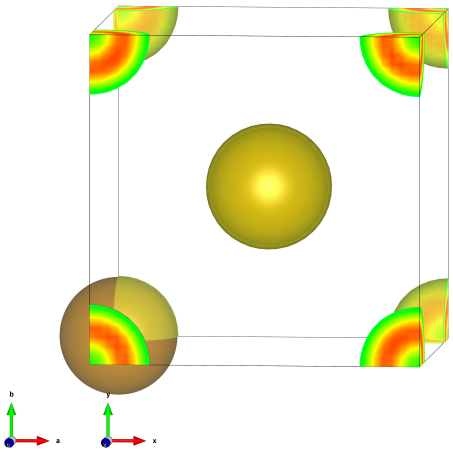


图 2. SPIN2_CHG.cube

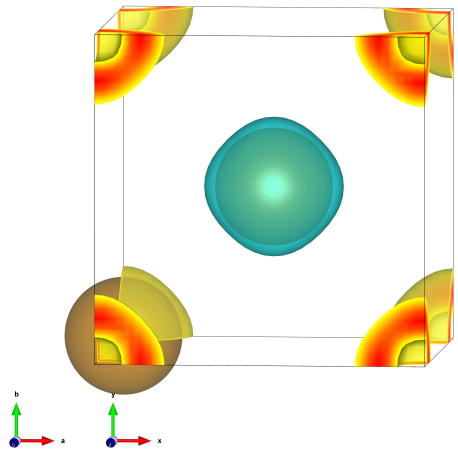


图 3. SPIN_DENSITY.cube

即对 yz 平面进行积分，进行简单的绘图：

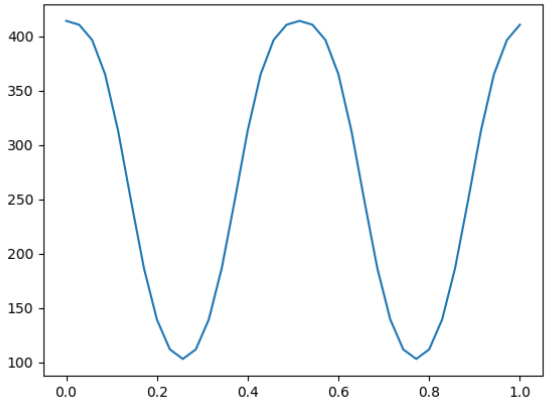



图 4. charge1d.dat，0 和 0.5 位置各有一个 Fe 原子，Bader 电荷切分的地方接近 0.25 和 0.75

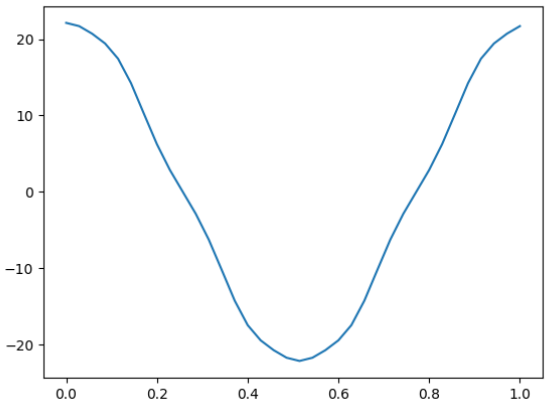



图 5. spin1d.dat，spin density 由正到负，符合反铁磁 Fe2 的预期

**Slice2d 操作**

有时我们会考察某个截面的电子密度，对原子周围电子的局域化程度进行分析。我们仍然分别使用 ELECTRON_DENSITY.cube 和 SPIN_DENSITY.cube 对该功能进行演示：

```
python3 cube_manipulator.py -i ELECTRON_DENSITY.cube --s2d x=0 -o charge2d_000.dat
python3 cube_manipulator.py -i ELECTRON_DENSITY.cube --s2d x=0.25 -o charge2d_025.dat
python3 cube_manipulator.py -i ELECTRON_DENSITY.cube --s2d x=0.5 -o charge2d_050.dat

python3 cube_manipulator.py -i SPIN_DENSITY.cube --s2d x=0 -o spin2d_000.dat
python3 cube_manipulator.py -i SPIN_DENSITY.cube --s2d x=0.25 -o spin2d_025.dat
python3 cube_manipulator.py -i SPIN_DENSITY.cube --s2d x=0.5 -o spin2d_050.dat
```

进行简单绘图，电荷密度：

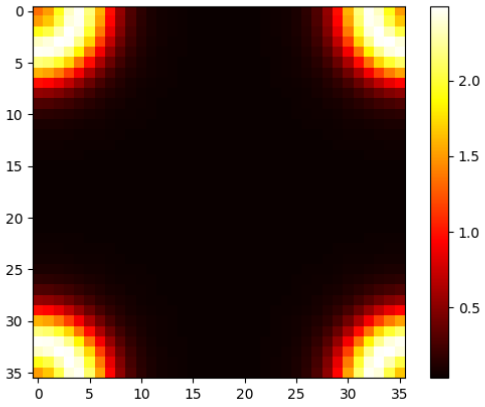



图 6. charge2d_000.dat

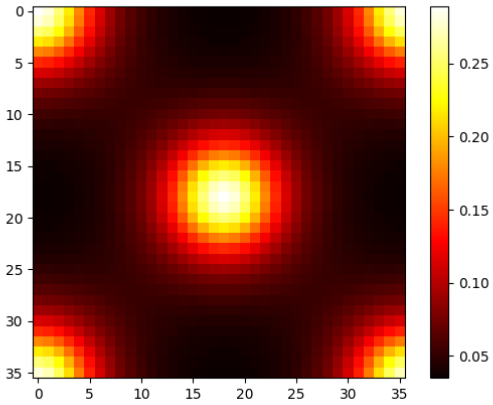



图 7. charge2d_025.dat

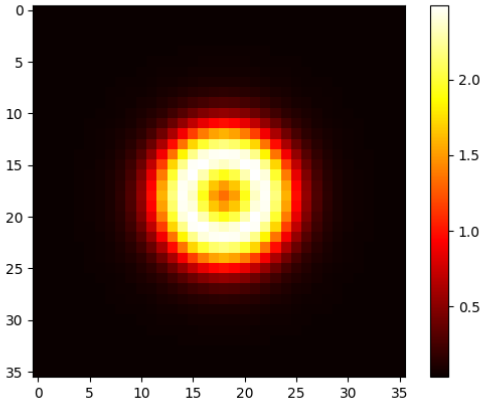



图 8. charge2d_050.dat

自旋密度:

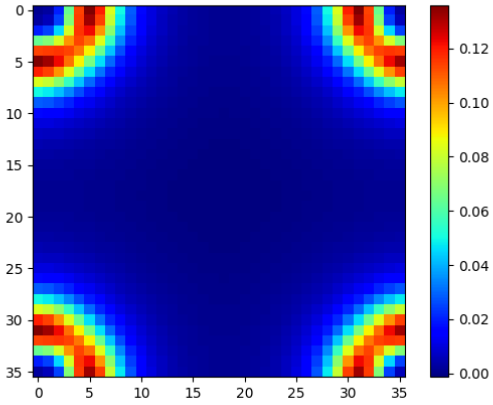



图 9. spin2d_000.dat

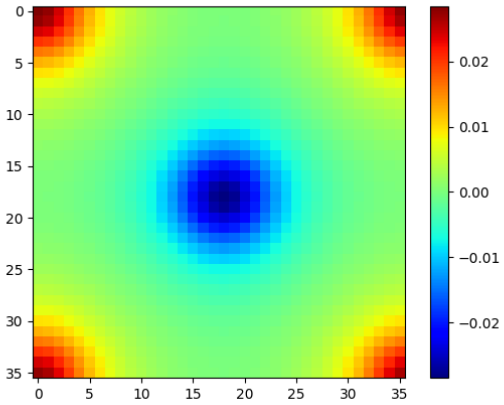



图 10. spin2d_025.dat

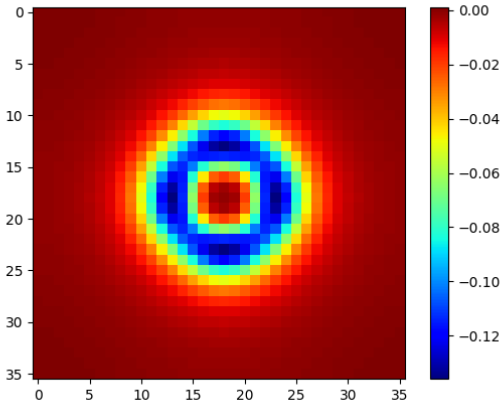



图 11. spin2d_050.dat

### 6.5.5 Have a question? Submit issue!

如果在使用 ABACUS+Bader Analysis 过程中发现了 bug 或者运行结果不达预期，可以在 deepmodeling/abacus-develop 仓库下提交 issue。

## 6.6 ABACUS+pymatgen 计算弹性常数

### 6.6.1 介绍

本教程旨在描述如何采用 ABACUS 来计算材料体系的弹性常数（elastic constants）。弹性常数是表征材料弹性的量，也是材料的重要力学性质，可以通过原子尺度模拟计算获得，例如通过密度泛函理论或者原子间的势函数方法。在晶体结构预测辅助的新材料计算设计中，弹性常数常用于检查结构的稳定性。

在晶体的线弹性范围内，应力 $\sigma$ 和相应应变 $\varepsilon$ 之间的关系符合胡克定律，

$$\sigma = C\varepsilon, \sigma_{ij} = C_{ijkl}\varepsilon_{kl}, \quad i,j,k,(x,y,z), \quad C_{ijkl}$$

由于 $\sigma$ 和 $\varepsilon$ 均是对称张量，基于 Voigt 表示法：

$$xx \mapsto 1, yy \mapsto 2, zz \mapsto 3, yz \mapsto 4, xz \mapsto 5, xy \mapsto 6,$$

$$\sigma_1 = \sigma_{xx}, \sigma_2 = \sigma_{yy}, \sigma_3 = \sigma_{zz}, \sigma_4 = \sigma_{yz}, \sigma_5 = \sigma_{xz}, \sigma_6 = \sigma_{xy}$$

$$\epsilon_1 = \epsilon_{xx}, \epsilon_2 = \epsilon_{yy}, \epsilon_3 = \epsilon_{zz}, \epsilon_4 = \epsilon_{yz}, \epsilon_5 = \epsilon_{xz}, \epsilon_6 = \epsilon_{xy}$$

此时，

$$C_{xxxx} \mapsto C_{11}, C_{xxyy} \mapsto C_{12}, \ldots$$

$$\begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \\ \sigma_4 \\ \sigma_5 \\ \sigma_6 \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} & C_{13} & C_{14} & C_{15} & C_{16} & \epsilon_1 \\ C_{12} & C_{22} & C_{23} & C_{24} & C_{25} & C_{26} & \epsilon_2 \\ C_{13} & C_{23} & C_{33} & C_{34} & C_{35} & C_{36} & \epsilon_3 \\ C_{14} & C_{24} & C_{34} & C_{44} & C_{45} & C_{46} & 2\epsilon_4 \\ C_{15} & C_{25} & C_{35} & C_{45} & C_{55} & C_{56} & 2\epsilon_5 \\ C_{16} & C_{26} & C_{36} & C_{46} & C_{56} & C_{66} & 2\epsilon_6 \end{bmatrix}$$

如果我们通过施加应变 $\varepsilon_i$ 使晶体变形并计算相应的应力，则可以从方程中获得弹性常数。晶体晶胞上的变形矩阵为 $D = I + \varepsilon$，其中 $I$ 是 3*3 的单位矩阵，$\varepsilon$ 是 Voigt 表示法中的应变矩阵。

在 3 维晶体中，应变矩阵为 $\varepsilon = \begin{bmatrix} \varepsilon_1 & \varepsilon_6 & \varepsilon_5 \\ \varepsilon_6 & \varepsilon_2 & \varepsilon_4 \\ \varepsilon_5 & \varepsilon_4 & \varepsilon_3 \end{bmatrix}$，在 2 维平面晶体中，应变矩阵为

$$\varepsilon = \begin{bmatrix} \varepsilon_1 & \varepsilon_6 & 0 \\ \varepsilon_6 & \varepsilon_2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

变形后，晶格矢量为 $A' = A \cdot D$，其中 $A$ 是初始的晶格矢量。

在实际计算中，6 种应变状态将逐一应用于初始驰豫（relaxation）后的结构，因此每次只考虑一种独立变形。对于 6 种应变状态中的每一种，应用 4 种不同的默认应变大小：

$$\varepsilon_i \in \{-0.01, -0.005, 0.005, 0.01\},$$

将产生 24 种构型。接着对这 24 种构型分别使用 DFT 计算（固定晶格矢量，但允许原子驰豫，即 relax），对所获得的每种应变状态的 4 个应力和应变的集合使用线性拟合来计算相应的弹性常数。

更多细节可以参考：Elastic Constants | Materials Project Documentation

### 6.6.2 准备

**下载例子**

首先可以下载一个 ABACUS 的计算实例，可以从 Gitee 上下载。具体来说，可以通过在网页右侧点击克隆/下载-> 下载 ZIP 得到算例，或者在 linux 终端执行如下命令得到算例：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

下载后解压，之后进入 abacus-user-guide/examples/elastic 文件夹

里面有 relax 文件夹，为 8 原子金刚石的算例，周期性边界条件。

此外，gene_dfm.py 和 compute_dfm.py 为计算弹性常数使用的 python 脚本。

run_task.sh 和 sub.sh 为批量运行 abacus 计算的 shell 脚本，INPUT、KPT、C_ONCV_PBE-1.0.upf 和 C_gga_7au_100Ry_2s2p1d.orb 为运行 abacus 计算所需的输入文件。

**下载并安装 pymatgen**

Python Materials Genomics (pymatgen，https://pymatgen.org/) 是一个 API 包，该软件可以与 materials project 结合进行高通量计算。该软件包是由加州大学圣地亚哥雅各布斯工程学院的纳米工程教授 Shyue Ping Ong 和他的材料虚拟实验室 (Materials Virtual Lab) 团队开发并维护的程序。

在本教程里会用到 pymatgen 来计算弹性常数，此外还会用到 dpdata，monty，numpy 这三个库，可以使用如下命令安装：

```
pip install monty numpy dpdata pymatgen
```

弹性常数所用方法相关文档：https://pymatgen.org/pymatgen.analysis.elasticity.html

### 6.6.3 金刚石的弹性常数计算

**结构弛豫**

进入 relax 文件夹，运行 ABACUS 进行结构弛豫，完成后在 OUT.C8 文件夹下出现 STRU_ION_D 文件。之后的应变都将在这个构型文件基础上产生。此外，计算弹性常数所用的应力也会减去这个构型的应力，因此也需要 running_relax.log 输出应力

**产生应变构型**

回到例子根目录，执行如下命令：

```
python gene_dfm.py abacus
```

默认应变的大小（代表晶格常数的倍数，例如 0.01，其形变量是 1

```
norm_strains = [-0.010, -0.005, 0.005, 0.010]
shear_strains = [-0.010, -0.005, 0.005, 0.010]
```

将会产生 task.000 到 task.023 共 24 个文件夹，分别对应第一节所说的 24 种构型。进入任意 task 文件夹，其中会有 INPUT、KPT、STRU 和 strain.json 四个文件。

其中 INPUT 和 KPT 拷贝自例子根目录下的 INPUT 和 KPT，由于使用的赝势和轨道文件放在例子根目录下，所以 pseudo_dir 和 orbital_dir 均设置为../。此外需要进行固定晶格矢量，但允许原子弛豫来计算应力，因此 calculation 设置为 relax。STRU 和 strain.json 分别是生成的构型文件和相应的应变大小。

**计算应力**

分别进入上述 task 文件夹，运行 abacus 计算相应构型的应力。也可以使用例子根目录下的 run_task.sh 脚本，但要注意根据实际修改其和 sub.sh 的内容。

**计算弹性常数**

回到例子根目录，执行如下命令：

```
python compute_dfm.py abacus
```

屏幕输出如下：

```
# Elastic Constants in GPa
1043.31  107.39  107.39    0.00    0.00    0.00
 107.39 1043.31  107.39    0.00    0.00    0.00
 107.39  107.39 1043.31    0.00    0.00    0.00
   0.00   -0.00   -0.00  557.05    0.00    0.00
  -0.00   -0.00   -0.00    0.00  557.05    0.00
   0.00    0.00    0.00    0.00    0.00  557.05
# Bulk   Modulus BV = 419.37 GPa
# Shear  Modulus GV = 521.42 GPa
# Youngs Modulus EV = 1105.91 GPa
# Poission Ratio uV = 0.06
```

第 2 到 7 行即为计算的各种弹性常数（单位：GPa），分别是体弹性模量（Bulk Modulus）、剪切模量（Shear Modulus）、杨氏模量（Youngs Modulus）和泊松比（Poission Ratio）。精度更高的计算结果存储在 elastic.json 内。

以下是常用的几个弹性常数相关的名词解释：

弹性模量（Bulk Modulus）：指当有力施加于物体或物质时，其弹性变形（非永久变形）趋势的数学描述。物体的弹性模量定义为弹性变形区的应力-应变曲线的斜率。

杨氏模量（Young's Modulus）：是材料力学中的名词，一般将杨氏模量习惯称为弹性模量。弹性材料承受正向应力时会产生正向应变，在形变量没有超过对应材料的一定弹性限度时，定义正向应力与正向应变的比值为杨氏模量。杨氏模量的大小标志了材料的刚性，杨氏模量越大，越不容易发生形变。大部分金属在合金成分不同、热处理在加工过程中的应用，其杨氏模量值会有 5%或更大的波动。

剪切模量（Shear Modulus）：又称刚度模量，描述材料在剪切力作用下抵抗形变的能力，即物体形变的大小与作用的剪切应力之间的关系。

泊松比（Poisson's Ratio）：当材料在一个方向被压缩，它会在与该方向垂直的另外两个方向伸长，这就是泊松现象，泊松比是用来反映泊松现象的无量纲的物理量。泊松比一般是正值，表示在一方向拉伸后，在其他方向收缩。不过也存在泊松比为零（在一方向拉伸后，在其他方向的尺寸不变），其至为负的材料。

## 6.7 ABACUS+Phonopy 计算声子谱

### 6.7.1 介绍

本教程旨在介绍采用 ABACUS（基于 ABACUS 3.2.2 版本）做密度泛函理论计算，并且结合 Phonopy 软件计算声子谱的流程。此外，本教程还用到 gnuplot 来绘图。

首先推荐大家阅读以下文档中的详细说明：

ABACUS 官方文档：Phonopy - ABACUS documentation

Phonopy 相关文档：ABACUS & phonopy calculation —Phonopy v.2.19.1

Gnuplot 主页：gnuplot homepage

### 6.7.2 准备

我们以 FCC Al 这个简单例子来演示使用有限位移方法来结合 ABACUS 和 Phonopy 计算声子谱。

**下载并安装 Phonopy**

```
git clone https://github.com/phonopy/phonopy.git
cd phonopy
python3 setup.py install
```

**下载 FCC Al 例子**

可以从 Gitee 上下载。可以在网页右侧点击克隆/下载-> 下载 ZIP 得到算例，或者在 linux 终端执行如下命令得到算例：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

下载后解压，之后进入 abacus-user-guide/examples/interface_Phonopy 文件夹

### 6.7.3 流程

**使用 ABACUS 优化结构**

这里我们已经给出一个已经优化好的 FCC Al 结构
ABACUS 中的结构文件名为 STRU：

```
ATOMIC_SPECIES
Al 26.982 Al_ONCV_PBE-1.0.upf upf201

NUMERICAL_ORBITAL
Al_gga_7au_100Ry_4s4p1d.orb

LATTICE_CONSTANT
1.88972612546

LATTICE_VECTORS
4.03459549706 0 0 #latvec1
0 4.03459549706 0 #latvec2
0 0 4.03459549706 #latvec3

ATOMIC_POSITIONS
Direct

Al #label
0 #magnetism
4 #number of atoms
0  0  0  m  0  0  0
0.5  0.5  0  m  0  0  0
0.5  0  0.5  m  0  0  0
0  0.5  0.5  m  0  0  0
```

**用 Phonopy 产生需要计算的超胞及相应微扰构型**

这里我们使用有限位移方法计算声子谱，因此需要对晶格进行扩胞并对原子位置进行微扰。执行如下命令即可生成 222 的扩胞并产生微扰结构：

```
phonopy -d --dim="2 2 2" --abacus
```

这一步 phonopy 会根据晶格对称性自动产生相应个数的微扰结构。由于 FCC 的晶格对称性较强，因此这个例子只产生一个微扰结构：STRU-001。这里类似 K 点的对称性分析，晶体结构对称性越强，所需的微扰结构就越少，对称性稍差的体系一般会产生多个微扰结构。

经验性设置：1）扩胞越大，计算结果越精确，但是计算量也会上升，一般来说扩胞三个方向的 cell 长度均在 10-20 Å 是比较合适的；2）对于优化后的晶胞（复杂体系），原子位置可能不处于高对称点上，phonopy 可能计算存在一定的误差，可以使用 Matertial Studio 等软件把对称性加回去之后，再做上述步骤，这样能够得到准确的声子谱数据（保证计算出来的声子谱满足体系的对称性特征）。

**产生 FORCE_SET 文件**

接着用 ABACUS 计算原子受力，其中需要注意的是 calculation 需要设置为 scf，并且设置 cal_force 为 1，因为这一步目的是输出原子受力。

小技巧：即为了计算不同的微扰结构的受力，可以在 INPUT 里添加关键字 stru_file 来指定 STRU 文件的路径和文件名：stru_file ./STRU-001

INPUT 内容如下：

```
INPUT_PARAMETERS
#Parameters (1.General)
suffix          Al-fcc
calculation     scf
esolver_type    ksdft
symmetry        1
pseudo_dir      ./psp
orbital_dir     ./psp
cal_stress      1
cal_force       1
stru_file       STRU-001

#Parameters (2.Iteration)
ecutwfc         100
scf_thr         1e-7
scf_nmax        50

#Parameters (3.Basis)
basis_type      lcao
gamma_only      0

#Parameters (4.Smearing)
smearing_method mp
smearing_sigma  0.015

#Parameters (5.Mixing)
```

```
mixing_type     pulay
mixing_beta     0.7
mixing_gg0      1.5
```

算完之后用以下命令产生 FORCE_SET 文件：

```
phonopy -f ./disp-001/OUT*/running_scf.log ./disp-002/OUT*/running_scf.log ...
```

即要指定所有微扰构型算完之后的 running_scf.log 文件位置。如果运行有错，需要首先检查是否所有构型都已正常结束，且其中有力输出（可以找“FORCE”来确认）。

**设置 band.conf 文件计算得到声子谱**

执行如下命令：

```
phonopy -p band.conf --abacus
```

band.conf 内容如下：

```
ATOM_NAME = Al
DIM = 2 2 2
MESH = 8 8 8
PRIMITIVE_AXES = 0 1/2 1/2  1/2 0 1/2  1/2 1/2 0
BAND = 1 1 1  1/2 1/2 1  3/8 3/8 3/4  0 0 0  1/2 1/2 1/2
BAND_POINTS = 101
BAND_CONNECTION = .TRUE.
```

这一步结束之后会有 `band.yaml` 文件输出

以上参数在 Phonopy 的线上文档中均有详细说明，这里再进行简单概述：

- **ATOM_NAME**：指定结构文件中的元素种类。
- **DIM**：扩胞的大小，需要跟 3.2 用 Phonopy 产生需要计算的超胞及相应微扰构型中的”dim” 一致。
- **MESH**：q 点的采样网格。`'888'` 意味着采用 `8x8x8` 的 q 点网格，默认以 `(0, 0, 0)` 为中心。
- **PRIMITIVE_AXES**：输入晶胞到目标原胞的转换矩阵，并将根据原胞基矢量作为声子计算的坐标系。这里是 FCC 的原胞转换矩阵。
- **BAND**：采样能带的 q 点路径。不同晶格的高对称点不同，具体可以使用 Seek-path，自动生成 q 点路径。
- **BAND_POINTS**：给出了包括能带路径末端的采样点的数量。
- **BAND_CONNECTION**：在能带交叉处辅助连接能带。

**绘制声子谱**

本教程使用 gnuplot 绘制声子谱，在 Ubuntu 上 gnuplot 的安装如下：

```
sudo apt-get install gnuplot
```

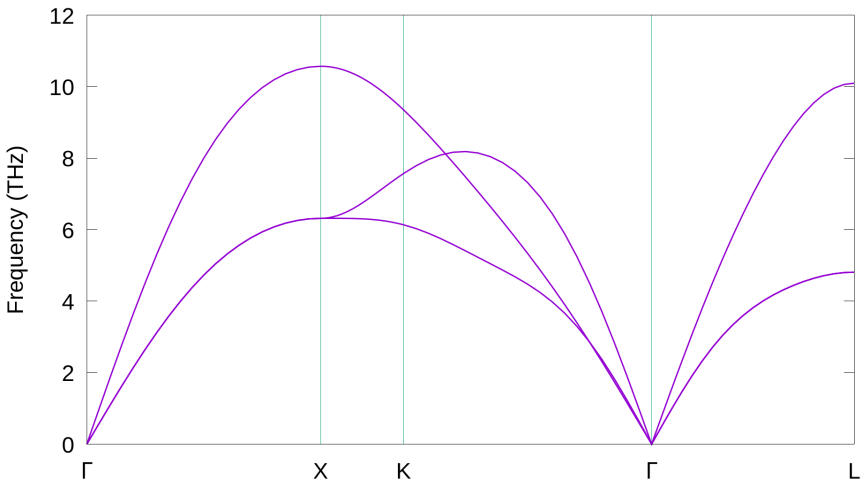
用如下命令输出 gnuplot 格式的声子谱，并使用 gnuplot 绘制声子谱并存为 Al-FCC_plot.png：

```
phonopy-bandplot --gnuplot > pho.dat
gnuplot plot_pho.gp
```

plot_pho.gp 内容如下：

```
set terminal pngcairo size 1920, 1080 font 'Arial, 36'   ## 格式，大小和字体
set output "Al-FCC_plot.png"  ### 输出的文件名

set ylabel 'Frequency (THz)'

set ytics 2
unset key

x1 = 0.13115990
x2 = 0.17753200
x3 = 0.31664810
xmax = 0.43023590
ymin = 0
ymax = 12

set xrange [0:xmax]
set yrange [ymin:ymax]
set xtics ("{/Symbol G}" 0, "X" x1, "K" x2, "{/Symbol G}" x3, "L" xmax)
set arrow 1 nohead from x1,ymin to x1,ymax lt 2
set arrow 2 nohead from x2,ymin to x2,ymax lt 2
set arrow 3 nohead from x3,ymin to x3,ymax lt 2

plot 'pho.dat' using 1:(\$2) w l lw 3
```

FCC Al 的声子谱：




也可使用 Origin 绘图，pho.dat 的第一列就是上图的横轴（K 点路径），其中高对称 K 点位置见 pho.dat 的第二行，第二列就是上图的纵轴（声子频率，单位 THz）。

# 6.8 ABACUS+ShengBTE 计算晶格热导率

## 6.8.1 介绍

本教程旨在介绍采用 ABACUS（基于 ABACUS 3.2.0 版本）做密度泛函理论计算，并且结合 ShengBTE 软件计算晶格的热导率的流程。其中，整个计算过程中还用到了：1）采用 Phonopy 程序来计算二阶力常数，2）采用 ASE 程序进行原子结构的转换，3）采用 ShengBTE 的 thirdorder 程序计算三阶力常数，4）最后使用 ShengBTE 来计算材料的晶格热导率。

上述提到了一些需要结合的外部软件，这里推荐大家阅读这些软件的相关文档和说明：

ShengBTE：https://bitbucket.org/sousaw/shengbte/src/master/

phonopy：http://abacus.deepmodeling.com/en/latest/advanced/interface/phonopy.html

ASE：http://abacus.deepmodeling.com/en/latest/advanced/interface/ase.html

thirdorder: https://bitbucket.org/sousaw/thirdorder/src/master/

## 6.8.2 准备

ABACUS 的软件包中提供了一个 ABACUS+ShengBTE 计算晶格热导率的算例，可以从 Gitee 上下载。算例中包含采用数值原子轨道的 LCAO（Linear Combination of Atomic Orbitals）和采用平面波基矢量的 PW（Plane Wave，平面波）两个文件夹。每个文件夹下分别又包含了 2nd、3rd 和 shengbte 这三个文件夹，分别保存了使用 phonopy 计算二阶力常数（2nd）、thirdorder 计算三阶力常数（3rd）和 ShengBTE 计算晶格热导率（shengbte）的相关文件。

## 6.8.3 流程

以 LCAO 文件夹为例，我们这里提供的测试案例是包含 2 个原子的金刚石结构 Si 结构，采用的模守恒赝势是 Si_ONCV_PBE-1.0.upf，以及原子轨道文件采用的是 Si_gga_7au_100Ry_2s2p1d.orb（GGA 泛函，7 au 截断半径，100 Ry 能量截断，以及包含 2s2p1d 的 DZP 轨道）。

**计算二阶力常数**

要计算二阶力常数，除了 ABACUS 之外，还需要结合 Phonopy 和 ASE。首先，进入 2nd 文件夹。

结构优化

做晶格热导率计算之前要先对模拟的材料体系的进行原子构型的优化。下面是采用 ABACUS 做结构优化（relax）后得到的原子构型文件 STRU。在这个例子里，为了简单起见，结构优化过程采用的是 2*2*2 的布里渊区 k 点采样，平面波的能量截断值 ecut（LCAO 里面也用到了平面波基矢量）为 100 Ry，注意实际计算中应该要采用更收敛的 k 点采样。

```
ATOMIC_SPECIES
Si 28.0855 Si_ONCV_PBE-1.0.upf

NUMERICAL_ORBITAL
Si_gga_7au_100Ry_2s2p1d.orb

LATTICE_CONSTANT
1.88972612546

LATTICE_VECTORS
0 2.81594778072 2.81594778072 #latvec1
```

```
2.81594778072 0 2.81594778072 #latvec2
2.81594778072 2.81594778072 0 #latvec3

ATOMIC_POSITIONS
Direct # direct coordinate

Si #label
0 #magnetism
2 #number of atoms
0.875  0.875  0.875  m  0  0  0
0.125  0.125  0.125  m  0  0  0
```

注意：第一行 Si 的质量 28.0855 在计算中不起作用。

计算二阶力常数

调用 Phonopy 软件产生需要计算的超胞及相应微扰的多个原子构型，命令如下：

```
phonopy setting.conf --abacus -d
```

其中 setting.conf 文件的内容为：

```
DIM = 2 2 2
ATOM_NAME = Si
```

这里我们采用的 Si 的例子只需要产生 1 个微扰构型 STRU-001 即可，对所有微扰构型（这里 Si 的例子只有 1 个）进行 SCF 计算（SCF 代表 Self-consistent field，这里代表进行密度泛函理论的电子迭代自洽计算）获得原子受力，算完之后用以下命令产生 FORCE_SET 文件：

```
phonopy -f OUT.DIA-50/running_scf.log
```

小技巧：在 ABACUS 的输入文件 INPUT 中可以设置变量 stru_file，该变量对应的原子构型文件为 STRU-001 则 ABACUS 可以直接读取该结构文件。

下一步，设置 band.conf 文件计算得到声子谱以及二阶力常数：

```
phonopy -p band.conf --abacus
```

这里出现的 band.conf 文件，其内容如下（具体参数含义可以查看 Phonopy 说明文档）：

```
ATOM_NAME = Si
DIM = 2 2 2
MESH = 8 8 8
PRIMITIVE_AXES = 1 0 0 0 1 0 0 0 1
BAND = 0.0 0.0 0.0  0.5 0.0 0.5  0.625  0.25  0.625, 0.375 0.375 0.75  00 0.0 0.0  0.5 0.5 0.5
BAND_POINTS = 101
BAND_CONNECTION = .TRUE.
FORCE_CONSTANTS = WRITE
FULL_FORCE_CONSTANTS = .TRUE.
```

这一步结束之后，Phonopy 软件会产生 band.yaml（用于绘制声子谱）和 FORCE_CONSTANTS 文件。其

中，FORCE_CONSTANTS 文件包含的数据即为二阶力常数，注意这里务必设置 FULL_FORCE_CONSTANTS = .TRUE.，输出全部的二阶力常数，否则 ShengBTE 读取数据会报错。

此外，可以使用如下命令输出 gnuplot 格式的声子谱，用于绘制声子谱：

```
phonopy-bandplot --gnuplot > pho.dat
```

后处理

注意 ShengBTE 软件要求 `FORCE_CONSTANTS_2ND` 文件里数据的单位为 `eV/A^2`，但是 ABACUS 结合 phonopy 计算的 `FORCE_CONSTANTS` 单位为 `eV/(Axau)`，其中 `au` 是原子单位制，`1 au=0.52918 A`。可以使用 `2nd` 目录下提供的 `au2si.py` 脚本进行单位转换，生成 `FORCE_CONSTANTS_2ND` 文件，命令如下：

```
python au2si.py
```

在 shengbte 文件夹中提供了 FORCE_CONSTANTS_2ND 文件供参考计算结果。

**计算三阶力常数**

要计算三阶力常数，需要结合 thirdorder 程序，计算后输出三阶力常数文件 FORCE_CONSTANTS_3RD。但是，thirdorder 目前只支持读取 VASP 和 QE 的输入输出文件。因此，这里我们是通过将 ABACUS 的结构文件和输出受力分别转换为 POSCAR 和 vasprun.xml 来使用 thirdorder，请先进入 3rd 文件夹，具体步骤将在以下叙述。

获得微扰构型

首先将 ABACUS 软件进行结构优化（relax）后的 STRU 文件转化为 POSCAR（目录下已给出转化过的 POSCAR，或者需要自己动手进行这个转换）。

之后，运行 thirdorder_vasp 程序，产生微扰过后的一系列原子构型文件 3RD.POSCAR.*，例如这个例子一共产生了 40 个构型：

```
thirdorder_vasp.py sow 2 2 2 -2
```

运行 pos2stru.py，将上述 POSCAR 转化为 STRU 文件，注意该脚本里调用了 ASE 软件包的函数（需提前安装好 ASE）：

```
python pos2stru.py
```

注意：这里不能调用 dpdata 软件进行转化。因为 dpdata 会强制将晶格改为下三角矩阵，相当于旋转了晶格，会导致原子间受力方向也相应旋转，从而发生错误。

计算微扰构型的原子受力

可以参考目录下 run_stru.sh 使用脚本批量产生 SCF-* 文件夹并提交计算，这里需要采用 ABACUS 对 40 个原子构型分别进行 SCF 计算，会有些耗时。建议每个 SCF 单独在 SCF-* 文件夹内运行，这里的 INPUT 中的 scf_thr 需要至少小到 1e-8 才能得到收敛的结果。

计算完成后，运行 aba2vasp.py，将 ABACUS 计算的原子受力包装成 vasprun.xml 格式，放置在每个 SCF-* 文件夹中，命令如下：

```
python aba2vasp.py
```

vasprun.xml 格式示意：

```
<modeling>
    <calculation>
        <varray name="forces">
            <v>1.865e-05 -0.04644196 -0.00153852</v>
            <v>-1.77e-05 -0.00037715 -0.00149635</v>
            <v>1.973e-05 0.002213 -0.00149461</v>
            <v>-1.976e-05 0.00065303 -0.0014804</v>
            <v>8.31e-06 -0.0003306 -0.00024288</v>
            <v>-8.25e-06 -0.00038306 -0.00025385</v>
            <v>1.071e-05 0.00060621 -0.00025797</v>
            <v>-1.05e-05 -0.00014553 -0.00027532</v>
            <v>0.00668053 0.00645634 -0.04642593</v>
            <v>-0.00668085 0.00645595 -0.00040122</v>
            <v>-0.00650454 0.00628877 -0.00025123</v>
            <v>0.00650504 0.00628892 -0.00028948</v>
            <v>-0.00039591 2.479e-05 0.00223371</v>
            <v>0.00039608 2.426e-05 0.0006732</v>
            <v>0.0003264 3.122e-05 0.00052874</v>
            <v>-0.00032589 3.415e-05 -0.00023577</v>
            <v>-2.908e-05 -0.00832477 0.00635709</v>
            <v>3.737e-05 -0.00125057 -7.444e-05</v>
            <v>-2.582e-05 0.00656076 0.00636285</v>
            <v>2.566e-05 -0.00049974 -6.661e-05</v>
            <v>-5.431e-05 0.00502637 0.00639077</v>
            <v>4.553e-05 -0.00180978 0.0001325</v>
            <v>-3.609e-05 -0.00676473 0.00638092</v>
            <v>3.806e-05 5.503e-05 0.00012759</v>
            <v>-0.00670704 0.00646596 0.01310437</v>
            <v>0.00670119 3.673e-05 0.00602948</v>
            <v>0.00036366 0.00627899 -0.00657272</v>
            <v>-0.00036508 2.288e-05 0.00026009</v>
            <v>0.00648649 0.0064463 -0.00036521</v>
            <v>-0.00648098 1.594e-05 0.00671469</v>
            <v>-0.00034493 0.00630074 0.00662932</v>
            <v>0.00034331 4.157e-05 -0.0002028</v>
        </varray>
    </calculation>
</modeling>
```

最后执行如下命令：

```
find SCF-* -name vasprun.xml|sort -n|thirdorder_vasp.py reap 2 2 2 -2
```

即可得到三阶力常数文件 FORCE_CONSTANTS_3RD。在 shengbte 文件夹中提供了 FORCE_CONSTANTS_3rd 文件供参考计算结果。

**运行 ShengBTE 得到晶格热导率**

进入 shengbte 文件夹，里面已经准备好 CONTROL（ShengBTE 的参数文件）、FORCE_CONSTANTS_2ND（二阶力常数文件）、FORCE_CONSTANTS_3RD（三阶力常数文件）这三个文件，使用如下命令运行 ShengBTE 即可得到晶格热导率，其中 Ref 文件夹中给出了计算结果供参考：

```
mpirun -n 10 ShengBTE
```

### 6.8.4 结尾

对于 ABACUS 中使用平面波（PW）来做 ShengBTE 的计算也是采用以上类似的流程，但要注意使用平面波时，计算三阶力常数的 INPUT 中 scf_thr 需要至少小到 1e-12。通过计算结果可以发现，PW 和 LCAO 基组计算出的 Si 的晶格热导率是接近的，300 K 下均在 100 W/(m K) 左右，而实验中 Si 在 300 K 的热导率在 150 W/(m K) 附近。这是因为作为教学例子，这里使用的是 2*2*2 的扩胞以及 2*2*2 的 K 点，导致计算结果偏小，实际科研中需要测试扩胞的大小以及 K 点的采样方案来达到收敛的结果。以上就是 ABACUS(3.2.0)+ShengBTE 计算晶格热导率的全部流程，如果有什么问题，欢迎通过邮件联系。

## 6.9 ABACUS+Wannier90 使用教程

### 6.9.1 教程使用说明

Wannier 函数是周期性晶格势场中局域表象下的波函数，Wannier 函数在 1937 年由 Gregory H. Wannier 提出，它可以通过傅立叶变换和布洛赫波函数互相转换。Wannier 函数对于理解半导体、过渡金属、分子晶体等价电子局域程度比较高的凝聚态系统有重要帮助，因而在物理中有广泛的应用。

在 ABACUS 等密度泛函理论（Density Functional Theory，简称 DFT）软件中，单电子近似下的布洛赫波函数可以通过自洽迭代求解 Kohn-Sham 方程得到。但是布洛赫波函数存在规范不确定性，进而导致 Wannier 函数 [1-4] 也不是唯一的。1997 年到 2001 年间，Nicola Marzari、David Vanderbilt 和 Ivo Souza 等人发展了最大局域化的 Wannier 函数（Maximally localized Wannier function, 简称 MLWF）的方法 [1,2,4]，提出可以通过优化 Wannier 表象下的电子位置的方差（即 spread 函数）来确定一个规范，在该规范下 Wannier 函数是最局域的；该方法可以在 DFT 电子结构计算完之后以后处理的方式实现计算 MLWF。2008 年，用于产生 MLWF 的 Wannier90 软件发布，并在随后几年获得了广泛的应用。

本教程分为两部分，第一部分最大局域化 Wannier 函数方法简介介绍最大局域化 Wannier 函数的算法，主要介绍 1997 年和 2001 年的两篇 PRB 文章中提出的优化算法，侧重物理与算法原理；第二部分即本文档，是 Wannier90 软件的 ABACUS 接口说明，主要介绍如何使用 ABACUS 计算 Wannier90 需要的输入文件，侧重具体软件使用。

### 6.9.2 Wannier90 软件介绍

Wannier90 软件 [5-7] 是 Arash A. Mostofi 同 Marzari、Vanderbilt 等人在 2008 年写的做 Wannier 函数最局域化的 Fortran 语言写成的软件，目前是做 Wannier 函数最局域化的主流软件。在 Wannier 函数方面，Wannier90 软件可以输出 Wannier 函数的中心（负电中心），也可以画出 Wannier 函数的等值面图像。除了做 Wannier 函数相关的计算，Wannier90 还具有计算能带与态密度、Berry Phase、输运性质等方面的功能，程序的下载和使用方法可以参考其官网：Wannier90。

### 6.9.3 使用 ABACUS 准备 Wannier90 需要的文件

目前的 ABACUS 平面波基组功能支持 Wannier90 接口，LCAO 的接口还在完善中。

使用 Wannier90 接口不需要编译 ABACUS 过程中做额外操作。

**ABACUS 软件的输入参数解释**

| 参数名 | 默认值 | 变量类型 | 参数解释 |
|---|---|---|---|
| towannier90 | 0 | bool | 是否打开 Wannier90 接口的计算功能 |
| nnkpfile | seedname.nnkp | string | Wannier90 产生的 nnkp 文件的路径，nnkp 文件包含了 k 点邻居关系等计算 mmn 和 amn 文件必要的信息。 |
| wannier_spin | up | string | 输出向上/向下的波函数及其内积 |
| out_wannier_mmn | 1 | bool | 是否输出 mmn 文件，mmn 文件存储了 $M_{mn}^{(}mathbf(k,b))$，即布洛赫波函数之间的内积。mmn 文件在计算 Wannier 最后域化时必须输出。 |
| out_wannier_amm | 1 | bool | 是否输出 amn 文件，amn 文件存储了 $A_{mn}$，即初猜的 Wannier 函数和布洛赫波函数之间的内积。在 Wannier 最后域化计算中最好有该文件（可以加快优化迭代的收敛），也可以没有。 |
| out_wannier_eig | 1 | bool | 是否输出 eig 文件，eig 文件存储了各个 k 点、各个能带的本征能量。在计算输出能带时需要输出，如果不需要输出能带则不需要输出该文件。 |
| out_wannier_unk | 1 | bool | 是否输出 UNK 文件，UNK 文件存储了布洛赫波函数 $u_nlmathbf(k)$。计算 Wannier 函数最后域化，输出 Wannier 函数的中心和 spread 不需要波函数信息。只有画 Wannier 函数图像时需要 UNK 文件。 |
| out_wannier_wvfn_formatted | 1 | bool | 输出的 UNK 文件是否是文本文件。非文本文件即二进制文件，输出速度会比文本文件快很多。 |

在运行 ABACUS+wannier90 程序前，需要准备自洽计算（SCF）的输入文件、非自洽计算（NSCF）的输入文件和 Wannier90 的输入文件，ABACUS 的输入文件有如下注意事项：

1. 自洽计算：需要注意 INPUT 中一定要指定 out_chg 参数为 1，即输出电荷密度，以供给非自洽计算计算使用；

2. 非自洽计算：需要注意 INPUT 中需要把 init_charge 参数设置为 file，即读取文件中的电荷密度；一定要指定 towannier90 参数为 1，同时指定 nnkpfile 参数为 Wannier90 产生的 nnkp 文件的地址；另外需要注意 KPT 文件中的布里渊区 k 点要和 Wannier90 输入文件中的 k 点一模一样，所以最好手动输入 K 点坐标指定；

3. 非自洽计算：INPUT 中有 out_wannier_mmn、out_wannier_amn、out_wannier_eig、out_wannier_unk、out_wannier_wvfn_formatted 五个控制 Wannier90 相关文件输出的参数，默认值都是 1，其中 out_wannier_eig 和 out_wannier_mmn 是必须打开的，out_wannier_unk 和 out_wannier_amn 在需要使用 Wannier90 绘制波函数图时需要打开，out_wannier_wvfn_formatted 控制输出的 UNK 文件是文本文件还是二进制文件，输出文本文件的时间成本和存储空间都会比较大，输出二进制文件的时间开销和存储开销则会相对较小。

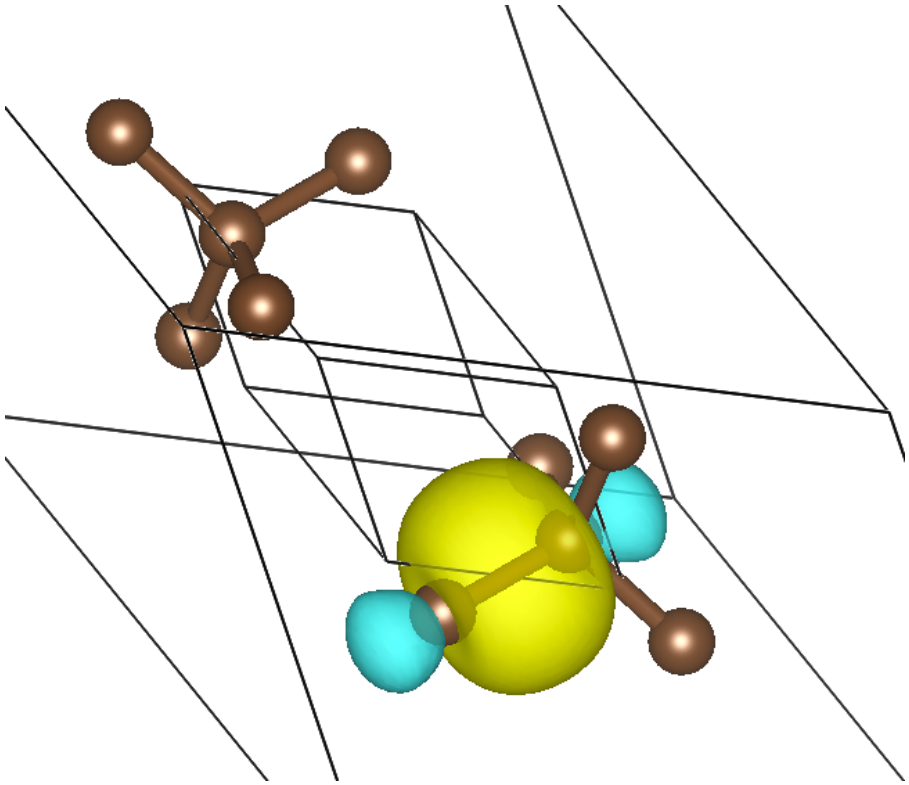
**运行流程**

这里给出了运行 ABACUS+wannier90 的一个脚本文件，结合 examples/interface_wannier90/ 中的例子讲解结合 ABACUS 和 Wannier90 计算最局域化 Wannier 函数的完整运行流程，这里的流程对平面波基组和原子轨道基组均适用，原子轨道基组的 NSCF 会比平面波基组慢一些。

```
export W90=wannier90.x
export ABACUS=abacus
export NP=12

# 1. 运行 Wannier90 软件，生成 nnkp 文件；
cd wannier
```

```
mpirun -np $NP $W90 -pp diamond.win

# 2. 运行 ABACUS 软件，执行电子结构的自洽计算；
cd ../ABACUS
cp INPUT-scf INPUT
cp KPT-scf KPT
mpirun -np $NP $ABACUS >> scf.out

# 3. 将 nnkp 文件放在 ABACUS 的非自洽计算的 INPUT 文件中指定的位置，运行 ABACUS 软件，执行非自洽计算；
cp ../wannier/diamond.nnkp .
cp INPUT-nscf INPUT
cp KPT-nscf KPT
mpirun -np $NP $ABACUS >> nscf.out

# 4. 将非自洽计算生成的 Wannier90 所需要的文件（mmn、amn、eig、UNK）放在 Wannier90 的运行路径下，运行 Wannier90 软
cd ../wannier
cp ../ABACUS/OUT.ABACUS/diamond.amn ../ABACUS/OUT.ABACUS/diamond.mmn ../ABACUS/OUT.ABACUS/diamond.eig ../ABACUS/
mpirun -np $NP $W90 diamond.win
```

在这个例子中，Wannier90 的输入文件 diamond.win 中指定要求画出 Wannier 函数的图像，运行结束之后会得到 diamond_0000[1-4].xsf 四个文件，存有 Wannier 函数的实空间格点数值，放在 VESTA 中可以画出如下图的 Wannier 函数的等值面。



**运行时间参考**

由于 Wannier90 接口只涉及上述步骤中的 NSCF 计算，所以这里只提供 NSCF 部分的时间统计。

在 12 个 CPU（型号：Intel(R) Xeon(R) Bronze 3104 CPU @ 1.70GHz）上并行运行样例，UNK 文件采用二进制格式输出，平面波基组耗时 4 秒，原子轨道基组耗时 98 秒。

**样例文件**

下载地址为（国内 gitee）：
https://gitee.com/mcresearch/abacus-user-guide/tree/master/examples/interface_Wannier90

### 6.9.4 参考文献

Nicola Marzari, Arash A. Mostofi, Jonathan R. Yates, Ivo Souza, and David Vanderbilt, Maximally localized Wannier functions: Theory and applications, Rev. Mod. Phys. 84, 1419 (2012). （关于最大局域化 Wannier 函数的综述文献）

Nicola Marzari and David Vanderbilt, Maximally localized generalized Wannier functions for composite energy bands, Phys. Rev. B 56, 12847 (1997). （提出最大局域化 Wannier 函数方法的文献）

Blount, E. I., Solid State Phys. 13, 305 (1962). （关于 Bloch 状态的诸多基础性质的讨论，最大局域化中 $\langle n\mathbf{R}|\mathbf{r}|m\mathbf{0}\rangle$ 在布洛赫表象下的表达式来源于此）

Ivo Souza, Nicola Marzari, and David Vanderbilt, Maximally localized generalized Wannier functions for entangled energy bands, Phys. Rev. B 65, 035109 (2001). （从纠缠能带中解耦出部分能带，做最大局域化 Wannier 函数的文献）

Arash A. Mostofi, Jonathan R. Yates, Young-Su Lee, Ivo Souza, David Vanderbilt, Nicola Marzari, wannier90: A tool for obtaining maximally-localised Wannier functions, Comput. Phys. Commun., 178, 9, 685 (2008). (发布 Wannier90 软件的文献)

Giovanni Pizzi et al. Wannier90 as a community code: new features and applications. J. Phys.: Condens. Matter, 32, 165902 (2020). (Wannier90 软件近期的更新)

ABACUS 的 Wannier90 接口的英文文档：Wannier90 ABACUS documentation

## 6.10 ABACUS+Candela 使用教程

### 6.10.1 介绍

本教程旨在介绍采用 Candela 分析 ABACUS（基于 3.2.4 版本）分子动力学轨迹，计算径向分布函数（Radial Distribution Function，简称 RDF），静态结构因子（Static Structure Factor，简称 SSF），离子-离子动态结构因子（Ion-ion Dynamic Structure Factor，简称 DSF）以及均方差位移（Mean Square Displacement，简称 MSD）流程。

上述性质的具体描述可以参考：Qianrui Liu et al 2020 J. Phys.: Condens. Matter 32 144002，径向分布函数（RDF）见文中公式（12），静态结构因子（SSF）见公式（13），离子-离子动态结构因子（DSF）见公式（15，16，17）以及均方差位移（MSD）见公式（18）。

Candela 全称 Collection of ANalysis DEsigned for Large-scale Atomic simulations，目前支持分析 QE、ABACUS、LAMMPS 和 VASP 的分子动力学轨迹，Github 主页：https://github.com/MCresearch/Candela

### 6.10.2 准备

**下载并安装 Candela**

```
git clone https://github.com/MCresearch/Candela.git
cd Candela
make -j4 # 使用 intel oneapi 编译器
```

若使用 Intel Oneapi 编译器，按照上述流程操作，在编译完成后即可在 bin 目录下看到 candela 可执行文件
若使用其他编译器，需要按照 Candela 目录下 Makefile.vars 修改相应的 CXX

**下载例子**

可以从 Gitee 上下载。在网页右侧点击克隆/下载-> 下载 ZIP 得到算例，或者在 linux 终端执行如下命令得到算例：

```
git clone https://gitee.com/mcresearch/abacus-user-guide.git
```

下载完成后解压，之后进入 abacus-user-guide/examples/candela 文件夹
算例中包含 RDF、SSF、DSF、MSD 以及 MSD_ntype_2 五个文件夹和 MD_dump 文件。其中 MD_dump 是 32 原子的铝的 MD 轨迹

### 6.10.3 流程

**计算径向分布函数**

进入 RDF 文件夹，INPUT 文件即为 Candela 的输入文件，具体如下：

```
calculation  pdf
geo_in_type  ABACUS
geo_directory  ../MD_dump
geo_1        0
geo_2        100
geo_interval 2
geo_ignore   50
geo_out      Al_rdf.txt

ntype        1
natom        32
rcut         2.3
dr           0.01
```

以上参数在 Candela 的线上文档中均有详细说明，这里再进行简单概述：

- **calculation**: 设置为 `pdf` (Pair Distribution Function) 即计算径向分布函数
- **geo_in_type**: 读取的 MD 轨迹的格式，目前支持分析 `pw.x` (QE2)，`cp.x` (QE)，`ABACUS` (ABACUS)、`LAMMPS` (LAMMPS) 和 `VASP` (VASP)
- **geo_directory**: MD 轨迹的位置
- **geo_1**: MD 轨迹起始的索引
- **geo_2**: MD 轨迹结束的索引
- **geo_interval**: Candela 读取 MD 轨迹的间隔
- **geo_ignore**: 需要跳过的 MD 轨迹帧数
- **geo_out**: 输出的文件名，默认为 `result.dat`
- **ntype**: 原子种类
- **natom**: 原子数
- **rcut**: 计算 RDF 的截断半径，一般取晶格的一半，单位为 `Angstrom`
- **dr**: 计算 RDF 的 r 的间隔，单位为 `Angstrom`

执行如下命令：

```
mpirun -n 2 candela
```

即可得到 RDF，输出到 Al_rdf.txt，其中第一列为 r，单位为 Angstrom，第二列即为对应的 RDF，第三列为对第二列积分的结果。

### 静态结构因子

进入 SSF 文件夹，INPUT 文件具体如下：

```
calculation   ssf
geo_in_type   ABACUS
geo_directory ../MD_dump
geo_1         0
geo_2         100
geo_interval  2
geo_ignore    50

ssf_out       Al_ssf.txt
ntype         1
natom         32
struf_dgx     1.32656
struf_dgy     1.32656
struf_dgz     1.32656
struf_ng      6
```

以上参数在 Candela 的线上文档中均有详细说明，这里再进行简单概述：

- **calculation**: 设置为 `ssf`，即计算静态结构因子
- **ssf_out**: 输出的文件名
- **struf_dgx**: 倒空间中的间隔，一般取 $2\pi/a$，其中 $a$ 为 x 方向的晶格长度，单位为 $\texttt{Angstrom}^{-1}$
- **struf_dgy**: 一般取 $2\pi/b$，其中 $b$ 为 y 方向的晶格长度，单位为 $\texttt{Angstrom}^{-1}$
- **struf_dgz**: 一般取 $2\pi/c$，其中 $c$ 为 z 方向的晶格长度，单位为 $\texttt{Angstrom}^{-1}$
- **struf_ng**: 上述倒空间中的间隔的数量

执行如下命令：

```
mpirun -n 2 candela
```

即可得到 SSF，输出到 Al_ssf.txt（原始的计算结果）以及 sm-Al_ssf.txt（平滑的计算结果），其中第一列为 q，单位为 Angstrom−1 ，第二列即为对应的 SSF。

### 离子-离子动态结构因子

进入 DSF 文件夹，离子-离子动态结构因子需要对中间散射函数（Intermediate Scattering Function）进行傅里叶变换得到，因此需要先计算中间散射函数，INPUT 文件具体如下：

```
calculation     isf2
geo_in_type     ABACUS
geo_directory   ../MD_dump
```

```
geo_1           50
geo_2           100
geo_interval    1

isf_outfile     isf.txt
ntype           1
natom           32
isf_nt1         11
isf_nt2         40
dt_snapshots    0.00006

isf_target_q    2.65
isf_dgx         1.32656
isf_dgy         1.32656
isf_dgz         1.32656
isf_ngx         6
isf_ngy         6
isf_ngz         6
```

以上参数在 Candela 的线上文档中均有详细说明，这里再进行简单概述：

- **calculation**: 设置为 `isf2`，即计算中间散射函数
- **isf_outfile**: 输出的文件名
- **isf_nt1**: 中间散射函数的时间长度
- **isf_nt2**: 用于平均中间散射函数的时间长度，注意需要控制 `interval*(nt1+nt2)<=geo_2-geo_1+1`
- **dt_snapshots**: MD 轨迹每一帧之间的时间步长，单位为 ps
- **isf_target_q**: 中间散射函数计算的目标 q，单位为 $\texttt{Angstrom}^{-1}$
- **isf_dgx**: 倒空间中的间隔，一般取 $2\pi/a$，其中 a 为 x 方向的晶格长度，单位为 $\texttt{Angstrom}^{-1}$
- **isf_dgy**: 一般取 $2\pi/b$，其中 b 为 y 方向的晶格长度，单位为 $\texttt{Angstrom}^{-1}$
- **isf_dgz**: 一般取 $2\pi/c$，其中 c 为 z 方向的晶格长度，单位为 $\texttt{Angstrom}^{-1}$
- **isf_ngx**: 上述倒空间中 x 方向的间隔的数量
- **isf_ngy**: 上述倒空间中 y 方向的间隔的数量
- **isf_ngz**: 上述倒空间中 z 方向的间隔的数量

执行如下命令：

```
mpirun -n 2 candela
```

即可得到 ISF，输出到 isf.txt，其中第一列为时间，单位为 ps，第二列即为对应的 ISF。

接着执行如下命令得到离子-离子动态结构因子，其中 PathtoCandela 指的是 Candela 的下载目录：

```
python PathtoCandela/examples/e3_dsf/onedsf.py 0.00006 0.0006
```

其中第一个参数 0.00006 = dt_snapshots * geo_interval，为 ISF 的时间步长，单位为 ps；第二个参数 0.0006，为对 ISF 做傅里叶变换的总时长，单位为 ps。这里 onedsf.py 默认读取 isf.txt，并输出到 dsf.txt，其中第一列为 $\omega$，单位为 meV，第二列即为对应的 DSF。

**均方差位移**

单种原子

进入 MSD 文件夹，INPUT 文件具体如下：

```
calculation   msd_multiple
geo_in_type   ABACUS
geo_directory ../MD_dump
geo_1         0
geo_2         100
geo_interval  1
geo_ignore    50

ntype         1
natom         32
msd_n         2
msd_t0        0.003
msd_t         0.0015
msd_dt0       0.0015
msd_dt        0.00006
msd_natom     32
```

以上参数在 Candela 的线上文档中均有详细说明，这里再进行简单概述：

- **calculation**: 设置为 `msd_multiple`，即计算均方差位移
- **msd_n**: 计算均方差位移的段数，目的是将 MD 轨迹分成若干段，分别计算均方差位移
- **msd_t0**: 计算均方差位移的起始时间，单位为 `ps`
- **msd_t**: 每段均方差位移的时长，单位为 `ps`
- **msd_dt0**: 两段均方差位移起始位置的间隔，单位为 `ps`。若设置等于 `msd_t`，则每段均方差位移之间的时间距离为 0
- **msd_dt**: 获取的 MD 轨迹每一帧之间的时间步长（等于 `geo_interval * dt`，其中 `dt` 为 MD 轨迹每一帧之间的时间步长），单位为 `ps`
- **msd_natom**: 需要设置为计算的原子数

执行如下命令：

```
mpirun -n 2 candela
```

即可得到 MSD，输出到 MSD_each.txt 和 MSD_total.txt。其中 MSD_each.txt 第一列为时间，单位为 ps，第二列到最后一列即为对应的每段的 MSD，单位为 Angstrom2；MSD_total.txt 第一列为时间，单位为 ps，第二列为平均的每段的 MSD，单位为 Angstrom2

多种原子

进入 MSD_ntype_2 文件夹，INPUT 文件具体如下：

```
calculation   msd_multiple
geo_in_type   ABACUS
geo_directory MD_dump
geo_1         0
geo_2         100
geo_interval  1
```

```
geo_ignore    50

ntype         2
natom         17
natom1        16
natom2        1
id1           C
id2           Li

msd_n         2
msd_t0        0.003
msd_t         0.0015
msd_dt0       0.0015
msd_dt        0.00006
ele_select    C
msd_natom     16
```

以上参数在 Candela 的线上文档中均有详细说明，与单种原子不同的参数如下：

- natom1、natom2：不同种类原子的个数
- id1、id2：不同种类原子的名称，需要与读入的 MD_dump 中原子名称一致
- ele_select：计算 MSD 的原子名称，需要与 id1 或 id2 对应

多种原子的计算过程与结果与单种原子并无区别。

## 6.11 ABACUS+USPEX 接口教程

### 6.11.1 背景介绍

近年来，随着高性能计算机的快速发展，晶体结构预测算法已然成为计算材料学的核心研究手段之一，极大地促进了新材料设计与研发的进程。Artem R.Oganov 教授课题组开发的进化类晶体结构预测算法 USPEX 是当前主流的全局搜索类算法之一。软件自 2004 年发布以来，该算法已被广泛应用于科研人员的研究工作中。晶体结构预测算法的主要作用是生成结构以及控制结构的演进，结构预测的效率及可靠性除却算法本身的影响外还对结构弛豫软件有极强的依赖性。ABACUS（原子算筹）作为一款国产开源密度泛函理论软件，同时兼具高精度和高效率，可适用于从小体系到上千原子体系的电子结构优化、原子结构弛豫、分子动力学等计算。因此实现晶体结构预测算法 USPEX（https://uspex-team.org/en）和第一性原理软件 ABACUS 的结合会给广大研究人员在结构预测研究中带来便利。

本教程旨在为大家提供一个 USPEX-ABACUS 接口的实现教程，方便大家程序配置。本教程以单质硅（Si）的结构预测为例, 介绍了该接口使用所需的输入文件和参数设置。

如需了解更多关于 USPEX 的使用信息，请参考 USPEX 官方文档：USPEX Documentation　USPEX

### 6.11.2 接口介绍

当前发行的 USPEX 提供了分别以 matlab 和 python 为核心编程语言的两个版本，前者的版本编号为 USPEX-vXX（如 USPEX_v10.5），后者的版本编号为 USPEX-YEAR-XX（如 USPEX 2023.0.2），需要注意的是当前只有 python 版本可以自定义外部接口。该教程中的接口是在 USPEX 2023.0.2 和 ABACUS 3.3.0 版本进行编写测试的，如发生版本变更引起的异常，根据报错再行修改。

首先通过 USPEX 官网（USPEX 下载 USPEX）下载 USPEX 2023.0.2 软件（注册后即可免费下载），解压后根据所使用的 python 版本，选择软件压缩包进行安装，例如 pip install uspex-2023.0.2-cp310-cp310-linux_x86_64.whl。

注：若系统提示需要安装虚拟环境，可以在 USPEX 2023.0.2 目录下激活虚拟环境后重新进行安装。

```
python -m venv tutorial-env
source tutorial-env/bin/activate
```

该接口共包含三个文件

(1) USPEX/components.py（对接口进行注册）

(2) USPEX/Stages/Interfaces/ASEInterfacesAdapter.py（完成 ASE 接口中的读 read 和写 write 功能）

(3) USPEX/Stages/Interfaces/ABACUS_Interface.py（接口实体，主要功能是准备 Abacus 的输入文件和读取 Abacus 的计算结果）

注：以上路径中 USPEX 为主目录。

上述文件可在 Github 仓库下载：https://github.com/gxq1219/Interface_USPEX-ABACUS/tree/master

安装完成后将以上文件复制到对应的路径下，并且需要手动添加环境变量，例如:export PATH=python3.10/site-packages/USPEX:$PATH，然后将 USPEX 2023.0.2 目录下的 random_cell 复制到设置的环境变量路径下。该接口需要安装 ase-abacus 版本的 ase 库，测试版本为 ase-3.23.0b1

安装 ase-abacus 接口

```
git clone https://gitlab.com/1041176461/ase-abacus.git
cd ase-abacus
python3 setup.py install
```

此次测试是在 QSUB 任务管理系统下进行测试，如果更换任务管理系统，应修改 USPEX 输入文件 input.uspex 中的 taskManager 和 TM，此外需修改 USPEX/Stages/TaskManager 中所选任务管理方式中对任务队列状态识别的方式，如在 QSUB.py 文件中应根据所使用平台修改 def _parseJobID 中的内容识别出提交任务的 ID，以及下文的 def isReady 和 def isExist 等函数。

QSUB.py 示例（第 56 行的函数，根据自己超算平台修改，功能是执行 qsub sub.sh 后从屏幕输出的文本中提取出 jobID）：

```
"""
USPEX.Stages.TaskManagers.QSUB
==============================
"""
import logging
from os.path import join as pj
logger = logging.getLogger(__name__)
class QSUB:
    shortname = 'QSUB'
    _RUNSCRIPT = 'jobscript'
    def __init__(self, header : str, connector):
        """
        :param header: description of params of TaskManager
        :param connector: for remote submission
        """
        self.connector = connector
        self.header = header
```

```
    def _prepareSubmission(self, COMMAND_EXEC : str, JOB_NAME : str,
                                inputFile : str, outputFile : str, errorFile : str) -> str:
        """
        Preparing jobscript for submission
        :param commandExec:
        :param jobName:
        :return: jobscript as string
        """
        content = ''
        for line in self.header.split('\n'):
            if ' -N ' in line:
                logger.info('Job name found in HEADER will be overwritten')
            elif ' -o ' in line:
                logger.info('Output file name found in HEADER will be overwritten')
            elif ' -e ' in line:
                logger.info('Error file name found in HEADER will be overwritten')
            else:
                content += line + '\n'
        content += f'#PBS -N  {JOB_NAME}\n'
        content += f'#PBS -o  {outputFile}\n'
        content += f'#PBS -e  {errorFile}\n'
        content += f'cd \$PBS_O_WORKDIR\n'
        content += f'\n{COMMAND_EXEC}\n'
        return ''.join(content)
    async def submit(self, command: str, jobname: str, input: str, output: str, error: str,
    calcFolder : str) -> int:
        content = self._prepareSubmission(command, jobname, input, output, error)
        with open(pj(calcFolder, self._RUNSCRIPT), 'wt') as f:
            f.write(content)
        await self.connector.sync_l2r(pj(calcFolder, self._RUNSCRIPT))
        returncode, out, err = await self.connector.execute(f'qsub {self._RUNSCRIPT}',
        cwd=calcFolder)
        logger.debug(f'process returned code {returncode}')
        if returncode != 0:
            logger.error(err)
            logger.error(out)
        jobID = self._parseJobID(out, err)
        logger.info(f"Job ID is {jobID}")
        return jobID

    def _parseJobID(self, output : str, error : str) -> int:
        """
        :param output:
        :param error:
        :return: jobID
        """
        if '.mgmt' in output:
            tmp = output.index('.mgmt')
```

```
            return int(output[:tmp])
        elif 'job' in output:
            tmp = output.index('job') + 4
            return int(output[tmp:])
        elif 'comput100' in output:
            tmp = output.index('comput100')-1
            return int(output[:tmp])
        else:
            return int(output)
    async def isReady(self, jobID):
        returncode, out, err = await self.connector.execute(f'qstat {jobID}')
        return not (' R ' in out or ' Q ' in out)
    async def isExist(self, jobID):
        returncode, out, err = await self.connector.execute(f'qstat {jobID}')
        return len(out) > 0
    async def kill(self, jobID):
        returncode, out, err = await self.connector.execute(f'qdel {jobID}')
        #logger.info(f'Process with jobID={jobID}  killed.')
```

### 6.11.3 流程（以 Si 为例）

**准备 USPEX 的输入文件 input.uspex**

```
{
    optimizer: {
        type: GlobalOptimizer
        target: {
            type: Atomistic
            conditions: {externalPressure: 0.00001}
            compositionSpace: {symbols: [Si]
                               blocks: [[4]]}
        }
        optType: enthalpy
        selection: {
            type: USPEXClassic
            popSize: 4
            initialPopSize: 4
            bestFrac: 0.6
            optType: (aging enthalpy)
            fractions: {
                heredity: (0.4 0.7 0.5)
                softmodemutation: (0.2 0.5 0.3)
                randSym: (0.05 0.5 0.1)
                randTop: (0.05 0.5 0.1)
            }
        }
    }
    stages: [abacus1 abacus2 abacus3 abacus4 abacus5]
```

```
    numParallelCalcs: 4
    numGenerations: 25
    stopCrit: 10
}

#define abacus1
{type : abacus, commandExecutable : 'OMP_NUM_THREADS=1 mpirun -machinefile \$PBS_NODEFILE -env
I_MPI_HYDRA_DEBUG 3 -genv I_MPI_FABRICS shm:ofi abacus', kresol: 0.20, taskManager: TM}

#define abacus2
{type : abacus, commandExecutable : 'OMP_NUM_THREADS=1 mpirun -machinefile \$PBS_NODEFILE -env
I_MPI_HYDRA_DEBUG 3 -genv I_MPI_FABRICS shm:ofi abacus', kresol: 0.16, taskManager: TM}

#define abacus3
{type : abacus, commandExecutable : 'OMP_NUM_THREADS=1 mpirun -machinefile \$PBS_NODEFILE -env
I_MPI_HYDRA_DEBUG 3 -genv I_MPI_FABRICS shm:ofi abacus', kresol: 0.12, taskManager: TM}

#define abacus4
{type : abacus, commandExecutable : 'OMP_NUM_THREADS=1 mpirun -machinefile \$PBS_NODEFILE -env
I_MPI_HYDRA_DEBUG 3 -genv I_MPI_FABRICS shm:ofi abacus', kresol: 0.08, taskManager: TM}

#define abacus5
{type : abacus, commandExecutable : 'OMP_NUM_THREADS=1 mpirun -machinefile \$PBS_NODEFILE -env
I_MPI_HYDRA_DEBUG 3 -genv I_MPI_FABRICS shm:ofi abacus', kresol: 0.05, taskManager: TM}

#define TM
{
type : QSUB,
header:"#PBS -S /bin/bash
#PBS -N single
#PBS -l nodes=1:ppn=16
#PBS -j oe
#PBS -V"
}
```

注意：ABACUS 在任务提交时，应注意指定 OMP_NUM_THREADS，防止内存不足引起计算错误

**准备 Specific 文件夹**

所需文件：INPUT_X（使用 ABACUS 弛豫）、ATOMIC_SPECIES、NUMERICAL_ORBITAL 以及轨道和赝势文件。

INPUT 文件内容和个数由用户自行设置（需要与 input.uspex 中 stages 对应），具体可参考 uspex 官方文档。

需要注意的是：INPUT 的 suffix 需指定为 USPEX；

需给出 ATOMIC_SPECIES 和 NUMERICAL_ORBITAL 两个文件指定所使用的 upf 文件和 orb 文件。

ATOMIC_SPECIES 内容：元素符号质量赝势文件名

NUMERICAL_ORBITAL 内容：元素符号轨道文件名

**根据计算实际需求添加 Seeds 结构（可选，非必须）**

**提交运算**

```
nohup ./uspex-sub.sh >> log &

### content of uspex-sub.sh

#!/bin/sh
while true;do
    data >> log
    uspex -r >> log
    sleep 300   ### Users can adjust this value to manipulate the frequency of the call to uspex.
done
```

**计算结果**

如果程序可以正常提交任务，log 文件中会打印如下内容：

```
2023-08-09 16:52:47,115 - USPEX.Optimizers.Target - INFO - Following utilities was not initialized:
['PowderSpectrumAnalyzer', 'SingleCrystalSpectrumAnalyzer'].
2023-08-09 16:52:47,115 - USPEX.Optimizers.Target - INFO - Permutation does not work when number
of symbols in calculation is 1.
2023-08-09 16:52:47,123 - USPEX.Stages.GenerationController - INFO - Calculation initialized from
input parameters.
2023-08-09 16:52:47,441 - USPEX.Selection.USPEXClassic - INFO - System 0 successfully created by
RandSym operator.
2023-08-09 16:52:47,866 - USPEX.Selection.USPEXClassic - INFO - System 1 successfully created by
RandSym operator.
2023-08-09 16:52:48,638 - USPEX.Selection.USPEXClassic - INFO - System 2 successfully created by
RandSym operator.
2023-08-09 16:52:48,767 - USPEX.Selection.USPEXClassic - INFO - System 3 successfully created by
RandSym operator.
2023-08-09 16:52:48,788 - USPEX.Stages.Executor - INFO - System 0 with tag 1 will be submitted now.
2023-08-09 16:52:48,788 - USPEX.Stages.TaskManagers.QSUB - INFO - Job name found in HEADER will be
overwritten
2023-08-09 16:52:48,792 - USPEX.Stages.Executor - INFO - System 1 with tag 1 will be submitted now.
2023-08-09 16:52:48,792 - USPEX.Stages.TaskManagers.QSUB - INFO - Job name found in HEADER will be
overwritten
2023-08-09 16:52:48,796 - USPEX.Stages.Executor - INFO - System 2 with tag 1 will be submitted now.
2023-08-09 16:52:48,797 - USPEX.Stages.TaskManagers.QSUB - INFO - Job name found in HEADER will be
overwritten
2023-08-09 16:52:48,801 - USPEX.Stages.Executor - INFO - System 3 with tag 1 will be submitted now.
2023-08-09 16:52:48,801 - USPEX.Stages.TaskManagers.QSUB - INFO - Job name found in HEADER will be
overwritten
2023-08-09 16:52:48,812 - USPEX.Stages.TaskManagers.QSUB - INFO - Job ID is 37768
2023-08-09 16:52:48,818 - USPEX.Stages.TaskManagers.QSUB - INFO - Job ID is 37769
2023-08-09 16:52:48,822 - USPEX.Stages.TaskManagers.QSUB - INFO - Job ID is 37770
2023-08-09 16:52:48,825 - USPEX.Stages.TaskManagers.QSUB - INFO - Job ID is 37771
```

如果程序可以正常输出结果，在 results1/Individuals 中会打印相关的信息，示例如下：

```
+----+-----------------+-------------+--------------+-------------+---------------+
-----------------+--------------+--------------+
| ID |      Origin     | Composition | Enthalpy (eV) | Volume (A^3) |  SYMMETRY (N) |
Structure order | Average order | Quasientropy |
+----+-----------------+-------------+--------------+-------------+---------------+
-----------------+--------------+--------------+
| 0  |     RandSym     |    Si: 4    |    -425.452   |    59.320    | P6/mmm  (191) |
0.334     |      0.334    |    -0.000    |
| 2  |     RandSym     |    Si: 4    |    -418.078   |    59.320    | I4/mmm  (139) |
0.287     |      0.287    |    -0.000    |
| 3  |     RandSym     |    Si: 4    |    -425.038   |    59.320    | P4_332  (212) |
0.392     |      0.392    |    -0.000    |
| 4  | Softmodemutation |    Si: 4    |    -428.592   |    59.320    | I4/mmm  (139) |
0.325     |      0.325    |     0.000    |
| 5  | Softmodemutation |    Si: 4    |    -428.592   |    59.320    | I4/mmm  (139) |
0.325     |      0.325    |     0.000    |
| 6  |     Heredity    |    Si: 4    |    -427.892   |    59.320    |  P2_1/m  (11) |
0.202     |      0.215    |     0.077    |
| 8  | Softmodemutation |    Si: 4    |    -427.882   |    59.320    |  P2_1/m  (11) |
0.201     |      0.215    |     0.076    |
| 9  | Softmodemutation |    Si: 4    |    -427.887   |    59.320    |  P2_1/m  (11) |
0.202     |      0.215    |     0.075    |
| 10 |     Heredity    |    Si: 4    |    -428.645   |    59.320    |  Cmce    (64) |
0.261     |      0.261    |     0.000    |
| 11 | Softmodemutation |    Si: 4    |    -428.644   |    59.320    |  Cmce    (64) |
0.261     |      0.261    |     0.000    |
| 12 | Softmodemutation |    Si: 4    |    -428.645   |    59.320    |  Cmce    (64) |
0.261     |      0.261    |     0.001    |
| 14 |     Heredity    |    Si: 4    |    -428.628   |    59.320    |  Cmce    (64) |
0.263     |      0.263    |     0.000    |
| 15 |     Heredity    |    Si: 4    |    -428.928   |    59.320    |  Cmcm    (63) |
0.291     |      0.291    |     0.000    |
| 16 |     Heredity    |    Si: 4    |    -428.949   |    59.320    |  P2_1/m  (11) |
0.283     |      0.284    |     0.006    |
| 17 |     RandSym     |    Si: 4    |    -422.987   |    59.320    |  C2/m    (12) |
0.250     |      0.250    |     0.000    |
| 19 | Softmodemutation |    Si: 4    |    -428.927   |    59.320    |  Cmcm    (63) |
0.291     |      0.291    |     0.000    |
| 20 | Softmodemutation |    Si: 4    |    -428.928   |    59.320    |  Cmcm    (63) |
0.290     |      0.290    |     0.000    |
| 21 |     Heredity    |    Si: 4    |    -428.926   |    59.320    |  Cmcm    (63) |
0.290     |      0.290    |     0.000    |
```

### 6.11.4 结语

建议大家在使用前，首先选择小体系进行测试；此外大家也可根据自己的实际需要对接口文件进行修改扩充，有相关问题可通过邮件方式沟通。

# 6.12 ABACUS+Hefei NAMD 使用教程

## 6.12.1 介绍

激发态动力学一直是凝聚态物理中的重要问题，在激发状态下，凝聚态体系中的准粒子会发生复杂的相互作用，涉及到不同时间尺度的超快过程，因此，研究凝聚态体系的激发态动力学不仅需要从时间、空间、能量和动量等多个维度对其进行描述，还需要描述不同准粒子之间的动态耦合，而目前商用的第一性原理软件无法满足这样的需求。针对这一问题，中国科学技术大学赵瑾教授团队发展了自主可控的激发态动力学软件 Hefei-NAMD（https://hefei-namd.org/code/），基于面跳跃算法（Surface Hopping）与经典路径近似，初步实现激发态动力学在时间、空间、动量、能量、自旋等多个维度上的描述，并可以研究激发态电子、空穴、激子、声子、极化子等准粒子的动态相互作用。Hefei-NAMD 程序采用模块化架构，包含单粒子动力学、自旋动力学、动量空间动力学以及 GW+rtBSE 模块。目前程序使用周期性边界条件，可处理上百个原子，且与密度泛函软件 VASP、Quantum Espresso 等存在接口，Hefei-NAMD 和 ABACUS v3.2 建立的接口（主要针对局域轨道算法，涉及波函数与局域轨道下的交叠矩阵等信息，用来计算非绝热耦合项和电子跃迁概率）和提供的相关案例，可以进一步拓宽 Hefei-NAMD 的使用范围。

本教程旨在介绍采用 ABACUS（基于 ABACUS 3.2.0 版本）做分子动力学计算，将结果作为 Hefei NAMD 软件的输入文件进行后续非绝热分子动力学计算的流程。

这里推荐大家阅读 Hefei NAMD 软件的相关文档和说明：

http://staff.ustc.edu.cn/~zqj/posts/Hefei-NAMD-Training/

https://github.com/QijingZheng/Hefei-NAMD

https://github.com/vtzf/abacus-namd（这个是拥有与 ABACUS 接口的 Hefei NAMD 仓库，功能与上面的仓库是一样的），第一步请先下载这个链接上面的 python 代码。

## 6.12.2 计算流程

我们这里提供的测试案例是包含 2 个原子的金刚石结构 Si 结构，采用的模守恒赝势是 Si_ONCV_PBE-1.0.upf，以及原子轨道文件采用的是 Si_gga_8au_60Ry_2s2p1d.orb（GGA 泛函，8 au 截断半径，60 Ry 能量截断，以及包含 2s2p1d 的 DZP 轨道）。

### 用 ABACUS 进行分子动力学 (MD) 计算

输入文件 INPUT

大部分参数为做分子动力学计算所需要参数，具体含义可以参考 ABACUS 的分子动力学教程。为了后续用 Hefei NAMD 进行非绝热分子动力学计算，ABACUS 会输出每一步分子动力学模拟的哈密顿量矩阵 H、交叠矩阵 S、波函数文件。因此需将相关的 ABACUS 的 INPUT 文件中的输入参数 out_wfc_lcao 和 out_mat_hs 都设置为 1，表示输出这些物理量。计算完成后，在 OUT.*** 文件夹中，每一个分子步都对应一个文件夹 MD_n，在每个文件夹中会有 data-0-H、data-0-S、LOWF_GAMMA_S1.dat 三个文件储存我们需要输出的物理量。

```
INPUT_PARAMETERS
 #Parameters     (General)
 suffix          autotest
 pseudo_dir      ../../tests/PP_ORB
 orbital_dir     ../../tests/PP_ORB
 nbands          8
 calculation     md

 #Parameters (Accuracy)
```

```
ecutwfc          50
scf_nmax         20

basis_type       lcao
md_nstep         10

cal_stress       1
stress_thr       1e-6
cal_force        1
force_thr_ev     1.0e-3

ks_solver        genelpa
mixing_type      pulay
mixing_beta      0.7

md_type          nve
md_restart       0
md_tfirst        10
init_vel         1

read_file_dir    ./

gamma_only       1

out_wfc_lcao     1
out_mat_hs       1
```

结构文件 STRU

```
ATOMIC_SPECIES
Si 1 Si_ONCV_PBE-1.0.upf

NUMERICAL_ORBITAL
Si_gga_8au_60Ry_2s2p1d.orb

LATTICE_CONSTANT
10.2

LATTICE_VECTORS
0.5 0.5 0 #latvec1
0.5 0 0.5 #latvec2
0 0.5 0.5 #latvec3

ATOMIC_POSITIONS
Cartesian

Si #label
0 #magnetism
```

```
2 #number of atoms
0  0  0  m  1  1  1  v  1.75205850628e-05  0.000155425594558  -3.99334763874e-05
0.241  0.255  0.250999999999  m  1  1  1  v  -1.75205850628e-05  -0.000155425594558  3.99334763874e-05
```

### 用 Hefei NAMD 进行非绝热分子动力学（NAMD）计算

Hefei NAMD 代码为 python 代码，在 src 目录下，建议使用 python 3.9。需要的 python 库有 Numpy，Scipy，Numba，MPI4py（>= 3.1.3）。

设置 NAMD 计算参数

在 Args.py 中设置 NAMD 的各种参数，在 Args.py 的注释中有关于各种参数具体含义的详细介绍。

```
# NAMD parameter
# manual input start
dftdir   = '/public/share/zhaojin/tuyy/abacus/sh/OUT.autotest1/'
namddir  = '../namd_test/' # output NAMD output file in namddir
dt       = 1      # MD time step (fs)
start_t  = 1      # start MD step
end_t    = 2000   # end MD step
istart_t = 901    # isample start MD step
iend_t   = 1000   # isample end MD step

LCHARGE  = True   # output atom projected charge density
atom     = [13,26]# atom number of all species (only needed in atomic basis)
orbital  = [27,13]# atomic orbital basis number (only needed in atomic basis)
whichA   = [0,13,14,15] # atom index for projected charge density (starts from 0)

LRANGE   = True   # select range of band, change iband
                  # if not given, LRECOMB specifies energy range
                  # if LRECOMB not given, energy range: [0,nbands]
LHOLE    = True   # Hole/electron transfer
dE       = 2.0    # initial energy from VBM/CBM (eV)

LPHASE   = True   # phase correction

TEMP     = 300    # temperature in Kelvin
NACTIME  = 1000   # time for used NAC (i_end_t-state_t+NACTIME<nstep)
NAMDTIME = 1000   # time for NAMD run
NELM     = 1000   # electron time step (per fs)
NTRAJ    = 5000   # SH trajectories number

LINTERP  = 2      # hamiltonian interpolation algorithm 1,2,3
LTA      = False  # Liouville-Trotter algorithm for small NELM
LSH      = 'FSSH' # run DISH, FSSH or DCSH

LRECOMB  = False  # consider electron-hole recombination

OutMode  = 'text' # output file format 'numpy(.npy)' or 'text'
# manual input end
```

读取 ABACUS 的输出文件

运行 NAC.py 即可将 ABACUS 的输出文件转成 Hefei NAMD 需要的 NATXT 和 EIGTXT 文件。

在超算上计算的话，可以直接使用用 sub_scripts 目录下的 sub_nac 脚本提交任务，用户可在 sub_nac 中自行修改节点数等信息。

```
sbatch sub_nac
```

进行非绝热分子动力学计算

运行 SurfHop.py 即可进行 NAMD 的计算。

在超算上计算的话，可以直接使用用 sub_scripts 目录下的 sub_sh 脚本提交任务，用户可在 sub_sh 中自行修改节点数等信息。

```
sbatch sub_sh
```

输出文件中，SUPROP 文件中包含了电子驰豫能量和电子占据数随时间的演化，PSICT 文件则包含波函数系数随时间的演化。可用 https://github.com/QijingZheng/Hefei-NAMD/tree/master/scripts 中的脚本进行画图。

### 6.12.3 结语

本篇主要介绍了如何利用 ABACUS 和 Hefei NAMD 进行非绝热分子动力学的计算，简单来说就是通过 ABACUS 跑分子动力学得到每一步的波函数和哈密顿量等信息，Hefei NAMD 再读入这些信息进行非绝热分子动力学的计算。关于非绝热分子动力学的详细介绍则可以查看 Hefei NAMD 的相关网站（见最上面的网站），如有问题欢迎联系。

### 6.12.4 参考文献

[1] Zheng, Q.; Chu, W.; Zhao, C.; Zhang, L.; Guo, H.; Wang, Y.; Jiang, X.; Zhao, J. Ab initio nonadiabatic molecular dynamics investigations on the excited carriers in condensed matter systems. Wiley Interdiscip. Rev. Comput. Mol. Sci. 2019, 9, e1411.

## 6.13 Pyabacus 文档一：用户手册

### 6.13.1 简介

pyabacus 是密度泛函理论软件 ABACUS（原子算筹）的 Python 接口，提供了与 ABACUS 软件交互的 Python API。

该项目基于 pybind11 和 scikit-build-core 构建。

### 6.13.2 模块

pyabacus 当前提供以下模块：

- **io**: pyabacus 中的输入/输出模块。
- **Cell**: 用于单元结构的模块，将 Python 中的 ModuleNAO 与用户输入连接。
- **ModuleBase**: 基本数学函数模块。
- **ModuleNAO**: 数值原子轨道 (NAO) 模块。
- **hsoiver**: 用于求解哈密顿量的模块。

### 6.13.3 安装

我们建议用户使用 conda 进行安装，conda 是一个开源的软件包管理系统和环境管理系统，可以方便的管理不同版本的 Python 和软件包。用户可以按照以下步骤安装 Pyabacus：

创建一个虚拟环境，并且激活虚拟环境 conda create -n pyabacus python=3.8 & conda activate pyabacus

下载 ABACUS 软件

```
cd {your_working_directory}
git clone https://github.com/abacusmodeling/abacus-develop.git
cd abacus-develop/python/pyabacus
```

安装 ABACUS 所需的依赖库 使用 pip install -v . 安装 Pyabacus，或者使用 pip install .[test] 安装 Pyabacus 及其测试环境的依赖库 (使用 pip install -v .[test] -i https://pypi.tuna.tsinghua.edu.cn/simple 加速安装过程)。

### 6.13.4 调用

用户可以通过以下代码调用 Pyabacus 的接口：

```
import pyabacus
s = pyabacus.ModuleBase.Sphbes()
s.sphbesj(1, 0.0)
0.0
```

上面的代码调用了 ABACUS 的 `ModuleBase::Sphbes` 模块。

目前，我们移植了下面几个模块：

- `ModuleBase::Sphbes`
- `ModuleBase::Integral`
- `RadialCollection`
- `TwoCenterIntegrator`
- `hsolver::dav_subspace` 在 Python 中调用这些模块的方法可见 `examples` 文件夹内示例。

### 6.13.5 示例

examples 文件夹中包含了一些 Pyabacus 的示例脚本，用户可以通过这些示例脚本了解 Pyabacus 的使用方法。

运行 examples 文件夹中的 python vis_nao.py 以可视化数值原子轨道。

```
$ cd examples/
$ python vis_nao.py
```

运行 examples 文件夹中的 python ex_s_rotate.py 来检查 S 矩阵。

```
$ cd examples/
$ python ex_s_rotate.py
norm(S_e3 - S_numer) =  3.341208104032616e-15
```

运行 examples 文件夹中的 python diago_matrix.py 来对角化一个矩阵

```
$ cd examples/
$ python diago_matrix.py
eigenvalues calculated by pyabacus: [-0.38440611 0.24221155 0.31593272 0.53144616 0.85155108
1.06950155 1.11142051 1.12462152]
eigenvalues calculated by scipy: [-0.38440611 0.24221155 0.31593272 0.53144616 0.85155108
1.06950154 1.11142051 1.12462151]
error: [9.26164700e-12 2.42959514e-10 2.96529468e-11 7.77933273e-12 7.53686002e-12 2.95628810e-09
1.04678111e-09 7.79106313e-09]
```

### 6.13.6 测试

Pyabacus 使用 pytest 进行单元测试，用户可以在 abacus-develop/python/pyabacus/tests 目录下使用 pytest -v 命令进行单元测试。

### 6.13.7 结语

以上就是 Pyabacus 的一些基本安装和使用方法，如果读者对进一步交流感兴趣，欢迎登录 ABACUS 的 github 网站进行进一步的交流。

## 6.14 Pyabacus 文档二：HSolver 模块

### 6.14.1 简介

目前，pyabacus.hsolver 内实现了三种对角化算法的 python 化，分别是 hsolver.dav_subspace、hsolver.davidson 与 hsolver.cg，其签名分别如下：

```
def dav_subspace(
    # 矩阵-向量组乘法函数，接收一个 numpy 数组，返回一个 numpy 数组
    mvv_op: Callable[[NDArray[np.complex128]], NDArray[np.complex128]],
    # 对角化过程中的特征向量组初猜
    init_v: NDArray[np.complex128],
    # 矩阵维度
    dim: int,
    # 要求的特征值个数，从最小的特征值开始，算法会求解 num_eigs 个特征值
    num_eigs: int,
    # 预处理向量
    precondition: NDArray[np.float64],
    # 基组允许容纳的向量个数 (dav_ndim * nband 个)
    dav_ndim: int = 2,
    # 迭代误差
    tol: float = 1e-2,
    # 最大迭代次数
    max_iter: int = 1000,
    # 是否使用子空间函数
    need_subspace: bool = False
) -> Tuple[NDArray[np.float64], NDArray[np.complex128]]:

# dav_subspace 会返回一个元组，分别为特征值数组与其对应的特征向量集合
```

```
23
24 def dav_subspace(
25     # 矩阵-向量组乘法函数，接收一个 numpy 数组，返回一个 numpy 数组
26     mvv_op: Callable[[NDArray[np.complex128]], NDArray[np.complex128]],
27     # 对角化过程中的特征向量组初猜
28     init_v: NDArray[np.complex128],
29     # 矩阵维度
30     dim: int,
31     # 要求的特征值个数，从最小的特征值开始，算法会求解 num_eigs 个特征值
32     num_eigs: int,
33     # 预处理向量
34     precondition: NDArray[np.float64],
35     # 基组允许容纳的向量个数 (dav_ndim * nband 个)
36     dav_ndim: int = 2,
37     # 迭代误差
38     tol: float = 1e-2,
39     # 最大迭代次数
40     max_iter: int = 1000,
41     # 是否使用 PAW 方法
42     use_paw: bool = False
43 ) -> Tuple[NDArray[np.float64], NDArray[np.complex128]]:
44
45 # davidson 会返回一个元组，分别为特征值数组与其对应的特征向量集合
46
47 def cg(
48     # 矩阵-向量组乘法函数，接收一个 numpy 数组，返回一个 numpy 数组
49     mvv_op: Callable[[NDArray[np.complex128]], NDArray[np.complex128]],
50     # 对角化过程中的特征向量组初猜
51     init_v: NDArray[np.complex128],
52     # 矩阵维度
53     dim: int,
54     # 要求的特征值个数，从最小的特征值开始，算法会求解 num_eigs 个特征值
55     num_eigs: int,
56     # 预处理向量
57     precondition: NDArray[np.float64],
58     # 迭代误差
59     tol: float = 1e-2,
60     # 最大迭代次数
61     max_iter: int = 1000,
62     # 是否使用子空间函数
63     need_subspace: bool = False,
64     # 计算模式：自洽场/非自洽场
65     scf_type: bool = False,
66     # 进程数
67     nproc_in_pool: int = 1
68 ) -> Tuple[NDArray[np.float64], NDArray[np.complex128]]:
69
70 # cg 会返回一个元组，分别为特征值数组与其对应的特征向量集合
```

### 6.14.2 安装 pyabacus

我们建议用户使用 conda 进行安装，conda 是一个开源的软件包管理系统和环境管理系统，可以方便的管理不同版本的 Python 和软件包。用户可以按照以下步骤安装 pyabacus：

创建一个虚拟环境，并且激活虚拟环境 conda create -n pyabacus python=3.8 & conda activate pyabacus

下载 ABACUS

```
cd {your_working_directory}
git clone https://github.com/abacusmodeling/abacus-develop.git
cd abacus-develop/python/pyabacus
```

安装 ABACUS 所需的依赖库使用 pip install -v . 安装 pyabacus，或者使用 pip install .[test] 安装 Pyabacus 及其测试环境的依赖库 (使用 pip install -v .[test] -i https://pypi.tuna.tsinghua.edu.cn/simple 加速安装过程)。

### 6.14.3 调用对角化算法

三种对角化方法签名几乎一致，只有微小的差别，调用方法也相同，下面统一用 diago 指代。

在调用这两个函数前，首先我们需要读取一个矩阵，我们可以从 https://sparse.tamu.edu/PARSEC 上下载，之后使用 scipy 读取，并且确定要求的特征值个数（示例中为 8 个）：

```
h_mat = scipy.io.loadmat(mat_file)['Problem']['A'][0, 0]
dim = h_mat.shape[0]
num_eigs = 8
```

我们也可以用 numpy 生成一个对角占优的 Hermitian 矩阵：

```
n = 500
h_mat = np.random.rand(n,n)
h_mat =  h_mat +  h_mat.conj().T + np.diag(np.random.random(n))*10
```

之后，我们可以定义 mvv_op 算子：

```
def mvv_op(x):
    return h_mat.dot(x)
# 如果你愿意的话，也可以定义为一个 lambda 函数
# mvv_op = lambda x: h_mat.dot(x)
```

选取初猜 v0：

```
v0 = np.random.rand(nbasis, nband)
```

计算预处理子 precond，由于这里我们读取的矩阵为对角占优的稀疏矩阵，所以我们可以取出矩阵的对角元组成新的矩阵 D，并且计算 D 的逆作为矩阵的近似逆。

```
diag_elem = h_mat.diagonal()
# 为了数值稳定，防止部分对角元过小（接近 0）导致
# 其倒数为无穷大，我们将小于 1e-8 的点设置为 1e-8
diag_elem = np.where(np.abs(diag_elem) < 1e-8, 1e-8, diag_elem)
```

```
precond = 1.0 / np.abs(diag_elem)
```

做完上述一系列准备工作后，我们即可调用 pyabacus 的算法来计算特征值问题了！

```
# 下面的 diago 可按需替换为 dav_subspace 或 davidson
e, v = diago(
    mm_op,
    v0,
    nbasis,
    nband,
    precond,
    dav_ndim=8, # cg 法无该参数
    tol=1e-8,
    max_iter=1000
)

print(f'eigenvalues calculated by pyabacus is: \n', e)
```

### 6.14.4 结语

以上就是 Pyabacus 的一些基本安装和使用方法，如果读者对进一步交流感兴趣，欢迎登录 ABACUS 的 github 网站进行进一步的交流。

## 6.15 Pyabacus 文档三：开发者指南

### 6.15.1 简介

欢迎来到 pyabacus 项目！pyabacus 是国产第一性原理软件 ABACUS（中文名原子算筹）的 Python 接口，它旨在为用户提供了便捷的 Python API，使用户能够利用 Python 语言更灵活地实现电子结构计算并分析结果。本文档主要针对开发者撰写，以下是本文档的目录

- 项目结构
  - 根目录 CMake 配置
  - 模块 CMake 配置
- Python 化上手指南
  - Python 化指南
  - 如何编写单元测试
  - 如何参与开源贡献
- 资料汇总

读者可以参考该系列文档的前两个文档来熟悉 pyabacus。如果读者希望做出贡献，本指南将帮助您了解项目结构、开发流程和代码贡献方式。

### 6.15.2 结构

**目录结构**

pyabacus 项目的目录结构如下所示：

```
pyabacus/
  CMakeLists.txt
  src
      pyabacus
          {your_module}
              {interface}.py
              __init__.py
      {your_module}
          {your_code}.cpp
          CMakeLists.txt
```

项目采用 pybind11 和 scikit-build-core 构建，借助 CMake 构建工具链。因此，CMakeLists.txt 的配置是深入理解项目结构的关键。

### 根目录 CMake 配置

根目录下的 CMakeLists.txt 是 pyabacus 项目的主要配置文件。它设置了项目、查找必要的依赖项、配置构建选项，并为不同模块包含子目录。以下是文件中每部分的详细说明：

```
cmake_minimum_required(VERSION 3.15...3.26)

# 项目设置
project(
  \${SKBUILD_PROJECT_NAME}
  VERSION \${SKBUILD_PROJECT_VERSION}
  LANGUAGES CXX)
```

此部分设置项目名称、版本和使用的编程语言（本项目中为 C++）。项目名称和版本分别从 SKBUILD_PROJECT_NAME 和 SKBUILD_PROJECT_VERSION 变量中获取。

```
# 查找 Python 和 pybind11
find_package(Python REQUIRED COMPONENTS Interpreter Development.Module)
find_package(pybind11 CONFIG REQUIRED)
```

该部分查找所需的 Python 和 pybind11 包。

```
# 设置源路径
set(ABACUS_SOURCE_DIR "\${PROJECT_SOURCE_DIR}/../../source")
set(BASE_PATH "\${ABACUS_SOURCE_DIR}/module_base")
set(NAO_PATH "\${ABACUS_SOURCE_DIR}/module_basis/module_nao")
set(HSOLVER_PATH "\${ABACUS_SOURCE_DIR}/module_hsolver")
set(PSI_PATH "\${ABACUS_SOURCE_DIR}/module_psi")
set(ENABLE_LCAO ON)
list(APPEND CMAKE_MODULE_PATH "\${PROJECT_SOURCE_DIR}/../../cmake")
```

此部分设置了各种源路径和配置选项，定义了不同模块的路径并附加了自定义的 CMake 模块路径。

```
# 添加数学库
if(DEFINED ENV{MKLROOT} AND NOT DEFINED MKLROOT)
    set(MKLROOT "\$ENV{MKLROOT}")
endif()
if(MKLROOT)
  set(MKL_INTERFACE lp64)
  set(ENABLE_MPI ON)
  if (ENABLE_MPI)
    find_package(MPI REQUIRED)
    include_directories(\${MPI_CXX_INCLUDE_PATH})
  endif()

  set(USE_OPENMP ON)
  if(USE_OPENMP)
    find_package(OpenMP REQUIRED)
    set (CMAKE_CXX_FLAGS "\${CMAKE_CXX_FLAGS} \${OpenMP_CXX_FLAGS}")
    add_link_options(\${OpenMP_CXX_LIBRARIES})
  endif()
  find_package(MKL REQUIRED)
  add_definitions(-D__MKL)
  include_directories(\${MKL_INCLUDE} \${MKL_INCLUDE}/fftw)

  if(NOT ENABLE_DEEPKS)
    list(APPEND math_libs IntelMKL::MKL)
  endif()

  if(CMAKE_CXX_COMPILER_ID MATCHES Intel)
    list(APPEND math_libs -lifcore)
  endif()
else()
    find_package(FFTW3 REQUIRED)
    add_compile_definitions(__FFTW3)
    find_package(LAPACK REQUIRED)
    include_directories(\${FFTW3_INCLUDE_DIRS})
    list(APPEND math_libs FFTW3::FFTW3 LAPACK::LAPACK)

  if(ENABLE_LCAO)
    find_package(ScaLAPACK REQUIRED)
    list(APPEND math_libs ScaLAPACK::ScaLAPACK)
  endif()
endif()
```

此部分配置数学库。它检查 Intel 数学核心库 (MKL) 的可用性并在可用时进行配置。如果 MKL 不可用，则改用 FFTW3 和 LAPACK。同时，若启用 MPI 和 OpenMP，也进行相应配置。

```
# 添加包含目录
include_directories(
    \${BASE_PATH}
```

```
    \${ABACUS_SOURCE_DIR}
    \${ABACUS_SOURCE_DIR}/module_base/module_container
    )
```

此部分添加了项目所需的包含目录。

```
# 添加基本库
set(CMAKE_POSITION_INDEPENDENT_CODE ON)
# 添加 base 模块
set(BASE_BINARY_DIR "\${PROJECT_SOURCE_DIR}/build/base")
add_subdirectory(\${ABACUS_SOURCE_DIR}/module_base \${BASE_BINARY_DIR})
# 添加 parameter 模块
set(PARAMETER_BINARY_DIR "\${PROJECT_SOURCE_DIR}/build/parameter")
add_subdirectory(\${ABACUS_SOURCE_DIR}/module_parameter \${PARAMETER_BINARY_DIR})
# 添加 orb 模块
set(ORB_BINARY_DIR "\${PROJECT_SOURCE_DIR}/build/orb")
add_subdirectory(\${ABACUS_SOURCE_DIR}/module_basis/module_ao \${ORB_BINARY_DIR})
```

此部分设置位置无关代码标志，并为 base、parameter、orb 模块添加子目录，指定这些模块的构建目录。

```
# 设置 RPATH
execute_process(
  COMMAND "\${PYTHON_EXECUTABLE}" -c "from distutils.sysconfig import get_python_lib;
  print(get_python_lib())"
  OUTPUT_VARIABLE PYTHON_SITE_PACKAGES
  OUTPUT_STRIP_TRAILING_WHITESPACE
)
```

此部分设置 Python 包目录的运行时搜索路径 (RPATH)，使用 Python 命令获取 site-packages 路径并将其存储在 PYTHON_SITE_PACKAGES 变量中。

```
# 将包名设置为 pyabacus
set(TARGET_PACK pyabacus)
set(CMAKE_INSTALL_RPATH "\${PYTHON_SITE_PACKAGES}/\${TARGET_PACK}")
```

此部分将包名设置为 pyabacus，并配置安装 RPATH 以包含 Python site-packages 目录。

```
# 为子模块添加子目录
add_subdirectory(\${PROJECT_SOURCE_DIR}/src/hsolver)
add_subdirectory(\${PROJECT_SOURCE_DIR}/src/ModuleBase)
add_subdirectory(\${PROJECT_SOURCE_DIR}/src/ModuleNAO)
```

此部分为模块添加子目录，每个子目录包含自己的 CMakeLists.txt 以进行进一步配置。

通过遵循此结构，CMakeLists.txt 文件确保了所有必要的依赖项被找到、配置并包含在构建过程中。它还设置了项目环境并包含不同组件的子模块。

### 模块 CMake 配置

以下示例为模块（如 pyabacus.hsolver）的 CMakeLists.txt 文件：

```
# 添加 diago 动态链接库
list(APPEND _diago
    \${HSOLVER_PATH}/diago_dav_subspace.cpp
    \${HSOLVER_PATH}/diago_david.cpp
    \${HSOLVER_PATH}/diag_const_nums.cpp
    \${HSOLVER_PATH}/diago_iter_assist.cpp
    \${HSOLVER_PATH}/kernels/dngvd_op.cpp
    \${HSOLVER_PATH}/kernels/math_kernel_op.cpp
    \${BASE_PATH}/kernels/math_op.cpp
    \${BASE_PATH}/module_device/device.cpp
    \${BASE_PATH}/module_device/memory_op.cpp
    \${PSI_PATH}/psi.cpp
)
add_library(diagopack SHARED \${_diago})
target_link_libraries(diagopack
    base
    parameter
    container
    orb
    \${math_libs}
    \${OpenBLAS_LIBRARIES}
    \${LAPACK_LIBRARIES}
)

list(APPEND pymodule_hsolver
    \${PROJECT_SOURCE_DIR}/src/hsolver/py_hsolver.cpp
)

# 使用 pybind11 添加 Python 模块
pybind11_add_module(_hsolver_pack MODULE \${pymodule_hsolver})
# 将依赖项和 pybind11 库链接到模块
target_link_libraries(_hsolver_pack PRIVATE pybind11::headers diagopack)
target_compile_definitions(_hsolver_pack PRIVATE VERSION_INFO=\${PROJECT_VERSION})

set_target_properties(diagopack PROPERTIES INSTALL
```

### Python 化上手指南

- Pybind 简单教程（外部链接，感觉写的比较简单，可以直接阅读官方文档）里边有我入门 pybind11 时做的一些笔记，可以理解为官方文档的部分摘抄和学习
- 阅读 pybind11 with cmake，了解如何搭配 cmake 使用 pybind11
- 我们的项目使用 pybind11+cmake+scikit-build-core 构建，所以可以下载 scikit-build-core template 仓库，搞懂里边的代码，尝试跑一个 demo 出来

  有任何与 scikit-build-core 相关的问题可以查阅 scikit-build-core 文档
- 阅读 pyabacus README，简单了解 pyabacus 目前的模块结构，尝试按照 README 编译 pyabacus 并跑通（可

以运行 python/pyabacus/examples 内的 python 脚本）

- 阅读 pyabacus How to contribute，同步阅读 pyabacus 下各文件夹内 CMakelists，理解它们之间的关系。本文档中给出了该部分的中文翻译，可对照查看。
- 同步地，可以看一下 src/hsolver 内的代码，其结构如下

  {method}.hpp 为对应算法的头文件，头文件中将 hsolver 内对应算法进行了一定的包装，加入了 pybind 的一些头文件以方便调用 py_hsolver.cpp 为 pybind11 绑定，将三个方法对应的类绑定到 python .../pyabacus/hsolver 内的两个 py 文件将类的细节隐藏，用函数在外层抽象，将三个算法以函数的形式暴露给 python

  了解了上面几点，你大概就了解了如何 Python 化 ABACUS 的一个模块，剩下需要搞懂的是
- 你要 Python 化的模块，按照面向对象的思想，你可以只搞懂对应模块的头文件中暴露出来的 public 方法的接口

  但是，如果想要优化/防止 python 化过度影响效率，最好要搞懂其原理 作为高性能计算软件，我们时常与裸指针/底层数据打交道，但是我们处理的数据通常是矩阵 or 张量，所以一定要从源码中弄清楚数据的存储方式：行优先/列优先
  - 不要想当然，作为有着” 历史积累” 的软件，不同模块可能使用不用方式存储
  - 例如：davidson 算法与 dav_subspace 中的裸指针采用列优先方式，与 fortran 一致，while CG 算法中的 tensor 的底层数据是行优先方式，与 C++ 习惯及 NumPy 中一致
- 如何编写单元测试

  可以阅读 tests 文件夹内的代码，同时查阅 pytest 文档 提交 PR 前一定要做好单元测试，确保功能没有问题！看到一片绿色写着 PASS 时候是很有成就感的

  对上面问题有了基本的了解以后，我们就可以进行开发了！祝你在开发的过程中好运

### 6.15.3 资料汇总

Pybind 简单教程
pybind11 with cmake
scikit-build-core template
scikit-build-core 文档
pyabacus README
pyabacus How to contribute
pytest 文档
寻找在 GitHub 上参与开源项目的方法
GitHub Issues 快速入门
https://zhuanlan.zhihu.com/p/697280920

### 6.15.4 结语

以上就是 Pyabacus 的一些基本安装和使用方法，如果读者对进一步交流感兴趣，欢迎登录 ABACUS 的 github 网站进行进一步的交流。

# 第 7 章 开发者规范

## 7.1 ABACUS 的 Github 仓库 Issues 处理流程

### 7.1.1 Github 的 Issues 机制

**简介 Github 的 Issues**

GitHub 上的 Issues 机制是一个用于跟踪和讨论项目相关问题、需求、任务和改进的功能。它为开发者们提供了一个集中式的平台，使得项目的参与者们能够跟踪项目进展、报告错误、提出新功能、讨论改进方向等。

**Issues 对 ABACUS 软件的作用**

ABACUS（原子算筹）是一款国产密度泛函理论（Density Functional Theory，简称 DFT）软件，其涉及到的功能繁多，仅 2023 年上半年统计到的输入参数就有三百多个，应用的对象也很广，在材料、物理、化学、能源和工程等多个领域都有应用。因此，常见的问题是开发者开发出的功能不能包含所有可能的应用对象，因此软件在某些特定场合出现效率不高甚至某个功能无法运行的情况是时有发生的。另外一方面，ABACUS 项目参与的开发人员广泛，不同开发者对同一个软件代码进行修改，容易引发新的 bug。因此，建立一个良好的用户和开发者，乃至开发者之间互动交流的平台，对于发现软件问题，不断改善软件质量和提高稳定性易用性方面具有重要作用。Github 上的 Issues 机制则承担起了这一部分功能。因此，将 Issues 机制建立好是不断提升 ABACUS 软件，使其快速迭代的重要措施。

**Issues 功能介绍**

- **创建 Issue**: 项目的任何参与者都可以新建一个 Issue。创建 Issue 时，需要填写标题、描述（可以使用 Markdown 语法格式化描述内容）、分配处理人（Assignees，可选）、添加分类标签（Labels，可选）、关联项目看板（Projects，可选）和设置里程碑（Milestones，可选）。
- **标签（Labels）**: 标签是用来对 Issues 进行分类和过滤的。项目维护者可以创建自定义标签，例如：`bug`、`feature`、`documentation`、`enhancement` 等。这有助于组织和优先处理不同类型的问题。
- **里程碑（Milestones）**：里程碑用于对 Issues 进行分阶段管理和规划。项目维护者可以创建里程碑并为其分配截止日期，这样有助于按照计划进行项目开发。
- **指派（Assignees）**：在 Issue 上指派处理人意味着将该 Issue 的负责人分配给特定的项目参与者。这有助于清楚地了解谁负责解决某个问题。
- **评论和讨论**: 项目的参与者们可以在 Issue 下面发表评论，进行讨论。评论支持 Markdown 语法，因此可以方便地插入代码、图片和链接等。
- **引用和关联 Issue**: 在评论和提交（Commit）中，可以通过输入类似 `#issue` 编号的格式引用其他 Issue，这有助于将讨论串联起来。此外，还可以使用关键字（如：`close`、`fix`、`resolve` 等）在提交信息中自动关闭相关 Issue。
- **通知和订阅**: 当您参与某个 Issue 时（如创建、评论或被指派），如果您希望及时收到回复，建议您关注是否会收到邮件通知（关注一下 Github 的功能）
- **关闭和重开 Issue**: 当某个问题被解决或者不再适用时，项目维护者或 Issue 创建者可以关闭 Issue。如果后续发现问题没有解决或者需要重新讨论，可以重新打开 Issue。

### 7.1.2 ABACUS 项目的 Issues 标签分类

提交 Issues 会让提交者（提交者可以是用户或者开发者）选择标签，以下是 ABACUS 开发者团队推荐的标签及其使用场景

**分类：**

- `Bug Report`（开发者/用户）：用于报告代码问题，例如崩溃、错误或意外行为。
- `Code Quality`（开发者）：用于报告与代码质量相关的问题，例如代码不规范、设计不良、全局类或去全局变量使用不当、或者可维护性问题，开发者会用重构来解决相关问题。
- `Docs`（开发者/用户）：用于报告与软件文档相关的问题，例如缺少或不完整的文档，或难以理解的文档。
- `Feature Request`（开发者/用户）：用于请求新功能或对现有功能进行增强。
- `Performance`（开发者/用户）：用于报告与用户角度下软件性能相关的问题，例如响应时间慢、资源使用率高或可扩展性问题。
- `Questions`（开发者/用户）：软件安装、软件易用性等用户或开发者需要帮助解决的问题。
- `Tests`（开发者）：用于报告与测试相关的问题，例如测试失败、缺少或不完整的测试或测试自动化问题

**优先级：**

- `Priority High`：通常指的是需要尽快解决的紧急问题，如 CI 不通过，新 PR 全部报错
- `Priority Medium`：重要且需要近期解决的问题，如...
- `Priority Low`：可能是一些不紧急但需要解决的问题，如需求调整、性能优化

**专题：**

- `DFTU`, `EXX`, `big system` 等用于记录一些专题类 Issues，以方便开发者系统解决同一个主题的 Issues

### 7.1.3 Issues 处理流程

**介绍**

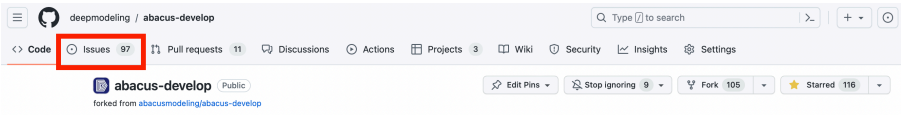
提交的 Issues 会在 24 小时内得到开发者团队的回复。当前 Issues 处理的闭环方式有：

- 提交 PR 修改代码，解决对应 Issues，之后关闭相应的 Issue。
- ISSUE 历史大于两周以上，且 ISSUE 提出者不对有效回应做回应，则提醒用户后关闭
- 属于 Discussion 机制而不属于 Issues 的范围，会转移到 Github 的 Discussion 模块：例如用户的想法，用户的建议，用户对社区的问题等以用户的满意程度为指标的问题

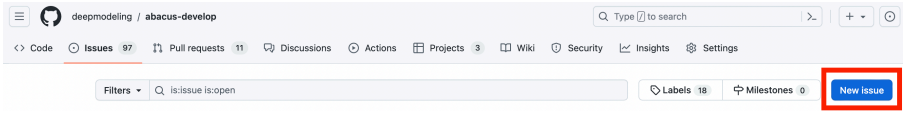
**创建一个 ISSUE**

有多种方法可以创建 ISSUE，您可以选择最方便的方法来。本例将使用 GitHub UI。有关其他创建 ISSUE 的方法的更多信息，请参考 Github 文档 “创建 ISSUE”。以下是具体步骤：

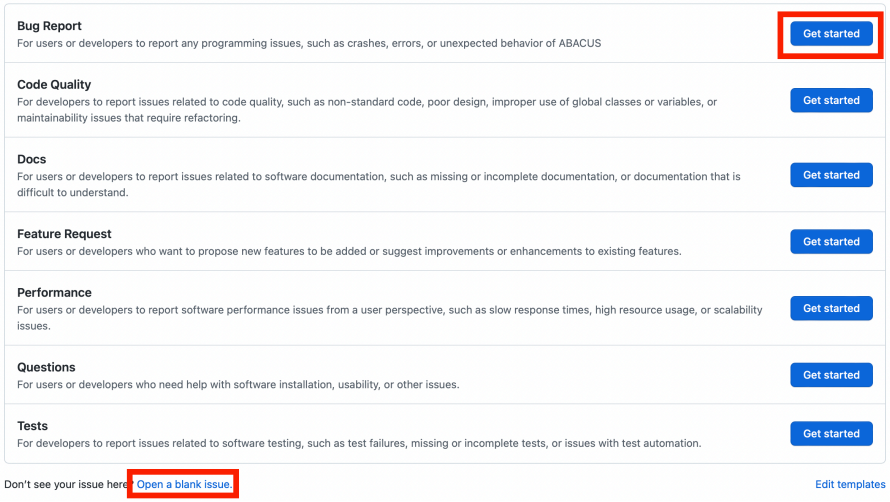
1. 在 GitHub 上，导航到 ABACUS 仓库的主页。https://github.com/deepmodeling/abacus-develop.git
2. 在 ABACUS 仓库名称下面，单击 “Issues”。




3. 单击 “New issue”。



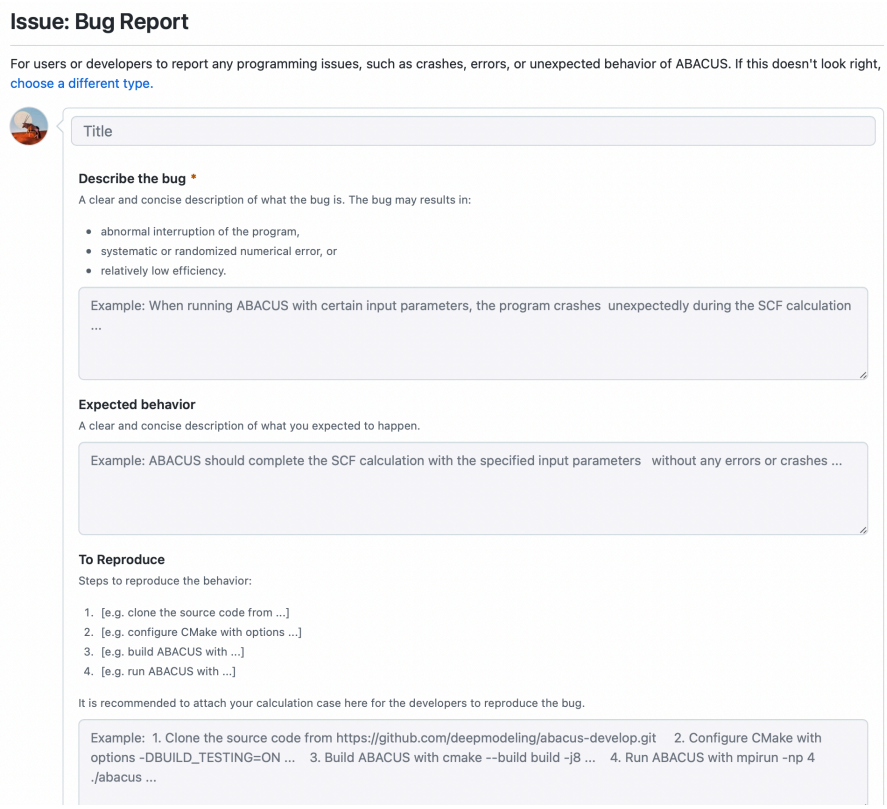

4. 选择一个最适合您的 ISSUE 的一个标签模版，点击“Get started”。如果 ABACUS 提供的默认模版不符合您的 ISSUE 类型，可以选择“Open a blank issue”。




### 填入 ISSUE 信息

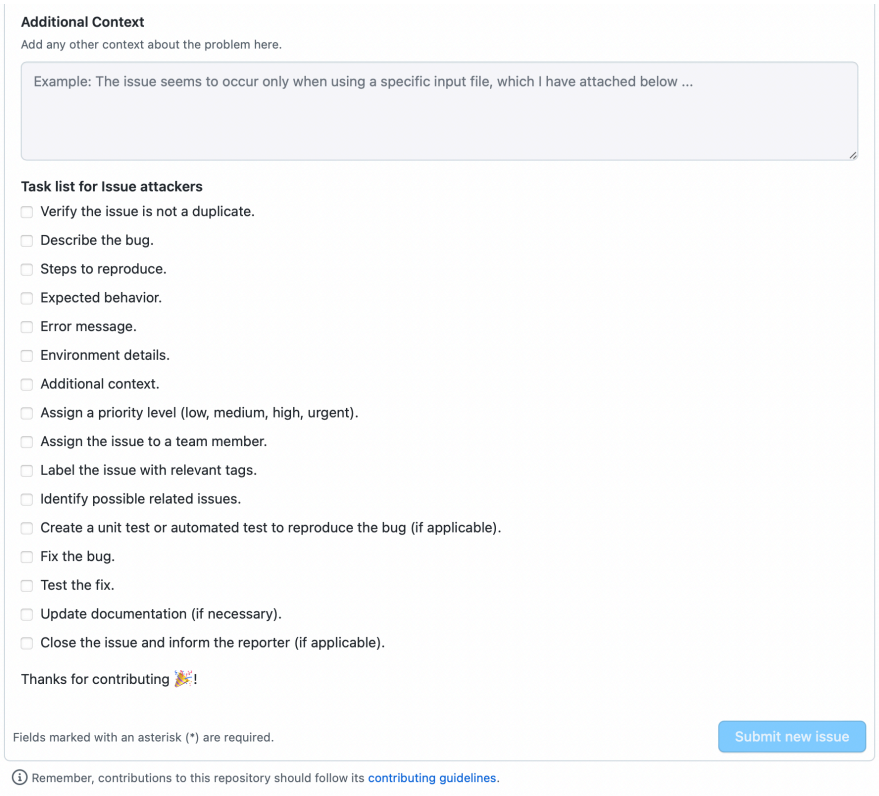
给您的 ISSUE 一个描述性的标题。标题应该一眼就能传达问题的内容。添加一个解释 ISSUE 目的的描述，包括任何可能有助于解决问题的细节。例如，如果这是一个 bug 报告，描述重现 bug 的步骤、预期结果和实际结果。您可以使用 Markdown 添加格式、链接、表情等。有关更多信息，可参阅“在 GitHub 上撰写”。

以下是 ABACUS 的 Bug report 类型 ISSUE 的模版，您可以根据 ISSUE 模版示例来填入 ISSUE 信息。其中带“*”的项是必填项。



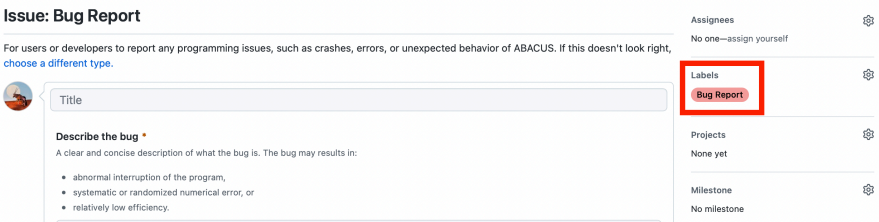

### ISSUE 的任务列表

请注意“Task list for Issue attackers”是为了帮助开发者检查 ISSUE 解决的进度，并不是要求您选择解决 ISSUE 需要完成哪些工作。所以您只需要提供足够详细的信息以帮助开发者来解决 ISSUE。如果您提供的信息已经完成其中的一些任务列表，可以在相应的任务前打对勾，但是如果没有可以不用关心“Task list for Issue attackers”。当然，了解解决一个 ISSUE 所需要经过的一般步骤，有助于您了解 ABACUS 代码的开发过程。如果您想通过自定义的任务列表给开发者一些提示，可以参考 Github 文档“About task lists”。

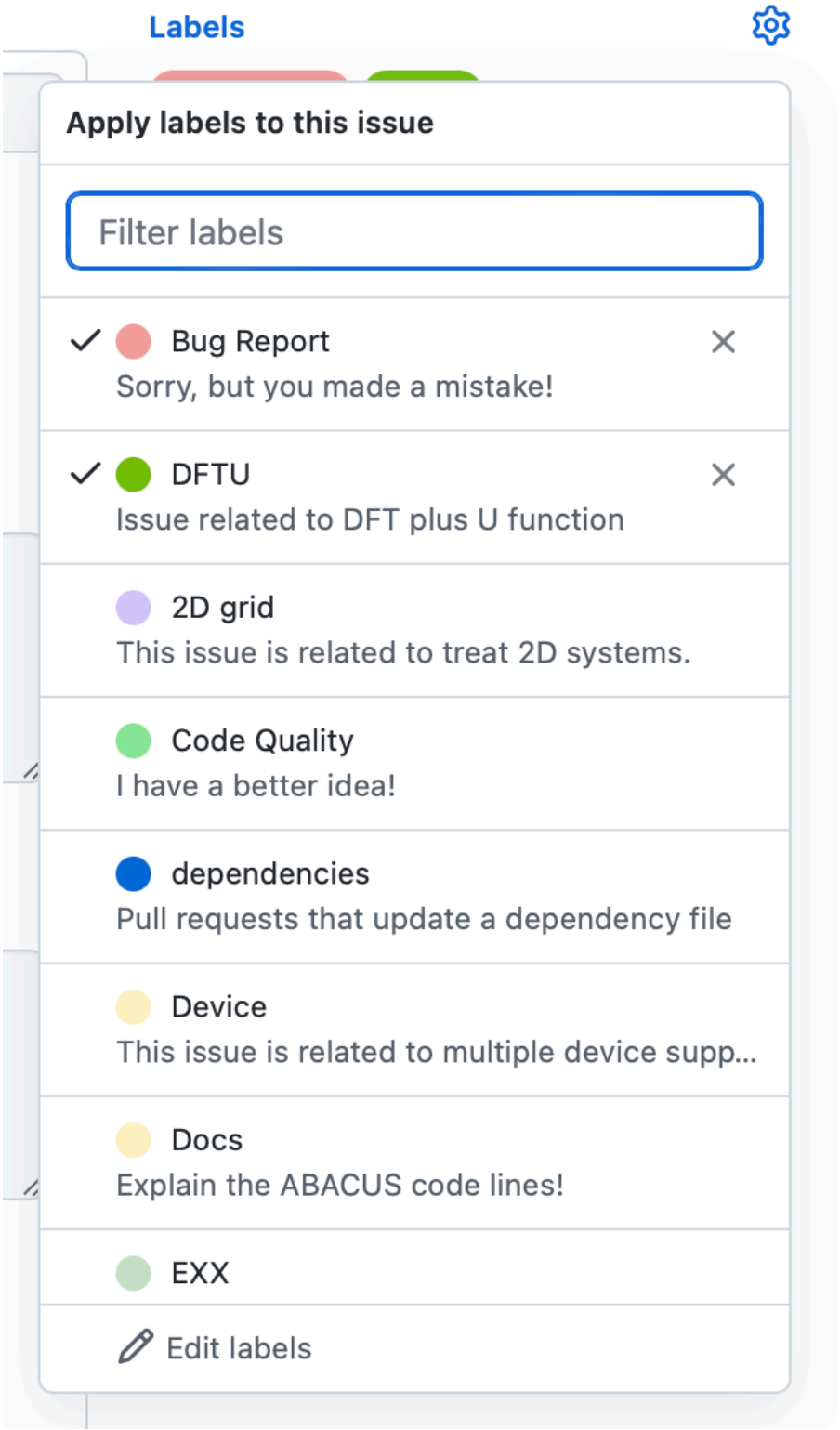



### 添加 ISSUE 标签

如果您选择 ISSUE 模版，创建完 ISSUE 之后会自然有相应的标签出现，如：

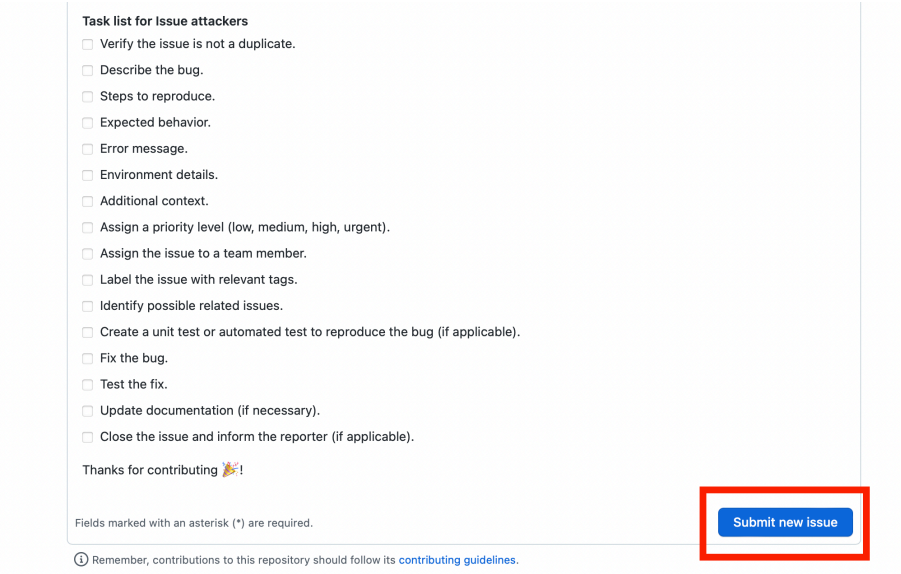



此外，您也可以参考 ABACUS 的专题标签（例如 DFTU, EXX 等）添加相应的标签，或者通过“Edit labels”创建新标签：



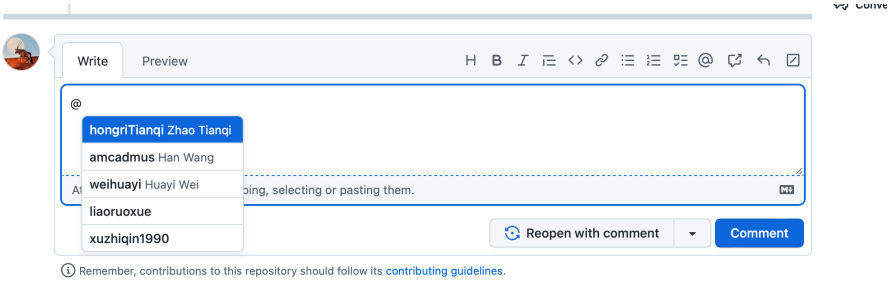

**提交 ISSUE**

点击“Submit new issue”以创建您的 ISSUE。您可以在创建问题后编辑上述任何字段。您的 ISSUE 有一个唯一的 URL，您可以与团队成员分享，或在其他 ISSUE 或 Pull Requat 中引用。

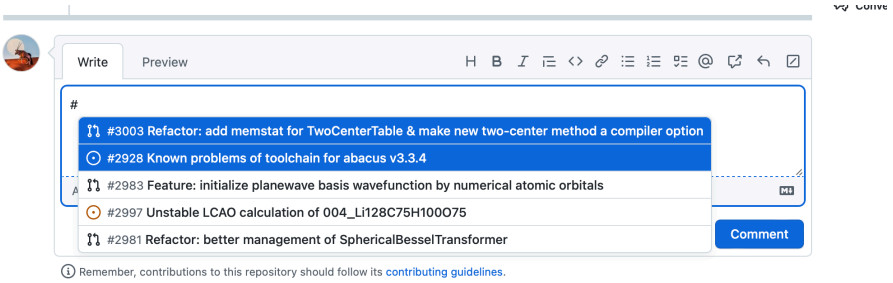



### 与开发者交流

创建 ISSUE 后，通过添加评论与开发者或其他协作者交流对话。您可以键入 @ 邀请协作者或团队关注评论。




要在同一仓库中可以链接相关问题，您可以键入 #，后跟 ISSUE 标号或部分标题，然后单击要链接的问题。有关更多信息，请参见 ““在 GitHub 上撰写””。

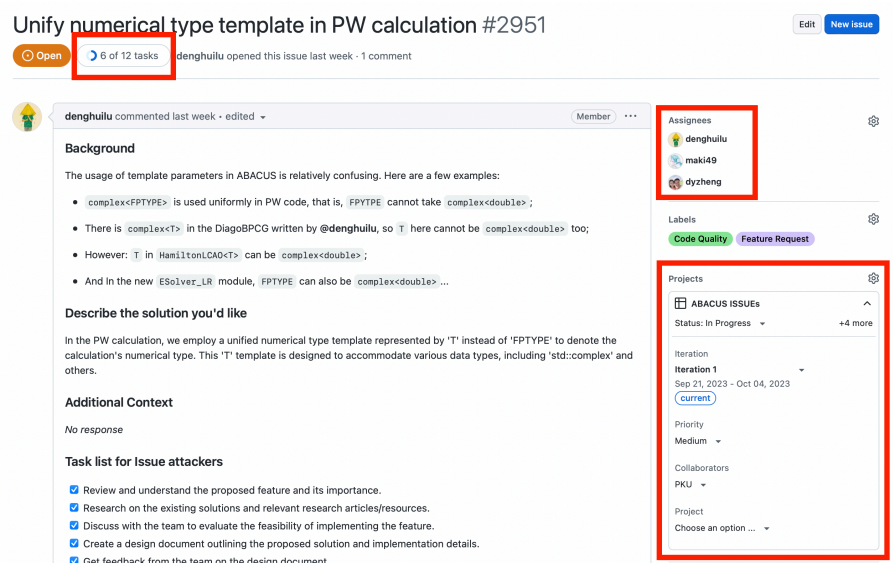



### 查看 ISSUE 解决进度

查看 ISSUE 解决有 3 种方式，如下图所示：
可以通过“task list”的完成进度查看 ISSUE 的解决进度

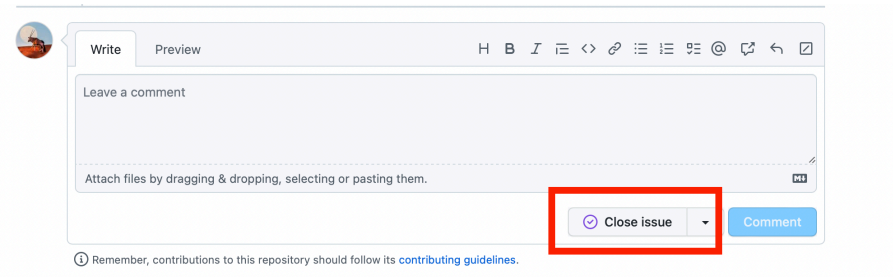
新建的 ISSUE 会被自动加入 ABACUS ISSUEs 项目，开发者通过讨论之后会在其中加入”Iteration” 解决周期，”Priority” 优先级，”Collaborators” 开发者单位，”Project” 是否纳入近期的开发者项目等信息，您也可以在这里看到 ISSUEs 的进展。

Assignees 是解决 ISSUE 的开发者，需要 ABACUS 管理员进行分诊。

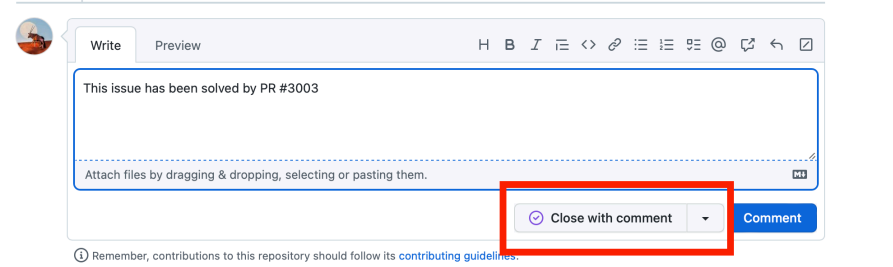



## 关闭 ISSUE

以第 1 步和第 2 步同样的方式找到想要关闭的 ISSUE。点击 “Close issue”，关闭 ISSUE。




如果要添加关闭 ISSUE 的原因，可以在 “Leave a comment” 处添加关闭原因，然后点击 “Close with comment”，关闭 ISSUE。



# 7.2 ABACUS 开源项目 C++ 代码规范

## 7.2.1 前言

让任何开发者都可以快速读懂别人的代码，这点对于 ABACUS 项目很重要。此外，使代码易于管理的方法之一是加强代码一致性，这需要从代码规范开始着手。

## 7.2.2 命名约定

**普通变量的命名统一是小写，并使用下划线命名法**

以下是四种命名方法：下划线命名法、匈牙利命名法、驼峰式命名法、帕斯卡命名法

```
<strong>string table_name; </strong>// 下划线命名法（推荐） - 用下划线区分单词（推荐使用）
string sTableName; // 匈牙利命名法 - 前缀字母用变量类型缩写，单词首字母大写
string tableName;  // 驼峰命名法 - 混合大小写（不推荐使用）
string TableName; // 帕斯卡命名法 - 每个单词首字母大写（不推荐使用）
```

下划线命名法：推荐使用

匈牙利命名法：目前 ABACUS 中有些变量（例如指针），首字母为数据类型（指针为 p），可以考虑用这种方式命名，但不推荐用大写字母

驼峰命名法/帕斯卡命名法：目前 ABACUS 大部分的代码不是用这种命名方式的，为了代码风格统一，所以不推荐使用

**函数命名**

建议使用下划线命名法且所有字母全部小写，建议小于 18 个字符

```
void calculate_area() {
  // function body
}
```

**private 的类数据成员在最后以下划线结尾**

private 的类数据成员和普通变量的命名方式一样，但要在最后以下划线结尾，以区分自己是 private 的类数据成员。

```
class TableInfo
{
...
private:
    string table_name_;  // 好 - 后加下划线
    string tablename_;   // 好
    static Pool<TableInfo>* pool_;  // 好
};
```

**只有全局变量用全大写命名**

在 ABACUS 中，例如 NBANDS（能带数），NLOCAL（局域轨道数目）这种全大写的变量名是全局变量，其它的不建议用大写字母给变量命名。另外，ABACUS 将通过重构或者删除的方式逐步淘汰全局变量，因此不建议增加新的全局变量。

### 7.2.3 关于头文件

**只有在真正需要使用某个库/头文件时才 #include 它们**

不需要的 #include 删除。

**#include 避免使用快捷目录. (当前目录) 或.. (上级目录)**

项目内头文件应按照项目源代码目录的树结构排列，避免使用 UNIX 特殊的快捷目录. (当前目录) 或.. (上级目录)。例如

source/module_hsolver/diago_cg.h 文件是这样引用 module_base 的头文件：

```
#include "diagh.h"
#include "module_base/complexmatrix.h"
```

**#include 顺序（clang-format 可以自动）**

使用标准的头文件包含顺序，这样可以可增强可读性，避免隐藏依赖。

foo.cpp 中包含头文件的次序如下：

与源文件对应的头文件：dir2/foo2.h（这种优先的顺序排序保证当 dir2/foo2.h 遗漏某些必要的库时，dir/foo.cpp 的构建会立刻中止）

C 系统文件

C++ 系统文件

其他库的.h 文件

本项目内.h 文件

**头文件需要 #define 保护**

```
#ifndef FOO_BAR_BAZ_H_
#define FOO_BAR_BAZ_H_
...
#endif // FOO_BAR_BAZ_H_
```

**避免使用前置声明**

使用 #include 包含需要的头文件，尽量避免使用前置声明（见以下例子解释什么是前置声明）。

```
// 什么是前置声明？
// MyClassB.h

// Bad: Overuse of forward declarations
class MyClassB;

```

```
class MyClassA {
public:
    void DoSomething(MyClassB* obj_b);
};

void MyClassA::DoSomething(MyClassB* obj_b) {
    // ...
}
```

在这段代码中，MyClassB 被声明为一个类，但没有给出其定义。这被称为前置声明（Forward Declaration），它告诉编译器 MyClassB 是一个存在的类，但不提供该类的详细信息。

**当函数只有 10 行甚至更少时，才将其定义为内联函数**

只有当函数只有 10 行甚至更少时才将其定义为内联函数（关键字 inline）。

### 7.2.4 关于类（Class）

**先声明类的成员变量，再声类的成员函数**

**将 public 部分放在最前，之后是 protected 数据，最后是 private**

**不使用运算符重载**

若要使用，建议提交 issue 讨论

**什么时候用 struct？**

只有数据成员、没有成员函数时可以用 struct

**类型转换使用 static_cast<>()**

建议使用 C++ 的类型转换，如 static_cast<>()。

不要使用 int y = (int)x 或 int y = int(x) 等转换方式。

用 static_cast 替代 C 风格的值转换，或某个类指针需要明确的向上转换为父类指针时。

用 const_cast 去掉 const 限定符。

用 reinterpret_cast 指针类型和整型或其它指针之间进行不安全的相互转换。仅在你对所做一切了然于心时使用。

**继承采用 public 继承**

如果你想使用私有继承，你应该替换成把基类的实例作为成员对象的方式去定义类。

**不过度使用实现继承，可以考虑组合**

例如，如下案例：

```
class Engine {
public:
    void start() { /* 启动引擎 */ }
};
```

```
5
6  class Car {
7  public:
8      Car(const std::string& n, Engine& e) : name_(n), engine_(e) {}
9      void drive() { engine_.start(); std::cout << name_ << " is driving\n"; }
10
11 private:
12     std::string name_;
13     Engine& engine_;
14 };
15
16 int main() {
17     Engine engine;
18     Car car("mycar", engine);
19     car.drive();
20     return 0;
21 }
```

### 7.2.5 关于函数（Function）

**函数传参：不变参数用 const 和 &**

函数的输入参数与输出参数:

在一个函数中，不变的量，我们可以看作是函数的输入参数；变化的量，我们可以看作是函数的输出参数。

在输入参数中可以选择 const T* （指向常量对象的指针，不能通过这个指针来修改其指向的对象的值。然而，你可以改变指针本身的值）。

也可以使用 const T& （不能通过这个引用来修改其引用的对象的值，在其生命周期内不能重新引用另一个对象）。

所以，建议使用 const T&，若要使用 const T*，则应给出相应的理由，否则会使读者感到迷惑。

```
1 void Foo(const string &in, string *out);
2 // 输入参数：in （const + 引用 &）
3 // 输出参数：out（指针变量）
```

**函数传参：会变的参数用指针**

什么是引用参数:

在 C 中, 如果函数需要修改输入变量的值, 参数必须为指针, 如 int foo(int *pval)。

在 C++ 中, 函数还可以声明为引用参数: int foo(int &val)。

引用参数在语法上是值变量却拥有指针的语义（变量可以被改变!）。

**每个函数不超过 50 行**

建议编写简短，凝练的函数，有特殊情况的除外。

**函数返回值（return 的值）多使用值返回和引用返回，避免使用指针返回**

### 7.2.6 关于作用域

**变量要初始化**

无初始化的变量可能会引起结果不稳定（例如出现随机数），因此建议养成习惯，对所有变量的值要初始化，见下面的例子：

```
int i;
i = f(); // 坏——初始化和声明分离

int j = f(); // 好——初始化时声明
```

```
vector<int> v;  // 坏——初始化和声明分离
v.push_back(1);
v.push_back(2);

vector<int> v = {1, 2}; // 好——初始化时声明
```

**将局部变量置于最小作用域**

在 if, while 和 for 语句中：

变量不是对象，则变量应当在内部声明与初始化，这样子这些变量的作用域就被限制在这些语句中了，举例而言：

```
while (const char* p = strchr(str, '/'))
{
str = p + 1;
}
```

如果变量是一个对象，每次进入作用域都要调用其构造函数，每次退出作用域都要调用其析构函数。这会导致效率降低。建议将构造函数调用次数减少以提高程序效率。

```
Foo f;  // 构造函数和析构函数只调用 1 次
for (int i = 0; i < 1000000; ++i)
{
    f.DoSomething(i);
}

// 低效的实现
for (int i = 0; i < 1000000; ++i)
{
    Foo f;  // 构造函数和析构函数分别调用 1000000 次!
    f.DoSomething(i);
}
```

### 仅在局部作用域使用 using namespace

使用 using namespace 语句来引入命名空间中的所有名称可能会导致名称冲突。因此，建议在需要时使用它并仅在局部作用域内使用（例如只在调用的时候使用）。

```
// 建议不在程序开头使用，而是在具体用到 std 库的函数内使用该语句
using namespace std;
```

### 用全局函数要加命名空间

以下是两种建议的方式，或者用类，或者用 namespace

```
// 类的静态成员函数
class MyMath {
public:
    static int add(int a, int b) {
        return a + b;
    }

    static int sub(int a, int b) {
        return a - b;
    }
};

// or
// 命名空间内的非成员函数
namespace my_math {
    int add(int a, int b) {
        return a + b;
    }

    int sub(int a, int b) {
        return a - b;
    }
}
```

### 优先使用命名空间的非成员函数

能使用命名空间的非成员函数，就不用类的静态成员函数。

```
// 使用命名空间内的非成员函数
namespace my_math {
    int add(int a, int b) {
        return a + b;
    }

    int sub(int a, int b) {
        return a - b;
    }
```

```
}

int main() {
    int x = 3, y = 5;
    int z = my_math::add(x, y); // 调用 my_math 命名空间中的函数
    return 0;
}
```

```
// 不要使用类的静态成员函数模拟命名空间
class MyMath {
public:
    static int add(int a, int b) {
        return a + b;
    }

    static int sub(int a, int b) {
        return a - b;
    }
};

int main() {
    int x = 3, y = 5;
    int z = MyMath::add(x, y); // 调用 MyMath 静态方法
    return 0;
}
```

### 7.2.7 其他 C++ 特性

**每个代码文件不超过 500 行**

太长的代码阅读理解和维护的成本都太高，因此不建议一个文件太长。如果有文件超过 500 行，建议重构，把对象进一步的划分。

**禁止用 C++11 之后版本的语法**

目前的主要考虑是用新语法会使得编译器编译成功的概率降低，另外提高开发者的开发门槛。因此，我们规定不能使用 C++11 之后的语法。

**多用前置自增 (++i)**

对于迭代器和其他模板对象使用前缀形式 (++i) 的自增，自减运算符。
不考虑返回值的话，前置自增 (++i) 通常要比后置自增 (i++) 效率更高

**尽可能用 sizeof(a) 代替 sizeof(int)**

这里 a 是一个参数名
原因：当代码中变量类型改变时会自动更新。

**多使用列表初始化**

C++11 中，该特性得到进一步的推广，任何对象类型都可以被列表初始化。示范如下：

```
// Vector 接收了一个初始化列表。
// 不考虑细节上的微妙差别，大致上相同。
// 可以任选其一。
vector<string> v{"foo", "bar"};
vector<string> v = {"foo", "bar"};

// 可以配合 new 一起用。
auto p = new vector<string>{"foo", "bar"};

// map 接收了一些 pair，列表初始化大显神威。
map<int, string> m = {{1, "one"}, {2, "2"}};

// 初始化列表也可以用在返回类型上的隐式转换。
vector<int> test_function() { return {1, 2, 3}; }

// 初始化列表可迭代。
for (int i : {-1, -2, -3}) {}

// 在函数调用里用列表初始化。
void TestFunction2(vector<int> v) {}
TestFunction2({1, 2, 3});
```

**初始化时：整数用 0，实数用 0.0，指针用 nullptr，字符 (串) 用'0'**

C++11 引入了一个新的关键字 nullptr，用来表示空指针。相对于传统的 NULL 或 0，nullptr 更加明确、类型安全。使用 nullptr 可以避免一些潜在的编程错误，比如将整数值误传给函数，导致出现不可预期的行为。

因此，建议在 C++11 及以上的版本中使用 nullptr 来表示空指针。

**少用 auto，使用前需对 auto 有更全面的了解**

auto 是 C++11 引入的关键字，它可以让编译器自动推导出变量的类型。之后，C++14 和 C++17 对 auto 的使用也有了一些扩展和改进。

C++11：auto 只能用于定义局部变量，并且必须初始化。例如：

```
auto i = 42; // 推导出 i 的类型为 int
auto f = 3.14f; // 推导出 f 的类型为 float
auto s = "hello"; // 推导出 s 的类型为 const char*
```

C++14：auto 可以用于定义函数返回值类型，使得函数定义更加简洁。例如：

```
auto add(int x, int y) { return x + y; } // 推导出返回类型为 int
auto divide(double x, double y) { return x / y; } // 推导出返回类型为 double
```

C++17：auto 进一步扩展为 auto 和模板结合使用时，可以直接指定模板类型参数，从而实现更加灵活的类型推导。例如：

```
std::vector<int> v{1, 2, 3};
auto it = v.begin(); // 推导出 it 的类型为 std::vector<int>::iterator
auto [first, second] = std::make_pair(1, 3.14); // 使用结构化绑定和 auto 推导出 first 和 second 的类型
```

但是大家注意，在 abacus 中，我们只支持 C++11 的标准，C++14/17 语法是不接受的。

**auto 和 for 的混合使用时注意事项**

在 C++11 中，auto 和 for 循环的结合使用已成为一种常见的编程范式，它可以让代码更加简洁、易读，并且减少了手动指定类型的错误。

```
int arr[] = {1, 2, 3};

for (auto i : arr)  // 相当于复制
{
    std::cout << i << " ";
}

for (const auto& i : arr) // 常量引用
{
    std::cout << i << " ";
}

for (auto& i : arr) // 能够实现对元素的直接修改。
{
    i *= 2;
}
```

**constexpr 替代宏定义和 const 常量**

在 C++11 里，用 constexpr 来定义真正的常量，或实现常量初始化。（真正的常量在编译时和运行时都不变）

```
#define PI 3.14159 // PI 是一个宏定义常量，它不会进行类型检查，容易出错；
const double kE = 2.71828; // kE 是一个 const 常量，它不能用于编译期计算。

constexpr double kGravity = 9.8;
```

constexpr 可以替代宏定义和 const 常量的主要原因是：

类型安全：使用 constexpr 定义的常量会进行类型检查，避免了宏定义可能带来的类型错误，同时也比 const 常量更加严格。

编译时计算：constexpr 声明的变量或函数在编译时就可以被求值，而不需要在运行时计算。这比宏定义和 const 常量更高效，尤其是在需要多次使用同一个值的情况下。

更好的可读性和可维护性：使用 constexpr 可以使代码更加清晰易懂，减少了宏定义可能导致的代码混乱问题。同时，由于 constexpr 可以使用函数、类等 C++ 语言特性，因此更加灵活，对于复杂的计算也更容易维护和修改。

因此，在 C++11 及以上的版本中，建议使用 constexpr 来替代宏定义和 const 常量，以提高代码的可读性、可维护性和效率。

### 7.2.8 关于 ABACUS 中常用的关键词缩写

有些名字很长，我们希望尽量言简意赅的表达出一些关键词的意思。原则是一般 3-5 个字母的范围下尽量说清楚一个变量的含义。这些统一的命名会出现在函数名或者变量名里。

**两个字符**

pw：代表 plane wave 平面波
op：代表具有 multi-device 和 multi-precision 支持的算子（operator），和 Operator 模块含义不同

**三个字符**

fft：快速傅里叶变换
kpt：布里渊区 kpoint 的缩写
nao：代表 numerical atomic orbitals （nao 经常用来表示 number of atomic orbitals，不知道会不会混）
orb：orbital，轨道
hmt：代表 hamilt 或者 hamiltonian
pot：代表 potential
chg：代表 charge
den：代表 density（电荷密度尽量都用 chg）
scf：代表自洽迭代 self consistent field
thr：代表 threshold
tab：代表 table
kin：代表 kinetic，动能的
cal：代表 calculate
opt：代表 optimize
gen：代表 generate

**四个字符**

iter：代表 iteration
init：代表初始化 initializaiton
read：读入
stru：代表 structure
veff：代表有效势
vloc：代表局域势

### 7.2.9 Reference

Google C++ Style Guide

# 7.3 ABACUS 注释规范：Doxygen 入门 (c++)

## 7.3.1 简介

开源密度泛函理论软件 ABACUS 目前由众多社区开发者共同开发，因此有一个统一的代码注释规范是非常重要的，本文主要推荐以 Doxygen 工具为辅助的代码注释格式。

### 什么是 Doxygen

Doxygen 是一款支持包括 C/C++, Python, Fortran 等多种语言的静态程序分析与文档生成工具。通过搜集并整理代码中的信息，包括但不限于

函数的调用/被调用

类的成员以及继承、协作关系

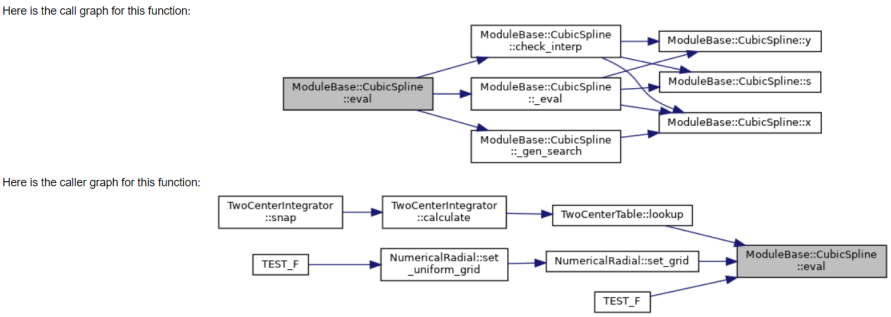
文件夹之间的依赖关系

头文件 include 关系

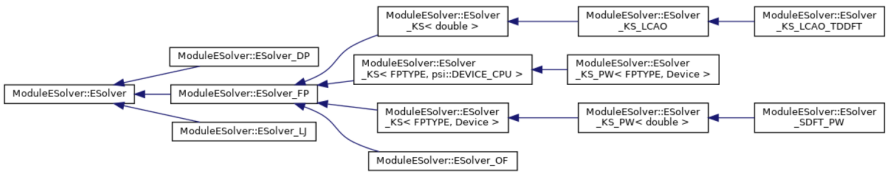
Doxygen 不仅能自动生成包含以上信息的图表与列表，也能将代码中采用特定格式的注释附到相应位置，最终生成内容翔实的文档。在适当的 Doxygen 配置与良好的代码注释下，Doxygen 自动生成的文档能为开发者提供诸多便利，降低阅读代码的难度，从而促进开发者社区的建设。

### 功能与文档范例

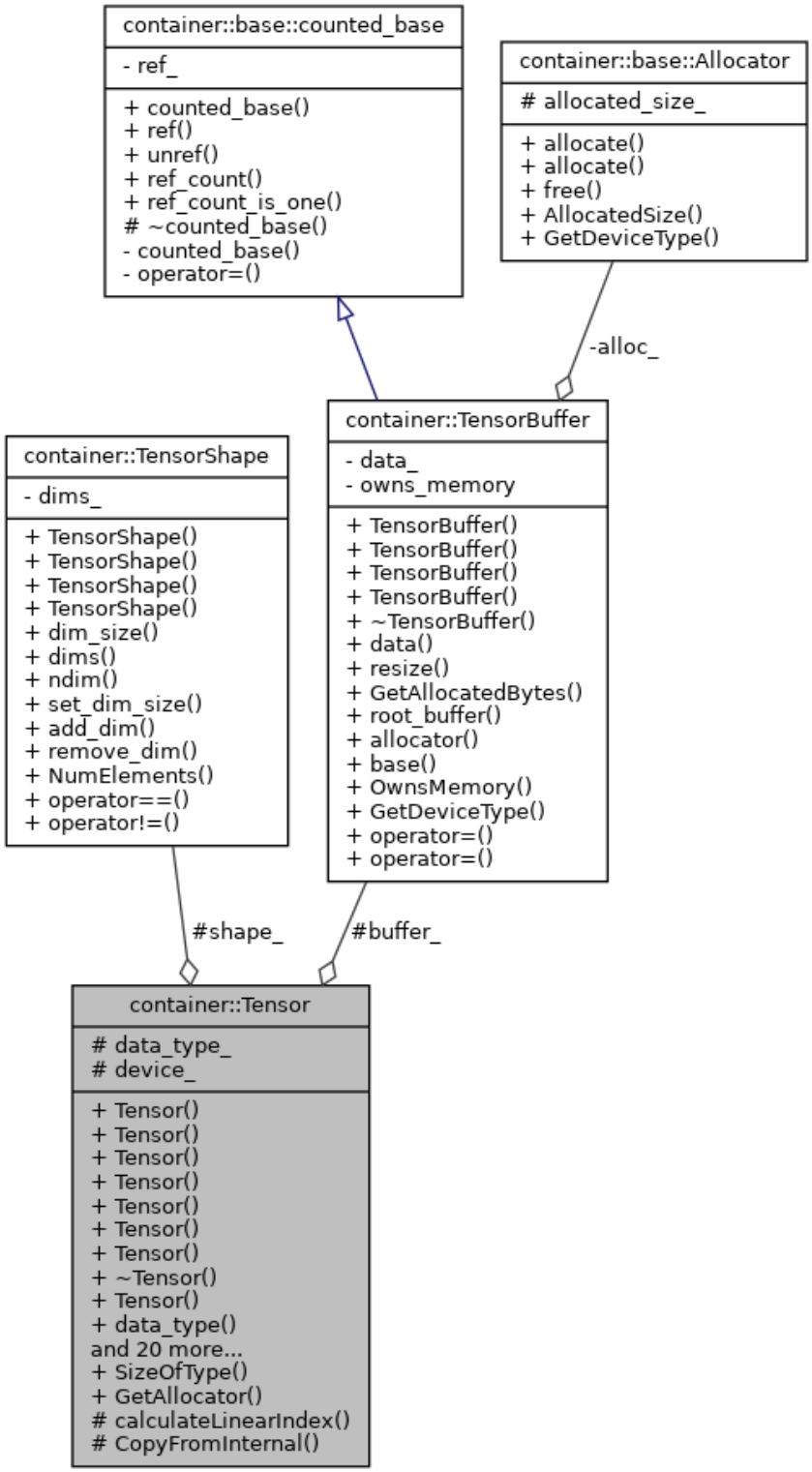
Doxygen 可以生成某个函数的调用与被调用图 (call & caller graph)。以开源密度泛函理论软件 ABACUS 代码为例，其中负责计算三次样条插值的函数 ModuleBase::CubicSpline::eval，doxygen 可将其调用关系自动可视化为如下图表：



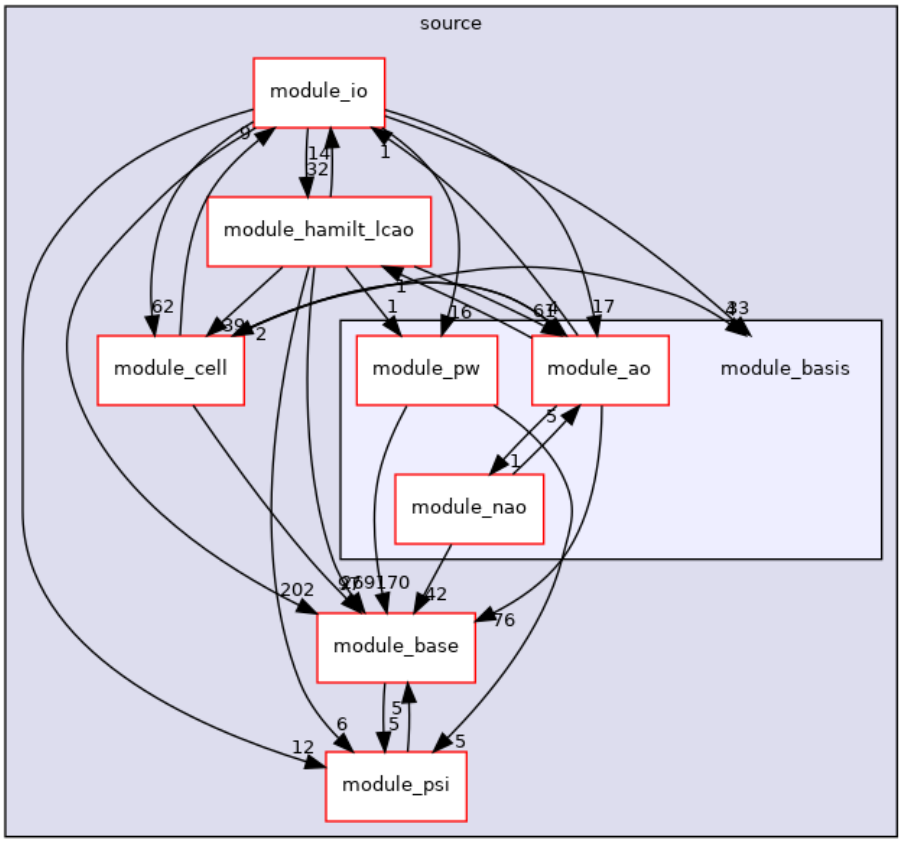

Doxygen 可以生成类的协作图（collaboration diagram）。下图展示了 container::Tensor 的成员以及成员的成员之间的关系：



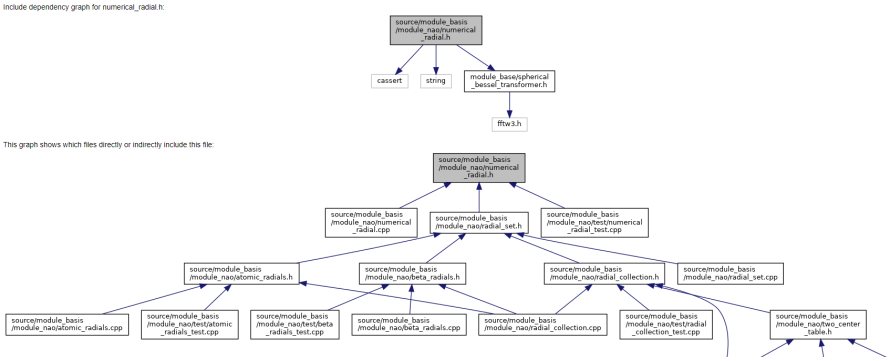

Doxygen 可以生成文件夹之间的依赖关系图。下图展示了 source/module_basis 及其内部成员文件夹与其他文件夹的耦合。



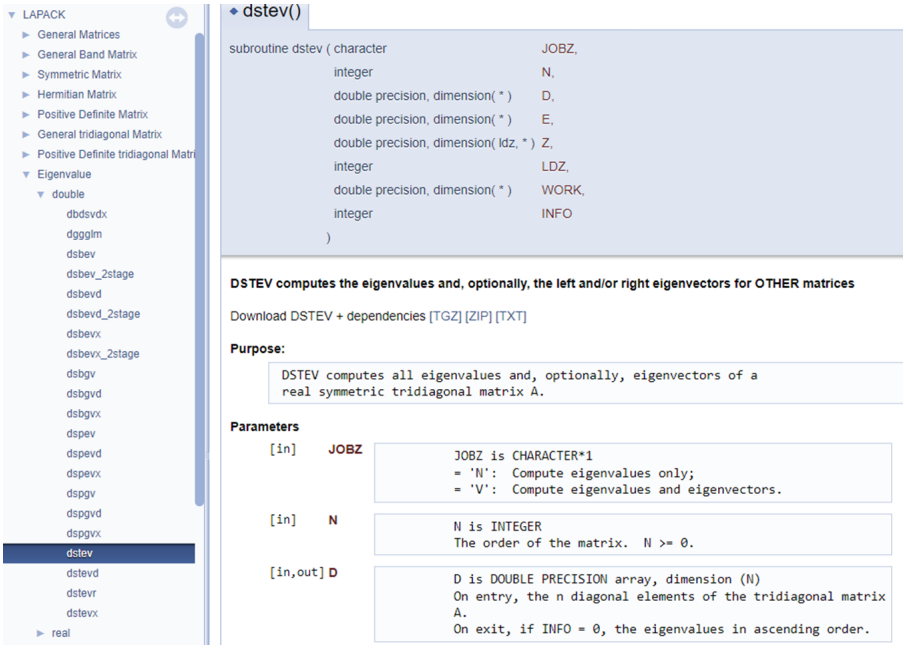

Doxygen 可以生成文件夹之间的依赖关系图。下图展示了 source/module_basis 及其内部成员文件夹与其他文件夹的耦合。




Doxygen 可以将头文件的 include 关系可视化成图表 (include dependency graph)。下图（部分地）展示了 module_basis/module_nao/numerical_radial.h 包含与被包含的情况：




大量项目使用 Doxygen 生成文档 (https://www.doxygen.nl/projects.html)。对科学计算开发者而言一个著名的例子是 LAPACK 的在线文档 https://www.netlib.org/lapack/explore-html/




## 7.3.2 如何使用 Doxygen

Doxygen 可以自动识别代码逻辑并生成以上例子中的函数调用、类的继承与协作、文件夹依赖以及头文件依赖等图表。但是，对理想的开发者文档而言代码注释中的信息亦不可或缺。在最终生成的文档中，除以上自动生成的图表外，Doxygen 会将类的成员与函数一一列举，并识别代码中特定格式的注释后将其附在对应位置。

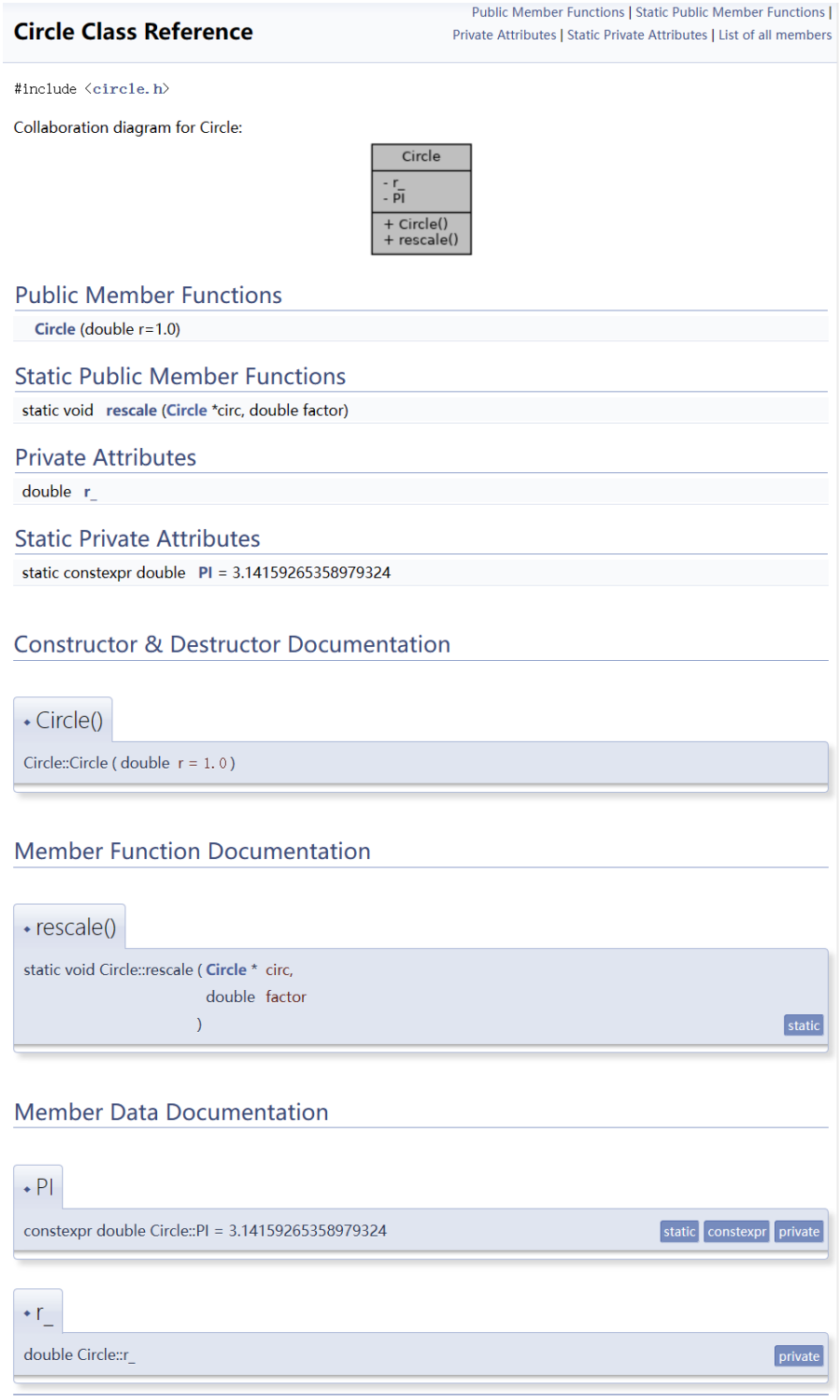
### 如何写能被 Doxygen 识别的注释

通常开发者会希望一部分精心撰写的注释——比如类的功能介绍或者函数参数的含义——出现在开发者文档里，而不希望文档充斥着临时变量的描述或某一行意义的解释。Doxygen 充分考虑了这一点。事实上，普通的注释均会被 Doxygen 忽略；只有特定格式的注释会被 Doxygen 识别。下面的代码是一个简单的例子。尽管其中各处都附上了注释，但这些均不会出现在最终生成的文档中：

```
// A class that represents a circle.
// This toy class will be used to demonstrate the usage of Doxygen.
class Circle {
public:
    // Initializes a Circle object with a given radius.
    // A unit circle is assumed by default.
    Circle(double r = 1.0);

    // Rescales a Circle object.
    // The radius of the given Circle object is multiplied by the given factor.
    static void rescale(Circle* circ, // pointer to the Circle object
                        double factor // radius rescaling factor
                       );

private:
    double r_; // radius of the Circle object
    static constexpr double PI = 3.14159265358979324;
```

```
18 };
```

以上代码生成的文档如下。Doxygen 会自动将成员变量和函数一一列举，但由于注释不符合特定格式，因而不会出现在文档相应位置。



**Circle Class Reference**

Public Member Functions | Static Public Member Functions | Private Attributes | Static Private Attributes | List of all members

#include <circle.h>

Collaboration diagram for Circle:



## Public Member Functions

**Circle** (double r=1.0)

## Static Public Member Functions

static void **rescale** (**Circle** *circ, double factor)

## Private Attributes

double **r_**

## Static Private Attributes

static constexpr double **PI** = 3.14159265358979324

## Constructor & Destructor Documentation

### Circle()

Circle::Circle ( double r = 1.0 )

## Member Function Documentation

### rescale()

static void Circle::rescale ( **Circle** * circ,
double factor
)

static

## Member Data Documentation

### PI

constexpr double Circle::PI = 3.14159265358979324

static constexpr private

### r_

double Circle::r_

private

下面介绍如何写能被 Doxygen 识别的注释以及一些基本用法。

### 基本格式

在 c++ 中 Doxygen 会识别以下三种注释格式：

Javadoc-style

```
/**
 *  This is a Javadoc-style comment
 */
```

Qt-style

```
/*!
 * This is a Qt-style comment
 */
```

Extended c++ style

```
/// This is the so-called extended c++-style comment
//! This also works!
```

以上三种格式本身均是合法的 C++ 注释，只是在 // 和 / / 的基础上添加了额外的'*'，'!' 或'/' 。一旦文档中出现了以上三者之一，Doxygen 便会将之后的注释内容填入文档的相应位置。这三种基本格式是所有 Doxygen 用法的基础，所有的进阶功能（如 group，module，equation 等）均需在这三者之一开启的注释行/块内实现。这三种基本格式可以混用，开发者可以按自己的习惯选择一种或多种使用。

Doxygen 默认注释作用于之后的类、变量或函数（允许有空行）。如我们希望注释作用于之前的对象，则可在基本格式后立刻添加'<' 符号，例如

```
//!  this comment line applies to print(), even though there's an empty line below

void print();
///<  this comment line also applies to print(), not get(); note the '<' character!
int get();
```

### 详细与简略描述

如之前例子所示，Doxygen 自动生成类的文档时会先把成员函数与变量罗列在开头，随后才是每个成员的介绍（当然，由于之前的例子里没有采用 doxygen 格式，详细描述为空）。只要使用了基本格式，Doxygen 就会自动将注释附在对应成员处。比如若将之前构造函数前的 // 注释替换为 /// :

```
/// Initializes a Circle object with a given radius.
/// A unit circle is assumed by default.
Circle(double r = 1.0);
```

这些内容即会出现在原先为空的 Constructor & Destructor Documentation 下 Circle() 的描述里:

**Circle Class Reference**

Public Member Functions | Static Public Member Functions | Private Attributes | Static Private Attributes | List of all members

A class that represents a circle. More...

#include <circle.h>

Collaboration diagram for Circle:



Public Member Functions

**Circle** (double r=1.0)
Initializes a **Circle** object with a given radius. More...

Static Public Member Functions

static void **rescale** (**Circle** *circ, double factor)
Rescales a **Circle** object. More...

Private Attributes

double **r_**
radius of the **Circle** object More...

Static Private Attributes

static constexpr double **PI** = 3.14159265358979324

Detailed Description

A class that represents a circle.

This toy class will be used to demonstrate the usage of Doxygen

Constructor & Destructor Documentation

Circle()

Circle::Circle ( double r = 1.0 )

Initializes a **Circle** object with a given radius.

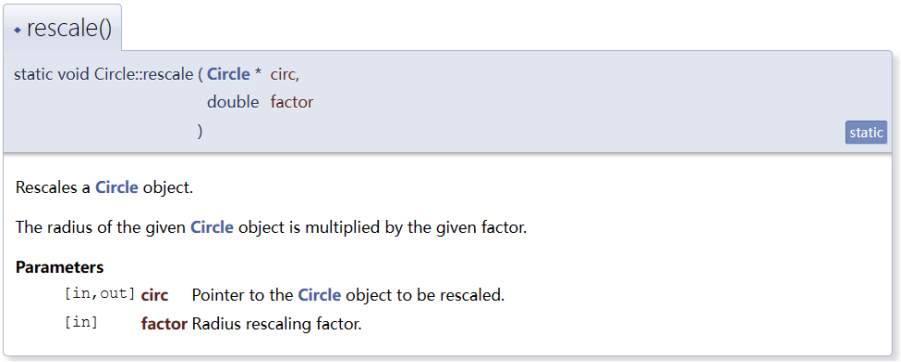
A unit circle is assumed by default.

Doxygen 还允许让一些简略描述出现在类文档的开头部分。有两种方式可以实现这一目的。较为正式的方式是在注释块里添加 @brief 或者 \brief。以这种方式指定的简略描述可以跨多行（如下面开头 Circle 类的注释）。另一种较为轻便的方式是使用单行的 extended C++ style 注释，如下面 Circle 的构造函数。【多行的 extended c++ style 注释只会被当做详细描述】

```
/**
 * @brief A class that represents
 *        a circle.
 *
 * This toy class will be used to demonstrate the usage of Doxygen
 */
class Circle {
public:
    /// Initializes a Circle object with a given radius.
```

```
    /*!
     * A unit circle is assumed by default.
     */
    Circle(double r = 1.0);

    /*!
     * \brief Rescales a Circle object.
     *
     * The radius of the given Circle object is multiplied by the given factor.
     *
     */
    static void rescale(Circle* circ,
                        double factor
                       );

private:
    double r_; ///< radius of the Circle object
    static constexpr double PI = 3.14159265358979324;
};
```

以上代码生成的 Doxygen 文档在开头罗列部分会出现对应的简略描述；详细介绍部分则会包括所有的注释（包括 brief）。




### 函数参数

一个开发者写的函数经常会被很多其他开发者使用。除对函数整体功能的描述外，每个参数含义的注释也十分重要。Doxygen 支持两种给函数参数添加注释的方式。较为正式的方式是在注释块内添加 @param 或 \param：

```
/*!
 * \brief Rescales a Circle object.
 *
 * The radius of the given Circle object is multiplied by the given factor.
 *
 * @param [in,out] circ     Pointer to the Circle object to be rescaled.
 * @param [in]     factor   Radius rescaling factor.
 */
static void rescale(Circle* circ,
```

```
                    double factor
                   );
```

较为轻便的方式是在函数声明的参数列表里直接添加（加在参数后注意要使用'<'）

```
/*!
 * \brief Rescales a Circle object.
 *
 * The radius of the given Circle object is multiplied by the given factor.
 */
static void rescale(Circle* circ, ///<[in,out] Pointer to the Circle object to be rescaled.
                    double factor ///<[in] Radius rescaling factor.
                   );
```

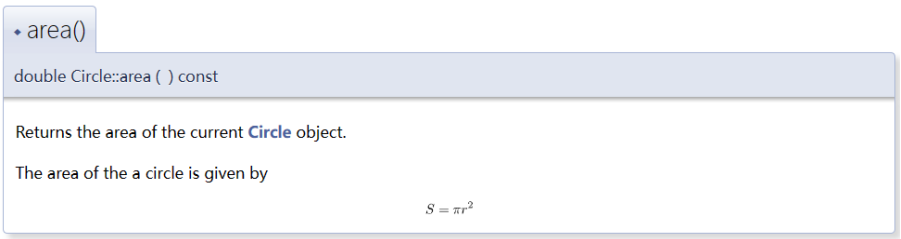
[in], [out] 或 [in, out] 可以提示该参数是输入还是输出参数。这并非 Doxygen 的强制要求，但十分推荐。

以上两种参数注释方式对 Doxygen 而言是等价的，开发者可按习惯选择一种使用，生成的函数描述如下：

### 数学公式

有时一些函数严格对应了某个公式，这种情况下直接用数学公式作为注释的效果可能胜过大段的文字描述。Doxygen 允许开发者以 Latex 的语法实现公式注释。如有在文字中嵌入公式的需求，可以采用 \f$ ... \f$：

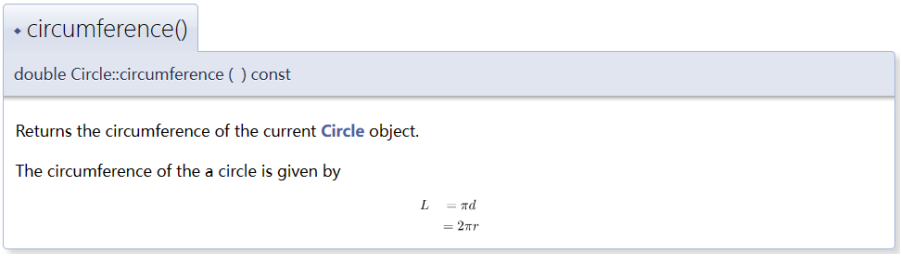
```
/// A constant that represents the value of \f\$ \pi \f\$.
    static constexpr double PI = 3.14159265358979324;
```




单行公式由 \f[ ... \f] 实现：

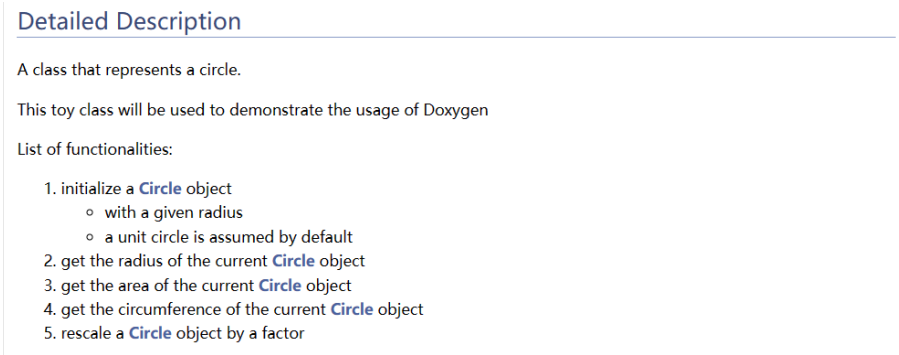
```
/*!
 * @brief Returns the area of the current Circle object.
 *
 * The area of the a circle is given by
 *
 * \f[ S = \pi r^2 \f]
```

```
 */
double area() const;
```




如有多行公式的需求，Doxygen 支持调用 eqnarray 等 latex 数学环境：

```
/*!
 * @brief Returns the circumference of the current Circle object.
 *
 * The circumference of the a circle is given by
 *
 * \f{eqnarray*}{
 *      L &= \pi d \\
 *        &= 2 \pi r
 * \f}
 *
 */
double circumference() const;
```




【在配置文件中采用“USE_MATHJAX = YES”可以调用 mathjax 渲染公式得到更好的视觉效果】

**列表**

Doxygen 还有大量用法可供用户生成理想的文档。
对齐的’-’, ’+’ 或’*’ 可自动生成列表 (bullet list)；’-#’ 则可用来生成带序号的列表：

```
/**
 * @brief A class that represents
 *        a circle.
 *
 * This toy class will be used to demonstrate the usage of Doxygen
 *
 * List of functionalities:
 * -# initialize a Circle object with a given radius
 *   - a unit circle is assumed by default
 * -# get the radius of the current Circle object
 * -# get the area of the current Circle object
 * -# get the circumference of the current Circle object
 * -# rescale a Circle object by a factor
 *
 */
class Circle                                */                                          */
```

Public Member Functions

Circle (double r=1.0)
Initializes a Circle object with a given radius. More...

**Properties**

Functions that return the properties of the current Circle object.

double radius () const
Gets the radius of the current Circle object. More...

double area () const
Returns the area of the current Circle object. More...

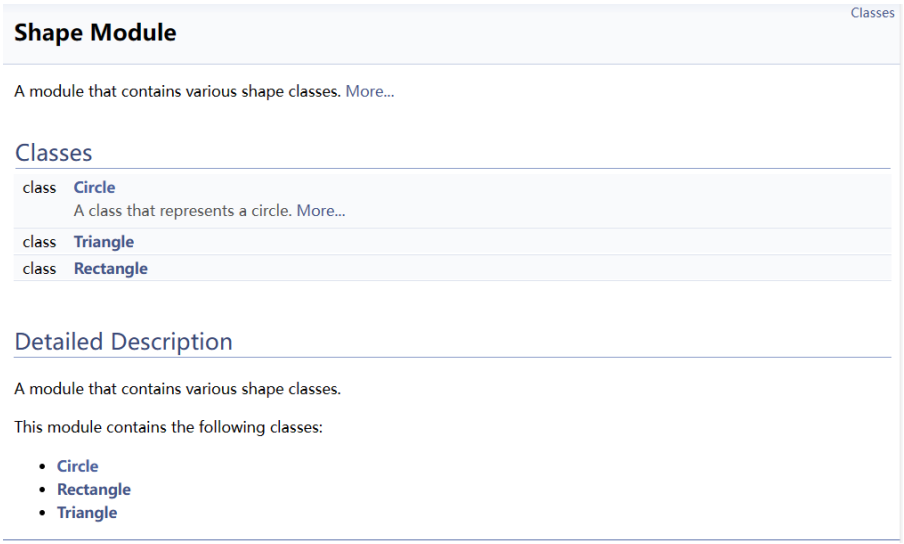
double circumference () const
Returns the circumference of the current Circle object. More...

### Member Group & Module

变量或函数可以由 @ ... "@ 归类为 member group 并附上相应注释，这能使文档更有层次感：

```
///@{ @name Properties
/// Functions that return the properties of the current Circle object.

/// Gets the radius of the current Circle object.
double radius() const { return r_; }

/*!
 * @brief Returns the area of the current Circle object.
 *
 * The area of the a circle is given by
 *
 * \f[ S = \pi r^2 \f]
 */
double area() const;

```

```
/*!
 * @brief Returns the circumference of the current Circle object.
 *
 * The circumference of the a circle is given by
 *
 * \f{eqnarray*}{
 *      L &= \pi d \\
 *        &= 2 \pi r
 * \f}
 *
 */
double circumference() const;
///@}
```



Classes

**Shape Module**

A module that contains various shape classes. More...

Classes

| | |
|---|---|
| class | **Circle**<br>A class that represents a circle. More... |
| class | **Triangle** |
| class | **Rectangle** |

Detailed Description

A module that contains various shape classes.

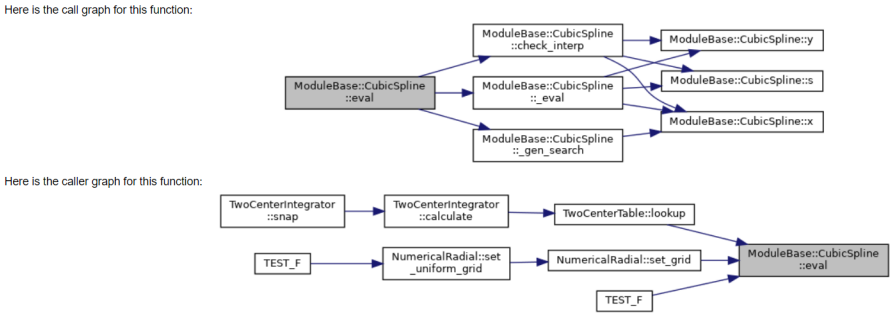
This module contains the following classes:

- **Circle**
- **Rectangle**
- **Triangle**

有共性的类之间亦可归类组成 Modules:

```
/// @defgroup shape Shape Module
/// @brief A module that contains various shape classes.
///
/// This module contains the following classes:
/// - Circle
/// - Rectangle
/// - Triangle

/// @addtogroup shape
/// @{
class Circle {
...
};
///@}

```

```
/// @addtogroup shape
/// @{
class Rectangle {
...
};
///@}

/// @addtogroup shape
/// @{
class Triangle {
...
};
///@}
```




更多用法可参考官方文档 Doxygen Manual: Overview

**如何生成文档**

上一节介绍了 Doxygen 支持的注释格式以及一些常见的使用方式。以下以 Ubuntu 用户为例介绍让开发者在本地生成 html 文档的方法：

1. 安装 doxygen 与 graphviz

```
sudo apt install doxygen graphviz
```

2. 生成 Doxygen 的配置文件

```
doxygen -g  <your_preferred_doxyfile_name>
```

默认的配置文件名为 Doxyfile。

3. 修改配置文件

配置文件内已经包含介绍各选项的注释。常用的选项包括：

INPUT：用以生成文档的文件夹或文件路径

RECURSIVE：是否遍历给定路径下的所有子文件夹

EXCLUDE：是否要排除一些子文件夹或子文件

EXTRACT_PRIVATE, EXTRACT_STATIC, EXTRACT_PRIV_VIRTUAL：文档是否要包括私有成员/静态成员/私有虚函数

CALL_GRAPH, CALLER_GRAPH: 文档是否要包含函数调用/被调图
PREDEFINED：预编译宏
MACRO_EXPANSION：是否要展开宏定义
EXPAND_ONLY_PREDEF：是否只展开 PREDEFINED 的宏定义
一个可供参考的 Doxyfile 可见于 Doxygen 使用 （赵天琦）
完整的配置选项可参考官网 Doxygen Manual: Configuration
4. 运行 doxygen
假设配置文件叫 my_doxyfile：

```
doxygen my_doxyfile
```

### 7.3.3 附录：Doxygen 与 VS Code

Visual Studio Code 上有 doxygen 插件 Doxygen Documentation Generator - Visual Studio Marketplace 可供 VS Code 用户方便使用。

【注：前文中有提到对 doxygen 而言三种基本注释格式可以混用；混用的注释会合并在一起作用于注释后的对象。这个特性与 VS Code 中鼠标悬停于类或函数上时出现的预览窗口不兼容；如注释格式混用则会导致前一种格式的注释被忽略】

## 7.4 ABACUS 线上文档输入参数撰写规范

### 7.4.1 背景

ABACUS 代码仓库地址：https://github.com/deepmodeling/abacus-develop
ABACUS 线上文档地址：https://abacus.deepmodeling.com/en/latest/
（ABACUS 线上文档是包括开发者在内的所有用户了解 ABACUS 软件使用方法的重要渠道。）
本文档关注：“关键字”部分的文档
对应的文档地址：https://abacus.deepmodeling.com/en/latest/advanced/input_files/input-main.html
对应的代码地址：https://github.com/deepmodeling/abacus-develop/blob/develop/docs/advanced/input_files/input-main.md
更新方式：目前对该文档的更新同样是通过 PR 的方式提交到 ABACUS 的 Github 仓库上，因此任何人都可以通过提交 PR 的方式对文档做出修改，我们也鼓励用户如果发现问题后可以主动修复文档。

### 7.4.2 规范格式

每个输入参数都应遵循以下格式：
Name of Parameter Type: Availability: (可选) Description: Default: Unit: (可选) Example: (可选)
按照上述顺序排列，上述所有字段需加粗，后面使用英文冒号（不加粗），冒号后空一格开始写内容（不加粗）
上述关键字用黑色实心圆，关键字后的介绍不用加粗
错误示范：Type:Integer 错误示范：Type: integer 错误示范：Type: integer 正确示范：Type: Integer 错误示范：Default: 0 错误示范: Default: false 正确示范：Default: False

#### Name of Parameter

这个参数代表了参数的名称，建议起参数名称的时候遵从以下规范（xxx 由设定参数名的开发者指定）：

- 输出某物理量的参数：建议以 `out_xxx` 为参数名，其中 out 是 output 的缩写。可参考的变量名有 `out_chg`（输出电荷密度），`out_pot`（输出势函数），`out_dm`（输出密度矩阵 density matrix）
- 收敛阈值相关参数：建议以 `xxx_thr` 为参数名，其中 thr 是 threshold 的缩写。可参考的变量名有 `scf_thr`（电子自洽迭代的收敛阈值），`force_thr`（结构驰豫时原子中的最大受力域值），和 `stress_thr`（晶格优化时的应力收敛阈值）
- 计算某物理量的参数：建议以 `cal_xxx` 为参数名，其中 cal 是 calculate 的缩写
- 初始化某物理量的参数：建议以 `init_xxx` 为参数名，其中 init 是 initialize 的缩写。可参考的变量名有 `init_wfc`（初始化波函数），`init_chg`（初始化电荷密度），`init_vel`（初始化原子速度）
- 结构驰豫相关参数：建议以 `relax_xxx` 为参数名，其中 relax 代表（结构）驰豫
- 并行策略相关参数：建议以 `xxxpar` 为参数名，其中 par 是代表 parallel，即并行。可参考的变量名有 `kpar`（k 点并行），`bndpar`（能带并行）
- 其它参数：建议先寻找 ABACUS 目前支持的参数名来设计相关参数

**Type**

Type：冒号之后首字母大写，输入参数的种类，有 Integer，Real 和 String 三种类型。对于 Boolean 类型，Default 字段后的内容统一写：True 或 False（首字母大写），且先写 True，再写 False

注意 Real 类型不要写成 double 或者 float 类型

**Availability（Optional）**

如果是完整的一句话就首字母大写，如果是单词或短语就小写

**Description**

- 冒号之后首字母大写
- 如果有 Note 的标注，可在 Description 字段下，单独写一个 Note，不用加粗。
- Description 字段里如果有涉及到 1 和 0 是表示 True 或 False，统一改成 True 或 False。
- 字段如果是完整句子，末尾要加英文句号”:”。若是单词和短语则不加。其他字段均不加。
- String 类型的参数直接列出所有选项
- 当一个参数有多种取值时，按照从小到大顺序描述不同的取值，例如 propagator 这个参数，就在 description 里面描述该参数是 0、1、2 三种情况下的波函数演化方法。
- 不写 “when set to”、“if set to”，直接写参数后，再写该参数的描述

**Default**

参数默认的数值，分成有单位的和没有单位的两种。如果是小数，建议采用科学计数法表示，“e” 一律小写，如: 2e-6

**Unit（Optional）**

如有需要使用 Unit 来标定输入参数的单位，紧接在 Unit 后面描述单位，另外内容里如果涉及到写单位，需要用空格隔开，如：U (eV)

单位撰写需符合标准：

错误示范：KBar 错误示范：ev 正确示范：kbar 正确示范：eV

**Example（Optional）**

如有案例，可添加相关例子的链接和简单描述。如果需要，可以添加上述的 Example 字段描述案例。

### 7.4.3 其他问题

- 所有简写应改大写，以及后面用括号写出全称，每一个关键字的描述出现了简写均需要全称，如：“soc”改成“SOC（spin-orbital-coupling）”。即使最常见的如 PW、MD 等也均写出全称
- 对参数的描述保持客观，如：不写“An important parameter”这种话
- 相关的参数建议放在文档上靠近的地方。举例：symmetry 参数是控制是否打开对称性操作的参数，而 symmetry_prec 是用来控制对称性操作的精度，这两个参数就可以放在一起先后介绍，并且在介绍的时候，可以提及相关的参数
- 一些太长的输出，单独开辟一个专区去介绍
- 关键字告诉了用户输出了文件，建议告诉用户文件名称和文件保存的路径
- 注意语法问题
- 标点符号格式要一致
- 注意不要出现乱码

以上若有问题或者建议，欢迎写信给作者！虽然我们邮箱基本不会收到类似邮件，但开源精神的意义就在于提供了这种可能性，然后坚信更好的事情会到来。

## 7.5 ABACUS 代码存放规范

### 7.5.1 背景

截止到 ABACUS 3.4 版本，ABACUS 的 Github 仓库（https://github.com/deepmodeling/abacus-develop）代码已超过 30 万行，涉及上千个源代码文件，有时一周内在 Github 仓库上就会 Merge 近 20 个 PR，涉及数千乃至上万行代码的修改。随着社区规模不断壮大，为了更好的在各个开发者之间协同合作，在过去已有的一些代码管理的共识的基础上，我们提出这个代码存放规范文档。

### 7.5.2 代码目录

进入主仓库之后，ABACUS 的源代码目录位于 source 文件夹下：

```
\$ tree source -d -L 1
source
   module_base            # 基础模块
   module_basis           #LCAO 和 PW 基组模块
   module_cell            # 晶胞模块
   module_elecstate       # 电子波函数相关性质模块
   module_esolver         # 能量求解器模块
   module_hamilt_general  # 对于 PW 和 LCAO 基组共享的哈密顿量模块
   module_hamilt_lcao     #LCAO 基组下哈密顿量的相关模块
   module_hamilt_pw       #PW 基组下哈密顿量求解的相关模块
   module_hsolver         # 对角化求解矩阵特征值/特征向量等哈密顿求解器模块
   module_io              # 输入输出模块
   module_md              # 分子动力学模块
   module_psi             # 波函数模块
   module_relax           # 结构优化模块
   module_ri              #Resolution of identity 相关的 Beyond DFT 的模块
```

其中，14 个主模块的核心功能如上面注释所言，代码存放建议按照模块化管理，相关功能的代码存放到对

应的模块当中。如果需要涉及到新建模块以及更新模块位置等需求，应该按照 ABACUS 的 Github 仓库 Issues 处理流程 · GitBook 中相关规范要求去 Github 仓库提交 ISSUE 并邀请开发者讨论来进行。

### 模块内部代码存放规范

以 module_base 目录为例，该目录下包含了 ABACUS 中内存管理，错误处理，基础通讯以及基础算法等等功能的代码实现，是 ABACUS 中最底层的代码，其他模块大多依赖于 module_base。该模块中的代码分布如下，相应文件夹功能如注释所言：

```
\$ tree source/module_base -d -L 1
source/module_base
CMakeLists.txt
   abfs-vector3_order.cpp      # 基础代码
   abfs-vector3_order.h        # 基础代码
   ...
   CMakeLists.txt              # 负责 module_base 中 cmake 编译的 CMakeLists.txt
   kernels                     #module_base 中的异构计算代码
   libm                        #libm 模块，包含一些优化过后的数学库运算
   module_container            #ABACUS 中底层容器 Tensor 的实现
   module_mixing               #charge-mixing 实现
   test                        #module_base 中的单元测试
   test_parallel               #module_base 中并行相关的单元测试
   ...
   ylm.cpp                     # 基础代码
   ylm.h                       # 基础代码
```

我们提出如下的模块代码存放规范：

1. 基础代码存放规范：模块中的基础代码（除去部分特殊代码外的大部分代码）应直接存放到当前模块目录下，头文件以.h 命名；源文件以.cpp 命名；代码规范应该遵循 ABACUS 开源项目 C++ 代码规范 · GitBook 中的要求。

2. 异构计算代码存放规范：模块中涉及到异构计算（CPU 结合 GPU 计算）部分的代码应直接存放于当前模块的 kernels 目录下；这部分代码的结构将会在稍后详细介绍。

3. 子模块代码存放规范：

在主模块下同样可能存在一些子模块，它们通常是具有特定功能的代码的集合：以 module_base 举例，libm，module_container 以及 module_mixing 分别表示了 libm 模块（后续建议改成 module_libm 模块），ABACUS 中底层 Tensor 容器模块（用来方便在不同硬件上存放数据）以及 charge-mixing 相关的功能模块，它们为 module_base 模块下的子模块，子模块中同样也可能有类似于主模块的文件代码排布。

如果要在当前模块中新建或者调整子模块应当去 Github 仓库提交相关的 ISSUE 讨论。

4. 单元测试代码存放规范：

模块中代码的单元测试应当存放到当前模块的 test 目录下，特殊情况可以存放到以 test 开头的目录下面。例如，test_parallel 存放了一些并行计算的单元测试，因为 test 目录下面的 CMakeList 文件有时候会把 MPI 宏定义去除，再比如 test_mpi 里面只测试了 mpi 代码，没有打开 OPENMP。module_base 下面还有一个针对数值精度的测试，命名成了 test_precision。

当前模块的子模块的单元测试应当存放到相应子模块的 test 目录下；

5.CMakeLists.txt 存放规范：各个模块以及子模块通过各自目录中的 CMakeLists.txt 管理 CMake 编译结构，添加或修改代码要注意维护 CMake 编译的完备性，此外还应该维护 Makefile 编译的实现，确保代码可以通过 Github 的 CI 测试。

**异构计算代码存放规范**

当前 ABACUS 中 PW 部分代码已经实现了初步的 GPU 化，Source 目录中包含 GPU 的 module 如下，相应文件夹功能如注释所言：

```
module_base           # 基础模块
    kernels           #module_base 模块下的异构计算代码
        cuda          #module_base/kernels 中函数的 CUDA 实现
        rocm          #module_base/kernels 中函数的 ROCm/DCU 实现
        test          #module_base/kernels 中函数的单元测试
    libm              # 子模块 libm
        test          #libm 的单元测试
    module_container  # 子模块，负责实现 ABACUS 中的底层 Tensor 容器
        ATen          # 定义了 Tensor 容器的基本实现
        base          # 定义了一些 Tensor 对象无关的基础函数
        test          #module_container 的单元测试
    test              #module_base 的单元测试
        data          #module_base 的但愿测试依赖的数据
    test_parallel     #module_base 的并行测试
<strong>...</strong>
module_elecstate      # 电子波函数相关性质模块
    kernels           #module_elecstate 中相关的异构计算代码
        cuda
        rocm
        test
    module_charge     #charge 子模块
    module_dm         #dm 子模块
        test          #module_dm 的单元测试
    potentials        #potentials 子模块
    test              #module_elecstate 中的单元测试
        support       # 放置单元测试需要的输入文件
    test_mpi          # 只测试 mpi 并行代码的单元测试
<strong>...</strong>
module_hamilt_pw      #PW 基组下哈密顿量求解的相关模块
    hamilt_ofdft      #OFDFT 子模块
    hamilt_pwdft      #PWDFT 子模块
        kernels
        operator_pw
        test
    hamilt_stodft
module_hsolver        # 对角化求解矩阵特征值/特征向量的模块
    genelpa
    kernels
        cuda
        rocm
        test
    test
<strong>...</strong>
module_psi            # 波函数存储模块
```

```
    kernels
        cuda
        rocm
        test
    test
        support
<strong>...</strong>
```

为了更进一步的规范异构计算代码在 ABACUS 中的维护和管理，以 module_base/kernels 举例，针对异构计算代码的位置和目录结构做出如下规范：

```
module_base/kernels/
  cuda                    # 存放算子的 CUDA 实现
     math_op.cu           # 算子的 CUDA 实现存放于 cuda 目录下
  math_op.cpp             # 算子的 CPU 实现
  math_op.h               # 算子的声明
  rocm                    # 存放算子的 ROCM 实现
     math_op.hip.cu       # 算子的 ROCM 实现存放于 rocm 目录下
  test                    # 提供了算子的单元测试
      CMakeLists.txt
      math_op_test.cpp
```

1. 异构计算代码存放规范：在每个 module 下，异构计算相关的代码都应该存放在 kernels 目录下。异构计算代码都是以算子（OP）的形式存在的，一个算子包含了：CPU，CUDA 以及 ROCM 的实现，使得该算子可以作为统一的计算接口供 ABACUS 调用。

2. 算子的声明和 CPU 实现存放规范：以 module_base 目录举例：module_base/kernels 目录应该直接存放当前 module 中涉及到的异构计算算子的声明.h 文件和其 CPU 实现.cpp 文件；

3. 算子的 CUDA 实现存放规范：kernels 目录下的 cuda 子目录中相应的.cu 文件存放算子的 CUDA 实现；

4. 算子的 ROCM 实现存放规范：kernels 目录下的 rocm 子目录存放算子的 ROCM 实现；

5. 算子的单元测试存放规范：kernels 子目录下的 test 目录存放当前 kernels 中涉及到的全部算子的单元测试。

### 7.5.3 总结

我们提倡在 ABACUS 中将各个模块的代码按照上述规范建议进行存放，目录结构可以延续上述风格以便代码识别；涉及到代码目录的问题欢迎到 Github 仓库进行讨论交流。

## 7.6 如何在 ABACUS 中新增一个输入参数（截至 v3.5.3）

### 7.6.1 在 input 类中申明和定义

**声明**

路径：source/module_io/input.h

首先现在 input.h 中添加相关输入参数的定义，如 nelec_delta：

```
    //==========================================================
    // electrons / spin
    //==========================================================
```

```
    std::string dft_functional; // input DFT functional.
    double xc_temperature; // only relevant if finite temperature functional is used
    int nspin; // LDA ; LSDA ; non-linear spin
    double nupdown = 0.0;
    double nelec; // total number of electrons
    double nelec_delta; // change in the total number of electrons
    int lmaxmax;
```

**设置初始值**

路径：source/module_io/input.cpp

接下来你需要在 Input::Default(void) 和 Input::Default_2(void) 函数里对参数进行初始化，其中：

Input::Default(void)——定义单个变量的初始值

Input::Default_2(void)——根据部分参数的值，对一些有关联的参数重新赋值，如 nelec_delta：

```
    //----------------------------------------------------------
    // electrons / spin
    //----------------------------------------------------------
    dft_functional = "default";
    xc_temperature = 0.0;
    nspin = 1;
    nelec = 0.0;
    nelec_delta = 0.0;
    lmaxmax = 2;
```

**读取**

路径：source/module_io/input.cpp

在 Input::Read() 中定义从 INPUT 文件中读取相应的参数，例如：

```
        else if (strcmp("nelec_delta", word) == 0)
        {
            read_value(ifs, nelec_delta);
        }
```

**MPI 广播**

路径：source/module_io/input.cpp

读取完之后，需要定义如何在 Input::Bcast() 将读取的参数广播到所有的节点，原因是 ABACUS 程序从一个进程读入参数，之后通过 MPI 的 Bcast 函数广播到其它的进程。例如：

```
    Parallel_Common::bcast_double(nelec_delta);
```

注意这里你需要根据参数的类型，使用不同的语句：

```
    Parallel_Common::bcast_bool(out_wannier_wvfn_formatted);
    Parallel_Common::bcast_string(dft_functional);
    Parallel_Common::bcast_double(xc_temperature);
```

```
    Parallel_Common::bcast_int(nspin);
```

**输出**

路径：source/module_io/write_input.cpp

ABACUS 会在 Input::Print(const std::string &fn) 中做参数的输出，你也需要在这个函数中进行添加：

```
ModuleBase::GlobalFunc::OUTP(ofs, "nelec_delta", nelec_delta,"change in number of total electrons");
```

**添加测试**

路径：

source/module_io/test/input_test_para.cpp

source/module_io/test/write_input_test.cpp

参数默认值和 MPI 广播的测试

TEST_F(InputParaTest, Bcast) 既测试了初始值的设定，也同样测试了 Bcast 的有效性，你也要将新增参数加入：

```
EXPECT_DOUBLE_EQ(INPUT.nelec_delta, 0.0);
```

这里同样需要考虑不同的数据类型：

```
EXPECT_TRUE(INPUT.out_wannier_mmn);
EXPECT_FALSE(INPUT.out_wannier_unk);
EXPECT_DOUBLE_EQ(INPUT.nelec, 0.0);
EXPECT_EQ(INPUT.basis_type, "pw");
```

参数输出的测试

TEST_F(write_input, General1) 测试了 ABACUS 参数输出的正确性，例如：

```
EXPECT_THAT(output, testing::HasSubstr("nelec
0 #input number of electrons"));
```

这里一定要注意空格和对齐。

### 7.6.2 将 input 成员转换为全局变量

参数参与计算一般会先转换为全局变量 GlobalV，再通过传参赋值给负责计算的类（但是注意将来我们会逐步弃用 GlobalV，所以这部分将来可能会变化）。

**声明**

路径：source/module_base/global_variable.h

在 GlobalV 这个命名空间中申明新增的参数，例如：

```
extern double nelec_delta;
```

**设置初始值**

路径：source/module_base/global_variable.cpp
为 GlobalV::var 设置初始值，例如：

```
double nelec_delta = 0;
```

**利用 Input 类的成员为 GlobalV 中成员赋值**

路径：source/module_io/input_conv.cpp
在 Input_Conv::Convert(void) 中为 Global 的参数赋值，例如：

```
GlobalV::nelec = INPUT.nelec;
```

注意：Input 类一个全局类，只要调用了头文件即可使用。

**添加测试**

路径：source/module_io/test/input_conv_test.cpp
在 TEST_F(InputConvTest, Conv) 测试这一转换是否正确，例如：

```
EXPECT_EQ(GlobalV::nelec_delta,0);
```

### 7.6.3 添加文档

路径：docs/advanced/input_files/input-main.md
每个新参数的 PR 必须包含相应的文档，否则不会被接收。请在 input-main.md 中添加参数描述。

## 7.7 如何在 ABACUS 中新增一个输入参数（v3.7.0 后）

### 7.7.1 开发者须知

**声明参数并给出初始值**

路径 module_parameter/input_parameter.h
首先现在 input_parameter.h 中添加相关输入参数的定义，如 nelec_delta：

```
bool init_vel = false;          ///< read velocity from STRU or not  liuyu 2021-07-14
double symmetry_prec = 1.0e-6;  ///< LiuXh add 2021-08-12, accuracy for symmetry
bool symmetry_autoclose = true; ///< whether to close symmetry automatically
                                ///< when error occurs in symmetry analysis
double nelec = 0.0;             ///< total number of electrons
double nelec_delta = 0.0;       ///< change in the number of total electrons
```

在声明的同时给出默认值，如 nelec 默认为 0.0。
注：对于任何类成员变量的 double, int, bool 等一般类型变量，在定义时就建议给一个初始值。如果类成员没有初始值，很可能会有难以察觉的 bug
注：string 类型变量的初始值不能为””, 如果想自动设置，可以设置初始值为”auto”, ”none” 等。

**在参数列表中添加参数**

路径 module_io/read_input_item_*.cpp（不同的参数分好了类，其对应不同文件，需要加到属于的文件里面），如 nelec 就在 read_input_item_general.cpp 中

添加 nelec 参数的 item

```
Input_Item item("nelec");
```

之后依次添加：（* 为必填）

注释（annotation*）：该注释会被打印在 OUT.* 文件夹的 INPUT 中

```
item.annotation = "input number of electrons";
```

重新赋值函数（reset_value）：如果其他某些参数满足特定条件是，需要对当前参数（nelec）进行修改的函数，该函数一定会执行。书写规范要求：只把修改当前参数的函数部分定义在当前参数的 reset_value，如果要修改其他参数，请定义在其他参数的 reset_value 。

```
item.reset_value = [](const Input_Item& item, Parameter& para) {
            if(para.input.somecondition)
            {
                para.input.nelec = 0.0;
            }
        };
```

检验函数（check_value）：用于检查目前参数是否合适，如果不合适就报错退出（warning_quit），该函数一定会执行。

```
item.check_value = [](const Input_Item& item, const Parameter& para) {
            if(para.input.nelec < 0)
            {
                ModuleBase::WARNING_QUIT("ReadInput", "nelec should be no less than 0.0");
            }
```

获取最终打印值函数（get_final_value*）：用于给 stringstream final_value 赋值，最终 final_value.str() 会被打印在 OUT.* 的 INPUT 文件中，该函数一定会执行。

```
item.get_final_value = [](Input_Item& item, const Parameter& para) {
            item.final_value << para.input.nelec;
        };
```

以及如果是并行版，需要添加广播函数（bcastfuncs*）：如何调用 Parallel_Common 进行 bcast 的函数，该函数一定会执行。该函数不一定只传输该参数，还可以传输其他引入的非 INPUT 参数，例如 gamma_only_local 这个变量不是 INPUT 参数，但是其由 gamma_only 和 basis_type 共同决定取值，这种额外引入的参数也需要在 gamma_only 或 basis_type 处添加他的 bcast 函数。

```
#ifdef __MPI
        bcastfuncs.push_back([](Parameter& para) {
            Parallel_Common::bcast_bool(para.input.nelec);
```

```
        });
#endif
```

最后将 item 添加到参数列表中：

```
this->add_item(item);
```

总结：

```
{
        Input_Item item("nelec");
        item.annotation = "input number of electrons";
        item.read_value = [](const Input_Item& item, Parameter& para) {
            para.input.nelec = std::stoi(item.str_values[0]);
        };
        item.reset_value = [](const Input_Item& item, Parameter& para) {
            if(para.input.somecondition)
            {
                para.input.nelec = 0.0;
            }
        };
        item.check_value = [](const Input_Item& item, const Parameter& para) {
            if(para.input.nelec < 0)
            {
                ModuleBase::WARNING_QUIT("ReadInput", "nelec should be no less than 0.0");
            }
        };
        item.get_final_value = [](Input_Item& item, const Parameter& para) {
            item.final_value << para.input.nelec;
        };
#ifdef __MPI
        bcastfuncs.push_back([](Parameter& para) {
            Parallel_Common::bcast_bool(para.input.nelec);
        });
#endif
        this->add_item(item);
    }
```

以上是完整的参数添加实例，可以根据需要自己修改函数，然而实际上大部分参数的 bcastfuncs、get_final_value、read_value 函数填写是几乎一致的，他们有共同的形式，为了书写方便，可以利用 module_io/read_input_tool.h 定义的宏函数来进行书写的简化：

1. 对于形如

```
bcastfuncs.push_back([](Parameter& para) {
            Parallel_Common::bcast_bool(para.input.nelec);
        });
```

的 bcastfuncs 函数书写可以分别使用 add_double_bcast，add_int_bcast，add_bool_bcast，add_string_bcast，

add_doublevec_bcast, add_intvec_bcast, add_stringvec_bcast 函数进行代替

2. 对于形如

```
item.get_final_value = [](Input_Item& item, const Parameter& para) {
            item.final_value << para.input.nelec;
        };
        bcastfuncs.push_back([](Parameter& para) {
            Parallel_Common::bcast_bool(para.input.PARAMETER);
        });
```

的 get_final_value 与 bcastfuncs 函数的合并书写可以使用 sync_double, sync_int, sync_bool, sync_string, sync_doublevec, sync_intvec, sync_stringvec 函数进行代替

3. 对于形如

```
item.read_value = [](const Input_Item& item, Parameter& para) {
            para.input.nelec = std::stoi(item.str_values[0]);
        };
        item.get_final_value = [](Input_Item& item, const Parameter& para) {
            item.final_value << para.input.nelec;
        };
        bcastfuncs.push_back([](Parameter& para) {
            Parallel_Common::bcast_bool(para.input.nelec);
        });
```

的 read_value, get_final_value 与 bcastfuncs 函数的合并书写可以使用 read_sync_double, read_sync_string, read_sync_int, read_sync_bool 函数进行代替

因此添加 nelec 的代码可以简化成：

```
{
        Input_Item item("nelec");
        item.annotation = "input number of electrons";
        read_sync_double(nelec);
        item.reset_value = [](const Input_Item& item, Parameter& para) {
            if(para.input.somecondition)
            {
                para.input.nelec = 0.0;
            }
        };
        item.check_value = [](const Input_Item& item, const Parameter& para) {
            if(para.input.nelec < 0)
            {
                ModuleBase::WARNING_QUIT("ReadInput", "nelec should be no less than 0.0");
            }
        };
        this->add_item(item);
```

**参数的使用**

目前已经舍弃使用 INPUT 与 GloablV::，请勿在里面添加新变量！

需要用到参数的地方，请以只读的方式使用

```
int nbands = PARAM.inp.nbands; //只读的访问元素
```

而其不能作为修改的对象，这时编译不能通过：

```
PARAM.inp.nbands = 0;  //会报错
```

**添加测试**

读入测试
路径：source/module_io/test/read_input_pteset.cpp

```
        <u>source/module_io/test/support/INPUT</u>
```

在 INPUT 中添加这一参数：

```
nelec                  10 #input number of electrons
```

在 TEST_F(InputParaTest, ParaRead) 测试这一读入是否正确，例如：

```
EXPECT_EQ(PARAM.inp.nelec, 10);
```

特殊场景的 reset_value 和 check_value 测试
路径：source/module_io/test/read_input_item_test.cpp
在 TEST_F(InputTest, Item_test) 添加特定参数的 reset_value 函数和 check_value 的覆盖性测试。

**添加文档**

路径：docs/advanced/input_files/input-main.md
每个新参数的 PR 必须包含相应的文档，否则不会被接收。请在 input-main.md 中添加参数描述。

### 7.7.2 设计文档参考

参考：林需泽博士提的方案 github.com

**背景**

v3.7.0 之前添加参考如何在 ABACUS 中新增一个输入参数（截至 v3.5.3）
原 input.cpp
当添加一个新的 INPUT 参数，我们需在 input 类里做以下事情：
A. 首先在 Default 函数中给初始值；
B. 再在 read 函数中的 if 中加一个判断分支：

```
if (strcmp("suffix", word) == 0) // out dir
{
    read_value(ifs, suffix);
}
```

C. 再在 Bcast 函数中添加 bcast

D. （大部分）在 GlobalV 定义一个相同功能的变量，并给个初始值（实际上这个初始值没有任何用，还容易让人误解到底这个和 input.cpp 哪个是初始值）

E. 再在 input_conv.cpp 中转化成 GlobalV 的变量

F. 再在 write_input.cpp 添加一行代码使其可以输出到 OUT.ABACUS 文件夹中 INPUT

痛点：

每加一个参数，对于 ABACUS 主代码需“翻山越岭”地添加代码，不易管理

input.cpp 的变量和 GlobalV 的变量大部分重复，没有必要，且容易误解

大量的 if 分支改进思路：

将相同的参数代码集中起来

input 类只实现读的功能，不存储参数，参数由另一个类存储

GlobalV

其实代码有全局变量不可怕，可怕的是全局变量会在运行中改变，而部分 GlobalV 就是这样，因此需要限制他人在初始化参数之后改变的行为。

目前很多也使用了 public 的类静态成员变量，这其实和全局变量是一样的。

### 重构设计

#### Parameter 类

该类只存储 ABACUS 运行的参数，将代替 GlobalV 的功能，该类将成员设成私有变量，只给 ReadInput 类修改的权限，其他类只有访值权限，没有修改权限：

```
class Parameter
{
  public:
    Parameter(){};
    ~Parameter(){};
  public:
    // We can only read the value of input, but cannot modify it.
    const Input_para& inp = input;
    // We can only read the value of mdp, but cannot modify it.
    const MD_para& mdp = input.mdp;
    // We can only read the value of other parameters, but cannot modify it.
    const System_para& globalv = system;

    // Set the rank & nproc
    void set_rank_nproc(const int& myrank, const int& nproc);
    // Set the start time
    void set_start_time(const std::time_t& start_time);

  private:
    // Only ReadInput can modify the value of Parameter.
    friend class ModuleIO::ReadInput;
    // INPUT parameters
    Input_para input;
    // System parameters
    System_para system;
```

```
};
```

**Input_Item 类**

用于存储 input 参数的信息，每一个参数用 input_item 的一个对象存储，不同 input_item 用 vector 打包。

```
class Input_Item
{
  public:
    Input_Item(const std::string& label_in)
    {
        label = label_in;
    }

    std::string label;                   ///< label of the input item
    std::vector<std::string> str_values; ///< string values of the input item
    std::stringstream final_value;       ///< final value for writing to output INPUT file
    std::string annotation; ///< annotation of the input item

    // ====== !!! These functions are complete.        ======
    // ====== !!! Do not add any more functions here.  ======
    /// read value if INPUT file has this item
    std::function<void(const Input_Item&, Parameter&)> read_value = [](const Input_Item& item,
    Parameter& param) {};
    /// check value
    std::function<void(const Input_Item&, const Parameter&)> check_value = nullptr;
    /// reset some values
    std::function<void(const Input_Item&, Parameter&)> reset_value = nullptr;
    /// get final_value function for output INPUT file
    std::function<void(Input_Item&, const Parameter&)> get_final_value = nullptr;
    // ====== !!! Do not add any more functions here.  ======
};
```

四个存储的变量：

label: 参数的名字

annotation: 参数的注释，用于生成 INPUT 的注释

str_values: 读入的 raw 数据，一个参数可以有多个数据，因此用 vector 存储，例如：kspacing 3 3 3

final_value: 最终的取值，用于生成 INPUT

四个 std::function 成员：

read_value（必填）：当从 INPUT 读入参数后，需要进行的赋值操作

reset_value（选填）：根据其读入的值，可能会修改其他参数的值时填入，例如 calculation 默认是 scf，当你读入 calculation 是 nscf 后，则需要将 init_chg 改成 file

check_value（选填）：根据其读入的值进行判断参数是否合适，例如：读入 ecut 为-100，则需要 warning_quit()

get_final_value（大部分必填）：此函数一定会执行（无论是否从 INPUT 读入这个参数），需要在此函数中给出 final_value 的赋值函数，这样用于打印 INPUT 文件

**ReadInput 类**

此类只负责读入 INPUT 文件，并赋值给 Parameter 类的对象，此类不负责存储参数

```
class ReadInput
{
 public:
     /**
      * @brief read in parameters from input file
      *
      * @param param parameters of ABACUS
      * @param filename_in read INPUT file name
      */
     void read_parameters(Parameter& param, const std::string& filename_in);

     /**
      * @brief write out parameters to output file
      *
      * @param param parameters of ABACUS
      * @param filename_out write output file name
      */
     void write_parameters(const Parameter& param, const std::string& filename_out);
  private:
     std::vector<std::pair<std::string, Input_Item>> input_lists;
     //----These functions are done only when INPUT file has them.------
     // read value if INPUT file has this item
     std::vector<Input_Item*> readvalue_items;

     /// bcast all values function
     /// if no MPI, this function will resize the vector
     std::vector<std::function<void(Parameter&)>> bcastfuncs;
```

主要两个 public 函数：

read_parameters：读入 INPUT 文件，赋值给 param

write_parameters：打印 INPUT 文件

3 个 vector:

input_lists: 用于存储所有的 Input_Item，用于遍历调用 resetvalue, getfinalvalue

readvalue_items: 用于存储读入 INPUT 中相应 Input_Item 指针，并进行读入赋值操作

bcastfuncs: 用于所有参数的 bcast

原则上 bcastfuncs 也可以用 vector 存储，一方面由于其不依赖 Input_Item，并且可能会 bcast 非 input 列表参数，另一方面但为了访存效率，用 vector 存储更好

read_parameters 函数

对于 ReadInput 的 read_parameter，其执行的顺序是:

先确定 INPUT 对应哪些 Input_Item，然后将 INPUT 中读入的信息存入 item 的 str_values 中，之后依次执行

1.readvalue_items

2.inputlist->resetvalue

3.inputlist->checkvalue

4.bcastfuncs

write_parameters 函数

如果要打印 INPUT，顺序执行 input_lists 的 getfinalvalue

然后

```
for (auto& item: this->input_lists)
{
   Input_Item* p_item = &(item.second);
   if (p_item->getfinalvalue == nullptr)
            continue;
   p_item->getfinalvalue(*p_item, param);
   ModuleBase::GlobalFunc::OUTP(ofs, p_item->label, p_item->final_value.str(), p_item->annotation);
}
```

## 7.8 如何在 ABACUS 中进行异构计算

### 7.8.1 前言

异构编程（Heterogeneous Programming）是一种利用不同类型计算单元协同工作来完成计算任务的编程模式。传统的同构系统中，所有计算任务都由单一类型的处理器（如 CPU）完成。但随着计算需求的不断增长，单一类型处理器在处理某些特定任务时效率有限。例如，CPU 虽然具有强大的通用性和控制能力，但在处理大规模并行计算任务时，性能表现不如专门的计算单元。因此，异构系统应运而生，它结合了不同类型的计算单元，如 CPU、GPU（图形处理器）、FPGA（现场可编程门阵列）、ASIC（专用集成电路）等，以充分发挥各计算单元的优势，提高计算效率，这里我们主要讨论 CPU 结合 GPU（或者类似 GPU）的异构编程。

在国产密度泛函理论软件 ABACUS（原子算筹）里，异构的主要对象之一是数学库。近期我们对数学库 blas 的调用进行了封装，相关文件我们称为 blas_connector。blas_connector 得到重构之后（在这个 PR 得到接收之后 https://github.com/deepmodeling/abacus-develop/pull/5799），可以通过直接调用 blas_connector.h 里面提供的算子，配合 device 和 memory 等基础模块即可直接编写异构代码。这套方案比起现在使用的方案来说，希望解决不同模块各自封装的问题，且对于新开发者来说可以避免使用模板和各自封装导致的理解困难。之后会逐渐使用这套方案替换原有的实现，从而实现整体异构方案的统一化与重构。用这种方法去除模板之后，能实现更灵活的计算，后续会用这个方式完成混合精度计算功能的实现。

下文将会介绍如何使用 ABACUS 现有的工具简单的进行异构计算的实现。

### 7.8.2 管理设备类型

ABACUS 目前使用这几种方式进行计算设备的标识。主要记录在 source/module_base/module_device/types.h 文件中。包括 struct 和 enum 两种方式

```
// !source/module_base/module_device/types.h
#ifndef MODULE_TYPES_H_
#define MODULE_TYPES_H_

namespace base_device
{

struct DEVICE_CPU;
struct DEVICE_GPU;
struct DEVICE_DSP;
```

```
11
12 enum AbacusDevice_t
13 {
14     UnKnown,
15     CpuDevice,
16     GpuDevice,
17     DspDevice
18 };
19
20 } // namespace base_device
21
22 #endif // MODULE_TYPES_H_
```

其中，DEVICE_XXX 是定义为结构体的标识符。这些结构体被作为模板参数使用，在之前是作为 ctx 这种标识符使用，使用方式有些反直觉。

另一个 AbacusDevice_t 就是一个枚举类型变量。是一个很早就出现在 ABACUS 的代码中但是几乎一直没有被使用的数据结构。在重构 blas_connector 之后会逐渐启用，用于提供一个更加简单清晰的设备标识。

除此之外，在 ABACUS 里的有些其他模块会存在自己的设计。如 source/module_basis/module_pw/module_fft/fft_base.h 的 fft 模块中，使用一个字符串来标记设备类型

```
1 std::string device = "cpu"; //或者是 gpu
```

这个类简单的使用一个字符串来标记 device 类型。本质上和使用 AbacusDevice_t 没有区别。这里建议新开发者如果希望加入新的异构功能，尽量都是用 AbacusDevice_t 来标记。

### 7.8.3 数据内存管理（旧）

由于这些 Operator 的实现存在一些冗余的参数，因此会在近期的 Pull Request 中迎来对应的重构工作。因此暂时不建议使用这里的 Operator 进行内存管理，但是其仍然具有一定参考价值。

#### 给指针分配内存

注：如果你需要进行大量数据的计算和转移（如需要储存一个矩阵），这里建议使用 ABACUS 提供的 Tensor 数据结构进行计算，见 Tensor 文档。你也可以用这种方式进行异构计算，它会更加灵活，但是你需要自己进行内存管理，防止内存溢出等问题。

现在可用的异构内存工具位于 source/module_base/module_device/memory_op.h 文件中。只需要包含这个头文件即可使用。目前这些算子的使用方式还是以模板为主。我们主要会使用以下 Operator。

```
1 template <typename FPTYPE, typename Device>
2 struct resize_memory_op
3 {
4     /// @brief Allocate memory for a given pointer. Note this op will free the pointer first.
5     ///
6     /// Input Parameters
7     /// \param size : array size
8     /// \param record_string : label for memory record
9     ///
10    /// Output Parameters
```

```
    /// \param arr : allocated array
    void operator()(const Device* dev, FPTYPE*& arr, const size_t size,
    const char* record_in = nullptr);
};
```

这个算子的主要作用是给一个指定指针分配内存。你也可以使用这个算子给以及包含了一段数据的指针分配内存，这样的话原来的内存区域会被释放，然后重新分配一段内存。你可以这样使用这个算子

```
double* ptr = nullptr;
base_device::memory::resize_memory_op<double, base_device::DEVICE_CPU>()(ptr, sizeof(double));
```

可以注意到，首先我们有两个模板参数叫 base_device::DEVICE_GPU 和 base_device::DEVICE_CPU，用于决定这个 Operator 会执行在哪个设备上。。

之后我们只要调用这个 Operator 来实现在不同设备分配内存。上文展示了在 CPU 的一个 double 指针上分配一定内存的方法。这个 Operator 的用法是这样的

```
base_device::memory::resize_memory_op<TYPE, DEVICE>()(pointer, size);
```

在尖括号模板参数中，TYPE 是需要分配的数据类型，如 float，double，std::complex<double> 等。DEVICE 决定空间会被分配到哪个计算设备上，可以选填 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU。在后面的参数中，pointer 是需要被分配内存的指针，size 是希望分配的空间大小，一般可以用 sizeof(TYPE) * count 的方式指定分配多少个数据。

**初始化一片内存的值**

memory_op.h 提供了以下算子来初始化一片内存区域的值

```
template <typename FPTYPE, typename Device>
struct set_memory_op
{
    /// @brief memset for multi-device
    ///
    /// Input Parameters
    /// \param var : the specified constant value
    /// \param size : array size
    ///
    /// Output Parameters
    /// \param arr : output array initialized by the input value
    void operator()(FPTYPE* arr, const int var, const size_t size);
};
```

同样的，可以这样使用这个 Operator

```
base_device::memory::set_memory_op<TYPE, DEVICE>()(pointer, var, size)
```

TYPE：被操作指针的数据类型
DEVICE：计算设备类型，可选 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU
pointer：被操作的指针

var：设置内存区域的值，只能为 0 或者-1

size：设置内存区域的大小

注意！由于底层实现的原因，这个算子只能用来将一片内存区域全部设置为 0 或者全部设置为-1，不能用于给这篇内存填充别的特定数值。一般情况下我们也不会需要使用这个来填充别的数值的使用场景。

#### 在不同设备间转移数据

memory_op.h 提供了以下算子来在不同设备间转移数据

```
template <typename FPTYPE, typename Device_out, typename Device_in>
struct synchronize_memory_op
{
    /// @brief memcpy for multi-device
    ///
    /// Input Parameters
    /// \param arr_in : input array
    /// \param size : array size
    ///
    /// Output Parameters
    /// \param arr_out : output array initialized by the input array
    void operator()(FPTYPE* arr_out,
                    const FPTYPE* arr_in,
                    const size_t size);
};

template <typename FPTYPE_out, typename FPTYPE_in, typename Device_out, typename Device_in>
struct cast_memory_op
{
    /// @brief memcpy for multi-device
    ///
    /// Input Parameters
    /// \param arr_in : input array
    /// \param size : array size
    ///
    /// Output Parameters
    /// \param arr_out : output array initialized by the input array
    void operator()(FPTYPE_out* arr_out,
                    const FPTYPE_in* arr_in,
                    const size_t size);
};
```

这个算子的作用是在不同计算设备间转移数据。其中 synchronize_memory_op 会转移同种数据类型的数据，cast_memory_op 可以在设备间转移不同类型的数据，但是效率可能稍低。这个算子可以这么使用

```
base_device::memory::synchronize_memory_op<TYPE, DEVICE_OUT, DEVICE_IN>
(pointer_out, pointer_in, size);
```

这个函数会将 pointer_in 长度为 size 的内容拷贝到 pointer_out。要求这两个指针的数据类型 TYPE 要相同。如果选在相同的 DEVICE_OUT 和 DEVICE_IN，会在同一个计算设备上的两个指针之间移动数据，也就是起到

一个拷贝的作用。

TYPE：被操作指针的数据类型

DEVICE_OUT：拷贝的目标计算设备类型，可选 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU

DEVICE_IN：被拷贝的计算设备类型，可选 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU

pointer_in：被拷贝的指针

pointer_out：拷贝的目标指针

size：拷贝内存区域的大小

另一个算子使用方法类似，但是会提供一个类型转换

```
base_device::memory::cast_memory_op<TYPE_OUT, TYPE_IN, DEVICE_OUT, DEVICE_IN>
(pointer_out, pointer_in, size);
```

TYPE：被操作指针的数据类型

DEVICE_OUT：拷贝的目标计算设备类型，可选 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU

DEVICE_IN：被拷贝的计算设备类型，可选 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU

pointer_in：被拷贝的指针

pointer_out：拷贝的目标指针

size：拷贝内存区域的大小

使用这两个算子可以将 CPU 上的数据拷贝到 GPU 上，或者将 GPU 数据转移到 CPU 上。也可以在同一设备上使用实现内存区域的拷贝。

#### 删除内存区域

为了防止内存溢出，一定要记得在开辟内存区域最后删除。在 memory_op.h 提供了以下算子来释放内存空间

```
template <typename FPTYPE, typename Device>
struct delete_memory_op
{
    /// @brief free memory for multi-device
    ///
    /// Input Parameters
    /// \param arr : the input array
    void operator()(const Device* dev, FPTYPE* arr);
};
```

这个使用方式非常简单，只需要

```
base_device::memory::delete_memory_op<TYPE, DEVICE>()(pointer);
```

TYPE：被操作指针的数据类型

DEVICE：计算设备类型，可选 base_device::DEVICE_CPU 或 base_device::DEVICE_GPU

pointer：需要删除内存区域的指针

#### 使用别名

在一般使用的时候，为了避免算子名称过长导致的代码可读性下降，我们一般使用关键字 using 来缩短对应模板函数的名称，如

```
using resmem_sh_op = base_device::memory::resize_memory_op<float, base_device::DEVICE_CPU>;

// Then you can directly using resmem_sh_op to resize memory
```

在 memory_op.h 中，已经做好了大量这种定义

```
using resmem_sh_op = base_device::memory::resize_memory_op<float, base_device::DEVICE_CPU>;
using resmem_dh_op = base_device::memory::resize_memory_op<double, base_device::DEVICE_CPU>;
using resmem_ch_op = base_device::memory::resize_memory_op<std::complex<float>,
base_device::DEVICE_CPU>;
using resmem_zh_op = base_device::memory::resize_memory_op<std::complex<double>,
base_device::DEVICE_CPU>;

using resmem_sd_op = base_device::memory::resize_memory_op<float,
base_device::DEVICE_GPU>;
using resmem_dd_op = base_device::memory::resize_memory_op<double, base_device::DEVICE_GPU>;
using resmem_cd_op = base_device::memory::resize_memory_op<std::complex<float>
, base_device::DEVICE_GPU>;
using resmem_zd_op = base_device::memory::resize_memory_op<std::complex<double>,
base_device::DEVICE_GPU>;

using setmem_sh_op = base_device::memory::set_memory_op<float,base_device::DEVICE_CPU>;
using setmem_dh_op = base_device::memory::set_memory_op<double,base_device::DEVICE_CPU>;
using setmem_ch_op = base_device::memory::set_memory_op<std::complex<float>,
base_device::DEVICE_CPU>;
using setmem_zh_op = base_device::memory::set_memory_op<std::complex<double>,
base_device::DEVICE_CPU>;

using setmem_sd_op = base_device::memory::set_memory_op<float, base_device::DEVICE_GPU>;
using setmem_dd_op = base_device::memory::set_memory_op<double, base_device::DEVICE_GPU>;
using setmem_cd_op = base_device::memory::set_memory_op<std::complex<float>,
base_device::DEVICE_GPU>;
using setmem_zd_op = base_device::memory::set_memory_op<std::complex<double>,
base_device::DEVICE_GPU>;

using delmem_sh_op = base_device::memory::delete_memory_op<float, base_device::DEVICE_CPU>;
using delmem_dh_op = base_device::memory::delete_memory_op<double, base_device::DEVICE_CPU>;
using delmem_ch_op = base_device::memory::delete_memory_op<std::complex<float>,
base_device::DEVICE_CPU>;
using delmem_zh_op = base_device::memory::delete_memory_op<std::complex<double>,
base_device::DEVICE_CPU>;

using delmem_sd_op = base_device::memory::delete_memory_op<float, base_device::DEVICE_GPU>;
using delmem_dd_op = base_device::memory::delete_memory_op<double, base_device::DEVICE_GPU>;
using delmem_cd_op = base_device::memory::delete_memory_op<std::complex<float>,
base_device::DEVICE_GPU>;
using delmem_zd_op = base_device::memory::delete_memory_op<std::complex<double>,
base_device::DEVICE_GPU>;
```

```
43
44 using syncmem_s2s_h2h_op
45     = base_device::memory::synchronize_memory_op<float, base_device::DEVICE_CPU,
46     base_device::DEVICE_CPU>;
47 using syncmem_s2s_h2d_op
48     = base_device::memory::synchronize_memory_op<float, base_device::DEVICE_GPU,
49     base_device::DEVICE_CPU>;
50 using syncmem_s2s_d2h_op
51     = base_device::memory::synchronize_memory_op<float, base_device::DEVICE_CPU,
52     base_device::DEVICE_GPU>;
53 using syncmem_d2d_h2h_op
54     = base_device::memory::synchronize_memory_op<double, base_device::DEVICE_CPU,
55     base_device::DEVICE_CPU>;
56 using syncmem_d2d_h2d_op
57     = base_device::memory::synchronize_memory_op<double, base_device::DEVICE_GPU,
58     base_device::DEVICE_CPU>;
59 using syncmem_d2d_d2h_op
60     = base_device::memory::synchronize_memory_op<double, base_device::DEVICE_CPU,
61     base_device::DEVICE_GPU>;
62
63 using syncmem_c2c_h2h_op
64     = base_device::memory::synchronize_memory_op<std::complex<float>, base_device::DEVICE_CPU,
65     base_device::DEVICE_CPU>;
66 using syncmem_c2c_h2d_op
67     = base_device::memory::synchronize_memory_op<std::complex<float>, base_device::DEVICE_GPU,
68     base_device::DEVICE_CPU>;
69 using syncmem_c2c_d2h_op
70     = base_device::memory::synchronize_memory_op<std::complex<float>, base_device::DEVICE_CPU,
71     base_device::DEVICE_GPU>;
72 using syncmem_z2z_h2h_op
73     = base_device::memory::synchronize_memory_op<std::complex<double>, base_device::DEVICE_CPU,
74     base_device::DEVICE_CPU>;
75 using syncmem_z2z_h2d_op
76     = base_device::memory::synchronize_memory_op<std::complex<double>, base_device::DEVICE_GPU,
77     base_device::DEVICE_CPU>;
78 using syncmem_z2z_d2h_op
79     = base_device::memory::synchronize_memory_op<std::complex<double>, base_device::DEVICE_CPU,
80     base_device::DEVICE_GPU>;
81
82 using castmem_s2d_h2h_op
83     = base_device::memory::cast_memory_op<double, float, base_device::DEVICE_CPU,
84     base_device::DEVICE_CPU>;
85 using castmem_s2d_h2d_op
86     = base_device::memory::cast_memory_op<double, float, base_device::DEVICE_GPU,
87     base_device::DEVICE_CPU>;
88 using castmem_s2d_d2h_op
89     = base_device::memory::cast_memory_op<double, float, base_device::DEVICE_CPU,
90     base_device::DEVICE_GPU>;
```

```
using castmem_d2s_h2h_op
    = base_device::memory::cast_memory_op<float, double, base_device::DEVICE_CPU,
    base_device::DEVICE_CPU>;
using castmem_d2s_h2d_op
    = base_device::memory::cast_memory_op<float, double, base_device::DEVICE_GPU,
    base_device::DEVICE_CPU>;
using castmem_d2s_d2h_op
    = base_device::memory::cast_memory_op<float, double, base_device::DEVICE_CPU,
    base_device::DEVICE_GPU>;

using castmem_c2z_h2h_op = base_device::memory::
    cast_memory_op<std::complex<double>, std::complex<float>, base_device::DEVICE_CPU,
    base_device::DEVICE_CPU>;
using castmem_c2z_h2d_op = base_device::memory::
    cast_memory_op<std::complex<double>, std::complex<float>, base_device::DEVICE_GPU,
    base_device::DEVICE_CPU>;
using castmem_c2z_d2h_op = base_device::memory::
    cast_memory_op<std::complex<double>, std::complex<float>, base_device::DEVICE_CPU,
    base_device::DEVICE_GPU>;
using castmem_z2c_h2h_op = base_device::memory::
    cast_memory_op<std::complex<float>, std::complex<double>, base_device::DEVICE_CPU,
    base_device::DEVICE_CPU>;
using castmem_z2c_h2d_op = base_device::memory::
    cast_memory_op<std::complex<float>, std::complex<double>, base_device::DEVICE_GPU,
    base_device::DEVICE_CPU>;
using castmem_z2c_d2h_op = base_device::memory::
    cast_memory_op<std::complex<float>, std::complex<double>, base_device::DEVICE_CPU,
    base_device::DEVICE_GPU>;
```

在这个命名中，s,d,c,z 分别代表 float,double,std::complex<float>,std::complex<double> 这四种数据类型。而 h 代表 host，一般都指的是 CPU，d 代表 device，一般都指 GPU。

所以一个名为 syncmem_s2s_h2d_op 的别名指的是一个将 float 转换为 float（s2s），将数据从 CPU 拷贝到 GPU（h2d）的 synchronize_memory_op。

### 7.8.4 数据内存管理（新）

在 Pull Request https://github.com/deepmodeling/abacus-develop/pull/5861 合并之后，添加了直接进行内存管理的函数。这些函数本质上是对上述算子的封装，但是使用它可以使代码更加清晰，且避免了增加重构工作的工作量。因此建议使用这些函数进行内存管理。

#### 给指针分配内存

```
template <typename FPTYPE>
void resize_memory(FPTYPE* arr, const size_t size, base_device::AbacusDevice_t
device_type = base_device::AbacusDevice_t::CpuDevice);
```

直接调用这个函数即可给指针 arr 分配内存。FPTYPE 可为包括 float, double, std::complex 和 std::complex 的任意数据类型。device_type 可选 base_device::AbacusDevice_t::CpuDevice 或者

base_device::AbacusDevice_t::GpuDevice。

#### 初始化一片内存的值

```
template <typename FPTYPE>
void set_memory(FPTYPE* arr, const int var, const size_t size, base_device::AbacusDevice_t
device_type = base_device::AbacusDevice_t::CpuDevice);
```

直接调用这个函数即可给指针 arr 管理的一片内存区域初始化值。

其中，var 是需要赋的数值，只能为 0 或者-1！size 是这片内存区域的长度。device_type 可选 base_device::AbacusDevice_t::CpuDevice 或者 base_device::AbacusDevice_t::GpuDevice。

注意！由于底层实现的原因，这个算子只能用来将一片内存区域全部设置为 0 或者全部设置为-1，不能用于给这篇内存填充别的特定数值。一般情况下我们也不会需要使用这个来填充别的数值的使用场景。

#### 在不同设备间转移数据

有如下两个函数可以用于在不同设备间转移数据。

```
template <typename FPTYPE>
void synchronize_memory(FPTYPE* arr_out, const FPTYPE* arr_in, const size_t size, base_device::
AbacusDevice_t device_type_out, base_device::AbacusDevice_t device_type_in);

template <typename FPTYPE_out, typename FPTYPE_in>
void cast_memory(FPTYPE_out* arr_out, const FPTYPE_in* arr_in, const size_t size, base_device::
AbacusDevice_t device_type_out, base_device::AbacusDevice_t device_type_in);
```

其中，synchronize_memory 负责转移同类型数据，cast_memory 可以转移不同类型的数据。

arr_out 是转移数据的目标指针，arr_in 是转移数据的原始指针。size 是转移数据的长度。

device_type_out 是目标设备类型，可选 base_device::AbacusDevice_t::CpuDevice 或者 base_device::AbacusDevice_t::GpuDevice。

device_type_in 是数据原本储存区域的设备类型，可选 base_device::AbacusDevice_t::CpuDevice 或者 base_device::AbacusDevice_t::GpuDevice。

如果 device_type_out 和 device_type_in 相同的话，则是进行一次简单的数据拷贝工作。

#### 删除内存区域

为了防止内存溢出，一定要记得在开辟内存区域最后删除。只需要简单的执行这个函数即可

```
emplate <typename FPTYPE>
void delete_memory(FPTYPE* arr, base_device::AbacusDevice_t device_type =
base_device::AbacusDevice_t::CpuDevice);
```

这个函数会删除指定设备上的 arr 指针的内存区域。

### 7.8.5 执行数学计算

现在可以直接调用 Blas_connector 进行异构计算。只需要包含 source/module_base/blas_connector.h 这个头文件即可。

在 blas_connector.h 中，链接了 blas 的诸多函数（不建议直接使用），且都实现了对应的异构算子，只需要调整一个参数即可选择对应的异构设备进行计算。

要使用实现好的算子，只需要调用 BlasConnector 命名空间下的算子即可。具体有哪些算子直接查看 source/module_base/blas_connector.h。以最常用的 gemm 为例，这些算子一般是这么构成的

```
static
void gemm(const char transa, const char transb, const int m, const int n, const int k,
    const float alpha, const float *a, const int lda, const float *b, const int ldb,
    const float beta, float *c, const int ldc, base_device::AbacusDevice_t device_type =
    base_device::AbacusDevice_t::CpuDevice);

static
void gemm_cm(const char transa, const char transb, const int m, const int n, const int k,
    const float alpha, const float *a, const int lda, const float *b, const int ldb,
    const float beta, float *c, const int ldc, base_device::AbacusDevice_t device_type =
    base_device::AbacusDevice_t::CpuDevice);
```

这个算子前面的所有参数都和标准 blas 算子的参数排列方式相同，只要以相同的方式传入参数即可。注意到最后一个参数是 base_device::AbacusDevice_t 类型的参数，这是一个来自上文所说的 source/module_base/module_device/types.h 的枚举类型，代表使用的计算设备。如果我们需要在 GPU 上调用对应的算子进行计算的话，就这么使用

```
BlasConnector::gemm(...parameters..., base_device::AbacusDevice_t::GpuDevice);
```

如果需要在 CPU 上进行计算，只需要把这个参数改成 base_device::AbacusDevice_t::GpuDevice 或者直接空着不写（不建议），因为默认值就是 CPU 参数。

可以注意到，对于相同的算子有时候会有两种类型的封装，其中一种有后缀 _cm，这代表这是一个列优先的封装，如果没有后缀则为一个行优先的封装。由于 blas 本身都是基于列优先实现的，所以部分算子可能没有行优先的封装。

这样你就可以使用上文所述的内存管理工具和 Blas 计算库，在完全不使用专用的异构 API，如 CUDA，来进行异构代码的编写，实现功能在不同的设备上都能够运行。

## 7.9 ABACUS formatter-2.0 版本使用说明书

### 7.9.1 前言

为系统解决 ABACUS 对输出可读内容到屏幕和文件中的需求，2023 第三季度 ABACUS 开发团队进行了 ABACUS formatter library 1.0 版本的开发。随着后期 ABACUS 各功能的输出需求对 formatter library 的功能要求越来越多样化，我们对 formatter library 代码进行了全面重构，推出了 formatter-2.0 版本。相较于 1.0 版本（ABACUS formatter 库使用说明书），重构进行了大量冗余代码删减，如今已经轻量化成为只具有头文件、有明显“即插即用”性质的工具库。该工具库功能包括简单的字符串格式化、制表和部分 Python 字符串处理函数实现三部分。

### 7.9.2 字符串格式化

相较于 1.0 版本中只能对字符串加以 1) 宽度、2) 小数保留位数、3) 正负号、4) 左右对齐和 5) 科学计数法的设定，2.0 版本基于 std::snprintf 函数（见 cppreference：https://en.cppreference.com/w/cpp/io/c/fprintf）实现了对 format string（例如 C++20 支持的 format library：https://en.cppreference.com/w/cpp/utility/format/format）的全面支

持（如%d, %o, %x, %u, %hd, %ld, %lu, %c, %s, %f, %e 等），弃置了基于 stream（iostream, ）的技术路线（见 https://stackoverflow.com/questions/15106102/how-to-use-c-stdostream-with-printf-like-formatting）。

另一方面，std::cout 是在 ABACUS 中最普遍出现，且经常被修改的全局变量。任何不刻意设置便输出的内容都将受到“上一次”对 std::cout 的 IOManipulator 的影响。

然而，使用新版的 formatter 需要注意数据类型，若数据类型出现不匹配，则很可能出现 undefined behavior（例如 issue #4540 中所报道），这一类型要求和 Python 自 3.12 版本开始功能健全且大范围流行和提倡的 f-string 一致。

**static format 函数的使用**

实际上，由于 formatter library 的轻量化重构，从单元测试中就足以可以明白 formatter library 这一基础函数的使用方法。又因为该函数是类中 static 函数，因此可以随时避免创建类对象而直接使用该函数。基本的使用方法展示在下面：

```
TEST(FormatterTest, FmtCoreStaticFormat) {
    // const char*
    std::string result = FmtCore::format("Hello, \%s!", "world");
    // remove the last '\0' character
    EXPECT_EQ(result, "Hello, world!");
    // std::string
    result = FmtCore::format("Hello, \%s!", std::string("world"));
    EXPECT_EQ(result, "Hello, world!");
    // int
    result = FmtCore::format("Hello, \%d!", 123);
    EXPECT_EQ(result, "Hello, 123!");
    // float
    result = FmtCore::format("Hello, \%f!", 123.456);
    EXPECT_EQ(result, "Hello, 123.456000!");
    // char
    result = FmtCore::format("Hello, \%c!", 'a');
    EXPECT_EQ(result, "Hello, a!");
    // invalid format
    result = FmtCore::format("Hello, \%z!", "world");
    EXPECT_EQ(result, "Hello, \%!");
    // varadic template case
    result = FmtCore::format("Hello, \%s, \%d, \%f, \%c!", "world", 123, 123.456, 'a');
    EXPECT_EQ(result, "Hello, world, 123, 123.456000, a!");
}
```

下面我们对 ABACUS 中常用于输出的数据类型进行详细举例。

int 类型输出：%d

```
#include "module_base/formatter.h"
#include <iostream>

std::cout << FmtCore::format("\%d", 1);

//output: "1"
```

如果想要修改其宽度，则需要在 d 之前设定数字，如设置宽度为 4：

```
std::cout << FmtCore::format("\%4d", 1);
//output: "   1"
```

如果需要输出左对齐而非右对齐的字符串，则在百分号% 之后添加负号：

```
std::cout << FmtCore::format("\%-5d", 10);
// output: "10   "
```

float/double 类型输出：%f 和%e

对于浮点型数据，最重要的是其是否使用科学计数法、保留位数和宽度各如何。请认真观察如下示例：

```
const double rough_pi = 3.1415926535897932384;
std::cout << FmtCore::format("\%8f", rough_pi);
// output: "3.141592"
std::cout << FmtCore::format("\%8.4f", rough_pi);
// output: "  3.1415"
std::cout << FmtCore::format("\%.4f", rough_pi);
// output: "3.1415"
std::cout << FmtCore::format("\%.0f", rough_pi);
// output: "3"
```

不难发现如果 f（或者 e）前面只有一个数字，该数字默认为最小宽度。如果在数字前有小数点，则该数字意为保留小数位数，因此%.0f 将直接对数字取整。

对于需要使用科学计数法的场景，只需要将 f 替换成 e 即可。需要注意的是科学计数法中诸如 e+00 也需要占位 4 个长度，需要在规定字符串输出长度时加以考虑。

std::string 类型输出：%s

对于字符串的输出最简单，另外一个具有相似功能的占位符为%c，意为为 char 类型在格式化字符串（format string）中占位。

```
std::cout << FmtCore::format("Hello, \%s!\n", "world");
// output: "Hello, world!"
```

**dynamic format 函数的使用**

为应对大批量、重复使用同一 format string 的需求，可以首先建立一个 FmtCore 对象，之后调用对象中的成员函数 format 时，就不需要再每次输入 format string：

```
FmtCore fmt("Hello, \%s!\n");
std::cout << fmt.format("world") << std::flush;
std::cout << fmt.format("again") << std::flush;
// output:
// "Hello, world!
//  Hello, again!"
```

因此在功能角度和 static 函数中的 format 基本无异。单元测试可以辅助理解：

```
TEST(FormatterTest, FmtCoreDynamic)
{
    FmtCore fmt("Hello, \%s!");
    EXPECT_EQ(fmt.fmt(), "Hello, \%s!");
    std::string result = fmt.format(std::string("world"));
    EXPECT_EQ(result, "Hello, world!");

    fmt.reset("Hello, \%d!");
    EXPECT_EQ(fmt.fmt(), "Hello, \%d!");
    result = fmt.format(123);
    EXPECT_EQ(result, "Hello, 123!");

    fmt.reset("Hello, \%f!");
    EXPECT_EQ(fmt.fmt(), "Hello, \%f!");
    result = fmt.format(123.456);
    EXPECT_EQ(result, "Hello, 123.456000!");

    fmt.reset("Hello, \%c!");
    EXPECT_EQ(fmt.fmt(), "Hello, \%c!");
    result = fmt.format('a');
    EXPECT_EQ(result, "Hello, a!");

    // varadic template case
    fmt.reset("Hello, \%s, \%d, \%f, \%c!");
    EXPECT_EQ(fmt.fmt(), "Hello, \%s, \%d, \%f, \%c!");
    result = fmt.format(std::string("world"), 123, 123.456, 'a');
    EXPECT_EQ(result, "Hello, world, 123, 123.456000, a!");
}
```

### 7.9.3 制表功能

formatter-1.0 的另一项亮眼功能为自动制表，在 formatter-2.0 版本中这一 feature 得以保留，并由 1.0 版本的 Table 类重构成为更加轻量级的 FmtTable 类，用于提供想要制作排列整齐的数据表，又不太清楚表格的列宽的场景。

由于 FmtTable 类的设计初衷为”每一个 FmtTable instance“代表了一个 Table，因此在 Table 中设计需要用户提供如下信息：

每列标题，组织成 std::vectorstd::string

每列 format string，组织成 std::vectorstd::string

每列数据

（可选）表中数据与表标题的左右对称

（可选）表格的各个边框

（可选）表格中每列的分隔符

#### 形参表一览

```
/**
     * @brief Construct a new Fmt Table object
     *
     * @param titles titles, its size should be the same as the number of columns
     * @param nrows number of rows
     * @param aligns Alignments instance, can be constructed with initializer_list<char> like
     {'r', 'c'}, for right and center alignment for values and titles
     * @param frames Frames instance, can be constructed with initializer_list<char> like
     {'-', '-', '-', ' ', ' '}, for up, middle, down, left and right frames
     * @param delimiters Delimiters instance, can be constructed with initializer_list<char> like
     {'-', ' '}, for horizontal and vertical delimiters
     */
    FmtTable(const std::vector<std::string>& titles,
             const size_t& nrows,
             const std::vector<std::string>& fmts,
             const Alignments& aligns = {},
             const Frames& frames = {},
             const Delimiters& delimiters = {}): titles_(titles), fmts_(fmts), data_(nrows,
             titles.size()), aligns_(aligns), frames_(frames), delimiters_(delimiters)
    { assert(titles.size() == fmts.size()); };
```

基于 RAII 原则，我们假设需要制表并进行输出的数据经常是已经全部准备好的状态，而非如同 SCF 的迭代信息一样，需要特别估算每列的大致宽度信息。

因此在 FmtTable 构造函数中，我们一定需要 titles, n_rows, fmts，并对 align 和 Delimiter 参数设置了默认值因此并非总是需要。如果想要特殊配置，则可以选择单元测试中例子 https://github.com/deepmodeling/abacus-develop/blob/develop/source/module_base/test/formatter_test.cpp#L323，仿照其进行制表调整。

#### 使用代码示例

除了单元测试(https://github.com/deepmodeling/abacus-develop/blob/develop/source/module_base/test/formatter_test.cpp)外，目前 FmtTable 也已经用于 ABACUS 运行时间统计的输出（https://github.com/deepmodeling/abacus-develop/blob/develop/source/module_base/timer.cpp#L280），但可进一步实现对表中数据区域的每列左右对齐控制。

### 7.9.4 static Python-style 字符串函数

#### 函数功能简介

C/C++ 的文件读入功能也较为繁琐，尤其涉及需要进行字符串操作时，Python 中 str 的函数则更胜一筹。为了避免每次手动 parse 字符串，又注意到 std::string 同样是 STL 的标准容器，因此对标准容器支持的算法，也都能支持 std::string。目前出于个人使用习惯，实现了下列 Python-style static 函数：

- `split`: 用于以固定分隔符切割字符串，返回 `std::vectorstd::string`，但也是因为该返回类型，注定 split 函数不会收录在 STL 中
- `startswith/endswith`: 返回 `boolean`，用于判断字符串是否以给定字符串开始，或者以给定字符串结束。
- `strip`: 用于消除出现在字符串行头行尾的空格、回车和 \0 等字符，返回 `std::string`。
- `center`: 为了支持下一步的 ABACUS 输出重构，首先实现 center 函数用于将字符串以某一个宽度居中，两段则填充以 center 函数第一个参数，使得 ABACUS 中所有内容都可以具有给定宽度的输出（即两端对齐）

- `replace`: 用于消除字符串中存在的所有某个字符
- `join`: split 的反函数

**代码示例**

以上函数的例子可见单元测试：https://github.com/deepmodeling/abacus-develop/blob/develop/source/module_base/test/formatte

### 7.9.5 Find a bug? Submit issue!

如果在使用 formatter 过程中发现了 bug 或者运行结果不达预期，可以在 deepmodeling/abacus-develop 仓库下提交 issue。

## 7.10 ABACUS 中使用格式化工具 clang-format

### 7.10.1 引言

良好统一的代码风格在多人协同开发中至关重要，这一点在 ABACUS 团队里也尤其重要，特别是考虑到大部分的开发者都是研究生，没有经过特别系统的编程训练，那么好的代码风格能够降低阅读代码的障碍，加快开发效率，甚至可以防止一些不必要的 bug 的产生。

clang-format 是一个开源的代码格式化工具，它可以帮助程序员自动调整源代码的格式，以符合指定的编码风格规范。clang-format 可以在保持代码功能不变的情况下，自动处理缩进、空格、括号、逗号等细节，提高代码的可读性和一致性。

### 7.10.2 在 ABACUS 中使用 clang-format

clang-format 支持包括 LLVM、Google、Chromium、Mozilla、WebKit 等在内的多种代码风格，如果不指定自定义的代码风格，每次对代码进行格式化时使用的将是 clang-format 默认的风格。

clang-format 还支持在已有的代码风格基础之上自定义一个名为.clang-format 的代码风格文件，clang-format 就可以根据文件中定义的规则对代码进行格式化。

在 ABACUS 开源项目中，开发者已经制定了针对 ABACUS 项目的代码风格文件.clang-format，其位于项目的根目录下。（https://github.com/deepmodeling/abacus-develop/blob/develop/.clang-format）

注意事项：ABACUS 开源项目中的.clang-format 是依赖于 clang-format-14 标准的，因此请确保你的 clang-format 不低于 14 版本。

**在系统上安装 clang-format**

clang-format 有诸多版本可供使用，目前最新版本为 clang-format 18。不同版本所支持的格式化选项不尽相同，但向后兼容。

安装统一版本的 clang-format（ABACUS 项目中统一选择 clang-format-14）

```
# Linux 上，以 Ubuntu 为例：
sudo apt install clang-format-14

# 验证是否安装成功，并查看安装的版本号：
clang-format-14 --version
```

这里应注意，不能通过 sudo apt install clang-format 命令进行安装，因为对于不同的 Ubuntu 发行版而言，通过该命令安装得到的 clang-format 版本是不同的。

安装完成后，我们能够使用的命令名是 clang-format-14，我们将其提级为 clang-format：

```
sudo update-alternatives --install /usr/bin/clang-format clang-format /usr/bin/clang-format-14 100
```

后面当我们使用 clang-format 命令时，指代的即为 clang-format-14。

```
# 此时再次验证安装的版本号：(应为 14 版本)
clang-format --version
```

**基于 vscode 使用 clang-format**

基于 vscode 使用 clang-format，我们可以选择安装支持 clang-format 的插件，即可更方便的使用 clang-format。

但由于插件商场有很多插件都支持 clang-format 功能，且其支持的 clang-format 版本经常变化。所以我们应尽量选择支持更为稳定的插件进行使用。

通过 vscode 进行 C++ 开发时，我们一般会安装微软官方的 C/C++ 插件，打开该插件的安装目录我们会发现，该插件已经封装了一个内置的 clang-format。那为什么不直接使用这个 C/C++ 插件内置的 clang-format 呢?

不同版本的 C/C++ 插件内置的 clang-format 版本往往不同，就带来不必要的麻烦。为了使用统一的 clang-format-14，我们建议在 vscode 中安装 Clang-Format（Xaver Hellauer）插件 。

安装完成后，打开 “File -> Preferences -> Settings”，搜索 “format”，如下图：

将 Default Formatter 刚换为新下载的 Clang-Format 插件。

之后在一个文件内，通过”option” + ”shift” + ”F” 快捷键，即可实现对当前代码文件的格式化。

**基于命令行使用 clang-format**

除了使用 vscode ，可能还会有使用命令行进行开发的开发者。

使用方式：

```
clang-format -style=file -i demo.cpp
```

### 7.10.3 格式化时的注意事项：

由于直接对整个文件格式化会导致每一行的历史修改信息被覆盖，因此我们要求在您修改某文件相应的代码内容之后，先 commit 一次，然后再对该文件进行格式化，再提交一次专门对文件格式化的 commit。这样细化每一个 commit 具体的信息，更有利于 PR 的审核和项目的管理。

### 7.10.4 参考文献：

使用 clang-format 进行 C++ 代码风格管理
Clang-Format 用法详解
Clang-Format Style Options —Clang 14.0.0 documentation
C++ programming with Visual Studio Code
基于 vscode 使用 clang-format（旧版）